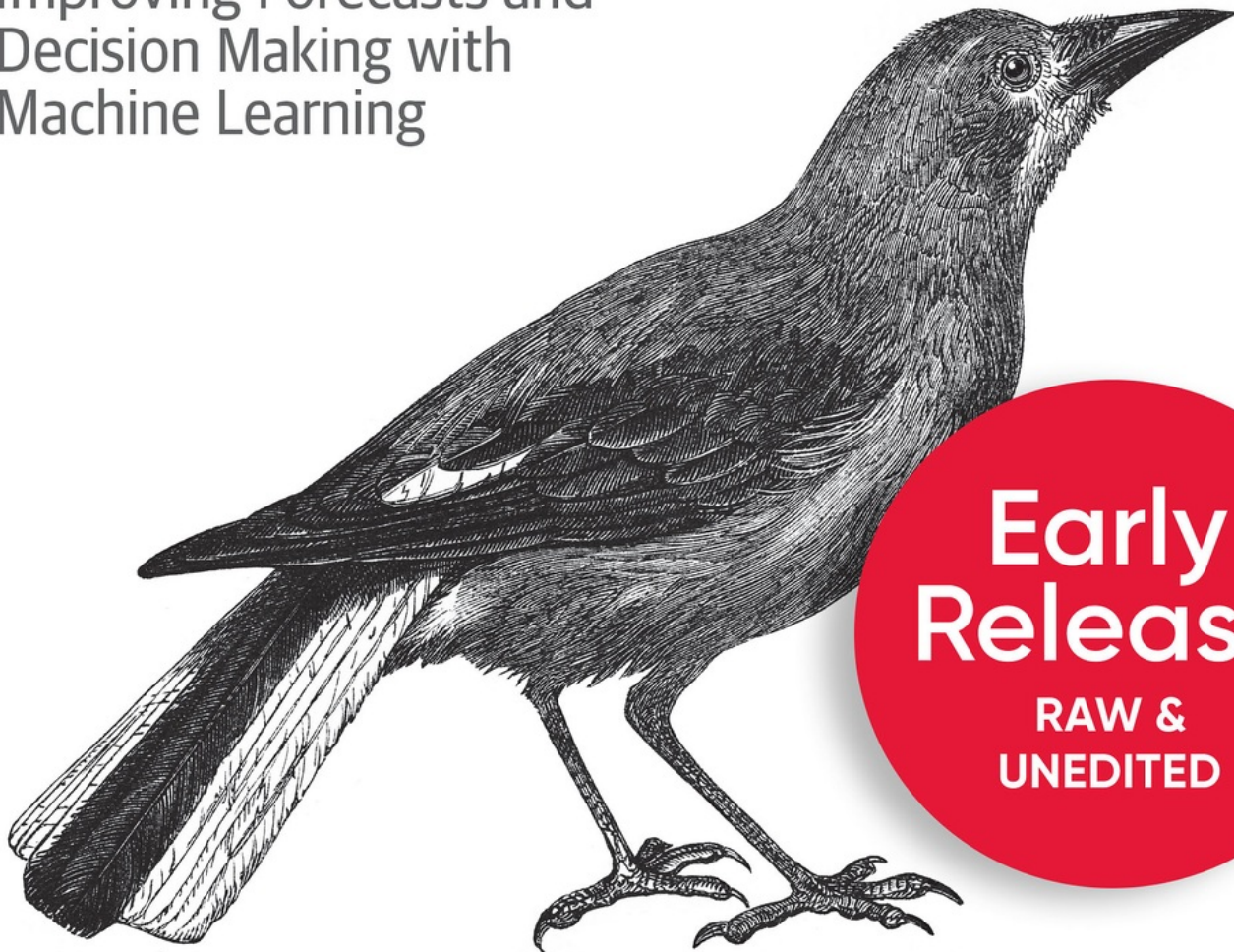


O'REILLY®

AI-Powered Business Intelligence

Improving Forecasts and
Decision Making with
Machine Learning



**Early
Release**

**RAW &
UNEDITED**

Tobias Zwingmann

AI-Powered Business Intelligence

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Tobias Zwingmann

AI-Powered Business Intelligence

by Tobias Zwingmann

Copyright © 2022 Tobias Zwingmann. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North,
Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Rachel Roumeliotis

Development Editor: Rita Fernando

Production Editor: Christopher Faucher

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

October 2022: First Edition

Revision History for the Early Release

- 2021-10-21: First Release
- 2021-12-20: Second Release
- 2022-03-18: Third Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492073215> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *The Value of AI-Powered Business Intelligence*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-11140-3

Chapter 1. Creating Business Value with AI

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 1st chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

In this chapter, we are going to explore why AI adoption in BI is becoming more important than ever and how AI can be utilized by BI teams. For this purpose, we will identify the typical areas in which AI can support BI tasks and processes, and we will look at the underlying machine learning capabilities. At the end of the chapter, we’ll go over a practical framework that will let you map AI/ML capabilities to BI problem domains.

How AI is changing the BI landscape

For the past 30 years, Business Intelligence has slowly but steadily become the driving force behind data-driven cultures in companies. The attention has shifted towards data science, machine learning and artificial intelligence. How did this even happen? And what does this mean for your BI organisation?

When we look back to the beginning of the first era of BI systems in the 1970s, we see technical systems that were used by IT experts to get insights from small (by today's scale) datasets. Analysing data was new and so even the most basic insights seemed jaw-dropping.

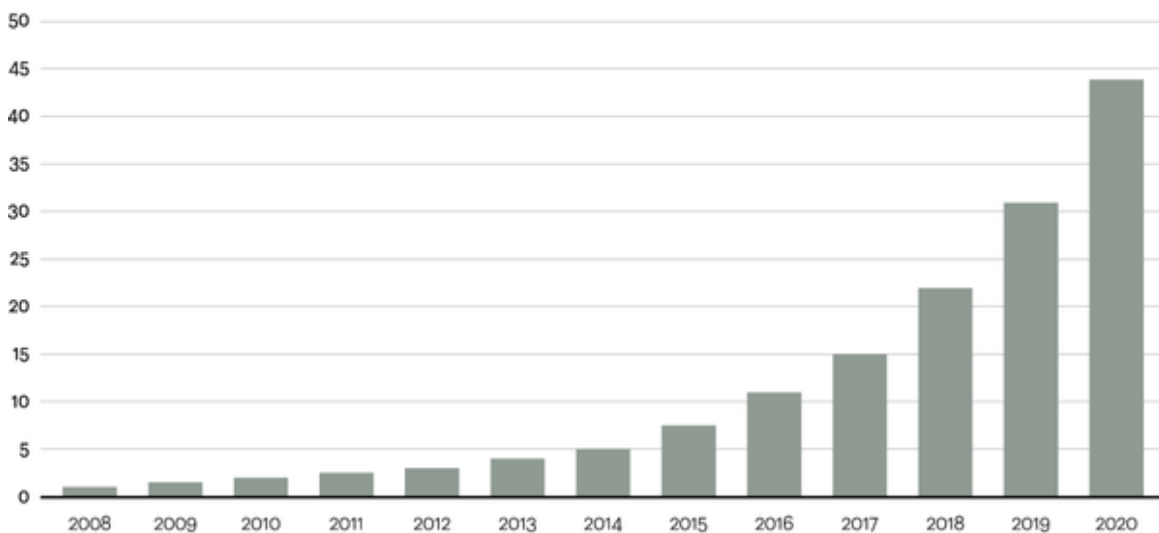
The second era of BI started in the mid-2000s and was dominated by self-service analytics. New tools and technologies made it easier than ever for non-technical audiences to slice and dice data, generate visualisations and get insights from increasingly large data sources.

Most large corporations today are still stuck in this second phase of BI. Why is that? For one, many technological efforts in the last years have been focused on trying to technically handle the exponential growth of underlying data that BI systems are supposed to process and generate insights from. Second, the increase of data as shown in Figure 1-1 fuelled an increasing shortage of data literate people who are well-versed in handling high-dimensional datasets with the appropriate tools (which, in this case, isn't Excel).

Figure 1

Data is growing at a 40 percent compound annual rate, reaching nearly 45 ZB by 2020

Data in zettabytes (ZB)



Source: Oracle, 2012

Figure 1-1. Data growth in recent years (Source: *Statista*)

Compared to the consumer market, AI applications are still underserved in the professional BI space. This is probably due to the fact that AI and BI talent sit in different teams within organizations, and if they ever meet, they have a hard time communicating effectively with each other. This is mainly due to the fact that both teams typically speak different languages and have different priorities. Meaning, BI experts usually don't talk much about training and testing data, and data scientists rarely chat about SSIS packages and ETL routines. The need for AI adoption in BI, however, is going to inevitably increase due to the following ongoing trends:

- **The need to get quick answers from data:** To remain competitive, organizations demand data-driven insights to help them grow. Data Analysts get overwhelmed with inquiries to explore this or that metric or examine this or that dataset. At the same time, the need for business users to get quick and easy answers from data increases. If they can ask Google or Alexa about the current stock price of a certain company, then why can't they ask their professional BI system about the sales figures from yesterday?
- **Democratization of insights:** Business users have become accustomed to getting insights from data with self-service BI solutions. However, the data itself is often too large and too complex to be handed off to the business for pure self-service analytics. High dimensionality and sometimes the sheer size of the data makes it difficult, if not impossible, today for non-technical users to analyse data with familiar tools on their local computers. In order to continue democratizing insights across an organization, BI systems are needed that are easy to use and surface insights automatically to the end-users.
- **Accessibility of ML services:** While use of AI continues to rise within organizations, so does the expectation for better forecasting or better predictions. This applies even more so for BI; Low-Code or No-Code platforms make it easier than ever before to make machine learning technologies available to non data scientists and puts pressure on the BI team to incorporate predictive insights into their reports. The same

advancements in data science are also expected to happen in the field of BI, sooner or later.

In order to get a better understanding of how Business Intelligence teams can leverage AI, let's briefly review the analytical insights model published by Gartner¹ as follows in Figure 1-2:

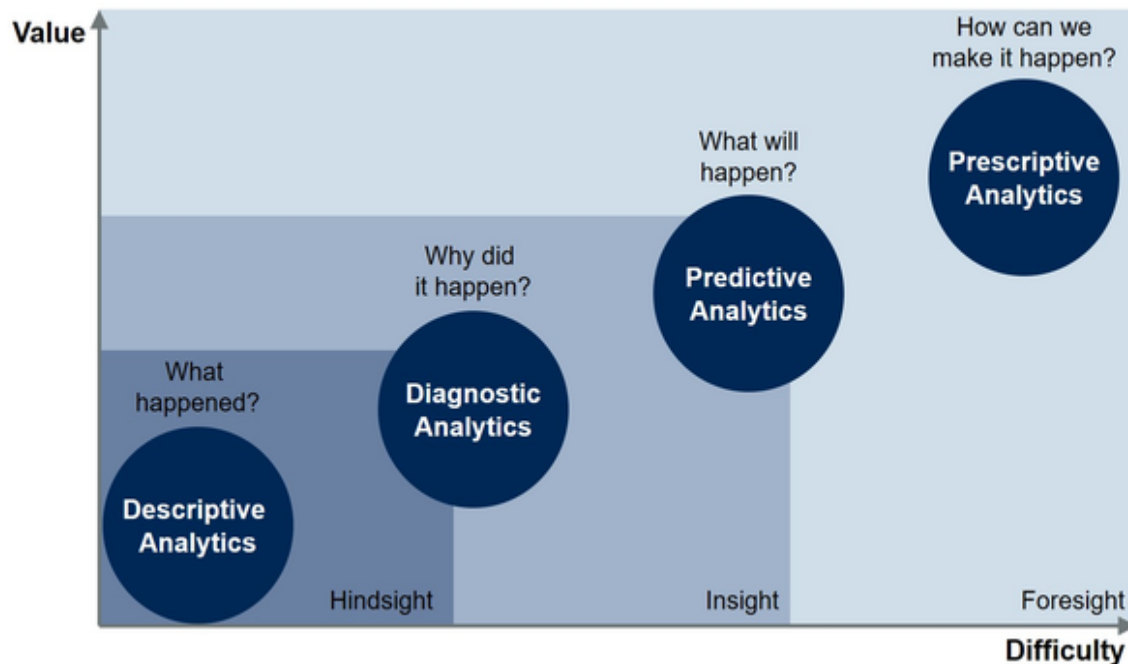


Figure 1-2. Types of insights and the analytic methodologies

The core functionality of every BI or reporting infrastructure is to deliver hindsight and insight using *descriptive* and *diagnostic* analytics on historic data. These two methods are paramount for all further analytical processes that layer on top of them.

First, an organization needs to understand what happened in the past and what was driving these events from a data perspective. This is typically called basic reporting with some insight features. The technical difficulty and complexity is comparably low, but so is the intrinsic value of this information.

Consider the following example: With a BI reporting system you could easily examine how many customers churned in the last week (descriptive).

You could even understand which customer segments churned and what actions they took - or didn't take - before they churned (diagnostic).

While this information by itself is quite useful, the value for the organisation is still limited. The insight itself won't prevent churn in the future.

Therefore, the next logical step of our BI and reporting systems would be to ask: How many (and which) customers are going to churn in the future, e.g. next week?

We are now exploring future events and entering the field of *predictive* analytics. Since we are leaving the realms of historic data, the complexity rises. We can't deliver insights any more in binary terms of true or false. Instead we are now talking about probabilities of certain events happening. Of course, this leads to an increased technical complexity and difficulty, but you can hopefully recognise the added value here.

Knowing which customers are likely to churn in the future gives us a handful of good options. Here are a couple examples: We can try to implement counter actions which hopefully retain customers or we can anticipate these changes with better planning.

Finally, once we know what is likely to happen in the future, we should care about which actions we take in order to prevent or anticipate our predictions. Welcome to the phase of *prescriptive* analytics.

Here is an example how this might work out: We predict customer churn for each User ID in our CRM system. A prescriptive model would consequently suggest a set of actions to take for each customer in order to reduce the churn likelihood, such as sending a certain email text or applying a certain price discount.

As we look into organisations with thousands or more customers it becomes clear that in order to optimize these tasks from a macro perspective, we need to rely on automation on a micro level. It is simply impossible to go through all these mini decisions manually. In contrast, a one-fits-all decision by the gut feeling of whoever is sitting in front of the computer is

not likely to achieve the best results. And this is where AI and BI go perfectly together.

Consider an AI that recommends churn likelihood together with a suggested next best action on a per customer base. We can now blend this information with classical BI metrics such as the customer's historical revenues or the customer's loyalty, allowing us to make a decision about these actions that have the highest business impact and best chances for success.

The relationship between AI and BI can therefore be summed up nicely in the following formula:

$$\textit{Artificial Intelligence} + \textit{Business Intelligence} = \textit{Decision Intelligence}$$

The most effective AI-powered BI application exists, once we blend automated and human decision making. We will explore this further in chapter 2.

Now, let's take a concrete look at how AI can systematically help us to improve our BI.

Common AI Use Cases for BI

AI can typically add value to BI in three ways: (1) Automating insights and making the analytical process more user friendly, (2) calculate better forecasts and predictions and (3) enable BI systems to drive insights even from unstructured data sources. Figure 1-3 gives you a high level overview how these application areas map to the different analytical methods.

How AI Capabilities Support Analytical Methods

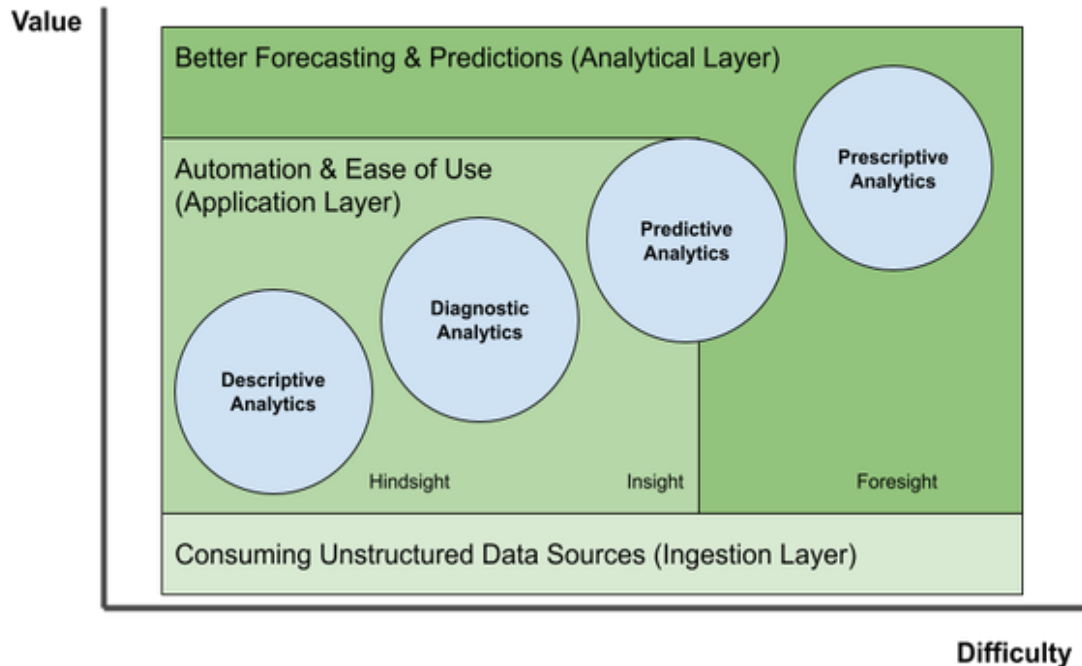


Figure 1-3. Mapping AI capabilities to analytical methods

Let's explore these areas in a bit more detail.

Automation & Ease of Use

Making the BI tool itself more intelligent and *easy to use* will make it even more accessible for non-technical users, taking workloads off from the hands of analysts. This ease of use is typically driven by *automation* happening under the hood. Smart algorithms make it possible to comb through piles of data in seconds and surface interesting patterns or insights to business users or analysts. As Figure 1-4 shows, these routines work specifically well for the descriptive and diagnostic analytical stage, where interesting correlations or unusual observations in data are being discovered. But automation and ease of use also touch upon the stage of

predictive analytics, for example by making it even easier for users to train and deploy custom ML models.

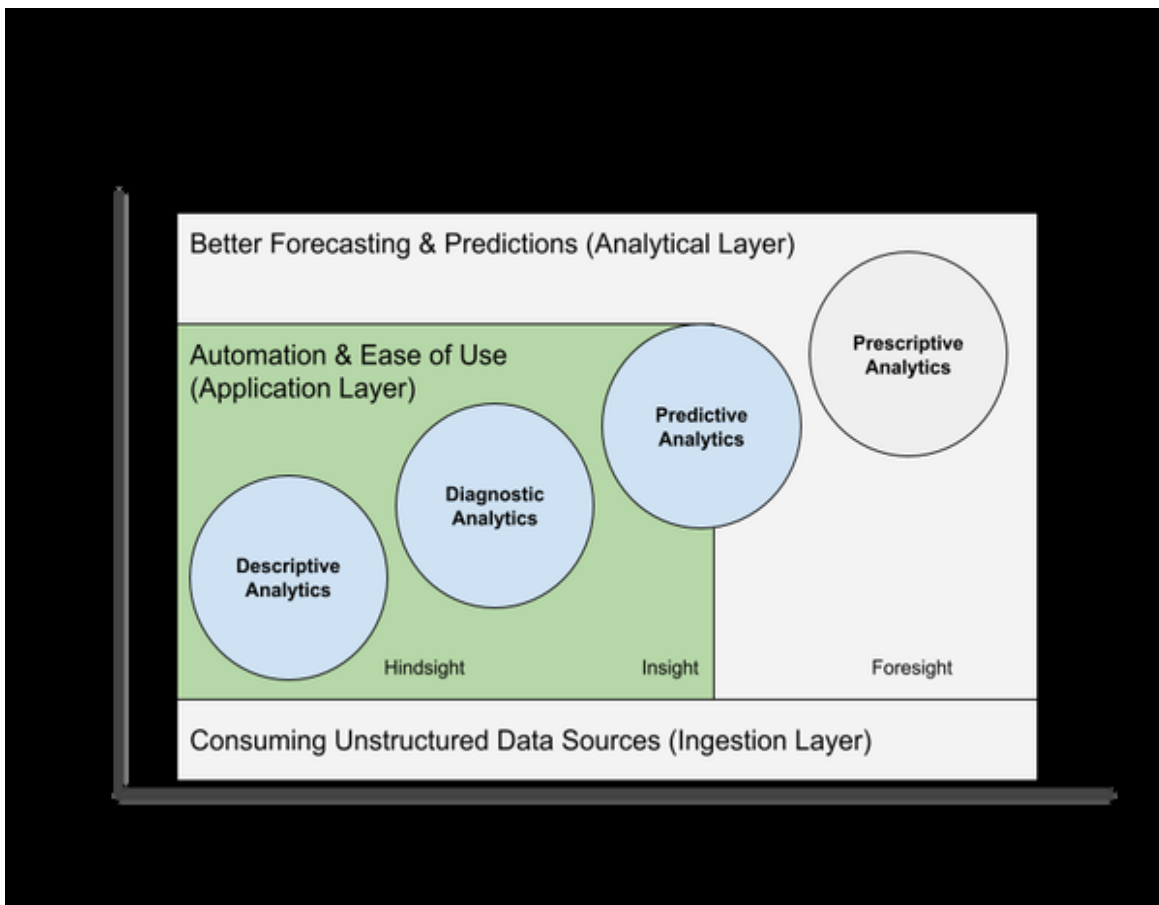


Figure 1-4. AI-powered BI: Automation & ease of use (application layer)

There is one important thing to note here: The AI capabilities at this stage are typically built into the application layer, which is your BI software. So it's typically nothing you can simply "add" to a BI platform with a few lines of Python code (in contrast to AI-powered predictions and unlocking unstructured data, which we will discuss later in chapters 6, 7 and 8.) If you are using modern BI platforms such as Microsoft PowerBI or Tableau, then you'll find these AI-enabled features inside these tools, sometimes they are hidden or happening so seamlessly that you don't even notice that AI is at work here.

Here are some indicators where AI is typically working under the hood to make your life as an analyst much easier:

- **You can interact with data using Natural Language**

By using AI-powered Natural Language Processes technologies, machines are much better at interpreting and processing textual input from users. For example, let's say you want to know the sales results for last month or how the sales were in the US last year versus this year. You might type in the following queries:

*How were my sales in Texas in the last 5 years? or
Sales \$ in Texas last year vs Sales \$ in Texas this year.*

No complicated code or query language needed. This layer of Q & A-like input makes BI much more accessible to non-technical users and also more convenient for analysts who really can't anticipate every question a business user might ask with a pre-made report. Most users will be quite familiar with this approach because it is similar to using a search engine such as Google.

Whether Q & A tools are either built-in to your BI tool or not, not all of these implementations work equally well. In fact, there's a huge complexity that needs to be solved behind the scenes to make these features work reliably in production work environments. Analysts have to track what kinds of questions business users ask and validate if the generated output is actually correct. Synonyms and domain-specific lingo need to be defined to make sure systems can interpret user prompts correctly. And as with all IT systems, these things need constant maintenance. The hope is that the systems will improve and the manual effort needed in the background will decrease over time.

- **Summarizing analytical results**

Even if a chart seems self-explanatory, it is good practice to summarize key insights in one or two lines, reducing the risk of misinterpretation. But who really enjoys writing seemingly all-obvious descriptions below plots in reports or presentations? Most people don't, and that's where AI can help.

AI-powered Natural Language Processing can not only help you to interpret natural language input, but it can also generate summary text for you based on some data. These auto-generated texts will include descriptive characteristics about the data as well as noteworthy changes or streaks.

Here's an example of an auto-generated plot caption from Microsoft PowerBI:

Sales \$ for Texas increased for the last 5 years on record and it experienced the longest period of growth in Sales between 2010 and 2014.

As you can see, these small AI-generated text snippets can make your life as an analyst much easier and save you a bulk of time when it comes to communicating insights to other stakeholders.

- **Automated pattern finding in data**

We have seen how Natural Language capabilities can help you to get descriptive insights from your data efficiently. The next logical step is to find out why certain observations happened in the past, such as: Why exactly did sales in Texas increase so much?

With diagnostic analytics, you would normally need to comb through your dataset in order to explore meaningful changes in underlying data distributions. In this example, you might want to find out if a certain product or a certain event was driving the overall change. This process can quickly become tedious and cumbersome. AI can help you to decrease the Time to Insights (TTS). Algorithms are great at recognizing underlying patterns in data and bringing them to the surface. For example, with AI-powered tools such as decomposition trees or key influencer analysis you can quickly find out which characteristic in your data led to the overall observed effect – on the

fly.

In Chapters 4 and 5, we are going to look at three concrete examples of how you can use AI-powered capabilities in Power BI to make your life as a data analyst or business user easier.

Improved Forecasting & Predictions

While descriptive and diagnostic analytics have been at the heart of every BI system, the immanent desire has always been to not only understand the past but also to foresee the future. As you can see in Figure 1-5, AI-enhanced capabilities can support end users to apply powerful predictive and prescriptive analytical methods for better *forecasting and predictions* based on historical data. This will add complexity since we leave the realms of binary data from the past and introduce probabilistic guesses about the future, which naturally contain many uncertainties. At the same time, the prospected value rises: If we were about to predict the future, we could make much better decisions in the present.

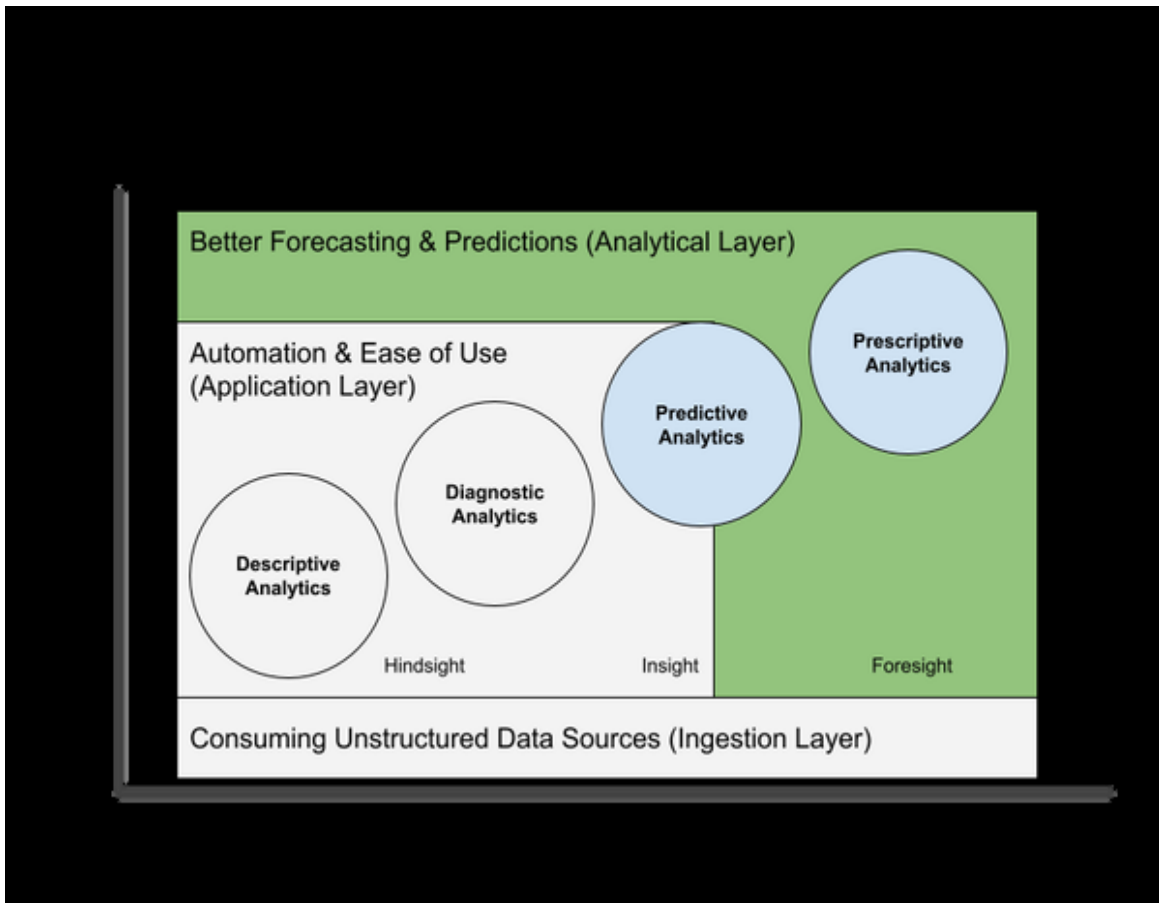


Figure 1-5. AI-powered BI: Better Forecasting & Predictions (Analytical Layer)

Now, if you heard about statistical methods like regression or Auto-Regressive Integrated Moving Average (ARIMA) before and wonder what is the big deal with AI, then take a note of the two following aspects:

- **AI can produce better forecasts with more data and less human supervision**

AI leverages old-school techniques such as linear regression at its core but at the same time applies these to multi-dimensional spaces in combination with stochastic algorithms such as gradient descent to find an optimal solution in very complex spaces in a very short amount of time without the need for big human supervision. Specialised algorithms for time series predictions are designed to recognize patterns in larger amounts of time series data. AI tries to optimize the forecast based on feature selection and minimizing loss functions. This can lead to better or more accurate predictions using a short time

horizon, or trying to predict more accurately over a longer period of time. More complex, non-linear models can lead to more granular and eventually better predictive results.

- **AI can calculate predictions at scale for optimized decision making**
Forecasting the total number of customers over the next quarter is nice. But what's even better is to calculate a churn likelihood for every customer in your database based on recent data. With this information we are not only able to tell which customers will probably churn next month, but we can also optimize our decision making, such as: Of all customers that will churn next month, which one should be targeted with a marketing campaign? Combining machine learning with BI creates a potentially huge value proposition for an organisation. And with the advance of novel techniques such as AutoML and AI-as-a-Service, which we will explore further in Chapter 3, organizations can reduce the bottlenecks of having not enough data scientists or ML practitioners to leverage these AI potentials.

AI-capabilities for enhanced forecasting or better predictions can be found both as an integral part of an existing BI software (application layer), or they can be applied independently directly on a database level (analytical layer). This makes them always available, no matter which BI tool you are using. We will explore how to apply these techniques in Chapters 6 and 7.

Leveraging Unstructured Data

BI systems typically work with tabular data from (preferably) relational databases such as enterprise data warehouses. And yet, with rising digitalization across all channels we see a dramatic increase in the use of unstructured data in the form of text, images, or audio files. Historically, these forms are difficult to analyse at scale for BI users. AI is here to change that.

AI can increase the breadth and depth of available and machine-readable data by using technologies such as computer vision or natural language

processing to access new, previously untapped data sources. *Unstructured data* such as raw text files, pdf documents, images, audio files, etc. can be turned into structured formats which match a given schema, for example in the form of a table or CSV file and can then be consumed and analysed through a BI system. As this is something that happens at the data ingestion level, this process will ultimately affect all stages of the BI platform (see Figure 1-6).

By incorporating these files into our analysis we can get even more information which can potentially lead to better predictions or a better understanding of key drivers. Chapter 8 will walk you through some examples of how this works in practice.

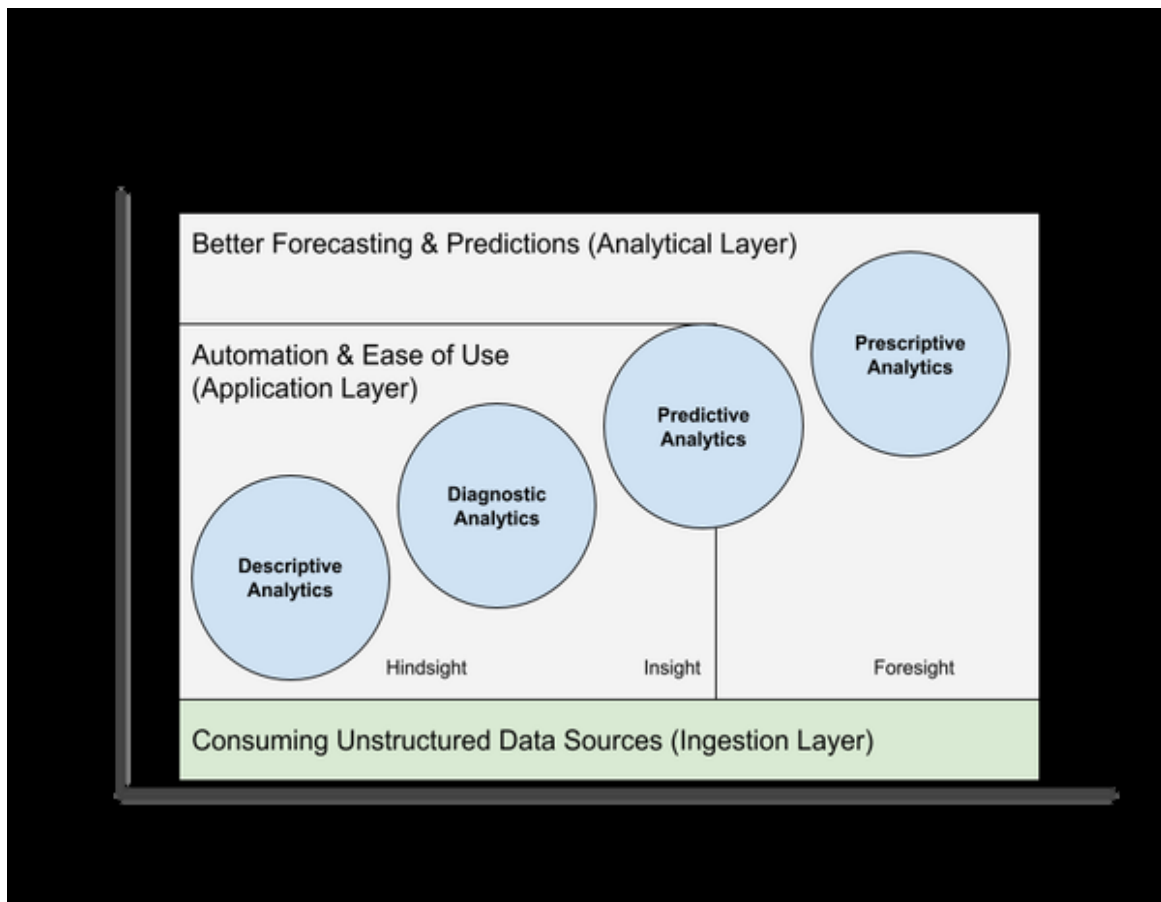


Figure 1-6. AI-powered BI: Unlocking unstructured data in the ingestion layer

Getting an Intuition for AI & Machine Learning

We've talked a lot about how AI can be used with BI. But in order to actually build AI-powered products or services, we need to dig deeper and understand what AI is and what it is capable (and not capable) of achieving.

So what is artificial intelligence, really? If you ask 10 people, you will probably get 11 answers. For the course of this book it is important to have a common understanding of what this term actually means.

Let's first acknowledge that the term "Artificial Intelligence" is not new. In fact, the term dates back to military research labs in the 1950s. Since then, researchers have tried many different approaches to accomplish the goal of having computers or machines replicate human intelligence.

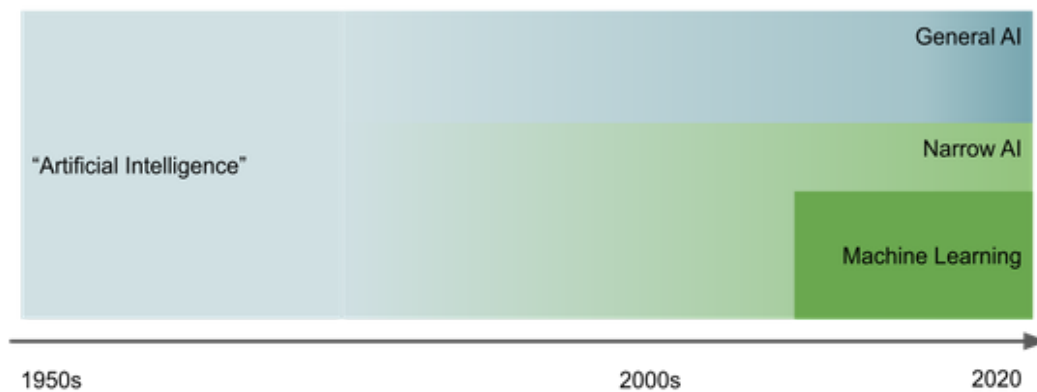


Figure 1-7. Development of Artificial Intelligence

As Figure 1-7 shows, two broad fields of AI have emerged since its inception: General AI and Narrow AI. General AI, or strong AI, refers to a technology that aims to solve any given problem that the system has never seen or exposed to before, similar to how the human brain works. General AI remains a hot research topic, it is still pretty far away: researchers are still uncertain it will ever be reached. Narrow AI, on the other hand, refers to a rather specific solution that is capable of solving a single, well-defined problem it has been designed for and trained on. Narrow AI has powered all the AI breakthroughs we have seen in the recent past both in research and practical or business-related fields.

At the core of Narrow AI there has been one approach that has stood out in terms of business impact and development advancements: machine learning (ML). In fact, whenever I talk about AI in this book, we look at solutions that have been made possible through machine learning.

That is why I will use AI and machine learning interchangeably in this book and consider AI as a rather broad term with a quite literal meaning: AI is a tool to build (seemingly) intelligent entities that are capable of solving specific tasks, mainly through machine learning.

Now that the relationship between AI and ML hopefully became a bit clearer, let's discuss what machine learning actually is about.

Machine learning is a programming paradigm that aims to find patterns in data for the sake of a specific purpose. It typically has two phases: Learning (training) and inference (also called testing or prediction). The core idea behind ML is that we use historic data to find patterns in them to solve a specific task, such as, putting observations into different categories, scoring probabilities, or finding similarities between different items. An example of a typical use case for machine learning is to analyse historic customer transaction data to calculate individual probabilities of customer churn likelihood. With inference, our goal is to calculate a prediction for a new data point given everything that we learned from the historic data.

To foster your understanding of machine learning, let's unpack the core components of our ML definition.

- Machine learning is a programming paradigm:
Traditional software is built by coding up rules to write a specific program. If you develop a customer support system, you come up with all the logic that should happen once a customer files a support ticket, e.g. notify support agents via email. You document all the rules, put them into your program and ship the software.

Machine learning, however, inverts this paradigm. Instead of hard-coding rules into a system, you present enough examples of inputs and desired outputs (labels) and let the ML algorithm come up with the rule set for you. While this setup is ineffective for building a customer support system, it works great for certain scenarios where the rules are not known or very hard to describe. For example, if you want to prioritize customer support tickets based on a variety of features such as the ticket text, the customer type, ticket creation date, etc. a machine learning algorithm could come up with a prioritization model for you just by looking at how past tickets have been prioritized. Instead of handcrafting a complicated if-then-else logic, the machine learning algorithm will figure it out given a certain amount of computation time and computational resources.

- Pattern finding in data: In order to find useful patterns in data, three important concepts play together: Model, algorithm, and training. A machine learning model is the set of rules or the mathematical function that will calculate an output value given a specific data input. Think of it as a big stack of weighted if-then-else statements. The machine learning algorithm describes the computational process a machine has to follow in order to get to this model. And the term training means iterating many times over an existing dataset to find the best possible model for this particular dataset, which yields both a low prediction error and a good generalization on new, unseen data inputs, so that the model can be used for a specific purpose.
- A specific purpose: Machine learning tasks are typically categorized by the problem they are trying to solve. Major areas are supervised and

unsupervised learning. Although this isn't a book on machine learning fundamentals, we are going to cover it in a bit more detail in Chapter 2.

If we consider all of the components, the task of a machine learning practitioner in a real-world situation then is to gather as much data about the situation of interest as is feasible, choose and fine-tune an algorithm to create a model of the situation, and then train the model so that it's accurate enough to be useful.

One of the biggest misconceptions about AI and machine learning that business leaders often have is the fact that AI and machine learning are super hard to implement. While designing and maintaining very specific, high-performing machine learning systems is a very sophisticated task, we also have to acknowledge that AI has become commoditized and commercialized so that even non-ML experts can build well-performing ML solutions using existing code libraries in their code or using no-code / low-code solutions. In Chapter 3 you will learn more about these techniques so you can implement ML solutions by yourself without the help of data scientists or machine learning engineers.

To conclude this section, recall that AI as a term can be scary and intimidating for people who don't really know what it means. The truth is, we are far off from Terminators and general AI. If you want to get broader acceptance and adoption of AI solutions inside your organization, you will need to communicate what AI is with a friendly and non-technical language. Thinking about AI as automation or being able to implement better decisions based on past learnings should make you comfortable enough to spot potentially good use cases and share that spirit with fellow co-workers.

Mapping AI Use Case Ideas to Business Impact

Now that you have learned more about AI and how it can be applied to BI, you might already have some ideas in mind for applying AI to your own use cases. To figure out which of those use cases have the most potential and are worth fleshing out, we will take a look at a story mapping framework you can use for exactly this purpose. The framework is inspired by agile project management techniques and should help you structure your thinking process.

The core idea of this AI story mapping framework is to contrast the present implementation of a process with how an AI-enabled implementation of the process would look like. This technique will give you a high level, end-to-end overview of what would be different, which things you would need to change, and, above all, help you to structure your thinking process.

The creation of a storyboard is straightforward. Take a blank piece of paper and divide it into a table with four columns and two rows. The four upper boxes will map your current process and the lower boxes will describe the future, anticipated implementation. Name the columns from left to right: Setup, actions, outcomes, results. Figure 1-8 shows how your piece of paper should look:

Current implementation	Setup	Actions	Outcomes	Results
Future implementation				

Figure 1-8. Storyboard template

To create your storyboard, you will need to populate the columns from left to right. You start with the first row, outlining how the current implementation of a given process works along the following dimensions:

“Setup” describes how the process starts, and lists your assumptions, resources or starting criteria.

“Actions” holds all tasks and action items that are executed by or on the resource outlined in the setup.

“Outcomes” describes the actual artifacts of the process. What exactly is being generated, created or modified?

“Results” holds the impacts the outcomes have on the business and/or subsequent next steps for the outcomes. For example, displaying a report in a dashboard is an outcome, but by itself does not have any impact. The

impact is what is happening based on the information shown in the dashboard and who is doing this.

In the next step you will do the same for the anticipated future implementation. In the end you will have a head-to-head comparison of the old and the new approach, giving you more clarity about how things are going to change and which impact these changes might have.

To give a little bit more context how this exercise works, Figure 1-9 shows a sample storyboard for our customer churn use case.

	Setup	Actions	Outcomes	Results
Current implementation				
Future implementation				

Figure 1-9. Storyboard example

Let's walk through our storyboard example. We start on the top left corner, laying out the current setup for the existing process.

Currently, customer churn is detected by sales people who get feedback from existing customers when talking to them in their regular meetings or

by customer support employees who receive feedback from customers that some things are not working out as they hoped or they face other issues.

In the next step, customer support or sales try to solve the problem directly with the customer, for example, providing onboarding help.

The main outcome of this process is that customer support (hopefully) resolves existing pain points and problems for the customer. Pain points might be reported to a management level or complaint management system.

As a result the customer will hopefully stay with the current service after the issue has been resolved.

Let's contrast this with how an AI-enabled implementation would look like, starting with the bottom left corner and proceeding right.

In our setup we would collect historic data about how customers were using different products and services and flag customers who churned and did not churn. We would also bring in people from sales and customer service to share their domain expertise with the analyst in the loop.

Our next action would be to analyze the historic data to determine if key drivers of customer churn can be identified in the dataset. If so, we would develop a predictive model which would calculate an individual churn risk for each customer in our database as well as provide insights for why the churn might be likely to happen.

As an outcome, these churn risk scores and churn reasons would be surfaced to the business, e.g., through a report in the CRM or BI system where this information can be blended with other metrics, e.g. customer revenue.

With this information, customer support could now reach out proactively to customers with a high churn risk and see if they can solve the problem or remove roadblocks before the customer actually flags a support ticket or churns without opening a ticket at all. As a result, the overall churn rate should reduce over time because the organization can better address reasons for customer churn at scale.

With both story maps - the existing and the new process - you should feel more confident to describe how a possible AI solution might look like, what benefits it could bring and if it is even reasonable to go for the new approach to either replace or blend it with the existing process.

As a conclusion, the idea of a storyboard is to provide a simple one-pager for each use case which intuitively contrasts the difference and benefits of the existing and the new solution. It will help you to structure your thinking process and is a solid starting point when it comes to prioritizing AI use cases.

Exercise: Use the storyboard template and map two or three AI use case ideas. Which of these ideas seem to be the most promising to you?

Summary

In this chapter we learned how AI is changing the BI landscape driven by the needs of business users to get quicker answers from data, the growing demand for democratized insights and an overall higher availability of commoditized machine learning tools. We explored how exactly AI can support BI through automation and better usability, improved forecasting and access to new data sources, thus empowering people to make better decisions. By now you should have a basic understanding how AI and ML work and what these technologies are capable of today. You also learned a framework which you can use to structure your thinking process and craft out ideas for ML use cases. In the next chapter we will take a deeper look on how AI systems are designed and which factors you need to consider before implementing these technologies in your enterprise BI services.

¹ <https://blogs.gartner.com/jason-mcnellis/2019/11/05/youre-likely-investing-lot-marketing-analytics-getting-right-insights/>

Chapter 2. Assessing Feasibility for AI Projects

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 2nd chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

In the previous chapter, you learned how machine learning capabilities could drive business impact. But to create a roadmap of prioritized use cases and make an informed decision about which use cases to pursue as a priority, we need to consider another dimension of criteria: Feasibility.

This chapter will dive much deeper into the fundamentals of machine learning to enable you to assess the complexity and overall feasibility of a given AI use case. We will explore feasibility based on three main topics: Data, Infrastructure/Architecture, and Ethics.

As a result, you will be able to create the first version of your AI-powered BI use case roadmap.

Putting Data First

AI projects require a different mindset than classic BI projects.

Most BI projects are done in a relatively straightforward manner, often following the traditional waterfall model: define the metric you want to show, design the data model, integrate the data, and make sure it works (hard enough). Iterate if necessary. Job done.

The main difference in AI projects is that - even under ideal circumstances - the outcome is highly uncertain. We simply do not know if an AI model will work with our data and be good enough to deliver value until we see it in action.

For this reason, AI projects typically require multiple, shorter iteration cycles in an agile-like project framework such as CRISP-DM. A common way to approach AI projects and deliver ROI is first to develop a minimum viable solution or prototype and, only after validating the prototype, launch a large-scale project. We will discuss this topic in more detail in the following Chapter 3.

Nevertheless, there are some things you can do even before you build a prototype: Think about what problem you want to solve and what data you will use to do it. At this stage, you do not need to worry about technical complexities like databases or data formats yet. Instead, I recommend you look at the data from a high level and use simple words to describe what you are trying to achieve.

In our previous example, it might look like this:

“ We want to use historical data mainly from the CRM and ERP system to predict customer churn for the next month.”

Your **essential data questions** about this use case might be:

- What data do you need for your use case?
- Do you have this data available? If so, what exactly do you have (tables, media files, text, etc.)?
- Do you have access to this data?
- Who can access it?

- Does your company own it, or do you have to buy it?
- Do you have legal authority to use the data?

All these questions still have nothing to do with concerns like “this is not in the data warehouse” or “we cannot stage PNG files”.

If you cannot answer these essential questions about the data, it is advisable not to proceed further with the data assessment. Instead, it would be better to develop ideas to answer these questions. How could you get the data? Who would you need to talk to? And so on.

But let us assume you could answer these essential questions and the data seems to be available (at least in theory). In that case, it’s worth exploring some of the data features in more detail before creating your prototype.

There is no gold standard for doing this systematically. However, one popular framework is the 4V framework, which I like for its simplicity. We will explore this framework in more detail in the next section.

Assessing Data Readiness with the 4V Framework

The 4V framework helps you better understand the characteristics of your data without getting too technical in the four dimensions of Volume, Variety, Velocity, and Veracity. This approach should enable you to consider which use cases are more or less realistic at a given stage.

Again, you should only perform this analysis after you have answered more fundamental questions, such as: Do you actually have the data, and are you allowed to use it?

Volume indicates how much data is available at any given time. You can interpret this in at least two ways: First, the number of observations (examples, rows), and second, the depth of information contained in the data (attributes, columns).

Different types of machine learning require different amounts of data, so it's hard to make a reasonable estimate of what "enough" data is. However, to give you at least a rough idea, see Table XX for data requirements for different machine learning techniques.

ML Capability	Rule of thumb volume needed
Regression	~ 100 examples
Classification	~ 1.000 examples per category for modest feature dimensionality (10)
Image classification/detection	~ 100 examples per category for clearly different categories (cats vs. dogs) ~ 1000 - 10.000 examples for more similar categories (e.g., dog breeds)
Text / Natural Language	~ 500 examples per class for tasks like sentiment or entity detection

In most cases, models get better as more data is added. Therefore, more data volume is almost always desirable. However, volume alone does not automatically add value.

Variety: Most of the data in your organization is not even stored in tabular form. Raw text, images, or log files, for example, fall into a category often referred to as unstructured data. Unstructured data means that the data does not have a reliable schema suitable for your purpose. While structured data

is most amenable to machine learning models, unstructured data may require extensive preprocessing or an AI service to convert it into structured data (as we will find out in later chapters). Variety, however, can also be interpreted in the context of tabular data and refers to how well your data represent the real world, including edge cases, or are generally representative. For example, do you have observations on all class labels in your dataset? Depending on how you view this term, variety may be something to aim for or avoid. Either way, you need to choose a coherent approach that fits your organization and an evaluation framework to assess different use cases in the same way.

Velocity is the rate at which data is produced and flows through your systems. Are you dealing with batch data that is updated perhaps once a day? Or are you dealing with streaming data that is constantly being updated? The velocity of the data affects two things: technology requirements and data drift. Streaming data tends to place higher demands on your infrastructure. And high-velocity data needs to be monitored more closely for data variance. High-velocity data that remains consistent can help train a machine learning system quickly because it helps accumulate volume. However, if this data changes rapidly over time due to its velocity, it can be difficult to maintain a consistent view of ground truth at any point in time.

Veracity: How accurate are the data as a representation of the real world? Do they exhibit inconsistencies, incompleteness, or ambiguity? Veracity is about whether your data is good enough for its intended use. And in the case of machine learning, this means first and foremost: does the data contain ground truth, i.e., do you have data labels? If your data does not contain labels, the only way to get them over time is to buy a labeling service or wait for a business process to produce labeled data as output.

Combining 4Vs to Assess Data Readiness

By looking critically at all 4Vs together, you can begin to see some of the relative strengths and weaknesses of your data for a particular use case. The

following table XX summarizes some of the key questions you should ask yourself for each category and what scale you might use to evaluate them.

Dimension	Questions	Possible scores from... to ...	
Volume	How much data do you have available? How much data will be produced?	Small Amount (1)	Large Amount (5)
Variety	Does the data contain enough variety to capture even rare events? Does the data contain so much variety that it holds too much noise and requires heavy data cleaning?	Undesired Variety (1) Poorly representative data (5)	Desired Variety (5) Highly representative data (5)
Velocity	How often are relevant data sources produced or updated? Are data sources updated often enough to retrain the model soon enough to mitigate the risk of data drift?	Low velocity (1) High risk of data drift (1)	High velocity (5) Low risk of data drift (5)
Veracity	How accurate is the data? How complete is the data? How consistent is the data? Do you have labels?	Poor data quality (1) No labels (1) Poor fit for use (1)	High data quality (5) All examples correctly labeled(5) Good fit for use (5)

With these scores you could for example create net charts as shown in Figure XX for the churn use case example. The chart can be interpreted as follows:

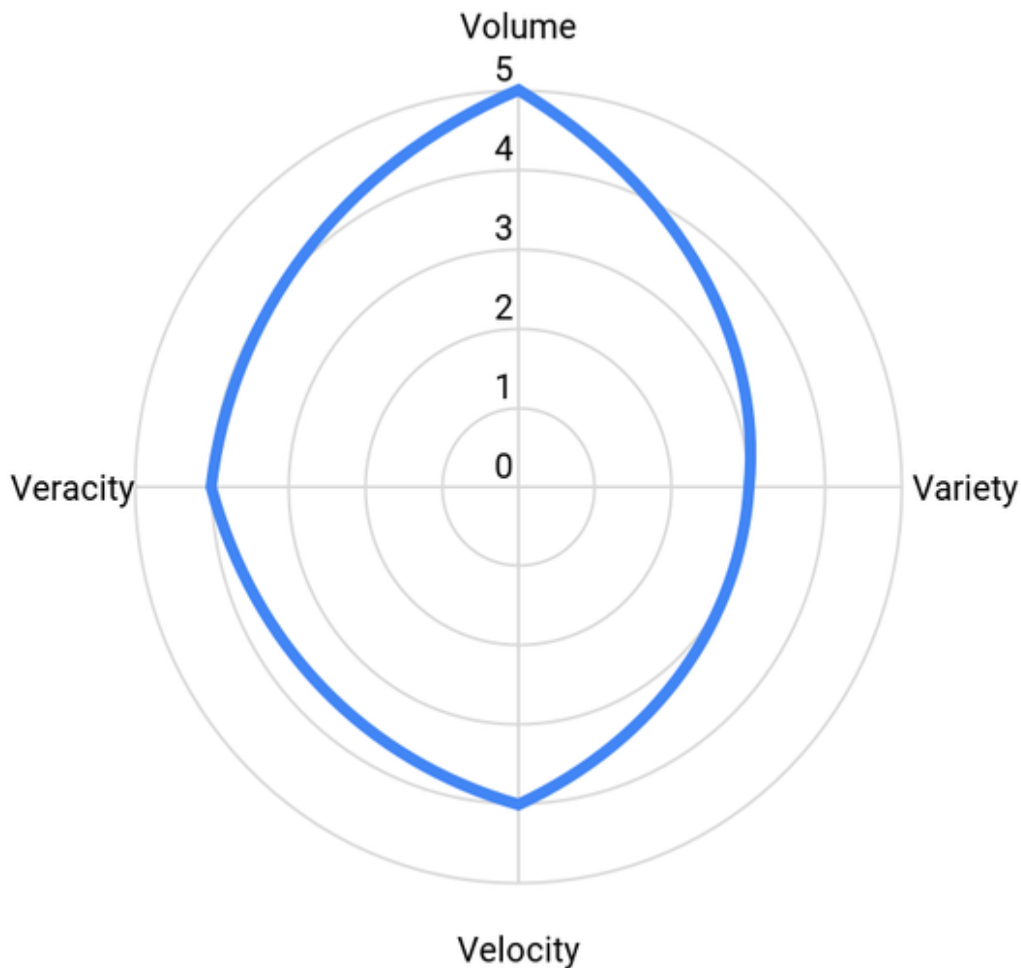


Figure 2-1. Figure caption placeholder

Example: Net chart for the customer churn use case

- Volume scored 5 out of 5: We have a large customer base with many attributes regarding the purchase behavior and we think it will be enough for training machine learning models.
- Variety score 3 out of 5: While all of our data is tabular and should be easy to digest for a machine learning model, we expect that we will

have only very few examples of long-term, high-value customers. On the other hand, there might be lots of new customers where we don't have much information available such as buying preference because this data might not be in the CRM system nor captured elsewhere.

- Velocity scored 4 out of 5: Data in our CRM and ERP system should be quite up to date and updated at least daily. We assume that this is frequent enough to avoid data drift.
- Veracity scored 4 out of 5: We can obtain the true labels (Churned) from the data itself, so no labeling service is needed. Furthermore, we expect data in the CRM system to be mostly correct due to a data governance process in place.

This net chart would indicate a feasible use case from a data perspective at this stage and allow us to compare this approach with other use cases as well.

Make or Buy Decisions for AI Services

Building an AI solution is a time-consuming process that requires a significant amount of ongoing maintenance. Figure XX shows an example process with all the steps required to build an end-to-end AI solution by yourself.

When you think about this and the complexity involved, sometimes the best AI solution is not to develop an AI solution at all. The good thing is that you do not necessarily have to do all of these steps yourself. One option might be to develop only some components on your own, another to rely entirely on off-the-shelf AI services. Ultimately, like any other business decision, AI solutions are a make-or-buy choice.

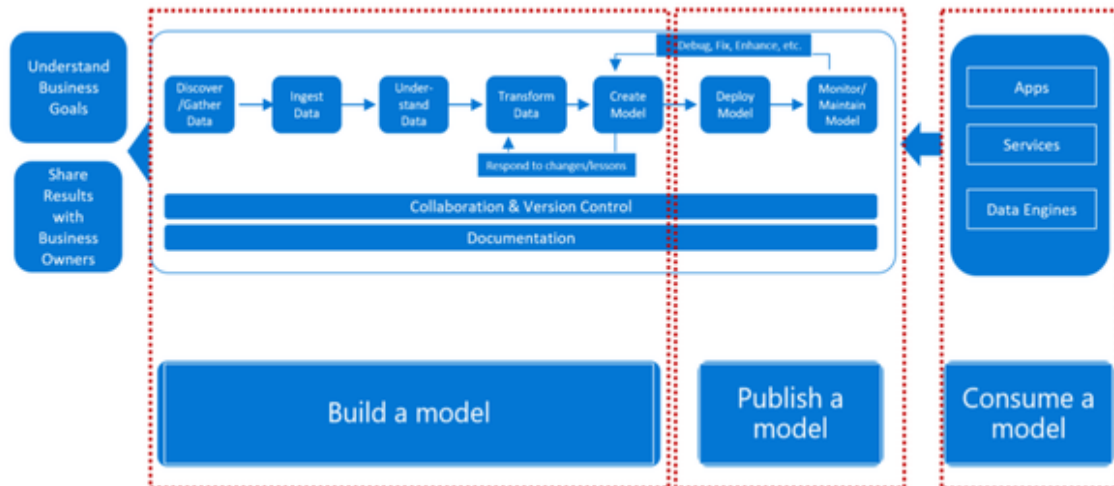


Figure 2-2. Source: <https://medium.com/microsoftazure/azure-machine-learning-deployment-workflow-475fc939e96>

To give you an overview of the possible stages of AI-related make-or-buy patterns, let's look at the different stages you could choose as a business and briefly review their relative pros and cons.

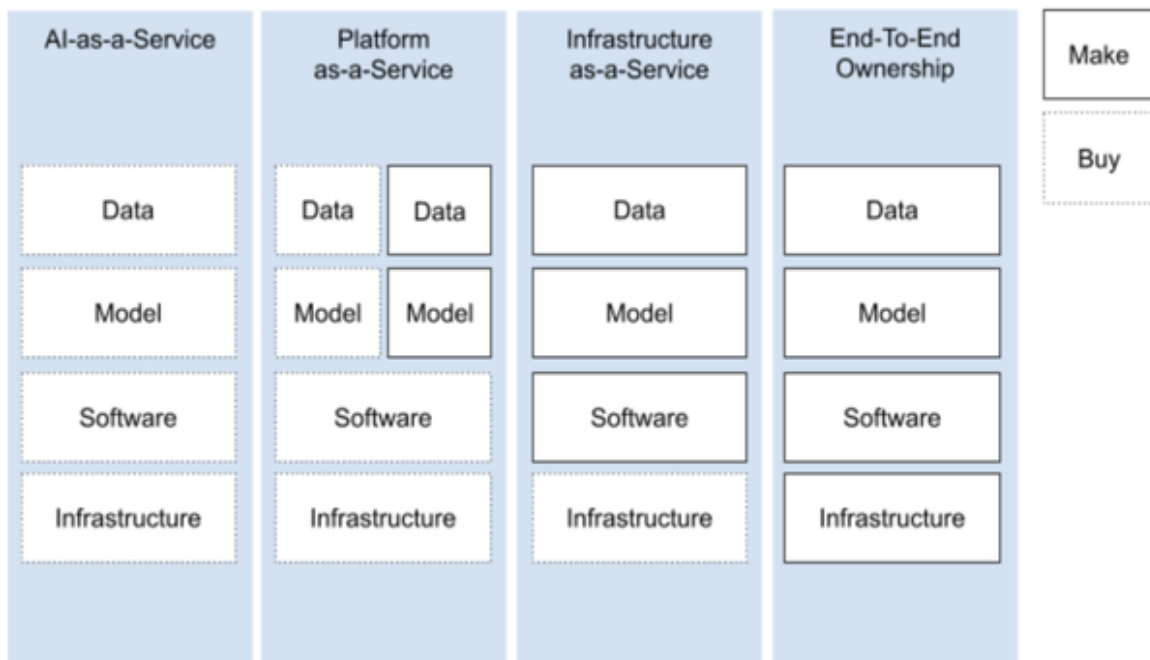


Figure 2-3. Figure caption placeholder

AI-as-a-Service

With AI-as-a-Service (AIaaS), you rent a fully managed AI service where you typically pay per use (e.g., API calls). For example, you send an image to an API and receive back a JSON document with the labels recognized in the image.

The advantage is that you do not have to develop or maintain anything. You only pay for what you use, and often there is a free tier so you can try out the AI service before spending money to see how well it works with your data. Also, AIaaS offerings do not require training data from you. The provider has trained the AI service on specific use cases, such as facial recognition or sentiment detection, and you can go straight into inference mode. Since there are a variety of AI services for different common use cases, it is relatively easy to switch between different services, so you can try different services without risk. Last but not least, AI services do not require ML expertise or knowledge. If a graphical user interface (GUI) is provided, business users can use the service without intensive training.

The biggest drawback of AIaaS offerings is that they are usually black boxes. That means you do not know what's happening inside the service. You only get input and output, nothing more. Also, you can not modify the AI yourself. So if you want to specialize in your own data or use different labels than the ones provided by the service, there's usually not much you can do. For this reason, off-the-shelf AI services like Azure Cognitive Services, Google Cloud Vision, Amazon Rekognition, etc. usually work well for very general use cases (e.g., detecting sentiment in user reviews), but they quickly deteriorate for more specific use cases. For example, you typically can not use an AI service to extract specific things like product names from text data. Last but not least, you need to trust the company providing the service. It may not be in its best interest to delete data for every API query, so you generally lack flexibility and control over your data.

Platform-as-a-Service

Platform-as-a-Service (PaaS) is the way to go for most enterprises. With the concept of PaaS, you get access to a managed machine learning platform

where you can train your own models or access already pre-trained models through marketplaces. Typically, you pay per license, per use, or a combination of both. If you want to build models from scratch, most ML platforms support you with tools like AutoML. AutoML, or automated machine learning, uses machine learning itself to find the best-fitting model for a given dataset, so even less experienced practitioners can quickly create a good initial machine learning model with the click of a mouse. We will explore this approach a bit later in this book. Of course, with concepts like AutoML, you'll need to bring your own training data.

If you have no or very little training data, most ML platforms provide marketplaces where you can buy ready-made AI models for various use cases. These are called pre-trained models. In some cases, you can tune these models to your data, which requires much less training data than training a model from scratch. For example, instead of requiring thousands of images to build a computer vision model from scratch, fine-tuning would only require perhaps a dozen images of each category to produce good results, depending on the use case.

Many PaaS offerings also support you with more advanced features such as a custom labeling service or support for deploying and monitoring your models.

In most cases, ML platforms run in the cloud (examples include Azure ML Studio, Amazon Sagemaker, Google Vertex AI), but there are also many vendors that support on-premise environments, such as Datarobot or H2O.

Here are the main advantages and disadvantages of PaaS:

Pros:

- Ease of use: In most cases, you won't have to worry about infrastructure as it is provided by the PaaS vendor.
- Speed: It's a big time saver not having to do infrastructure setup and maintaining it. Faster onboarding for non-ML experts.

Cons:

- Expensive at scale: If you need to train many different models or use cases, it can get very expensive quickly.
- Vendor lock-in: You are locked into one platform for training your models and deploying them. What if you need to swap?

I would say that for most companies that want to experiment with machine learning, this is the perfect way to start. PaaS gives you a well-maintained and easy-to-use infrastructure that is still flexible enough to adapt to your unique needs. You can experiment quickly without having to deal with too much overhead.

Infrastructure-as-a-Service

In the Infrastructure as a Service (IaaS) scenario, you rent storage, compute and network services from a cloud provider on a pay-per-use basis. You can deploy any machine learning frameworks you want. The IaaS approach makes sense for companies that are willing to invest heavily in machine learning and software talent, but do not want the burden of dealing with infrastructure hardware. This approach is typically recommended when your organization has reached a certain level of maturity in developing and maintaining machine learning models.

The pros and cons of IaaS in a nutshell:

Pros:

- High flexibility: Easy to scale up and down. This allows for quick adjustments of your resources according to business needs.
- Avoiding lock-in: If you're just using infrastructure you can switch providers relatively easily if necessary to reduce costs or improve quality of service.
- Focus on data science: In IaaS scenarios you have full control over your data and software. Therefore, you can focus on the development of your ML algorithms without being distracted by infrastructure-related issues while having full customizability.

Cons:

- Cloud experts needed: Managing cloud infrastructures needs expertise. If you don't have the talent, this will be a critical bottleneck.
- ML experts needed: Since you are not relying on a pre-built ML platform, you will need to do the software integration yourself, which is time-consuming and requires expertise.

IaaS is well suited for companies that want to focus on their data science and software development while having the ability to scale up and down quickly.

End-To-End Ownership

With end-to-end systems ownership, you are responsible for everything from infrastructure hardware, networking, software frameworks, databases, etc. Fun fact: Many companies unknowingly start here because they already run an on-prem infrastructure and have their data scientists doing things in Jupyter notebooks on their computers. This approach will soon run into bottlenecks if you try to implement it at scale without proper platform management.

End-to-end ownership is usually the most complex scenario, and also the most resource-intensive if you are doing it for more than a handful of use cases. To justify the overhead, your company needs at least one very strong AI use case that will make or break the business (think autonomous driving at Tesla). End-to-end ownership is mandatory if you want to create an initial POC but are not allowed to move data to the cloud or your company policies do not allow you to adopt an ML platform.

When choosing your AI infrastructure approach, remember that not every company needs to own the entire process from the ground up. The level you need depends on your use case, your budget, and most importantly, the talent available.

Machine learning solutions are not a one-time process but require continuous development and maintenance.

Think twice about which of these pieces you want to focus on and how you will benefit from them.

In our customer churn analysis example, we would likely choose a managed ML platform because we need to build our own model for this use case (there is no AI that can perform customer churn analysis as a service without training) and the data from our source systems can likely be easily uploaded to the ML platform.

Basic Architectures of AI Systems

Now that we have covered the feasibility aspect of AI use cases in terms of data and infrastructure setup, let us briefly cover the general architecture of modern AI systems. I promise we will not get too technical here. But if you understand how AI use cases are built, you will be in an even better position to assess the technical feasibility of a particular AI use case.

At the top level, we can typically identify three layers on which AI solutions are built: The data layer, the analytics layer, and the user layer, as shown in Figure XX.

User Layer

Analysis Layer

Data Layer

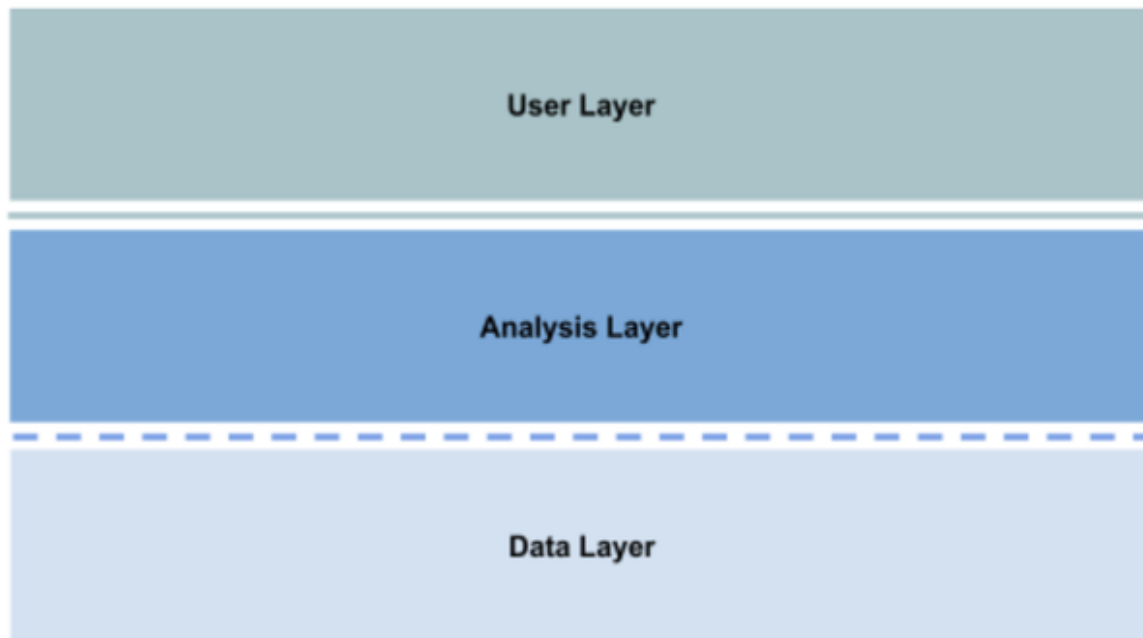


Figure 2-4. Figure caption placeholder

This high-level framework will be enough for us to understand

1. what problem we want to solve for the user and how the outcome should look like (User layer),
2. what ML capabilities we are going to use for this (Analysis Layer)
3. and which data we need with which processing. (Data Layer).

The underlying technical architectures are of course much more complex, but this high abstraction will get you very far.

Later in this book, you'll find these high-level architectures for the use cases we will cover, so you can quickly see how the solution works.

Let me briefly explain each layer, what it means, followed by two example architectures.

User Layer

The first layer is the user layer and it is usually a good step to start here. This is where we determine what problem we want to solve and, more importantly, what our solution should look like. Are we going to host an

API that users can access? Or is our end result a dashboard that displays a specific forecast in your BI? Being clear about our end result will give us the scope we need for our AI use case, even though the desired outcome may change during the development process. Here are some popular elements that can be used in the user layer:

- API
- Web Application
- Dashboard / BI Integration
- Files (Excel, CSV, ...)
- Legacy Connector (SAP, custom enterprise software, ...)

Data Layer

Following closely on the introduction to this chapter, let us first look at the data we want to use to achieve the proposed outcomes in the user layer before we get into the technical details of the analysis. The data layer contains components that describe the sources or source systems of the data we want to use. Some example components are:

- Databases
- Media files
- Data Warehouse
- APIs
- Files (CSV)
- User inputs
- Legacy Connectors (SAP, custom enterprise software, ...)
- ...

In addition, we can use the data layer to describe at a high level what we intend to do with the data before applying a ML service. The steps include:

- Labeling data
- Merging data
- Aggregating data
- Reshaping data
- Simulating data
- ...

How far this goes depends on your use case. For some use cases, reasoning about the data sources and their manipulation will be the main part whereas for others, the focus might be more shifted towards the User or Analysis layer.

Analysis Layer

Now that we have scoped the problem and the data we have, it's time to move on to the middle part of our architecture, where the magic happens with our ML capabilities. Note that incorporating ML capabilities is not necessarily required here. Instead, you can also consider filling the middle layer entirely without ML to get a first baseline. Are there any business rules or heuristics you can apply? Are there other existing models? If so, integrate them and put them together as your “base architecture.” Once you have created this, you can start replacing your baseline components with the ML-supported functions. This way you will have a clear indication of where your ML-supported approach is likely to add value.

Be aware that your architecture is not necessarily one-way, which means that data can flow back and forth between the different layers. You'll see what this means if we briefly look at two examples:

The first architecture example is shown in Figure XX and illustrates a possible setup for the customer churn use case.

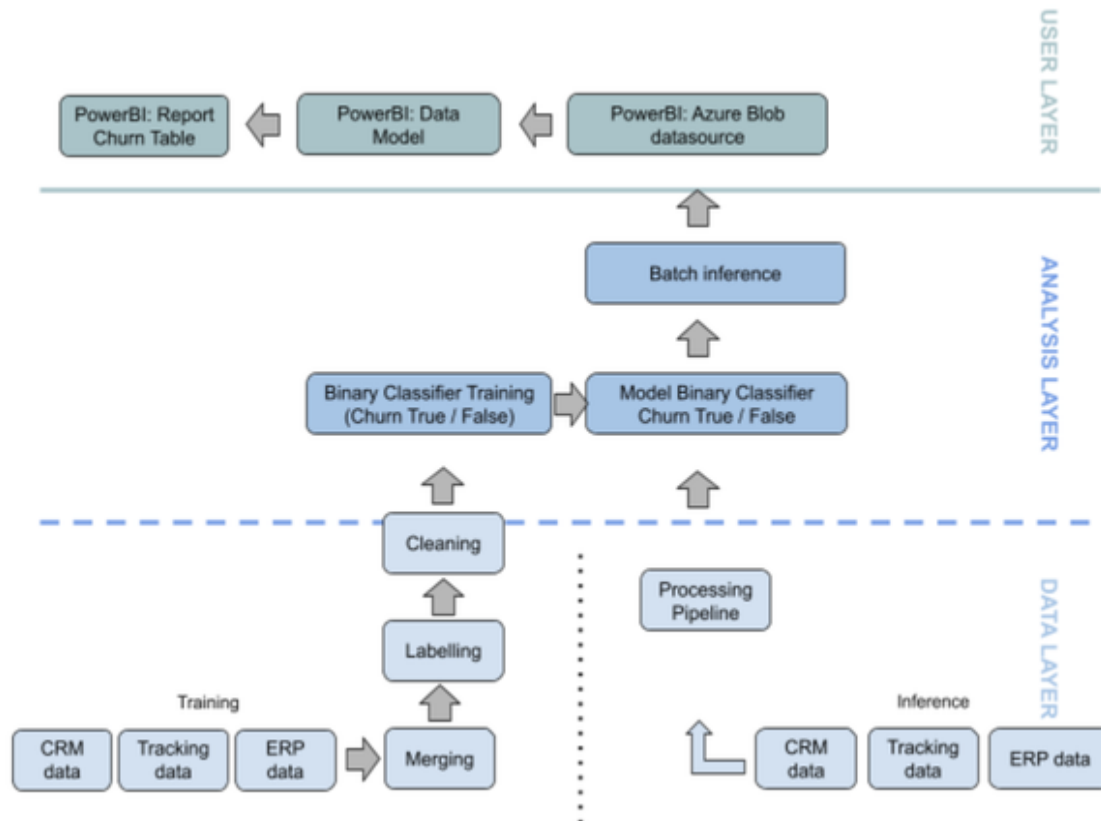


Figure 2-5. Figure caption placeholder

Let us go through this architecture in reverse order, starting with the user layer. From the box in the upper left corner “Power BI: Report Churn Table” we can see that the end result should be a report in Microsoft Power BI showing the predicted churn table. To do this, we will provide a data model (different tables and their relations) that the user can see in PowerBI (hence, this is still at the user layer). Power BI accesses the data from an Azure Blob storage. Moving down to the analysis layer: the data from the Azure Blob store comes from a batch inference job that uses a binary classifier to create the churn true/false labels for the data that has gone through a pre-processing pipeline from the CRM, ERP and web tracking systems. To obtain the model, we used the same data sources for training, merged, labeled and cleaned them, and used this data to train the binary classifier that performs the predictions.

Without having written a single line of code or even touched a single data source, we can use this architecture to discuss it with business stakeholders

and ask questions like:

- Is this the way the solution was intended to be?
- How complex do we think this will be?
- What is the minimum viable product or prototype we could ship?
- Does that fit the story mapping we did?

Chat application prompt: How can I help you today?

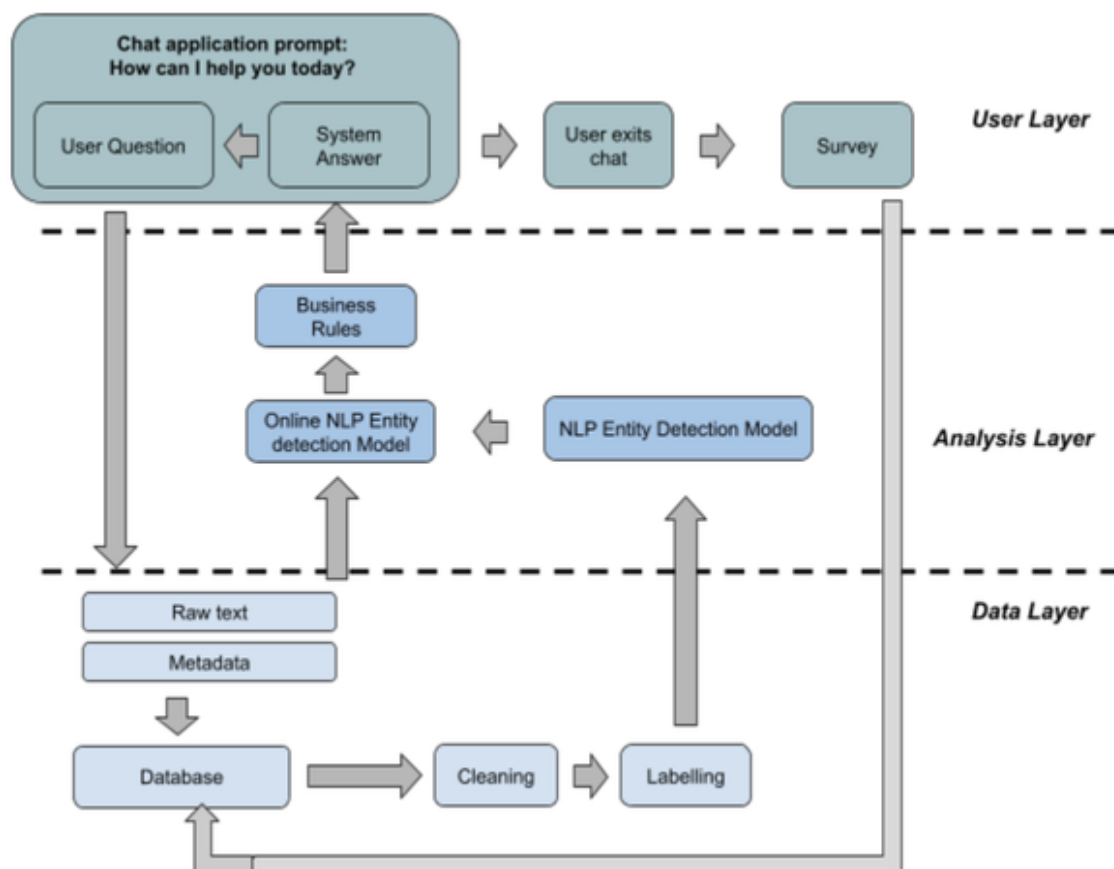


Figure 2-6. Figure caption placeholder

Let us take a look at another architecture.

Figure XX shows an example of a chatbot use case where the data moves back and forth between the different layers. In the upper left, you can see that the final user interaction should be a chat application on a website, which typically starts the conversation with a prompt like “How can I help

you today?”. The user responds with a question, which we store in a database along with some metadata (timestamp, user information, if allowed). In parallel with storing the data, we feed the user text into a real-time NLP model for entity recognition that extracts entities from the user input. These entities can be general topics such as “billing” or “technical issues” or more detailed entities such as product names. In the case of this very simple chatbot, the entities are used to trigger a business rule. This could be a response with pre-written answers related to the entity mentioned by the customer or asking a clarifying question. The user responds with another input and the process continues until the user leaves the chat. When the user leaves the chat, they are invited to a survey and asked for feedback. This feedback is stored in our database and combined with previous chat histories that we have collected. We clean this data and assign labels, i.e., we label entities in the historical conversations. We can then use these labels to re-train the NLP entity recognition model and improve the existing solution.

As you can see from this use case, the architecture can be bidirectional and data can flow back and forth between these different layers.

This high-level framework has helped me a lot in scoping AI projects and discussing use case ideas with technical and non-technical stakeholders. I hope this framework will also be helpful to you as you think about your next AI use case.

Ethical Considerations

The final topic we will address to assess the feasibility of our AI use case is the concept of ethics. AI ethics is an immensely growing field, and even if you intuitively would not think about it (“I do not want to hurt anyone!” I hear you say), it’s still important to consider it, at least in principle.

In this section, I want to provide you with a concise framework to help you identify for which areas certain AI use cases might be critical or less critical.

Let us get this out of the way right away: When we talk about AI ethics, we are talking about AI services that are highly specialized tools, not self-aware entities that think about the consequences of their actions (and are thus exempt from responsibility for themselves). Rather than assessing the AI service itself, therefore, we need to assess the use case or application for which AI technologies are being adapted. And it is the people who are responsible for them who ultimately bear the responsibility and must be held accountable.

In almost every AI use case, there are potential ethical considerations that must be addressed both before and after the solution is implemented. In some cases, these issues are so serious that the use case cannot proceed. In some cases, you need to make changes to the use cases to be morally safe. And there are use cases where the risk of discriminating against people on a large scale is very low.

The point is that as a business leader or someone trying to think about developing AI solutions, you are responsible for going through this thought process before you try your first prototype in the wild.

“Critical” in this case has two dimensions:

The first dimension, “Ethical Criticality,” ranges from applications that have no impact on other humans to those that involve the life and death of one or more humans.

The second dimension, “Privacy Criticality,” ranges from non-personal machine data to personal-relatable data to highly sensitive personal data based on privacy categories.

The framework is shown in Figure XX. Some applications are presented and classified here as examples.

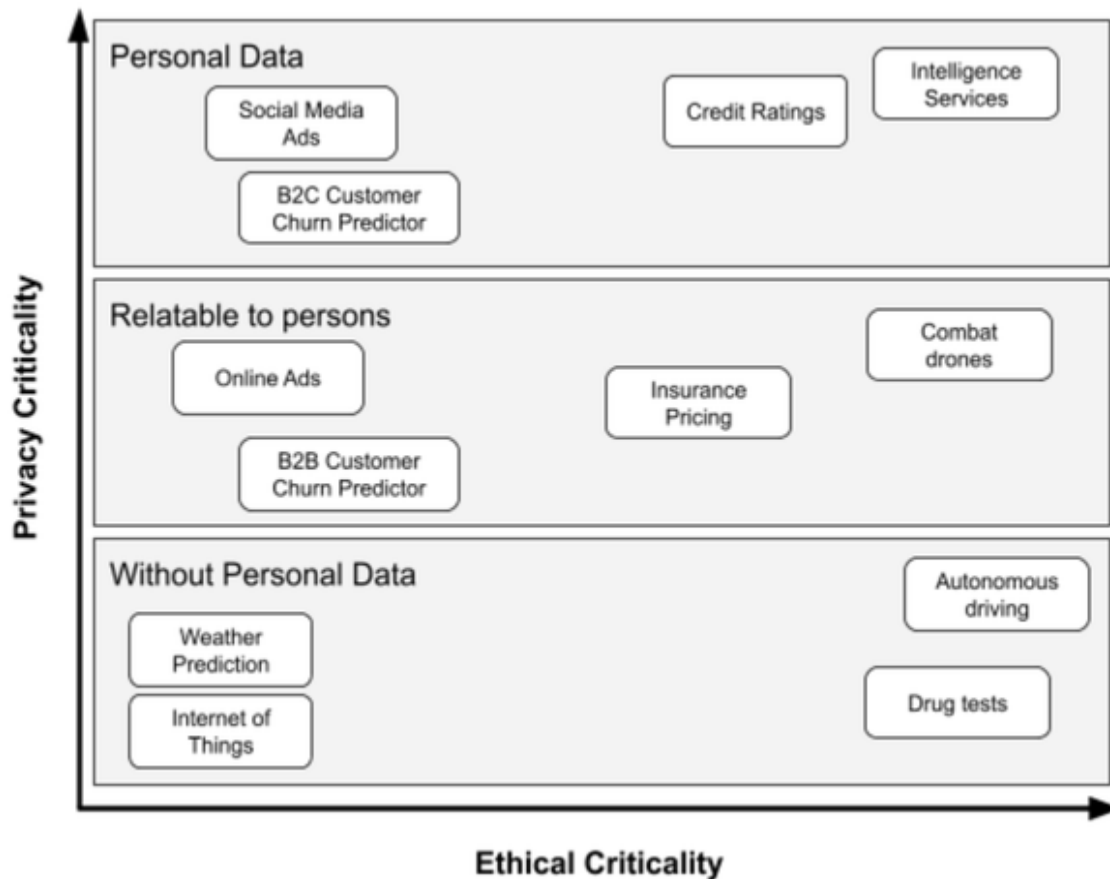


Figure 2-7. Source: *AI Ethics*, Springer (Zwingmann, Gärtner)

The applications in the lower left corner of this diagram seem to be the most harmless, since data without personal reference is sufficient and the results have little or no impact on others. Many Internet of Things applications are in this category, for example. Think of quality control in a factory, where an AI-based system determines whether a workpiece meets quality criteria. When dealing with items that pose little risk to humans (think computer keyboards), the manufacturer's quality requirements for AI can be enforced without extensive ethical considerations.

However, when the products have a significant impact on people's lives, the use case is ethically very critical, even if no personal data is used. Take medical drug testing, for example. At some point in a medical experiment, the data is so highly anonymized that it is impossible to associate data points with specific individuals. Now imagine a use case where you are evaluating the development of a new AI service that predicts whether or not

a drug will be successful in a particular cohort of patients, rather than performing a manual statistical analysis. While the data required for this does not include personal data, the entire use case is ethically very critical, as the end result (produced using AI) can make the difference between life and death for people.

Let us look at the other extreme at the top right of the diagram. These are the highest risk areas that require the most careful ethical consideration. They include use cases that directly tailor or build on personal data and directly impact people's lives - on a large scale. Intelligence services fall into this category, but so do credit ratings. If your algorithm goes wrong in production and you serve millions of customers, you prevent potentially huge numbers of people from accessing financial resources or other services like housing, shopping, etc. that rely on that rating data.

Taking the example of our customer churn use case, we could position this use case in the middle left portion of the chart if it is a B2B churn predictor. In this case, we would calculate the churn probability for accounts (companies) rather than individual customers (people). This makes the data related to individuals, but not as immediate as, say, a B2C churn example where we calculate churn scores at the individual level. In any case, the impact of our churn predictor on individuals is rather small. In the worst case, some individuals would receive a bad performing (or no) personal offer with an incentive to stay.

When developing AI use cases, you should be aware of both - the amount of personal data needed to realize the use case, and secondly, the impact your solution will have on people. Both categories should help you prioritize which use cases require deeper ethical engagement and which seem less critical.

Whether you take this approach or another: I strongly recommend that you develop and implement a consistent ethical framework for evaluating machine learning use cases. This framework does not have to be super complex and lead to a result with double-digit accuracy. It can be broad and highly customized to your needs. Most importantly, it must be consistent

and comprehensive. Both your customers and your business will thank you for it.

Creating a Prioritized Use Case Roadmap

By the end of this chapter, you should be able to assess the feasibility of your use case based on the three themes of data, infrastructure/architecture, and ethics. This will give you the second dimension you need to complement the impact score we determined with the story mapping exercise from Chapter 1.

We can now create a simple diagram for each use case that shows impact and feasibility, as shown in Figure XX:

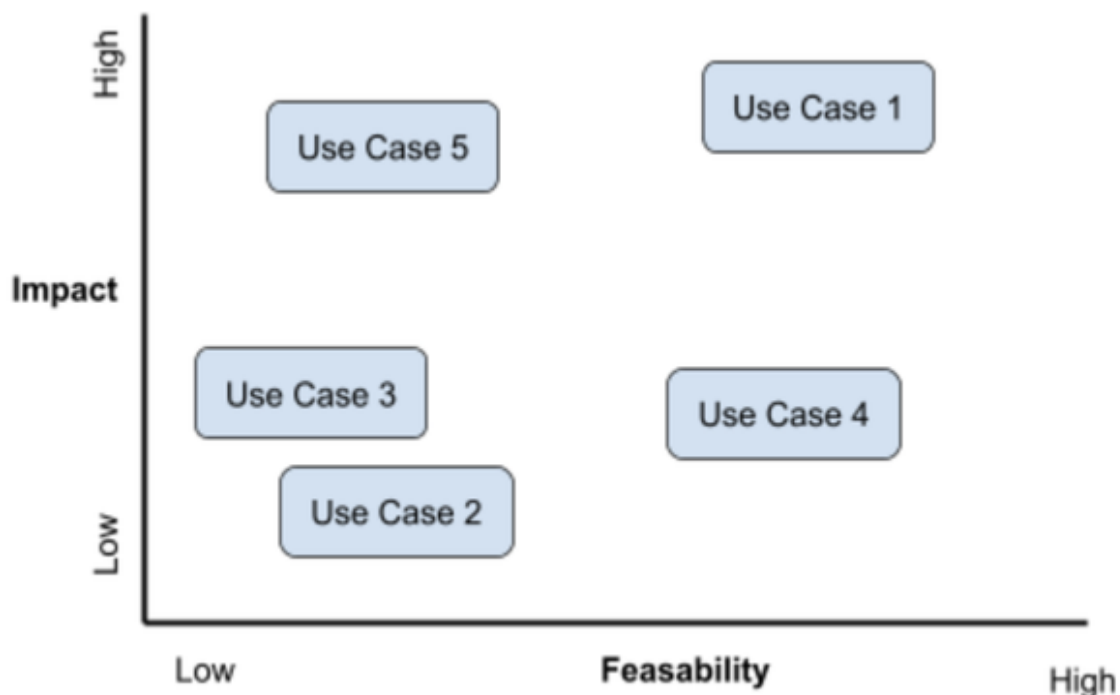


Figure 2-8. Figure caption placeholder

As you may have noticed, we have not used a formal scoring metric in this chapter to calculate actual numerical values for each use case. You can do that, and more sophisticated approaches do just that, but my advice is to start simple and compare all your ideas or use cases on a relative scale among each other. Which use case is likely to have the greatest impact,

which the least. Which use case do you think will be relatively easy to implement and which will be more challenging?

Following this system will give you four types of use cases to map to your diagram, as shown in Figure XX.

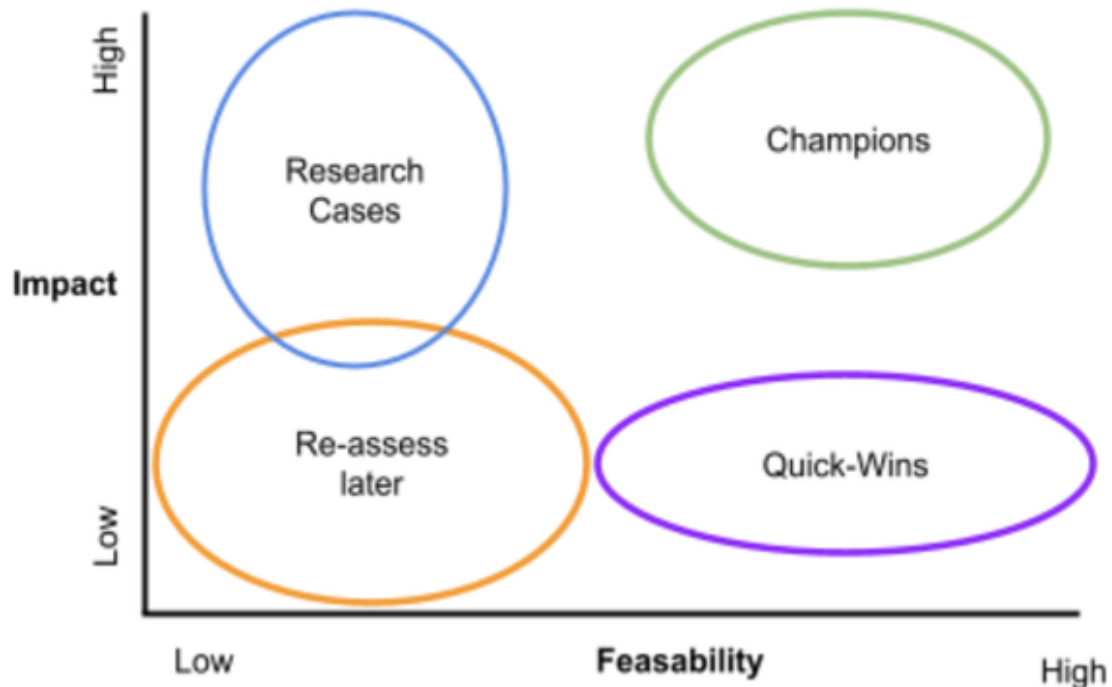


Figure 2-9. Figure caption placeholder

These use case areas are the following:

Champions: Use cases that have high impact and high feasibility. These should be your first priority and drive your ML roadmap.

Quick wins: use cases have similar high feasibility to champions, but lower business impact. Because these use cases are often less complex, they are easier to implement. However, they can be a good showcase and demonstrate the value of machine learning and AI applications. Think of this as an area where off-the-shelf AI services can be used, for example.

Research Cases: These are areas that have a very high impact, but are generally not feasible in the short term. Either you do not have the data, you are not allowed to use it (yet), or there are certain technical limitations.

However, because these use cases potentially have such a big impact on your business, do not neglect them, keep them on your radar.

Reassess later: These use cases have the potential to consume enormous resources without contributing much business value. They should be put on the back burner and re-evaluated later. Use cases with low impact and low feasibility can turn into quick wins as technology changes. For example, some time ago, it was very complex to create an automated priority ranking for customer support tickets. However, with new developments in AI services, this could suddenly be feasible overnight, making it a potentially interesting quick win for your business.

How would we evaluate our churn use case? It's hard to say, because we do not have much information yet. Stop for a moment and think: In which cluster would you place it?

Based on what we know so far, I would rate the technical feasibility as relatively high, which makes the Churn use case a good candidate for a Quick Win or a Champion. From the storyboard in the previous chapter and also from my experience with other churn use cases, I estimate the impact to be high if done correctly. Therefore, I would lean more towards Champion. Would you agree with that?

In conclusion, here are some guidelines and best practices for translating your diagram into a prioritized use case roadmap:

Mix Champions and Quick-Wins

It is good to set ambitious goals, but it is even better to achieve them from time to time. Again, AI projects are often open-ended and you never know if things will go as expected. Therefore, it's always good to have some use cases in your pipeline that have a relatively short development time and a high probability of successful implementation, even if the impact is not that big.

Identify Common Data Sources

If you have to choose between different use cases, take the ones that use the same data sources instead of mixing too many different data domains at once. For example, I would rather run 3 use cases based on CRM data rather than looking at CRM and sensor data from a production facility. With each new data source, you will most likely identify new problem areas that you would not have expected before. Once you thoroughly understand a data source, you should try to exploit it as much as possible.

Build a Compelling Vision

Do you think your company has a "killer use case" but it is still far from technically feasible? Do you see a competitive advantage for your company in an AI area, but you can not capitalize on it yet? Put it on your roadmap, but make it clear that there could still be a long way to go. Crafting a compelling story and pursuing a moonshot goal will give you a strong narrative to align other (less complex, less impactful) use cases until you finally get there. As the saying goes, Rome was not built in a day. But you could start now with the Circus Maximus.

Conclusion

In this chapter, you learned how to evaluate the feasibility of AI use cases considering data, infrastructure/architecture, and data. You should know how to perform an initial data assessment using the 4V framework and what your options are when choosing an ML infrastructure such as AI-as-a-Service or Platform-as-a-Service. We also looked at a high-level architecture you can use to describe your AI use cases. And I hope you never fall for the seven pitfalls of machine learning. It was a lot, but you have laid the perfect foundation for developing successful AI-powered prototypes on your own. Before we get into the practical application of AI use cases in various BI scenarios, the next chapter will introduce you to the prototyping toolkit for this book.

Chapter 3. Machine Learning Fundamentals

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 3rd chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rpyd.ai*.

This chapter contains everything you need to know about machine learning – at least for this book. The following sections should give you enough knowledge to follow along with the use cases in this book and help you build your own first prototypes. We’ll cover supervised machine learning, popular machine learning algorithms, key terms and you will learn how to evaluate machine learning models.

If you’re already familiar with these topics, feel free to consider this chapter as a refresher. Or, you could skip ahead to the next chapter.

Still here? Then let’s get started with Supervised Machine Learning!

The Supervised Machine Learning Process

Supervised machine learning means you train a machine learning model based on historical data where the ground truth was known. For example, if you want to predict real estate prices, in a supervised learning scenario you

need data on historical house prices and some other information that describes the houses (e.g., house size, bedrooms, location, etc.). Supervised machine learning is in contrast to unsupervised machine learning, where this ground truth isn't known and the computer has to group data points into similar categories. Clustering is a popular example of unsupervised learning.

Most enterprise machine learning problems fall into the realm of supervised learning.

I'll walk you through the basic process of supervised machine learning by showing you the key steps so you can apply them later when we build our first machine learning model starting in Chapter 7.

Step 1: Collect historical data

In order for a machine learning algorithm to learn, it must have historical data. As such, you need to collect data and provide it in a form that the computer can process efficiently.

Most algorithms expect your data to be “tidy.” **Tidy data** has three characteristics:

- Every observation is in its own row,
- every variable is in its own column and
- every measurement is a cell.

Figure 3-1 shows an example of how tidy data looks like. Bringing your data into tidy form might be the most cumbersome process of the entire ML workflow, depending from where you got the data.

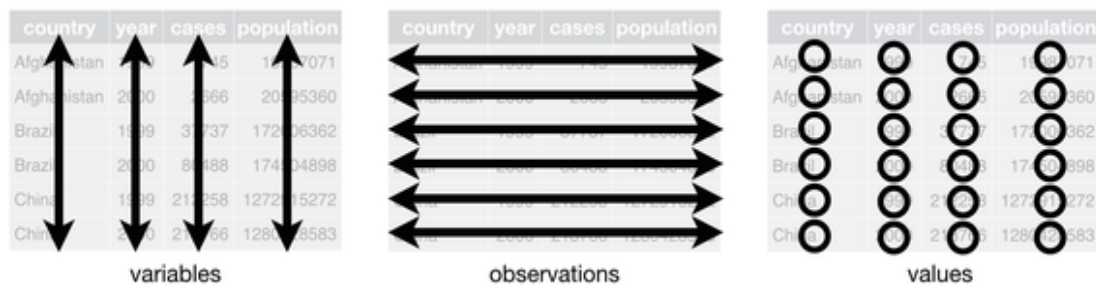


Figure 3-1. Tidy data, Source: R for Data Science (O'Reilly)

Step 2: Identify Features and Labels

Everything your algorithm is trying to do is to build a model that will predict some output y given some inputs x .

Let's unpack these concepts briefly:

Labels (also called **targets**, or **outputs**) are the variables that you want to predict based on other variables, which are called **features (attributes, inputs)** in machine learning jargon. For example, if you want to predict house prices, the house price would be your label and variables such as bedrooms, size, location would be your features. In a supervised learning setting you have historical data which contains features and labels for a given number of observations, also called **training examples**.

If your label is numeric, this process is called a **regression** problem. If your label is categorical, this process is called a **classification problem**.

Different algorithms are used for classification or regression problems. In the next section ("Popular Machine Learning Algorithms") I will introduce you to three popular algorithms that will be enough for our prototyping purposes.

Step 3: Split Your Data in a Training and Test Set

When you have your data organized in tidy form and defined your features and labels, you usually have to split your dataset into a training and a test set.

The **training** set is the part of the historical dataset which will be actually used to train the machine learning model (usually between 70% - 80% of the data).

The **test** (or **holdout**) set is a part of your data which will be used for the final evaluation of your machine learning model.

In most Automated Machine Learning (AutoML) scenarios you don't need to take care of this split manually, as it will be handled automatically by the AutoML tool. However, it's important to understand why you have to make this split and why you can only evaluate your model once on the test set.

The reason for doing this split is that you want to make sure that the model will not only perform well on the data it knows, but also on new, unseen data from the same distribution. That's why it's essential to keep some data aside for final evaluation purposes.

The performance your model has on the test set will be your performance indicator for new, unseen data.

Step 4: Find the Best Model Using Different Algorithms

In this step you will try to find the model which represents your data best and which has the highest predictive power. You do that by trying out different machine learning algorithms with different parameters.

Again, let's quickly go through these concepts:

The **model** is the final deliverable of your machine learning training process. It is a (arbitrarily) complex function that calculates an output value for any given set of input values.

Let's consider a very simple example. Your model could be the following:

$$y = 200x + 1000$$

This would be the equation of a simple linear regression model. Given any input value for x (for example house size), this formula would calculate the final price by multiplying the size by 200 and then adding 1,000 to it. Of

course, machine learning models are usually much more complex than that, but the idea stays the same.

So, how can a computer, given some historical input and output data come up with such a formula? Two components are needed for this: First, we need to provide the computer with the **algorithm** such as in this case linear regression. The computer will know that the output must look something like the regression equation:

$$y = b_1x + b_0$$

The computer will now use the historic training data to find out the best parameters b_1 and b_0 for this problem. This process of parameter estimation is what is called “**learning**” or “**training**” in machine learning. There are different ways to achieve and accelerate this learning process for big datasets, but this would be out of scope for this book and also not necessary for our purposes. The only thing you need to know is that you need to define a machine learning algorithm in order to train a model on training data. This is the place where experts like Data Scientists or Machine Learning experts can ace.

In our case, we will rely mostly on AutoML solutions to find the best model for us.

Step 5: Evaluate the Final Model

Once our training process is complete there will be one model which has shown the best performance on the test set. Different evaluation criteria are used for different machine learning problems, which we will explore in further detail in the section “Machine Learning Model Evaluation”. It is important to understand how these metrics work as you will need to specify these metrics to the AutoML service to find the best model.

Again, don’t worry - mostly the service will give a suggestion for a good first-shot evaluation metric.

Step 6: Deployment

Once you decide on a model, you usually want to deploy it somewhere so users or applications can use it. The technical term for this part of the machine learning process is **inference**, **prediction** or **scoring**. At this stage, your model doesn't learn anymore but it's only calculating the outputs as it receives new input values. The way this works depends on your setup. Very often the model is hosted as an HTTP API which takes some input data and returns the predictions (**online prediction**). Alternatively, the model can be used to score a lot of data at once, also called **batch prediction**.

Step 7: Maintenance

While the development process of the ML model has finished after deployment, there is actually never an end to it.

As data patterns change, models need to be retrained and one needs to look if the initial performance of the machine learning model can be kept high over time. We don't have to deal with these things during the prototyping phase, but it's important to note that machine learning models need a considerable amount of maintenance after deployment which you should consider in your feasibility analysis. We will explore this topic further in Chapter 11.

Now that you understand on a high level how the process for supervised machine learning looks in general, let's look at the most popular machine learning algorithms used to train a model for both classification and regression problems.

Popular Machine Learning Algorithms

In this section, I introduce you to three popular families of machine learning algorithms, namely Linear Regression, Decision Trees and Ensemble methods.

If you know these three classes, you'll be able to tackle probably 90% of all supervised machine learning problems in business.

Table 3-1 gives an overview of these classes of algorithms, but we'll look at them in a little more detail.

*T
a
b
l
e*

*3
-*
1

*.
C
o
m
p
a
r
i
s
o
n*

*o
f
p
o
p
u
l
a
r*

*m
a*

*c
h
i
n
e

l
e
a
r
n
i
n
g

a
l
g
o
r
i
t
h
m
s*

Algorithm Class	Used for	Performance	Interpretability
Linear Regression	Regression problems	Low - High	High

Decision Trees	Regression and classification problems	Average	High
----------------	--	---------	------

Ensemble Methods (Bagging & Boosting)	High	Low
---------------------------------------	------	-----

High	Low
------	-----

Linear Regression

Linear regression is one of the most essential algorithms to understand in Machine Learning because it powers so many different concepts. Logistic regression, regression trees, time series analysis and even neural networks use linear regression at their heart.

The way linear regression works is simple to explain, but hard to master: Given some numeric input data, the regression algorithm will fit a linear function that calculates (predicts) corresponding numeric output data.

Two main assumptions are important for regression: 1) The different features should be independent from another (low correlation between features) and 2) the relationship between the features towards the outcome variable should be linear (e.g. when one variable increases, the output variable should also increase or decrease with a fixed pattern).

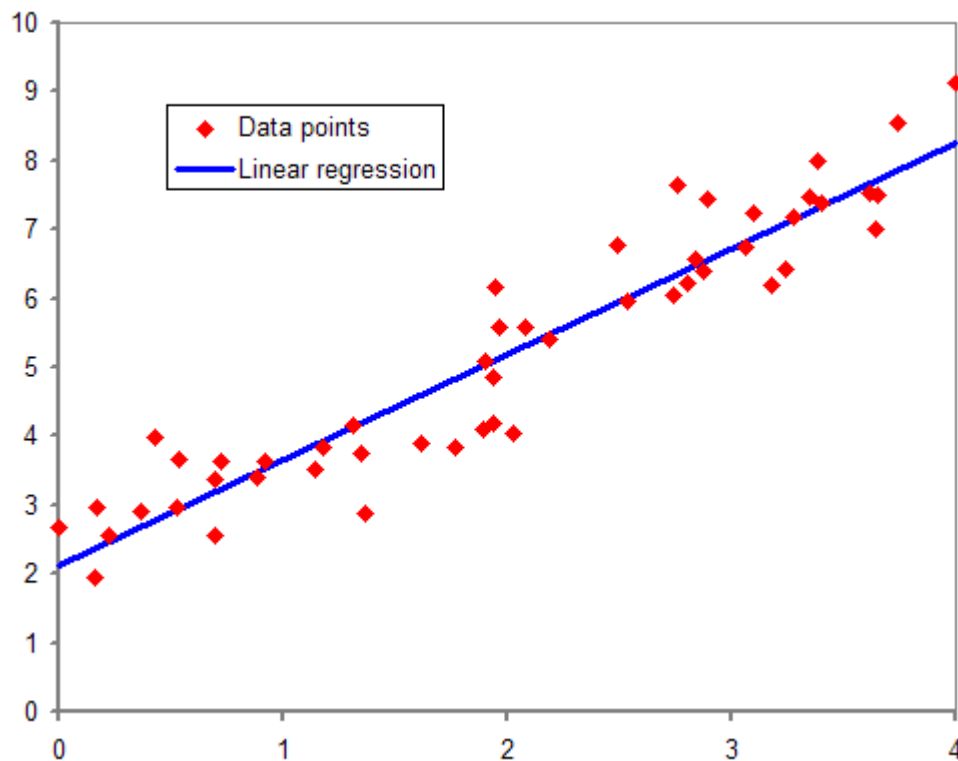


Figure 3-2. Simple Linear Regression

Figure 3-2 shows the example of a simple linear regression where the input variable on the x-axis would predict the variable on the y-axis. The dots in the plot are the actual data points (labels) which were observed for each value for x in the historical data set. The prediction from the regression would be the y-value on the straight line for each value of x. Of course, this is just a very basic example for illustration purposes. Linear regression works also with many more than just one input variable and it can also model different shapes than just straight lines (Polynomial Regression).

As you can see in the first line of Table 3-1 the model performance of Linear Regression can range anywhere from very low to very high. That's because the performance of the linear regression algorithm depends heavily on the data and how well the data meets the assumptions of linear regression. If the input variables are really independent and have a linear relationship to the output variables, regression can beat any neural network.

That said, regression is not a good choice if your data contains non-linear patterns. For example if a certain value is met, then suddenly the relationship to the target variable changes. These sort of if-then-else rules can better be modeled with other algorithms, such as decision trees.

Decision Trees

In contrast to a regression model, tree-based models can also work with both categorical and numerical data (regression trees) as you can see in Table 3-1. The way decision trees work is that they split the data at different levels, variable after variable until the data slices become so small that you can make a prediction. Imagine decision trees as a hierarchical order of if-then rules.

Decision trees are usually good allround-algorithms which can be used on almost any tabular dataset as a first baseline model. Decision trees can be easily shared and explained even to non-technical stakeholders. On the downside, single decision trees often don't deliver the best results in all cases because the algorithm is "greedy". That means it will do those splits first where the data shows the highest discrepancy. This process isn't ideal in all situations.

Figure 3.3 shows an example decision tree which predicts the survival of passengers on the Titanic, a popular research dataset. You read the tree from the top to the bottom where each node represents a step where a decision is made.

Survival of passengers on the Titanic

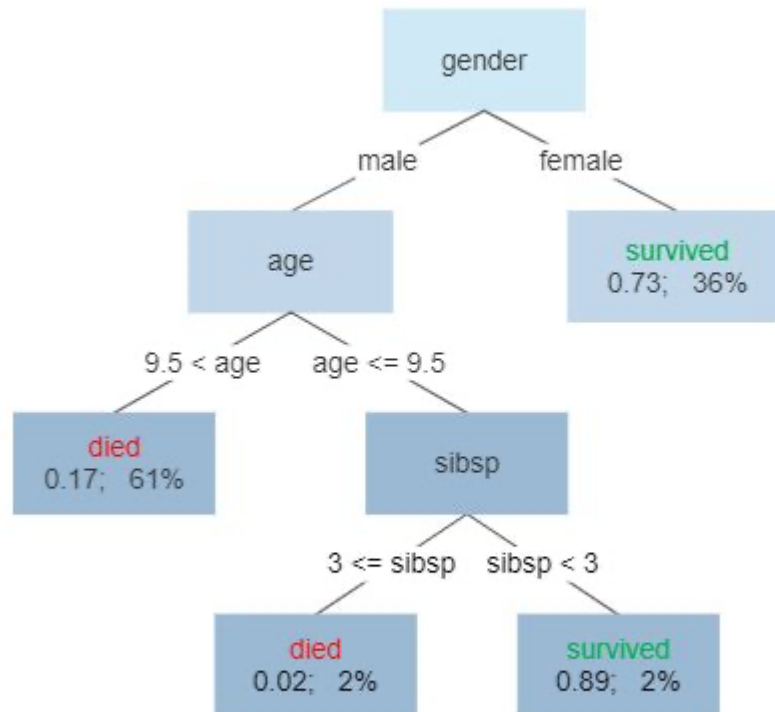


Figure 3-3. Decision Tree

Because the decision tree is “greedy” it will make the first split on the gender. If you pick a random female passenger from the Titanic dataset, the chances that this person survived are 73%. Not bad for a first guess. For men, the tree looks a little more complicated. The prediction will be “died” or “survived” considering multiple factors such as the person’s age and the number of siblings or spouses (“sibsp”).

We don’t go much more into detail here. It’s only important that you understand how decision trees work conceptually, because we will build on them in the next concept: Ensemble learning.

Ensemble Methods

When Linear Regression and Decision Trees are pretty straightforward and rather easy to explain in the way they work, Ensemble methods sit pretty much on the other end of the spectrum.

As you can see from Table 3-1, Ensemble methods provide generally high predictive power, but at the same time are not so easy to interpret in the way they make a decision.

Both linear regression and decision trees are typically considered to be ‘weak’ learners, which means that they typically struggle if data relationships become more complex. For example, regression can’t work with non-linear data and decision trees can make a split only once for each variable in your data set.

To combat these issues, ensemble methods allow you to combine several weak learners into a strong learner that shows a strong performance also on complex datasets.

The two typical methods of ensemble learning are bagging and boosting which differentiate by the way they combine weak learners.

Bagging (from bootstrap aggregating) tries to combine multiple weak learners by training different models individually and then combining them using an averaging process.

If you look at Figure 3-4, you can see a schema of how Bagging works. In the middle layer you can see four different models which are all trained on the same training dataset. This could be four decision trees, for example, with different parameters for the tree depths or the minimum node sizes. For each datapoint, each of these four models will make an individual prediction (top layer). The final prediction will be an aggregation of these four results, for example a majority vote in the case of a classification example or an average value for a regression problem.

A popular bagging ensemble method is called Random Forest where the decision trees are split into multiple sub-trees to have less correlation between them.

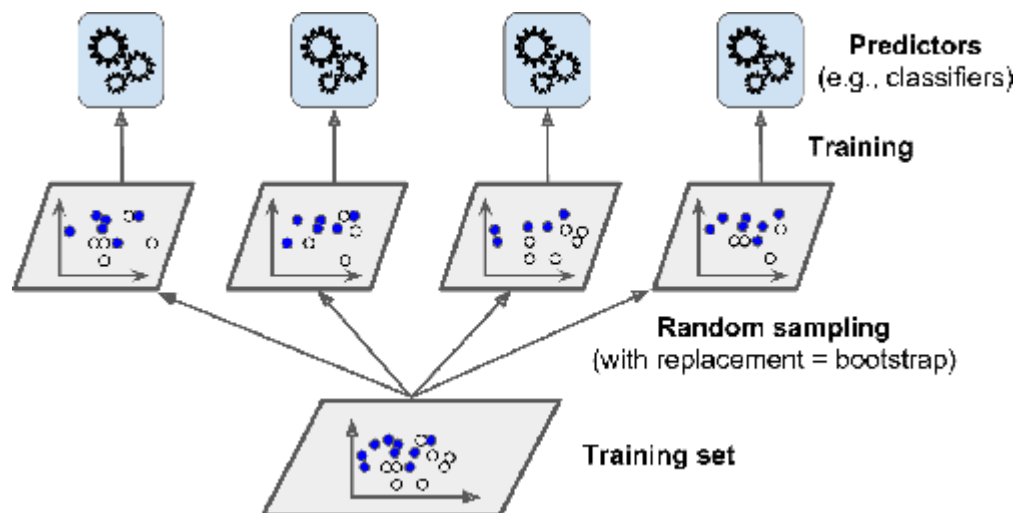


Figure 3-4. Bagging

Boosting, on the other hand, relies on the idea of sequentially adding models to an ensemble, each one correcting the errors of its predecessor. It's a very popular and powerful technique. Two popular boosting methods are AdaBoost (Adaptive Boosting) and Gradient Boosting.

Figure 3-5 shows a conceptual overview of how Bagging works:

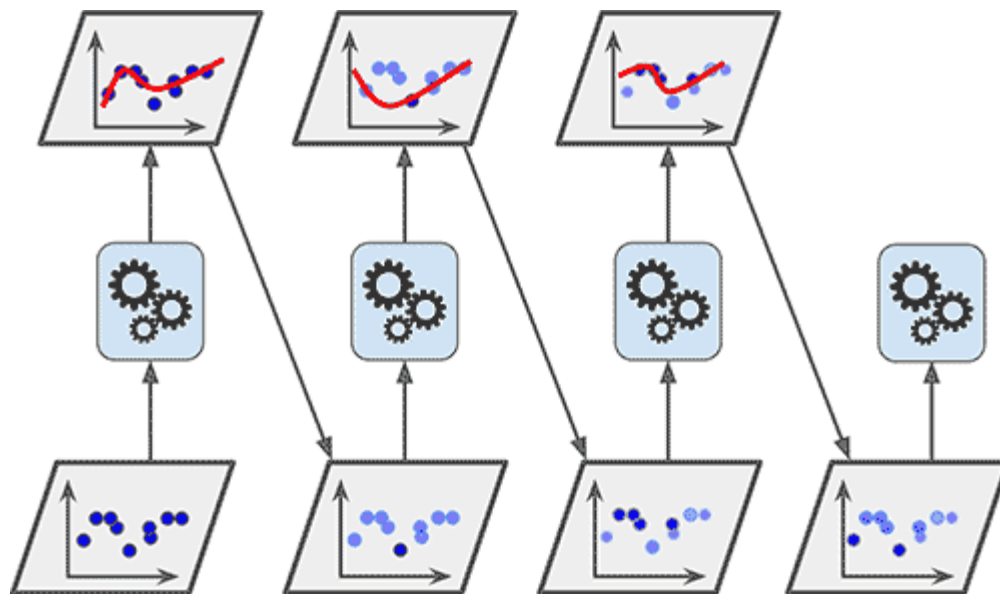


Figure 3-5. Boosting

The training process starts on the bottom left where the first model (for example a decision tree) is trained and gives a first prediction with relatively high errors (top left). Next, another decision tree is learned which

tries to correct exactly the errors that the preceding model made (second model in the bottom row). This process will be repeated several times until the errors become smaller and smaller and a well-performing model has been found. In Figure 3-5 there are three iterations which result in the final model on the bottom right.

As you might see from this explanation, ensemble methods are much harder to interpret than linear regression or decision trees. However, in most cases they provide better performance. Which approach you should take, depends on your goals.

If explainability is not so important as accuracy for your projects, choosing a strong learner such as Adaptive Boosting is typically the better choice.

On the other hand, if you want that your machine learning model must be well understood and interpreted by users or other stakeholders, choosing a slightly less-accurate model such as a decision tree or regression might be the better choice.

When in doubt, choose the simpler model if it provides similar performance.

Deep Learning

The algorithms we have covered so far work very well on structured, tabular data as you will find it in spreadsheets or data warehouses.

For other data types, such as image data or text data, you will need different algorithms. This is a field which is often also referred to as “Deep Learning”.

We won’t go into much detail here. The AI services we will later use from Azure Cognitive services are based on Deep Learning techniques, but we are not going to build them out ourselves. For our purposes, it’s enough to have a general idea of what these deep learning concepts mean and how they work.

Deep learning refers to a variety of approaches and techniques in machine learning that are specialized to work with high-dimensional dataset, that is data with many features. There isn't a formal definition where "shallow" machine learning ends and where "deep learning" starts but it mostly comes down to the data types.

Consider the following example: Even if you build an ultra-high dimensional sales predictor, you will probably not create more than a few dozen, maybe even hundreds of features from your tabular CRM data.

Now imagine you're analyzing images for object detection. Even for rather small images with a resolution of 300 x 300 pixels you would get 90.000 dimensions (one dimension for each pixel). With colored RGB pictures this amount triples.

You can see that there is still a considerable difference between a high-dimensional table of maybe 1,000 columns (dimensions) and a small image representation of 90,000 dimensions - for every picture. And that's why deep learning is often considered for non-tabular data such as images, videos and unstructured text files.

Two broad categories have emerged in deep learning: Computer Vision (dealing with picture or video data) and Natural Language Processing (dealing with text data).

Natural Language Processing

You have probably already heard about AI services that are able to interpret human language, do translations more accurately than ever before or even generate new texts completely automatically. Large language models which have been trained on billions of human text examples combined with a breakthrough technology called Transformers has catapulted the NLP field from a small research domain to one of the hottest areas in AI development.

We won't train NLP models ourselves in this book, but you will be exposed to using state-of-the-art language models to analyze text data at scale.

Computer Vision

Computer vision is a technology that enables machines to “see” and interpret image and video files. A technology called Convolutional Neural Networks (CNNs) has led to incredible breakthroughs in this field in recent years. Applications range from identifying commodity entities in images such as cars, bicycles, street signs, animals and people, to face recognition and extracting text structures from images. It’s a diverse field with lots of things happening.

In Chapter 9, we will use a Computer Vision AI service to detect and count cars in image files.

Reinforcement Learning

The last deep learning category I want to highlight here is a field called Reinforcement learning. We will see a Reinforcement Learning service in action in Chapter 8: AI-Powered Prescriptive Analytics.

Reinforcement Learning works differently than supervised or unsupervised learning and is somehow a separate field on it’s own. The reason lies in the way reinforcement learning models are learned.

Instead of relying on historical data, Reinforcement learning approaches require a constant stream of new data coming in where the model can learn the best strategies given a current state of a system and policies the model is allowed to act in.

Reinforcement learning has seen major breakthroughs by being able to beat world-class human players in games like *Go* and *Starcraft*.

But as you will find out later, it is also a great way to personalize user experiences and learn over time what users want and how to interact with them.

Machine Learning Model Evaluation

With the knowledge so far you should be confident enough for this book to build and apply machine learning models in a prototyping stage to real-world use cases.

One component is still missing, though. And that is how to assess how good your model is actually working.

Being able to measure the performance of a machine learning model is critical not only during prototyping but even later in production. The evaluation metric we choose should not differ between both, only the value of the metric. So how do we evaluate the performance of a machine learning model? Welcome to the world of evaluation metrics!

Evaluation metrics are calculated for two reasons:

1. Evaluation metrics are used to compare different models among each other to find the best predictive model.
2. Evaluation metrics are used continuously to measure the model's true performance on new, unseen data.

The basic idea behind model evaluation is always the same: We will compare the values that our model predicts with the values that the model should have predicted according to our ground truth, e.g., the labels in our dataset.

What sounds intuitively simple, carries indeed a good amount of complexity. In fact, you could write a book on evaluation metrics alone. In our case, we'll keep it short and just focus on the most important concepts you will most likely encounter when working with machine learning algorithms.

There are different evaluation metrics for classification and regression problems.

Evaluating Regression Models

Remember that regression models predict continuous numeric variables such as revenues, quantities, sizes, etc. The most popular metric to evaluate

regression models is the *Root Mean Squared Error (RMSE)*. It is the square root of the average squared error in the regression's predicted values ($\hat{y}$). The RMSE can be defined as so:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}$$

RMSE measures the model's overall accuracy and can be used to compare different regression models amongst each other. The smaller the value is, the better the model seems to perform. The RMSE comes in the original scale as the predicted values. For example, if your model predicts a house price in US dollars and you see an RMSE of 256.6, this translates to “the model predictions are wrong by 256.6 dollars on average”.

Another popular metric that you will see in the context of regression evaluation is the *coefficient of determination*, also called the *R-squared* statistic. R-squared ranges from 0 to 1 and measures the proportion of variation in the data accounted for in the model. It is useful when you want to assess how well the model fits the data. The formula for R-squared is:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

You can interpret this as one minus the explained variance of the model divided by the total variance of the target variable. Generally, the higher R-squared, the better. An R-squared value of 1 would mean that the model could explain all variance in the data.

While summary statistics are a great way to compare different models among each other at scale, it's still hard for us to figure out if our regression is working as expected from these metrics alone.

Therefore, a good approach in regression modeling is also to look at the distribution of the regression errors, also called the residuals of our model. The distribution of the residuals gives us good visual feedback on how the regression model is performing. A residual diagram plots the predicted values against the residuals as a simple scatter plot. Ideally, a residual plot should look like in Figure XX to back up the following assumptions:

Linearity

The points in the residual plot should be randomly scattered without exposing too much “curvature”.

Heteroscedasticity

The “spread” of the points across predicted values should be more or less the same for all predicted values.

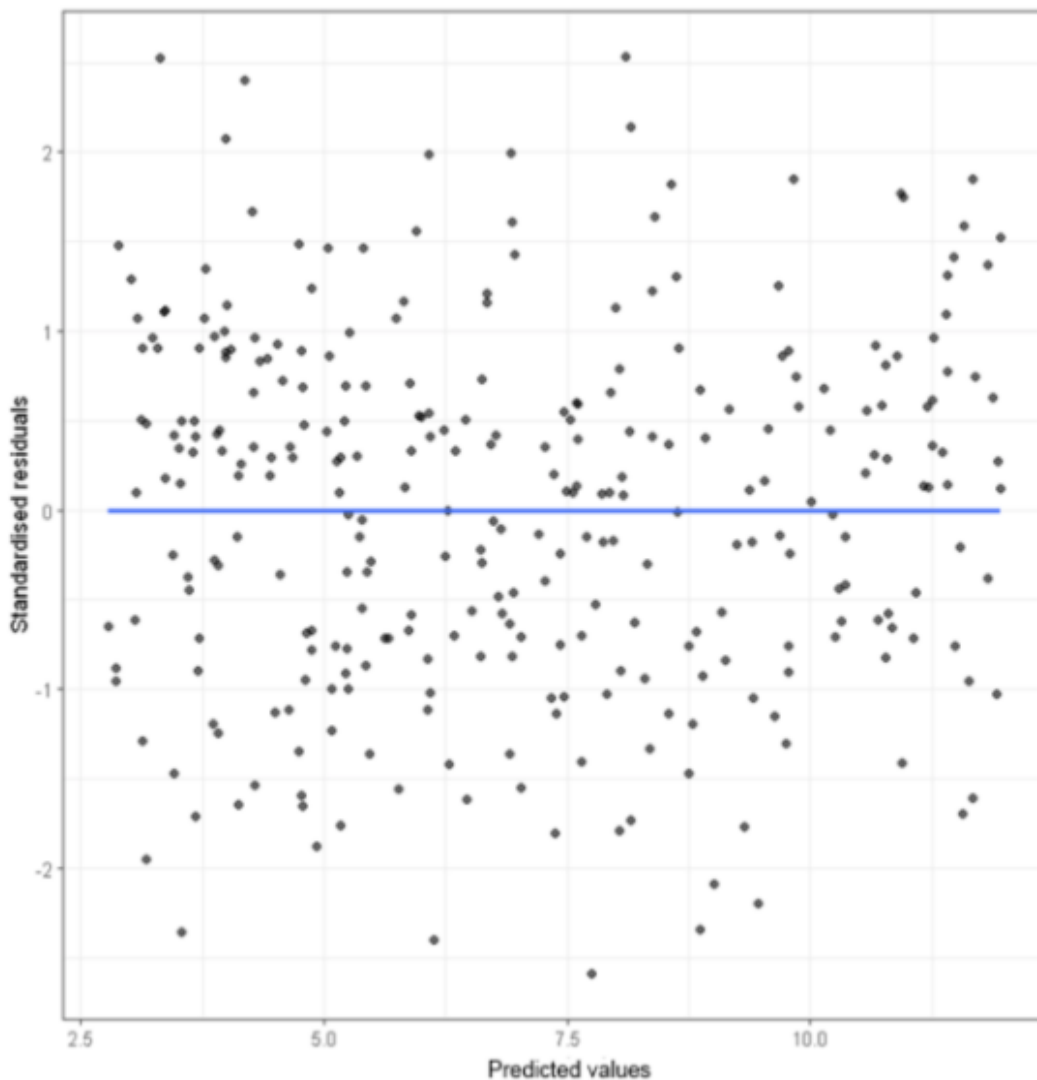


Figure 3-6. Residual analysis

Residuals plots will show you on which parts of your data the regression model works well and on which parts it doesn't. Depending on your use case you can decide if this is problematic or still acceptable given all other factors. Looking at the summary statistics alone would not give you these insights. Keeping an eye on the residuals is always a good idea when evaluating regression models.

Evaluating Classification Models

How do we evaluate the performance of a predictive model if the predicted values are not numeric but categorical? We can't just apply the metrics we learned from the regression model. To understand why, let's take a look at some data in Table 3-2.

*T
a
b
l
e*

*3
-
2
.
E
x
a
m
p
l
e*

*c
l
a
s
s
i
f
i
c
a
t
i
o
n*

o
u
t
p
u
t
s

Targets	Predicted	Probabilistic output
1	1	0.954
1	0	0.456
0	0	0.012
0	1	0.567
0	0	0.234
...	...	...

The table shows the true labels of a classification problem (column “Targets”) as well as the corresponding outputs (“Predicted” and “Probabilistic output”) of a classification model.

If you look at the first four rows, there are four things that could happen here:

- The true label is `1`, and our prediction is `1` (correct prediction)
- The true label is `0`, and our prediction is `0` (correct prediction)
- The true label is `1`, and our prediction is not `1` (incorrect prediction)
- The true label is `0`, and our prediction is not `0` (incorrect prediction)

These four outcomes would also happen if we had more than two categories. We could simply observe these four outcomes for each category in our dataset.

The most popular way to measure the performance of a classification model is to display these four outcomes in a so-called confusion table or confusion matrix as seen in Figure XX.

T
a
b
l
e

3
-
3
.
C
o
n
f
u
s
i
o
n

M
a
t
r
i
x

Predicted Class

Actual Class

Negative

Positive

Negative	2 388 True Negative (TN)	558 False Positive (FP)
----------	-----------------------------	-------------------------------

Positive	415 False Negative (FN)	2 954 True Positive (TP)
----------	-------------------------------	--------------------------------

This confusion table tells us that for our example classification problem, there were `2388` observations where the actual label was `0` and our model labeled them correctly. These observations are called True Negatives, because the model correctly predicted the Negative class label. Likewise, `2954` observations were identified by our model as “True Positives”, where both the actual and the predicted outcome had the label Positive class label (in this case the number `1`). But our model also made two kinds of errors. The first error type is that it wrongly predicted Negative classes to be Positive (558 cases), also known as “Type I error”. The second error type is that it incorrectly predicted 415 cases to be negative which were in fact positive. These ones are also known as “Type II Errors”. Statisticians were rather less creative in naming these things.

Now, there are different metrics we can calculate from this confusion table.

The most popular metric which you will see and hear often is accuracy.

Accuracy describes how often our model was correct, based on all predictions.

We can easily calculate the model accuracy by dividing the number of correct predictions through the number of all predictions:

$$\text{ACC} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

For the confusion matrix above, the accuracy is $(2388 + 2954) / (2954 + 2388 + 558 + 415) = 0.8459$, meaning that the predictions of our model were correct in 84.59% of all cases.

While accuracy is a widely used and very popular metric, it has one big caveat: Accuracy works only for balanced classification problems.

Imagine the following example:

We want to predict credit card fraud, which happens only in 0.1% of all credit card transactions. If our model consistently predicted 'No Fraud', it would yield an astonishing accuracy score of '99.9%' because it would be correct in most cases.

While this model has an excellent accuracy metric, it's clear that this model isn't useful at all.

That's why the confusion matrix allows us to calculate even more metrics that are more balanced towards correctly identifying positive or negative class labels.

The first metric we could look at is called *Precision*. Precision measures the proportion of true positives (predicted true labels that were actually true). Precision, or also called the positive predictive value (PPV) is defined as:

$$\text{PPV} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

In our example, the precision is $2954 / (2954 + 558) = '0.8411'$. This value can be interpreted as that when the model labeled a data point as positive ('1'), it was correct in 84% of all cases. We will pick this metric if the costs of a false positive are very high and the costs of a false negative are rather low, for example, when we want to decide if we are going to show an expensive ad to an online user.

Another metric that looks at the positive classes but has a slightly different focus is *Recall*, sometimes also called “Sensitivity” or true positive rate (TPR). It gives the percentage of all positive classes that were correctly classified as positive.

Recall is defined as:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

In our example, this would be $2954 / (2954 + 415) = 0.8768$. The value means that the model could identify 87.68% of all positive classes in the dataset. We would pick this metric to optimize our model for finding the positive classes in our dataset (e.g., fraud detection). Using recall as an evaluation metric makes sense when the costs of missing these positive classes (i.e., False Negatives) are very high.

If you don't want to lean either towards Precision or recall and accuracy seems to be not capturing your problem well enough, there is another commonly used metric which is called the 'F1-Score'. Albeit its technical name, this metric is relatively intuitive. The F1-score is simply the harmonic mean of precision and recall.

We can thus easily calculate the F1-Score if we already know the precision and the recall:

$$F_1 = 2 \times \frac{PPV \times TPR}{PPV + TPR} = \frac{2TP}{2TP + FP + FN}$$

In our case, the F1-Score would thus be $2 * (0.8411 * 0.8768) / (0.8411 + 0.8768) = 0.8586$.

A higher F1-Score indicates a better performing model, but what does it tell us exactly? Well, the F1-Score isn't as easily explained in one sentence as the metrics above, but it is usually a very good go-to metric to assess the quality of a model. Since the F1-Score uses the harmonic of precision and recall instead of the arithmetic mean it will tend to be lower if one of these would be very bad. Imagine an extreme example where precision was 0 and recall was 1. The arithmetic mean would return a score of 0.5 which roughly translates to "If one is great, but the other one is poor, then the average is okay.". The harmonic mean, however, would be 0 in this case. Instead of averaging high and low numbers out, the F1-Score will raise a flag if either recall or precision are extremely low. And that is much closer to what you would expect from a combined performance metric.

Evaluating Multi-Classification Models

The metrics we used for binary classification problems also apply to multi-class classification problems (more than 2 classes to predict). For example, consider the following target and predicted values:

```
targets = [1, 3, 2, 0, 2, 2]
predicted = [1, 2, 2, 0, 2, 0]
```

The only difference is that you would calculate the evaluation metrics from above for each class (0, 1, 2, 3) and compare it against all other classes. The way you do that can follow different approaches, usually referred to as the "micro" and "macro" value.

“Micro” scores typically calculate metrics globally by counting all the total true positives, false negatives and false positives together and only then dividing them by each other.

‘Macro’ scores on the other hand calculate Precision, Recall, etc. for each label first and then calculate the average of all of them. These approaches produce similar, but still different results.

For example, our dummy series for the 4 classification labels above has a ‘micro’ F1-Score of 0.667 and a macro F1-Score of 0.583. If you like, go try it out and calculate the numbers by hand to foster your understanding of these metrics.

Neither of the micro or macro metrics is better or worse. These are just different ways to calculate and the same summary statistic based on different approaches. These nuances will become more critical once you work on more complex multiclass problems.

There are even more performance metrics to look at, but these would be out of scope for this book. For example, so far we haven’t even looked at performance metrics that consider the probabilistic outputs of the models, such as the AUC (area under ROC-Curve).

There’s no silver bullet when it comes to evaluation metrics. It is important that you at least understand on a high level what these metrics are trying to capture and identify which metric is practical for your use case. And: Which performance is good enough to stop training more and more complex and (seemingly) accurate machine learning models? We will dive deeper into that in the next section.

Common Pitfalls of Machine Learning

Machine learning is a powerful tool for solving many modern problems, but like any other tool, it is easy to misuse and can lead to poor results. Here are some beginner’s mistakes I’d like you to avoid.

Pitfall 1: Using Machine Learning When You Don't Need It

If you have a hammer, everything looks like a nail. The worst thing you could do is take your newly acquired knowledge of machine learning and look for business problems that seem like a good fit. Instead, look at it from the opposite angle: Once you have identified a relevant business problem, consider whether machine learning could help solve it. And even then, it's important to start with a simple baseline first. This baseline is important to get a performance benchmark that the machine learning algorithm needs to outperform. If you have never done churn analysis before, you should not start with a ML-based approach, but rather use simple heuristics or hard-coded rules. These have very low complexity and are easy for business stakeholders to understand. Switch to ML models as soon as you find that they perform significantly better than your baseline solution.

Pitfall 2: Being Too Greedy

Don't try being too greedy and maximizing the performance metrics of your training set. This can cause your model to perform poorly on new data that it has never seen before. It may sound counterintuitive, but you should be more conservative in your training, especially on smaller data sets, as this can lead to better generalization of your model to unseen data. This concept is also known as "Occam's Razor" and means that when in doubt, a simple solution is better than a complex one.

Pitfall 3: Building Overly-Complex Models

Building too complex models goes hand in hand with the previous point: to achieve high accuracy, inexperienced people tend to create overly complex models such as neural networks. In many cases, a less accurate model with lower complexity is preferred to a highly complex, highly accurate model.

Complex models have two disadvantages:

- First, they can be difficult to maintain and debug in production. If something goes wrong or your model predictions are off (which they certainly will be from time to time), highly complex models are harder to correct than simpler models.
- Second, complex models can lead to a phenomenon called overfitting. *Overfitting* is when your model performs very well on your training data but poorly on new data. Predictive analysis is about finding a sweet spot (see Figure 3-7) where the prediction errors in training (historical data) and testing (new data) are approximately equal. To avoid overfitting, you should always monitor the performance of your model on new data as well as the complexity of your model.

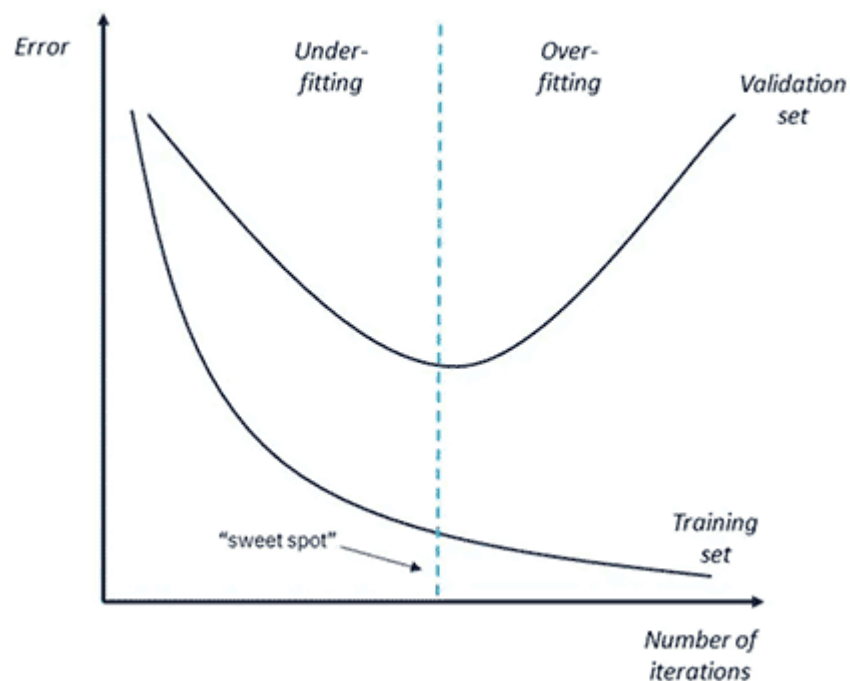


Figure 3-7. Training error vs. Testing error

Pitfall 4: Not Stopping When You Have Enough Data

Rarely will you have perfectly labeled machine learning data lying around in your organization. As we have already learned, usually the more examples (observations) we have, the better. However, experience shows

that machine learning algorithms reach a plateau at a certain point where additional training examples no longer significantly increase accuracy. This effect is illustrated in Figure 3-8. When you reach this point, it is simply no longer cost-effective to spend more money on data labeling. For this reason, it is important to have a clear goal in mind. How accurate does your model need to be for you to use it? The threshold can range from “anything better than baseline” to “99.99 accuracy required,” such as in medical cases. Knowing what your goal is and what you want to achieve can help you avoid high costs.

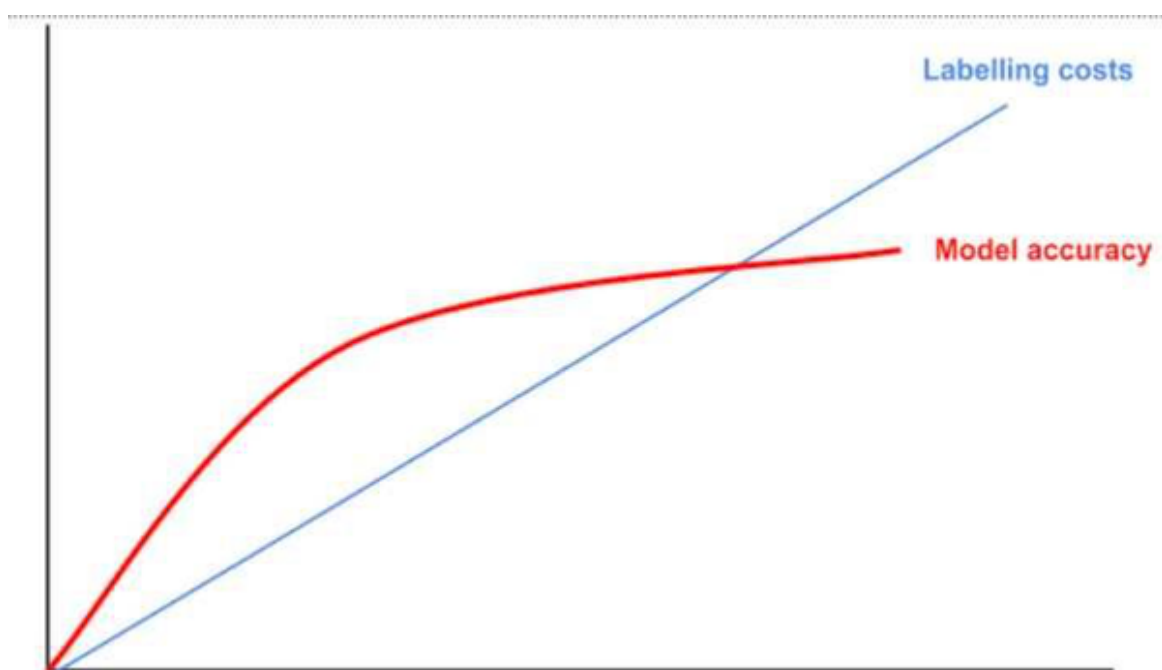


Figure 3-8. Model accuracy vs. labeling costs

Pitfall 5: Falling For The Curse Of Dimensionality

While the “the more data, the better” principle works for observations (rows), it can actually be counterproductive for features (columns). Imagine the following example.

You want to predict US home prices based on the variables size, bedrooms, and location (zip code). Size and bedrooms can be modeled as two individual features (columns), which only increases the dimensionality of your dataset by the order of two as seen in the example in Figure 3-9.

 Data points across multiple dimensions. Source <https://aiaspirant.com/curse-of-dimensionality>

Figure 3-9. Data points across multiple dimensions. Source: <https://aiaspirant.com/curse-of-dimensionality/>

However, once you start using zip codes, most machine learning algorithms require that you encode this categorical variable into a single-column feature that is exactly as wide as the number of unique values. This increases the dimensionality of the data not just by order 1, but by order 41,692 (possible US zip codes), since it would be single columns containing either the value “0” or “1” for each example.

Most machine learning algorithms will struggle to find actual patterns in the data because the data is too sparse. There are no absolute rules to avoid the curse of dimensionality, but here are some tips:

- Be careful when adding new features to your dataset and do not hesitate to delete redundant or irrelevant features. This can sometimes be difficult, as you may need solid domain expertise for this step.
- Try to reduce the number of relations between attributes by coding these dependencies as a single attribute. For example, instead of having two attributes “price” and “size”, you could calculate a new attribute “price per square foot”. The fewer dependencies you have between variables in your dataset, the easier it will be for your ML algorithm to understand them.

Pitfall 6: Ignoring Outliers

Outliers are data points that are far above the average value of your data set. For example, imagine a data set that contains people’s salaries and net worth. If you plot the data, it might look like Figure 3-10.

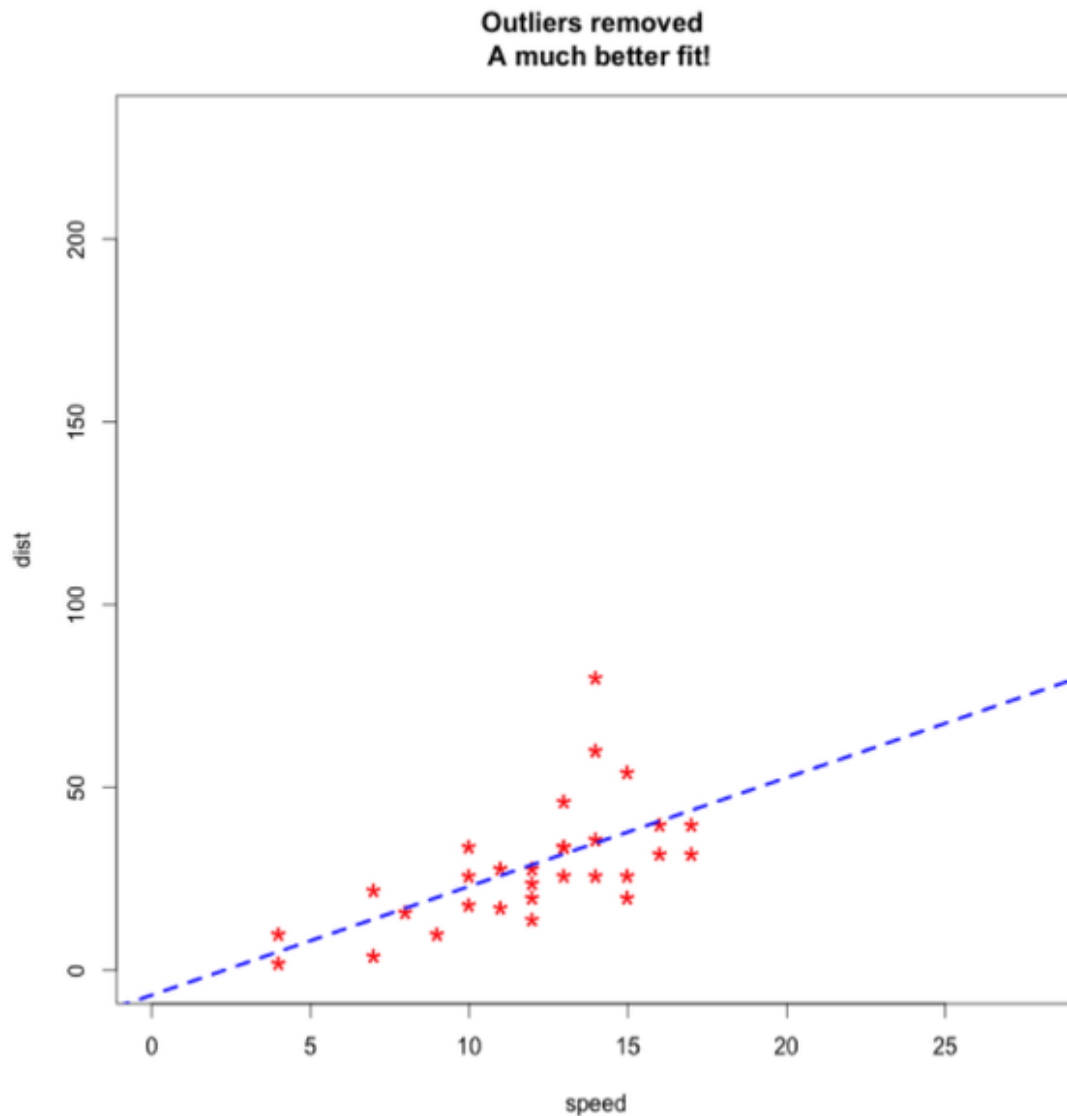


Figure 3-10. Sample data without outliers

You can see that there is a strong correlation between the two variables, as most of the points are not far from the diagonal.

However, if you plot an extra point on top of your data, you will see the true significance of this value. Figure 3-11 shows the same data set, but with an additional value (the red dot) that is far above the others. It represents someone who closed a \$10 million deal in their first year of work:

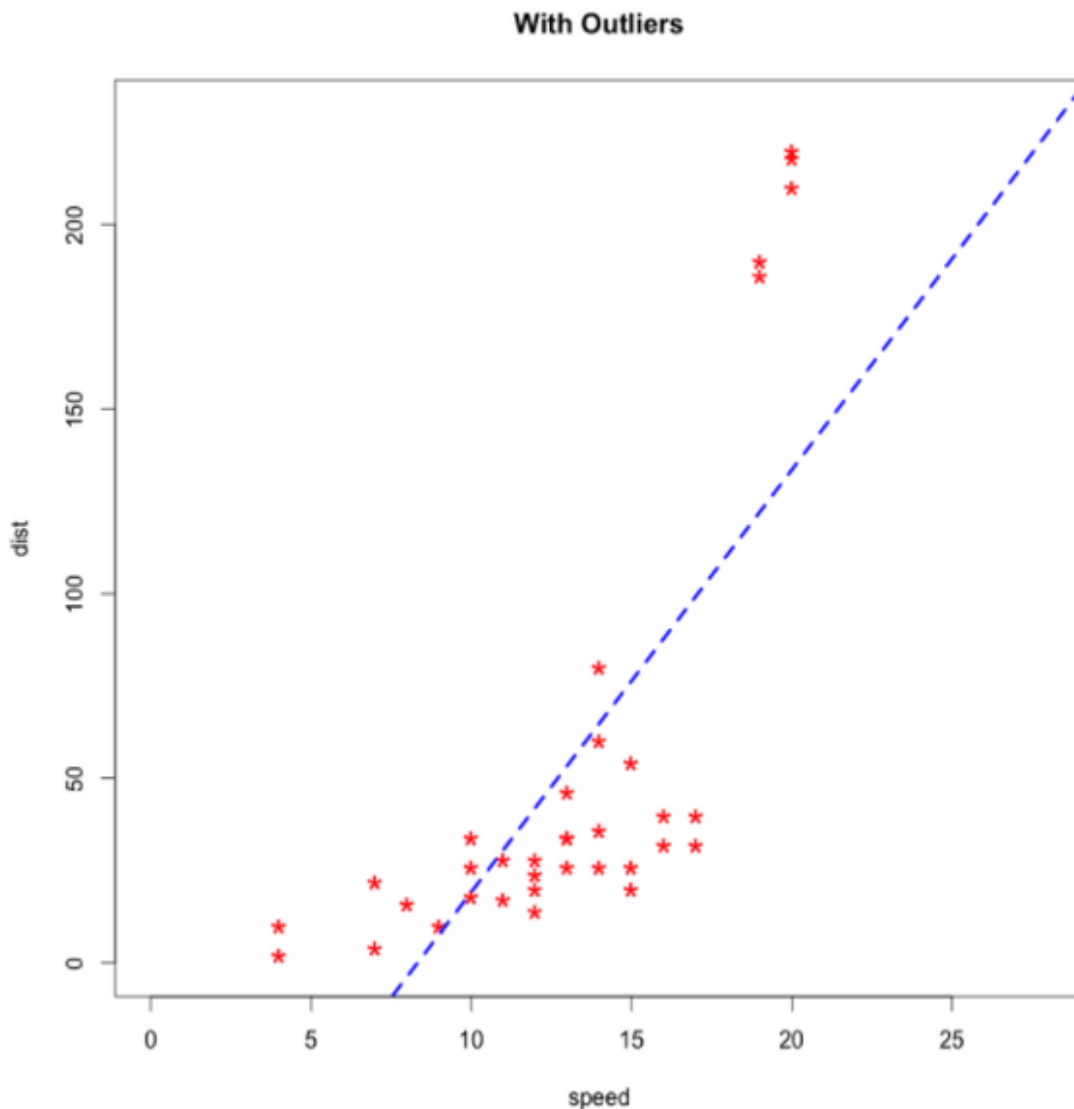


Figure 3-11. Sample data with outliers

The impact of outliers can be huge for many algorithms, especially those that work with regression tasks. This means that you should pay close attention to detecting outliers in your data set.

Pitfall 7: Taking Cloud Infrastructure for Granted

In this book, I make the bold assumption that you have access to cloud computing for both model training and model inference. But let us face it, in many companies that may not be the case (yet). Although cloud computing adoption is growing rapidly, most companies are still largely

using on-premise solutions. The reasons for this are many, but one important reason is the fear of losing control of data.

I strongly recommend that you try cloud computing (AI-as-a-Service or ML platforms) at least for prototyping with non-critical data, as described in the next chapter. While this may not yet be a lived practice in your organization, it will give you a quick head start and put you in touch with cutting-edge ML workflows and tools. This will also enable you to discuss with your team how cloud computing could be an investment that benefits your overall AI/ML strategy.

Conclusion

In this chapter, you learned the basic concepts of supervised machine learning and got an idea of which algorithms are important for machine learning. We also touched on terms like Deep Learning, Computer Vision, or Natural Language Processing.

Of course, there's much more to learn about machine learning, but these pages will give you everything you need to build your own ML-powered prototypes with the help from the tools of this book. I hope you feel a little more confident to do this by now. If not, don't worry!

Later, as we approach practical use cases, the concepts from this chapter will probably seem much more accessible to you as they're connected to some practice.

If you want to dig deeper into machine learning, I do recommend any of the the following books:

- Machine Learning Design Patterns By Valliappa Lakshmanan, Sara Robinson and Michael Munn (O'Reilly)
- Introduction to Machine Learning with Python By Andreas C. Müller and Sarah Guido (O'Reilly)
- Building Machine Learning Powered Applications By Emmanuel Ameisen

I also recommend bookmarking this chapter and come back here anytime you want to review a section.

Chapter 4. Prototyping

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

By now, you should have enough theoretical background knowledge to get hands-on with some AI use cases finally. In this chapter, you’ll learn how to use a prototyping technique borrowed from product management to quickly create and put machine-learning use cases into practice and validate your assumptions about feasibility and impact.

This chapter will introduce you to the theoretical concepts about prototyping but also the concrete tools we are going to use for the examples throughout this book.

What Is a Prototype and Why Is It Important?

Let’s face the hard truth: most machine learning projects fail. And that’s not because most projects are underfunded or lack talent (although those are common problems, too).

The main reason machine learning projects fail is the incredible uncertainty surrounding them: Requirements, solution scope, user acceptance, infrastructure, legal considerations, and most importantly, the quality of the

outcome are all things that are very difficult to predict in advance of a new initiative. Especially when it comes to the result, you never really know if your data has enough signals in it until you go through the process of data collection, preparation, and cleansing and build the actual model. Many companies start their first AI/ML projects with a lot of enthusiasm. Until they find that the projects turn out to be a bottomless pit. Months of work and thousands of dollars are spent just to get to a simple result: It doesn't work.

Prototyping is a way to minimize risk and evaluate the impact and feasibility of your ML use case before scaling it. With prototyping, your ML projects will be faster, cheaper, and deliver a higher ROI.

For this book, a prototype is an unfinished product with a simple purpose: validation. Concepts such as a Proof of Concept (POC) or a Minimum Viable Product (MVP) serve the same purpose, but these concepts have a different background and involve more than just the concept of validation.

The idea of validation is so important because you don't need to build a prototype if you don't have anything to validate.

In the context of AI or machine learning projects, there are two dimensions in which you want to validate your use case: Impact and Feasibility. That's why it's so essential that you have a rough idea of these categories before you go ahead and build something.

Here are some typical examples of things you can validate in the context of machine learning and AI:

- Does your solution provide value to the user?
- Is your solution being used as it was intended?
- Is your solution being accepted by users?
- Does your data contain enough signal to build a useful model?
- Does an AI service work well enough with your data to provide sufficient value?

To be an effective validation tool, machine learning prototypes must be end-to-end. Creating an accurate model is good. Creating a useful model is better. And the only way to get that information is to get your machine learning solution in front of real users. You can build the best customer churn predictor in the world, but if marketing and sales aren't ready to put the predictions into action, your project is dead. That's why you need quick user feedback.

When you prototype your ML-powered solution, don't compromise on data. Don't use data that wouldn't be available in production. Don't do manual cleanups that don't scale. You can fix a model, you can fix the UI. But you can't fix the data. Treat it the way you'd treat it in a production scenario.

A prototype should usually have a well-defined scope:

- Clear goal: e.g. "Can we build a model which is better as our baseline B in predicting Y given some data X?"
- Strict timebox: e.g. "We will build the best model we can in maximum two weeks"
- Acceptance criteria: e.g. "Our prototype passed the user acceptance test if feedback F happened."

When it comes to technology, a prototype should leave it up to you to decide which technology stack is the best to achieve the goals within the timeframe.

Prototypes give you valuable insight into potential problem areas (e.g., data quality issues) while engaging business stakeholders from the start and managing their expectations.

Prototyping in Business Intelligence

For decades, the typical development process for business intelligence systems was the classic waterfall method: projects were planned as a linear process that began with requirements gathering, then moved to design,

followed by data engineering, testing, and deployment of the reports or metrics desired by the business.

Even without AI or machine learning, the waterfall method is reaching its limits. That's because several factors are working together: Business requirements are becoming increasingly fuzzy as the business itself becomes more complex. In addition, the technology is also becoming more complex: Whereas ten years ago you'd have to integrate a handful of data sources, today even small companies have to manage dozens of different systems.

When your project plan for a large-scale BI initiative is ready, it's often already overtaken by reality. With AI/ML and its associated uncertainties, things aren't going to get much easier.

To get a successful AI-driven solution off the ground as part of a BI system, you also need prototyping in your toolbox. How can you do that?

Reports in BI production systems are expected by default to be reliable and available, and the information they contain is accurate. So your production system isn't the best choice for testing something among users.

What else do we have?

Companies typically work with test systems to try out new features before putting them into production. However, the problem with test systems is that they usually involve the same technical effort and challenges as the production system. Usually, the entire production environment is replicated for testing purposes: A test data warehouse, a test front end BI, a test area for ETL processes, etc., because you want to make sure that what works on the test system also works in production.

In the sense of our definition, a prototype takes place before testing, as shown in Figure. XX. For the prototype, you need deep vertical integration from data ingestion over the analytical services to the user interface while keeping the overall technical complexity low.

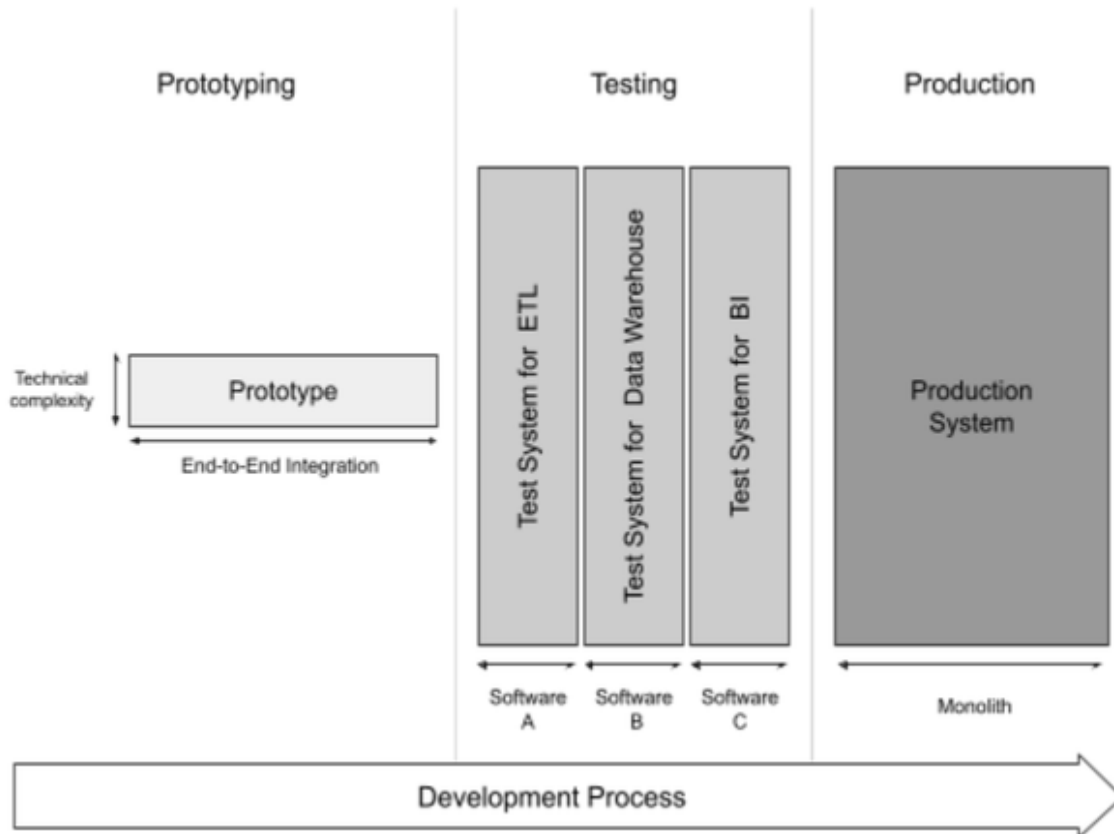


Figure 4-1. In most cases, especially with monolithic BI systems, your production and test stack aren't suitable for prototyping in just a few days or weeks.

For our purposes, we need something less complex and lightweight.

Whatever tech stack you use for this, you should make sure that:

- The tech stack supports all three layers of the AI use case architecture (data, analytics, and user interface) to ensure that data is available and fit-for-use, your models perform well enough on your data and that users accept and see value in your solution.
- Your stack should be open and provide high connectivity to allow future integration into the test and perhaps even the production system. This aspect is essential because, if successful, you want to enable a smooth transition from the prototyping phase to the testing phase. If you prototype on your local machine, it'll be complicated to transfer the workflow from your computer to a remote server and expect it to work as smoothly. Tools like Docker will help you ease the transition,

adding additional technical friction that no longer has anything to do with your original validation hypothesis.

Therefore, the perfect prototyping platform for AI-powered BI use cases offers high connectivity, many available integration services, and low initial investment costs. Cloud platforms have proven to meet these requirements quite well.

In addition, you should use a platform that gives you easy access to the various AI/ML services we covered in Chapter 2. If you have the data available, prototyping the ML model should take no more than a maximum of 2 - 4 weeks, sometimes only even a few days. Use Auto ML for regression/classification tasks, AI-as-a-Service for commodity services like character recognition, and pre-trained models for custom Deep Learning applications rather than training them from scratch.

To avoid wasting time and money unnecessarily on your next machine learning project, build a prototype first before going all in.

The AI Prototyping Toolkit For This Book

We'll use Microsoft Azure as our platform for prototyping in this book. There are many other tools like AWS or Google Cloud Platform that you can use. These platforms are very comparable in their features, and what we do here with Azure, you can basically do with any other platform.

I'm choosing Azure for the following two reasons:

- Many businesses are running on a Microsoft stack, and Azure will be the most familiar cloud platform to them
- Azure integrates smoothly with Power BI, a BI tool which is very popular in many enterprise.

You may be wondering how a full-fledged cloud platform like Azure can ever be lightweight enough for prototyping? The main reason is that we don't use the entire Azure platform, but only some parts of it that are

provided “as a service”, so we don’t have to worry about the technical details under the hood.

The main services we’ll use are Azure Machine Learning Studio, Azure Blob Storage, Azure Compute, and a set of Azure Cognitive Services, Microsoft’s AI-as-a-Service offering.

The good thing is that while we’re still prototyping, the infrastructure would stand up to a production scenario as well. We would need to change the solution’s processes, maintenance, integrations, and management, but the technical foundation would be the same. We’ll go into this in more detail in Chapter 11.

I’ll try not to rely too much on features that work exclusively between Azure and PowerBI, but keep it open enough so you can connect your own AI models to your own BI. Also, most Azure integrations into Power BI require a Pro license, which I don’t want to put you through.

Let’s start with the setup for Microsoft Azure.

Working with Microsoft Azure

The following section gives you a quick introduction to Microsoft Azure and lets you activate all the tools we need for the use cases starting in Chapter 7. While you may not need all of these services right away, it’s recommended that you set them up in advance so you can focus on the actual use cases later instead of dealing with the technical overhead.

Sign Up For Microsoft Azure

To create and use the AI services from Chapter 6 onwards, you need a Microsoft Azure account. If you are new to the platform, Azure gives you free access to all the services we cover in this book for the first 12 months (at the time of writing). You also get a free \$200 credit for services that are billed based on usage, such as compute resources or AI model deployments. You can explore free pricing [here](#).

To sign up, visit <https://azure.microsoft.com/free/> and start the sign-up process.

NOTE

If you already have an Azure account you can skip over to the next section “Create An Azure Compute Resource”. Be aware that you might be billed for these services if you are not on a free trial. If you are unsure, contact your Azure administrator before you move ahead.

To sign up, you will be prompted to create a Microsoft account. I recommend you create a new account for testing purposes (and take advantage of the free bonus!). If you have a corporate account, you can try to continue with that, but there might be restrictions from your company. Therefore, it may be best to start from scratch, play around a bit, and once you have everything figured out, continue with your company’s account.

On the following page, select “Create one!” (see Figure XX)

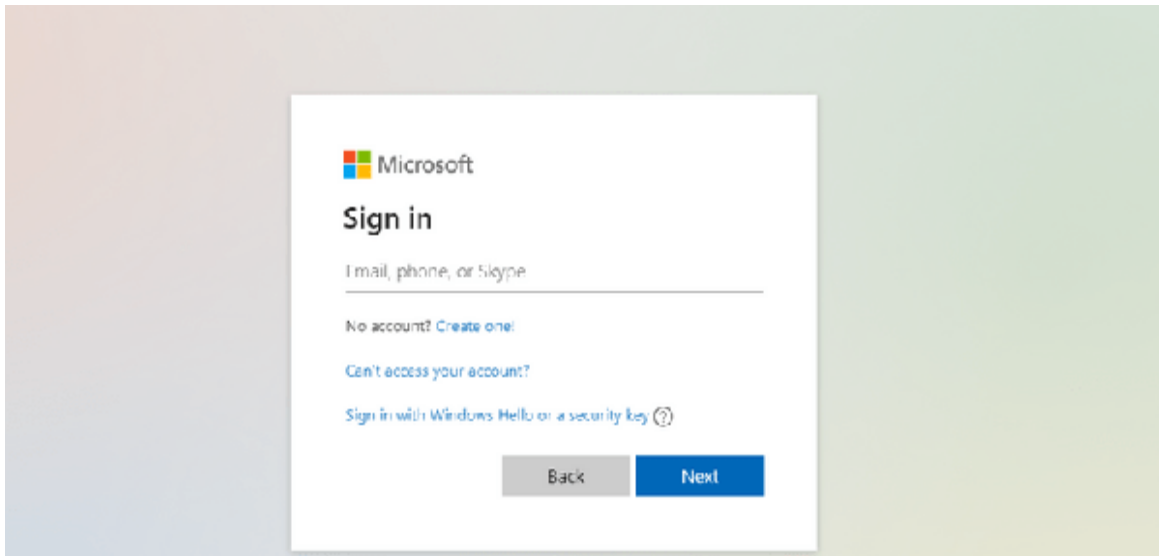


Figure 4-2. Figure caption placeholder

Next, enter your email address and click “Next”. Create a password and continue. After confirming that we are not a robot, you should see the following screen, as shown in Figure XX:

The screenshot shows the Microsoft Azure sign-up page in a web browser. The browser's address bar displays 'signup.azure.com'. The page header includes the Microsoft Azure logo and the user's email 'ai-powered-bi@outlook.com' with a 'Sign out' link. The main content is divided into two sections. On the left, the 'Your profile' section contains a form with the following fields: 'Country/Region' (a dropdown menu currently showing 'United States'), 'First name', 'Last name', 'Email address' (pre-filled with 'ai-powered-bi@outlook.com'), and 'Phone' (with an example '(425) 555-0100'). Below these fields is a checkbox labeled 'Use a different phone number to verify your identity.' and a blue 'Text me' button. On the right, the 'Create your Azure free account' section lists three benefits: 'Popular services free for 12 months', '25+ services always free', and '\$200 credit to use in your first 30 days'. Below these is a dark box titled 'No automatic charges' with explanatory text. At the bottom right, there is a blue 'Chat with Sales' button.

Figure 4-3. Figure caption placeholder

You must confirm your account with a valid phone number. Enter your information and click “Text me” or “Call me”. Enter the verification code and click on “Verify code”. After successful verification, check the terms and click “Next”.

We are almost there! The last prompt will require you to provide your credit card information. Why is this? It’s the same with all major cloud platforms. Once your free credit is used up or the free period expires, you will be charged for using these services. However, the good thing about Microsoft

for beginners is that you will not be charged automatically. When your free period ends, you'll have to opt for pay-as-you-go billing.

After successfully signing up, you should see the following screen in the Azure portal, as shown in Figure XX:

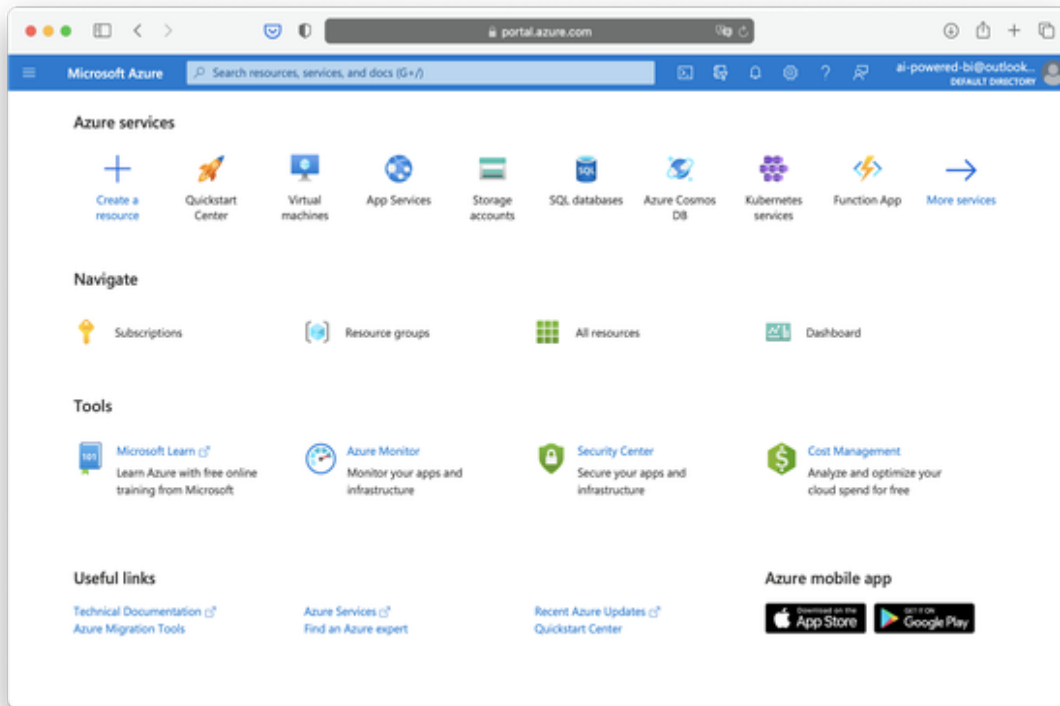


Figure 4-4. Figure caption placeholder

Welcome to Microsoft Azure! You can access this homepage at any time by visiting portal.azure.com.

Let's quickly explore the Azure portal home page:

1. Is the menu bar. You can open the portal menu on left or navigate to your account settings on the top right
2. Azure Services: Recommended services for you. These may look different for you, especially if you are newly signed up.
3. Create a resource: This will be one of the most commonly used buttons. You use it to create new resources like services on Azure.

4. Navigate and all other menu items: Takes you to various settings. You will not need this for now.

When you see this screen, you are all set with the setup for Microsoft Azure.

If you are having trouble setting up your Azure account, I recommend the following resources:

<https://azure.microsoft.com/en-us/resources/videos/sign-up-for-microsoft-azure/>

<https://azurelessons.com/create-azure-free-account/>

Create An Azure Machine Learning Studio Workspace

Azure Machine Learning Studio will be our workbench in Microsoft Azure to build and deploy custom machine learning models and manage datasets.

The first thing we need to do is create an Azure Machine Learning workspace.

The workspace is a basic resource in our Azure account that contains all of your experiments, training, and deployed machine learning models.

The workspace resource is associated with your Azure subscription. While you can programmatically create and manage these services, we will use the Azure portal to navigate through the process.

Remember, everything you do here with your mouse and keyboard can also be done through automated scripts if you need it later.

Log in to the Microsoft account you used for the Azure subscription and visit portal.azure.com.

Click the “Create Resource” button, as shown in Figure XX.

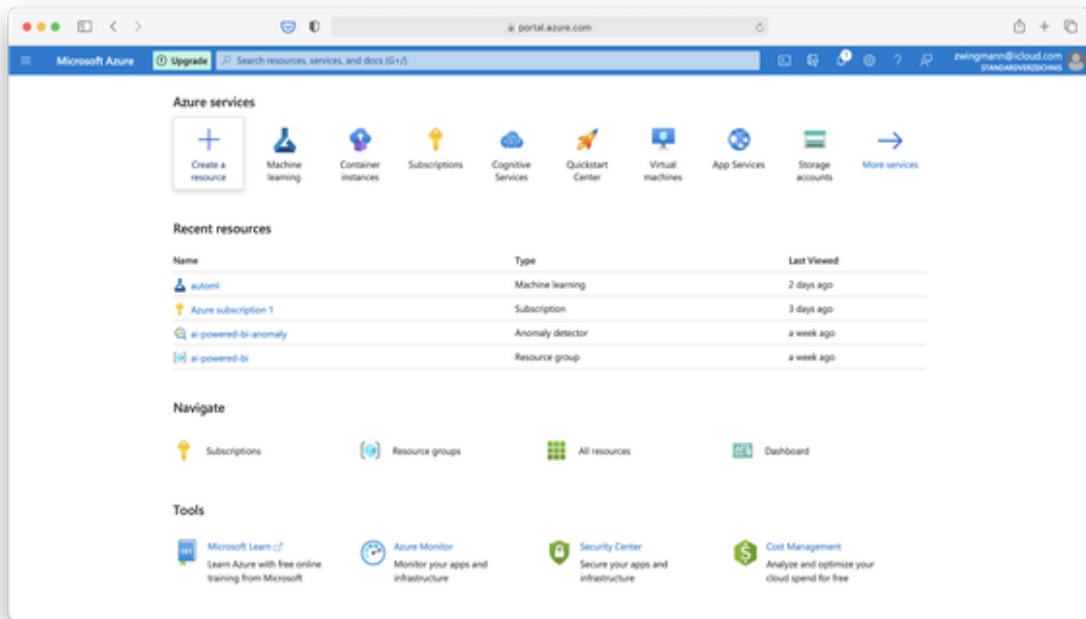


Figure 4-5. Figure caption placeholder

Now use the search bar to find “Machine Learning” and select it.

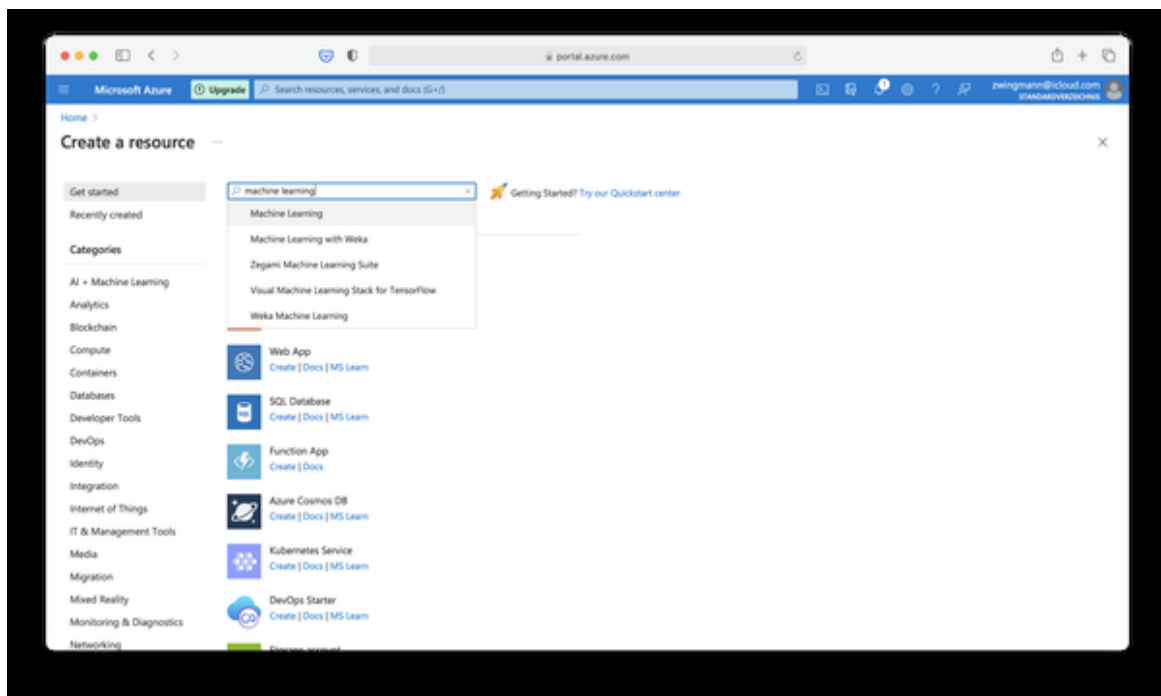


Figure 4-6. Figure caption placeholder

In the Machine Learning section, click Create.

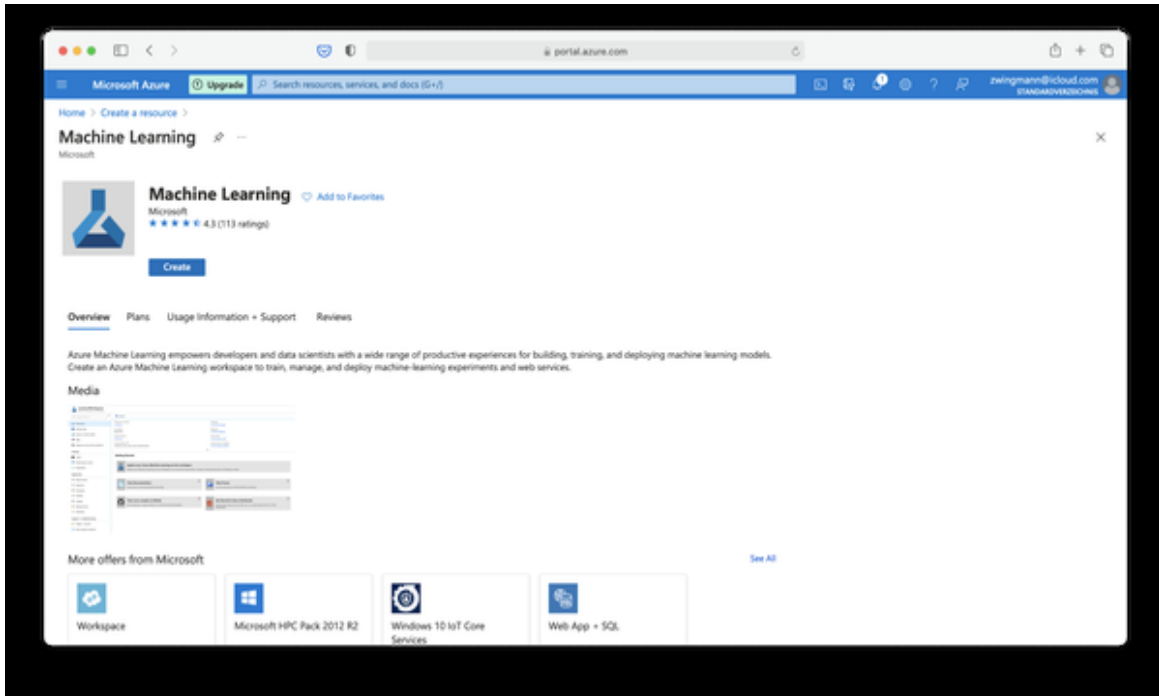


Figure 4-7. Figure caption placeholder

You will need to provide some information for your new workspace:

- **Workspace Name:** Choose a unique name for your workspace that differentiates it from other workspaces you create. Project names are typically good candidates for workspace names. In this example, we use automl. The name must be unique within the selected resource group.
- **Subscription:** Select the Azure subscription you want to use.
- **Resource group:** Enter a name to create a resource group or choose an existing one. A resource group bundles different resources in your Azure subscription. This could be your department for example. In my example, I am using `ai-powered-bi`. Whenever you see this reference somewhere you will need to replace it with your own resource group name.
- **Location:** Select the physical location closest to your users and your data resources. Be careful when transferring data outside of protected

geographical areas such as the EU. Most services are typically available first in US regions.

Once you've entered everything, hit "Review + create". After the initial validation has passed, hit create again.

The screenshot displays the Microsoft Azure portal interface for creating a machine learning workspace. The browser address bar shows `portal.azure.com`. The navigation bar includes the Microsoft Azure logo and a search bar. The breadcrumb trail indicates the path: Home > Create a resource > Machine Learning >. The main heading is "Machine learning" with a subheading "Create a machine learning workspace". The "Review + create" tab is active, showing a summary of the configuration. The "Project details" section includes a "Subscription" dropdown set to "Azure subscription 1" and a "Resource group" dropdown set to "ai-powered-bi", with a "Create new" link below. The "Workspace details" section includes fields for "Workspace name" (auto-ml), "Region" (East US 2), "Storage account" ((new) automl5982979842), "Key vault" ((new) automl7900833397), "Application insights" ((new) automl3820033333), and "Container registry" (None), each with a "Create new" link. At the bottom, there is a blue "Review + create" button, a grey "< Previous" button, and a white "Next: Networking" button.

Figure 4-8. Figure caption placeholder

It can take several minutes to create your workspace in the Azure cloud. When the process is finished, you will see a success message as in Figure

XX.

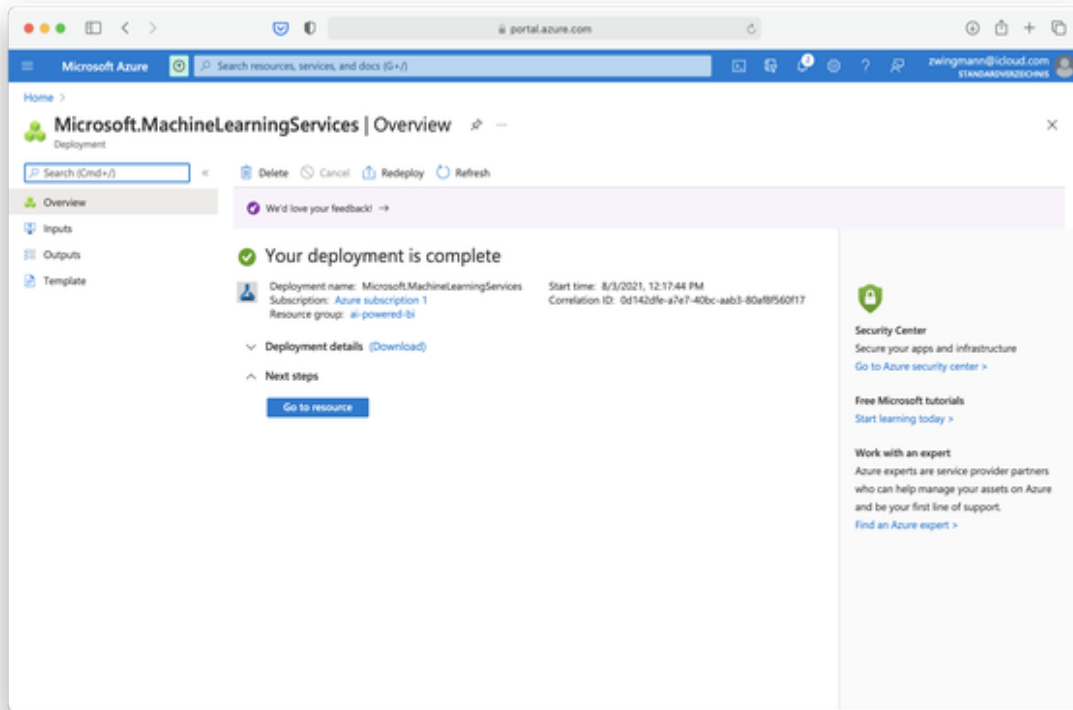


Figure 4-9. Figure caption placeholder

To view your new workspace, click on “Go to resource”. This will take you to the Machine Learning resource page as in Figure XX:

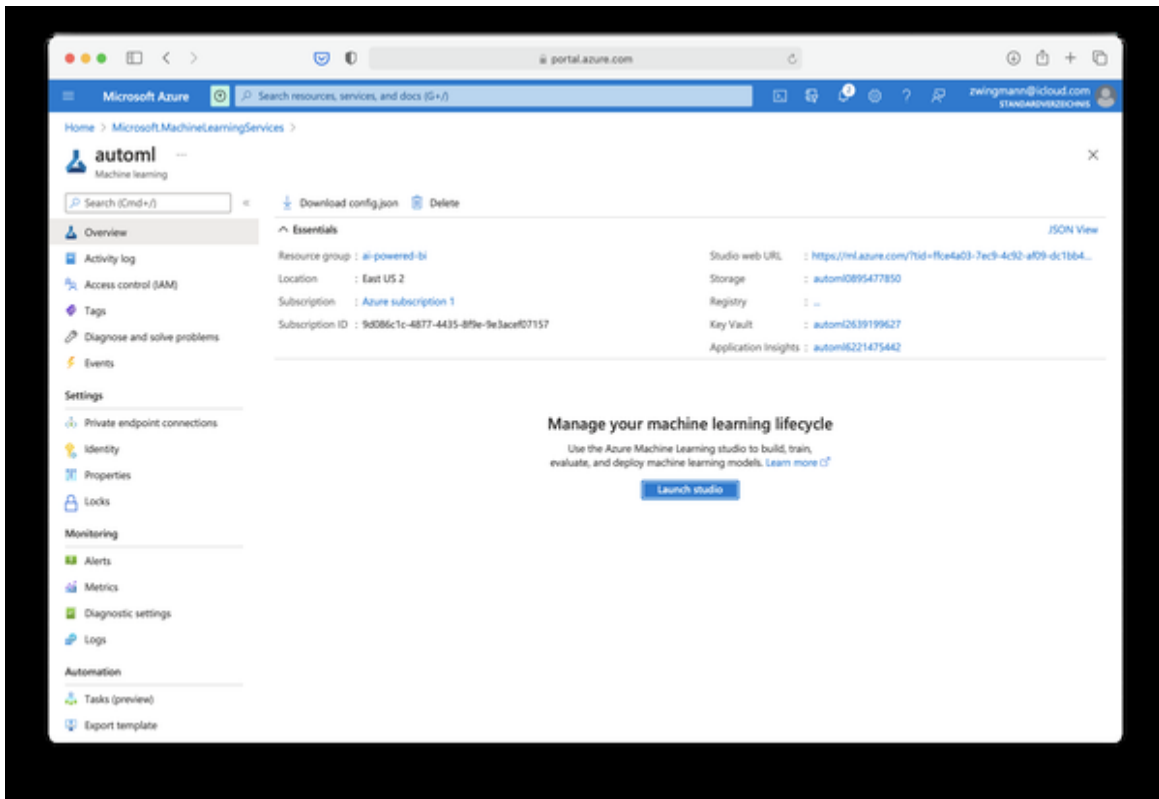


Figure 4-10. Figure caption placeholder

We have now selected the previously created workspace. Verify your Subscription and your resource group here. If we were to train and deploy our machine learning models via code programatically we could now also head over to our favorite code editor and continue from there.

But since we want to continue our no-code journey, we click on “Launch Studio” to train and deploy Machine Learning models without writing a single line of code. This will take us to the welcome screen of the Machine Learning studio. You can access this also directly via ml.azure.com.

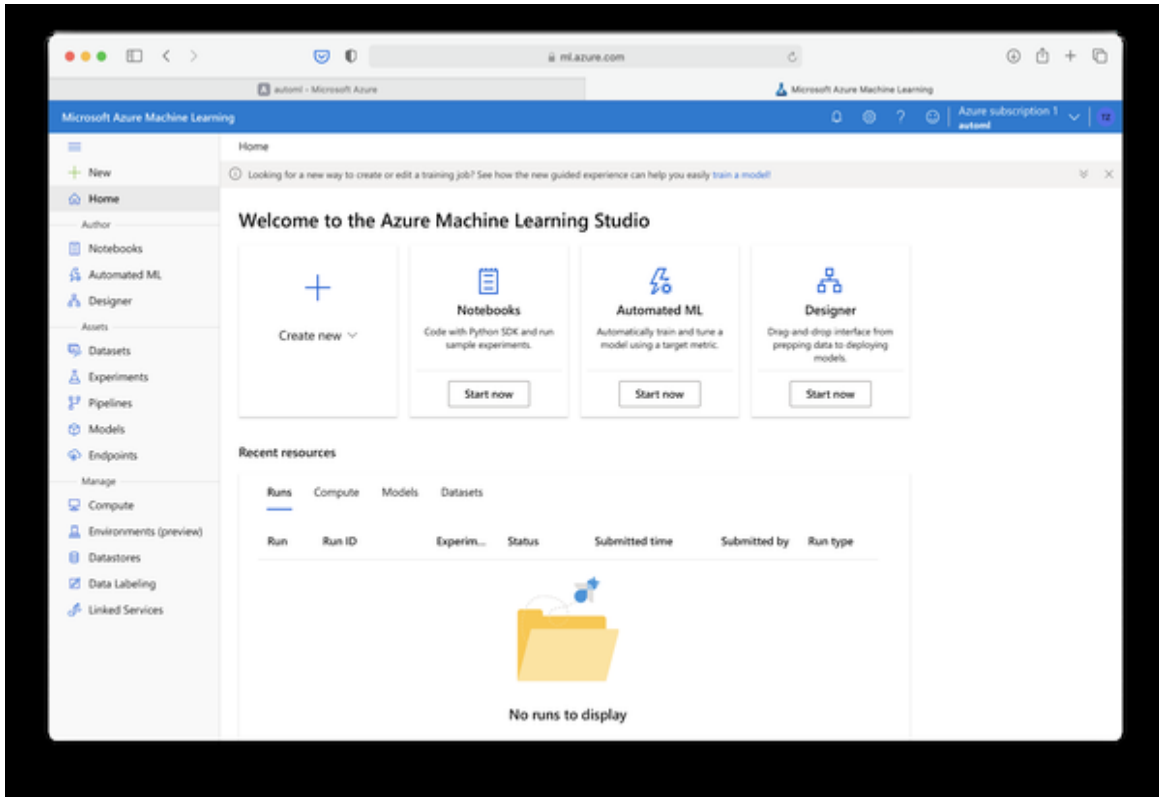


Figure 4-11. Figure caption placeholder

The Machine Learning Studio is a web interface that includes a variety of machine learning tools to perform data science scenarios for data science practitioners of all skill levels. The studio is supported across all modern web browsers (No, this does not include Internet Explorer).

The welcome screen is divided into three main areas:

- On the left you will find the menu bar with access to all services within ML Studio.
- On the top you will find some suggested services and you can directly create more services by clicking create new.
- On the bottom you will see an overview of recently launched services or resources within ML studio (which should still be empty by now)

Create An Azure Compute Resource

So far we have created an Azure subscription and a workspace for Machine Learning Studio. You can consider these things still as “overhead”. So far, we haven’t launched any service or resource that actually does something.

That’s about to change now.

In the following steps we will create a so-called “compute resource”. You can imagine a compute resource as a virtual machine (or a cluster of virtual machines) where the actual workload of our jobs is running. On modern cloud platforms such as Microsoft Azure, compute resources can be created and deleted within a few clicks (or prompts from the command line) and they will be provisioned or shut down in a matter of seconds. You can choose from a variety of machine setups - from small cheap computers to heavy machines with GPU support.

NOTE

Once you create your compute resource and this resource is online you will be charged according to the price mentioned in the compute resources list (e. g. 0.6\$/hour). If you are still using the free Azure credits this will be deducted from them. If not, then your credit card will be charged accordingly. We will only need this compute instance for an hour or so to run the simulation. Once you complete this chapter, make sure to delete your resource. For this go to Manage → Compute and select your compute resource from the list. Choose either “stop” to pause the compute service or “delete” to remove it completely

We’ll create a computational resource that we can use for all the examples in this book. In practice, you may want to set up different compute resources for different projects, also to have more transparency about which project incurs which costs. However, in our prototyping example, one resource is sufficient, and you can easily track whether the resource is running or not so that it doesn’t eat up your free trial budget.

We can create a compute resource directly in Azure ML Studio. Click on “Compute” in the left navigation bar.

Then click on “Create new compute” and a screen like the one in Figure XX will appear. To select the machine type, select “Select from all options”.

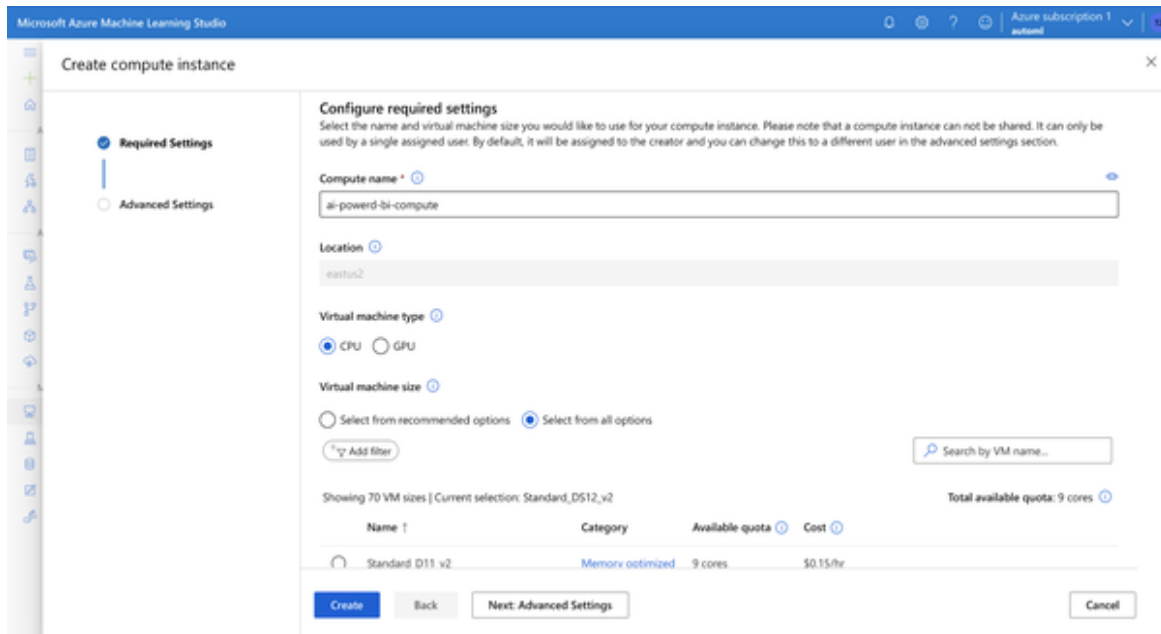


Figure 4-12. Create new compute instance

Here you can see all available machine types. The more performance you allocate, the faster a machine learning training usually will be. The cheapest resource available is a “Standard_DS1_v2” machine with 1 core, 3.5 GB RAM and 7GB storage which costs about \$0.06 per hour in the US East region, as shown in Figure 7-10.

Type “Standard_DS1” in the “Search by VM name” and select the machine with the lowest price, as seen in Figure XX. You can really choose any type of machine here, just make sure that the price is reasonable, because we don’t need high performance at the moment.

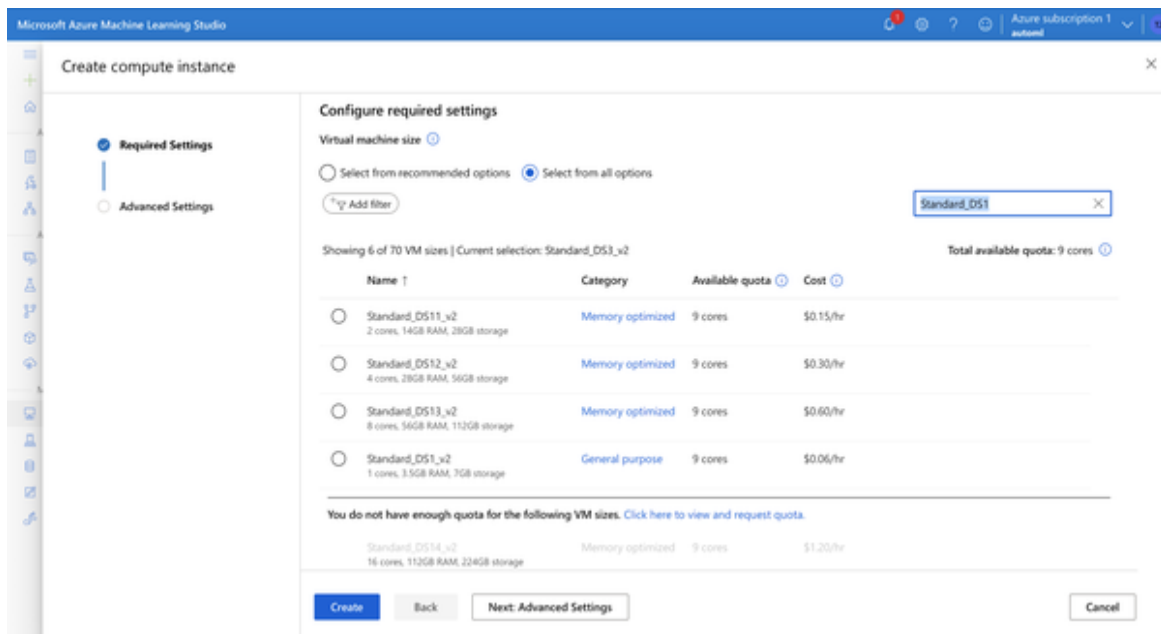


Figure 4-13. Choose a Standard_DS1 machine

Click on “Create.” This should take you back to the overview page of your computing resources. Wait for the resource to be created, which usually takes between 2 and 5 minutes.

The compute resource will start automatically once it’s provisioned (see Figure XX).

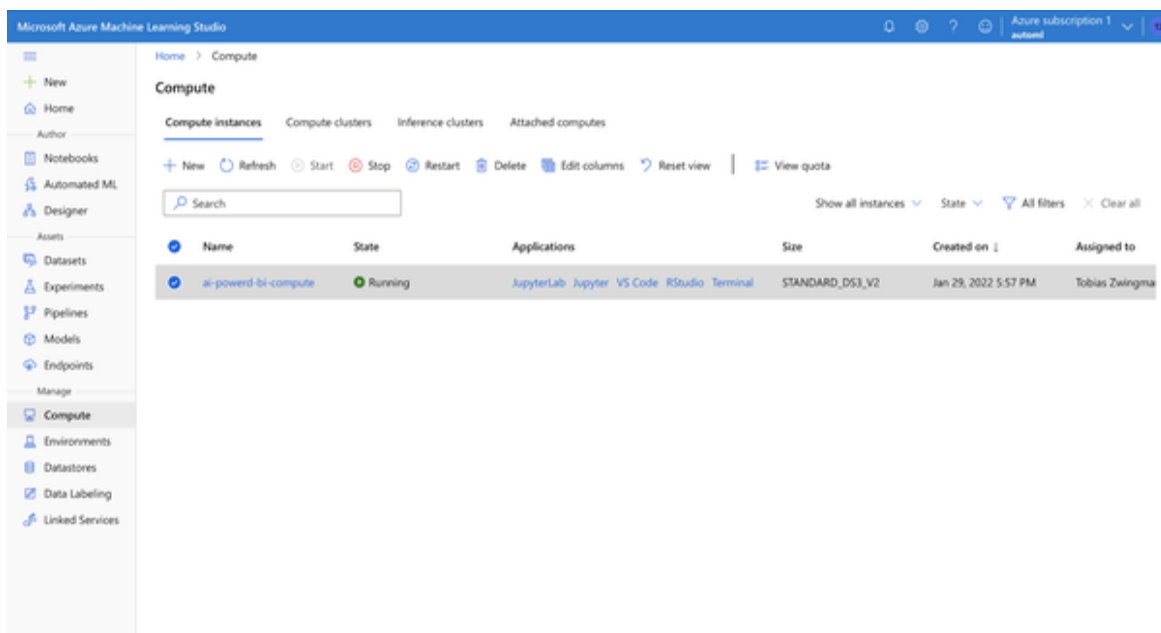


Figure 4-14. Figure caption placeholder

Since we won't need this resource until Chapter 4, select the resource and click “Stop” to prevent it from being encumbered. In the end, your compute resource summary page should look similar to Figure XX. There should be a compute resource created but stopped.

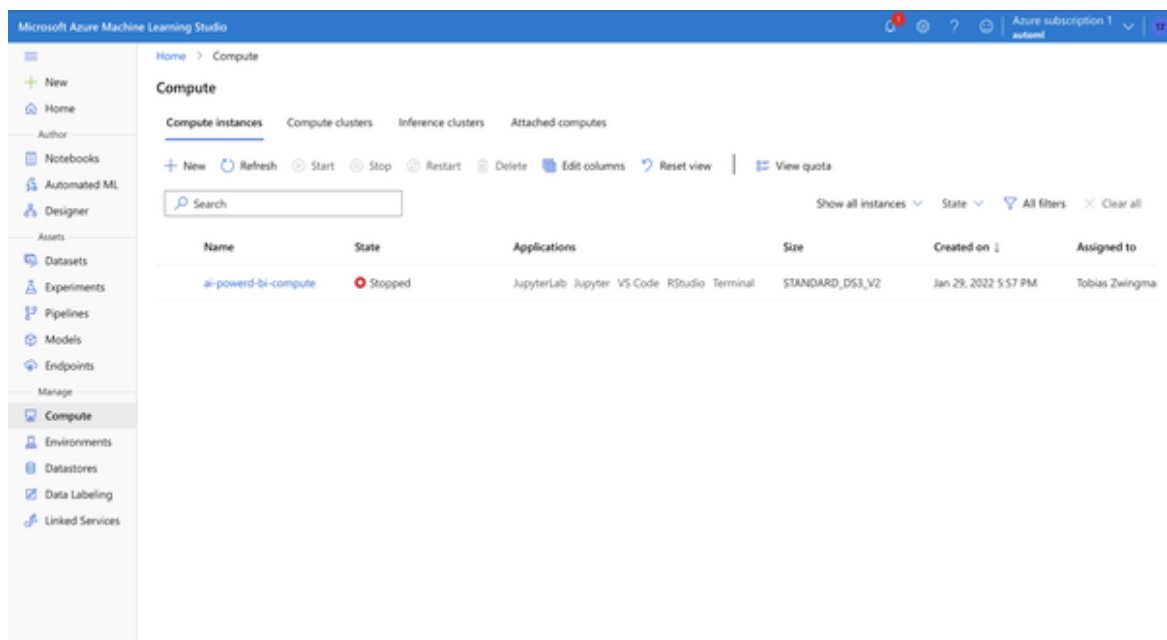


Figure 4-15. Figure caption placeholder

Create An Azure Blob Storage

The last building block we will need for our prototyping setup is a place where we can store and upload files such as CSV tables or images. The typical place for these “binary” files is a so-called Blob storage where blob stands for binary large object - basically a file storage for almost everything with almost unlimited scale. We will use this storage mainly for writing outputs from our AI models or staging image files for the use cases in Chapter 7.

To create a blob storage in Azure, visit portal.azure.com and search for “storage accounts”. Click “Create” and create a new storage account in the same region where your machine learning studio resource is located. Give this account a unique name, select standard performance and Locally-redundant storage as shown in Figure 3-x. As this is only test data we don't need to pay more for higher data availability standards. Again, we will only

save data here temporarily and the expenses will be more than covered by your free trial budget.

The screenshot shows the Microsoft Azure portal interface for creating a new storage account. The browser address bar shows 'portal.azure.com'. The page title is 'Create a storage account'. The 'Basics' tab is selected, with other tabs like 'Advanced', 'Networking', 'Data protection', 'Tags', and 'Review + create' visible. The instructions state: 'Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.' The 'Subscription' dropdown is set to 'Azure subscription 1'. The 'Resource group' dropdown is set to 'ai-powered-tp', with a 'Create new' link below it. Under 'Instance details', there is a link: 'If you need to create a legacy storage account type, please click [here](#).' The 'Storage account name' field contains 'ai-powered-tp'. The 'Region' dropdown is set to 'US East US'. The 'Performance' section has two radio buttons: 'Standard: Recommended for most scenarios (general-purpose v2 account)' which is selected, and 'Premium: Recommended for scenarios that require low latency.' The 'Redundancy' dropdown is set to 'Locally-redundant storage (LRS)'. At the bottom, there are three buttons: 'Review + create' (highlighted in blue), '< Previous', and 'Next: Advanced >'. The user's email 'rweingmann@cloud.com' and 'STANDARDVERSIONS' are visible in the top right corner.

Figure 4-16. Create a storage account

Once the deployment is complete click on “Go to resource”. You will see an interface as shown in Figure 3-xx.

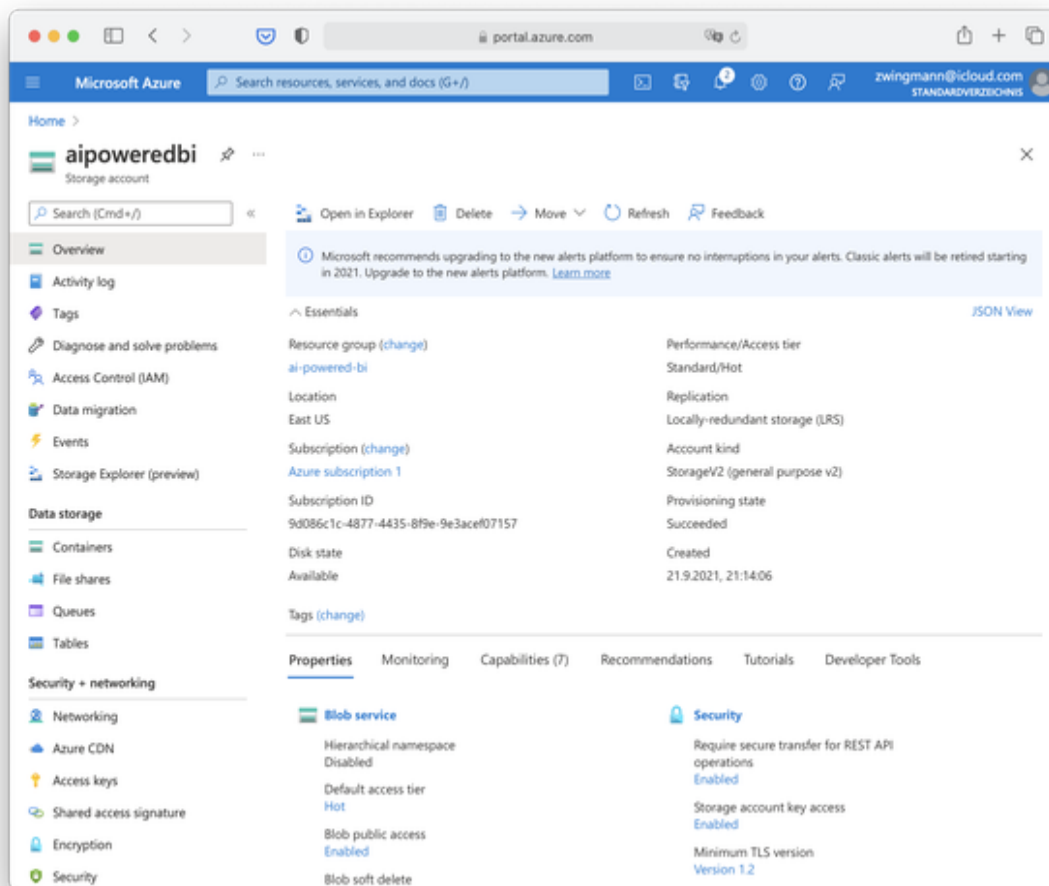


Figure 4-17. Azure storage account interface

Navigate to “Access Keys” as shown in Figure 3-xx. You will need these access key when you want to access objects in that storage programmatically.

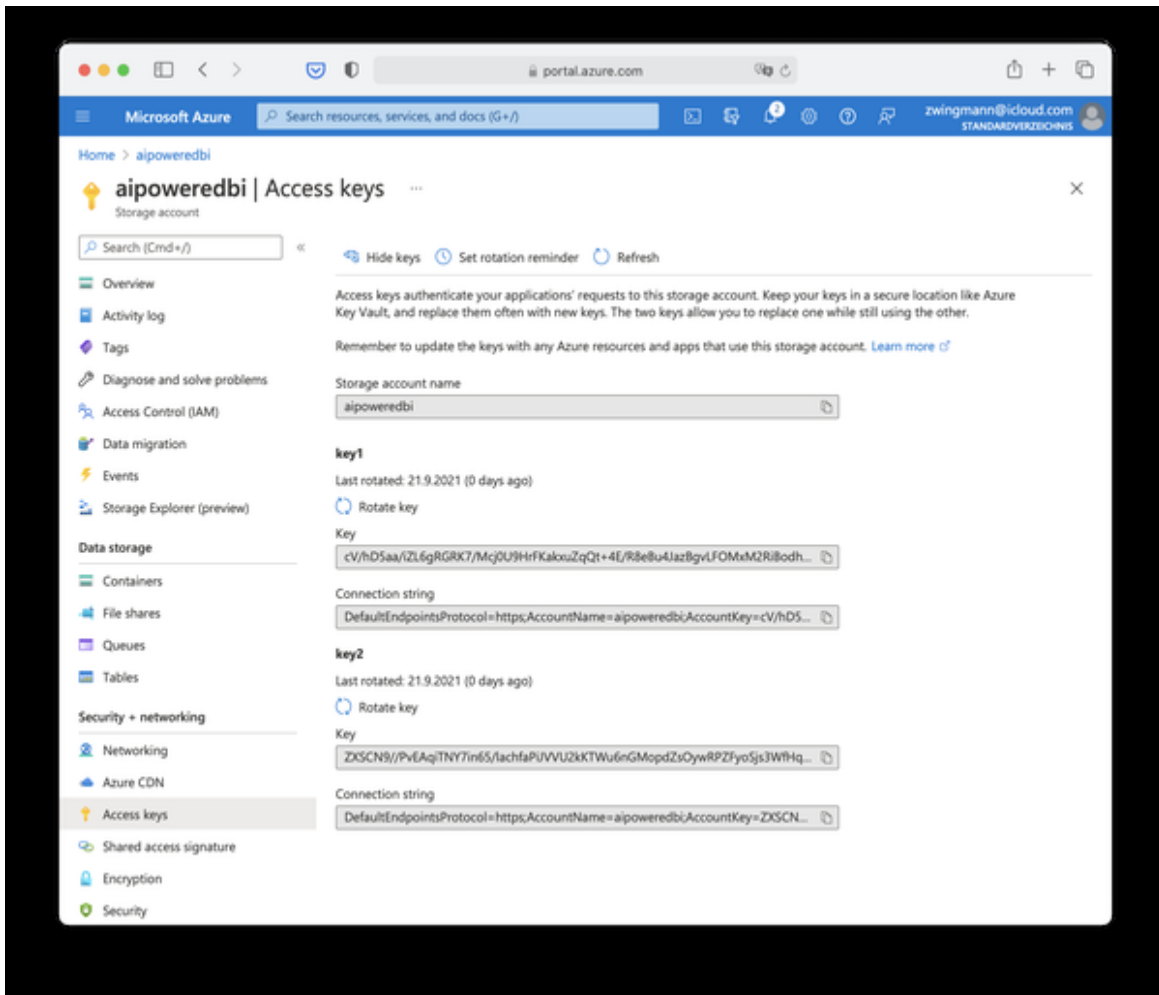


Figure 4-18. Access keys for Azure storage account

Now that we have a storage account and keys to manipulate data here, we have to create a so-called container which is something like a file folder that helps us organize our files.

Navigate to “Container” on the right side of the menu and click “+ Container” on the top. Create a container called “tables”. Set the access level to “Container” which will make it easier for us to access the files later through external tools such as Power BI.

Beware, if you deal with sensitive or production data you would of course choose “Private” here to make sure that authorization is needed before the data can be accessed.

Voilà, the new container should appear in the list and is now ready to host some files for us.

Working With Microsoft Power BI

For the user layer, we will rely on Microsoft Power BI as our tool to display reports, dashboards and display the results from our AI models.

Power BI comes in two versions: Power BI Desktop and Power BI Service.

Power BI Desktop is currently only available for Windows. It is an offline application that is installed on your computer and has all features you can expect from Power BI. If you have a Microsoft Office subscription, Power BI is most likely part of that. You can download Power BI here:

<https://powerbi.microsoft.com/de-de/downloads/>

Power BI Service is an online version of Power BI that runs across all platforms, including Macs. It doesn't currently have all features as the desktop version but new features are added regularly. I can't give you a guarantee that all things we cover in this book will work on Power BI service, but it's worth a try if you can't or don't want to install Power BI on your computer. You can get started with Power BI service here:

<https://powerbi.microsoft.com/en-us/getting-started-with-power-bi/>

If you have never worked with Power BI before, it might be useful to watch this 45-min introductory course on Power BI from Microsoft, which gives you a high-level introduction on how the tool works:

Introduction to Power BI

In case you don't like using Power BI or want to use your own BI, you should be able to recreate everything we will cover in the case studies (except for Chapter 4 & 5) in any other BI tool as well, as long as it supports code execution of either Python or R scripts.

Chapter 5. AI-Powered Descriptive Analytics

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 5th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

In this chapter we will explore two different use cases and look at how AI can help us to do descriptive analytics faster and provide a more intuitive and seamless way to interact with large datasets - even for non-technical people. We will also find out how natural language capabilities of modern BI tools - in our particular example Power BI - will take away some of the mundane tasks from us.

Use Case 1: Querying Data with Natural Language

The typical analytical thinking process of a business user often starts with a simple question such as:

- How were my sales last month?
- Did we sell more this year than last year?

- What's the top selling product?

These questions are mostly descriptive in nature: Users first need to understand the status quo before they can dive deeper into the analysis or even anticipate future events.

The “classical” way to solve this problem would be for an analyst to generate a bunch of static reports for the most common questions. Another way to answer this problem is with self-service BI. Instead of pre-generating all important reports and visualisations, the business users themselves could slice and dice data and create the visualisations they need on their own.

However, there are two problems with the self-service approach:

First, for a business analyst or BI designer, it is immensely difficult to anticipate all of the most common questions with static reports or visualisations. The problem is, depending on which problem you look at and which people you ask, the exact area of interest can be fundamentally different. In corporate settings, this typically leads to long and exhaustive meetings with different business stakeholders discussing which plots to show in a dashboard.

And second, it is simply not possible to enable every possible thinking pathway through customizable drill-downs or additional dimensions. One topic often leads to one another and high level reports are often only the beginning to a plethora of more questions. These factors lead to complex dashboards just to show descriptive analytics from the past. Also, not all business users understand how to drill down or slice data. Dragging together different measures and dimensions to custom reports still means too much friction for non-technical consumers who are not trained data analysts.

What business users do instead is ask questions. This is where they would typically call in a data analyst for help, solving the problem of technical friction by looping in human support. Of course, this approach does naturally not scale well since in most corporate settings you will typically

have more business users than data analysts. And even if you can afford having a lot of analytical know-how in house, you don't want to keep your analysts busy with repetitive, easy-to-solve tasks.

Let's now lay out the scenario for our example use case.

Problem Statement

We are looking at sales data in Power BI. Sales management has approached the BI team because they recognized a positive revenue trend and want to get some deeper understanding of what's currently going on. As BI analysts, we were tasked to provide support to sales staff and help them to get insights with regards to revenue development in their particular areas of interest. In the same sense of self-service BI, we want to provide "help for self-help" and give sales people the possibility to interact with the data set seamlessly and without any technical friction.

Solution Overview

A more convenient way to get answers from data in an intuitive fashion is to simply let users ask questions against the data using natural language and have them answered automatically without any, or with very little, human assistance. Most people don't realize that they've been trained in this method for years, or for however long they have been using an internet search engine like Google.

For example, if a business user wants to compare sales figures for a certain country with the numbers from last year, they might want to shoot off a query such as:

"Show me sales in the USA last year vs this year"

In fact, AI-powered Natural Language features let users just do that. What is happening under the hood is that an AI-powered Natural language model is interpreting the user input and tries to map it to available data in the dataset and come up with an appropriate visualisation for it. The experience should be quick, interactive, and frictionless.

We are using Power BI as our BI tool and we will leverage the built-in AI capabilities that Power BI has for our purpose. Other BI tools such as Tableau offer similar functionalities, but the details might be different. In any case, choosing a good BI front-end for this task is key, since you cannot add NLP capabilities to a BI software if there is no native support for this (or add-ons that are offering this).

In Power BI the NLP feature is called the Q&A visual. The Q&A visual lets you use natural language to explore your data in your own words. Q&A is interactive, and can be even fun. It seems like a good candidate for our problem statement.

The Q&A visual can be dragged onto a report just like any other visual so the user can perform custom queries against the data. The Q&A tool can also be added as a button in the ribbon, independent from a report and also be integrated into dashboards. The Q&A visual is free and available in both Power BI desktop and Power BI service. Mobile devices are supported with the Power BI apps on iOS and Android. The Q&A tool works with the following data sources: Imported data, live connection to Azure Analysis Services and SQL Server Analysis Services (via gateway) as well as Power BI datasets.

Power BI Walkthrough

To follow along with the following example, you will first need to download the [Sales & Marketing sample PBIX file](#).

The dataset contains various sales and marketing data from a fictitious manufacturing company. The dataset comes pre-populated with some reports to keep track of the company's market share, product volume, sales figures and sentiment scores. For our case study, we assume that all manufacturers in this dataset belong to one holding group so that we can analyse the total revenue as a metric of interest. In return, we are not so interested in the individual manufacturers' market share as highlighted in the original scenario. You can learn more about the dataset from the [website](#).

Let's start to load the dataset into Power BI. You can either use Power BI Desktop or Power BI Service to follow along.

In the upper left section of Power BI, select File > Open report > Browse reports. Then,

select the Sales & Marketing sample PBIX file.

You should see the intro screen of this report file, as shown in Figure 4-1.

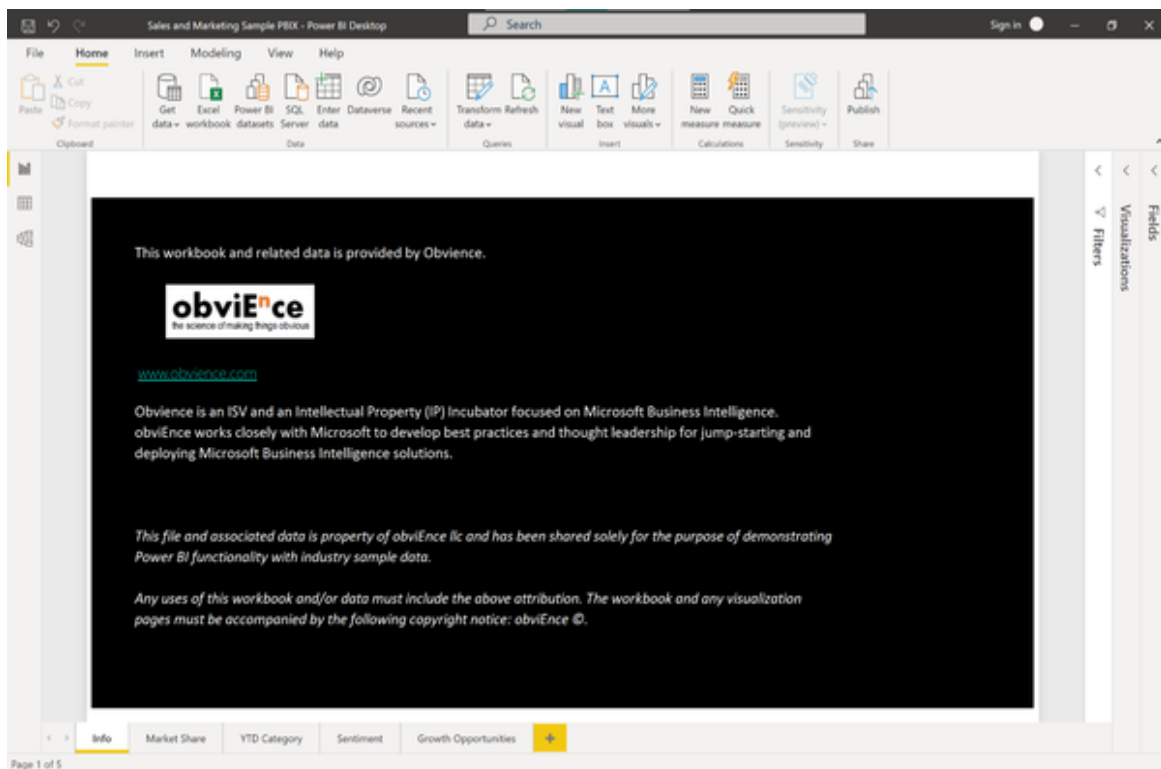


Figure 5-1. Sample Power BI Report

For now, we won't bother with the pre-written reports and dashboards. Instead, we will start from scratch to explore revenues and sales figures.

We first need to add a new blank page to the report by clicking the plus sign at the bottom of the screen.

Select the Q&A visual icon from the visualizations pane (Figure 4-2) and drag it onto the canvas or simply double click the icon.

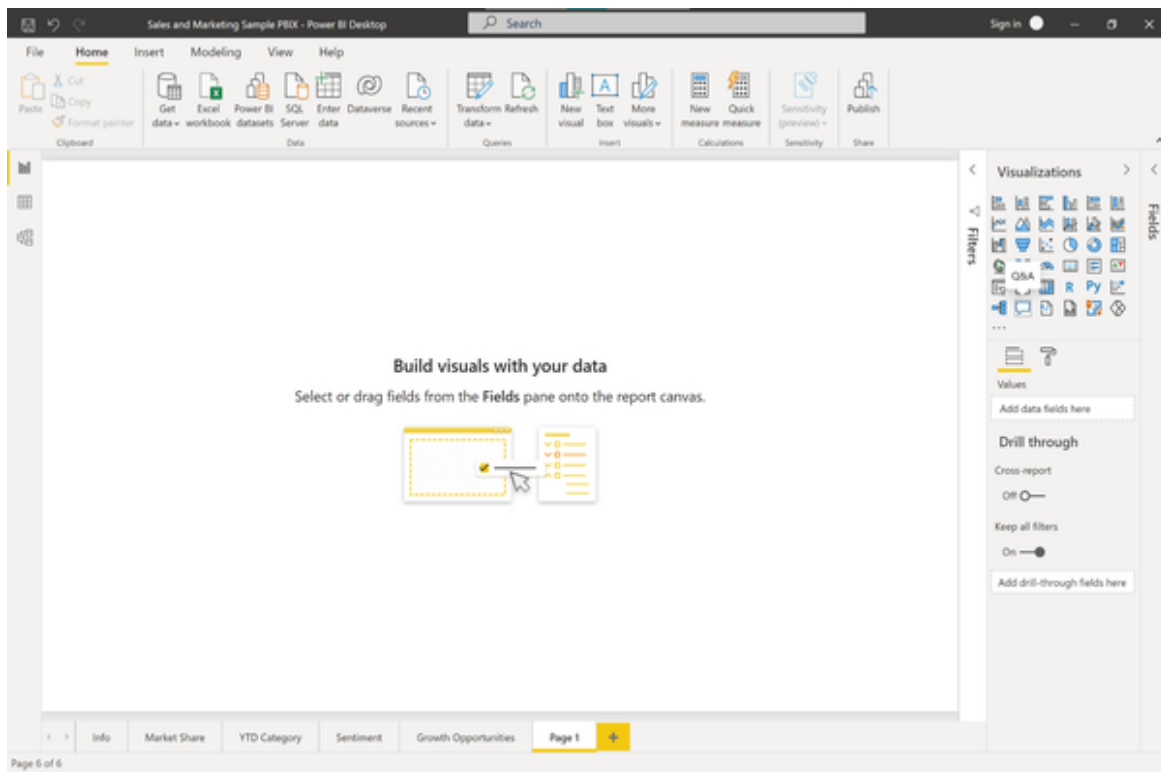


Figure 5-2. Q&A Visual

Drag the border so the visual fills the whole width of your report. Your report screen should now look similar to Figure 4-3.

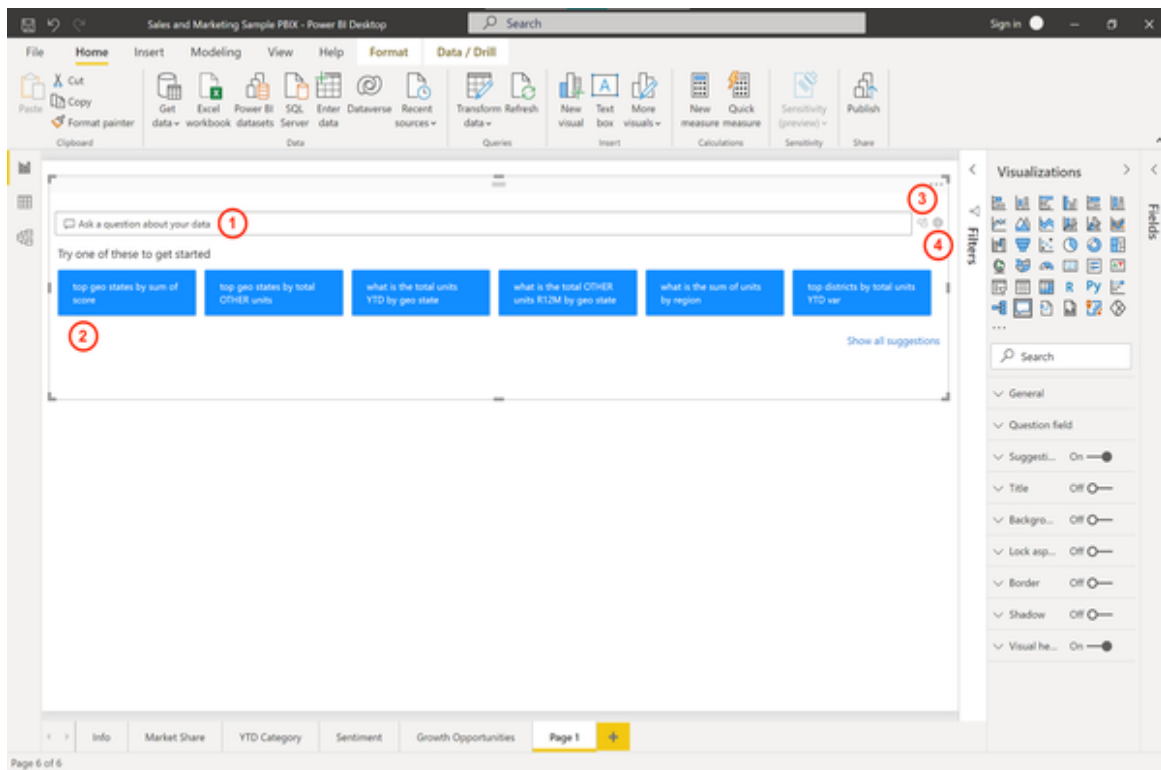


Figure 5-3. Q&A Visual Default Screen

Now, let's explore this visual in a bit more detail. The Q&A visual consists of four core components:

- **The text box (1):** this is where users type their questions or queries and where they will see autocomplete and auto suggest features.
- **The list of suggested questions (2):** This pre-populated list will contain some sample questions the user can run by a single click.
- **The convert icon (3):** This will convert the output from the Q&A tool into a standard Power BI visual.
- **The Q&A cog icon (4):** This will open a settings menu which allows designers to configure the underlying natural language engine.

Let's give this visual a try by selecting the first suggested question for us. Click on the first sample question:

“top geo states by sum of score”

Power BI will respond with the visual which seems most appropriate for this analysis. In this case it is presenting a map visual to us, showing the top states. We can change this to be almost any visual by making it part of our query. Try:

“top geo states by sum of score as bar chart”

This query will present a horizontal bar chart instead of a map.

Now, let's head over to the text box and explore how users would input questions here. The Q&A tool can answer a variety of queries, including but not limited to those listed in Table 4-1.

*T
a
b
l
e
5
-
1
.
E
x
a
m
p
l
e
C
o
m
m
a
n
d
s
f
o
r
t
h
e
P
o
w*

e
r
B
I
Q
&
A

V
i
s
u
a
l

Type	Example
Ask natural questions	Which sales has the highest revenue?
Relative date filtering	Sales in the last year
Filter by variables	Sales in the USA
Filter by conditions	Sales where product category is Category A or Category B
Show a specific visual	Sales by product as pie chart
Show aggregations	Median sales by product
Sorting	Top 10 countries by sales ordered by country code
Comparisons	Date by total sales vs total cost
Time	Sales over time

Since we are interested in revenue trends, we want to create a simple line chart showing the total revenue over time, ideally broken down by year. To achieve this, let’s ask Power BI a simple query:

“Show me revenue over time”

Wait, the result we see in figure 4-4 isn't quite what we expected. The output simply shows some dates from 1999. What's going on here?

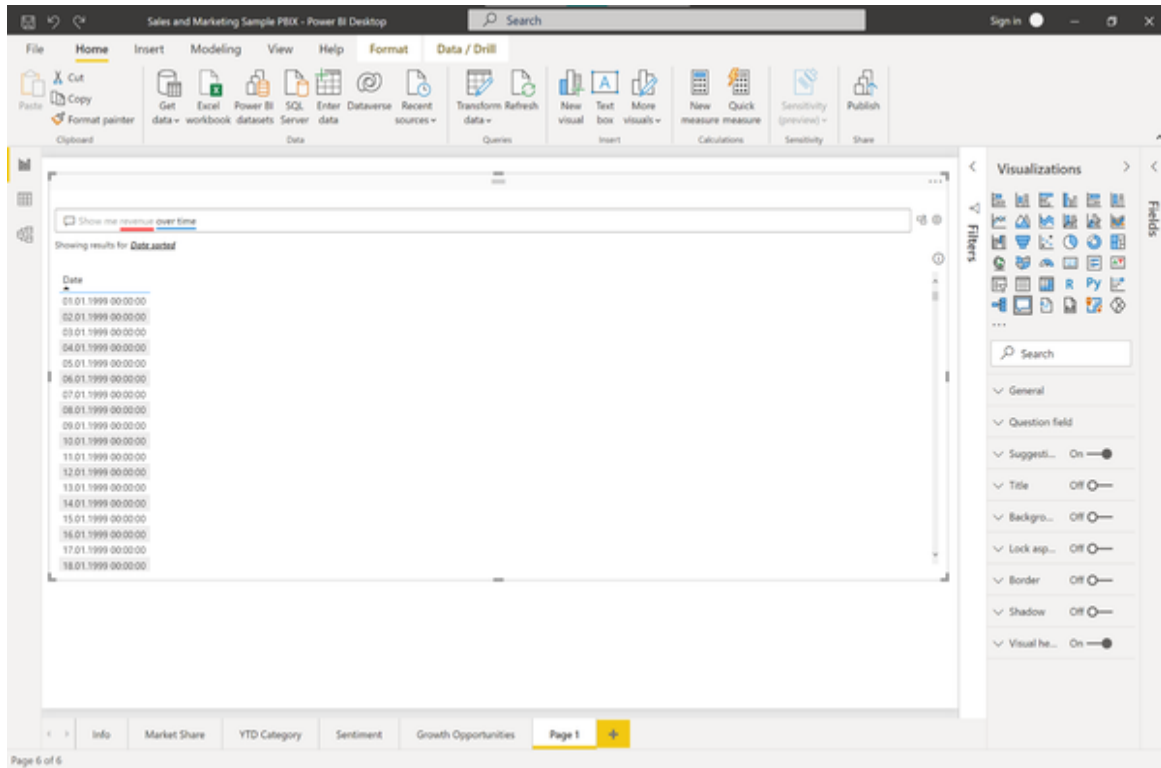


Figure 5-4. Showing revenue over time

We can find out more by looking at the text area box where Power BI helps us to identify problematic phrases in our query: The red line below “revenue” tells us that Power BI could not interpret this variable. In fact, the underlining here follows a distinct color code:

- A solid blue underline indicates that the system successfully matched the word to a field or value in the data-model.
- An orange underline indicates that the expression was matched with low confidence. This happens if the expression is ambiguous, for example because there are multiple fields that contain an expression like “sales”.
- A red underline means that the Q&A tool could not match the word at all with anything in the data model.

If you click on an underlined word you will see some suggestions how Power BI would solve the conflict. In the case of the “revenue” expression Power BI will come up with the following suggestion:

“Did you mean: Show me **sum of revenue** over time”.

As you can see, you have to be specific with the query if you rely on the default values. We’ll fix this later, but for now, let’s accept the suggestion and see what happens in Figure 4-5.

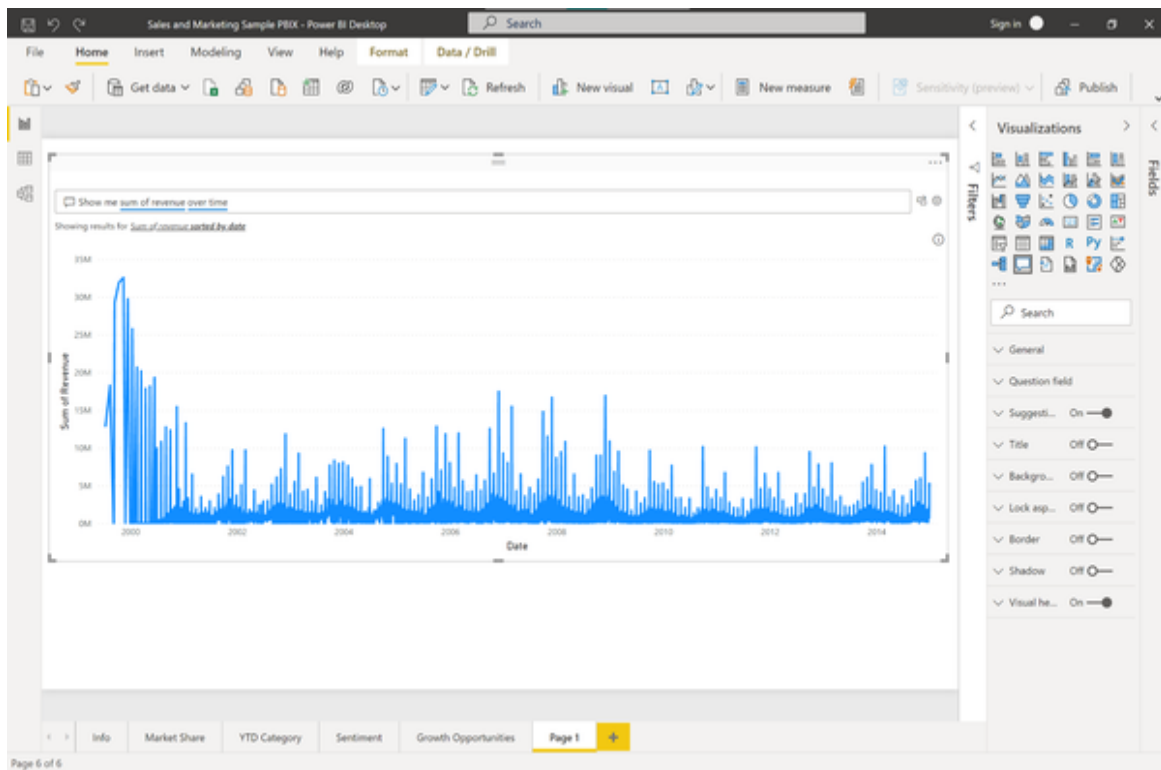


Figure 5-5. Showing sum of revenue over time

This looks more to what we expect. Power BI shows a line chart listing the total revenue over time, splitted up by day. Although this is not bad, it is hard for us to see the big trends.

So let’s be even more specific and ask:

“Show me sum of revenue over time by year”

There we go. Power BI shows us a clean line chart where we can instantly see that the revenue spiked in 2006 and then saw a negative trend

afterwards (see Figure 4-6). Since 2010 the trend seems to be again positive.

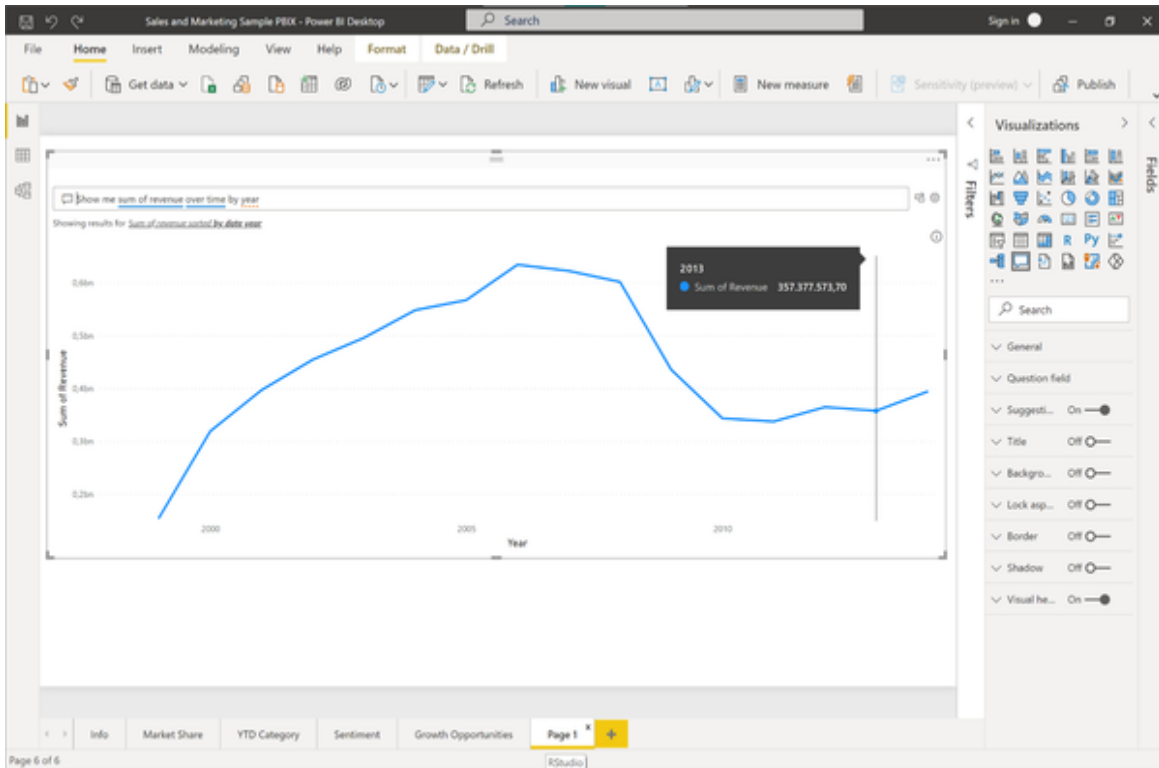


Figure 5-6. Showing sum of revenue over time by year

We want to keep this visual as our high level visual for consumers, so let's save this to a static visual by using the icon "Turn this Q&A visual into a standard visual". Power BI will convert the Q&A visual into a plain line chart.

Now it is a good time to make our report look a little nicer.

Shrink the revenue visual horizontally to make a little more space below. Add a headline on top of it reading "Self-Service Revenue Analysis". Rename this report sheet from "Page 1" to "Total Revenue by Year"

Drag and drop another Q&A visual from the visualizations pane so it appears below the total revenue line chart. Your report should now look similar to Figure 4-7:

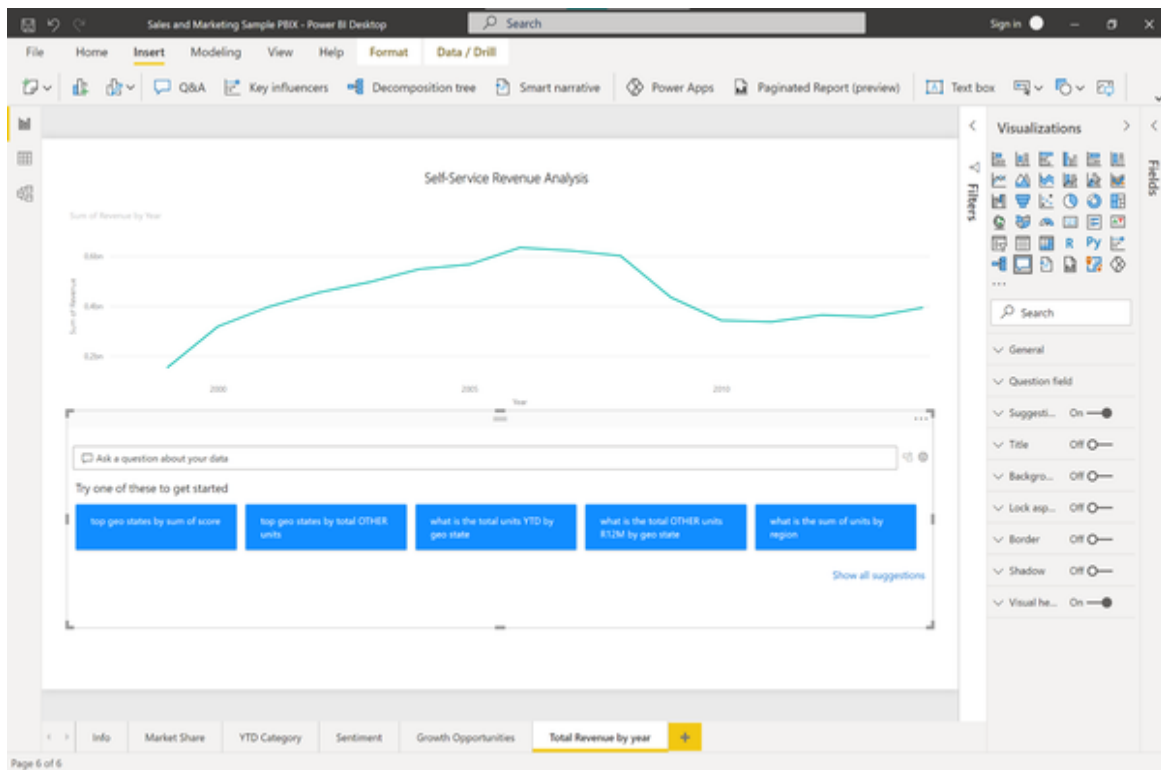


Figure 5-7. Q&A report layout

The idea is that we provide business users with a standardized high level overview of current trends or business metrics. Below we allow them to query the data with custom questions.

To do this we will edit the Q&A visual to suggest more meaningful questions relevant to the revenue.

Click the cog icon to the right of the text box to edit the settings of the Q&A visual. You will see four options here: Field synonyms, review questions, teach Q&A and suggest questions.

Head over to the field synonyms section first (see Figure 4-8). Power BI will present you with a list of your data dimensions and allow you to add synonyms to the different fields in the business lingo that your users speak. For our purpose, open the SalesFact list. You will see all measures in this list together with some auto-suggestions for synonyms and a toggle button whether this field should be included in the Q&A widget or not.

Scroll down to Sum of Revenue. Hit the “Add” button and include “total revenue” here as a synonym. Also add “total of revenue”, “revenue”, “total

income” and “total of revenue” to the synonyms. We could go through the other fields as well, but for now let’s end this exercise.

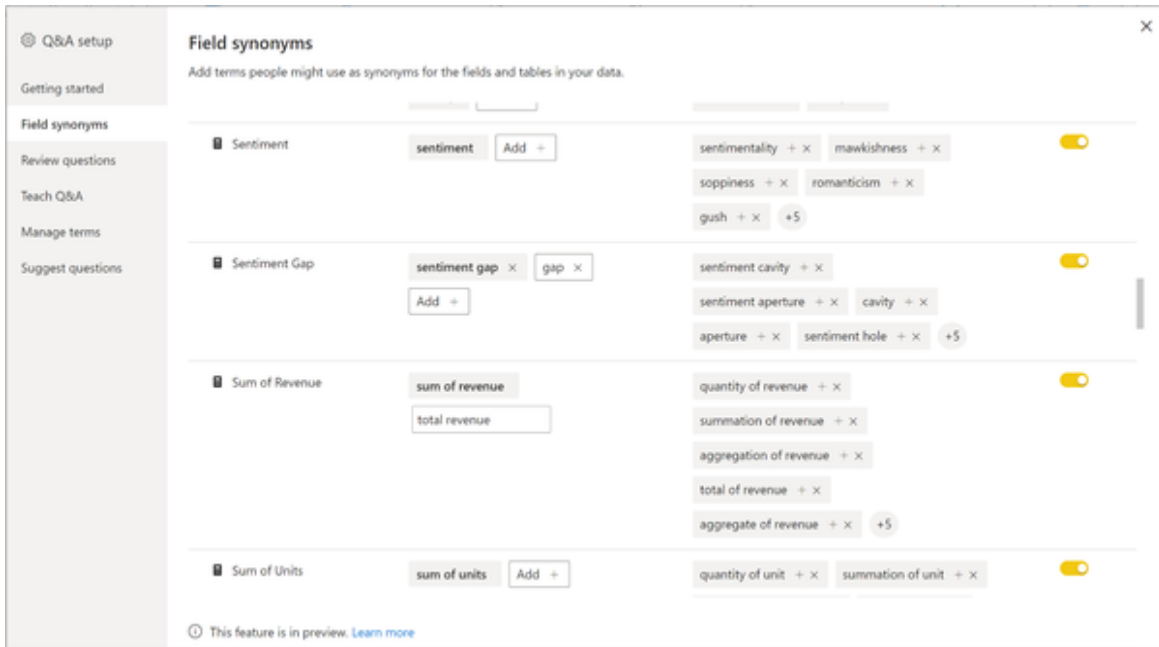


Figure 5-8. Q&A field synonyms

Click “Manage terms” in the left menu and you will see a list of all the terms and definitions that were added to the Q&A visual. You can see an example in figure 4-9. This way, you can easily keep track of any changes you made and delete terms which are not appropriate.

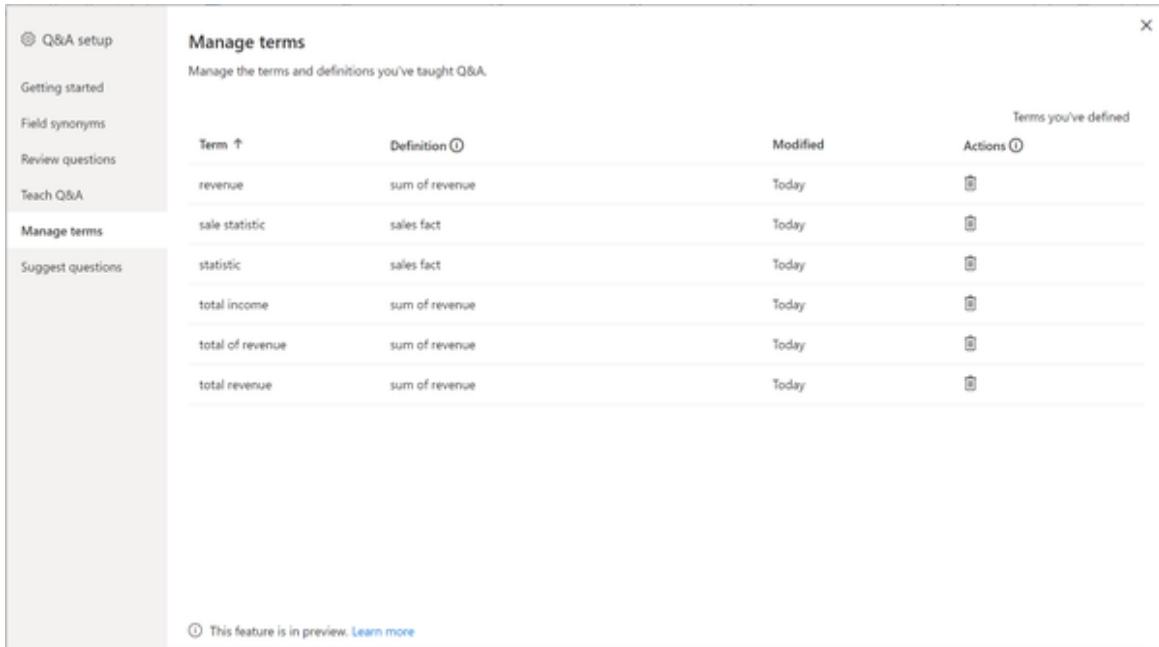


Figure 5-9. Q&A manage terms

Head over to “Suggest questions” in the left menu now. This section allows us to edit the questions which are suggested to users by default, which you can see in Figure 4-10.

Let’s start with a revenue breakdown by region. Type in:

“Show me revenue by region”

This should yield a plot in the preview window, as shown in Figure 4-10.

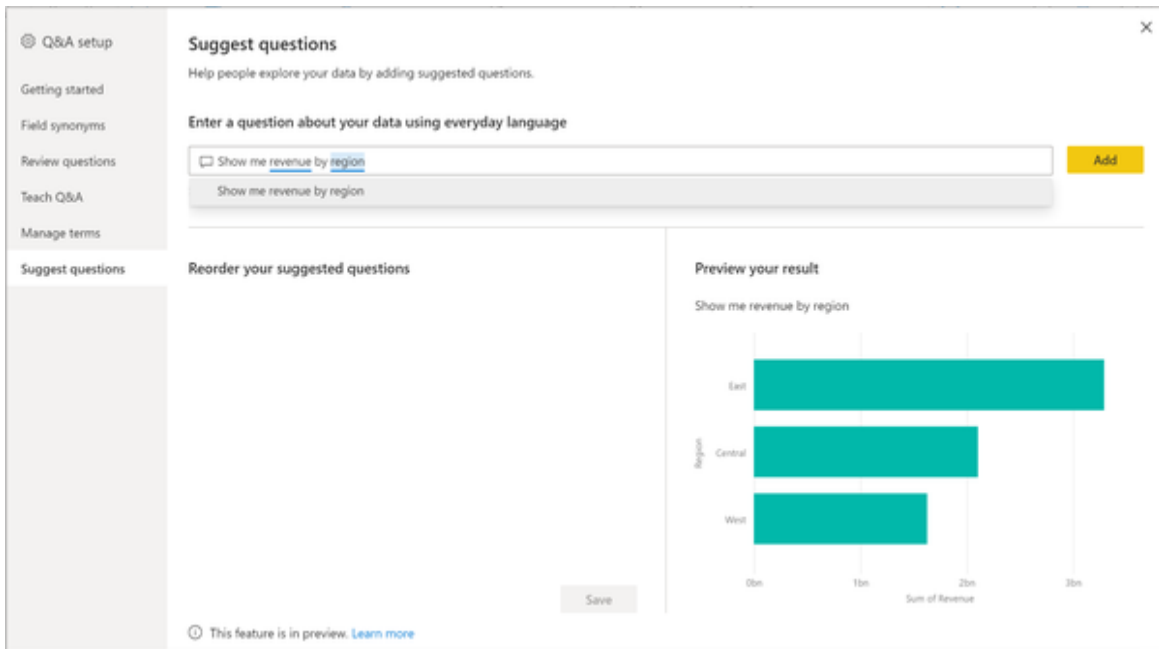


Figure 5-10. Q&A suggest questions

Fine, let's add that to our suggestions by clicking the “Add” button.

Business users are usually very good at learning by examples. To give users more intuition on how they can use the Q&A tool, let's add a variation to the previous question. Business users later should be comfortable replacing things like region with “product” or whatever it is that is on their mind.

Add the following suggestion:

“Show me revenue by region by year”

This will produce three line charts and leads over to the idea of comparing data points. So let's add an example of how business users can compare sales performance between different products. Let's add this slightly more complex suggested question:

“Show me revenue of Maximus UM-01 vs. Maximus UM-02 by year”

This will result in this very insightful chart, showing us that Maximus UM-01 had decreasing revenues since its peak in 2005 and the new product Maximus UM-02 seems to slowly take off after its initial release in 2007. By 2012, the new product contributed more revenue than its predecessor as you can see in figure 4-11.

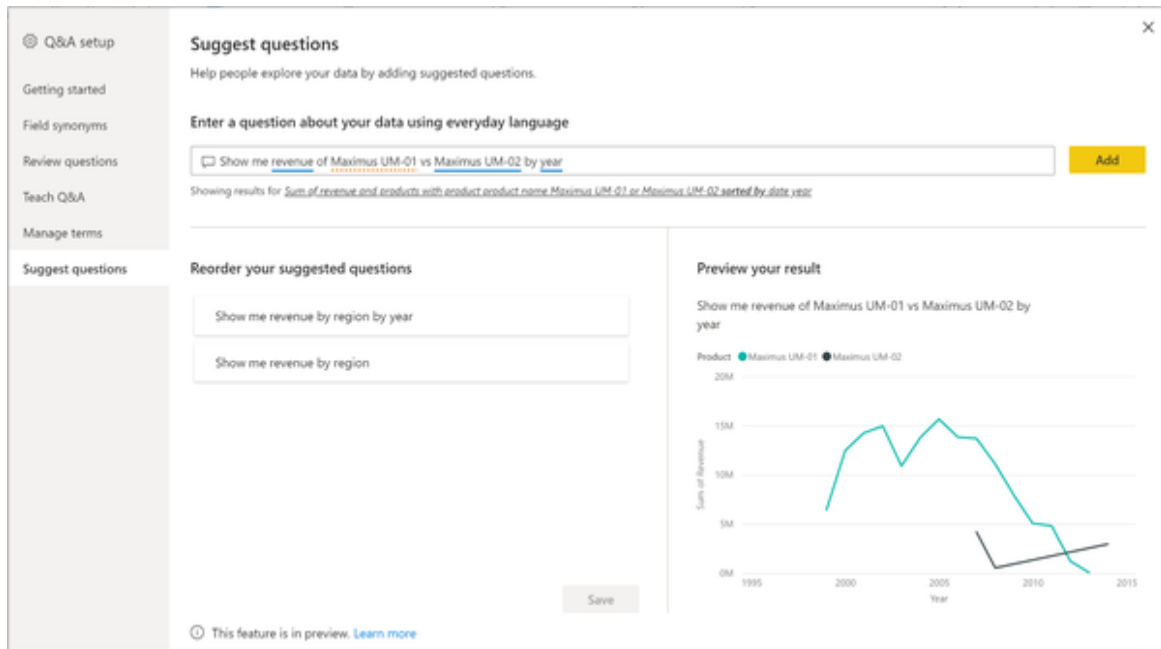


Figure 5-11. Q&A revenue comparison

To organize the list of suggested questions, you can drag the questions into the order you want. Let's put the most simple query to the first position and the most complex one to the last. Save the suggestions. Close the Q&A setup window.

Taking a look again at our report we can now see the three suggestions we just created now show up in the Q&A visual. Business users can start their data exploration by simply clicking on a suggestion and then modifying the query as they like, for example by replacing region with "geo state". You can see the final example in figure 4-12.

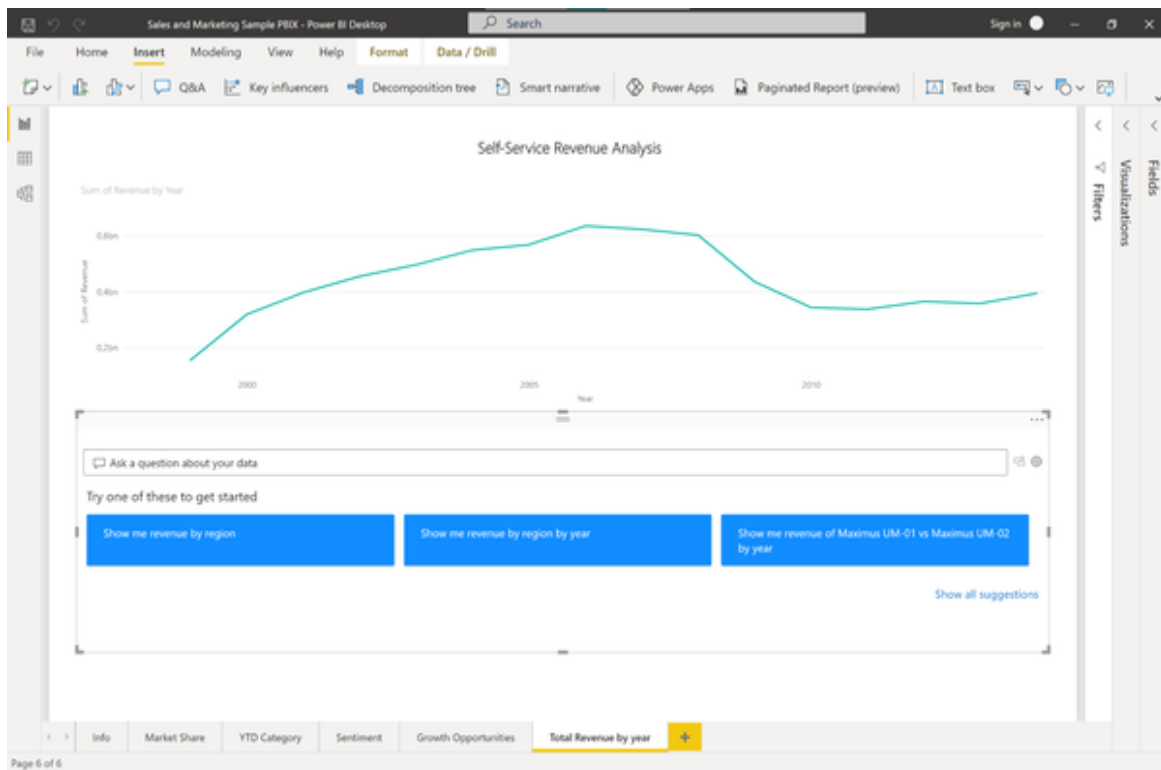


Figure 5-12. Final report with Q&A visual

This basic setup would be sufficient enough to ship an early prototype of this report to a sample of beta business users. You have the chance to collect valuable feedback, see if this is accepted and also find out where there are areas for improvement.

In the settings of the Q&A tool you can find an option to review questions asked in the last 28 days and thus see exactly how people intend to use your report and which synonyms, suggestions or fields might be still missing. This is a great place to start iterating over your product and building it so far that you can scale to more users. If you want to share this report you will need a Power BI Pro licence.

If you like to read more about how the Q&A tool and its administration work in detail, you might find the following resources quite useful from the documentation:

- [Best Practices](#)
- [Intro to Q&A Tooling](#)

- **Edit Q&A Linguistic Schema and Add Phrasings in Power BI Desktop**

Use Case 2: Summarizing data with natural language

Business analysts often have to craft data stories to communicate insights to business stakeholders. While most of their time is typically spent on creating compelling visuals, the process of annotating with conclusions can be cumbersome and tedious. Too often, analysts therefore skip the step of adding verbal comments to their visuals since the charts speak for themselves, don't they? No, in fact, many charts can get interpreted the wrong way, especially when it comes to more complex visualisations.

Let's imagine the following scenario.

Problem Statement

Building off the visuals we created in Use Case 1 we now want to add captions to assist non-technical stakeholders in interpreting the data. We want to provide captions to make insights clear, but we don't want to spend much time writing them out manually.

Solution Overview

Natural Language capabilities do not only let us analyse and interpret human language with machines, but also generate text based on defined inputs.

In our case, we can use AI technology to create captions or annotations for a given plot and its underlying data. The annotations should address key takeaways, point out trends and allow editing the language and format tailored to a specific audience. In Power BI this feature is called Smart narration and it is available in both Power BI Desktop and Power BI service for designers & developers. The feature is available for all licence types and does not require a Pro or Premium licence.

We will explore how we can use these auto-generated captions to generate a data story for our visuals we created in Use Case 1.

Power BI Walkthrough

Open the Sales & Marketing PBIX file from Use Case 1. Create a new report page and call it “Story”.

We want to create the following four visualisations:

- Revenue by year
- Revenue by state
- Revenue by segment
- Revenue by year and segment

You can use the Q&A tool to quickly create these graphs. Just convert them to standard visuals after entering the query and you’re all done. Do you see how fast it is to create these visuals with the AI NLP capabilities in Power BI compared to the traditional approach of manually choosing a visual in combination with mapping data and filtering by hand?

Arrange the visuals on the page in a way so that there is some space to the right for the narratives. Your page should look similar to Figure 4-13.

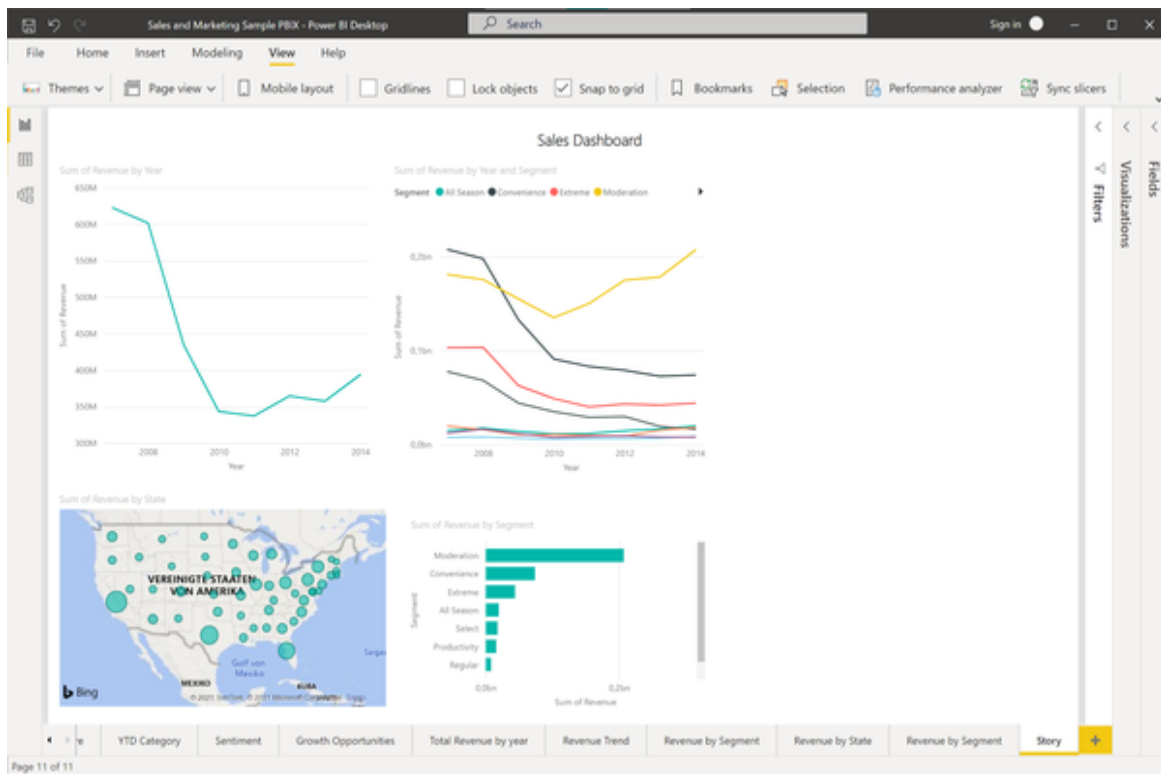


Figure 5-13. Sales Dashboard

Open the Visualizations pane. You should see the “Smart narrative” visual here as in Figure 4-14.

NOTE

If you don't see “Smart Narrative” in the visualization pane, you may be using an older version of Power BI and will need to turn on the smart narrative option. Go to File > Options and Settings > Options > Preview feature and make sure the Smart Narrative visual is turned on.

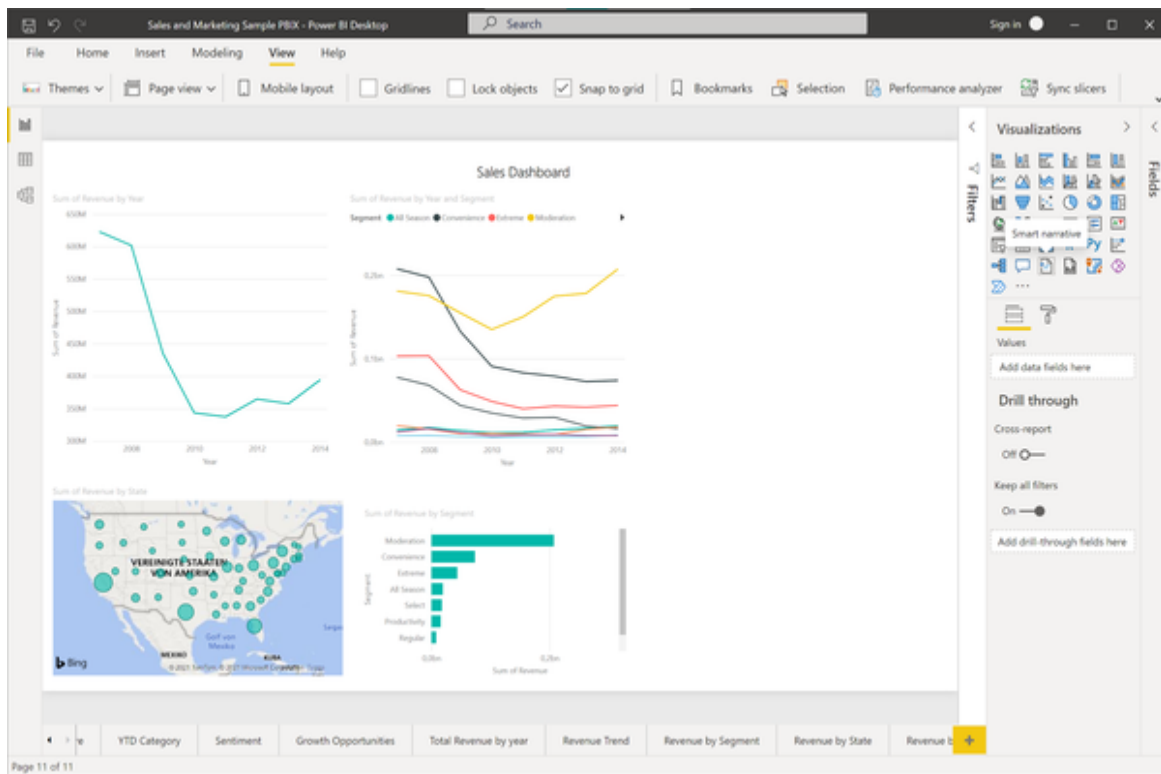


Figure 5-14. Smart narrative visualization

A double click on the Smart narrative visual should add a text box to the right side of your page. Alternatively, you can drag and drop the visual to a place on the page you like. This text box will contain the description that Power BI suggests for all visualisations on the page. In our case the description will look like the output in Figure 4-15. If you click on the text box you will see the highlighted values that Power BI calculated here.

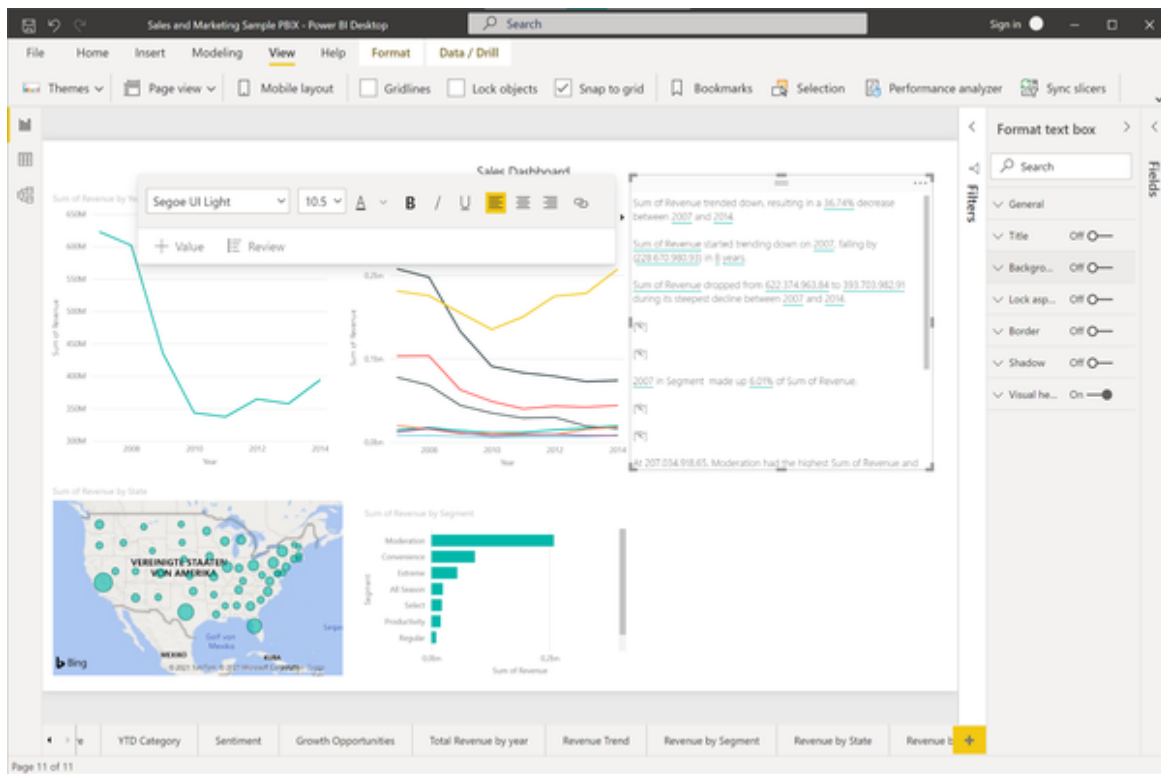


Figure 5-15. Smart narrative output

There are a couple of interesting things to note here.

On the first line, see how Power BI recognized that the revenue trended down and calculated the percentage of the beginning and the end of the revenue time series (see Figure 4-16). It also suggests that the product segment “Moderation” accounted for 52.59% of the total revenue in 2014. When you find that a statement is not relevant enough or even redundant, you can simply delete this line from the summary. You can also explore the calculated values in more detail and apply some formatting to them. For example, click the values that calculate the revenue figures. In the context menu, we can select that this value is displayed as a currency without any decimals after the comma.

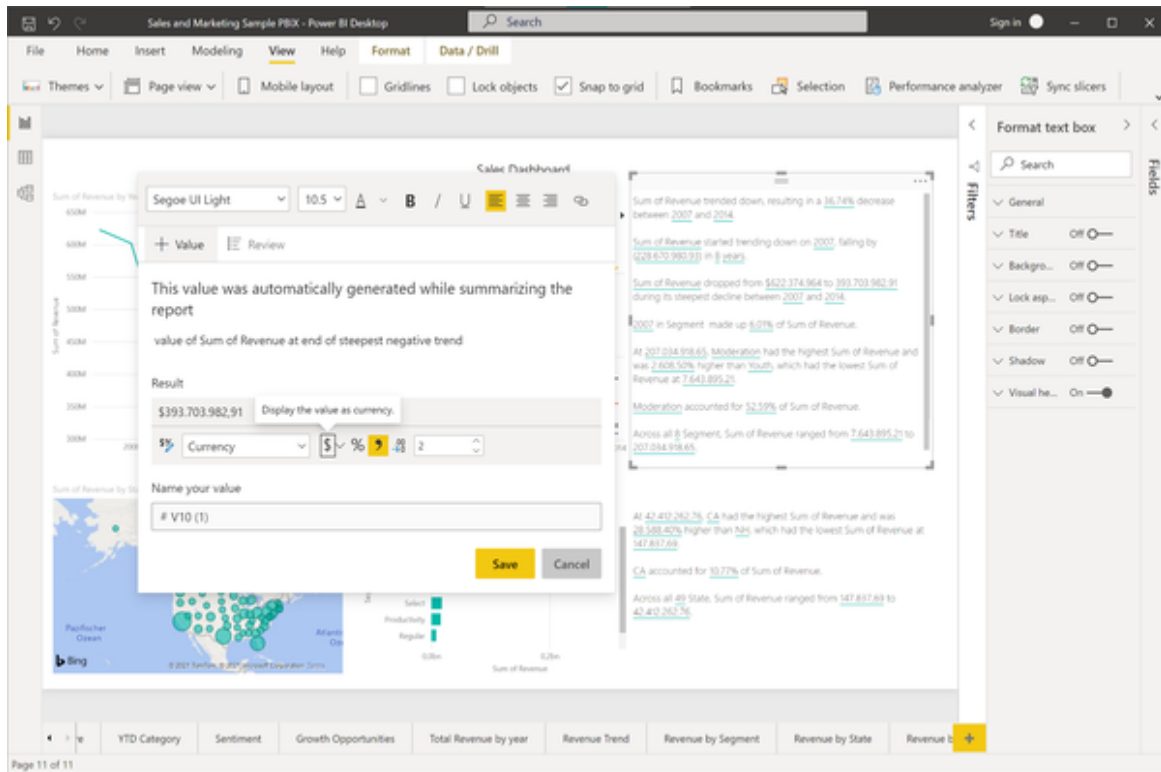


Figure 5-16. Smart narrative formatting

Formatting the automated text output will make it more accessible to the reader and overall look much nicer.

When we look at the summary, however, we realize that no information was given about the regional sales breakdown. Maybe they were not important enough to Power BI? To add them manually, right click on the map plot that shows the sales figures by state and click “Summarize” in the context menu. Power BI will add another text box that shows descriptions for just the plot which you selected. Again, you can delete or modify the descriptions as you like.

One important thing to note about the automated captions is that they are fully dynamic and respond to changes in the underlying graphs. To demonstrate this, let’s add a page filter to our dashboard to only show data from after 2010 as shown in figure 4-17. To do this, open up the Fields pane and then drag and drop the field “year” to the “Filters on this page” option pane. Once you apply the filter, see how the content of the text box changes.

Power BI recognizes that the revenue trend is now going upwards and updates the corresponding percentages accordingly.

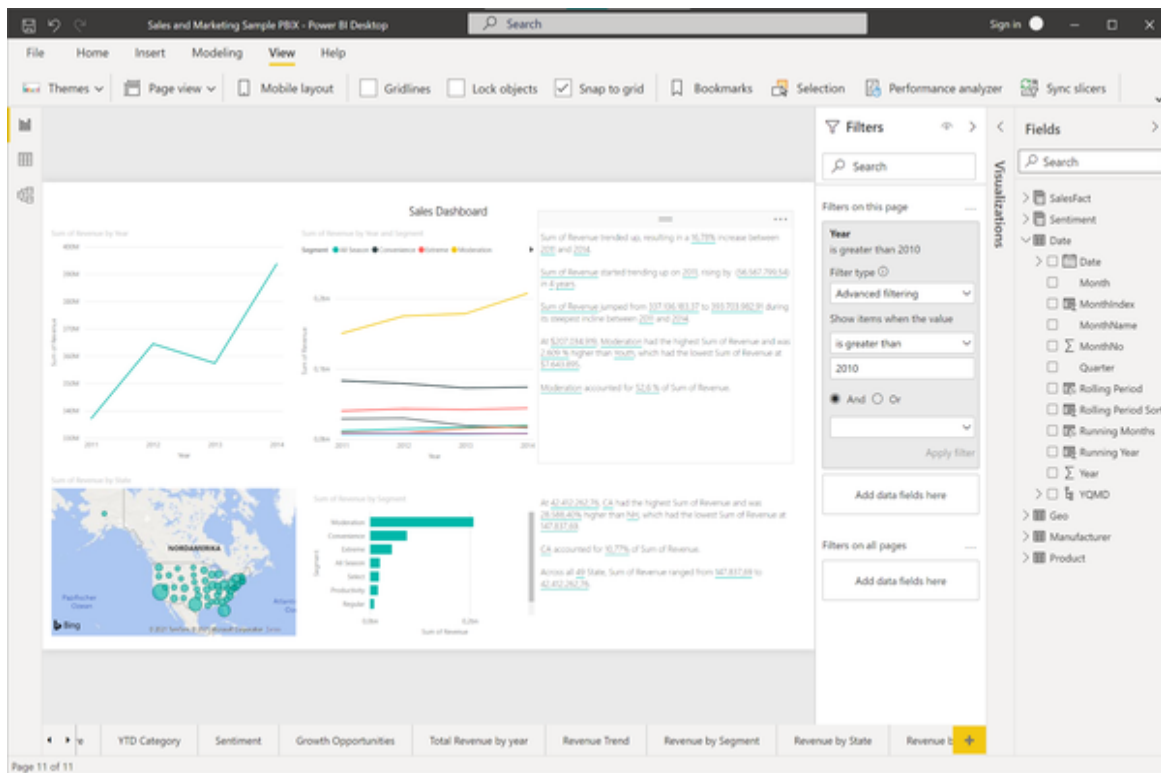


Figure 5-17. Smart narrative responds to global filtering and data changes

What is still missing is an explanation for one of the most important charts on this dashboard: The revenue breakdown by segment. You can clearly see that the segment “Moderation” is the only segment with a relevant positive trend—this is something that should be easy to recognize by the Smart narrative tool, isn’t it? Right click on the plot and click “Summarize” from the context menu. But what happens? Power BI just responds with a single line of text in the smart narrative box which is very hard to interpret.

We have hit the boundaries of the Smart narrative feature here. As of the time of this writing, it is not possible to summarize very complex charts with many different categories or underlying trends. Other limitations are that the Smart narrative tool cannot generate summaries of visuals whose columns are grouped by other columns and for visuals that are built on a data group field. Also, renaming dynamic values or editing automatically generated dynamic values is currently not supported and summaries cannot

be produced for visuals that contain on-the-fly calculations like complex measures or percentages.

Since the Smart narrative feature was initially released only in 2021 it will be exciting to see which of these limitations are going to disappear. You can find a full documentation on the Smart narration tool using this [Microsoft resource document](#)

For this time, let's just add the final conclusion for our dashboard by hand. Add a new line to the last text box and type: "Conclusion: Sales from product segment Moderation were the main reason for the positive revenue trend between 2011 and 2014." Your final sales dashboard should now look similar to figure 4-18.

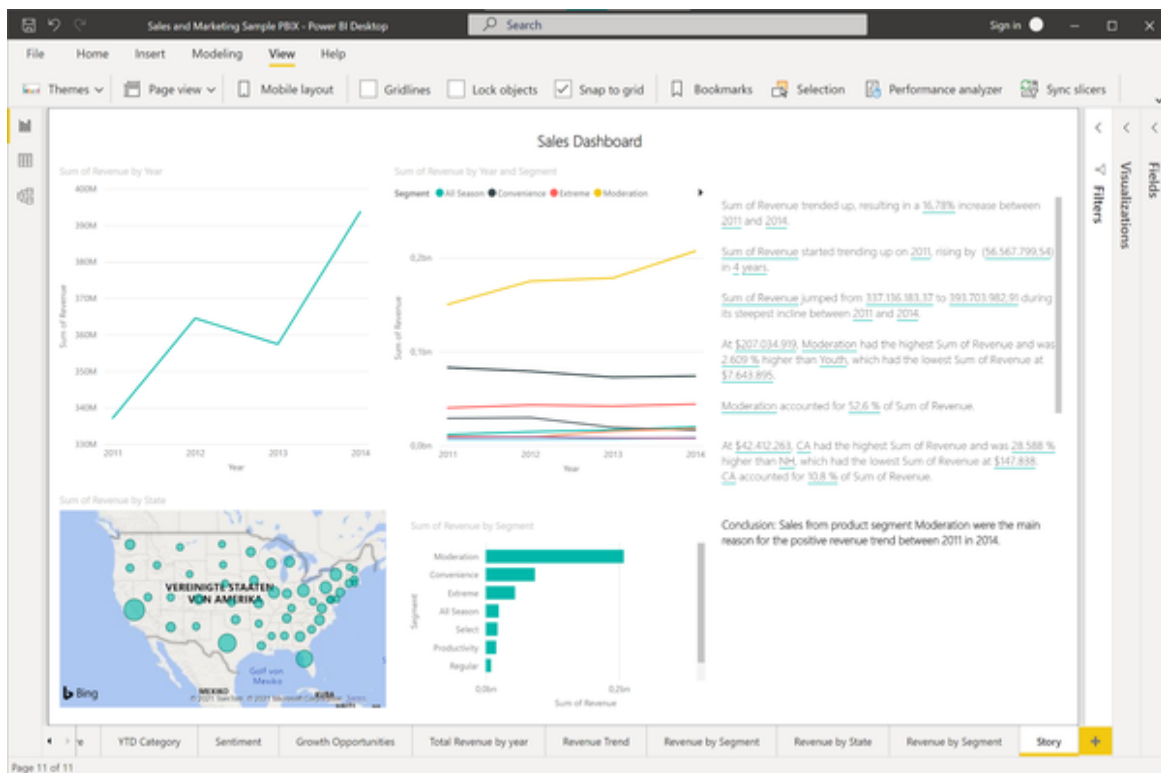


Figure 5-18. Final sales dashboard with annotations

You could easily export this page to a PDF document or publish it to a Power BI server.

While there are many more customizations and configurations for the Smart narration tool than this book could cover (for example, defining custom

values using the Q&A-style language queries), we will leave it like this.

If you like to find more about the Smart narration tool, check out this [video tutorial from Microsoft](#).

Summary

In this chapter you have learned how to use the AI-powered Natural Language capabilities in the form of the Q&A tool in Power BI to create visualisations faster and give your users a better BI experience. You have also learned how to automate the annotation process of your visualisations and add descriptions on the fly using the Smart narrative tool in Power BI. In the next chapter we will explore our data further and find out which factors affect the revenue trend in our data. We will use AI-powered tools to reduce our time to insights and comb through the data automatically for interesting patterns.

Chapter 6. AI-Powered Diagnostic Analytics

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 6th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

Finding out what happened is only half as interesting as finding out why it happened. Although data can never really tell us causal reasons (that’s left for logic), data can certainly point us to those areas where we humans can put our brains to work. In this chapter you will learn how AI can help you to surface those interesting patterns in data automatically so you and your colleagues can concentrate on the interpretation and impacts of this data.

Use Case 3: Automated Insights

We continue with the case study from the previous chapter: In a fictional manufacturing company we support sales management in their decision making process. In the last chapter we found out that sales slowly recovered after a period of strong declines. Now, we will dig deeper and concentrate on understanding why certain trends evolved.

Problem Statement

As the business analyst on the team we want to help sales management to find explanations for two of the observed revenue trends: Why did the sales figures melt so dramatically between 2006 and 2010 and which factors explain the slow recovery between 2010 and 2014?

This process would normally involve analysts who go through the data manually which takes time and is subject to the analyst's availability. Since we don't have the capacity to do one-on-one sessions with each of the business users, we are looking for ways to provide them with insights in a more automated and interactive way.

Solution Overview

Our goal is to find patterns in our dataset automatically by scanning through all available information with at little human assistance as possible. For this purpose, we are leveraging AI-powered techniques to deliver these insights in a fast and interactive manner. This will allow business users to get insights with less or even without the help of professional data analysts. To deliver this experience we are again using two Power BI tools here: The Power BI Key influencers and the Power BI Decomposition Tree tool.

The Key influencers visual aids in comprehending the aspects that influence a metric of interest by showing the top contributors to the selected metric. It examines the information you provide and surfaces the most important aspects to the top by labelling them as key influencers.

Power BI's decomposition tree visual allows you to visualize data in numerous dimensions. It aggregates data automatically and allows you to drill down into any dimension in any order. AI will suggest the next most relevant dimension to dive into depending on your particular area of interest. As a result, it's a useful tool for ad hoc investigation and root cause analysis.

We are going to apply these techniques to the Sales and Marketing example Power BI report to find out: Which factors were driving both revenue trends

and which patterns can we observe from our data?

In a corporate scenario, both tools could be provided to the business users in the form of Power BI report pages so they can interact with the data on their own.

Power BI Walkthrough

Open the **Sales & Marketing sample PBIX file** which we used already in the previous chapter. To recap real quick, this dataset contains various sales and marketing data from a manufacturing company and comes pre-populated with some reports for market share, product volume, sales figures and sentiment scores.

In Chapter 2 we found out that the overall revenue in the last four years has slowly recovered after a hard bounce in the previous year. Here is a snapshot of the sales dashboard we created before:

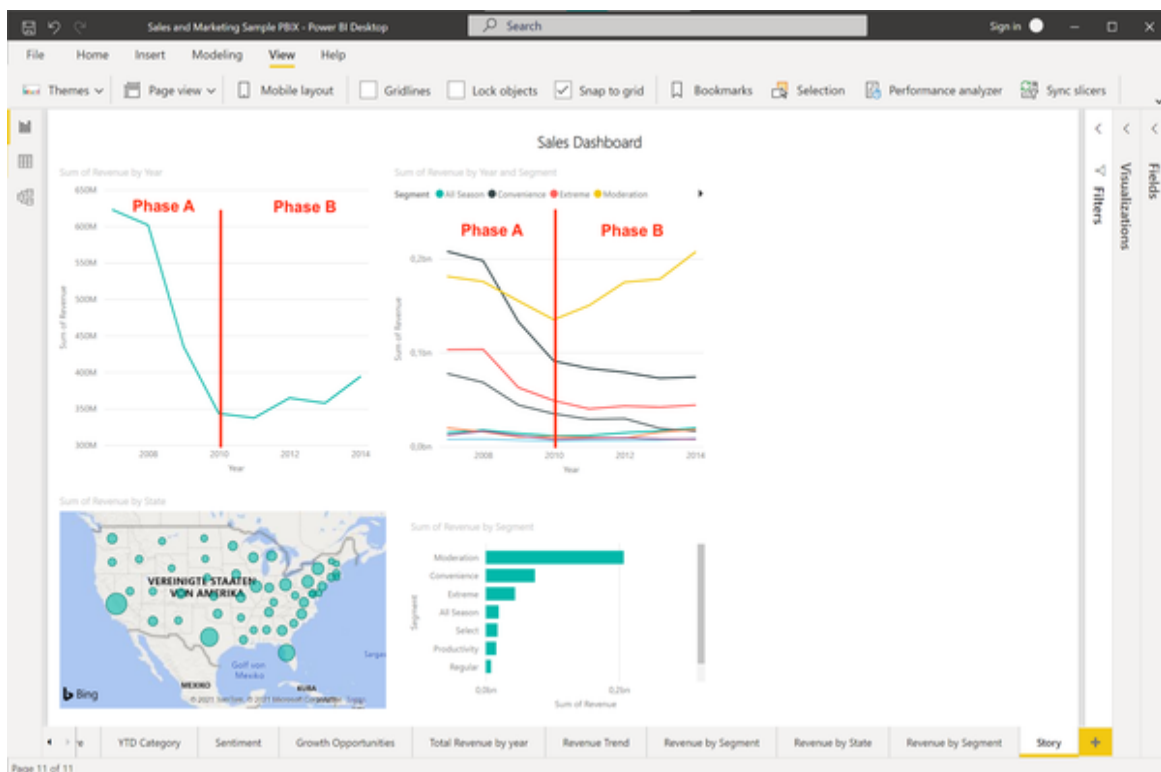


Figure 6-1. Two phases of revenue trends

We are especially interested in what happened during the phase of declining revenues between 2006 and 2010 (“Phase A”) and the phase of slowly recovering revenues between 2010 and 2014 (“Phase B”). For both of these phases we want to answer the simple but profound question: What happened?

We will start by examining Phase A. To recreate the revenue visual, create a new Power BI report page, drag and drop the Q&A visual and type:

“Revenue by year between 2006 and 2010”

NOTE

If you didn’t do the use case from the previous chapter you might need to replace Revenue with “sum of revenues”.

Click on the convert icon to turn the Q&A visual into a standard Power BI visual. The chart should look similar to this:

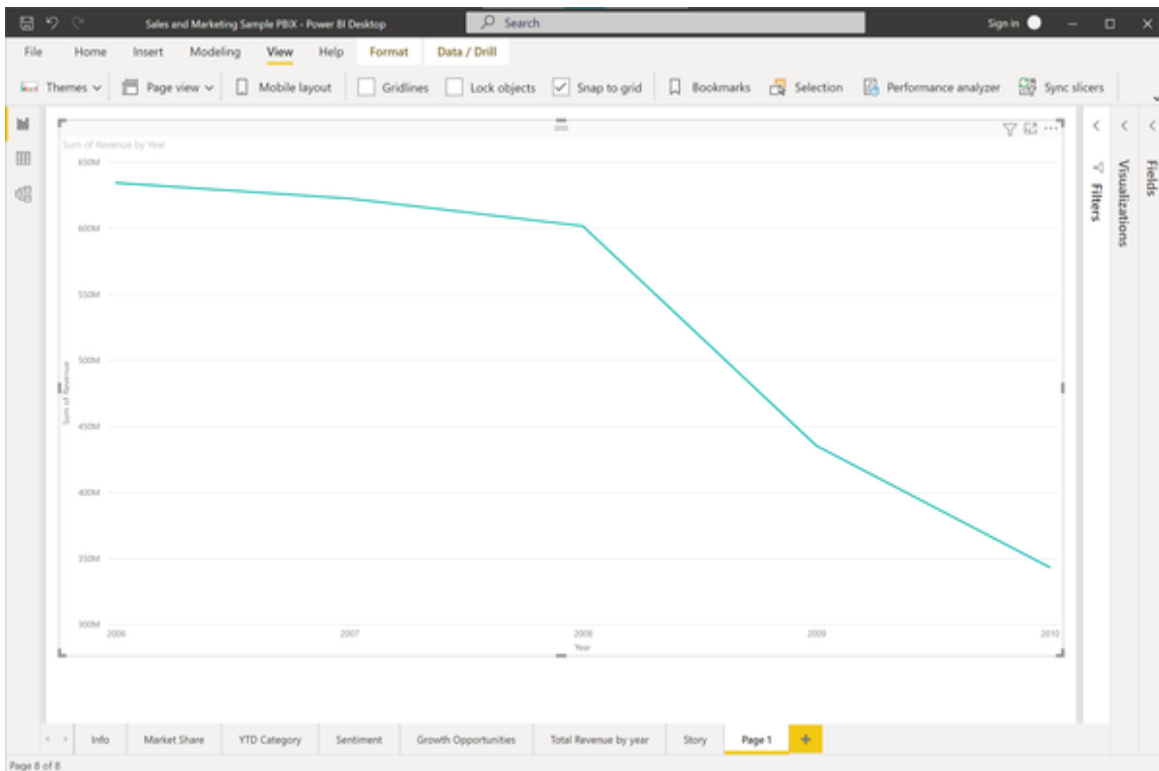


Figure 6-2. Revenue by year between 2006 and 2010

Select this chart and open the visualizations pane on the right. Click the “Key influencers” icon to convert the line chart into the Key influencers visual.

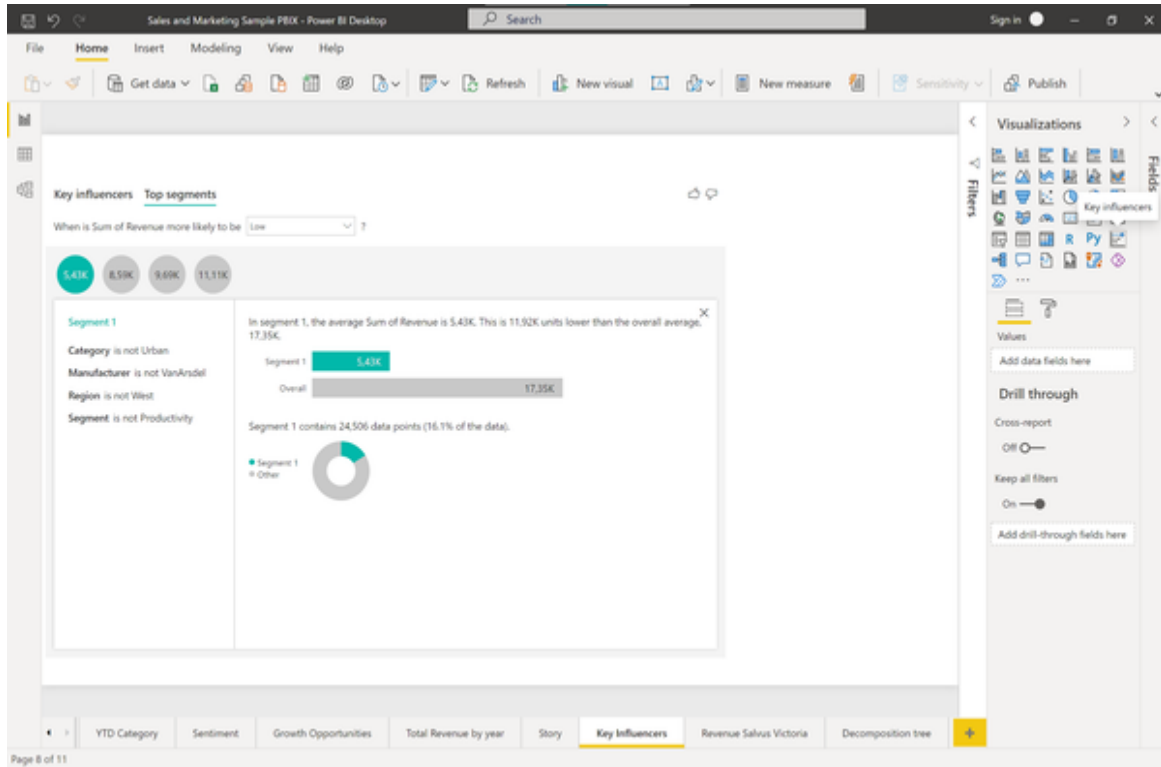


Figure 6-3. Key influencers visual icon

Once you select the new visual type, the chart will automatically update to something which should look similar to figure 5-4. Let’s explore the output of this visual in a bit more detail.

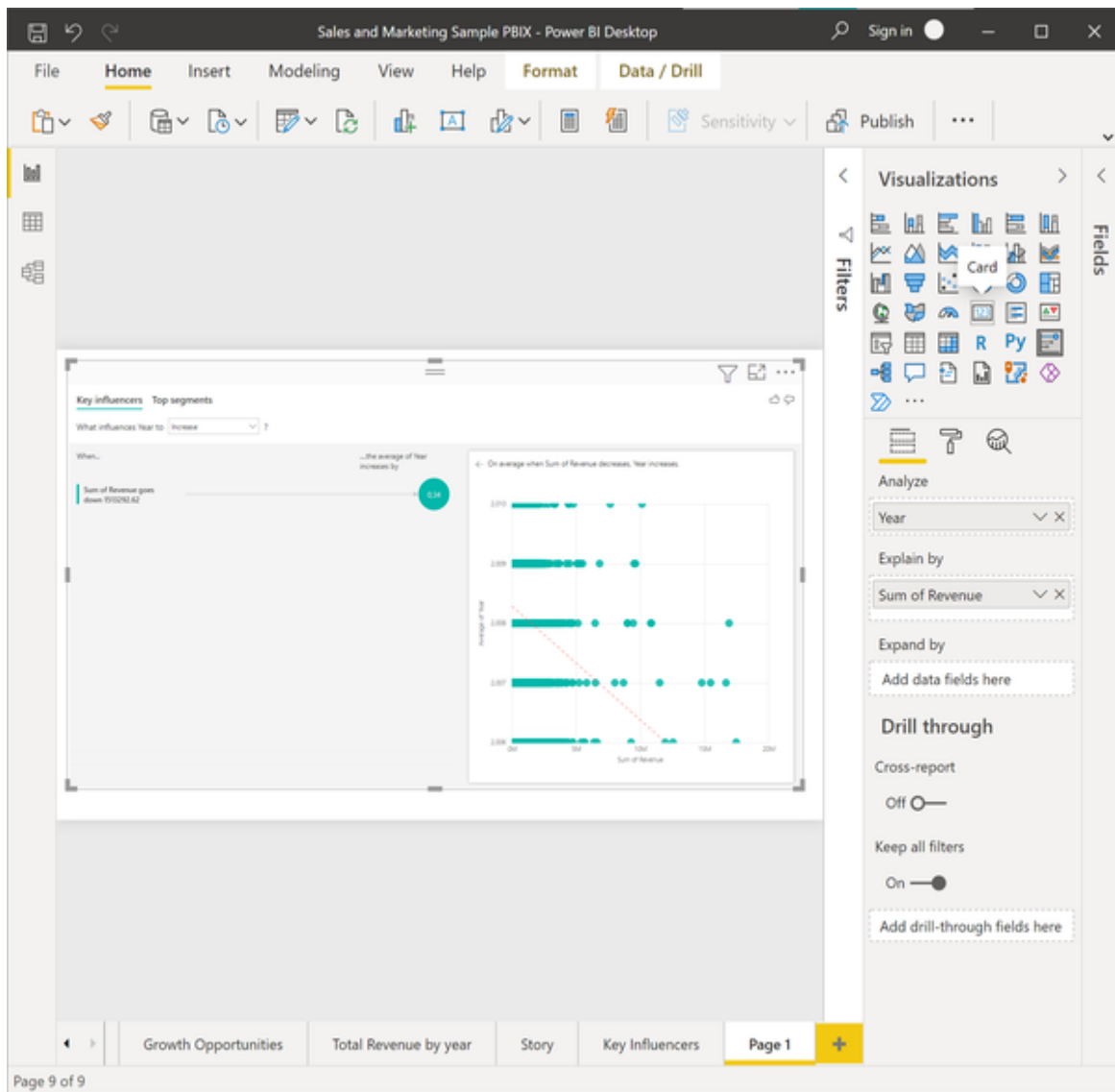


Figure 6-4. Key influencer standard output

Now, this output is probably not what you expected at all and definitely looks a bit weird. What went wrong? Open the visualisations pane and inspect the properties of the Key influencers visual. This should still look like shown in figure 5-5.

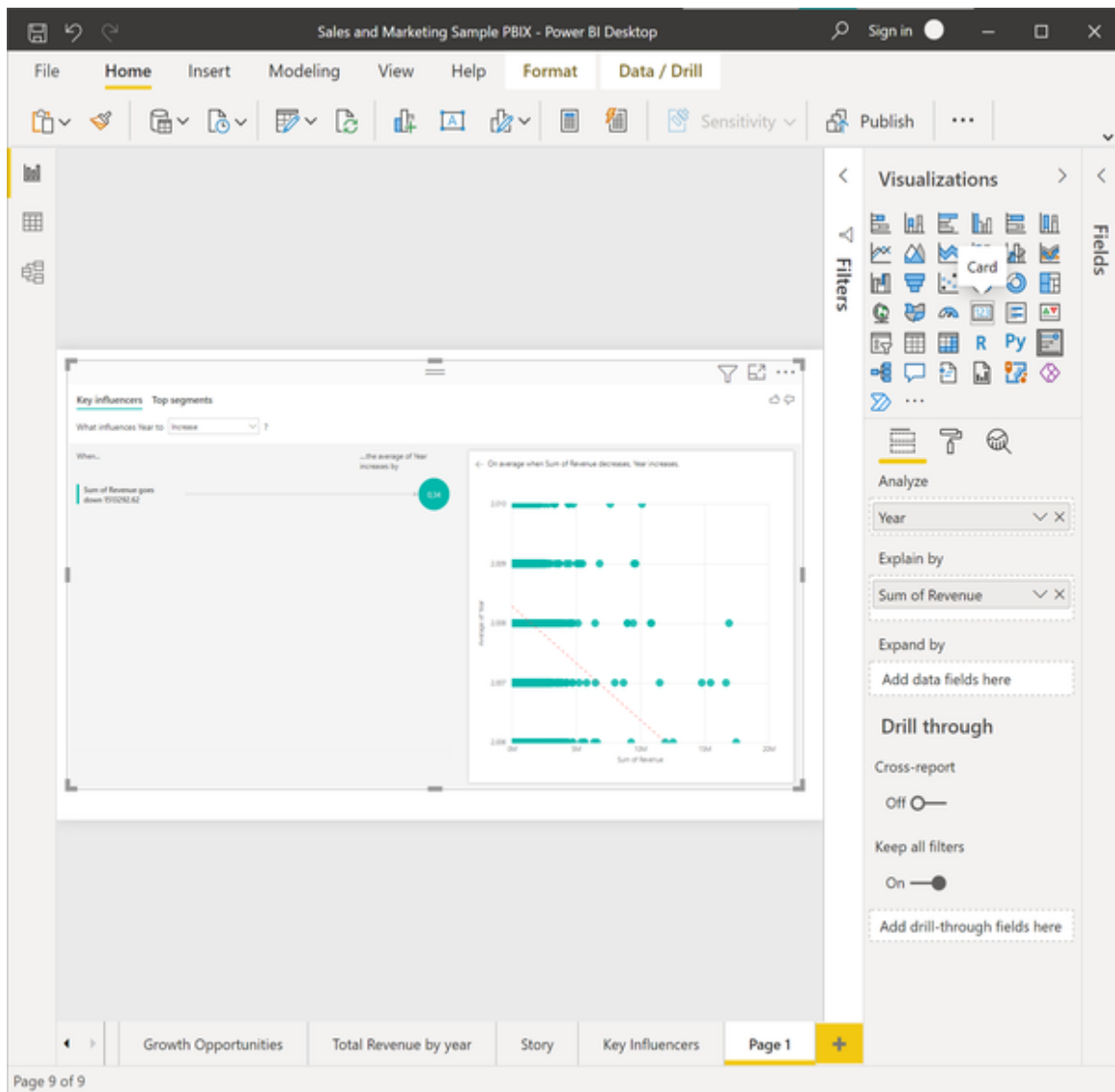


Figure 6-5. Key influencer properties after conversion from line chart

In the visual properties you will find two critical fields: “Analyze” and “Explain by”. As the names suggest, “Analyze” refers to the metric which you want to explore and “Explain by” refers to the various dimensions you might consider to have an effect on it. Per default, given our previous line chart, Power BI suggests to explain the dimension Year by the field Sum of Revenue which absolutely does not make any sense.

Let’s fix this by dragging “Sum of Revenue” from the “Explain by” field to the “Analyze” field instead of “Year”. Consequently, add additional dimensions into the “Explain by” field by pulling them from the data fields

repository on the right. Reasonable candidate fields from a business point of view might be: Segment, City, State, Region, Average of Score, Category and Manufacturer. We will leave the dimension Product out since too many data points are missing. For example, some products were replaced by others, some products were just launched, some products were deprecated. If we wanted to analyse Key influencers down to a product level we should rather look at a single year rather than a four year period.

The updated properties of the Key influencers tool should now look like in figure 5-6:

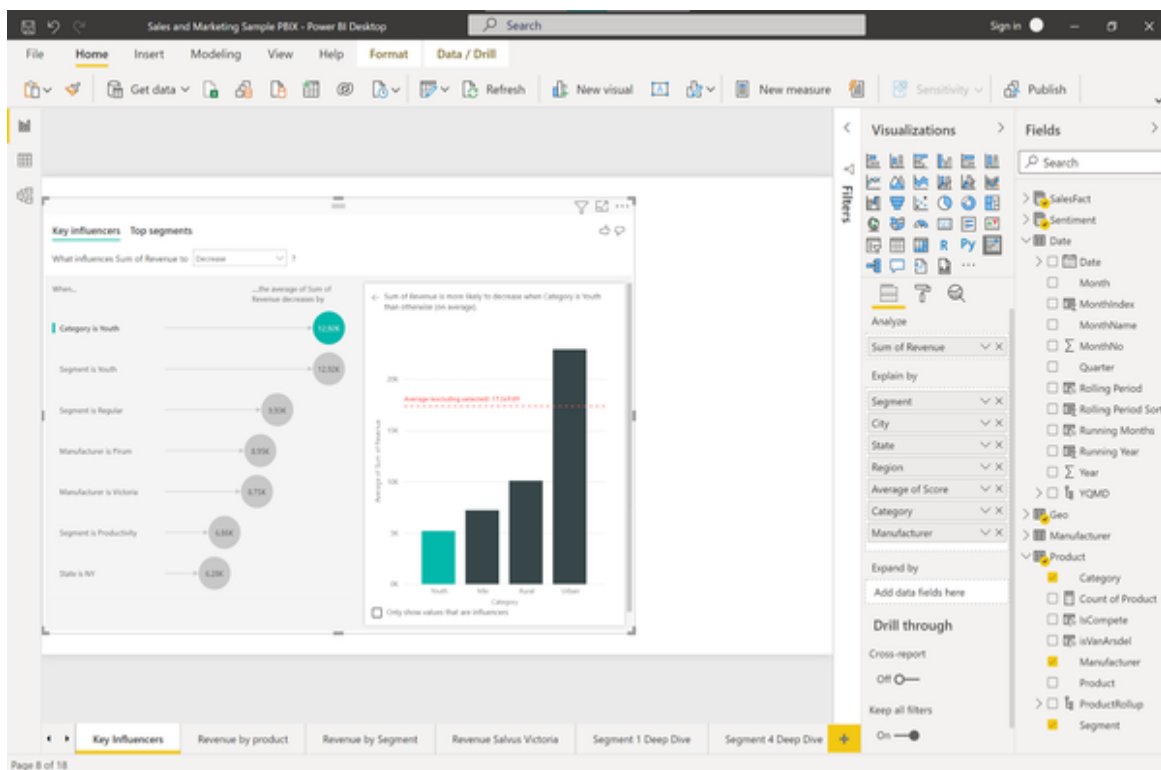


Figure 6-6. Key influencers properties (edited)

Within the Key influencers visual, switch the drop down menu on the top from Increase to Decrease, since we are interested in finding out what made the revenue drop. The updated

visual should by now look similar to the one in figure 5-7.

This one intuitively makes more sense than the version before. Note that this would be the report page you could share or publish for business users

so they can interact with the data on their own.

Let's dive into this visual to see how it works in a bit more detail.

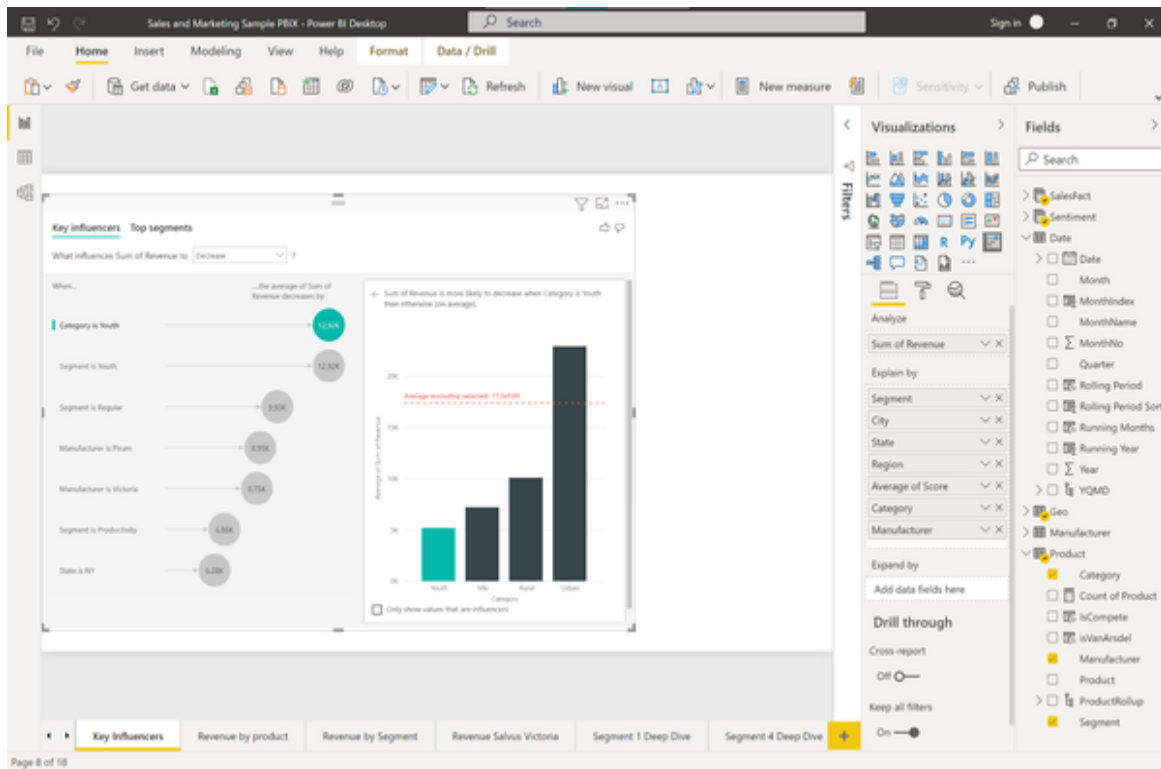


Figure 6-7. Key Influencer tool edited

On the left side of the visual, we can see which variables Power BI identified as key influencers for the given metric, in this case revenue. Can see that Power BI identified the attribute “Youth” across the dimensions Category and Segment as the main key drivers that cause the Revenue metric to decrease. On the right side, we are presented with the underlying data distribution for the selected influencer, showing the average of the revenue metric across all categories from the selected dimension, in the example in Figure 5-7 the Category dimension. You can read the chart on the right as follows: The average revenue for the Category Youth was just \$5,181.98 while the average revenue for all other Categories except Youth was \$17,349,89. That means, when the Category is Youth the average revenue is \$12.92K lower compared to all other values of the Category.

It is important to point out that in this case we are not judging based on the absolute values of a variable but on their averages and number of

observations. Depending on the measure you choose to analyse, whether it is categorical or continuous, the key influencer tool will adapt. Instead of averages it will indicate probabilities such as that the outcome is x-times more likely if a certain value is met.

The Key influencers tool does not always consider the lowest value of a category as an influencer. Take for example the influencer “Manufacturer is Victoria” in our analysis. If you click on this influencer you will see the chart in figure 5-8. We can see that Victoria is not the manufacturer with the smallest revenue on average, but Salvus. Now why has Salvus not been identified as a Key influencer?

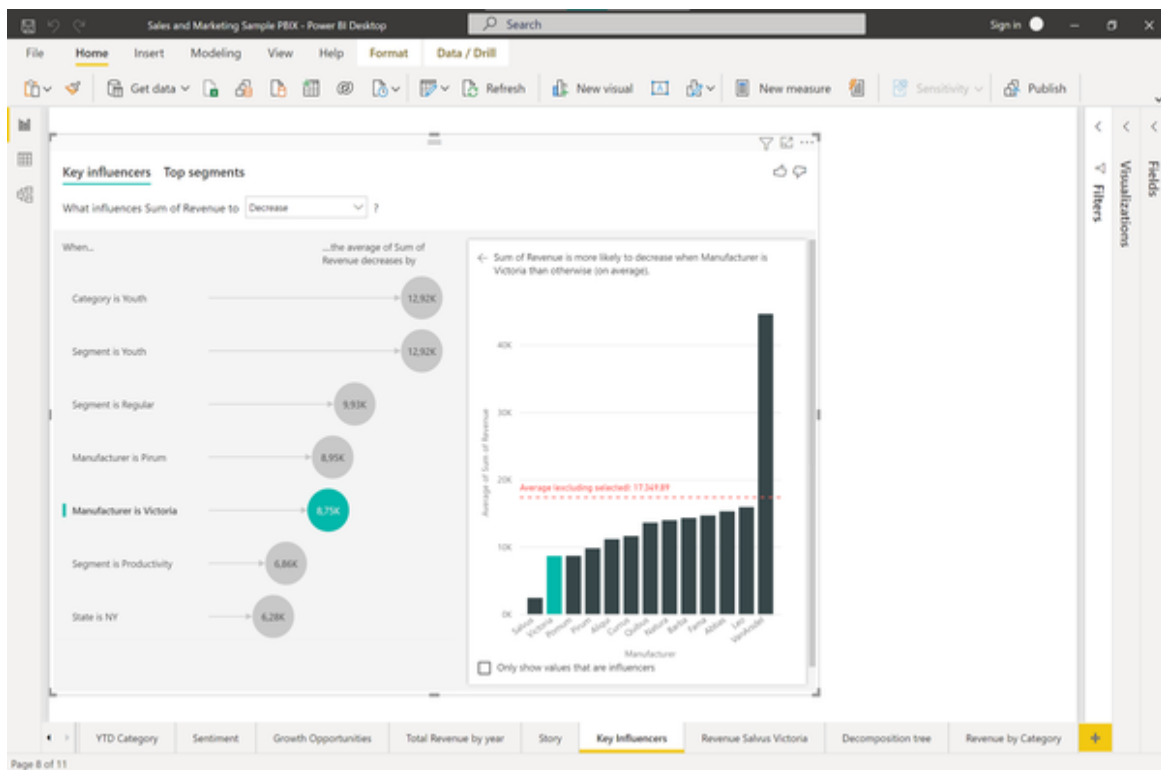


Figure 6-8. Key Influencer visual for Phase A (Negative revenue trend)

The reasons becomes clear if we look at the head to head comparison of Salvus and Victoria in figure 5-9, just for the sake of fostering our understanding how the Key Influencer tool works:

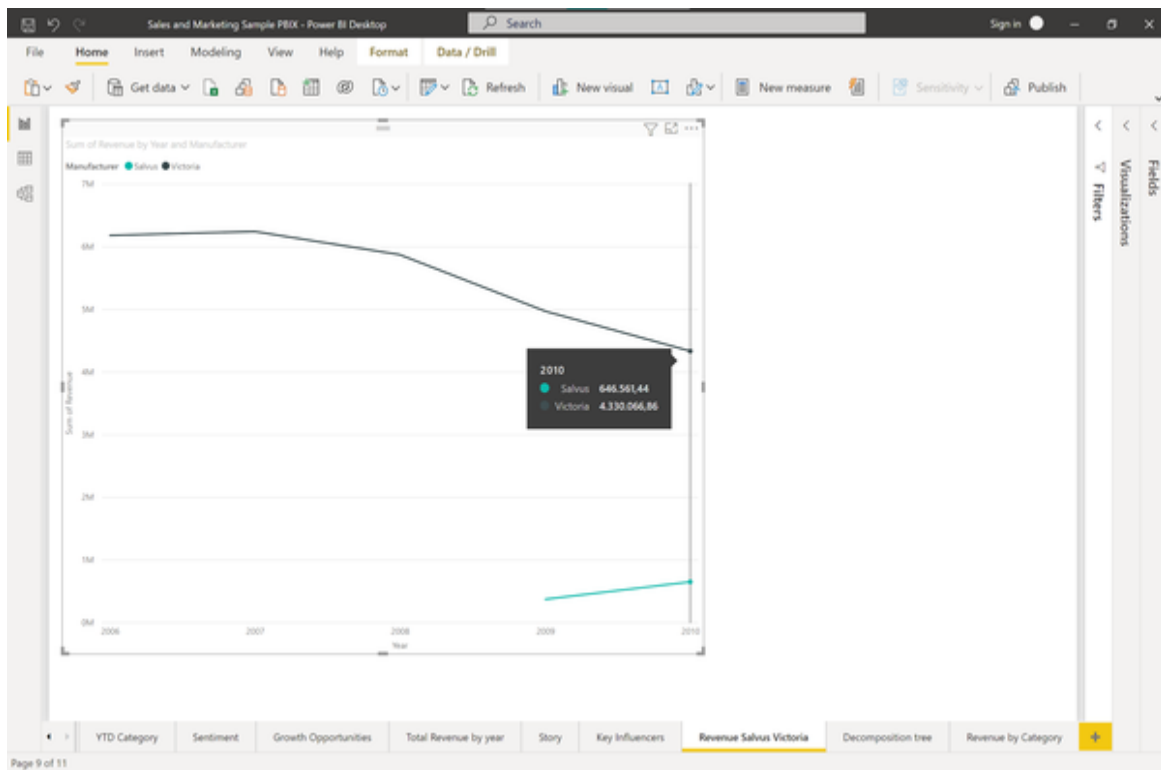


Figure 6-9. Comparing sales of manufacturers Salvus vs. Victoria

There are three factors which disqualify Salvus to be a key influencer: First, this manufacturer was only introduced in 2009, so there are fewer data points in total compared to the other manufacturers. Second, the absolute revenue contribution of Salvus is so little that it probably lacks overall impact. And last but not least, the actual revenue trend of this manufacturer is not negative, but positive. While Victoria dropped from 6.2M revenue in 2006 to 4.3M revenue in 2010, Salvus actually improved their revenues between 2009 and 2010. With this background information it is legit to not consider Salvus a Key influencer for overall decreasing revenues. The key influencer tool makes it easier for us to find such interesting patterns without combing through all of the data manually.

Let's explore another useful feature of the influencer tool: The segments. Segments can be considered as groups in your data which show a high impact on a metric of interest. Switch over to Top Segments and select "When is Sum of Revenue more likely to be Low" from the dropdown at the top. You should see the following screen in Figure 5-10 as a result:

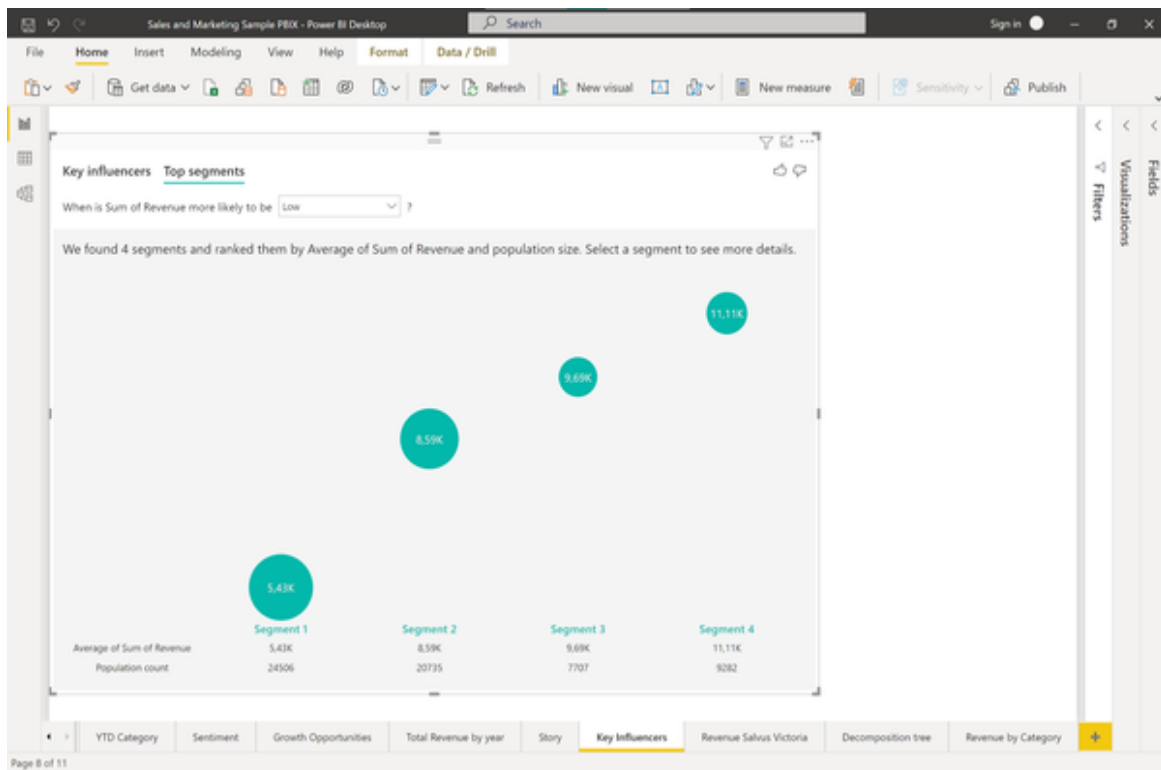


Figure 6-10. Analysing segments that contributed to negative revenue trend

We can see that there are four segments which Power BI identified for us. The biggest segment, indicated by the size of the bubble, holds 24,506 observations (i.e. sold products) in it and accounts for an average revenue of just 5,4K per sale. The low performer segment with the highest (yet below total) average revenue of 11,11K contains 9,282 observations. To find out more about what is happening here, click on the 5,43K bubble.

You will see a detailed view of this segments which in this case looks like this in Figure 5-11:

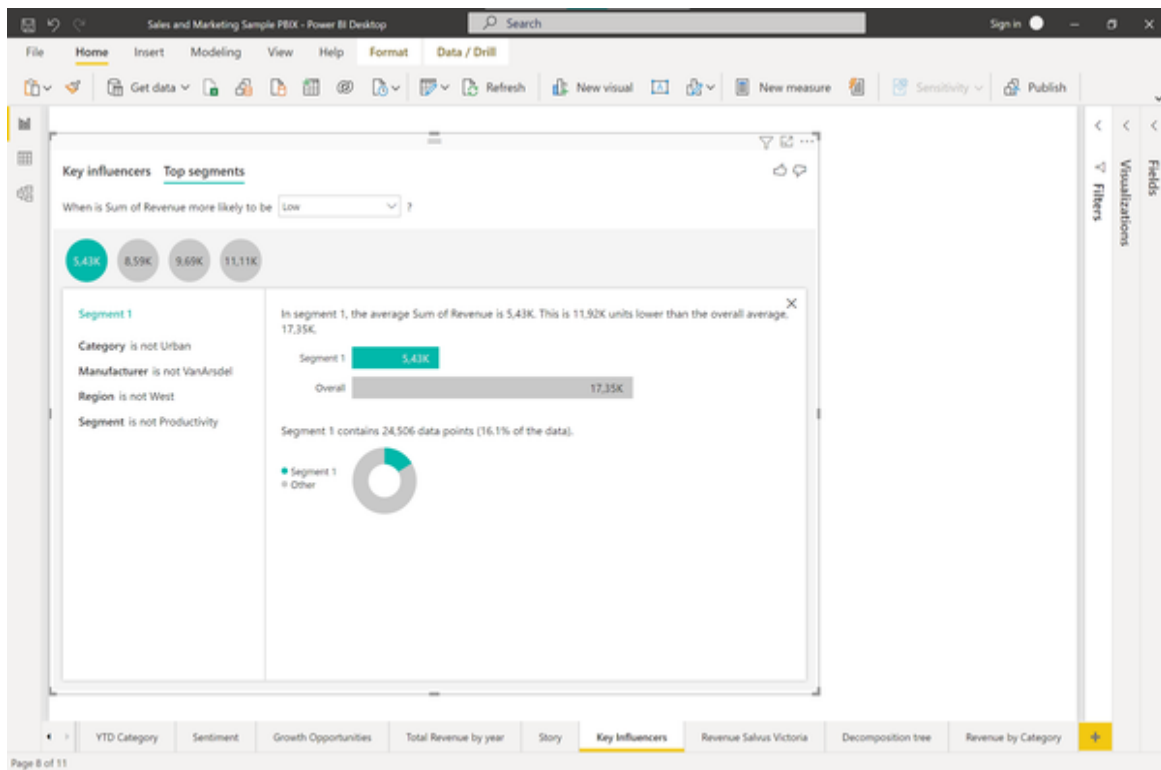


Figure 6-11. Segment 1 details

From this chart we can see that this segment contains almost 16% of all product sales and therefore has a very high relevance for us. The segment contains all sales where the category was not Urban, the manufacturer was not VanArsdel, the region was not West and the Segment was not Productivity. This segment contributed the largest group of data points and had the lowest average revenue which makes it an important segment to look at when it comes to explaining the falling revenues trend in Phase A (2006 - 2010).

To see what this means in detail, let's examine the revenue trend in this segment 1 vs. the total revenue trend between 2006 and 2010 as shown in Figure 5-12.

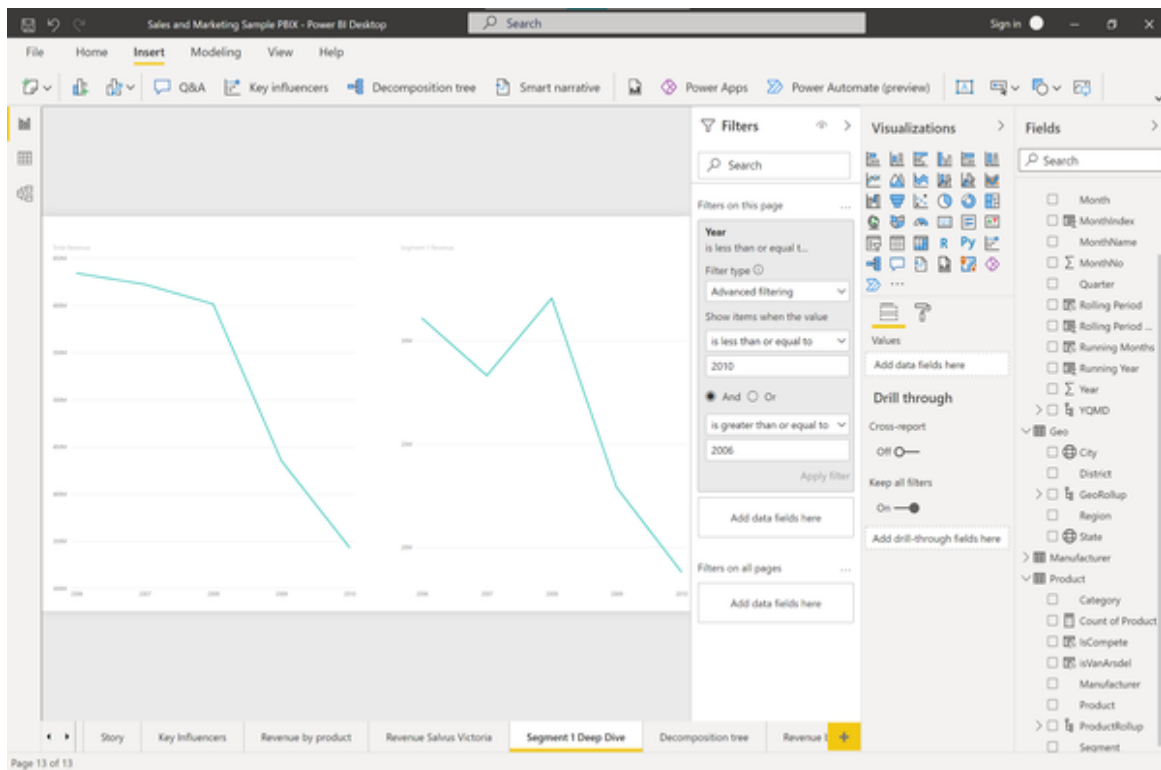


Figure 6-12. Overall revenue trend between total revenue and segment 1 revenue

While the overall revenue was down 46.6% from 643.28M to 343.13M in the total dataset, segment 1 alone contributed a loss of 12.3M from 31.10M in 2006 to 18.8M in 2010. These more focused segments should be easier to tackle for business users and identify potential problem areas as opposed to larger trends such as the Category “Youth”.

We could explore the other segments in a similar way, but let’s conclude our analysis for the past revenue drop for now and head over to finding out what happened during the recovering period between 2010 and 2014, called Phase B.

If you are interested in further features and descriptions on how the Key influencers tool works you might check out [this comprehensive resource from Microsoft](#).

For the analysis of Phase B we will switch over to the decomposition tree feature in Power BI.

The decomposition tree is a great tool to conduct a root cause analysis. You can use it as a smart drilldown where Power BI suggests the next drill down

level automatically based on a specific metric you want to explore.

For this purpose, start with a blank report page in your Power BI file. Recreate the revenue by year chart, this time for the period between 2010 and 2014. Convert this chart into a static visual if you used the Q&A tool for this. The revenue chart should now look similar to Figure 5-13:

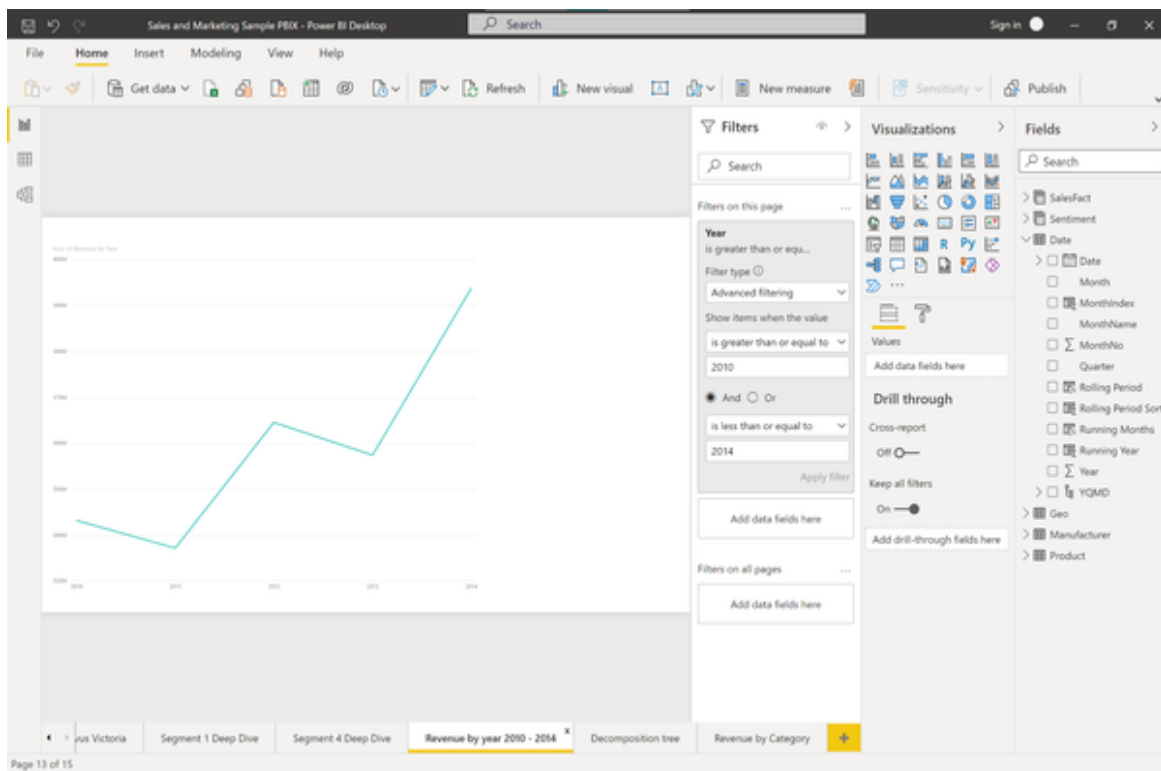


Figure 6-13. Total revenue between 2010 and 2014

Select the line chart and change the visualisation type to “Decomposition Tree” as shown in Figure 5-14:

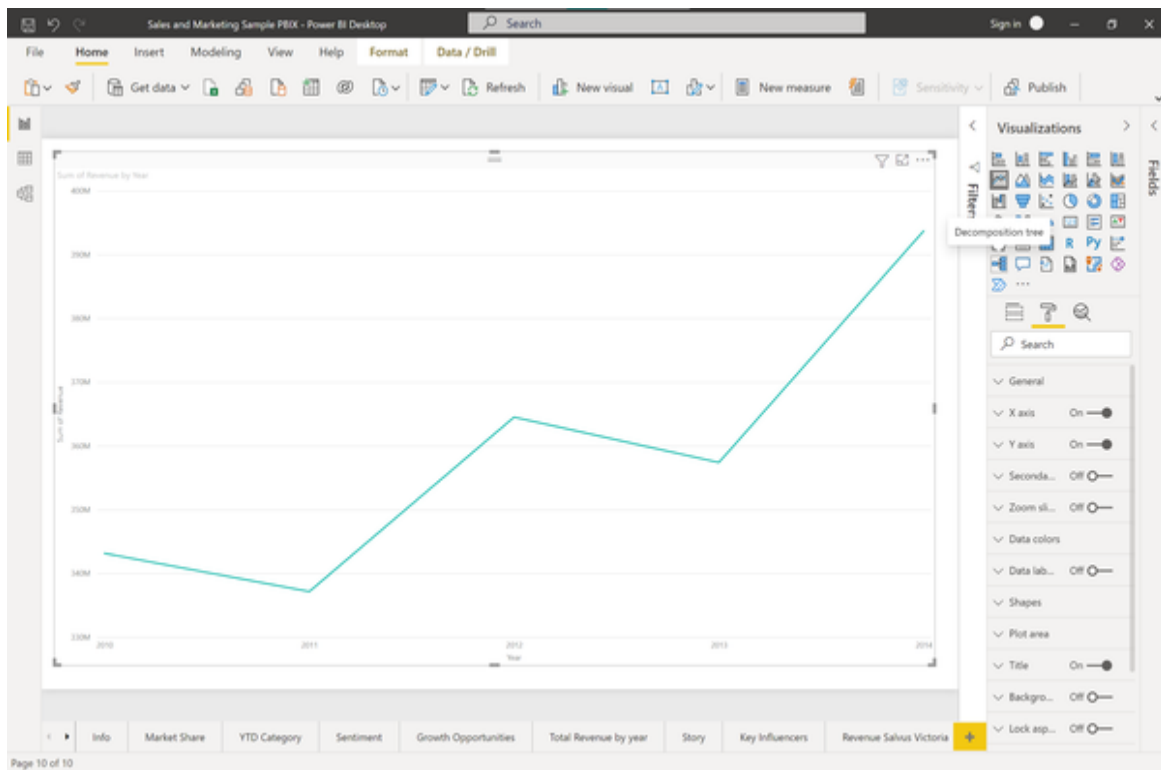


Figure 6-14. Decomposition tree visual

Similar to the influencer tool, we have to tell Power BI which variable we want to analyse and which dimensions we consider as explanatory factors. Modify the visual properties so that they are looking like the ones in Figure 5-15.

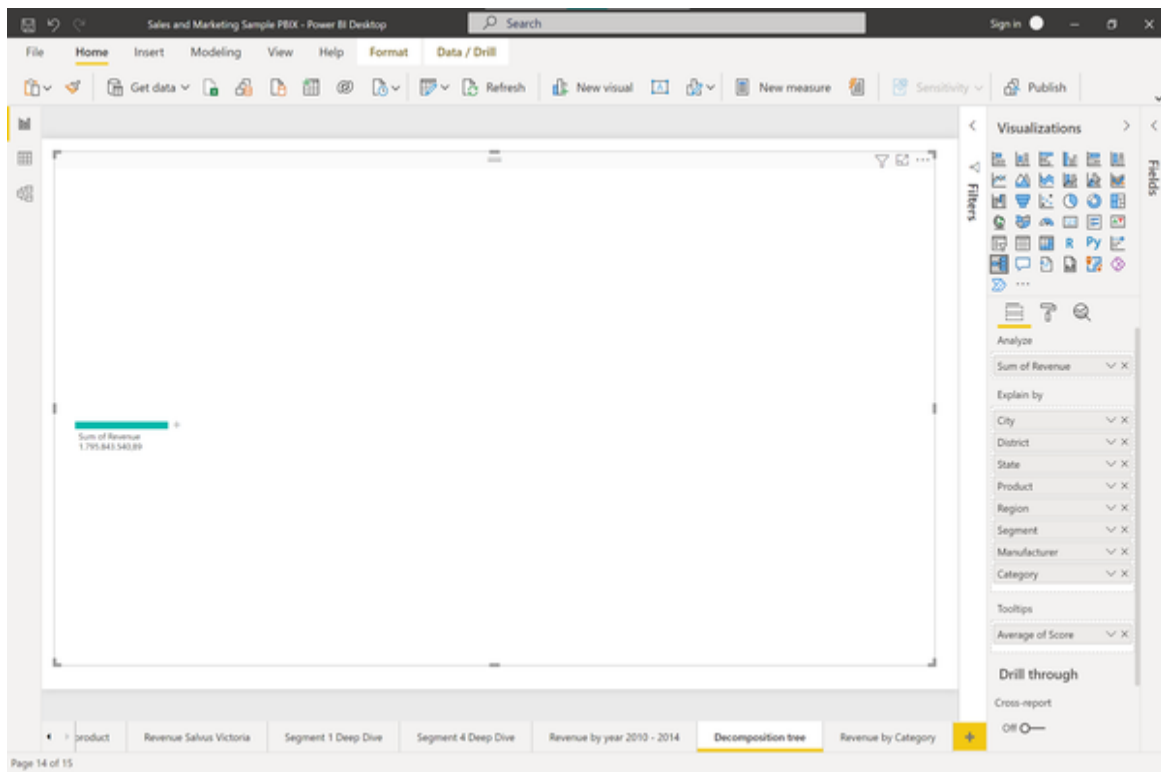


Figure 6-15. Decomposition Tree properties

The initial output of the decomposition tree visual will just be a plain summary of our metric of interest on a blank canvas. You can see this output in figure 5-16.

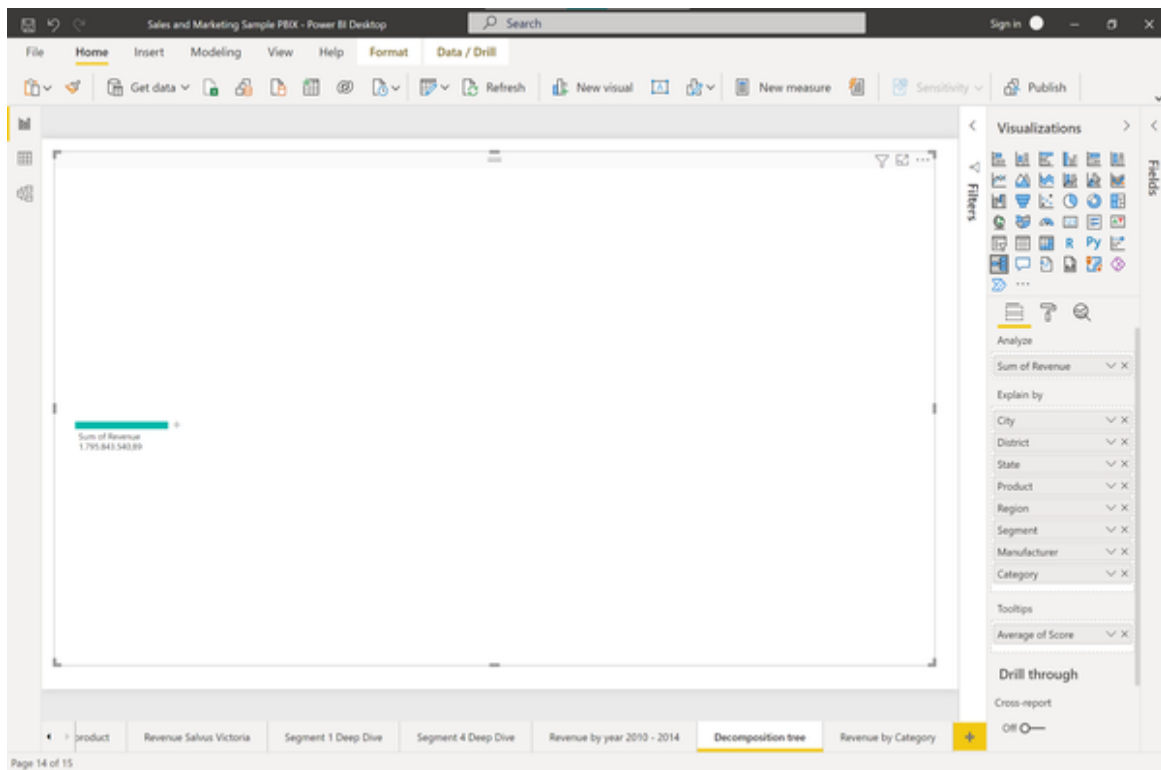


Figure 6-16. Decomposition Tree start

The task at hand is now to decompose this aggregated metric into smaller chunks, ideally splitting it in such a way that we capture the most important metrics at the top of the tree.

When you click the small “plus” icon on the right next to the aggregated revenue, you will see a menu popping up where you can select the next split you want to make. You could either select the split manually, for example based on some business logic or your own preferences, or you can use a feature called “AI splits”. We will come back to this concept later to find out in which cases it might be reasonable to select drill down levels manually. AI splits are indicated by small light bulbs and will automatically choose the next best drill down level for you, depending if you want to influence your top-level metric to be higher or smaller, as you can see in Figure 5-17.

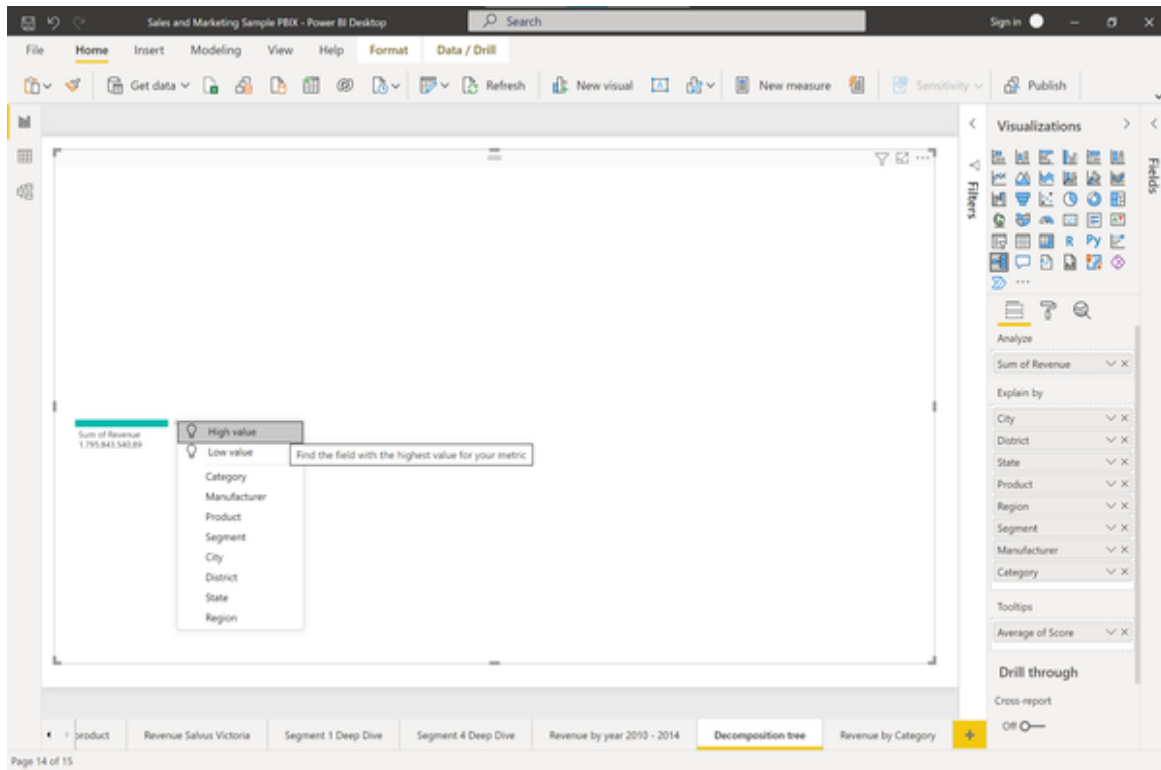


Figure 6-17. Using AI splits in the decomposition tree

Since we are interested in explaining the revenue growth in Phase B, let's choose an AI split for high values. You will see that Power BI creates the first split for you on the criteria "Category" as shown in Figure 5-18.

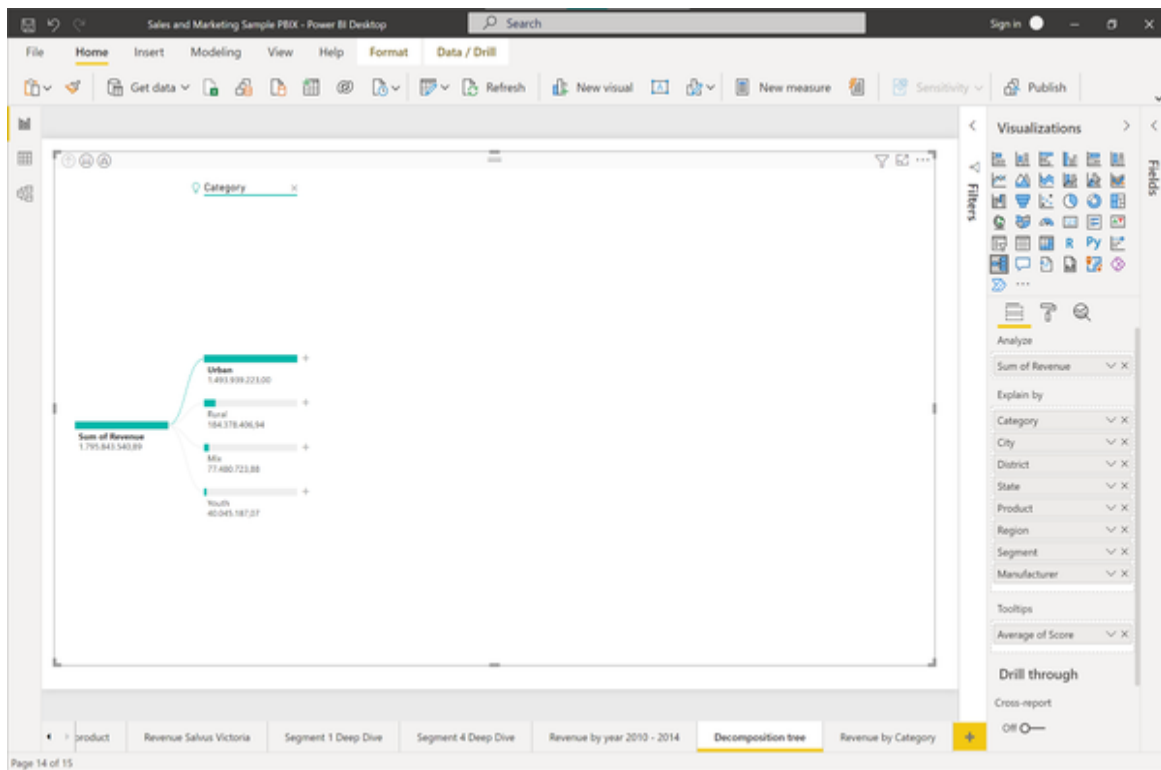


Figure 6-18. First Level Split in the decomposition tree

If we hover over the light bulb next to the Category split, you will see a Power BI tooltip explaining the current node of the tree. In this case, this first split tells us that the revenue between 2010 and 2014 was highest when the Category was Urban (see Figure 5-19).

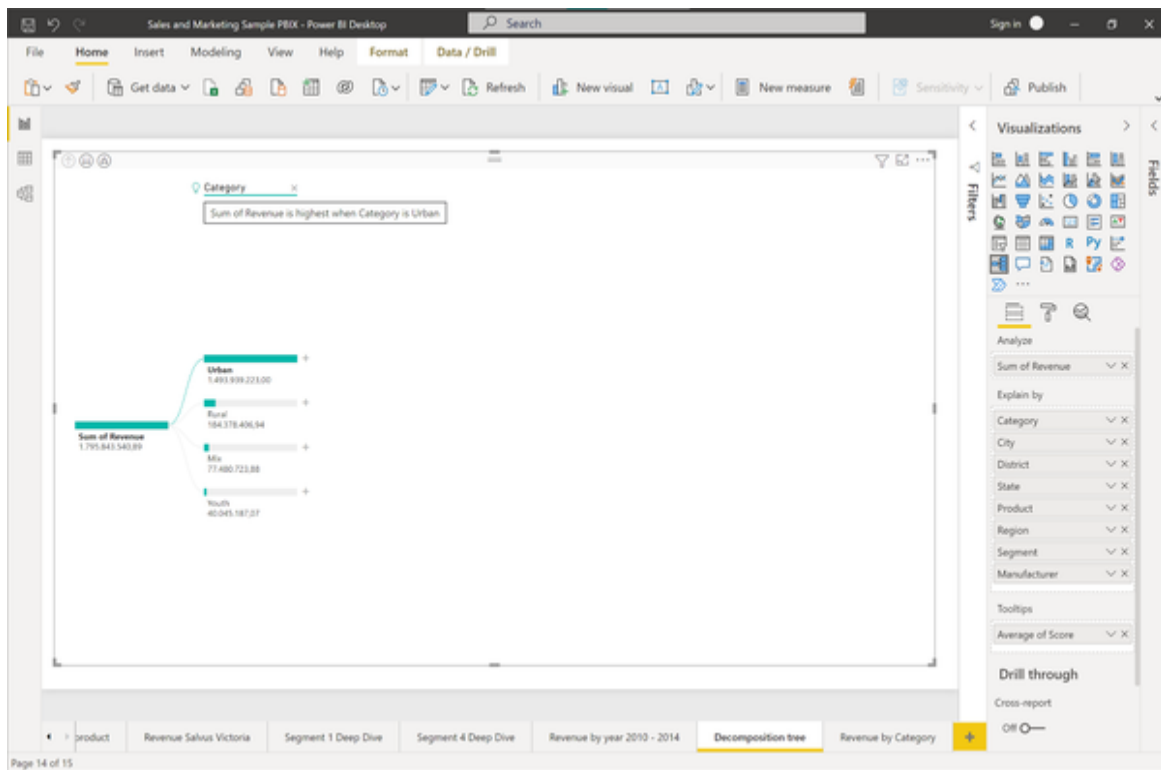


Figure 6-19. AI Split Explanation

We can now drill down further as we like, for example by finding out which criteria led to the revenue growth within the Urban category. Again, we could select the next drill-down level manually or let Power BI figure it out for us, depending on whether we're interested in High or Low Values (See Figure 5-20)

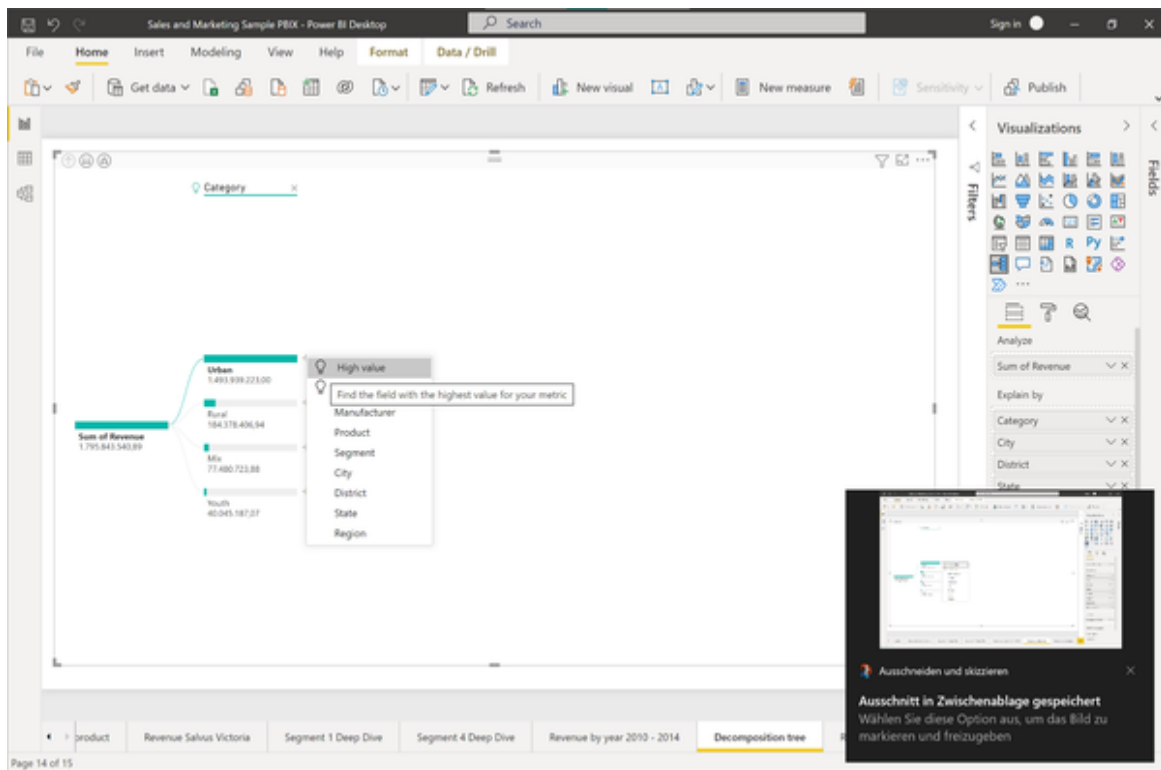


Figure 6-20. Second Level AI Split

If we choose “High values” again, we can see that the next split will be made upon the field “Manufacturer” as shown in Figure 5-21.

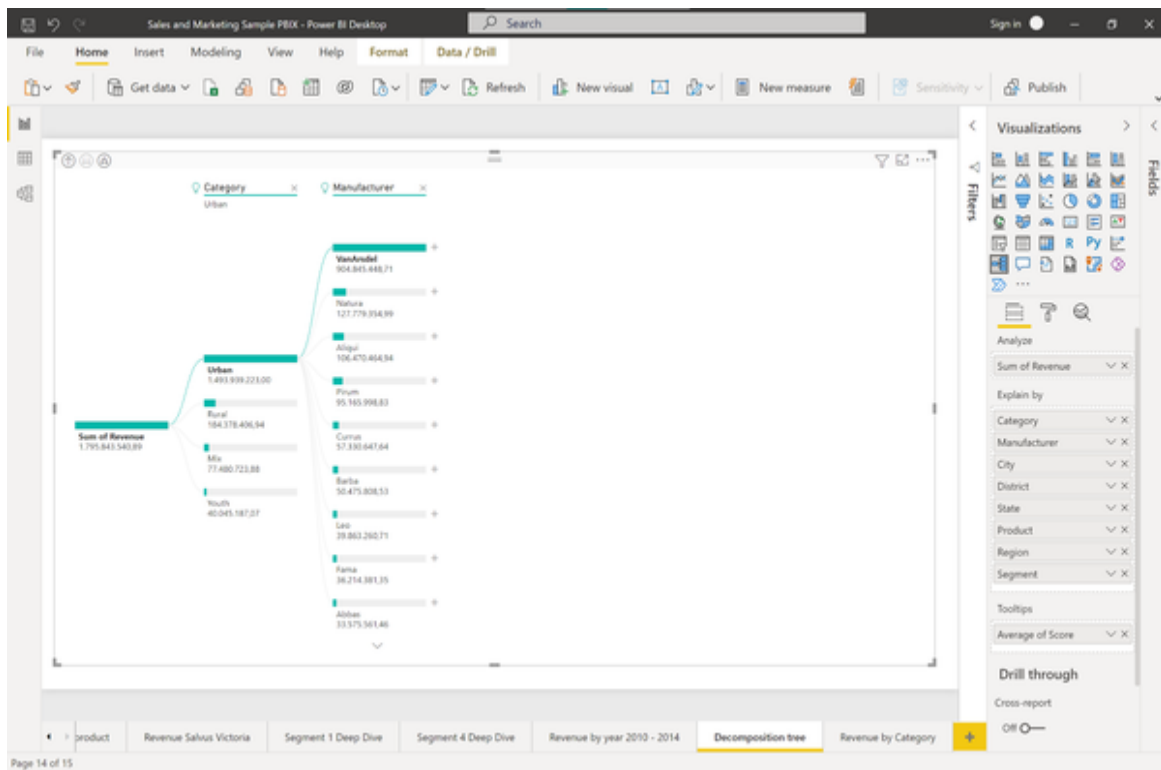


Figure 6-21. Second Level Split in the decomposition tree

At this point, let's stop for a second and quickly recap why exactly we are seeing these split in this order. Why isn't the data splitted first for the Manufacturer and only afterwards according to Category? At least we have seen that the manufacturer "VanArsdel" claims most of the revenues.

This happens because the underlying algorithm in Power BI which makes the splits is greedy. It will choose the next drill down category where the splits will bring the biggest advantage. To demonstrate this phenomenon let's look at a side by side comparison of revenue by category and revenue by manufacturer as shown in figure 5-22.

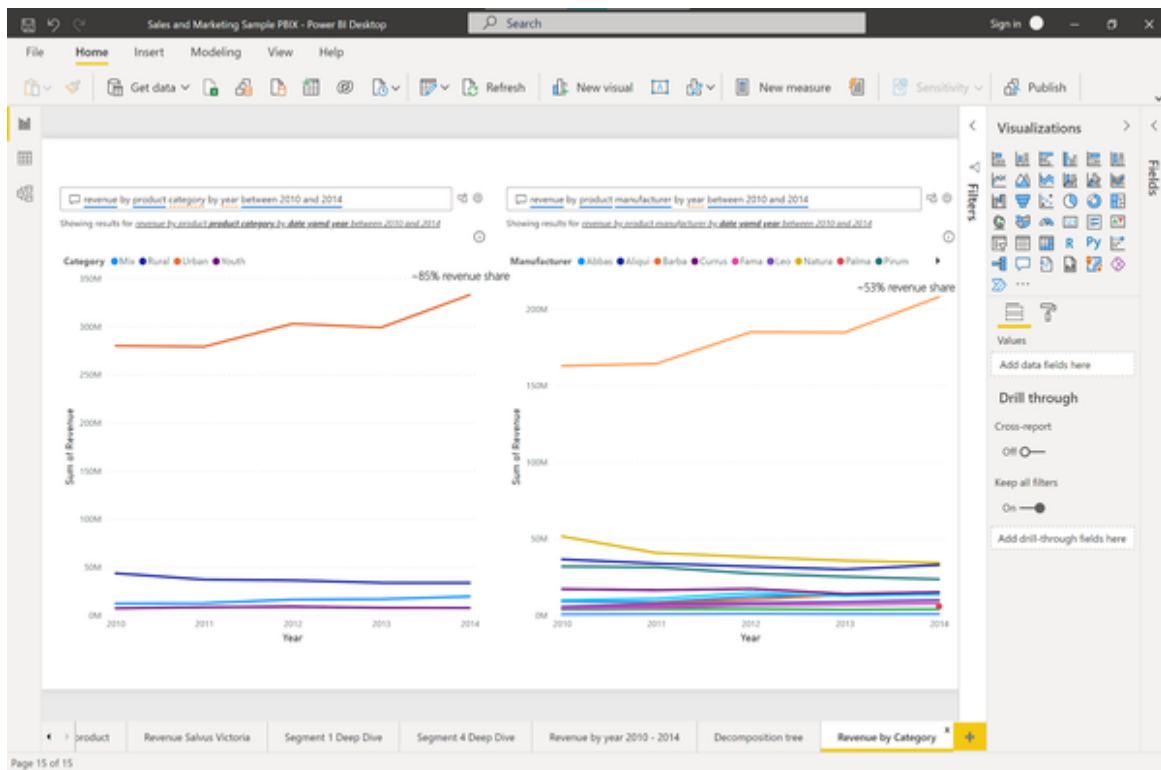


Figure 6-22. Revenue vs. Manufacturer vs. Category

We can clearly see that both Urban and VanArsdel stand out in their classes as main revenue drivers. However, looking more closely we will note that Urban claims around 85% of the revenue share and VanArsdel only gets 53% of the revenue share. This is because there are many more manufacturers than categories which take away plenty of small shares from the leading key driver in this field. And that is why Power BI splits the data first according to Category and only afterwards according to Manufacturers.

While this greedy approach works quite well most of the time, you have to be careful in some cases. When we examine data after a certain split we can only analyse data which actually exist in this split. For example, by splitting for Category first and then exploring the Urban category, we will only see manufacturers in that tree branch which actually produce products in the Urban category. If there was a manufacturer in the dataset which does not produce any goods for the Urban category, this manufacturer would not show up in the downstream splits of that branch of the tree.

and within the region East were the main contributors to the revenue growth in the period 2010 - 2014. If we create a line chart with exactly this filter criteria and compare it to the overall total revenue we can clearly see how this plays out in figure 5-24.

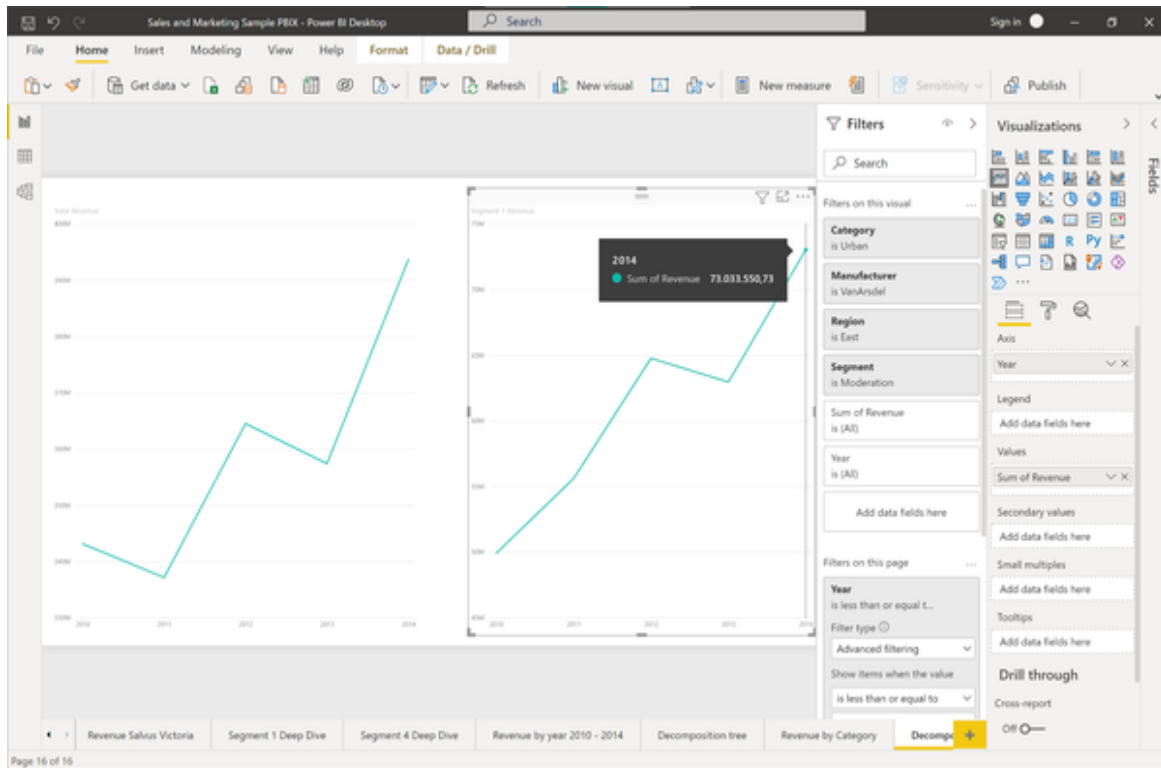


Figure 6-24. Comparison between revenue total and Decomposition Tree branch

Sales from this segment alone contributed more than a 20M revenue plus to our total revenue gain and grew even stronger on a relative level (+46%) than the absolute revenue trend (+15%).

With the Decomposition Tree business users can easily interact with the data and find the relevant contributors on the granularity level which they need. After having combed the data automatically using the AI features of Power BI it is much easier to loop in a data analyst, if necessary, to verify their conclusions or raise questions which came up during the analysis process.

The Power BI report with the Decomposition Tree can be shared with business users so they can browse through the data on their own. To share

reports with other users in your organization you will need a Power BI server or a Power BI pro license.

To find out more about the Decomposition Tool you might want to check the following [Microsoft resource](#).

Summary

In this chapter we have discovered two powerful AI-backed features in Power BI that help business users and analysts alike to reduce their time to insights by automating parts of the data analysis process. The Key Influencer tool can be used to surface key drivers of a given metric of interest and identify relevant segments. The Decomposition Tree in combination with AI splits is very useful for directed drill downs in any data dimensions while keeping an overall business metric fixed. This makes it a great tool for ad hoc data exploration and self-service root cause analysis. In the next chapter we will leave the realms of the past and present data and look how AI can help us to anticipate future events or make better predictions about future outcomes. As such, we will start with an automated classification task which helps us to categorize future business events based on our past experience.

Chapter 7. AI-Powered Predictive Analytics

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 7th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

Take your seats! In this chapter we’ll be stepping into the role of an BI analyst at American Airlines. Using a real world data set we will explore business-critical information from different angles and ask different questions.

Remember that our goal is still to build a prototype version of an AI-powered BI solution. We want to evaluate how well the AI works on our data, show it around and get support for the new approach in our company by demonstrating business value (build fast, show fast, learn fast). For that purpose we will abstract things like setting up data pipelines, handling ETL jobs or integrating AI services into our corporate data warehouse for now. But rest assured - all these things will be possible as we will find out later in Chapter 10.

What will stay is the AI model we build. As early as in our prototype, we will use enterprise-ready AI services provided by Microsoft Azure that will also endure production workloads. The only thing that will change later is

the way we integrate these services into our overarching system architecture. Once we can demonstrate business value and build a solid business case around our solution, we can afford the effort to put the prototype into production and build it “properly” there.

Prerequisites

To follow along with this Chapter you will need an account on Microsoft Azure to train and host the AI model as demonstrated in Chapter 4, some basic knowledge in Python or R to get predictions from the model and a BI tool which is able to execute Python or R scripts to integrate the model predictions into your dashboard.

Although we are using Microsoft Azure to train the model, the same functionality is more or less also available on other cloud platform providers such as Google Cloud or Amazon Web Services. Once you get a feel for how things work on Azure it should be relatively easy to get up and running on the other platforms quickly.

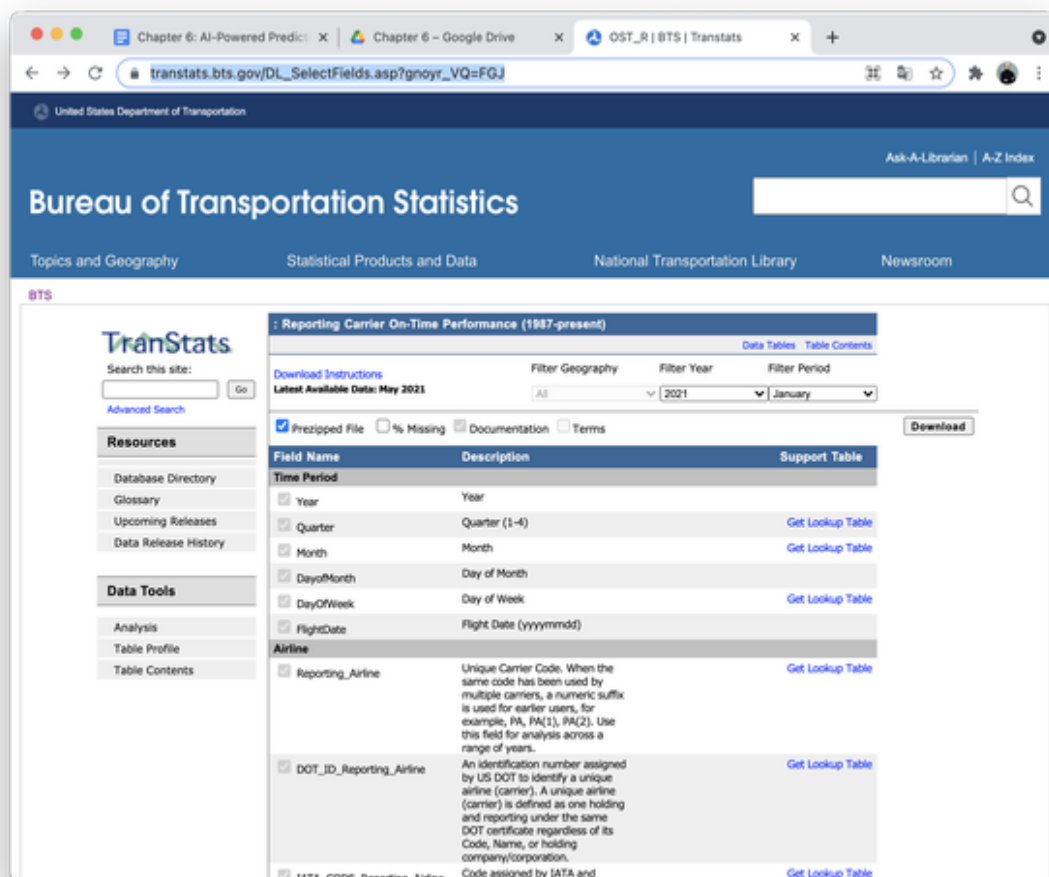
To build our dashboard we will use Power BI later on to achieve consistency with the previous use cases (and also because Azure works nicely together with Power BI). However, this book is not limited to Power BI users. That is why I will show you how to get inference from your model using a popular programming language like Python or R. You can integrate Python or R codes into Power BI or any other BI tool. Alternatively, you could run the predictions directly on the database (batch predictions) and just display the results in any BI tool. More on that will be covered later in chapter 10.

Get the dataset

Beware: We will be working on a real world real world dataset. This dataset hasn't been tweaked or tuned for machine learning exercises. So don't expect any jaw-dropping insights or nicely looking charts. But instead, you will get a feel for how a real world scenario would look like, which gains

you can expect from machine learning and how you can use this technology to beat your own baselines and automate predictions. It might get messy in between but it's closer to your real life than any academic exercise you will find.

To download the dataset, visit the website of the TranStats service of the **US Bureau of Transportation Statistics**. This data source maintains updated information about flight details for flights in the US. We will use flight data from January 2021 and February 2021. You can either download the dataset directly from the website by selecting the “Prezipped file” option and hitting the Download button as shown in Figure XX.



To follow along with the book tutorials I suggest to use the Excel files provided in the book resources. The following processing steps have been

applied to make the file match the case study in this chapter:

- The dataset has been filtered for American Airlines flights only
- Cancelled flights have been deleted from the dataset
- Flights which did not have any information on the arrival status have been removed (<1%>)

Feel free to go back to the original source if you want to try the exercises with an updated dataset or train the model on even more data than just one month.

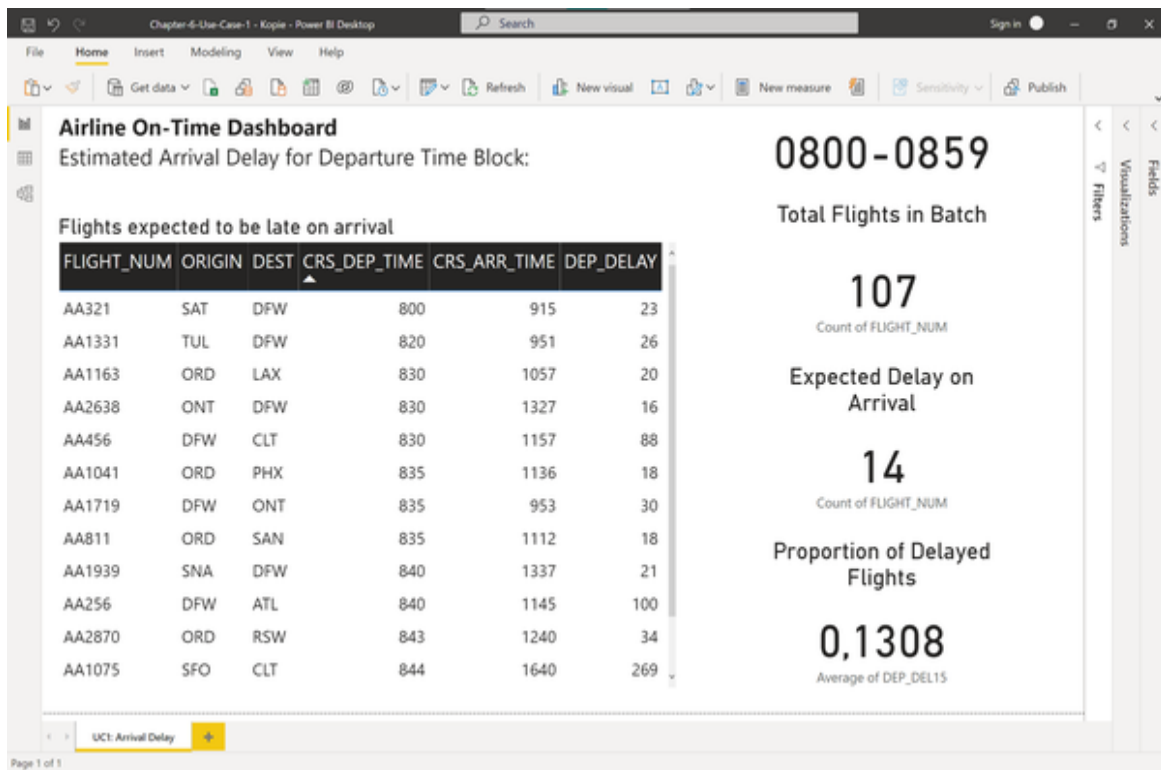
Use Case: Automating Classification Tasks

For our first use case, we are tackling a classification problem where we predict if a given flight will be delayed on arrival or not. Even if you are not an expert in the aviation industry, you should be familiar with the overall challenge. To lay it out in more detail, let's introduce the concrete scenario and problem statement as follows.

Problem Statement

As a member of the BI team of American Airlines we are equipped with the task to build a reporting dashboard on a business critical metric. The metric is the percentage of American Airlines flights which are currently in the air and which are expected to be delayed for more than 15 mins. This metric is important for quality control reasons and keeping customer satisfaction high as well as maintaining the overall flight schedule. Recent flight data is coming in in hourly batches for reasons we can't influence nor change. So for example, when we report the metric anytime between 09.00am and 9.59am the latest data we could see would be from the batch of departures between 08.00am and 08.59am

The metric is currently reported through a BI dashboard which looks as like Figure XX:



Our goal is to come up with a new classifier that manages to beat this 0.72 F1-Score baseline, so we can make even more accurate predictions about flight delays.

The dashboard shows the total number of current flights (107) scheduled for the departure time block 08.00am - 08.59am on February 7, 2021.

Most of these flights will be still in the air before the next data batch comes in. In order to estimate if there will be an arrival delay or not, we are currently using the departure delay information as a proxy. Up to now the BI team has used the variable `DepDel15`, which indicates if the flight was delayed at least 15 minutes upon departure, to approximate if the flight will also be late more than 15 minutes upon arrival.

This estimate has been a good first shot, but it is far from perfect. Let's open up the Excel workbook `AA_Flights_2021_1.xlsx` as seen in Figure XX to find out how this estimate has performed historically.

Year	Quarter	Month	DayofMonth	DayOfWeek	FlightDate	Reporting_Airline	DOT_ID	Reporting_Airline	IATA_CODE	Reporting_Airline	Tail Num
2021	1	1	1	5	2021-01-01	AA	19805	AA	AA	AA	N767AJ
2021	1	1	2	6	2021-01-02	AA	19805	AA	AA	AA	N793AN
2021	1	1	3	7	2021-01-03	AA	19805	AA	AA	AA	N772AN
2021	1	1	4	1	2021-01-04	AA	19805	AA	AA	AA	N789AN
2021	1	1	5	2	2021-01-05	AA	19805	AA	AA	AA	N781AN
2021	1	1	6	3	2021-01-06	AA	19805	AA	AA	AA	N771AN
2021	1	1	7	4	2021-01-07	AA	19805	AA	AA	AA	N781AN
2021	1	1	8	5	2021-01-08	AA	19805	AA	AA	AA	N781AN
2021	1	1	9	6	2021-01-09	AA	19805	AA	AA	AA	N780AN
2021	1	1	10	7	2021-01-10	AA	19805	AA	AA	AA	N794AN
2021	1	1	11	1	2021-01-11	AA	19805	AA	AA	AA	N757AN
2021	1	1	12	2	2021-01-12	AA	19805	AA	AA	AA	N781AN
2021	1	1	13	3	2021-01-13	AA	19805	AA	AA	AA	N772AN
2021	1	1	14	4	2021-01-14	AA	19805	AA	AA	AA	N772AN
2021	1	1	15	5	2021-01-15	AA	19805	AA	AA	AA	N787AL
2021	1	1	16	6	2021-01-16	AA	19805	AA	AA	AA	N730AN
2021	1	1	17	7	2021-01-17	AA	19805	AA	AA	AA	N791AN
2021	1	1	18	1	2021-01-18	AA	19805	AA	AA	AA	N783AN
2021	1	1	19	2	2021-01-19	AA	19805	AA	AA	AA	N787AL
2021	1	1	20	3	2021-01-20	AA	19805	AA	AA	AA	N761AJ
2021	1	1	21	4	2021-01-21	AA	19805	AA	AA	AA	N787AL
2021	1	1	22	5	2021-01-22	AA	19805	AA	AA	AA	N793AN
2021	1	1	23	6	2021-01-23	AA	19805	AA	AA	AA	N783AN
2021	1	1	24	7	2021-01-24	AA	19805	AA	AA	AA	N788AN
2021	1	1	25	1	2021-01-25	AA	19805	AA	AA	AA	N760AN

The Excel spreadsheet contains all American Airlines flights from January 2021 - almost 38,000 rows of data. Each row is an American Airlines flight which happened during January 2021.

Since this is a historic data set we can see the actual arrival delay that happened. Take the first row for example. The flight Number AA1 from John F. Kennedy Airport to Los Angeles took off on January 1, 2021. The flight didn't have a delay on departure and thus the column DepDelay15 is set to 0. Since it landed even ahead of schedule without any delay the ArrDelay15 column is also set to 0. The first flight which had a departure delay of more than 15 minutes happened in row 21: It is again flight AA1 from JFK to LAX, but this time on January 20, 2021. The DepDelay15 attribute is 1 since the departure delay was 75 minutes. The flight landed with 61 minutes delay and therefore the attribute ArrDel15 is 1, see figure XX.

he	DestWac	CRSDepTime	DepTime	DepDelay	DepDelayMinutes	DepDel15	ArrDelay	ArrDelayMinutes	ArrDel15
21	91	0900	1015	75,00	75,00	1,00	61,00	61,00	1
22	91	0900	0850	-10,00	0,00	0,00	-16,00	0,00	0
23	91	0900	0857	-3,00	0,00	0,00	-11,00	0,00	0
24	91	0900	0859	-3,00	0,00	0,00	-11,00	0,00	0

So in this case the attribute DepDelay15 was a good indicator whether a flight was actually also delayed on arrival. Now, how good would our prediction have been if we only relied on the DepDelay15 information? Head over to the second worksheet of the Excel file to see the “Confusion Table”. You should see the table as in figure XX.

	A	B	C	D
1	Classifier Evaluation			
2				
3			COUNT	PERC
4	BASELINE PREDICTION	0	34.169	90,1%
5	ACTUAL	0	32.991	96,6%
6	ACTUAL	1	1.178	3,4%
7	BASELINE PREDICTION	1	3.737	9,9%
8	ACTUAL	0	988	26,4%
9	ACTUAL	1	2.749	73,6%
10		Total	37.906	100,0%
11				
12		ArrDelay15	True	10%
13			False	90%

Before we dive into the details, look at the last two rows where it reads ArrDelay15 True = 10% and False = 90%. This summary tells us that we are dealing with a so-called “imbalanced classification” problem. Only 10% of all flights are actually flagged with the ArrDelay15 attribute and are thus more than 15 minutes late on arrival.

Why should this imbalance be important to us? Because it affects our evaluation metric. Imagine that if we predicted that all flights will be on time with less than 15 minutes delay, this would give us an astonishing

accuracy score of 90%. Would this prediction be useful? No. That is why we have to come up with some smarter evaluation metric.

Look at the pivot table above. This shows us how our baseline prediction (the DepDelay15 label) performed. If we used the Departure Delay heuristic in the past, we would have misclassified 2.166 observations: 1.178 flights were actually delayed although we predicted they would be on time. And 988 flights were actually on time, even though we predicted there would be a delay.

How do we compress this information into a metric which is easy to track for us? We are using a metric called “F1-Score” which is a combination of two other metrics: Precision and recall.

Precision is a measure that captures a classifier’s exactness, where higher precision indicates fewer false positives. If we manage to improve this metric, the 988 flights which have been predicted as delayed, but were actually on time, would be reduced. The precision is calculated by dividing the true positive values (2.749) by the sum of the true positives (2.749) and false positives (988).

Recall measures a classifier’s completeness, where higher recall indicates fewer false negatives (1.178 flights that were delayed although predicted on time). Recall is calculated by dividing the true positives (2.749) by the true positives plus the false negatives (1.178)

The F1-score is the harmonic mean of precision and recall and calculated as follows:

$$F1 = 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall}) = 2TP / (2TP + FP + FN)$$

As you can see in figure XX, the F1-Score for our baseline classifier is 0.72.

BASELINE			
True Positive	2.749		
False Positive	988		
True Negative	32.991		
False Negative	1.178		
Precision = $TP / (TP + FP)$		0,74	
Recall = $TP / (TP + FN)$		0,70	
F1-Score = $2TP / (2TP + FP + FN)$		0,72	

Our goal is to come up with a new classifier that manages to beat this 0.72 F1-Score baseline, so we can make even more accurate predictions about flight delays.

Solution Overview

In order to improve the prediction for a late arrival, we will use an AI classifier trained on historic data.

The AI will learn patterns from the past which factors influence a late arrival and help us classify new (unseen) data points based on the examples from the past. This problem is called a classification problem.

There are many approaches to solve classification problems. Since we don't want to involve AI or ML experts, we will use AutoML to train a model for us.

In our particular case we are using Microsoft Azure as the AutoML platform, but it could really be anything - they all work more or less the same.

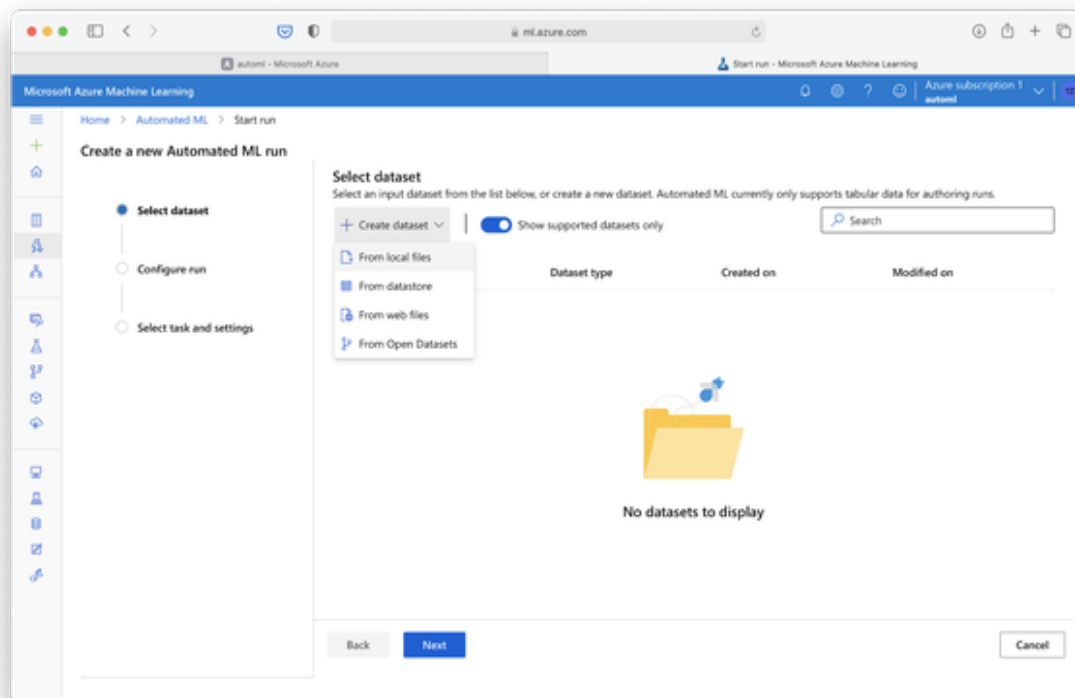
We will first build the model on Azure and then test the model on new, unseen data. After this final evaluation we will learn how to send requests to the model using an online API and we will finally integrate these insights

into a BI dashboard. We will use Power BI for this purpose to present the results in a dashboard format.

Model Training with Microsoft Azure Walkthrough

Visit the [Azure Machine Learning Studio](#) and select the resource that you created in chapter 3. Since we want to train an AI model using AutoML, select “Start Now” on the Automated ML pane and choose “New Automated ML run” to fill up the blank rows.

Before we can set up any AutoML run, we will need some data. On Azure, data is organized in datasets which are linked to your workspace. Datasets will ensure that your data is correctly formatted and can be consumed by machine learning runs. As shown in Figure XX select create dataset.

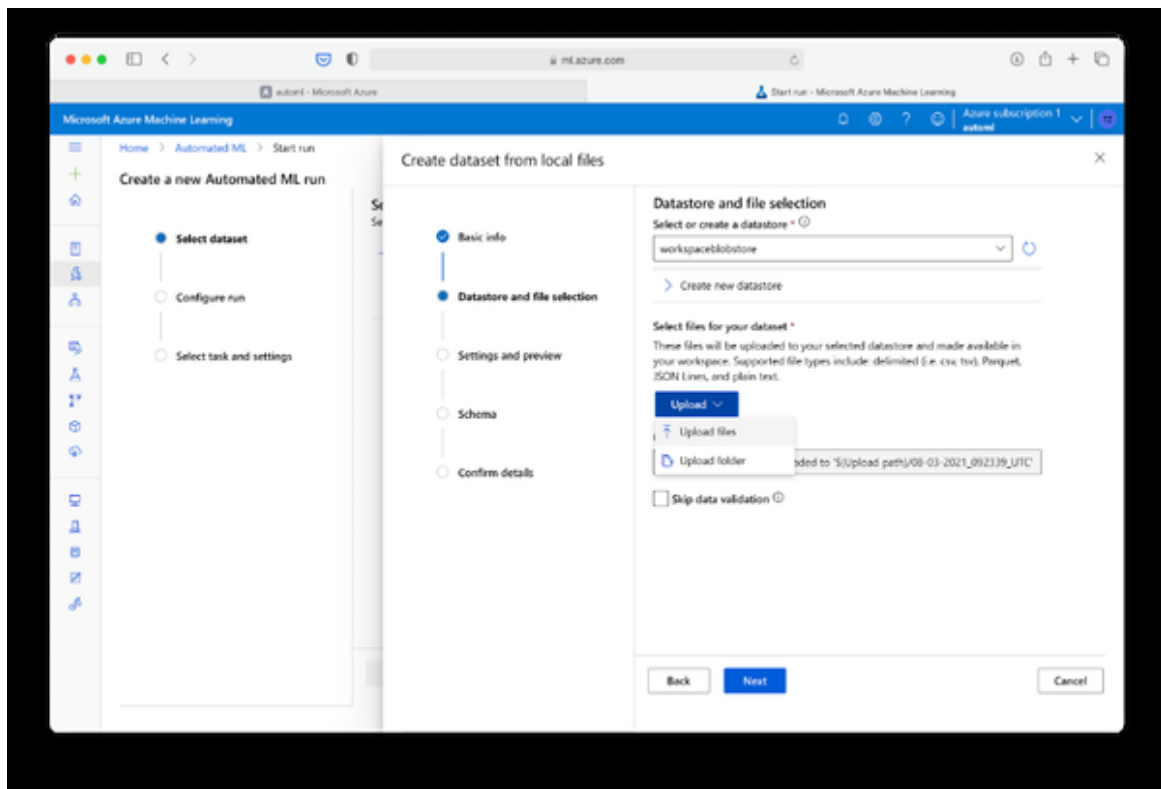


You can import data from various sources. In our case, we will upload a local file, so go ahead and select “From local files” as shown in Figure XX. In the Basic info form shown in Figure XX give your dataset a telling name and add a description if you like. In our case, I named the dataset

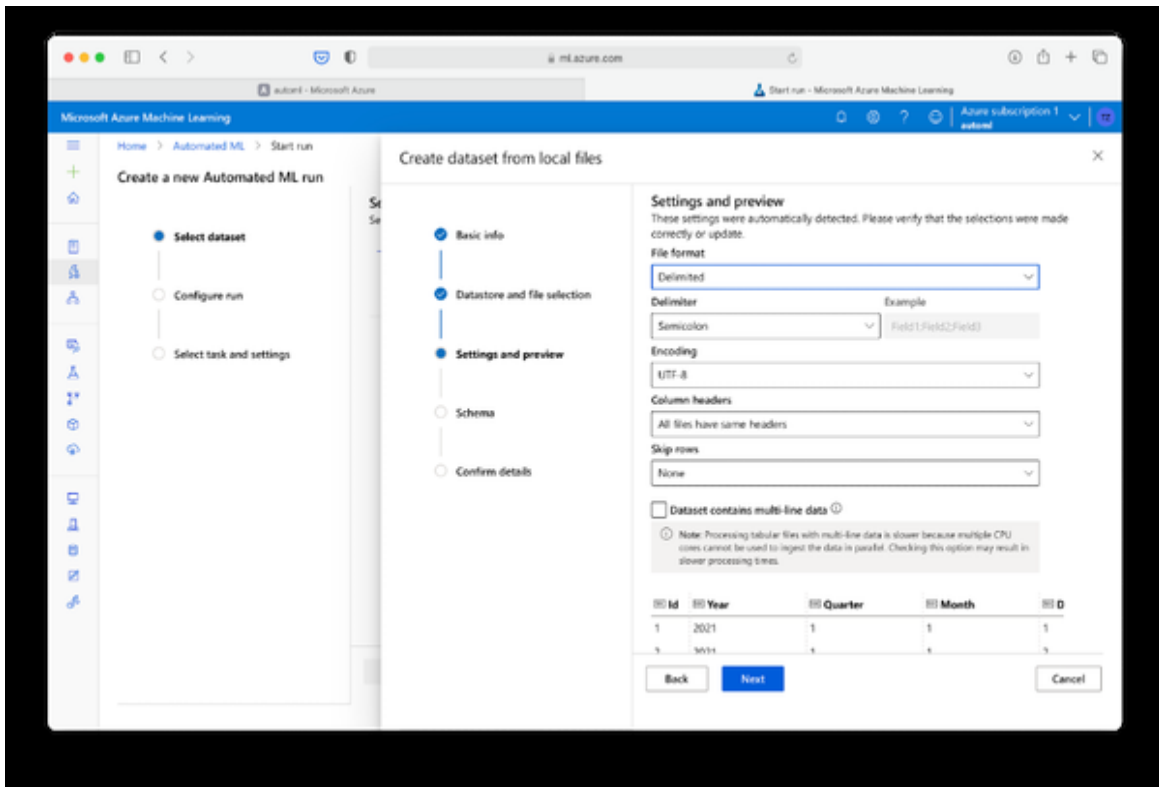
aa_flights_ontime. The version will default to 1 and the dataset type will be Tabular as AutoML currently only supports tabular data.

The screenshot shows the 'Create dataset from local files' wizard in the Microsoft Azure Machine Learning interface. The left sidebar shows the 'Create a new Automated ML run' workflow with steps: Select dataset, Configure run, and Select task and settings. The main panel is titled 'Create dataset from local files' and has a progress bar with steps: Basic info, Datastore and file selection, Settings and preview, Schema, and Confirm details. The 'Basic info' step is active, showing a form with the following fields: Name (aa_flights_ontime), Dataset version (1), Dataset type (Tabular), and Description (Dataset description). At the bottom of the form are 'Back', 'Next', and 'Cancel' buttons.

Click Next and head over to the next section. You will need to provide information as shown in Figure XX. You need to choose where the data should be stored and from where it should be uploaded. Select the default datastore that was automatically set up during your workspace creation (in my example it is called “workspaceblobstore”. This is where you’ll physically upload your data file to make it available to your current workspace. Click upload and choose the file “AA-Flights-2021-01-Training.csv”. This file contains the same data as the “AA_Flights_2021_1.xlsx” worksheet we have explored before. I have just converted it to CSV for your convenience. Select Next on the bottom of your screen to upload the file. Depending on your network connection this might take a few moments.

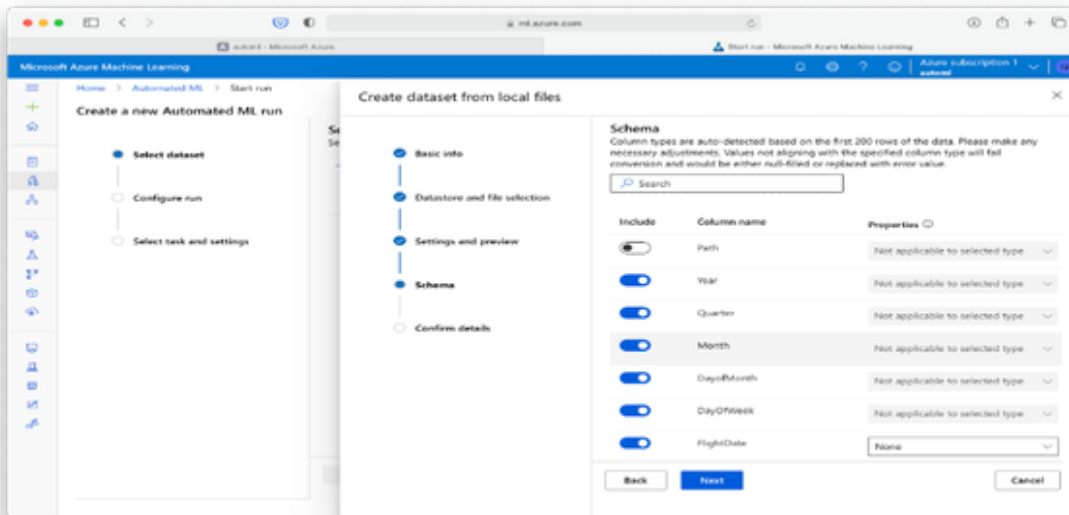


After the file was successfully uploaded you should see the “Settings and preview” form with some pre-populated information based on the uploaded file.



For our example table, double check that the settings are set as shown in Figure XX and that the data preview on the button is displaying the first couple of rows correctly. If everything looks good, click **Next**.

The following form will define the Schema of your dataset and this one is very important for two reasons: First, the schema will allow you to define which columns in your dataset should be considered by the AutoML algorithm. And secondly, the schema allows you to define the data type of the respective columns.



This step is important because both, used columns and data type can only be hardly inferred by the data itself. It requires your knowledge to specify this upfront. In our example, we don't need all the data provided in the dataset. Some values don't provide any meaningful insight at all and would only increase the complexity of the project (such as Year which is all the same). And some other values we don't want to include in our prediction because it contains something which is called "target leakage" in the context of ML. For example, arrival delay does tell us perfectly if ArrDel15 is true or false. And second, we want to set the data types correctly. For example, the respective data groups should be string and not numeric because you can't calculate properly with them (weekday 7 + 1 is not weekday 8, but weekday 1). So select only the following data with the respective data types:

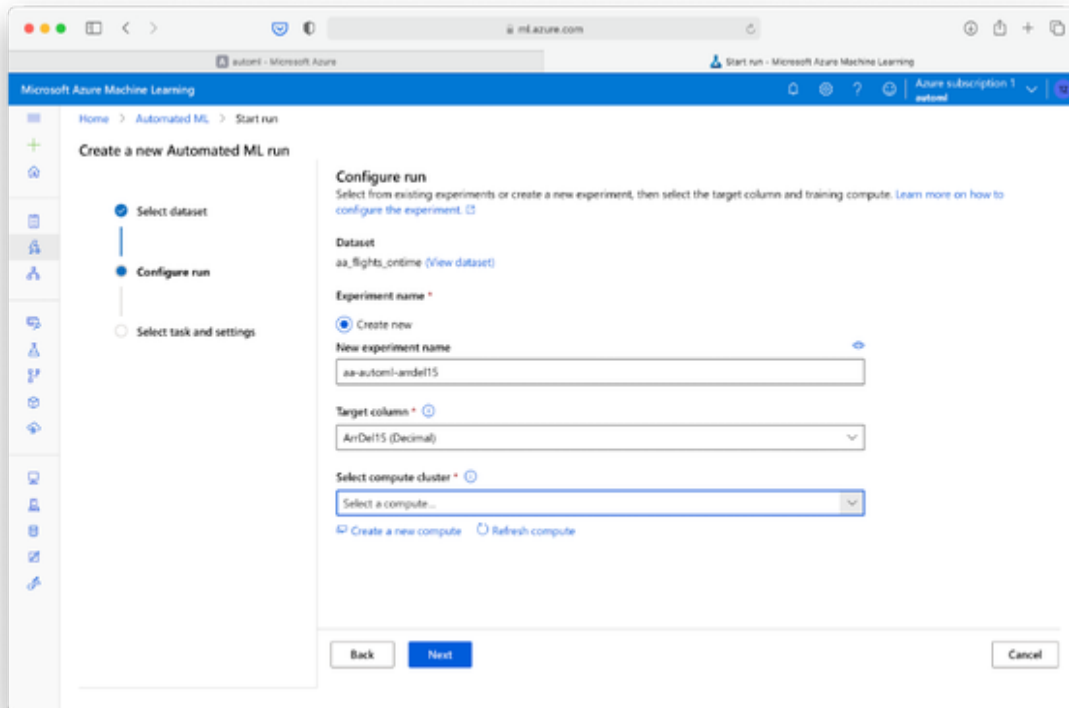
- DayofWeek - String
- Origin - String
- Dest -String
- DepDelay - Dec
- DepDelayMinutes - Dec

- DepDel15 - String
- DepartureDelayGroups - String
- DepTimeBlk - String
- TaxiOut - Dec
- ArrDel15 - String
- ArrTimeBlk - String
- Distance - Dec
- DistanceGroup - Str

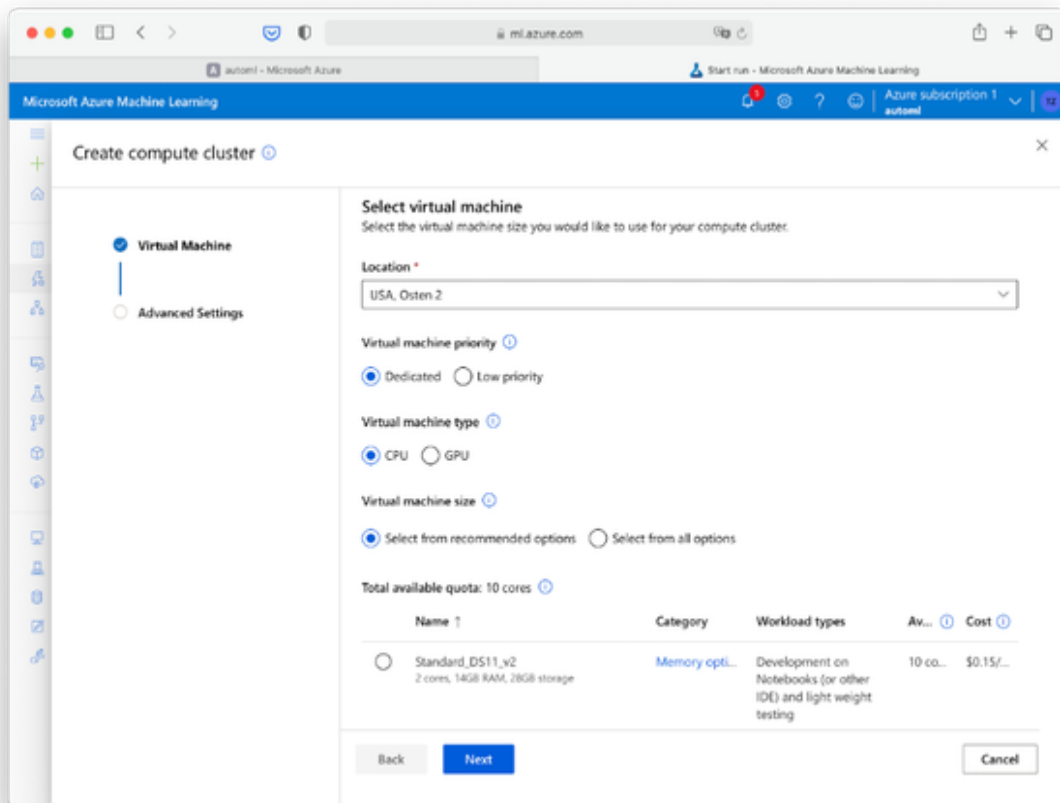
Select Next. On the Confirm details page, double check the information you provided and click Create to finish the creation of your dataset. You can select your dataset from the list that appears.

Process by selecting Next. You will see the Configure Run screen as shown in Figure XX.

What is an AutoML Run? Now that we have defined our dataset we have to define which computation resources should be used for the Auto ML training. In Azure, one Auto ML training on a specific dataset is called a AutoML run. Runs are organized in experiments. An experiment holds the information such as the size of your compute environment and specifies what column you want to predict.



Since we have not set up an experiment yet, select the “Create new” radio button. Enter a custom experiment name. In this case I chose aa-automl-arrdel15. Specify ArrDel15 (String) as the target column. Finally, we have to specify which computing resources should be used for this training run. Click “Create new compute” and you will see a screen as shown in Figure XX.

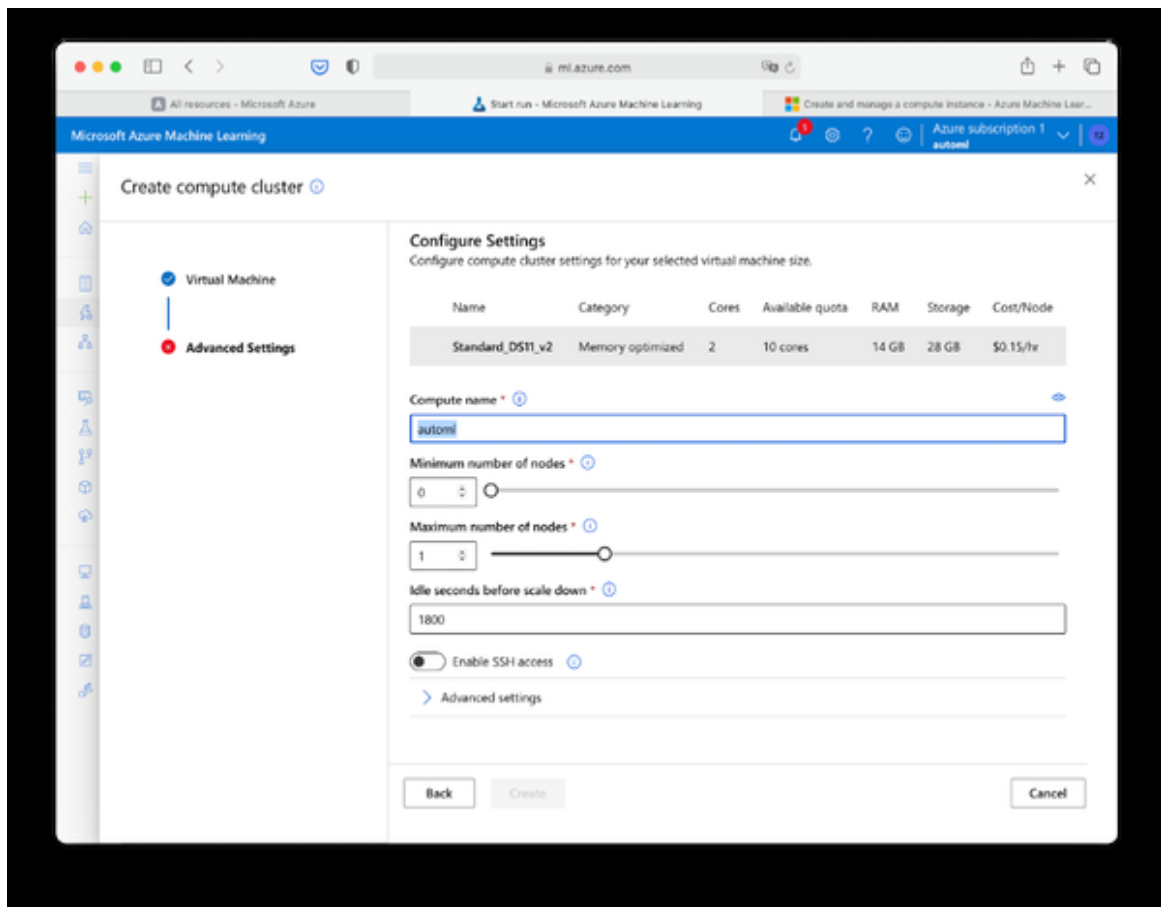


At this place you can configure the computing resources used for the experiment. The more performance you allocate, the faster the training will take place. Note that you will be charged for both, the compute resource and the AutoML training. While a larger compute resource might benefit the AutoML speed (and thus decrease the Auto ML service costs), it is typically more cost-effective to go for a cheaper compute resource and accept a slightly longer AutoML training. You can see the costs noted below in the suggested resources: decrease the time needed for the AutoML to take place. A “Standard_DS11_v2” machine with 2 cores, 14GB RAM and 28GB storage costs about \$0.15 per hour in the US East region. Azure will suggest a list of recommended sized based on your data and experiment type. For our experiment, choose the following settings:

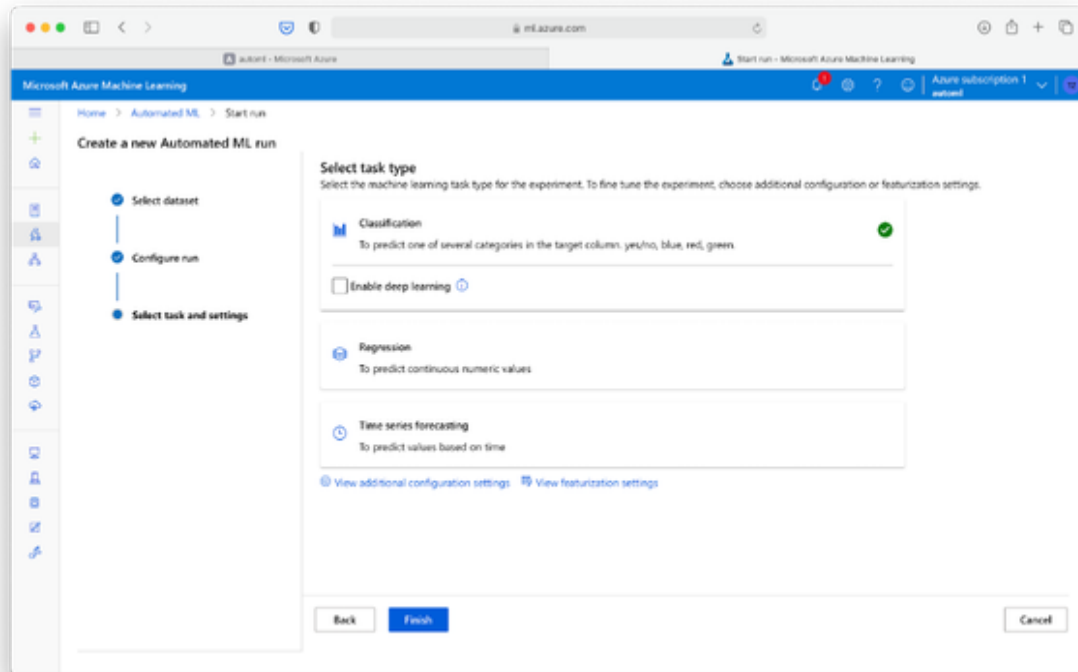
- Location: US East 2
- Virtual machine priority: Dedicated

- Virtual machine type: CPU
- Virtual machine size: Standard_DS12_V2

Select Next. You should see a form similar to the one in Figure XX. Enter a name for your compute instance. I chose “automl” in the example below. Also, provide a minimum and a maximum number of compute nodes. As you can see from this, the cloud compute resource is actually designed to be a cluster of virtual machines to handle even very large datasets. Since our dataset is not too large at all we don’t need a cluster of machines but a single machine will do. Select Minimum number of nodes 0 and maximum number of nodes 1. Set “Idle seconds before scale down” to 1800. This will ensure that our machine will go to standby after 30 Minutes of inactivity and prevent us from costs being charged. Leave SSH access disabled and also skip the advanced settings. Click Create. This process might take a couple minutes to complete. Once finished, you will be redirected to the “Configure run” screen in Figure XX with the new compute resource pre-populated in the compute resources dropdown menu. Proceed with Next.



Now it is time to specify the AutoML task we want to do.

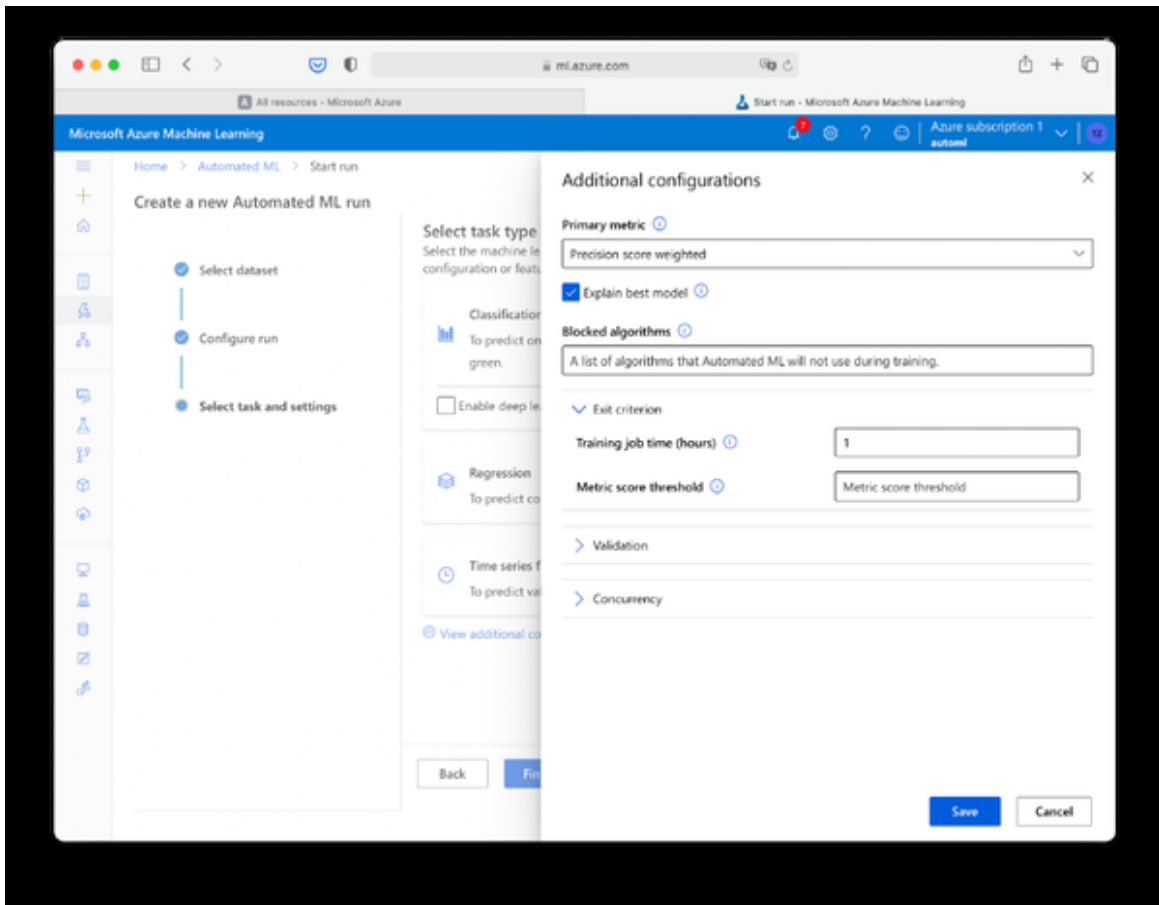


We can choose between three different types here:

- Classification: Predicting a categorical variable
- Regression: Predicting a continuous numeric variable
- Time series forecasting: Regression for time series data

Do you recognize why it was important to set the data types correctly before when we created our dataset? If we had left the datatype of the ArrDel15 variable to Decimal, then we would not be able to select Classification here. Similarly, if we want to predict a numeric value, we have to make sure it is correctly being set in the schema to enable regression. There is no way Auto ML could reliably guess the datatype on its own. That is something we have to define manually, especially for the target column.

Before you choose Finish, click “View additional configuration settings”. You will see the additional configurations pane as shown in Figure XX.



There are some things we want to customize here. First, we want to change the Primary metric. Per default this is set to “Accuracy”. As we have learned previously, Accuracy isn’t a good measure for imbalanced classification problems. We can’t choose F1-Score here. Instead, choose “Precision score weighted”. This will also consider Recall and therefore a result in a better F1-Score.

Also, make sure that “Explain best model” is ticked.

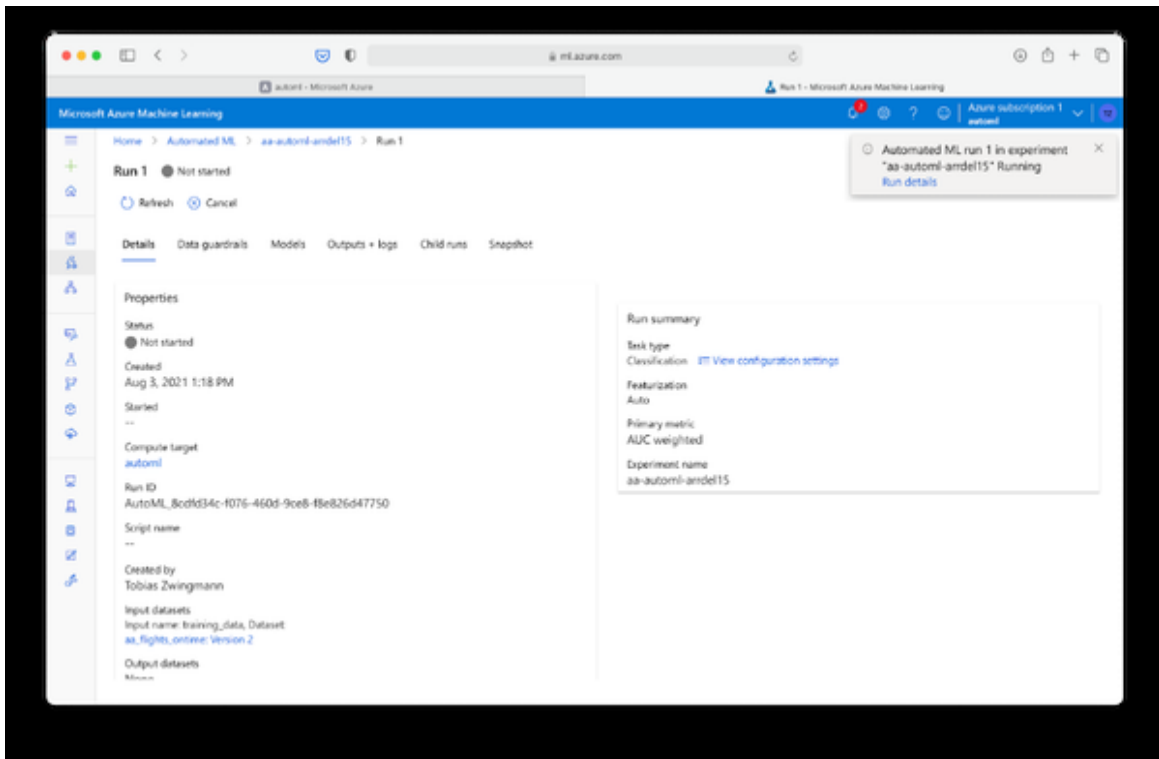
NOTE

Choosing the correct evaluation metric is a vital step in your Auto ML process and can significantly affect the results. At the same time it is hard to guess for the Auto ML service which metric you need, since this depends on the problem at hand. For example, do you prioritize more False Positive over False Negative results such as in clinical tests? Become familiar with different metrics and experiment with them in different AutoML runs. Visit “Experiments” in Azure ML Studio to compare your Auto ML Runs according to different metrics.

[Resource](#)

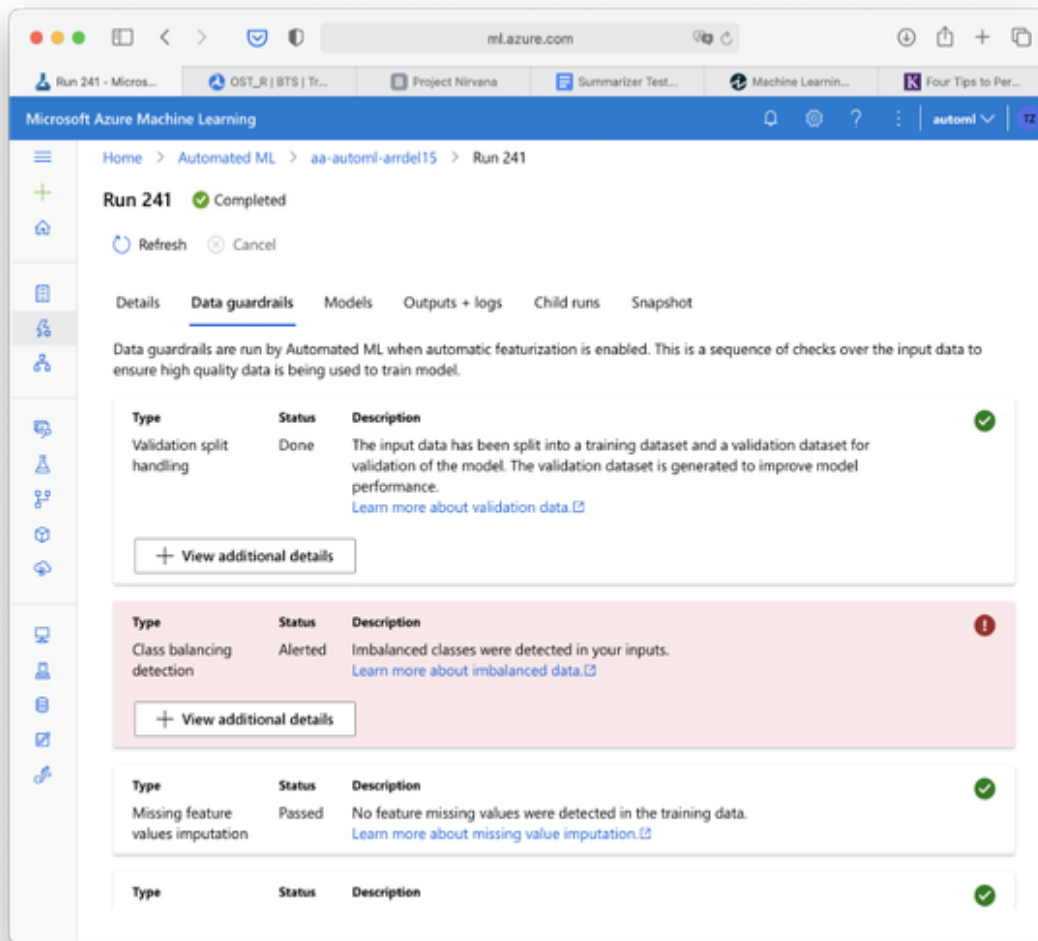
Another setting we want to customize is the “Exit criterion”. This will tell our Auto ML algorithm when the model is “good enough”. If a criteria is met, the training job is stopped. There are typically two approaches you can take here. You can end the training by time limit or by a minimum acceptance criteria for the metric. In our case, choose 1 hour as the maximum time limit. In fact, chances are that the training might even end before, if the Algorithm can’t get any more significant gains on training an even better model.

Confirm the additional settings with Save. And click finish to start your AutoML run. This will initialize your AutoML run and bring you to the “Run Detail” screen as shown in Figure XX.



This status updates as the experiment progresses. After a couple minutes the Status should change from “Not started” to “Running”.

It will probably take 10 minutes or so before the first model is actually being trained. There are some set up procedures and pre-works being done before. One of these preparations is a feature called “Data guardrails”. Click the “Data guardrails” tab to find out more. After some minutes, it should look like in figure XX. If this area is still empty, check back after some minutes and refresh the page.

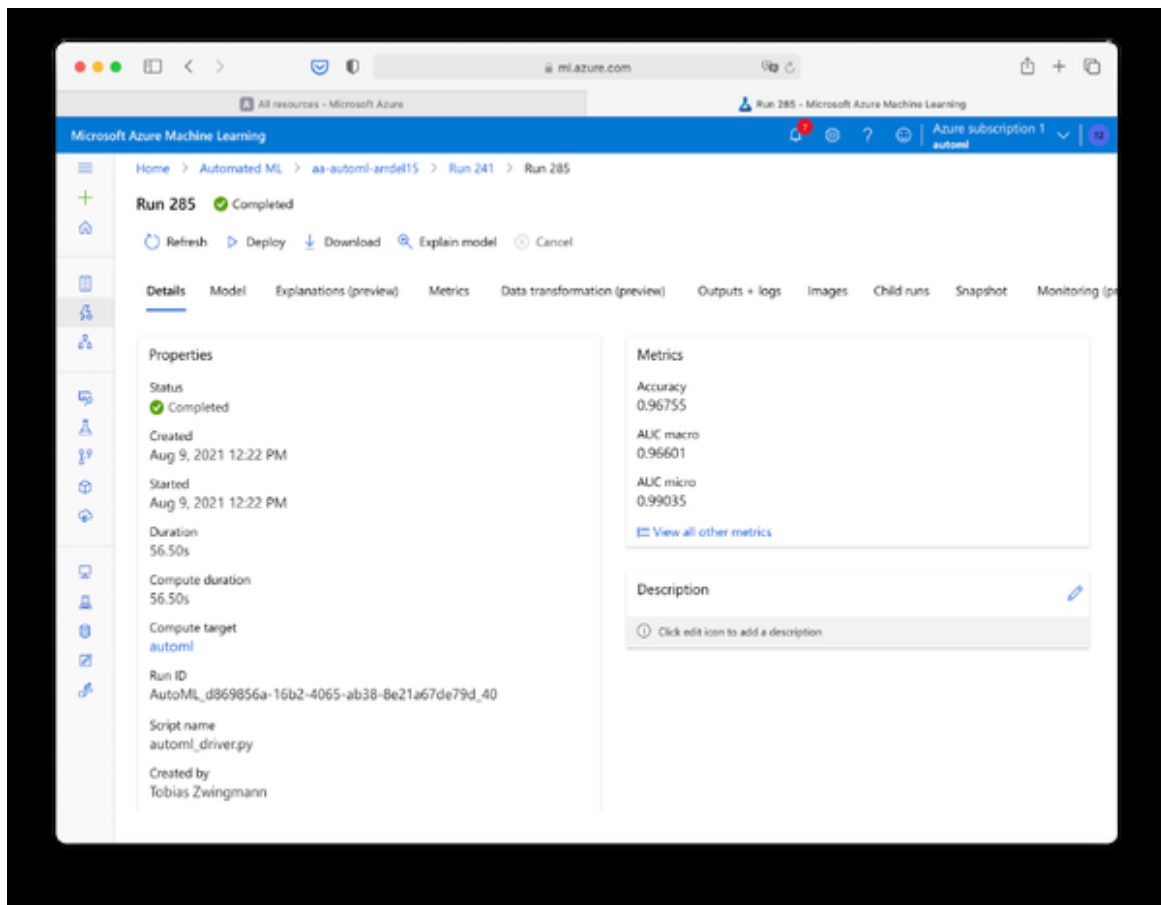


Data guardrails on Azure Auto ML is a series of automated checks over the training data to increase the chances of a high quality training outcome. These checks include splitting the data into training and validation sets automatically, or checking for missing values in the training columns. It also checks for some assumptions related to the training task at hand, for example it recognizes the imbalanced classes in the label and flags this as a potential problem area. If you click the button “View additional details” you will see the underlying distribution of the class labels. To find more about how to tackle this problem, Azure provides a link to “**Learn more about imbalanced data**”. Besides some built-in features that help to tackle class imbalances such as adding a weight column, one major point they suggest is to pick an evaluation metric which is robust against imbalanced classes.

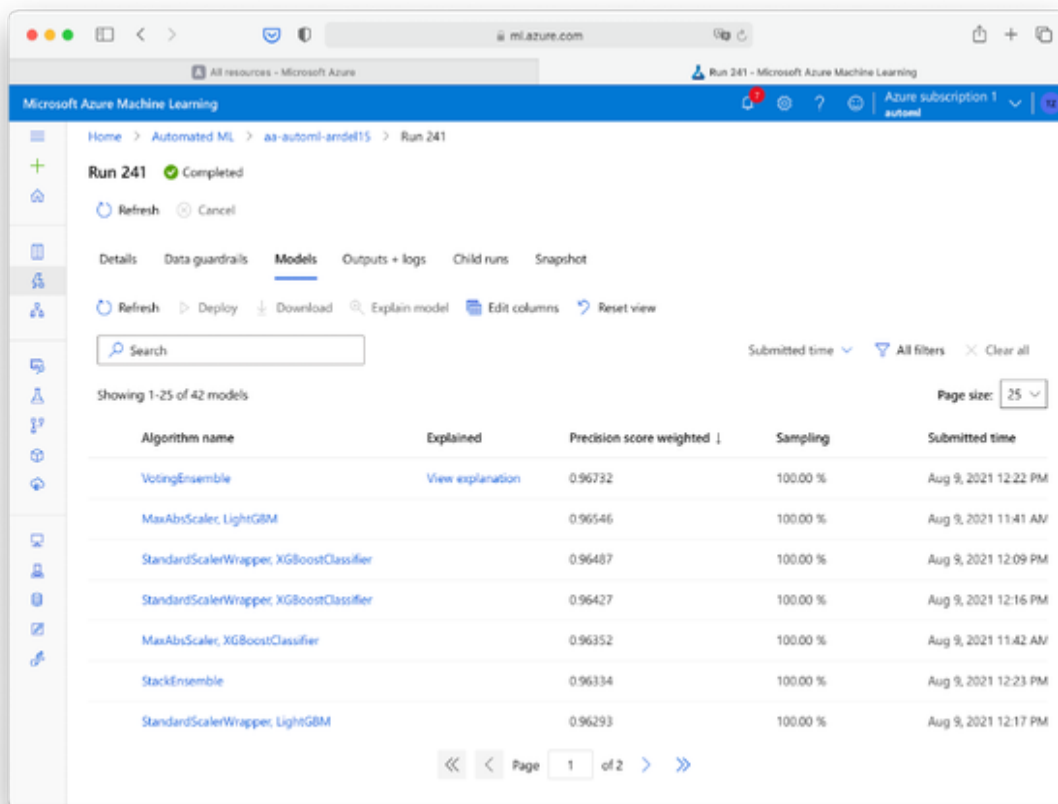
Good choice that we chose Precision over Accuracy as our leading metric before.

While the overall training process is still running, head over to the tab “Models”. Here you will see a list of all the models and the preliminary evaluation metrics the AutoML algorithm has come up with so far. After 15 to 20 minutes you should already see some preliminary model with the respective evaluation metric. Consider these models as it can only become better from here. If the Auto ML algorithm finds an even better model, it will appear on top of this list. You can explore each model here and look down further into the respective details. But the idea of Auto ML is that we don’t worry too much about what’s going on during training. Let the algorithms do their job, grab a coffee and check back in 30 minutes or so.

Has the training been finished? If so, you should see a screen as in figure xx where the status says “Completed”.



Now it's time to look at the best model the algorithm has found and explore it in a bit more detail. On the right side of the “Details” screen you will find the “Best model summary”. This part shows you some important metrics such as the algorithm name, in this case it was a Voting Ensemble” and the final Precision score that has been reached, in this case 0.96732. The actual results depend on the computing resources and the training time spent. They might look a little different if you chose different settings for the run configuration.



Microsoft Azure Machine Learning

Run 241 - Microsoft Azure Machine Learning

Run 241 Completed

Details Data guardrails Models Outputs + logs Child runs Snapshot

Refresh Deploy Download Explain model Edit columns Reset view

Search Submitted time All filters Clear all

Showing 1-25 of 42 models Page size: 25

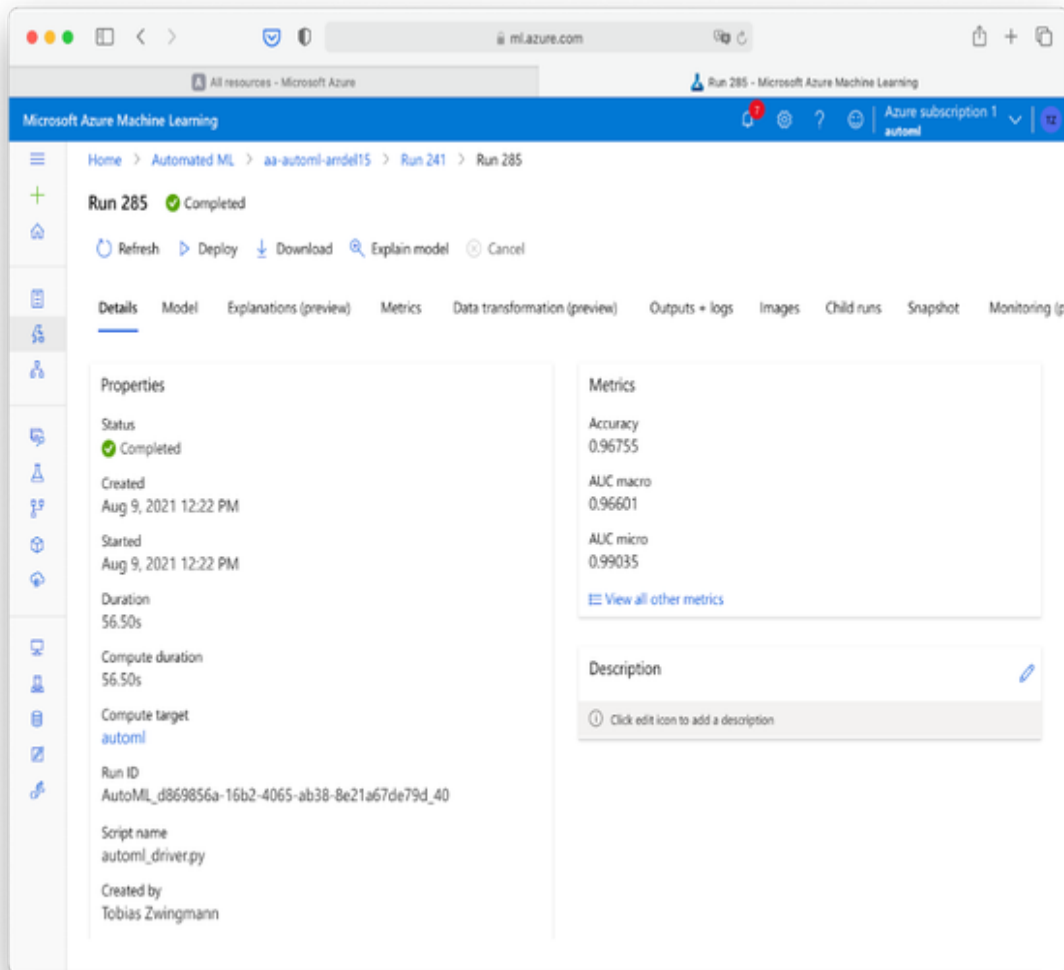
Algorithm name	Explained	Precision score weighted ↓	Sampling	Submitted time
VotingEnsemble	View explanation	0.96732	100.00 %	Aug 9, 2021 12:22 PM
MaxAbsScaler, LightGBM		0.96546	100.00 %	Aug 9, 2021 11:41 AM
StandardScalerWrapper, XGBoostClassifier		0.96487	100.00 %	Aug 9, 2021 12:09 PM
StandardScalerWrapper, XGBoostClassifier		0.96427	100.00 %	Aug 9, 2021 12:16 PM
MaxAbsScaler, XGBoostClassifier		0.96352	100.00 %	Aug 9, 2021 11:42 AM
StackEnsemble		0.96334	100.00 %	Aug 9, 2021 12:23 PM
StandardScalerWrapper, LightGBM		0.96293	100.00 %	Aug 9, 2021 12:17 PM

<< < Page 1 of 2 > >>

Head over to the models by clicking the “Models” tab. You should see a list similar to Figure XX. From this overview we can see that the top 5 models are actually very close together, with Precision scores ranging between 0.96352 and 0.96732 for the best model. As we have selected before, the best model also provides explanations. We will come back to this in a bit. But before, click the algorithm name “VotingEnsemble” to see the model’s detail page as shown in Figure XX.

NOTE

The actual results for your AutoML run might slightly differ from the results in this books. While the Auto ML process is deterministic you might see different results due to stochastic procedures on the data preparation process (e.g. automated data sampling for training and testing). The big picture, however, should be very similar.

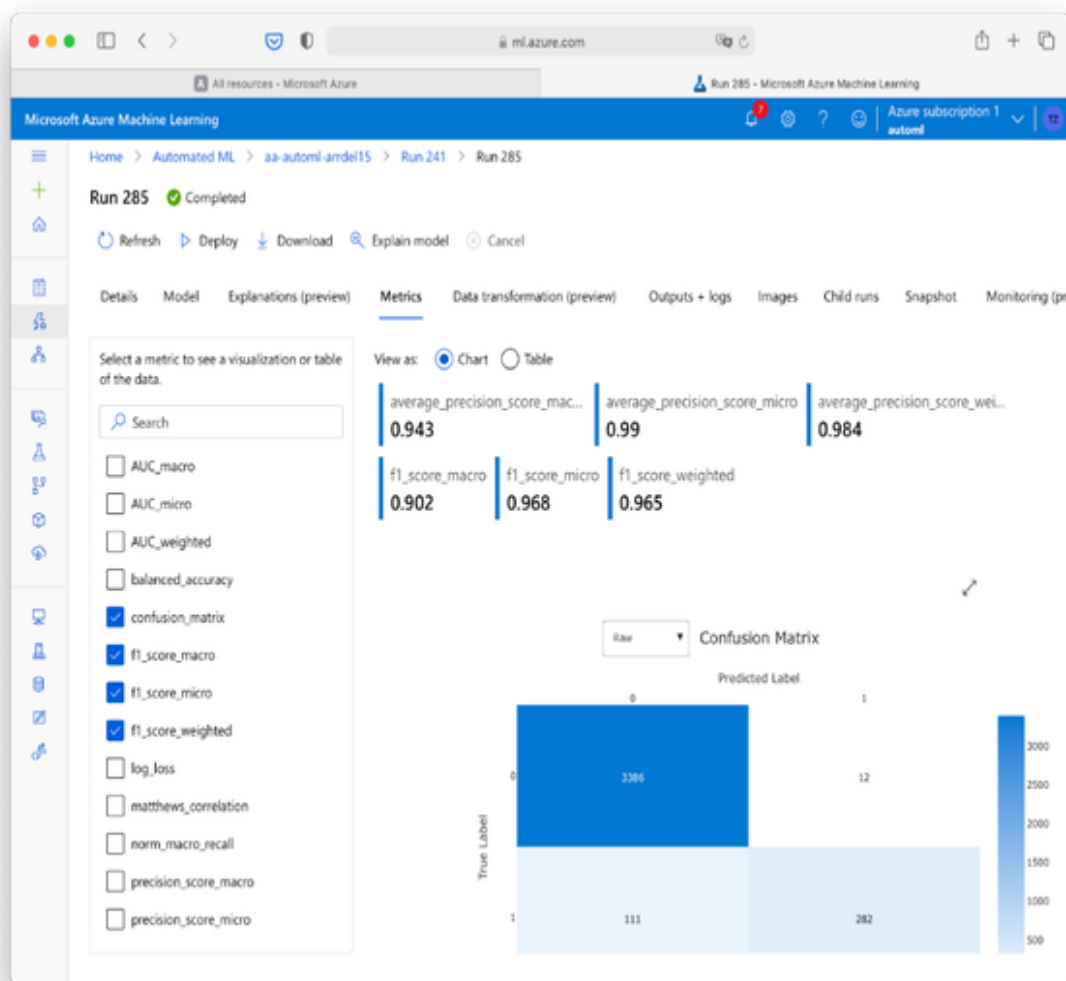


Most importantly, we want to find out how this model performs on our data. To find out more, click the “Metrics” tab. On this section, enable the following evaluation metrics and keep all other ones disabled for now:

- `average_precision_score_macro`

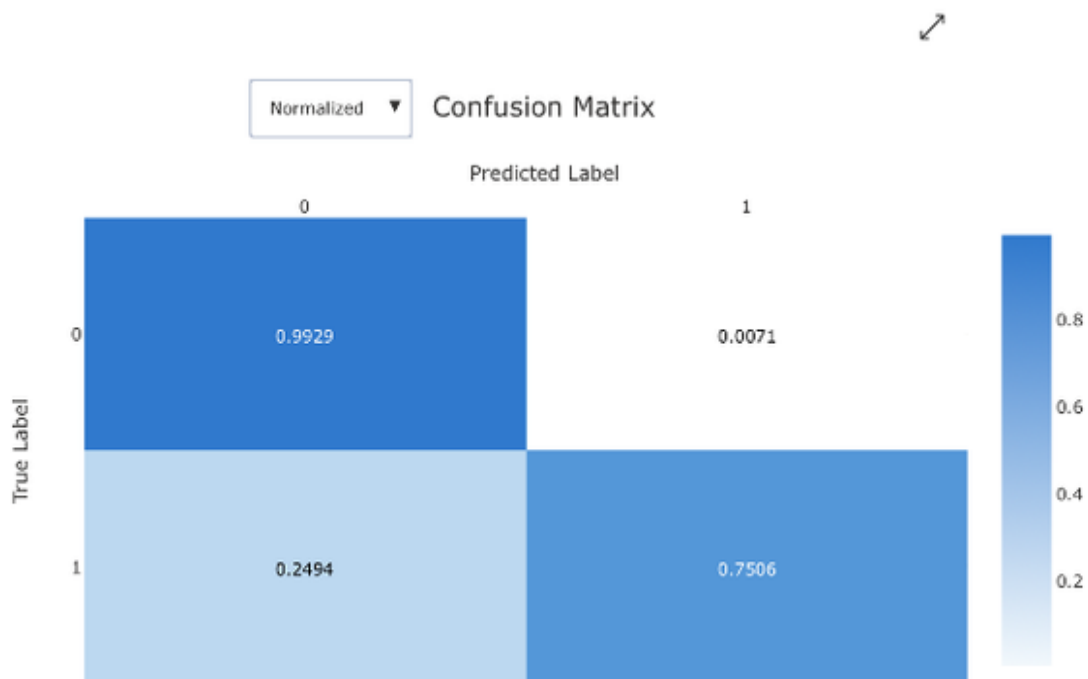
- average_precision_score_micro
- average_precision_score_weighted
- confusion_matrix
- f1_score_macro
- f1_score_micro
- f1_score_weighted

Your screen should look similar to the one in figure XX.



Note: Your model and result might look slightly different. Try different formats. Show experiments table.

Before we worry too much about what micro, macro and average metrics mean, let's take a look at the confusion table that the model evaluation provided for us. This confusion table contains exactly the same information as our Excel table in the problem statement, just the layout is a bit different. The table contrasts the true data labels with the predicted labels by the model. For example, there were 3374 observations where the actual label was "0" (no arrival delay > 15 Minutes) and the predicted value by the model was also "0". By contrast, 98 observations were predicted to be ontime (Predicted Label = "0"), but were in fact delayed by more than 15 minutes (Actual Label = "1"). Note that this table does not contain all observations from the dataset, but only the sample data points from the validation set. To make it more comparable to the Excel spreadsheet we have seen in the problem set, switch the dropdown menu from "Raw" to "Normalized" which will show you the respective percentages as seen in figure XX.



Let's compare the model against our baseline model, the prediction based solely on the fact whether the plane was Delayed more than 15 minutes upon departure. To review, see figure XX with the original confusion table.

	A	B	C	D
1	Classifier Evaluation			
2				
3			COUNT	PERCENTAGE
4	BASELINE PREDICTION	0	34.169	90,
5	ACTUAL	0	32.991	96,
6	ACTUAL	1	1.178	3,4
7	BASELINE PREDICTION	1	3.737	9,9
8	ACTUAL	0	988	26,
9	ACTUAL	1	2.749	73,
10		Total	37.906	100
11				
12		ArrDelay15	True	10
13			False	90

As you can see, our model made significant gains compared to the baseline when it comes to flights which were predicted to be delayed ("1") but were actually on time ("0"). The AI model misclassified only 0.71% observations here incorrectly where our baseline had an error of 3.4% in this category. Likewise, for flights that were predicted to be on time ("0") but actually landed with more than 15 minutes delay ("1") we could reduce the error from the baseline slightly from 26.4% to 24.94%.

Let's take a look at what this means for our acceptance criteria, the precision, recall and F1-Score. The metrics for the baseline model where as follows:

- Precision: 0.74
- Recall: 0.70
- F1-Score: 0.72

We can calculate the same metrics for our AI model by hand:

- Precision = $TP / (TP + FP) = 295 / (295 + 24) = 0.92$
- Recall = $TP / (TP + FN) = 295 / (295 + 98) = 0.75$
- F1-Score: $2TP / (2TP + FP + FN) = 2*295 / (2*295 + 24 + 98) = 0.83$

As you can see, we could improve our baseline prediction by 11 percentage points thanks to the approach of Automated Machine Learning.

Let's come back to the Precision and F1-Score metrics in the model metrics tab. Why do we see much higher values here? That is because we are provided with the micro, macro and weighted scores for our classification algorithm. These terms mean the following:

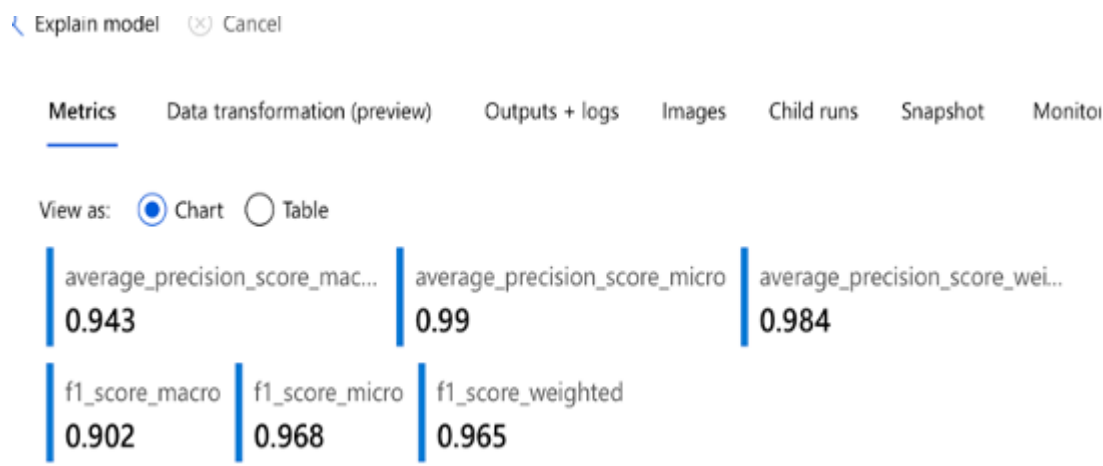
- Micro: Calculate metrics globally by counting the total true positives, false negatives and false positives.
- Macro: Calculate metrics for each label, and find their unweighted mean. This does not take label imbalance into account.
- Weighted: Calculate metrics for each label, and find their average weighted by support (the number of true instances for each label).

For imbalanced classification problems it is more informative to use a macro average where minority classes are given equal weighting to majority classes.

In the case of a binary (two classes) classification problem, the Macro F1-Score is simply the average of the F1-Score of the positive class and the negative class:

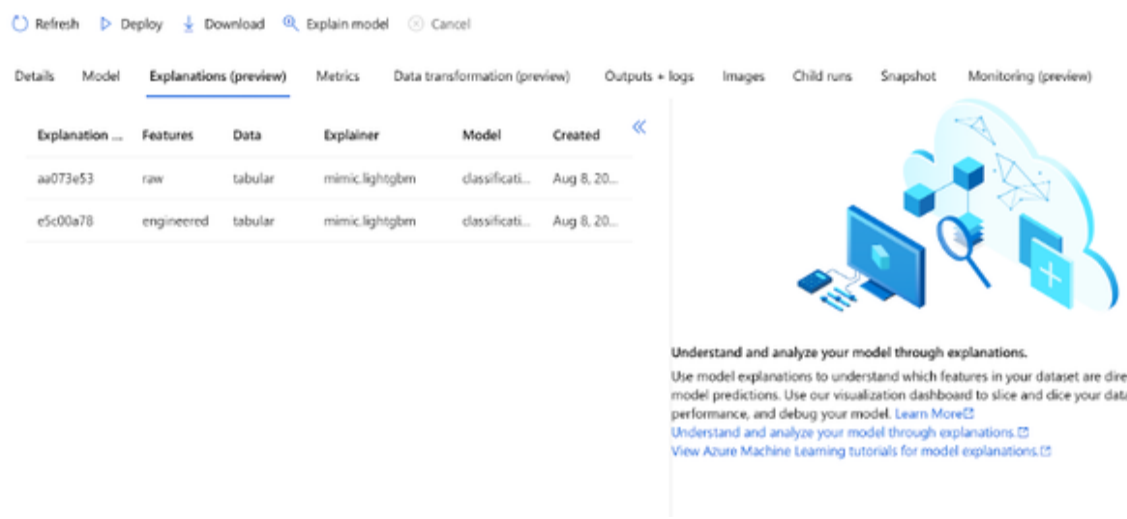
Macro F1-Score = $(\text{F1-Score of positive class} + \text{F1-Score of negative class}) / 2 = (0.83 + 0.98) / 2 = \mathbf{0.905}$. As you see, that is exactly what the

evaluation metric for the Macro F1-Score in Azure is showing us in figure XX:

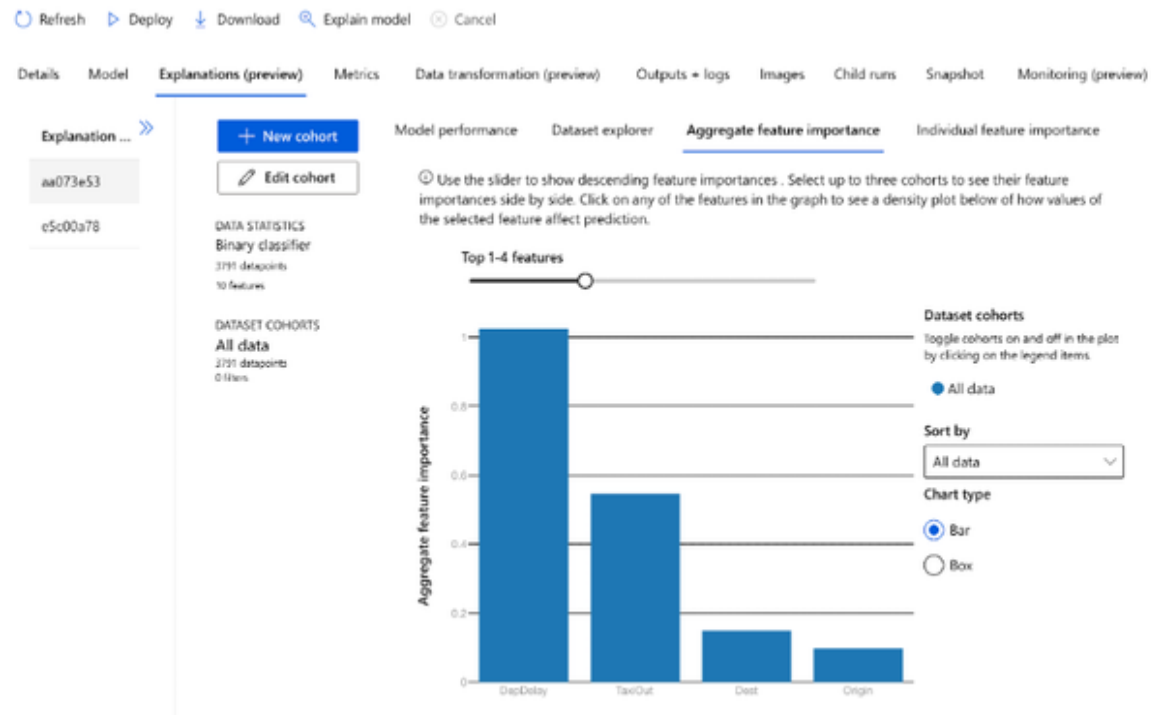


Be careful when you compare these aggregated evaluation metrics. Looking at things at a granular level, for example the confusion table, gives you a better picture of what is going on overall.

Now that we’ve got the confirmation that our model is actually working better as our naive baseline prediction, let’s understand why exactly this is the case. For this purpose, head over to the “Explanations” tab of our best model. Select “Raw” from the table in Figure XX and shrink the list pane by clicking the double arrow.



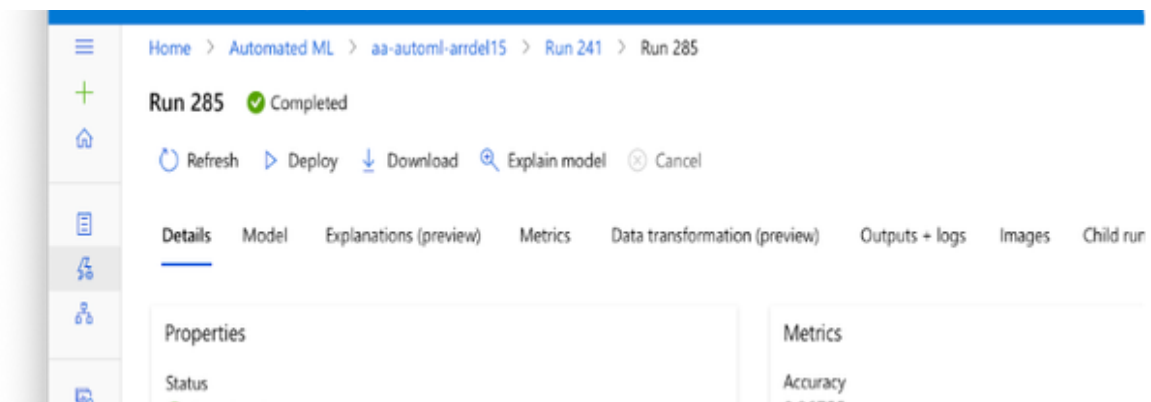
Next, click on the tab “Aggregated feature importance”. This should result in a screen similar to Figure XX. This visual briefly shows you the overall most important variables (features) that influence the outcome of a certain prediction. As we can see from the chart, the attribute “Departure Delay” is unsurprisingly by far the most important influence factor. But interestingly, there are three more variables influencing the predictions by a good bit. The first is the TaxiOut attribute which is nearly half as important as the Departure Delay attribute. This shouldn’t be no surprise since Taxi Out is actually linked very closely to the departure delay. But surprisingly we see that also the variables Destination and Origin play a role when it comes to predicting arrival delays. Depending on the place a plane takes off and the airport it is scheduled to arrive at, the probability for an on time arrival increases or decreases. These features are not as important as departure delay and taxi out, but they do play a role when it comes to making better predictions. With more than 400 unique combinations of departure and arrival airports in our dataset, this is an area where Auto ML can play out its strengths, since it would be very cumbersome for us to come up with handcrafted rules for these combinations, just to increase the overall quality of our predictions by some percent.



Now that we have the confirmation that our model works better than our baseline and we have some rough idea why this is happening, let's move ahead and deploy our model for making predictions on new, unseen datasets.

Model Deployment with Microsoft Azure Walkthrough

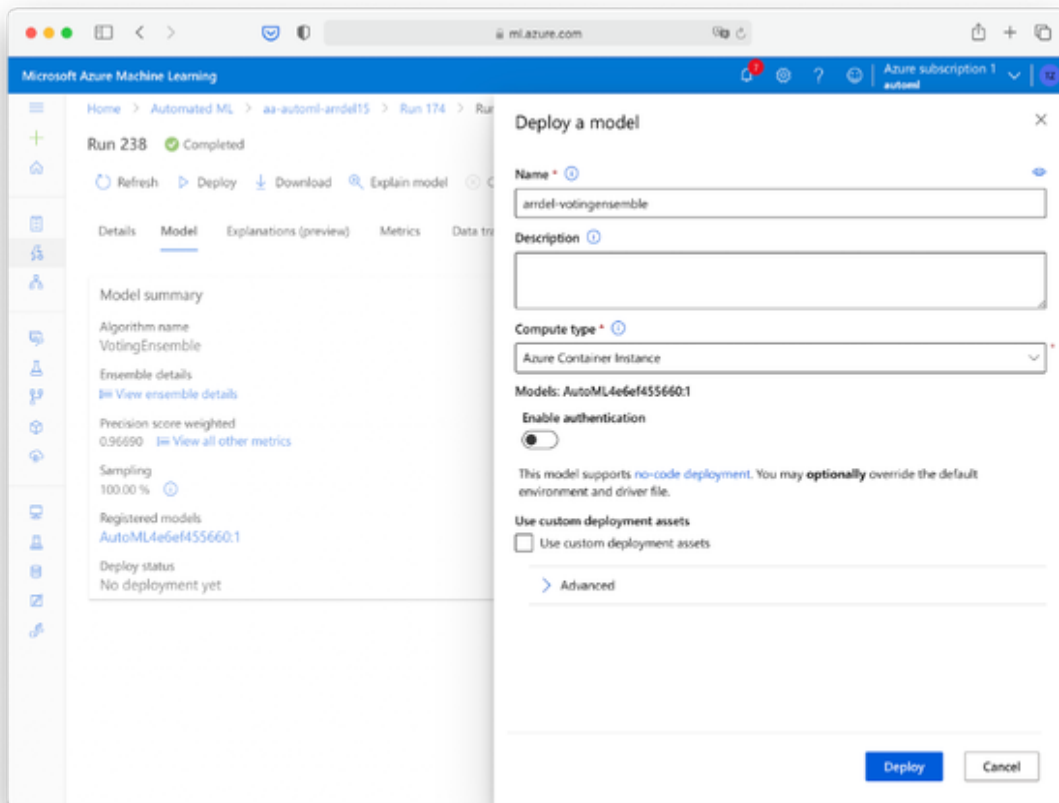
After we trained, validated and understood our machine learning model it is time to deploy it so we can use it to make predictions for new data.



In your Azure Machine Learning Studio, choose the model you want to make available and hit “deploy”. The “Deploy a model” pane will pop up on the right side as shown in Figure XX. Give your model deployment a name. I suggest using a combination of the model purpose and the model type you are using. In our case, a good model name could be “arrdel-votingensemble”. Next, select the compute type. You can choose here between Azure Kubernetes Services and Azure Container Instance. Both resources are typically used for real-time inference. The Kubernetes service is typically used for high-scale production deployments where fast response time and autoscaling are needed. That’s just the opposite of our prototyping requirements. We’re good with Azure Container service which is used for low-scale CPU-based workloads that require less than 48 GB of RAM. Make sure that authentication is disabled for now and hit “Deploy”.

NOTE

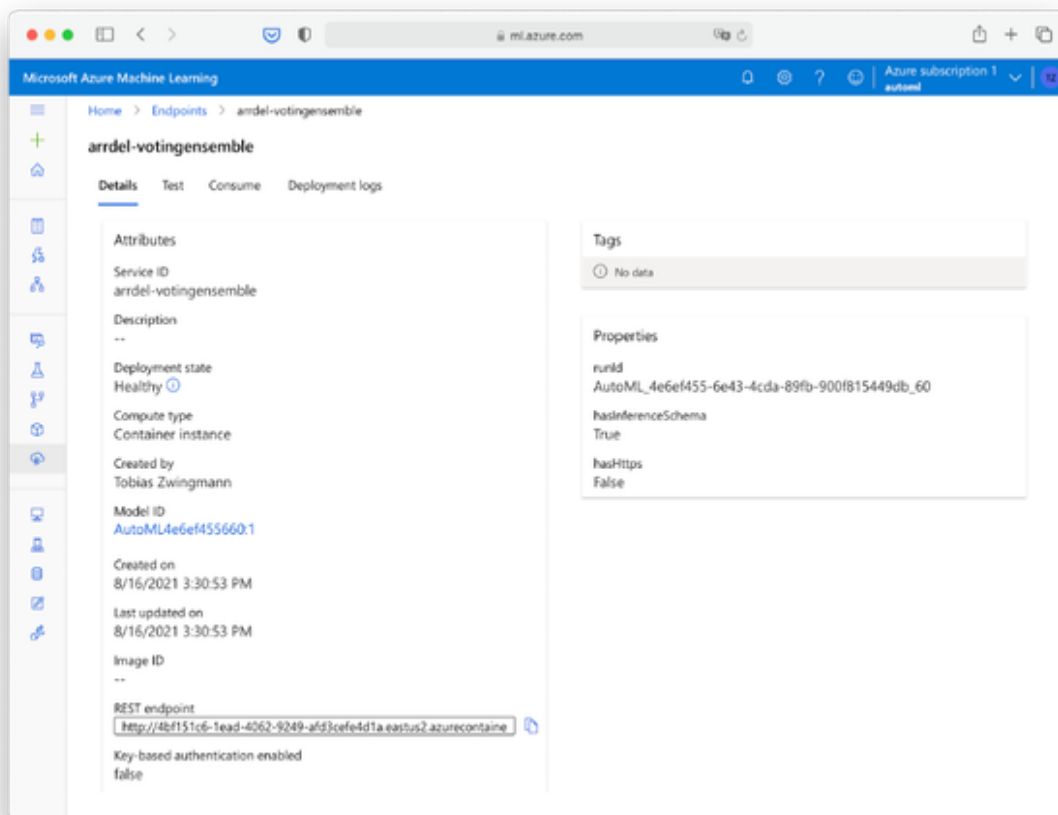
Sometimes the best model is not the best model for your purpose. Especially when the evaluation metrics are closely together, you might want to check the first three models or so and explore the details, for example the confusion table as we did before. Depending on your preference it makes sense to take the second- or third-best model if it offers a better suit to your problem at hand. Maybe you’ll find a model that offers a better Recall while sacrificing some precision?



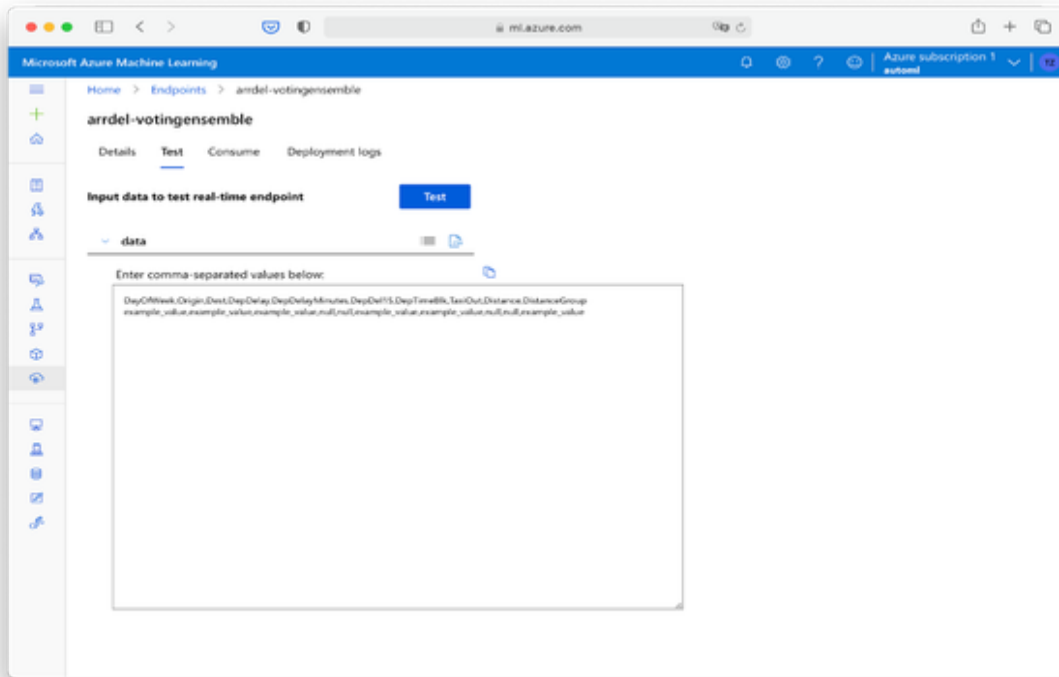
After that, you should see a notification telling you that the deployment is in progress.

After a couple minutes you should see another notification that the deployment is finished. Clicking on “Endpoints” and select your deployed model here.

Check the details of your deployed model. Most importantly check that the status of the model is showing “Healthy” as in Figure XX. Now, the model is ready to accept inference (prediction) requests. The most important parameter you will need from this page is the REST endpoint listed on the bottom of the screen. We will use this REST endpoint later.

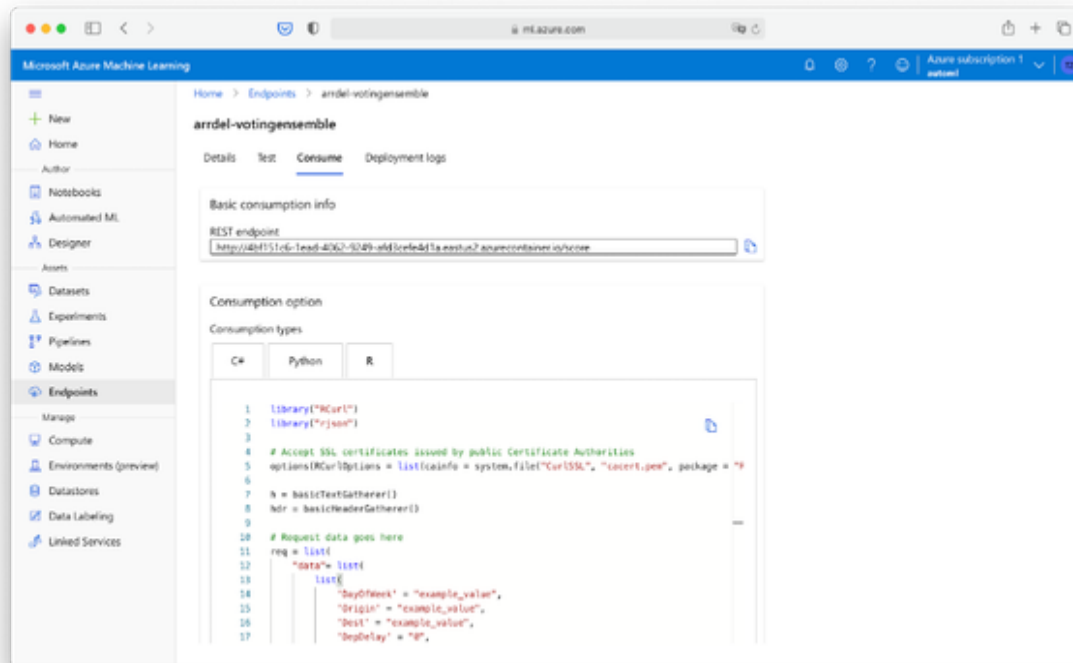


Before we continue to get predictions from our BI, let's quickly test the model using built-in testing features in Azure ML studio. From the tab menu, choose "Test". Here, you can quickly get online predictions for some sample data points. You can either input these values manually using the provided form. Or you can toggle the small CSV icon to paste tabular data as shown in Figure XX.



In the book resources, find the file “AA_Hourly_Batches_2021-02-07_0800-0859-inference-testing”..

This is one batch of hourly flight data which we want to use to make predictions about the expected arrival delay. Open the file in a text editor, select all, copy and paste the contents into the text area on the testing page. Hit the button “Test” and within a few seconds you should see the results on the right as shown in Figure XX.



The pre-written script is good enough if you want to process a single request for a single observation. If you want to process a whole table or R dataframe you would need to rewrite the script in a way that multiple records are sent to the API, and not only one. In the book resources you can find a file called “arrdel15-inference.Rarrdel15-inference.R”.

The input of this script is a data frame which contains the columns that are needed for the inference requests. The output is a modified data frame which contains all information as the original one plus one column called “ArrDel15_Prediction” which contains the actual predicted values.

The main change here is the function `inference_request` which takes a list of vectors (dataframe columns) and submits a single API request in JSON format. The output is then returned as a vector.

Replace the endpoint URL with the one from the “Consume” tab in your Azure portal. Also, if you made any changes to the dataset or chose different columns for predictions, you need to update them in the script as well, both within the function and for the actual function call.

```

# API request function
inference_request <- function(DayOfWeek, Origin, Dest, DepDelay,
DepDell5, DepTimeBlk, TaxiOut, ArrTimeBlk, Distance, DistanceGroup)
{

  # Accept SSL certificates issued by public Certificate
  Authorities
  options(RCurlOptions = list(cainfo = system.file("CurlSSL",
"cacert.pem", package = "RCurl"), ssl.verifypeer = FALSE))

  h = basicTextGatherer()
  hdr = basicHeaderGatherer()

  # Bind columns to dataframe
  request_df <- data.frame(DayOfWeek, Origin, Dest, DepDelay,
DepDell5, DepTimeBlk, TaxiOut, ArrTimeBlk, Distance, DistanceGroup)

  # Dataframe to request list of lists
  req = list("data"=apply(df,1,as.list))

  body = enc2utf8(toJSON(req))
  api_key = "" # Replace this with the API key for the web service
  authz_hdr = paste('Bearer', api_key, sep=' ')

  h$reset()
  curlPerform(
    # Insert your custom endpoint URL here
    url = "http://xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx.azurecontainer.io/score",
    httpheader=c('Content-Type' = "application/json",
'Authorization' = authz_hdr),
    postfields=body,
    writefunction = h$update,
    headerfunction = hdr$update,
    verbose = FALSE
  )

  headers = hdr$value()
  httpStatus = headers["status"]
  if (httpStatus >= 400)
  {
    return(paste("The request failed with status code:",
httpStatus, sep=" "))
  }

  result = h$value()

```

```

    result = fromJSON(fromJSON(result))$result
    result = strtoi(result)
    return(result)
}

# Submit request
result <- inference_request(df$DayOfWeek, df$Origin, df$Dest,
df$DepDelay, df$DepDel15, df$DepTimeBlk, df$TaxiOut, df$ArrTimeBlk,
df$Distance, df$DistanceGroup)

# Add column to dataset
df$ArrDel15_Prediction <- result

df

```

Model Inference with Python Walkthrough

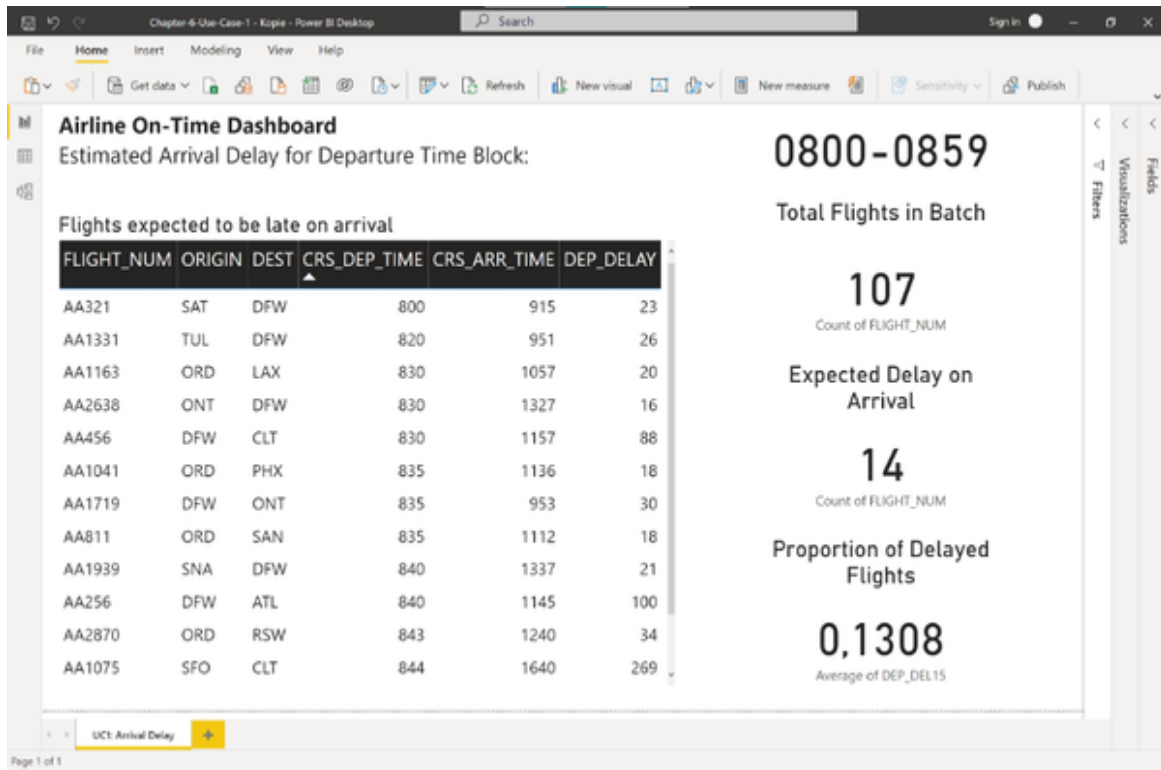
- explain model inference (API Call) in Python

Model Inference with Power BI Walkthrough

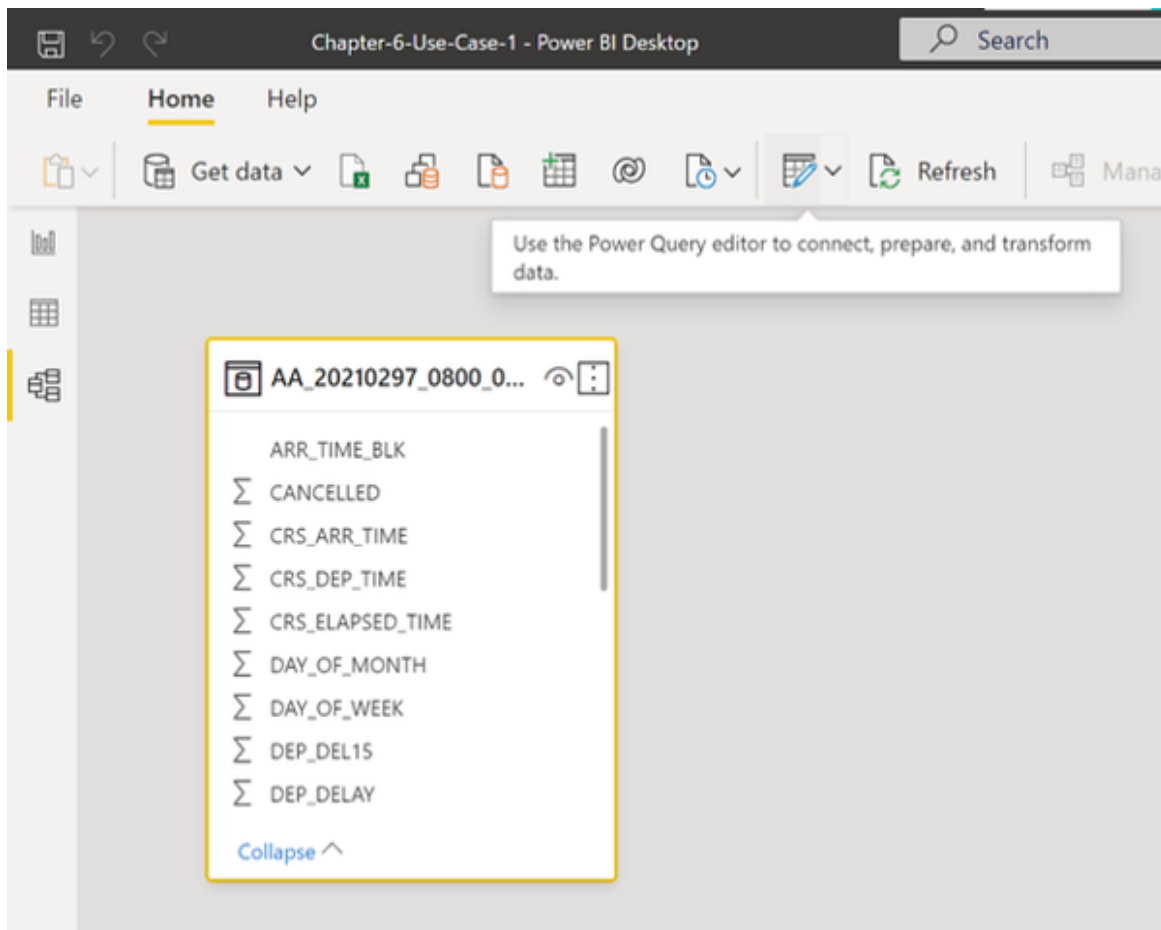
Now that we know how to get predictions from our model programmatically it is finally time to put our knowledge into action into Power BI. Note that although we are using Power BI here, you could literally use any BI tool which is able to process R or Python code.

Our goal is to enhance the arrival delay predictions for the dashboard we have found in the problem statement (see Figure XX). To recap: the Dashboard reads hourly batches of flight data and shows the expected proportion of delayed flights on arrival. We will now take this data, send it to our hosted model, get the predictions and display the aggregated score for further use and improved decision making.

To follow along in Power BI open the file “Chapter-6-Use-Case-1.pbix”. Make sure the data is properly loaded and you can see the dashboard as shown in Figure XX.



To add our AI predictions, we have to apply some data transformations. Head over to “Model” on the left and select Power Query from the toolbar as shown in Figure XX:



This will open the Power Query Editor. This tool allows you to manipulate your data, apply calculations, transformations and: execute Python and R scripts!

From the toolbar, choose Transform and then Run R script or Run Python script, depending on what you prefer (see Figure XX). For this tutorial I will continue with the example of R.

Chapter-6-Use-Case-1 - Power Query Editor

Queries [1]

AA_20210297_0800_0859

	DAY_OF_MONTH	DAY_OF_WEEK	FL_DATE	OP_UNIQUE_CARRIER	OP_CARRIER_FL_NUM
1	7	7	07.02.2021	AA	
2	7	7	07.02.2021	AA	
3	7	7	07.02.2021	AA	
4	7	7	07.02.2021	AA	
5	7	7	07.02.2021	AA	
6	7	7	07.02.2021	AA	
7	7	7	07.02.2021	AA	
8	7	7	07.02.2021	AA	
9	7	7	07.02.2021	AA	
10	7	7	07.02.2021	AA	
11	7	7	07.02.2021	AA	
12	7	7	07.02.2021	AA	
13	7	7	07.02.2021	AA	
14	7	7	07.02.2021	AA	
15	7	7	07.02.2021	AA	
16	7	7	07.02.2021	AA	
17	7	7	07.02.2021	AA	
18	7	7	07.02.2021	AA	
19	7	7	07.02.2021	AA	
20	7	7	07.02.2021	AA	
21	7	7	07.02.2021	AA	
22	7	7	07.02.2021	AA	
23	7	7	07.02.2021	AA	
24	7	7	07.02.2021	AA	
25	7	7	07.02.2021	AA	
26	7	7	07.02.2021	AA	

21 COLUMNS, 107 ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED ON MONTAG

Query Settings

PROPERTIES

Name

AA_20210297_0800_0859

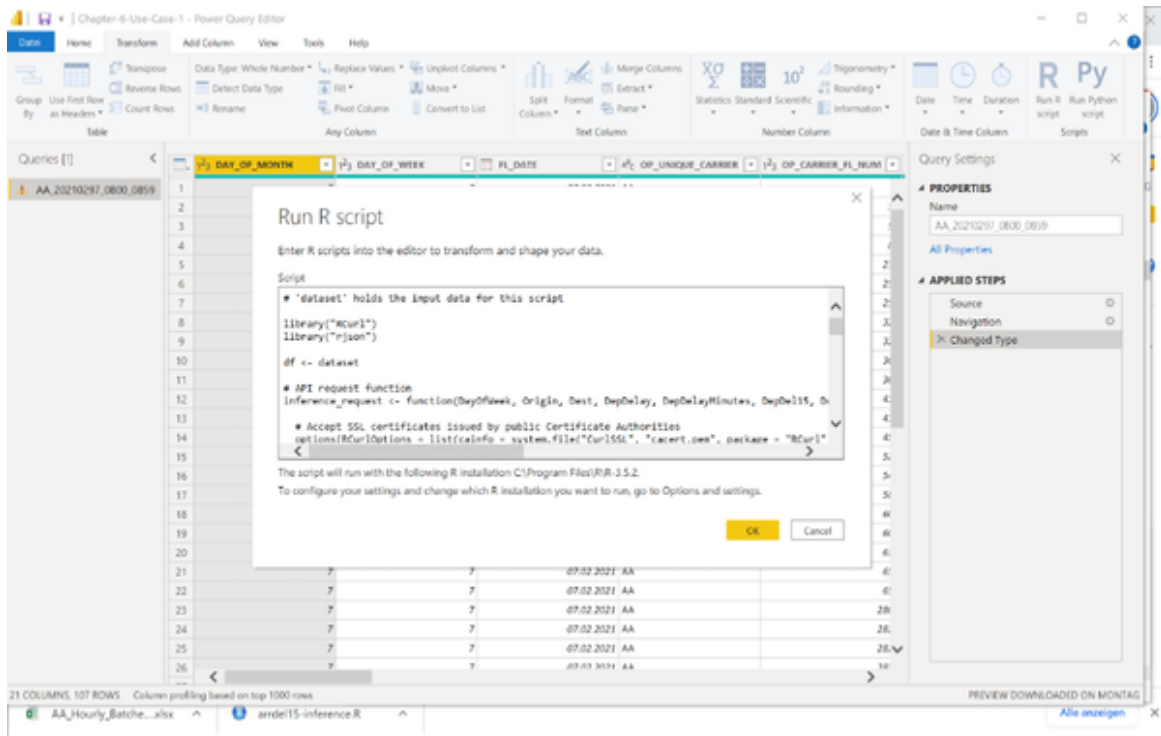
APPLIED STEPS

Source

Navigation

Changed Type

In the code editor, just paste the code from the arrdel-inference.R file as shown in Figure XX. Double check that you have replaced the endpoint URL with the one from your model as shown previously when we discussed the script.



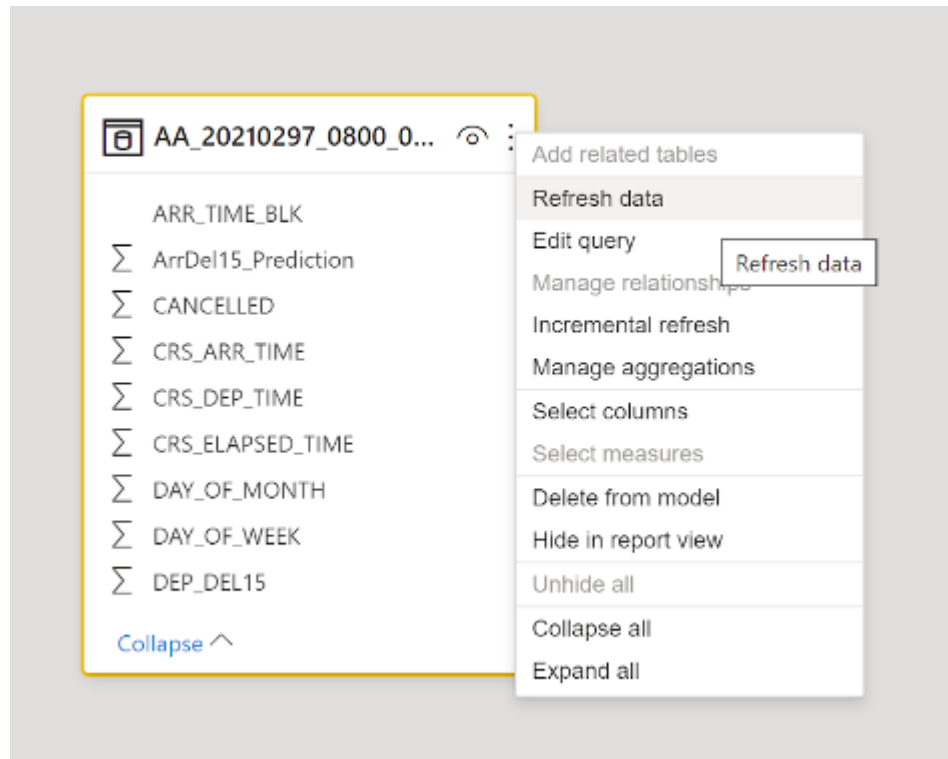
Hit OK and Power BI will evaluate the script. This should take a moment depending on how many rows you send to the API endpoint. Remember we are still in the phase of prototyping here. In a production setting you would rather apply these AI predictions on a database level and just consume it from your Power BI or reporting system. Check out chapter 10 for more details. After it is completed you will notice two things in the editor: First, there is another step added under the “Applied Steps” pane on the right. And second, you will see a new column “ArrDel15_Prediction” if you scroll to the right as seen in Figure XX.

The screenshot shows the Power Query Editor interface. The ribbon at the top has several groups: 'Text Column' (Split Column, Format, Merge Columns, Extract, Parse), 'Number Column' (Statistics, Standard, Scientific, Rounding, Information), and 'Date & Time Column' (Date, Time, Duration). There are also buttons for 'Run R script' and 'Run Python script'. The main area displays a table with the following data:

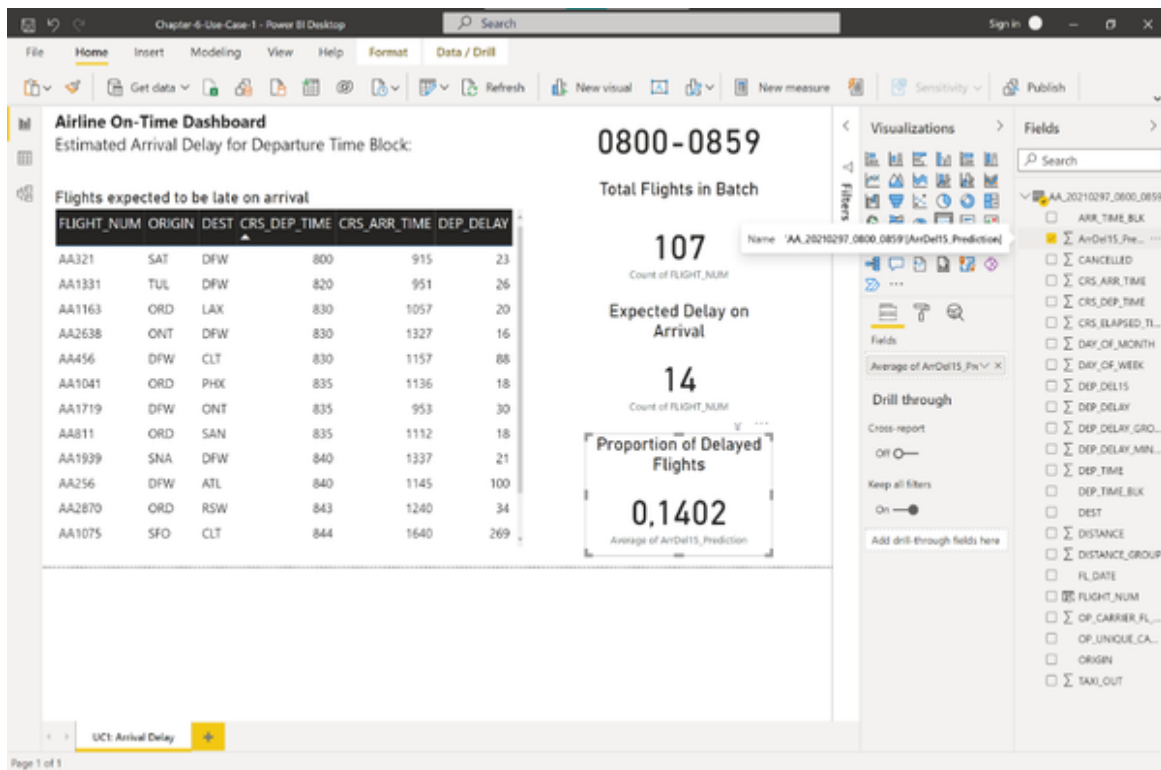
	1^{23} DISTANCE_GROUP	1^{23} ArrDel15_Prediction
2475	10	0
2556	11	0
1055	5	0
1007	5	0
2486	10	0
2486	10	0
731	3	1
247	1	1
1927	8	0
190	1	0
2342	10	0
1085	5	0

The right sidebar shows 'Query Settings' with a 'Name' field containing 'AA_20210297_0800_0659'. Below this are 'APPLIED STEPS' including 'Source', 'Navigation', 'Changed Type', 'Run R script', and a selected step 'df'.

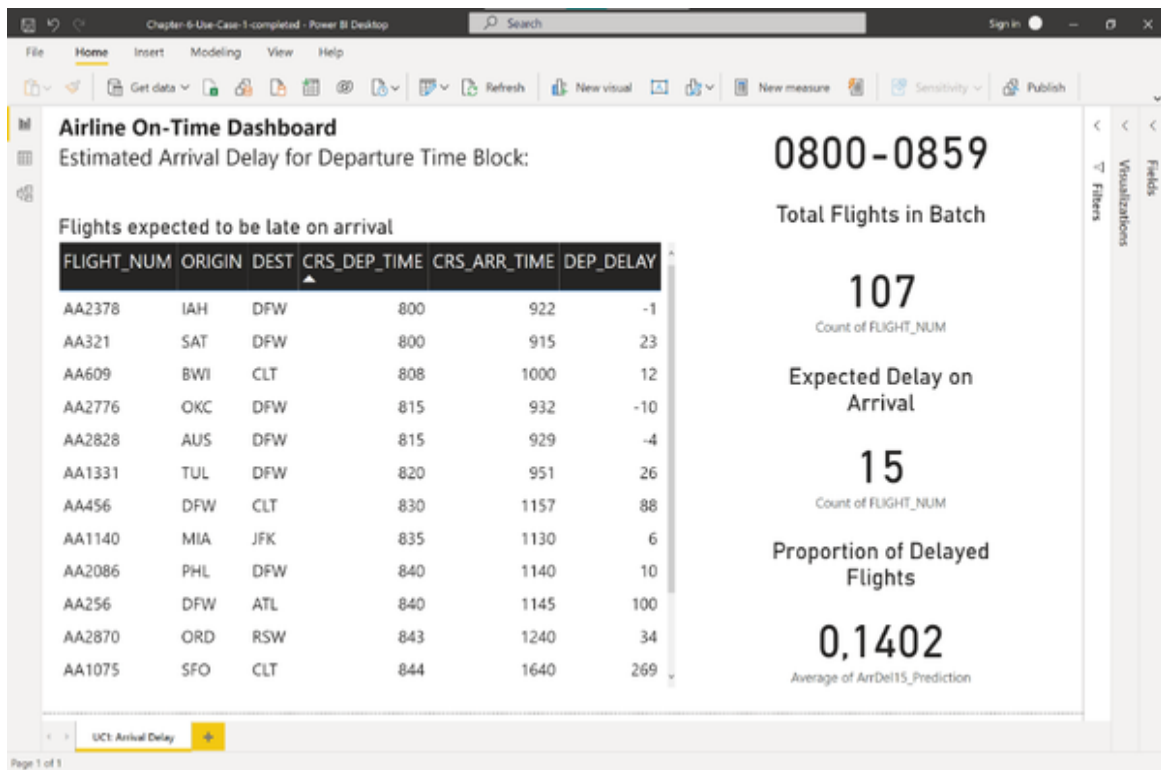
In the Power Query Editor select File and then Close and Apply. Your data model is being updated and populated with the new additional information. You can see the new column in the updated model. Whenever you want to update the data or the predictions, you need to refresh the data model as shown in Figure XX. This will reload the data from the file or an external source and re-run the steps in the Power Query Editor.



We are now ready to populate our dashboard with the new information. Go back to Report and select the Card visual with the “Proportion of Delayed Flights”. Drag and drop the new dimension “ArrDel15_Prediction” from the fields pane to the value field of the card visual replacing the old metric DEP_DEL15. Right click the on the updated field and select “Average”. The settings pane should now look like in Figure XX and the metric should update to 0.1402.



Go ahead and replace the DEP_DEL15 field with the new ArrDel15_Predicted dimension for the rest of the visuals in the dashboard. This includes the filter value for the Card visual “Expected Delay on Arrival” as well as the filter on the flights table. Alternatively, you can open the file Chapter-6-Use-Case-1-completed.pbix and follow along from there. Your final dashboard should look like in Figure XX.



From the count of delayed flights alone we might intuitively assume that the predictions from our baseline and the AI model are not that different. But don't fall for that trap! Just because the AI predicts one more flight in total to be delayed, the actual flights that are expected to be delayed are quite different. Have a look at the flights table. We can see in the AI-powered dashboard flagged flights which did not have a departure delay of 15 minutes. Examples include flights AA2378 from IAH to DFW, flight AA2776 from OKC to DFW and flight AA2828 from AUS to DFW that even started ahead of time.

As we know from the real dataset as shown in Figure XX we can confirm that flights AA2378 and flights AA2828 were in fact delayed by more than 15 minutes. Flight AA2776 did not have the ArrivalDelay15 flag, but the plane landed 12 minutes late, despite having a head-start of 10 minutes. While technically the AI prediction here was wrong, it was definitely a good guess.

You can check these and further results by exploring the table AA_Hourly_Batches_2021-02-07_0800-0859-ground_truth.xlsx from the

book resources.

FL_DATE	OP_	OP_CAR	ORIGIN	DEST	CRS_	DEP_	DEP_	DEP_	CRS_ARR	ARR_T	ARR_	ARR_
	UNI	RIER_FL			DEP_	TIME	DELAY	DEL15	_TIME	IME	DELAY	DEL15
	QUE	_NUM			TIME							
07.02.21	AA	2828	AUS	DFW	815	811	-4	0	929	954	25	1
07.02.21	AA	2378	IAH	DFW	800	759	-1	0	922	1003	41	1
07.02.21	AA	2776	OKC	DFW	815	805	-10	0	932	944	12	0

I hope you enjoyed building this dashboard and found out how AI can help you to make better predictions for your dataset. Keep track of a baseline and see if and how well AI can possibly outperform this.

NOTE

When you look carefully at the flights table you will find a small hitch of our approach which is also very likely to happen in real life. See for example the last row, the flight from SFO to CLT. The flight is 269 minutes delayed. This is what we call a training-serving-skew. When this was real-world data, this information would actually only be available almost 4.5 hours later. So depending on which point in time we are looking at the data we might have information available or not. In training we have all information available. In reality, though we don't. There are different approaches we could combat this issue, for example introducing a „minimum delay“ which counts the delay „up to now“ or by not serving a prediction for these flights. Or we could take off the “Departure Delay” variable from our AI model and just keep the field “Departure_Delay_15 as a proxy for the flight delay (in addition to all other attribute) and see how our AI model performs. When you decide to put a prototype into production, these are the questions you want to ask: How often do we get the data? Is the frequency enough? To which extent will the business benefit from a better prediction and is it worth the effort of scaling up the data pipeline? These are the questions you want to discuss and focus on. Thanks to your prototype you don't need to discuss whether AI could beat the current baseline. You build the proof of concept yourself. Now it's time to focus on the actual details of the implementation work.

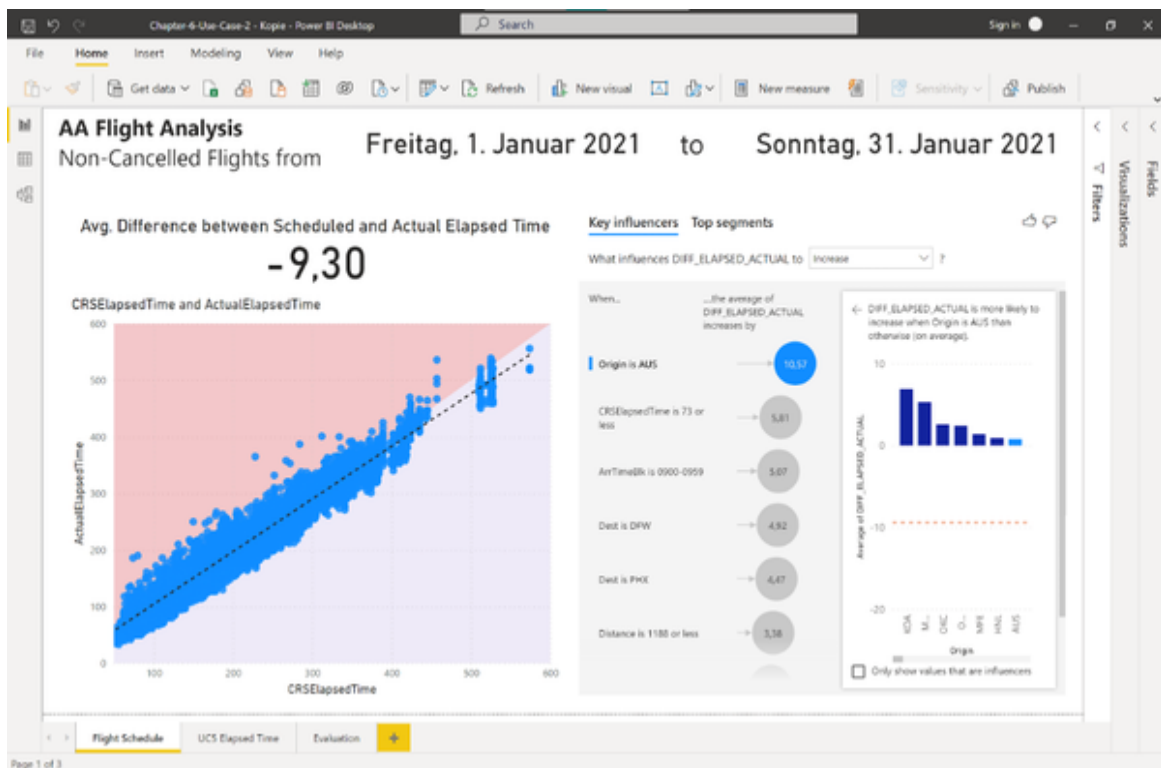
Use Case: Improving KPI Prediction

For this use case we will continue with the scenario from before: We are on the BI team of American Airlines. Only this time we don't want to improve our live reporting, but we want to improve our planning even ahead of time.

For that purpose we will need to deal with a metric called “Elapsed Time” which measures the difference between actual departure time and actual arrival time of a given flight. The elapsed time is planned well ahead and called CRS Elapsed Time. In contrast to the use case before, this is not a categorical, but a continuous numerical variable. Welcome to the world of regression problems! Let’s find out more about the problem by introducing the problem statement as follows.

Problem Statement

We want to identify bottlenecks in the flight schedule for the next month and flag those flights that have a high chance of being delayed, meaning the actual elapsed time was higher than the planned elapsed time. The BI team has built a dashboard which analyses historical data from the past and contrasts CRS (scheduled) Elapsed Time with Actual Elapsed Time. You can see this dashboard in Figure XX or open it yourself in Power BI with the file “Chapter-6-Use-Case-2.pbix”.

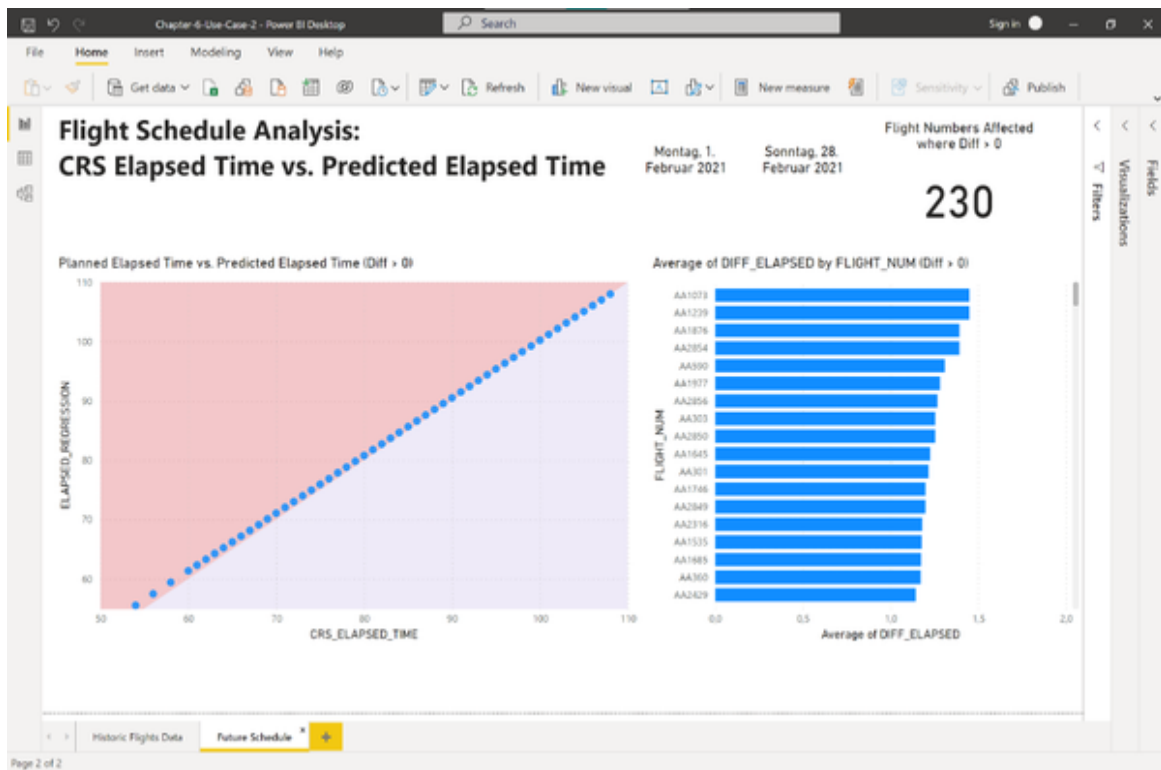


From the dashboard we found out that on average flights need 9.30 less than actually planned. This intuitively makes sense as we know that the schedule is optimized to minimize delays. At the same time, we can see from the scatterplot on the left that there are flights which exceed the planned elapsed time. The scatterplot shows CRS Elapsed Time vs. Actual Elapsed Time. Each point is a flight. Points in the upper dark area of the plot needed more time than originally scheduled. The key question is: Are we able to create a model which will flag these flights at times when the flight schedule is generated?

Based on the historic data the analysts have built a very simple regression model which is also shown as a trend line in the dashboard above. The model takes the scheduled CRS Elapsed time and calculates a prediction for the actual elapsed time based on the historic data. You can see the regression model in Figure XX and in the worksheet “Regression Model” of the file “AA_Flights_2021_1.xlsx”. The regression model has an R² value of 0.96 and a Standard Error (RMSE) of 14.3. That means that [Explain error metrics here]. While the model seems to be good at the paper it turns out to be quite useless for predicting the actual elapsed time.

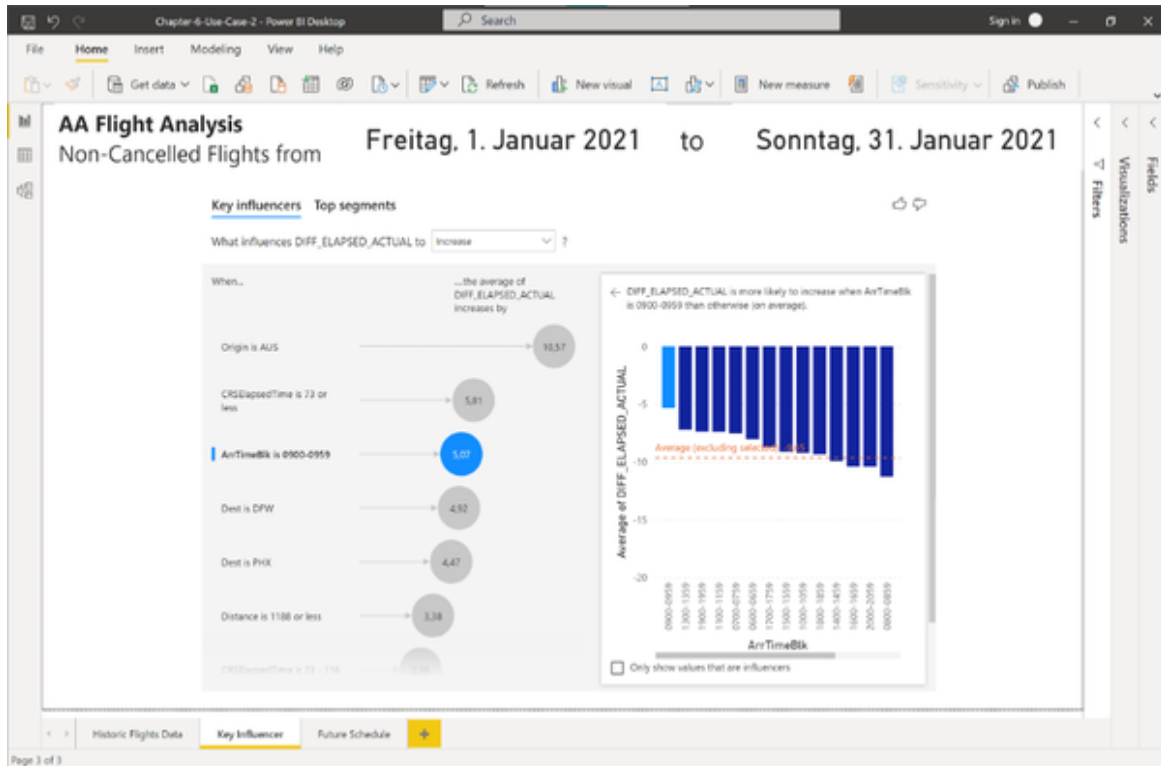
A	B	C	D	E	F	G	H	I
SUMMARY OUTPUT								
<i>Regression Statistics</i>								
Multiple R	0,98047362							
R Square	0,96132851							
Adjusted R Square	0,96132749							
Standard Error	14,3004254							
Observations	37906							
ANOVA								
	<i>df</i>	<i>SS</i>	<i>MS</i>	<i>F</i>	<i>Significance F</i>			
Regression	1	192692108	192692108	942249,708	0			
Residual	37904	7751450,18	204,502168					
Total	37905	200443558						
	<i>Coefficients</i>	<i>Standard Error</i>	<i>t Stat</i>	<i>P-value</i>	<i>Lower 95%</i>	<i>Upper 95%</i>	<i>Lower 95,0%</i>	<i>Upper 95,0%</i>
Intercept	-4,4558099	0,19001978	-23,44919	9,781E-121	-4,8282538	-4,0833661	-4,8282538	-4,0833661
X Variable 1	0,97229165	0,00100164	970,695476	0	0,9703284	0,9742549	0,9703284	0,9742549
Squared Errors	7751450,18							
Root Mean Square Error (t	14,300							

If we apply the model to the flight schedule for February 2021, the model will flag 230 flight numbers where the actual elapsed time will probably exceed the scheduled elapsed time. However, by looking at the bar chart on the right we can see that the biggest difference between the scheduled elapsed and actual predicted elapsed time is only very marginal with 1.45 minutes more than planned. That is the highest difference the regression model can predict. Look at the scatterplot on the left. It shows all flights where the expected elapsed time exceeds the scheduled elapsed time. You can see that all of these flights are actually very close to the threshold of being exactly in line with the scheduled elapsed time.



The regression model that has been built is just not flexible enough to capture variations in the elapsed time that are caused by other variables than the scheduled CRS time. But what are these influencing variables anyway? The BI team has further conducted an analysis and identified key drivers that cause the difference between the scheduled elapsed time and the actual elapsed time to increase. You can find a screenshot of this analysis in Figure

4 or in the report page “Key Influencer” in the Power BI file “Chapter-6-Use-Case-2.pbix”.



From the analysis we found out that a variety of different factors drive the discrepancy between scheduled and elapsed time. Among them are the origin airport, flight distance, destination airport, arrival time block and the weekday. So, you probably don't want to schedule a meeting shortly after arrival if you take a flight to Dallas which is expected to land between 09.00 am - 09.59am. But how can we as an Airline turn these insights into actionable information for our customers? How can we flag those critical bottlenecks in a flight schedule and surface them automatically without having to run through the key influencer analysis over and over again?

Solution Overview

In order to identify bottlenecks in the flight schedule we will take all information that is available to us when the schedule is created and train a machine learning model on historical data to predict the actual elapsed time using this information. The information included attributes such as Origin

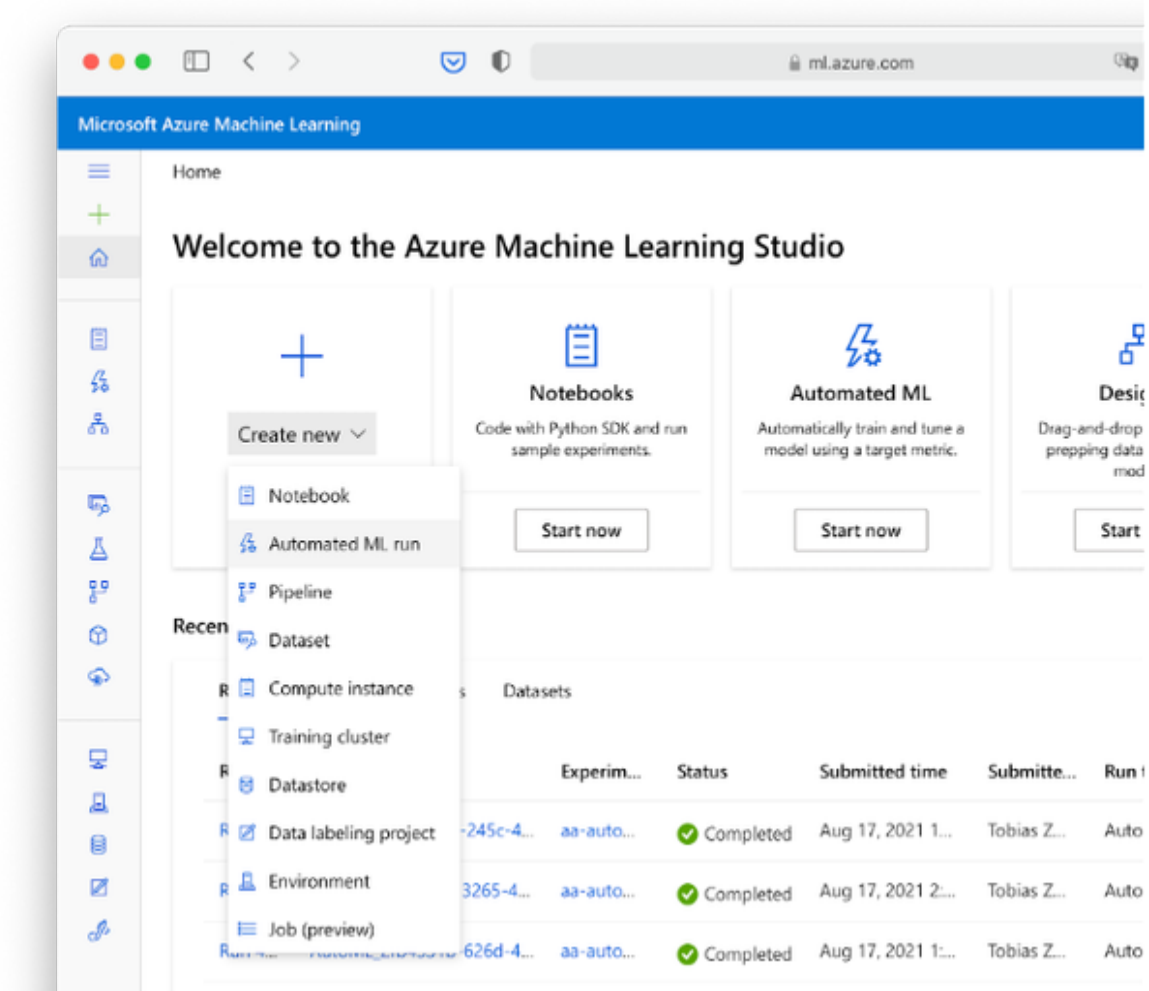
airport, destination airport, arrival and departure time blocks and more. All these attributes are known well ahead of time so we don't run into a training-serving skew and can react to the model outputs well ahead of time.

We are again using Microsoft Azure Auto ML to build a custom machine learning model for us and deploy it on the Azure cloud. Then, we will get inference from that model using a R or Python script which we call from within our BI tool. In this case, we are also using Power BI to make the request, but it is generally possible for other BI tools as well.

Model Training with Microsoft Azure Walkthrough

Head over to Azure Machine Learning Studio by navigating to ml.azure.com. If you skipped the use case before, then please set up your Azure environment accordingly as mentioned in the beginning of this chapter.

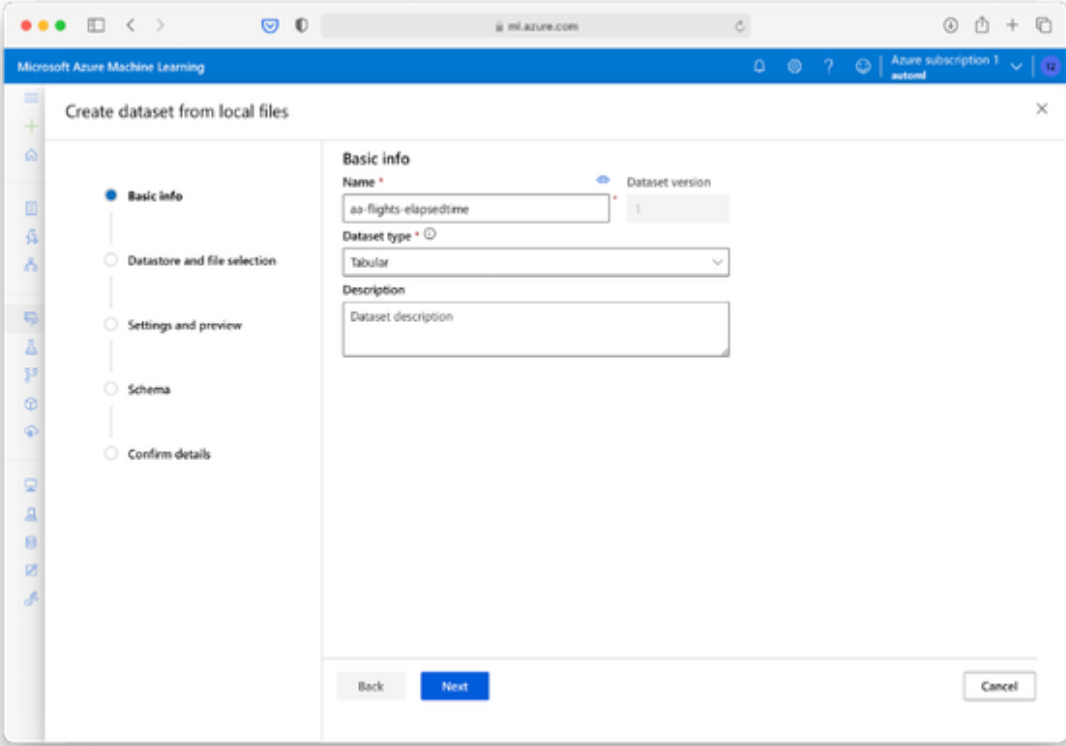
From the home screen, select “New” -> “Auto ML run” as shown in Figure XX



Choose “Create Dataset” -> From local file. You will see the form to provide basic information about your dataset as shown in Figure XX. Give your dataset a name which you can relate to the Auto ML training job. In my case I chose “aa-flights-elapsedtime”. Proceed with “Next”.

NOTE

While we are technically using the same data source as in the previous use case it is still a good idea to re-upload the file as a separate dataset in Microsoft Azure in order to keep things organized. The training dataset we are using now has a different schema than the training dataset from the first use case. For example, this time we are predicting the “Actual Elapsed Time” attribute. You can also update the schema in Azure for a dataset by adding a new version, but this will be complicated to track if you want to re-train your ArrDel15 classifier from the use case before. A good rule of thumb: Keep a separate dataset for each Auto ML task, meaning that the target column should stay fixed. Use the schema update function and version control to add or remove features or tweak the data types of your input data, but don’t touch the target column.



The screenshot shows the 'Create dataset from local files' wizard in the Microsoft Azure Machine Learning interface. The 'Basic info' step is selected in the left-hand navigation pane. The main form area contains the following fields:

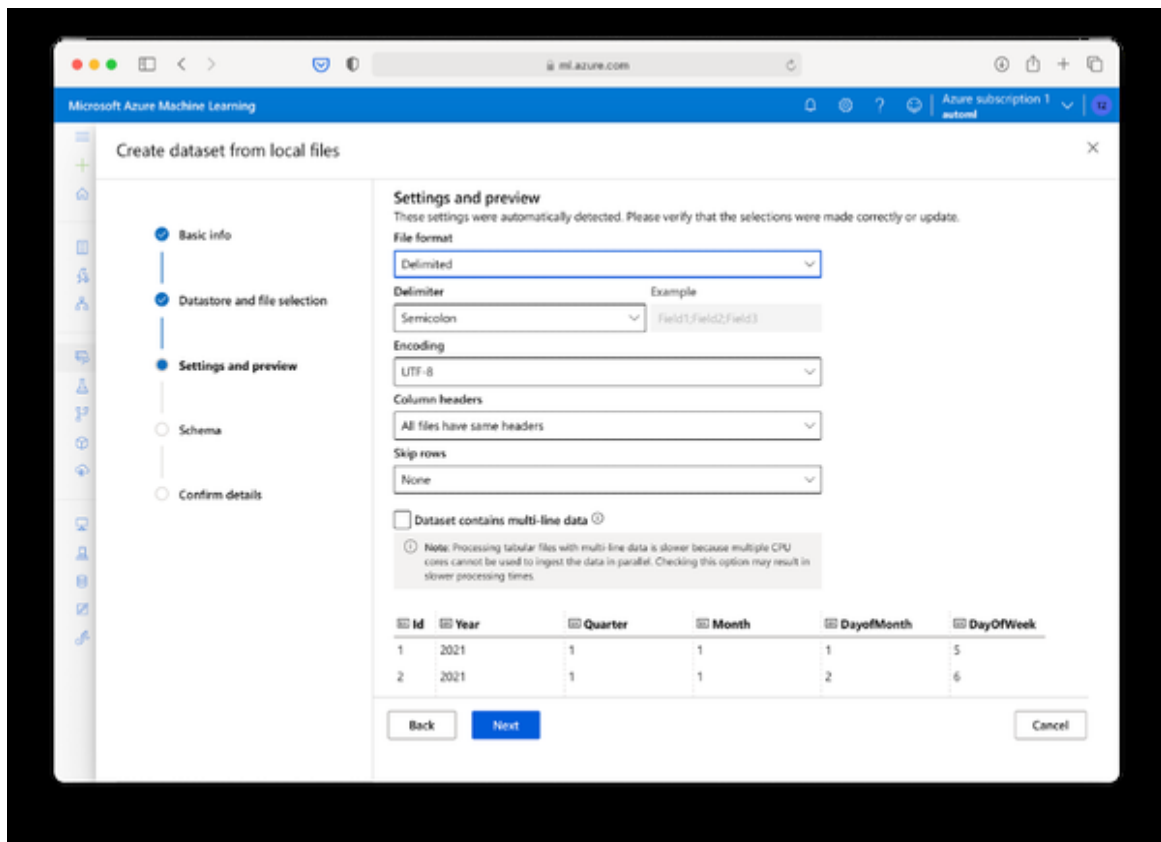
- Name:** A text input field containing 'aa-flights-elapsedtime'.
- Dataset version:** A numeric input field containing '1'.
- Dataset type:** A dropdown menu set to 'Tabular'.
- Description:** A text area with the placeholder text 'Dataset description'.

At the bottom of the form, there are three buttons: 'Back' (disabled), 'Next' (active/highlighted in blue), and 'Cancel' (disabled).

Click the “Upload” button and select the file “AA-Flights-2021-01-Training.csv”. Once the upload is complete the button “Next” should appear on the bottom of the screen.

Proceed here to continue to the “Settings and Preview” form. Double check that the file format is set to “Delimited”, the Delimiter to “Semicolon” and

the encoding to “UTF-8”. You should see a properly formatted preview of the data below as shown in Figure XX.

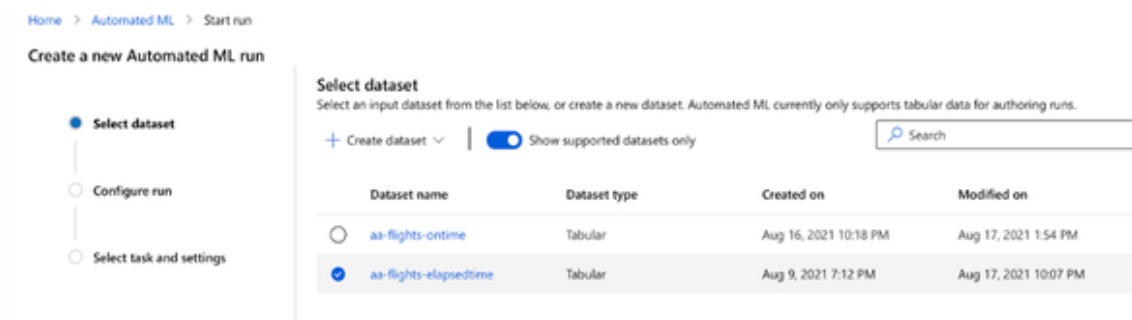


Click next to proceed to the schema settings. Enable just the following attributes with the according data type:

- DayOfWeek: String
- Origin: String
- Dest: String
- DepTimeBlk: String
- ArrTimeBlk: String
- CRSElapsedTime: Decimal (Comma)
- ActualElapsedTime: Decimal (Comma)
- Distance: Decimal (Comma)

- DistanceGroup: String

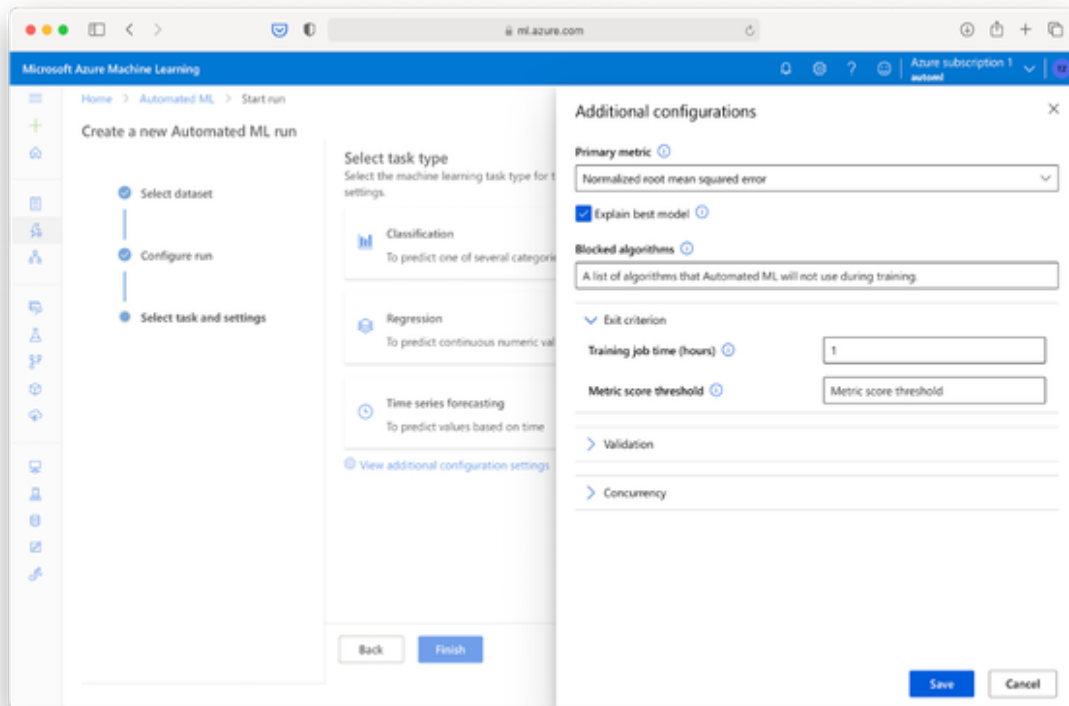
Choose Next, confirm the details and you should see the new dataset appear in the list of available datasets as shown in Figure XX.



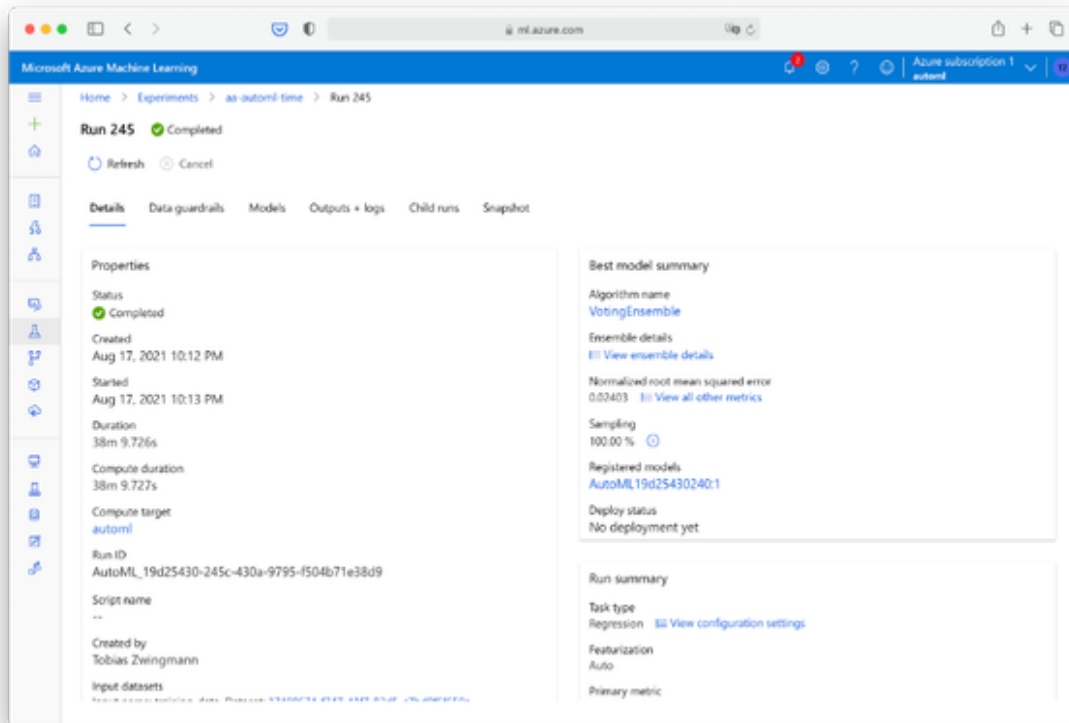
Select the new dataset and click Next. In the following screen I suggest to create a new experiment to keep the models and results from the classification and regression tasks separated. Choose an experiment name which will differentiate well from the previous experiment, such as “aa-automl-time”. As the target column choose “ActualElapsedTime (Decimal)”. For the compute resource, you can use the same resource that we have created in the previous use case “automl”. Your settings should look like in Figure XX. Proceed with “Next”.

In the following screen, Auto ML will guess that you are trying to solve a regression problem as you specified a continuous numeric target variable. In our case, this is absolutely correct! Before we finish it off, open the advanced settings by clicking “View additional configuration settings”.

In the additional configurations make sure that the Primary metric is set to Normalized root mean squared error. This way we can compare the Auto ML model pretty well with the existing baseline from the regression model we already have. Also double check that the option “Explain best model” is checked. Under exit criterion set the training job to a maximum of 1 hour. In fact, we probably don’t even need that but the algorithm should converge after roughly 30 minutes. Confirm the settings with save and submit the Auto ML run by pressing “Finish”.



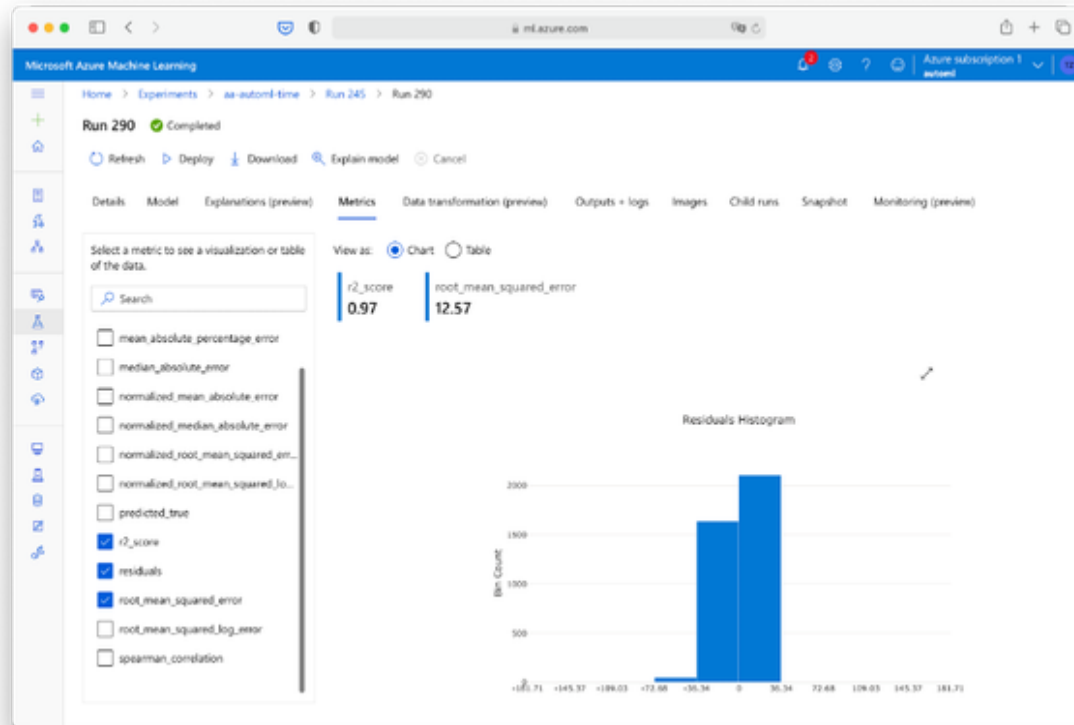
Take a break and after some time, check the status of your run by going to Experiments -> aa-automl-time (or whichever name you gave to it) and examining the status of your current run. If you see the green checkbox here, click on the Run to proceed to the Run details as shown in Figure XX.



As you can see, in my case the overall run took about 38 minutes and it resulted in a model that which uses VotingEnsemble as an algorithm - again! And this should be no surprise to you. In fact, AutoML often comes up with a VotingEnsemble or Stacked Ensemble as the final model. Both are techniques which combine the results of a number of different models that either “vote” for the final result or are “stacked” on top of each other to come up with the final decision. It is very common for Auto ML algorithms to end up with ensemble models.

In our case, the model achieved a Normalized RMSE of 0.2403. How can we compare that to the RMSE baseline of 14.30 from our simple regression model?

Click on the link “VotingEnsemble” and choose metrics. Uncheck all boxes except for the R2-Score, the residuals and the Root Mean Squared Error as shown in Figure XX

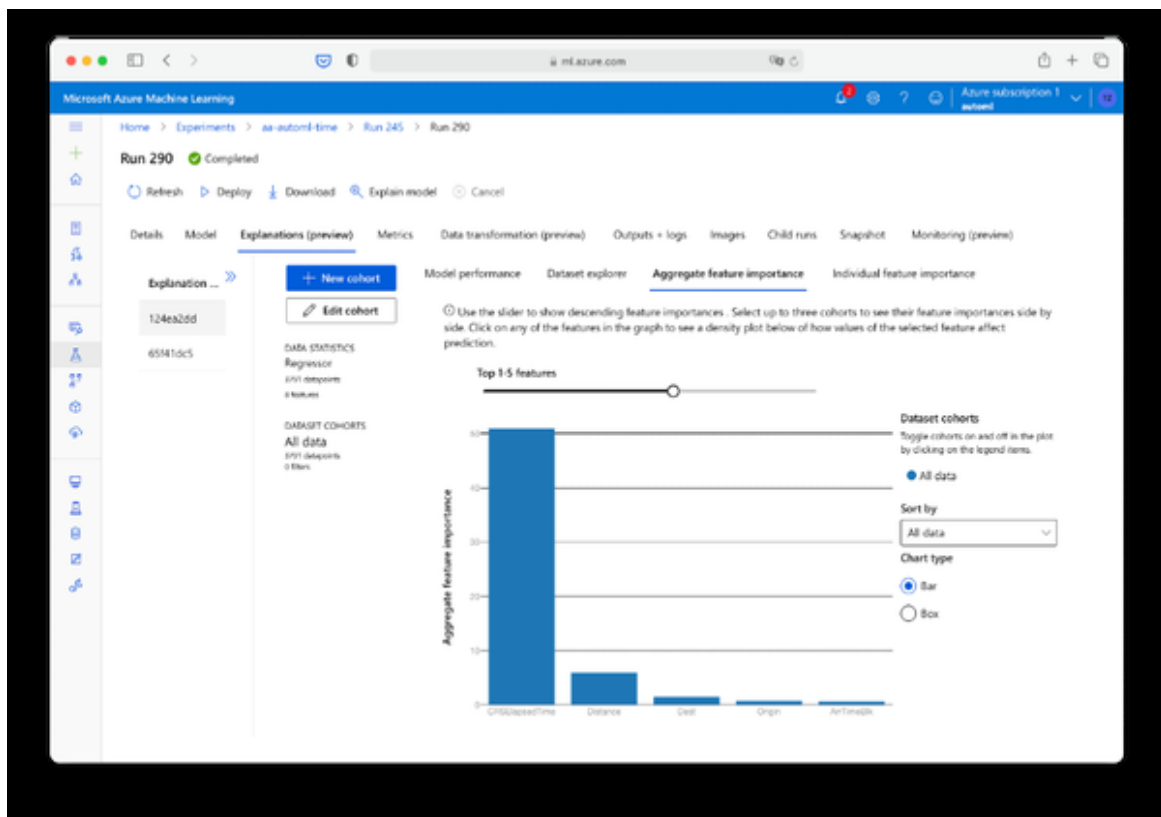


From the metrics you can see that the RMSE of our Auto ML regression model is 12.57. That is less (and therefore better) than the simple regression model, but actually not that much considering the amount of computing power we threw at this problem. But as we have learned previously from the first use case, we should be sceptical about aggregated quality metrics and rather look at the results in a bit more detail.

Take a look at the Residuals histogram. A residual is the error a regression model makes, that is the difference between the actual and the predicted outcome. In case of our simple regression model, the regression almost always predicted less than what was actually the real value. In our Auto ML scenario here we can see that the residuals are centered around the 0 value with some predictions being a little bit on top of the actual metric and some predictions below that. This type of variance is actually a good sign, because this variation is what we need to identify flights that have a high probability of exceeding the scheduled elapsed time. So with some good faith into our Auto ML model and the confirmation that at least on paper it

is better than our regression baseline, head over to the Explanations tab to find out which factors are influencing our predictions.

In the Explanations tab select the first list item with the raw features. Then select the tab “Aggregated feature importance” from the sub menu. Select the Top 5 features and you should see a screen similar to Figure XX:



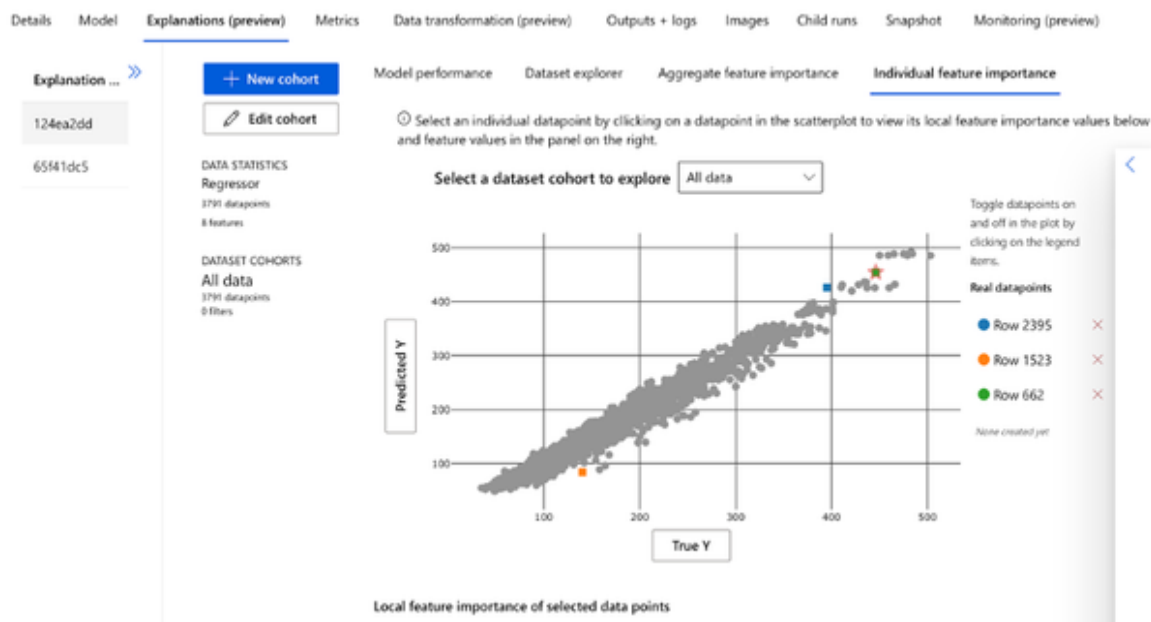
In my case, the Top 5 features were CRS Elapsed Time (No surprise, by far) and Distance, Dest, Origin and ArrivalTimeBlk. This selection makes sense from a logical standpoint. Bigger airports are busier than smaller ones, so there should be a higher probability that a delay happens. Also the ArrivalTimeBlk confirms what we have seen from the Key influencer tools above. There are certain arrival times, especially in morning hours, when airports are typically busier and delays are more likely to happen. Note that the actual results in your case might look a little different as there are some random elements involved in the Auto ML process (for example the way the data is split into training, testing and validation sets). Unfortunately we can't set the seed for these random procedures fixed in Azure so we have

100% reproducible results. But the overall picture should still remain the same.

Let's take a minute to understand our model even a little better and see what is going on. Head over to the tab "Individual feature importance". In the following scatter plot, click on the title of the Y axis and choose "Predicted Y" and do the same for the X-Axis choosing "True Y". This will give us an overview of all data points (flights) and their corresponding true and predicted value for the Elapsed Time attribute. The great thing about this visualisation is that we can individually select single data points and inspect the concrete feature importance which led to the individual predicted result. Choose three data points from this chart:

- One point where the predicted value was higher than the actual value,
- one point where the predicted value almost matches the actual value and
- one point where the predicted value was much lower than the actual value.

See Figure XX for how this looks like for my selection:



To provide a little more context you can also open the menu pane on the right to reveal more details about the data points. For example, data point Row 662 (Green) was a flight from Honolulu Airport to Dallas Fort Worth departing on a Saturday evening and landing in the morning, as you can see in Figure XX. The prediction for this point was pretty accurate with a predicted Elapsed Time of 454.2 minutes and an actual time of 446 metrics.

dual feature importance

its local feature importance values below

>

Data point info

What-If is currently not supported in studio. Run this widget in a jupyter notebook to enable What-If.

Data index

Row 662

True value: 446

Predicted value: 454,2

Feature values

Search features

DayOfWeek

6

Origin

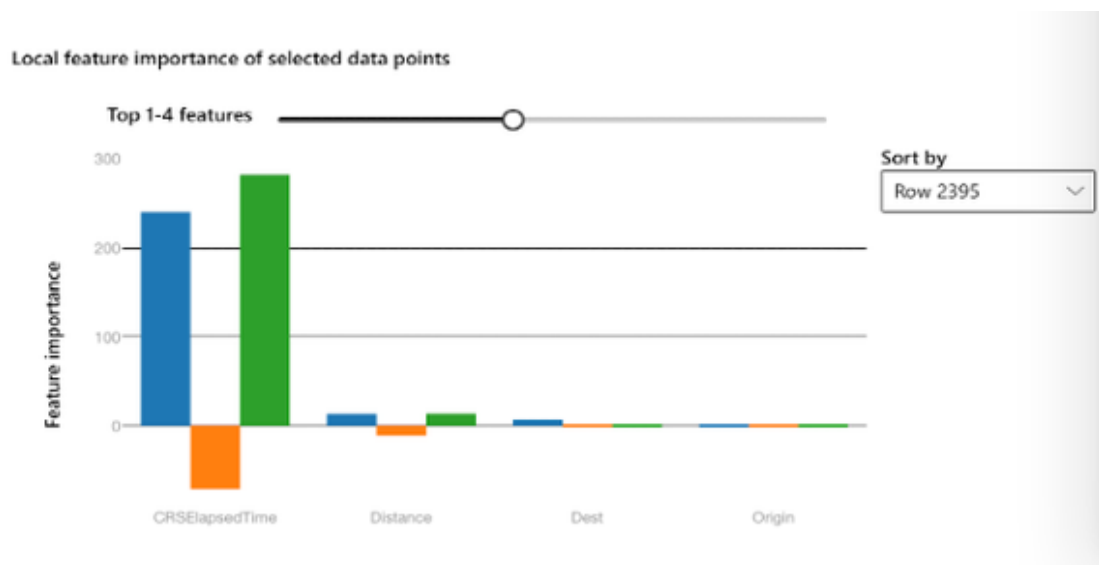
HNL

Dest

DFW

Likewise I chose a point where the prediction was much higher than actual (426 minutes vs. 395 minutes for Row 2395 (Blue) and one point where the prediction fell short (84 vs. 140 minutes) in case of Row 1523 (Orange).

Now with three points selected, scroll down and you should see a graphic which looks similar to the one in Figure XX:



This graphic will show you what led to the individual prediction for the selected data points. For example, we can see that in the case of Row 1523 (the orange bar in the middle) - the point where the prediction was too low - the feature "CRSElapsedTime" was actually weighted down by the model. For some reason it decided that in this special case the original scheduled time should not be considered as much. As it turns out, in this case this was a bad decision. In contrast, for Row 662 (the Green bar on the right) where the prediction was almost exactly like the true outcome, the scheduled departure time had a high influence factor. The model did not consider any other factors here as important that could cause a delay.

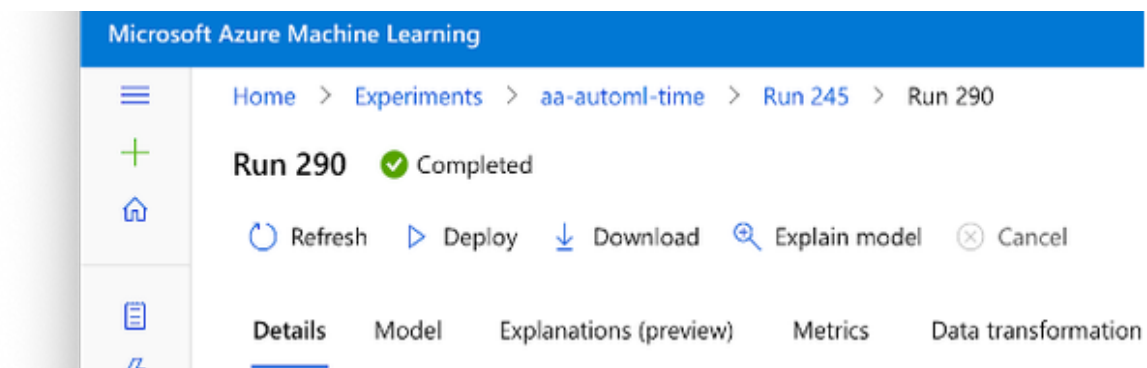
Looking at single predictions and understanding which factors have an influence on the predictions will help you to feel more comfortable in applying and also explaining your machine learning model, instead of just accepting it as a "black box". It is also a good tool for debugging. In our case we can see that the model is pretty much working as expected.

Let's head over and deploy this model to production, so we can get some inference from it.

Model Deployment with Microsoft Azure Walkthrough

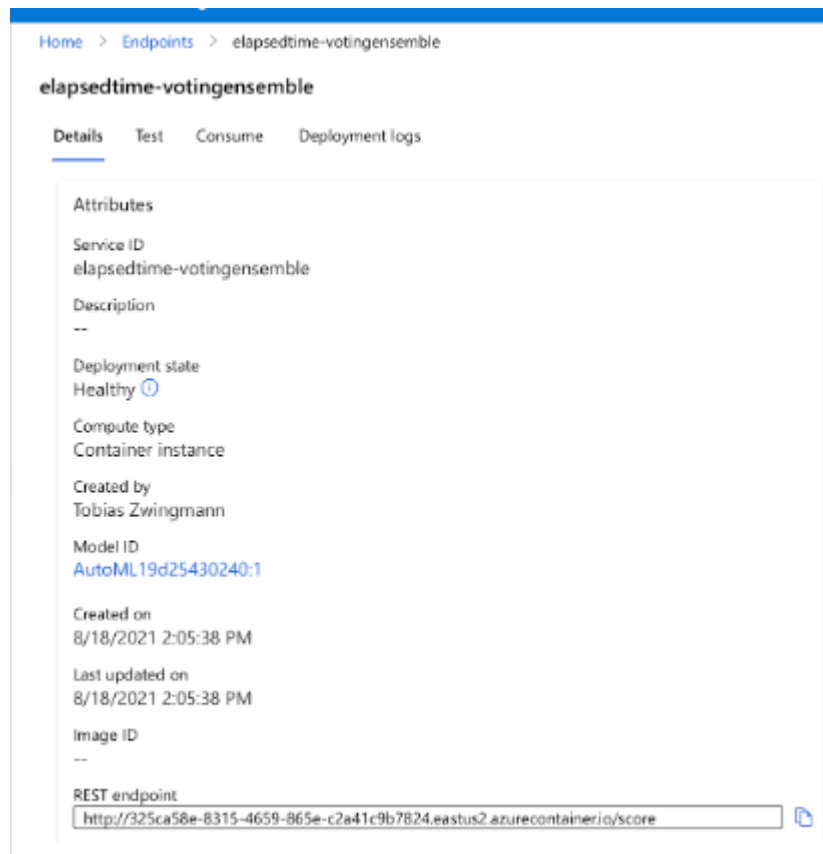
The deployment steps will be identical to the previous use case so I will keep it short here.

Select the model you want to deploy and click “Deploy” from the menu bar as shown in Figure XX.



In the pane that pops up give your model a name such as “elapsedtime-votingensemble”. Choose Azure Container Instance as Compute Type. Hit “Deploy”.

The deployment is complete when your model is listed under “Endpoints” and has the status “Healthy” as seen in Figure XX. Select the model to see the REST endpoint URL which you will need in the further steps.



Feel free to test your model online using the “Test” from the navigation menu or explore how to make requests against your model programmatically using the “Consume” tab.

Let’s go ahead and get some predictions from our online endpoint.

Model Inference with R

In order to send data to our hosted model endpoint and get predictions back, we will use the same script as provided by the Azure Portal, this time again with some small modifications. You can find the script file in the book resources elapsed-inference.R. The first modification is, that we are again wrapping the request inside a function that accepts a list of vectors so we can request inference for more than one observation simultaneously. The function takes these vectors and builds a JSON object which is sent to the API containing all of the individual observations. This is exactly the same as in the use case before.


```

# API request function:
# Expects a list of columns and gets predictions for each row
inference_request <- function(DayOfWeek, Origin, Dest, DepTimeBlk,
ArrTimeBlk, CRSElapsedTime, Distance, DistanceGroup) {

  # Accept SSL certificates issued by public Certificate
  Authorities
  options(RCurlOptions = list(cainfo = system.file("CurlSSL",
"cacert.pem", package = "RCurl"), ssl.verifypeer = FALSE))

  h = basicTextGatherer()
  hdr = basicHeaderGatherer()

  # Bind columns to dataframe
  request_df <- data.frame(DayOfWeek, Origin, Dest, DepTimeBlk,
ArrTimeBlk, CRSElapsedTime, Distance, DistanceGroup)

  # Dataframe to request list of lists
  req = list("data"=apply(request_df,1,as.list))

  body = enc2utf8(toJSON(req))
  api_key = "" # Replace this with the API key for the web service
  authz_hdr = paste('Bearer', api_key, sep=' ')

  h$reset()
  curlPerform(
    url = "http://325ca58e-8315-4659-865e-
c2a41c9b7824.eastus2.azurecontainer.io/score",
    httpheader=c('Content-Type' = "application/json",
'Authorization' = authz_hdr),
    postfields=body,
    writefunction = h$update,
    headerfunction = hdr$update,
    verbose = FALSE
  )

  headers = hdr$value()
  httpStatus = headers["status"]
  if (httpStatus >= 400)
  {
    return(paste("The request failed with status code:",
httpStatus, sep=" "))
  }

  # Get results
  result = h$value()
  result = fromJSON(fromJSON(result))$result

```

```
    return(result)
}
```

The second modification addresses the amount of data which will be sent to the API. While in the previous use case we only looked at a one hour batch of data (roughly 100 rows), in this case we are dealing with a whole month of data - more than 32.000 rows in total! While this would probably work for the request size, we are reaching the limits here of online inference - the processing time for this request would almost be a minute. Also, this data contains a lot of redundant information for the inference API. Remember that we trained on weekdays. Some flights in the schedule go daily, meaning that each weekday will be in the dataset four times or more. It would be both computationally but also financially inefficient to send a datapoint with the same information 4 times to the API. We won't do that and here is how using our R script:

```
df <- dataset
# Create subset of dataframe that holds distinct flight information
for prediction
df_inference <- df %>% select(DAY_OF_WEEK, ORIGIN, DEST,
DEP_TIME_BLK, ARR_TIME_BLK, CRS_ELAPSED_TIME, DISTANCE,
DISTANCE_GROUP) %>%
  distinct()
```

The dataframe df contains the original dataset, meaning all of the 32.000+ rows. We will create a subset of this dataset using the dataframe df_inference. The dataframe df_inference contains only these columns which are needed for predictions. We don't include the flight date here so some rows will now be redundant. Using the function distinct() from the dplyr package we delete all duplicate rows. This will shrink our dataframe from 32.116 rows to only 13.832 rows which contain unique information for inference. That is a reduction of our data workload by more than 50%! We will send the dataframe df_inference to the API and get predictions for that. Eventually, however, we want to report on all flights, not only the distinct combination that we sent to our model. And that is how we will join the prediction results back to the original dataframe with the following function:

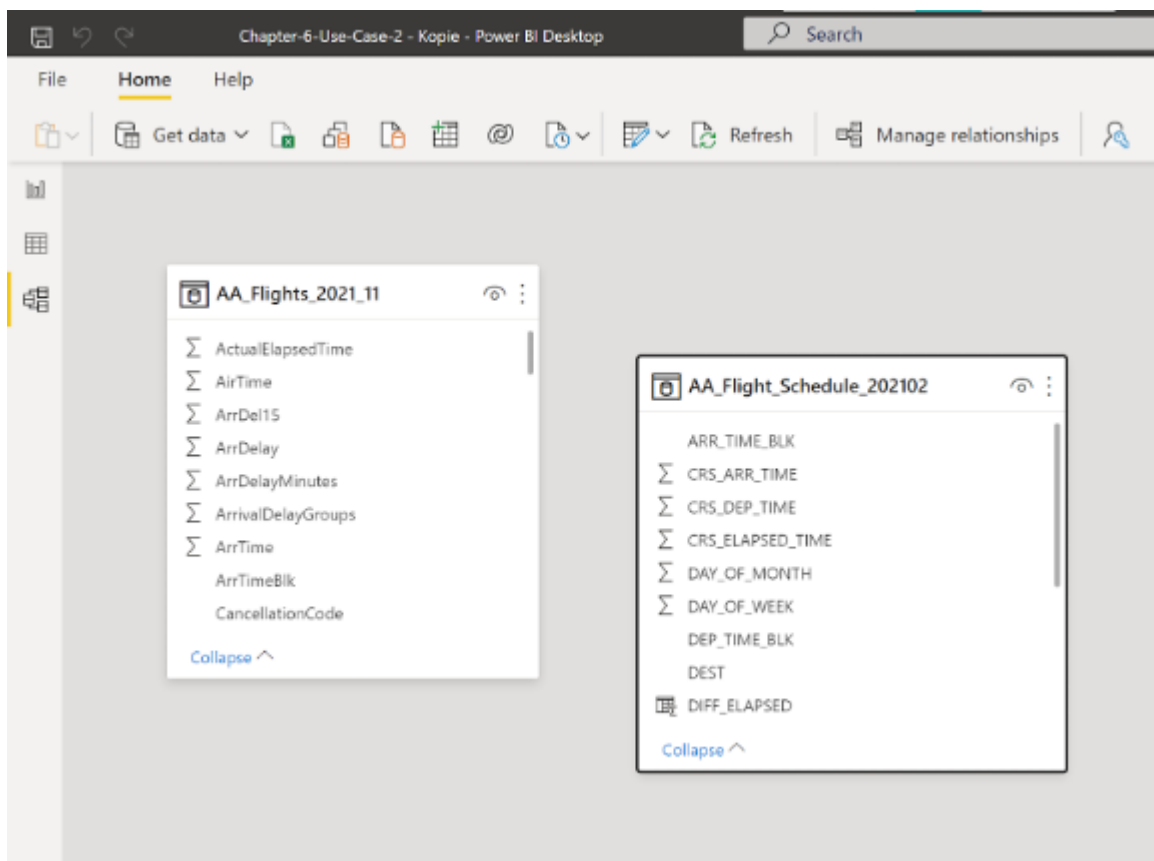
```
df <- df %>%
  left_join(df_inference, by = c("DAY_OF_WEEK", "ORIGIN", "DEST",
    "DEP_TIME_BLK", "ARR_TIME_BLK", "CRS_ELAPSED_TIME", "DISTANCE",
    "DISTANCE_GROUP"))
```

As a result, the dataframe df contains all of the 32.116 original observations with their respective prediction results, but we only send 13.832 rows from that to our model. Isn't that beautiful?

Time to put our script at work and see our model in action within our BI tool!

Model Inference with Power BI Walkthrough

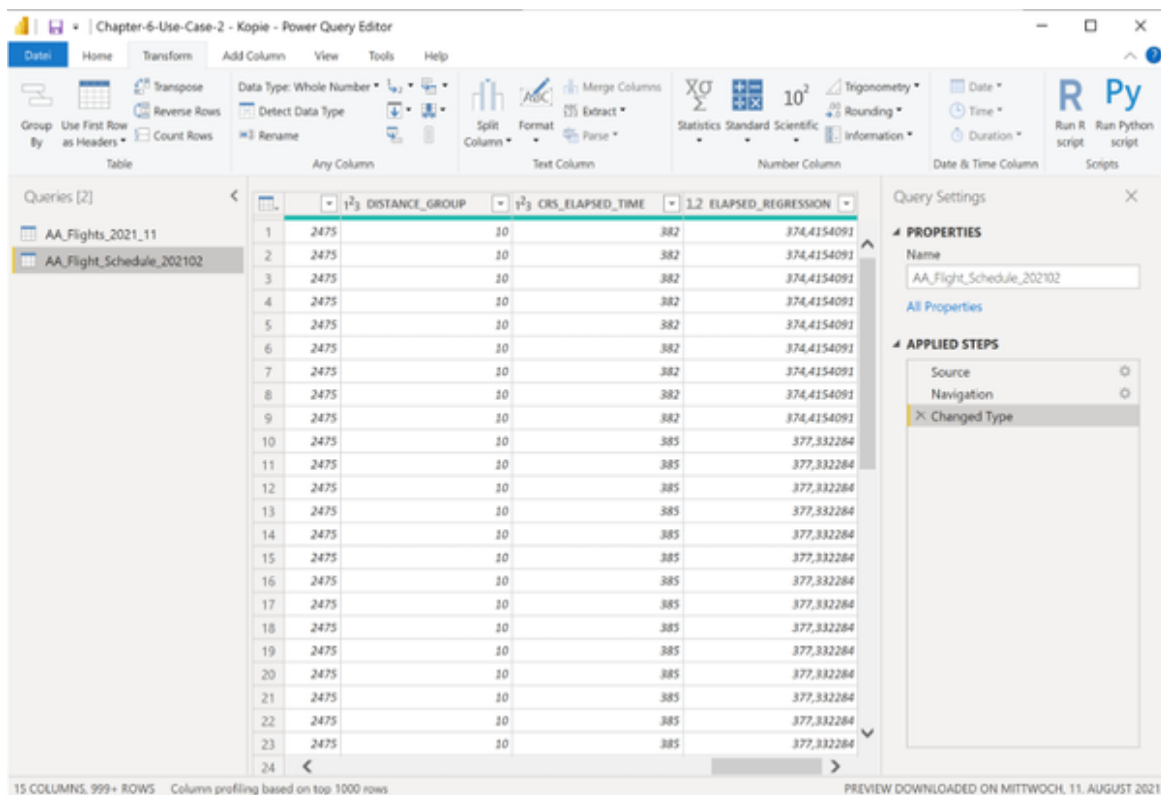
Open the file “Chapter-6-Use-Case-2.pbix” in Power BI desktop or Power BI service. Choose “data model” from the navigation pane on the left. You will see two data sources here as seen in Figure XX.



The data source “AA_Flights_2021_1” contains all the historic flights from January 2021. This is the data we used to train our model on. We don’t need to modify this data. The second source “AA_Flight_Schedule_202102” contains the flight schedule data we are interested in. For this data model we want to run predictions for the Elapsed Time attribute.

We will do this by applying the R or Python script we have written before to create a new column in the model.

Select the Power Query Editor and make sure you choose the “AA_Flight_Schedule_202102” table on the left. In the menu pane, select “Transform” and then choose “Run Python Script” or “Run R Script” depending on your preference as shown in Figure XX.



The screenshot shows the Power Query Editor window titled "Chapter-6-Use-Case-2 - Kopie - Power Query Editor". The interface includes a ribbon with tabs: Data, Home, Transform, Add Column, View, Tools, and Help. The "Transform" tab is active, showing options like Transpose, Reverse Rows, Count Rows, Detect Data Type, Rename, Split Column, Format, Extract, Parse, Merge Columns, Statistics, Standard Scientific, Trigonometry, Rounding, and Information. The "Queries [2]" pane on the left lists "AA_Flights_2021_11" and "AA_Flight_Schedule_202102". The main data view displays a table with 24 rows and 5 columns: "1", "2", "3", "4", and "5". The data values are as follows:

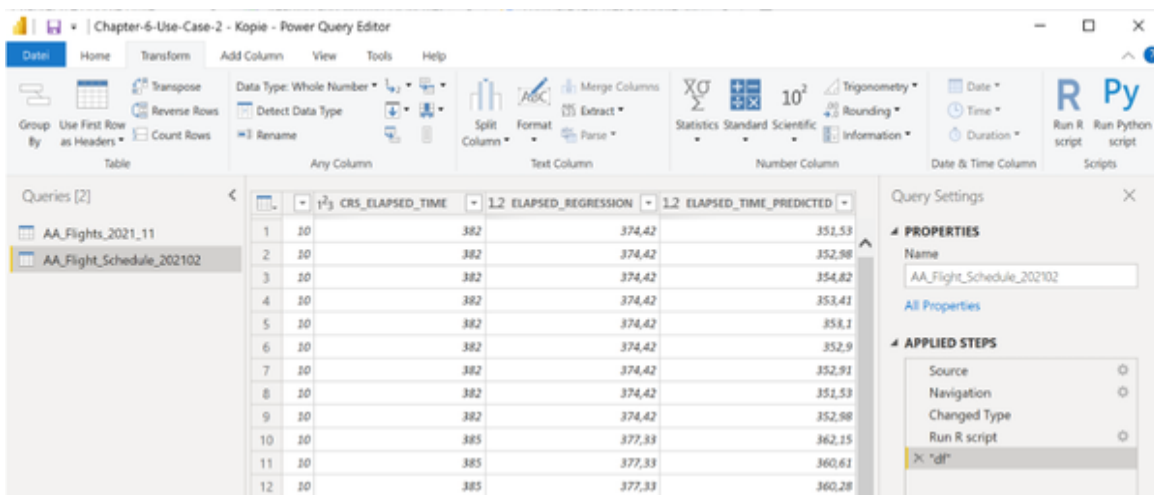
1	2	3	4	5
1	2475	10	382	374,4154091
2	2475	10	382	374,4154091
3	2475	10	382	374,4154091
4	2475	10	382	374,4154091
5	2475	10	382	374,4154091
6	2475	10	382	374,4154091
7	2475	10	382	374,4154091
8	2475	10	382	374,4154091
9	2475	10	382	374,4154091
10	2475	10	385	377,332284
11	2475	10	385	377,332284
12	2475	10	385	377,332284
13	2475	10	385	377,332284
14	2475	10	385	377,332284
15	2475	10	385	377,332284
16	2475	10	385	377,332284
17	2475	10	385	377,332284
18	2475	10	385	377,332284
19	2475	10	385	377,332284
20	2475	10	385	377,332284
21	2475	10	385	377,332284
22	2475	10	385	377,332284
23	2475	10	385	377,332284
24	2475	10	385	377,332284

The "Query Settings" pane on the right shows the "Name" as "AA_Flight_Schedule_202102" and the "Applied Steps" list including "Source", "Navigation", and "Changed Type". The status bar at the bottom indicates "15 COLUMNS, 999+ ROWS" and "Column profiling based on top 1000 rows". A footer note states "PREVIEW DOWNLOADED ON MITTWOCH, 11. AUGUST 2021".

Copy and paste the contents from the file of the R or Python script and paste it into the code window in Power BI. Double check that the URL provided in the script is the one of your healthy REST endpoint. Also make sure that you have all the required packages (RCurl, rjson and dplyr) installed on the R environment used by Power BI. Press “OK” and Power

BI will run your script. Depending on your machine this process might take some time. Remember that we are doing calculations here for more than 32.000 rows of data. And while we reduced the data amount that has been sent to the API, these are still over 13.000 rows. If you want to see what your computer is doing, feel free to open the Windows Task Manager, choose the “Performance” Tab and see your computer at work.

After the process has been completed you should see that there has been one step added on the right under “Applied Steps” and that your dataset now contains an additional column called “ELAPSED_TIME_PREDICTED” as seen in Figure XX.



The screenshot shows the Power Query Editor interface. The main area displays a table with 12 rows and 5 columns. The columns are: Index, CRS_ELAPSED_TIME, ELAPSED_REGRESSION, ELAPSED_TIME_PREDICTED, and an unnamed column. The 'Applied Steps' pane on the right shows a list of steps: Source, Navigation, Changed Type, Run R script, and a new step named 'd1'.

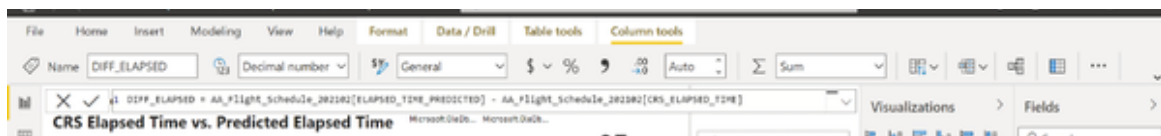
	CRS_ELAPSED_TIME	ELAPSED_REGRESSION	ELAPSED_TIME_PREDICTED
1	382	374,42	351,53
2	382	374,42	352,98
3	382	374,42	354,82
4	382	374,42	353,41
5	382	374,42	358,1
6	382	374,42	352,9
7	382	374,42	352,91
8	382	374,42	351,53
9	382	374,42	352,98
10	385	377,33	362,15
11	385	377,33	360,61
12	385	377,33	360,28

Select “File” -> “Close and Apply” to exit the Power Query editor and apply your transformations. Power BI will update the data model accordingly, which again can take a couple of minutes. The waiting time in this case should not bother us too much, since this process will only be done once for each monthly flight schedule.

After the data model has been successfully updated, head over to the “Reports” in Power BI. We want to put our new predictions at work. Select the report “Future Schedule” where you can see the baseline regression predictions. Time to tune them up.

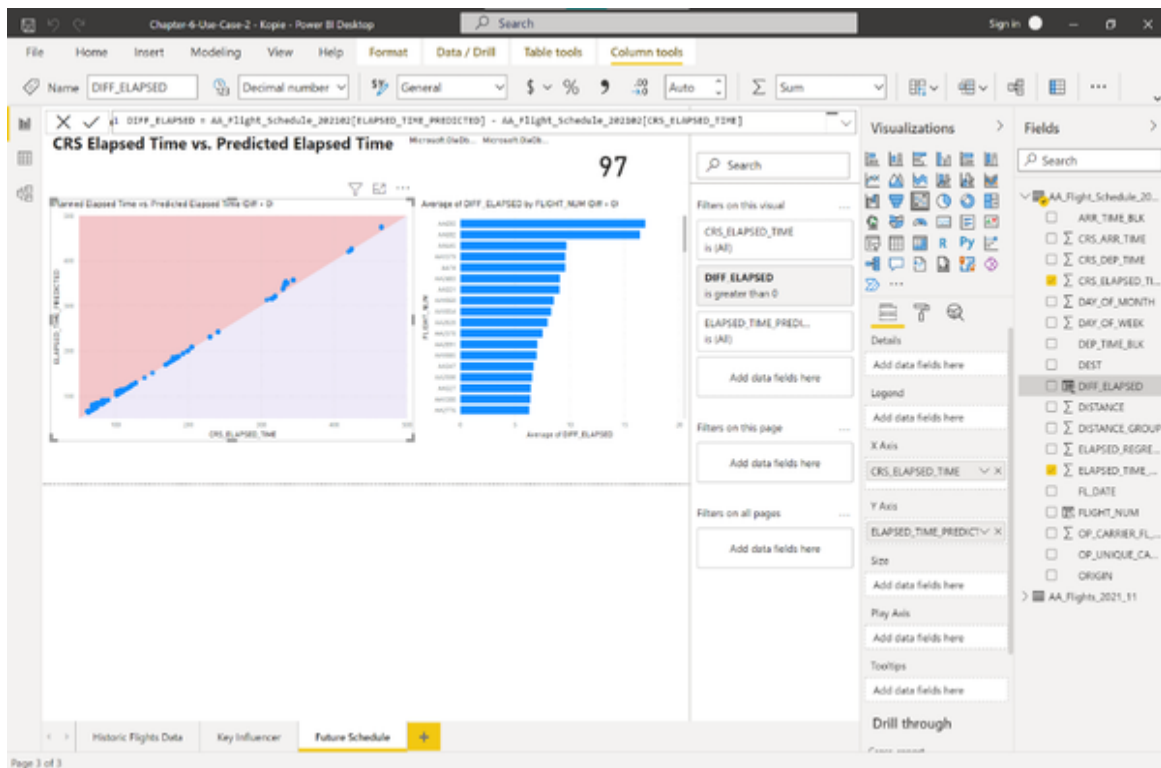
Select the scatter chart on the left. Replace the value for the Y-axis. Instead of “ELAPSED_REGRESSION” put the new measure “ELAPSED_TIME_PREDICTED” here. Open the Filter for this visual.

You can see that the data is still filtered for $\text{DIFF_ELAPSED} > 0$, where DIFF_ELAPSED is the difference between the predicted time by the baseline (regression) model and the scheduled elapsed time. We want this field to show the difference between our AI model and the scheduled elapsed time. Since this is a calculated field in Power BI, it is very easy to change that. Select the measure and you should see a formula popping up at the top of the report page. Replace `AA_Flight_Schedule_202102[ELAPSED_REGRESSION]` with `AA_Flight_Schedule_202102[ELAPSED_TIME_PREDICTED]` as shown in Figure XX and hit Enter.



With the updated values for the DIFF_ELAPSED measure, all other visuals on the page should update as well according to the new filter criteria.

Figure XX shows how the updated dashboard looks like:



What have we achieved so far?

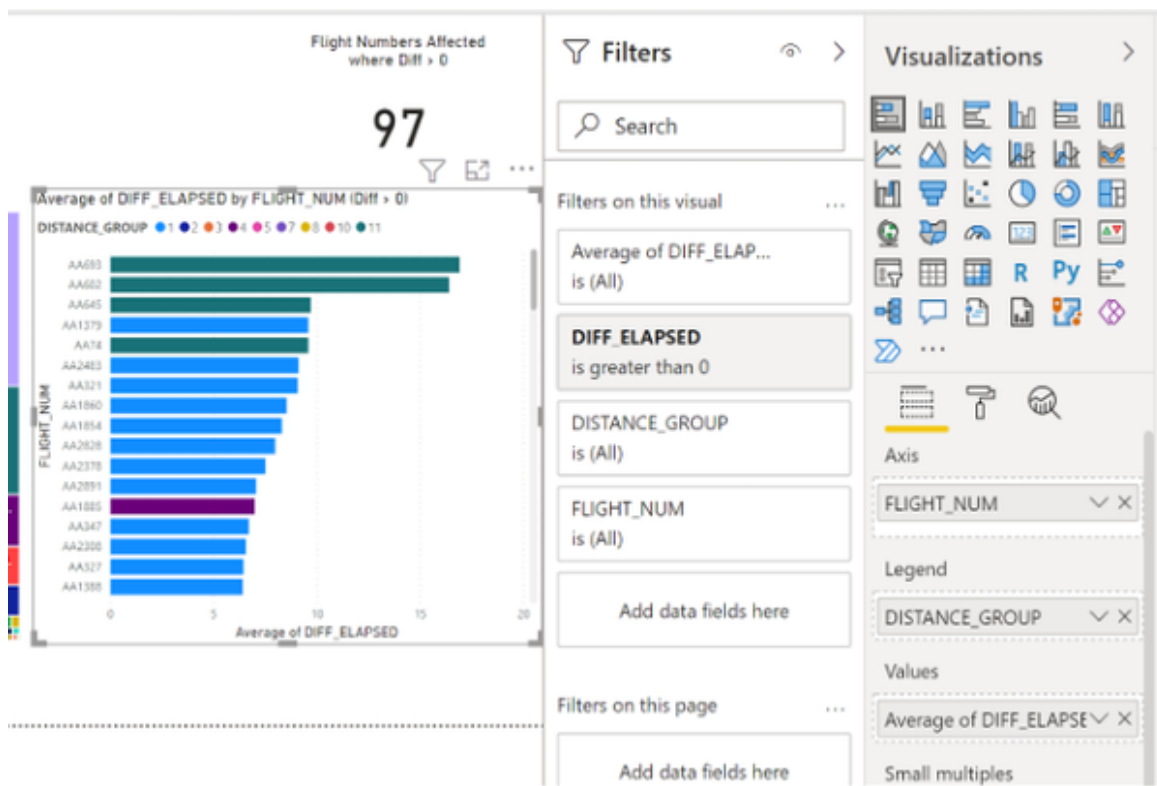
First, by applying our AI model we have reduced the amount of flights which are predicted to need longer than expected to 97 observations.

Second, if we look at the scatter chart on the left, we can see that there is more variation and the points do not form a single straight line as in the simple regression model.

And third, if we look at the bar chart on the right, we can see that we have identified two flights which are predicted to take 15 minutes longer than planned. If we compare that to the previous regression model, where the highest difference was only about 1.5 minutes, this is a significantly bigger amount.

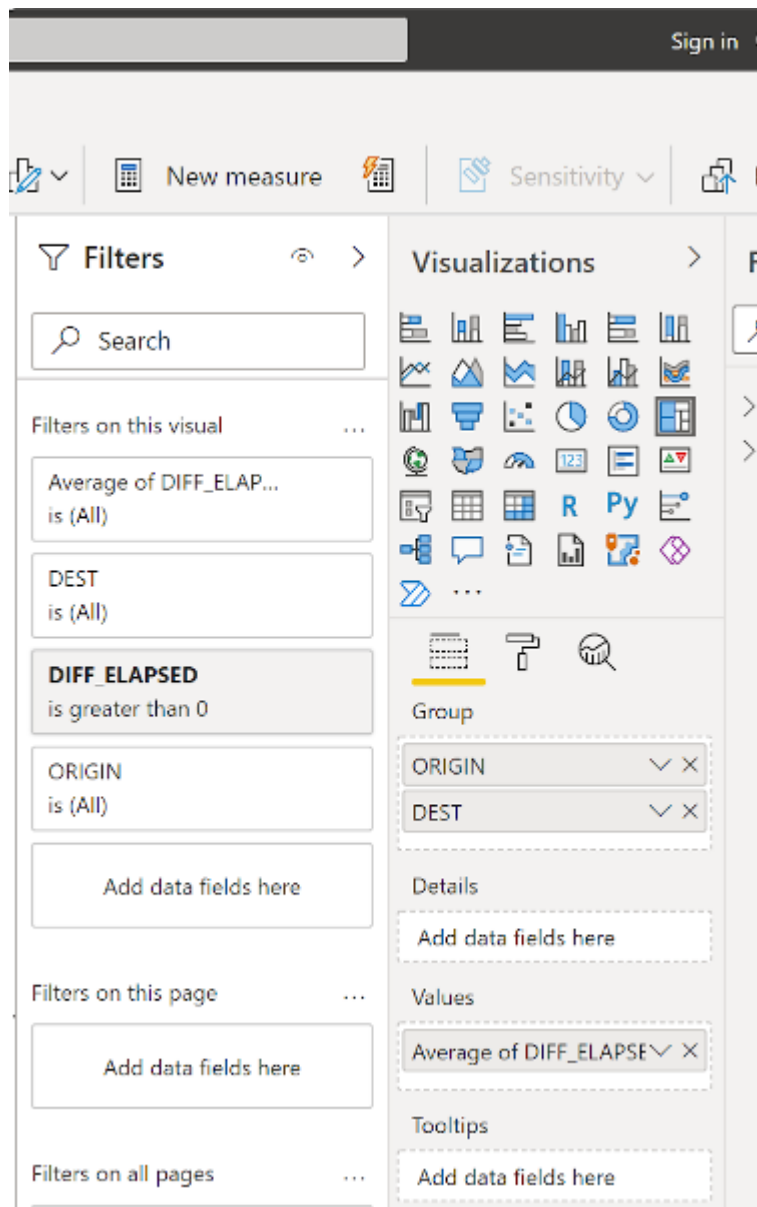
Let's see if we can tweak this report even a little bit more to make it more interactive and actionable.

First, select the bar chart on the right and add the field "DISTANCE_GROUP" to the legend as shown in Figure XX



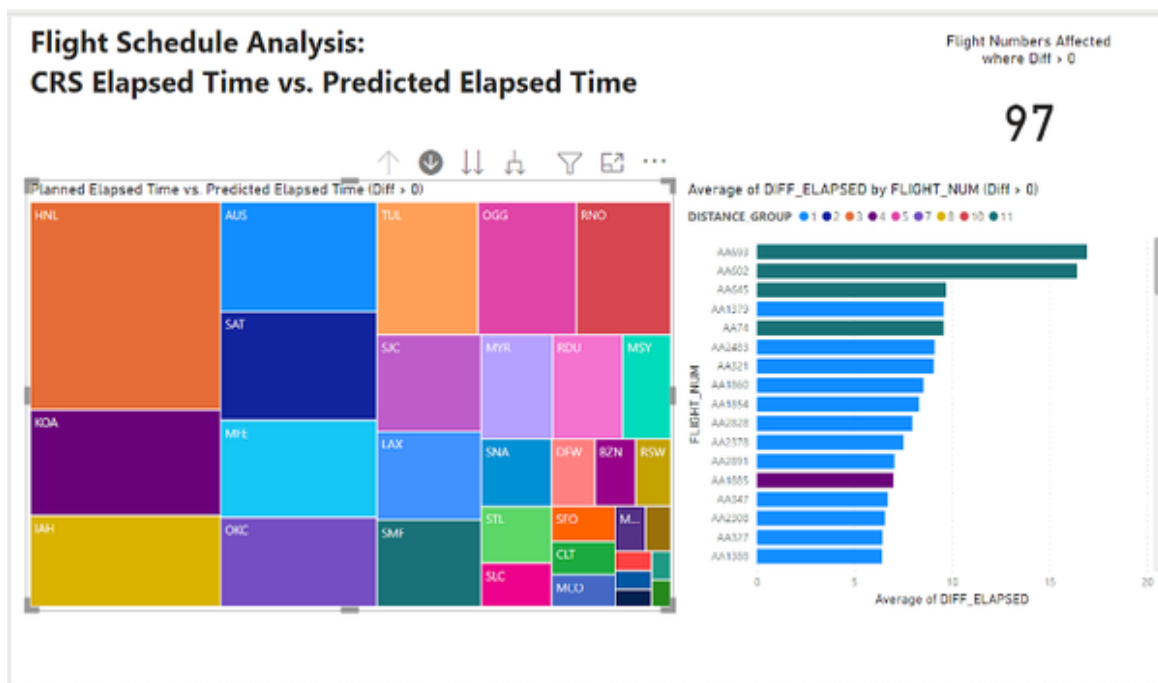
From this additional information we can see that the flights which have the highest discrepancy between predicted and scheduled elapsed time are actually long distance flights (distance group 11). The majority of flights on the chart, however, are short distance flights.

Also, let's get rid of the scatter plot. It has proven a good overview of how the model is working, but it is not so interactive. Select the Scatterplot and replace it with a Treemap visual. Adjust the settings for the treemap as seen in Figure XX:

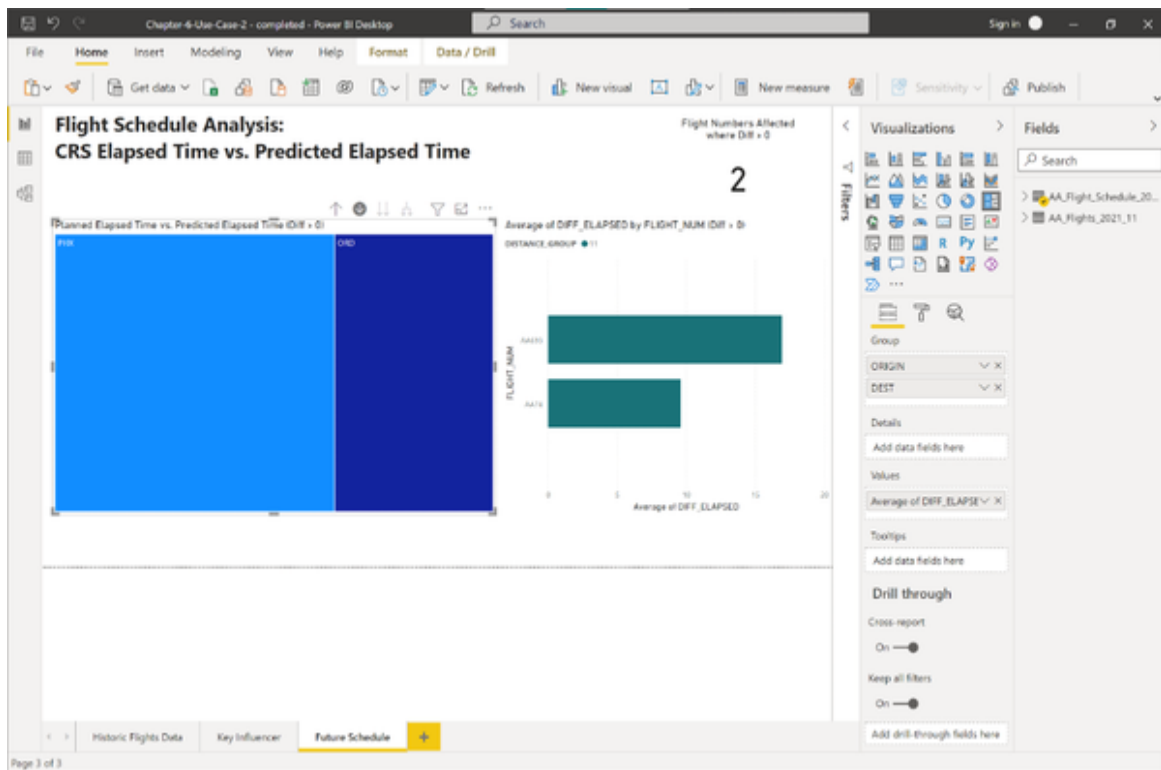


- Add the fields ORIGIN and DEST to Group.
- Put the field DIFF_ELAPSED into the values.
- Aggregate DIFF_ELAPSED by Average
- Filter for this visual is set to “DIFF_ELAPSED is greater than 0”.

As a result you should see an updated visual as shown in Figure XX. Enable the drill-down function on the top.

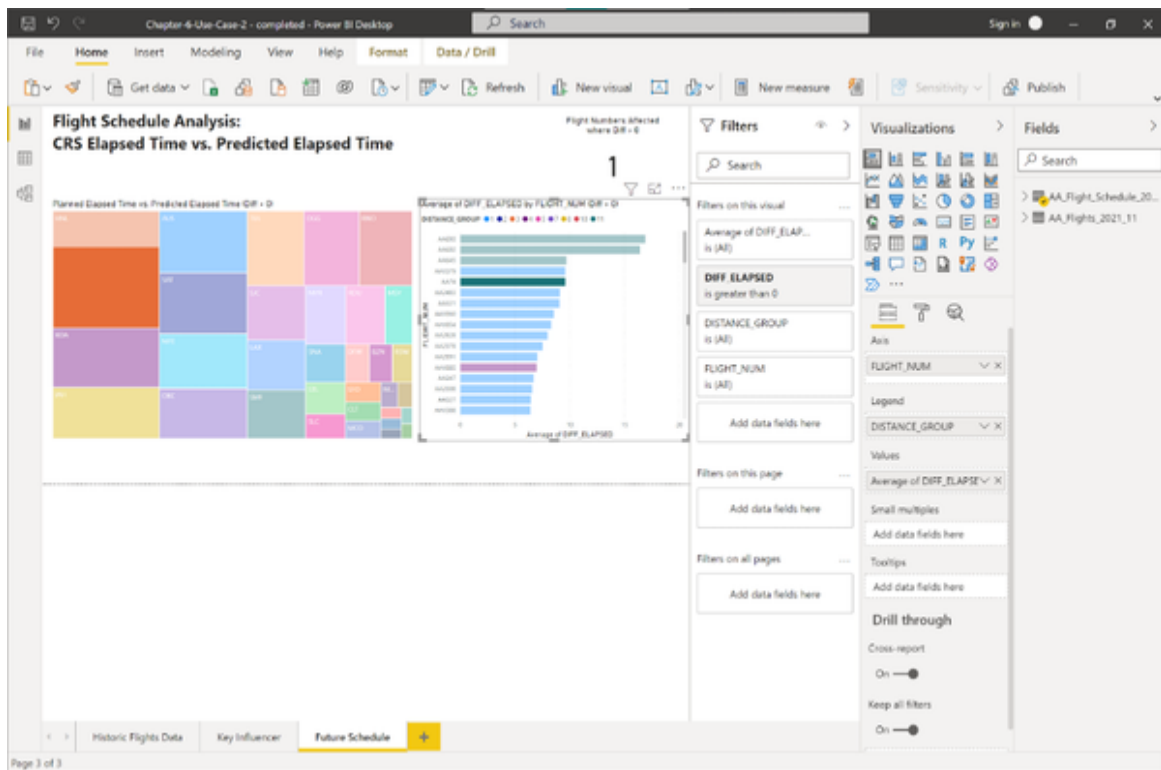


The tree map shows Origin airports where the field size is the average of the difference between the predicted and scheduled elapsed time for flights from this airport. Everytime we interact with this tree the bar chart on the right will change and we can drill down. For example, click the origin airport Honolulu. You will see that the treemap updates and the barchart as well as shown in Figure xx



- 2 flights, one has 10 and the other 15 minutes delay

Similarly, we can select a bar from the bar chart on the right and Power BI will highlight from which origin airport this flight comes from as shown in Figure XX



This provides us with a great interactivity and resource to find out where the bottlenecks in our flight planning are. We can clearly see that flights scheduled to depart from Honolulu are on a tough schedule and that the connection from Honolulu to Phoenix, AZ has the highest chance of exceeding the scheduled time, at least according to the best knowledge we have so far.

While the model is not perfect, we have improved our baseline and who knows - maybe we can think of even more criteria which can make the model more accurate? At least now we have a good showcase to demonstrate how a better AI model could be beneficial for improving our schedule ahead of time.

Find the final PBI file in the book repository - Chapter-6-Use-Case-2 - Completed.pbix

Use Case: Automated Anomaly Detection

In the previous use cases we learned how to leverage AI to predict categorical and numeric variables and how to use that skill to create business value. In the following use case we are not so interested whether a value is categorical or numeric, but we want to know if a certain value is abnormal. To identify whether a value is an anomaly or not, the algorithm will consider the points to be a series of events in time, also known as time series. Let's head over to the problem statement of our use case.

Problem Statement

We are part of the Operations team of American Airlines and our job is to maintain a service level. This included observing what is happening at the airports where American Airlines takes off, especially the most frequent ones. The most important airports that counted more than 1.000 flight departures in January 2021 were Dallas Fort Worth (DFW), Charlotte Douglas Airport (CLT), Phoenix Sky Harbor (PHX), Miami International Airport (MIA), Chicago O'Hare (ORD), Philadelphia International (PHL) and Los Angeles International Airport (LAX). Amongst other metrics, we have to closely monitor the taxi out time that is occurring at these airports.

Taxi-out time is defined as the time between the actual pushback of an aircraft from the gates and wheels-off. A high taxi-out time is a proxy for busy or overcrowded airports and can lead to unexpected flight delays (as we have seen in the previous use case).

While a natural fluctuation of the taxi out time is normal and considered by the flight schedule, too many peaks can lead to recurring delays. A stable taxi-out time is an important precondition for guaranteeing a smooth departure process at an airport. Also, long taxi-out times increase congestion and excessive emission of greenhouse gases.

Monitoring the taxi-out status and flagging unwanted spikes to airport operators are effective approaches to eliminate delays and improve the utilization of resources.

The way this process is currently handled is through a BI dashboard which shows an overview of the daily average taxi-out time for the high traffic

airports mentioned above. The dashboard is shown in Figure XX

<6-taxi-dashboard-baseline>

The primary goal of the analyst looking at this dashboard is to identify those peaks which are abnormal for an airport in order to raise these issues for further investigation to the Operations team. The main problem is: What is an abnormal point? The solid line in the graphs shows the average daily taxi-out time for the month of January for the 7 airports mentioned above. The dashed horizontal line displays the average taxi-out time for each of these airports in January. As we can see from the dashboard, the value of these averages varies depending on the airport. The average taxi-out range is between 15.3 minutes for Los Angeles International airport and almost 20 minutes for Dallas Fort Worth. That is why it is hard to judge abnormal taxi-out time generally, but instead they must be considered by the airport. Flagging all above-average taxi-out times however, is not practical because the time has too many fluctuations. Instead, the team has decided to set a 90-percentile threshold for each airport. This threshold covers 90% of all taxi-out averages for a specific airport. All daily taxi-out averages that exceed this threshold have been decided to be flagged and considered as anomalies. For example, the average taxi-out values for DFW on 4 and 10 January have been flagged.

However, this approach does have its limits:

- The 90-percentile mark is very static and does not consider any trends that are happening in the data.
- Also, it is very sensitive to extreme values, meaning one day of high average taxi-out values can raise the bar to unnecessary heights.
- And thirdly, it is very tedious and time-consuming to look at these charts manually and flag problematic events on a case-by-case basis.

The team is looking for an overall improved approach of identifying peak values for taxi-out times for these airports.

Solution Overview

Our goal is to make the prediction of abnormal data points more dynamic and more responsive to the overall development of the timeline. The approach we take for this is an AI-powered anomaly detection which analyses a series of events (value) over a certain period of time (timestamp). The anomaly detection will calculate dynamic upper and lower margins for the expected value and flag all values which exceed these boundaries as positive (above upper boundary) or negative (below lower boundary) anomalies.

This time, we are not going to train our own AI model. Instead, we will use an off-the-shelf AI service for the first time in this book. This means, we will consume an AI model that has been pre-trained before on the task which we want to achieve.

In our case we are using the Anomaly Detector by Azure Cognitive Services. To do this, we will enable the endpoint through our Azure portal, learn how to prepare our data and get predictions from this endpoint using Python and R and finally apply these predictions in our BI to build an enhanced version of the taxi-out anomaly detection report.

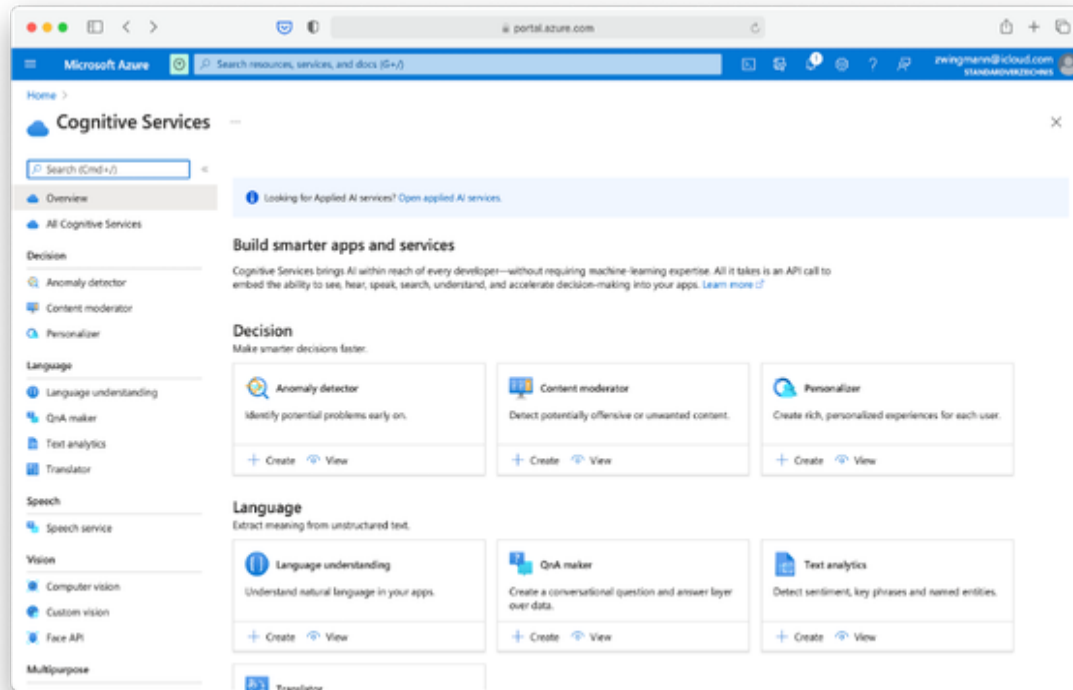
Note that the anomaly detection service can be used for both batch and real-time prediction. Real-time prediction in this case means that we send any points we observe directly to the API and will get the prediction back whether this point is an anomaly based on the points we sent previously. For our use case, however, we are taking the batch approach, meaning we upload all available data at once to the API.

Without further ado, let's head over to the Azure portal.

Enabling AI Service on Microsoft Azure Walkthrough

Our first task is to enable the Cognitive Services Anomaly Detector API in Azure by creating a resource for it. To do that, you can either visit [this link](#) or navigate there manually: Visit portal.azure.com, search for “Cognitive

Services” and in the “Decision” section you will find the card “Anomaly detector” where you can hit create as shown in Figure XX.



The “Create Anomaly Detector Form” requires you to specify some more details. Choose your desired Azure subscription (which will be just a single one if you just signed up) and select a resource group. Choose the same resource group as you created for the previous use cases. In addition, you need to specify the geographic region where the service should be running. Choose your preferred region and give your service a telling name. Note that this name is globally unique, think of it as a subdomain for the final endpoint. Finally you will need to choose the pricing tier. Different pricing tiers affect the performance and pricing model of the AI service. For our purpose, the Free tier should be enough which allows 10 Calls per second and 20.000 transactions per month. See Figure XX for an example of the filled out form. Proceed by selecting “Review + Create”. Review your settings on the subsequent page and confirm by clicking “Create”.

portal.azure.com

Microsoft Azure

Search resources, services, and docs (G+/I)

Home > Create a resource > Anomaly Detector >

Create Anomaly Detector

Basics Virtual network Identity Tags Review + create

Easily embed anomaly detection capabilities into your apps so users can quickly identify problems. Through an API, Anomaly Detector ingests time-series data of all types and selects the best-fitting detection model for your data to ensure high accuracy. Customize the service to detect any level of anomaly and deploy it wherever you need it most. Azure is the only major cloud provider that offers anomaly detection as an AI service. No machine learning expertise is required. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure subscription 1

Resource group * ai-powered-bi
[Create new](#)

Instance details

Region * East US 2

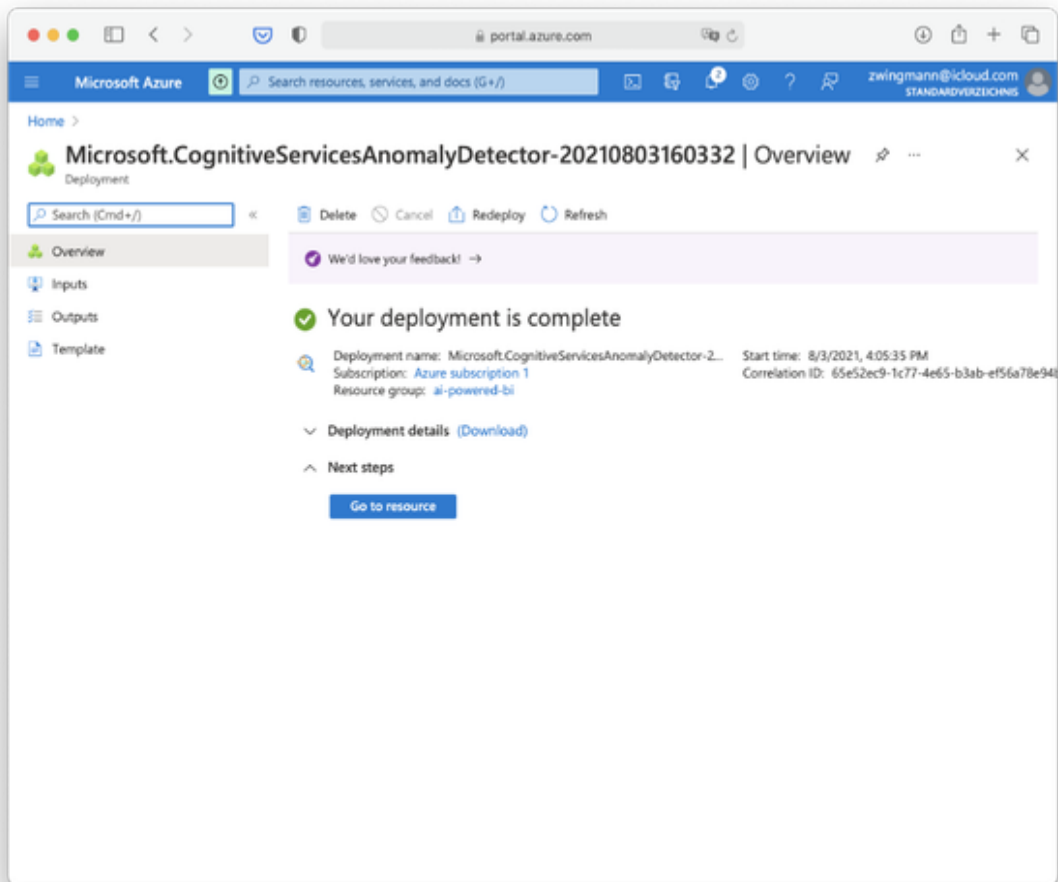
Name * ai-powered-bi-anomaly ✓

Pricing tier * Free F0 (10 Calls per second, 20K Transactions per month)

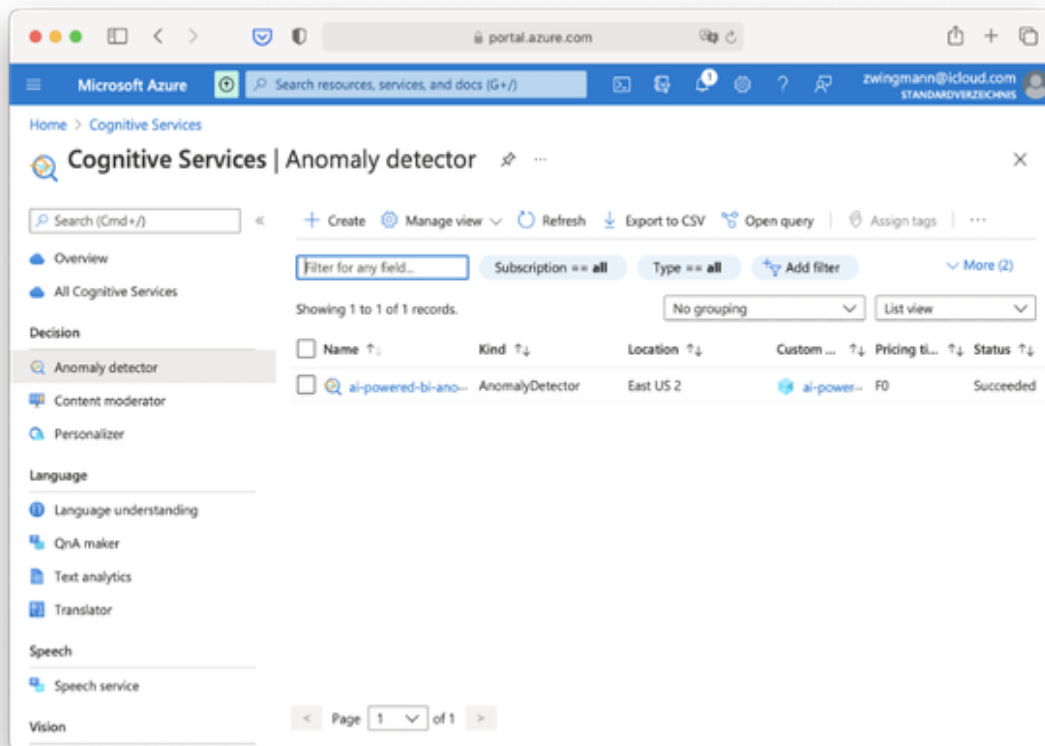
[View full pricing details](#)

[Review + create](#) < Previous Next: Virtual network >

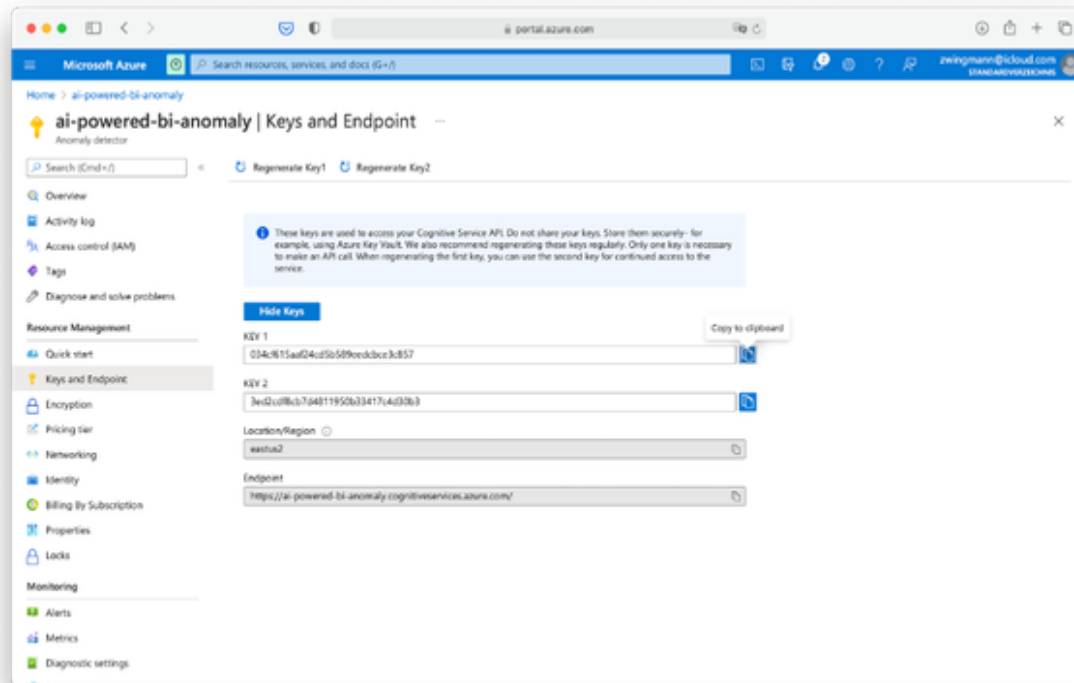
The deployment of the service might take a few minutes. After it is completed you should see a notification in Azure and a success message as shown in Figure XX.



After the deployment is completed successfully, navigate to the newly created resource by clicking “Go to resource” as shown in Figure XX. Alternatively, you could navigate from the Azure Portal home page to Cognitive Services, choose “Anomaly detector” from the menu on the left and click on the name of the endpoint which you will find in the list as shown in Figure XX.



When you have selected the Anomaly detector resource, you will be prompted with the quick start guide. This is a good place to come back to if you want to integrate the endpoint into further applications. The most important page we will need for our needs is the “Keys and Endpoints” section, which you can find in the menu on the left as shown in Figure XX.



On this page you will find three essential components: The first two, are your secret credentials which you will need to access the AI service. Think of them as something like a password that authorizes you to make requests against the service. As it goes for passwords, you should not save these keys somewhere in plain text and you should not share them publicly anywhere. If you did, either accidentally or on purpose (like printing them in a book), hit the button “Regenerate Key 1” or “Regenerate Key 2” to generate a new pair of keys. This will void the previous key and create a new one for you. Why do we need two keys you might ask? Think about it as having a guest key for your home. There is one key, which you can share and distribute to colleagues for quick prototyping purposes, so there is no harm when the key is regenerated often. The other key should be safely integrated in your running applications where you don’t want to replace this key that much (most likely never).

And finally, the third component on this page is the API endpoint. Be careful, however - this is not the complete endpoint you can use to make inference requests as we did in our previous examples. Instead, you need to

add the path to the actual service, the service version and the service settings to it. To find out how to construct the final REST endpoint, you can refer to the [API documentation](#) or the quick start you visited before.

For example, the final REST endpoint for the anomaly batch inference looks like the following (replace “ai-powered-bi” with your own service name):

```
https://ai-powered-bi-  
anomaly.cognitiveservices.azure.com/anomalydetector/v1.0/timeseries  
/entire/detect
```

That’s it. Now we’re all set to start making prediction requests from our newly set-up AI service. Let’s find out how to do it in either Python or R depending on your preference.

Model Inference with Python Walkthrough

- explain model inference (API Call) in Python

Model Inference with R Walkthrough

In the book repository you will find a file called “anomaly-inference.r” which contains the script we will use to get inference for our use case example. Let me quickly walk you through what the script is doing:

At the core of the script is the function called `inference_request`. It is very similar to the request function we used before, but contains some variations. The biggest change is, that the request body has been structured differently to fit the requirements of the Azure Anomaly Detector API. The API expects a JSON object which contains information about the time series granularity first (in our case that is daily) and secondly the actual time series with the corresponding timestamps and values. The two vectors you provide as input to the function (timestamp vector and value vector) will be put here in the right format.

The second change is that we added the `api_key` for authorization purposes.

When you use the script for your own exercise you need to replace to things:

- url: Replace “ai-powered-bi-anomaly” with your custom service name
- api_key: Replace with either key 1 or key 2 from the Azure Portal as described previously.

```
inference_request <- function(timestamp, value) {

  # Accept SSL certificates issued by public Certificate
  Authorities
  options(RCurlOptions = list(cainfo = system.file("CurlSSL",
"cacert.pem", package = "RCurl"), ssl.verifypeer = FALSE))

  h = basicTextGatherer()
  hdr = basicHeaderGatherer()

  # Bind columns to dataframe
  request_df <- data.frame(timestamp, value)

  # Build JSON object for request body
  granularity = c("granularity" = "daily")
  series = list("series"=apply(request_df,1,as.list))
  req = c(granularity, series)
  body = enc2utf8(toJSON(req))
  api_key = "c3580b4786e44ba8ae95578ed83eb21f" # Replace this with
the API key for the web service

  # Perform HTTP request
  h$reset()
  curlPerform(
    url = "https://ai-powered-bi-
anomaly.cognitiveservices.azure.com/anomalydetector/v1.0/timeseries
/entire/detect",
    httpheader=c("Content-Type" = "application/json", "Ocp-Apim-
Subscription-Key" = api_key, "Ocp-Apim-Subscription-
Region"="eastus2"),
    postfields=body,
    writefunction = h$update,
    headerfunction = hdr$update,
    verbose = FALSE
  )

  headers = hdr$value()
  httpStatus = headers["status"]
}
```

```

    if (httpStatus >= 400)
    {
        return(paste("The request failed with status code:",
httpStatus, sep=" "))
    }

    # Get results
    result = h$value()
    result = fromJSON(result)
    return(result)
}

```

Now that we have covered the inference function, let's take a look at some other things the script is doing for us:

In lines 11 and eleven we set the threshold for the airport selection. Remember that we don't want to check for anomalies for all airports, but just for the busiest ones, defined as airports with a minimum of 1000 departing flights.

```

# Threshold for minimum flights per airport
MIN_FLIGHTS <- 1000

```

This variable will be used in lines 60 - 64 where we get a data frame that contains a list of the airports that match this criteria.

```

# Get airports where threshold is met
airports <- df %>%
  group_by(Origin) %>%
  count() %>%
  filter(n >= MIN_FLIGHTS) %>%
  arrange(desc(n))

```

Now that we have our inference function and the list of airports for which we will need to do this inference, we need some code that applies the inference function to the daily average taxi out times for the different airports individually. We do that by running a preprocessing script inside a for loop that loops through the list of the airports which we selected by the threshold criteria. The following script creates an aggregated data frame over taxi out time for each airport and passes this information to the

`inference_request` function. All results will be collected in a dataframe called `df_inference_all`.

```
for (airport in airports$Origin) {
  # Create aggregated dataframe for average TaxiOut per Airport
  df_inference <- df %>%
    select(FlightDate, Origin, TaxiOut) %>%
    filter(Origin == airport) %>%
    group_by(FlightDate, Origin) %>%
    summarize(AvgTaxiOut = mean(TaxiOut)) %>%
    mutate(Timestamp = paste0(FlightDate, "T00:00:00Z"),
           AvgTaxiOut = round(AvgTaxiOut,2))

  # Submit request
  result <- inference_request(df_inference$Timestamp,
df_inference$AvgTaxiOut)

  # Convert result object to dataframe
  results_df <- as.data.frame(result)

  # Add inference results to inference_df
  df_inference <- bind_cols(df_inference, results_df) %>%
    select(-Timestamp)

  df_inference_all <- bind_rows(df_inference_all, df_inference)
}
```

Finally, we join the information back to the original data frame so we can make use of the new columns in our BI tool, just as we did previously. This process is handled by the following code:

```
# Add inference information to original dataframe
df <- df %>%
  left_join(df_inference_all, by = c("FlightDate", "Origin")) %>%
  mutate(upperMargins = expectedValues + upperMargins,
         lowerMargins = expectedValues - lowerMargins)
```

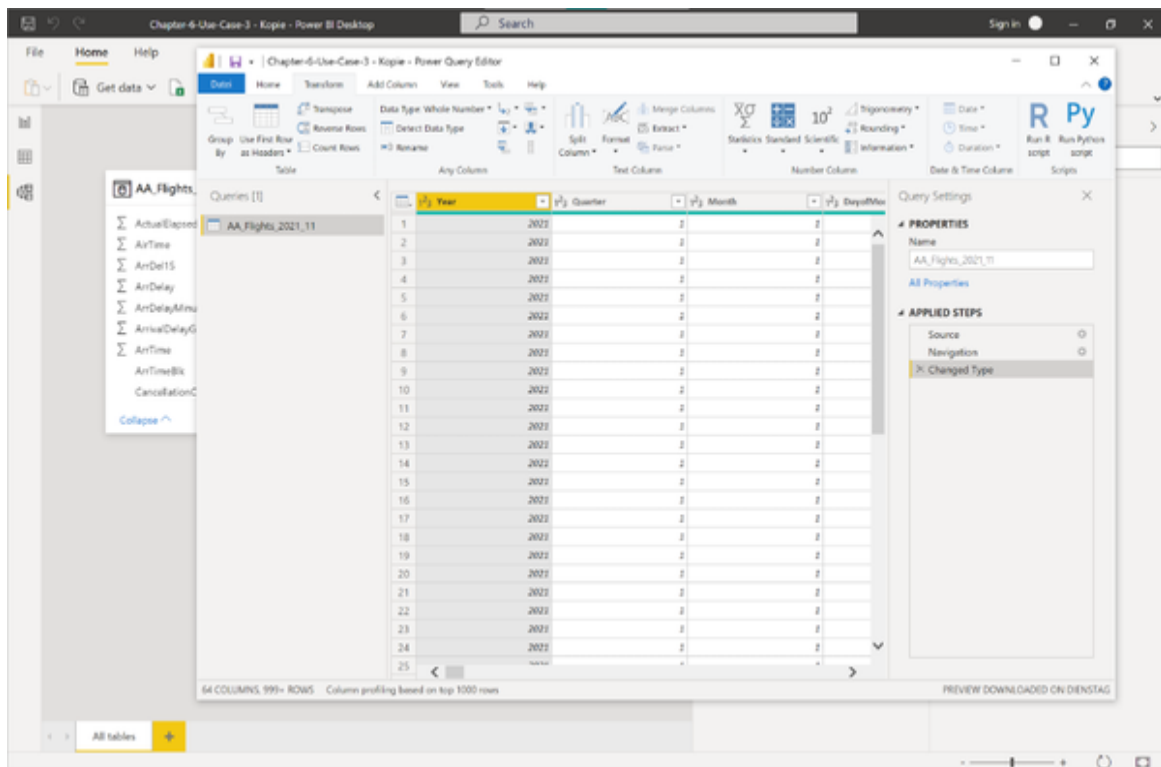
Don't worry if the code might look too overwhelming or too complicated at the first glance. You can always go back and revisit it step by step if you need to. Otherwise, feel free to just copy and paste everything while replacing your custom variables (`url` and `api_key`).

Model Inference with Power BI Walkthrough

Our goal is to utilize the anomaly detector to create a better report and more actionable insights. We want to make the overall report more accurate, interactive and free it up from manual efforts.

First, let's plug the AI into our data model.

Open the file Chapter-6-use-case-3.pbix in PowerBI Desktop or Power BI Service and navigate to the data model. Open Power Query Editor as shown in Figure XX.



Select Transform in the ribbon menu and choose either “R script” or “Python script” as per your preference. I’ll continue the example with R but the same steps apply to Python as well.

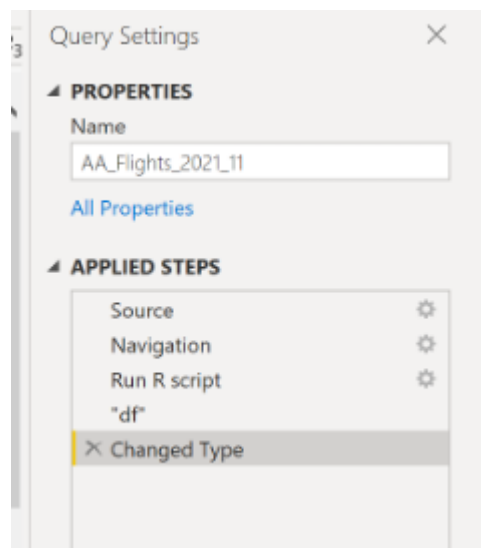
When the script editor opens, don’t paste the code from our source file yet. Instead insert the following line, no matter if you’re using R or Python:

```
df = dataset
```

Confirm with OK and you should see two steps added under “Applied Steps” on the right.

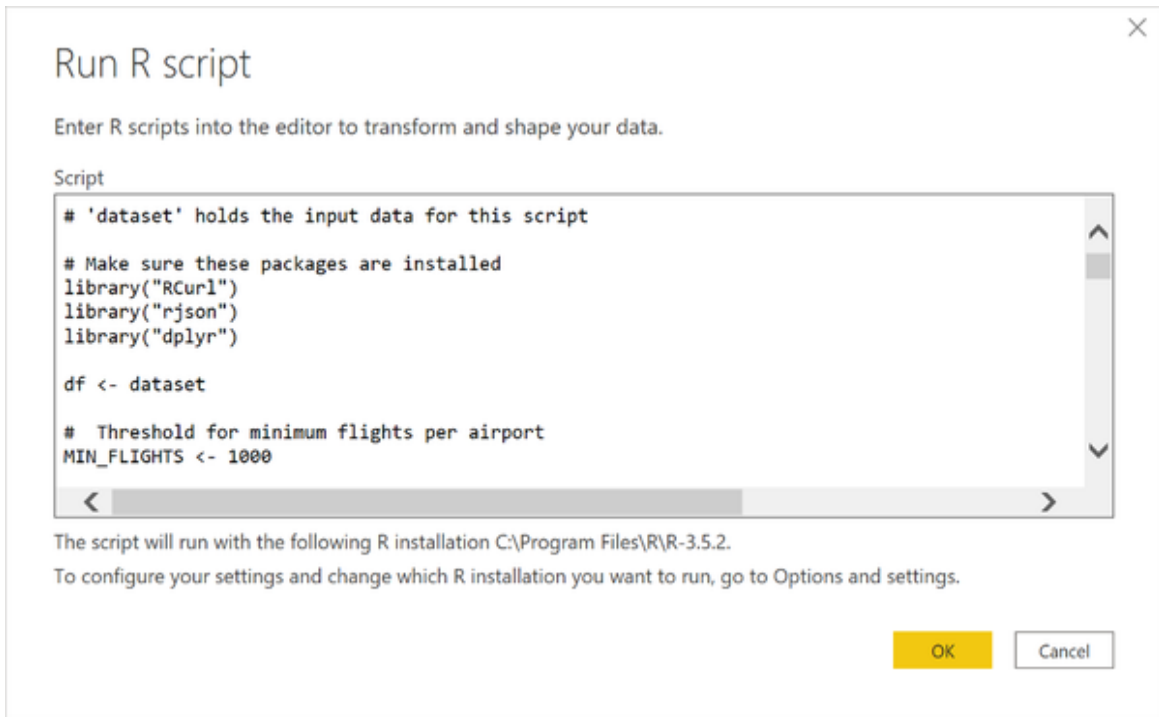
The reason why we didn't include our script from the source file yet is that at this stage it would not work. Power BI converted the FlightDate column from String to a proprietary Datetime format, which is difficult for us to process in Python or R. But we don't want to delete this type conversion, because it will allow a better reporting experience.

What we need to do is apply our R or Python script before the data type conversion takes place. Therefore select the "Change Type" step and pull it to the end of the list. The "Applied Steps" should now look like shown in Figure XX.



With the steps applied in the right order we can now select the "Run R script" or "Run Python script" step once more and edit the script by clicking the cog icon on the right.

Now, we can paste the script from the file to the Power BI code editor as shown in Figure XX. You can replace the existing contents, also the line of code that we just wrote before.



Click OK and wait for the script to be executed. This could take a few minutes, depending on the performance of your computer.

If the script is completed, select the last step “Changed Type” from the list. Double check that the new columns such as expectedValues, isAnomaly, isPositiveAnomaly and upperMargins were properly added to the table. It is OK if the first rows show null values, since we did not do inference for the whole dataset but just for some airports. If you scroll down you should see some results popping up from line 32 onwards as seen in Figure XX.

The screenshot shows the Power Query Editor interface. The main area displays a table with 72 columns and 999+ rows. The columns are: `isNegativeAnomaly`, `isPositiveAnomaly`, `lowerMargins`, and an unnamed column. The data is filtered to show rows 29 through 53. The `isNegativeAnomaly` and `isPositiveAnomaly` columns contain boolean values (TRUE, FALSE, or null). The `lowerMargins` column contains numerical values. The unnamed column contains a mix of null and numerical values.

	<code>isNegativeAnomaly</code>	<code>isPositiveAnomaly</code>	<code>lowerMargins</code>	
29	null	null	null	null
30	null	null	null	null
31	null	null	null	null
32	FALSE	FALSE	FALSE	14,04499899
33	FALSE	FALSE	FALSE	14,00909904
34	TRUE	FALSE	TRUE	14,27461747
35	FALSE	FALSE	FALSE	13,79409105
36	FALSE	FALSE	FALSE	14,12661208
37	TRUE	FALSE	TRUE	14,3505298
38	FALSE	FALSE	FALSE	14,24612203
39	FALSE	FALSE	FALSE	14,36145381
40	FALSE	FALSE	FALSE	13,52106165
41	FALSE	FALSE	FALSE	13,37438712
42	FALSE	FALSE	FALSE	13,31124084
43	FALSE	FALSE	FALSE	13,62827324
44	FALSE	FALSE	FALSE	13,43386079
45	FALSE	FALSE	FALSE	13,74972164
46	FALSE	FALSE	FALSE	13,72290332
47	FALSE	FALSE	FALSE	14,01249075
48	FALSE	FALSE	FALSE	13,97599578
49	FALSE	FALSE	FALSE	14,36467209
50	TRUE	FALSE	TRUE	14,47652056
51	FALSE	FALSE	FALSE	14,55869268
52	FALSE	FALSE	FALSE	14,61404654
53	TRUE	FALSE	TRUE	14,64952233

72 COLUMNS, 999+ ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED ON DIENSTAG

Hit File -> Close and Apply to exit the Power Query Editor. Back in the data model menu, you should see the new list of columns in your model.

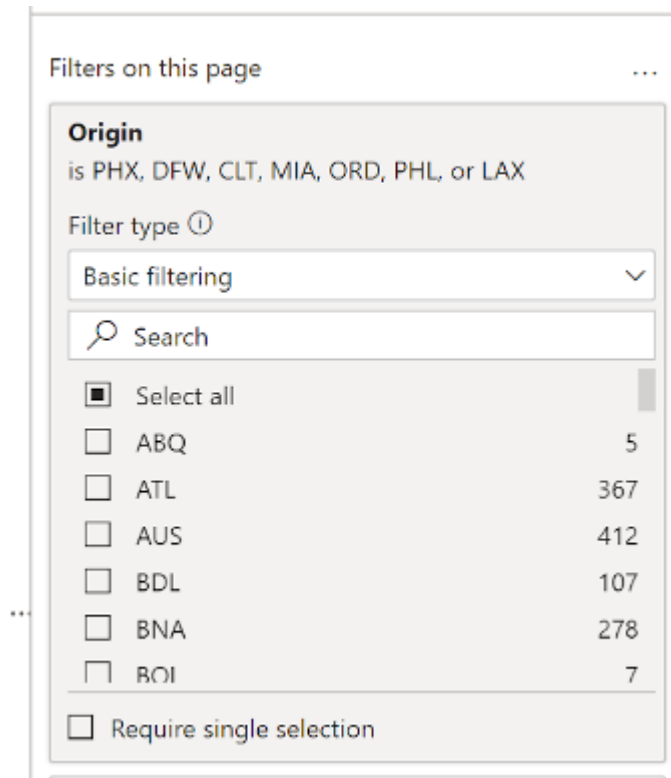
The heavy lifting is done by now. Now, we can proceed to the dashboard and make our predictions visible for report users.

In the “Report” section, add another report page by clicking the plus icon next to the first report page. We don’t want to update the existing visuals, but come up with an approach that fits better to the information we have.

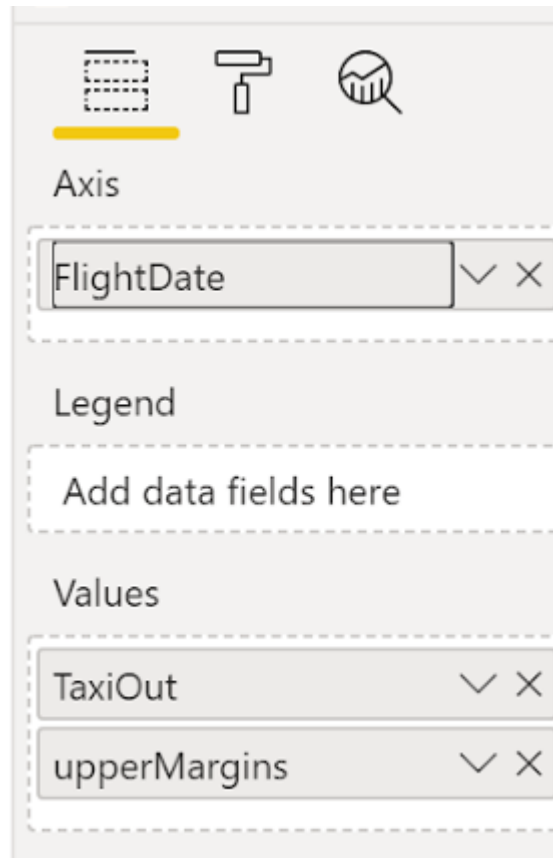
Our goal is to build a line chart where not only the actual average taxi-out time is shown, but also the predicted upper boundaries for the time series. Also, we want to highlight those data points that exceeded the upper boundaries and therefore mark them as anomalies. And finally, we want to provide the user an interactive way to switch between different airports easily,

Before we add any visual, open the Filters pane and add a page filter to this report page as shown in Figure XX. Remember that we only want to

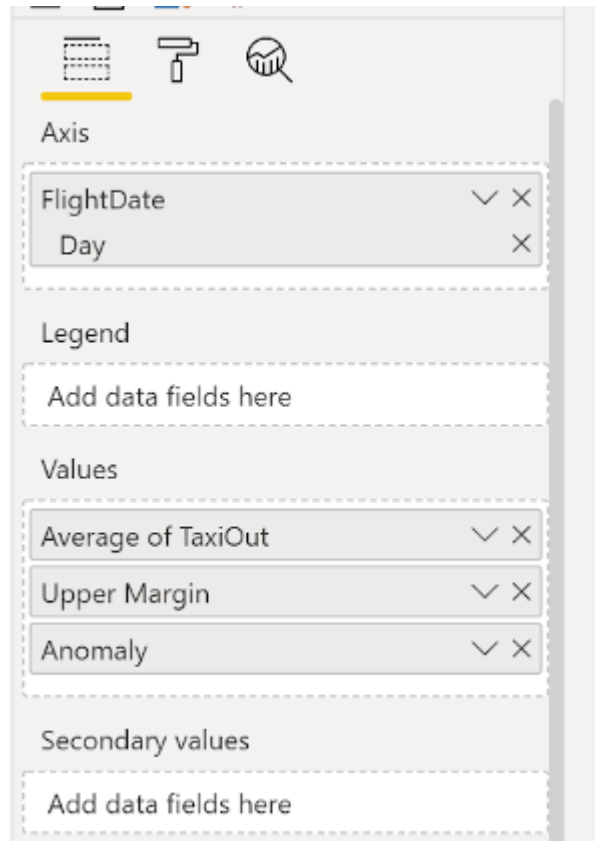
analyse a selected group of airports, so it is convenient to filter them out for all charts. Add the data field “Origin” to the “Filters on this page” area and select the following airports: CLT, DFW, LAX, MIA, ORD, PHL and PHX. The final filter should look like in Figure XX.



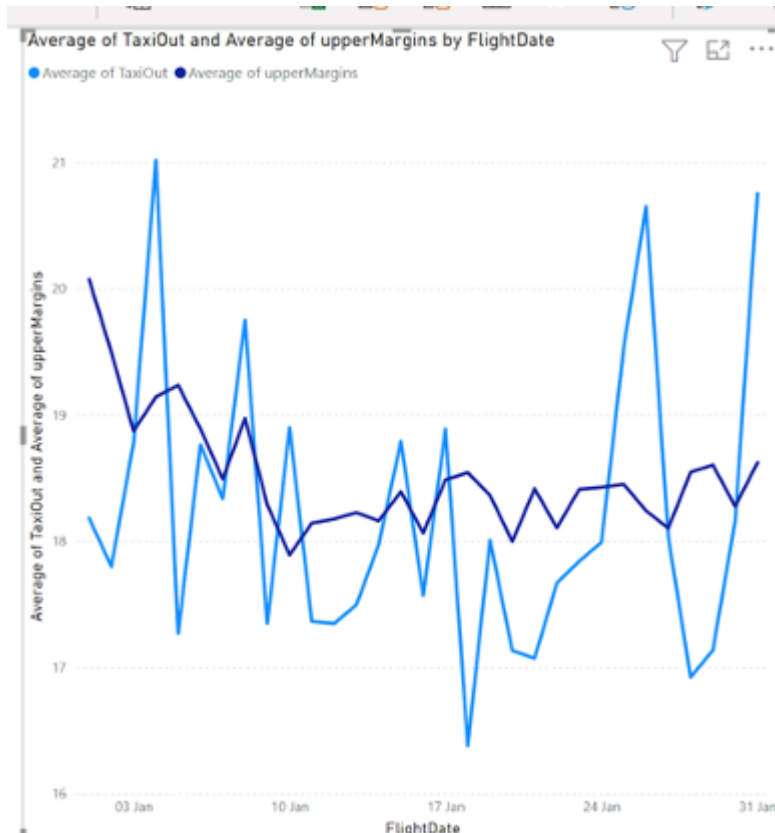
Let's continue by adding the line chart visual. Double click the line chart icon from the visualizations pane. Drag the line chart so that it covers roughly half of the report page. Add the following fields to the axis and values areas as shown in Figure XX: FlightDate (Axis), TaxiOut, upperMarings (Values).



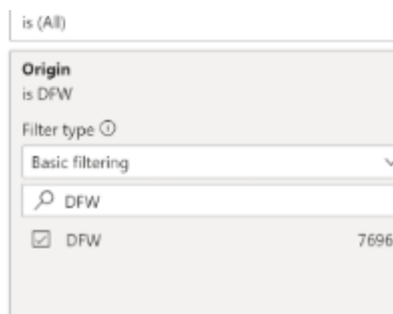
Below FlightDate, delete the Year, Quarter and Month hierarchy by clicking on the small “x” icon so that there is only Day left as shown in Figure XX. Also, right click on TaxiOut and upperMargins and choose “Average” as the aggregation function instead of sum. The field labels should now read “Average of TaxiOut” and “Average of upperMargin”.



By now you should see two lines as shown in Figure XX



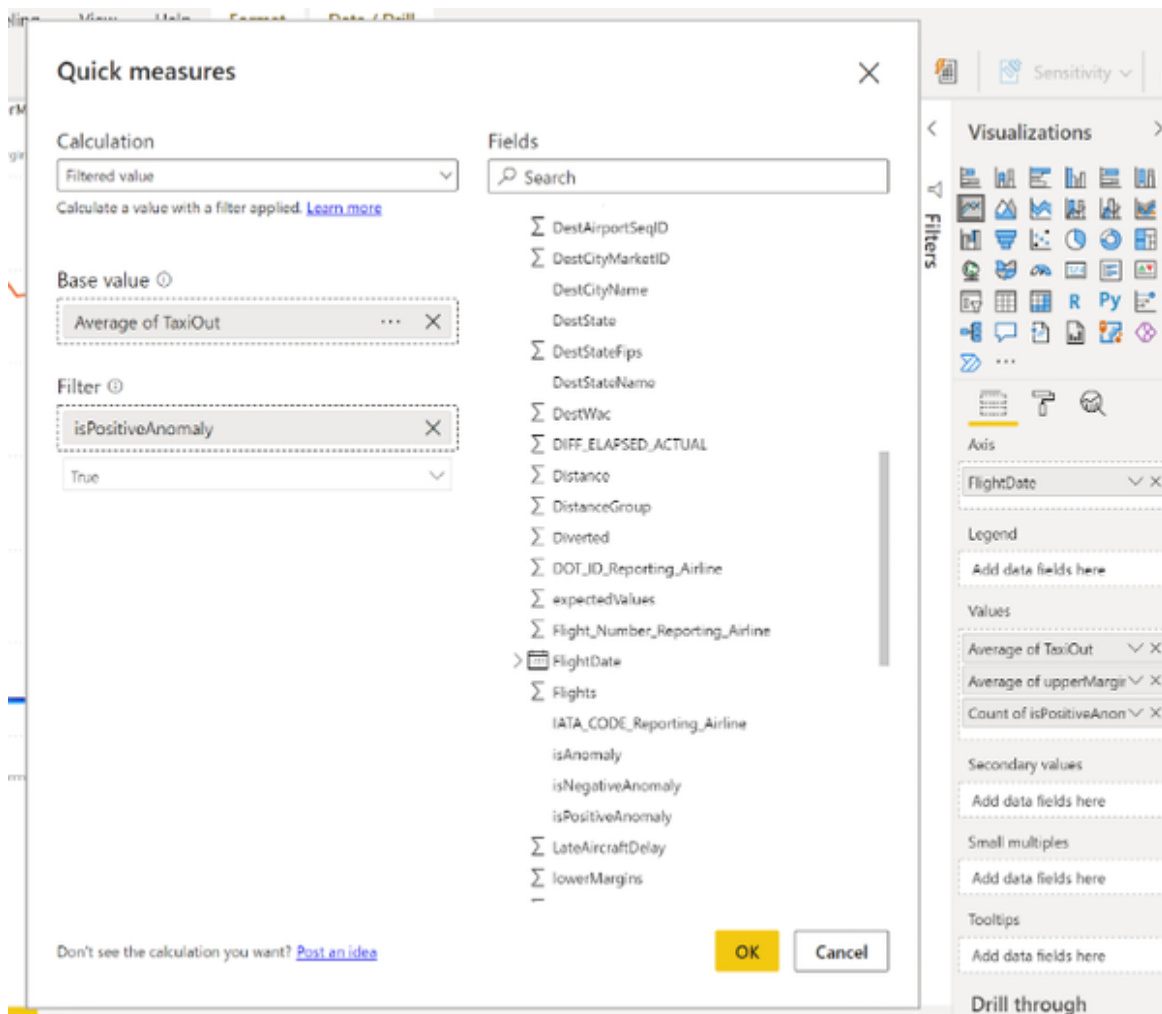
The chart shows the average taxi-out and the average upperMargins for the airports we specified on the page filter. But we want to analyse the airports individually. Therefore, open the filter pane once again and set a filter just for this visual which is “Origin” is “DFW” as shown in Figure XX. We will add the interactivity later so that users can choose the airport they want to analyse.



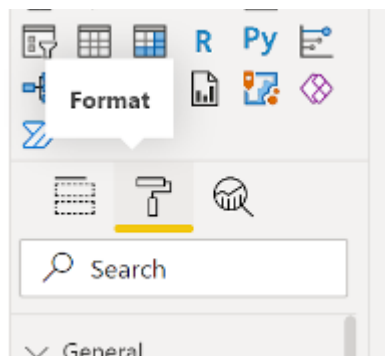
You should have set two filters by now: One filter on the page for 7 airports, and one filter on the visual for DFW airport. Also, we have two lines already on the chart: One line showing the daily average taxi out and

one line showing the predicted upper boundary. What's missing still is the highlight of outliers. We're getting there!

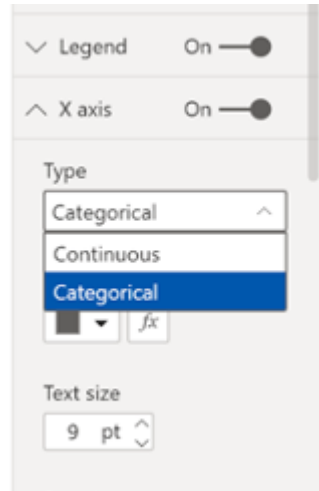
Add the field `isPositiveAnomaly` to the values as the third line. Right click and choose "New Quick Measure". In the window that pops up, set the settings as shown in Figure XX: Select the calculation to be "Filtered Value", set the base value to "Average of TaxiOut" and the Filter to `isPositiveAnomaly equals true`. Confirm with OK. Delete the "Count of `isPositiveAnomaly`" from the values area if it still appears here. There should be 3 values by now, "Average of TaxiOut", "Average of upperMargin" and "Average of TaxiOut for True". Right click on "Average of TaxiOut for True" and choose "Rename for this visual". Rename this field to "Anomaly". Likewise, change the name for "Average of upperMargin" to "Upper Margin" for this visual.



With these three values we have all the information we need on the plot. The rest is “just” formatting. So head over to the Format settings as shown in Figure XX.

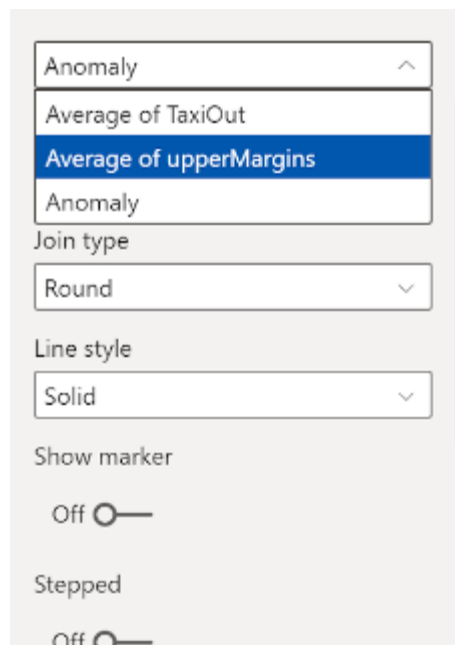


Choose X-Axis and change the type from continuous to categorical.



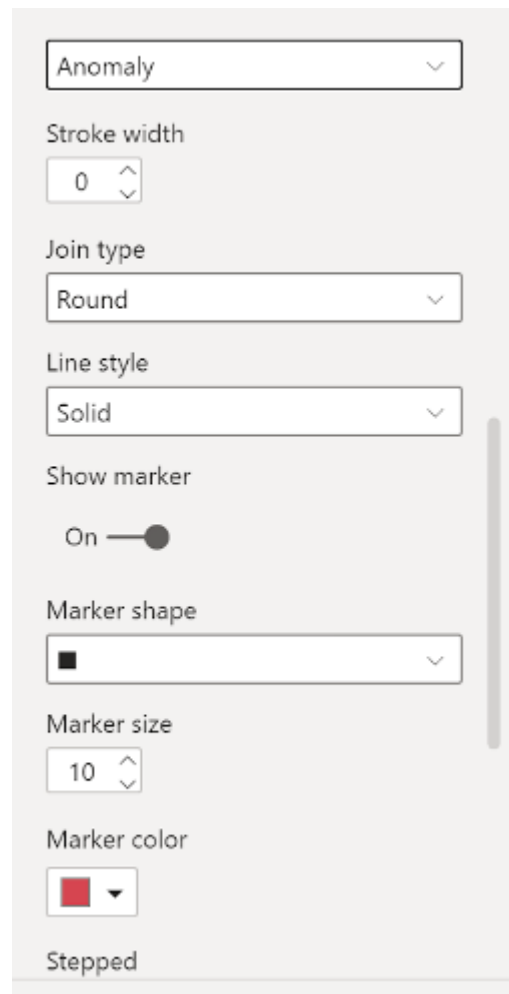
Head over to “Data colors” and change the color for both “Upper Margin” and “Anomaly” to Red.

Next, select “Shapes” and set the stroke width to 6. Toggle “Customize series” on. From the dropdown menu as shown in Figure XX select “Upper Margin”.



We want to give this upper band a different style to make clear that this is a boundary, not an actual data value. So set the line style to “Dotted” and toggle “Stepped” to on. Next, select “Anomaly” from the dropdown menu in “Shapes” and toggle “Show marker” on. Switch the marker shape from

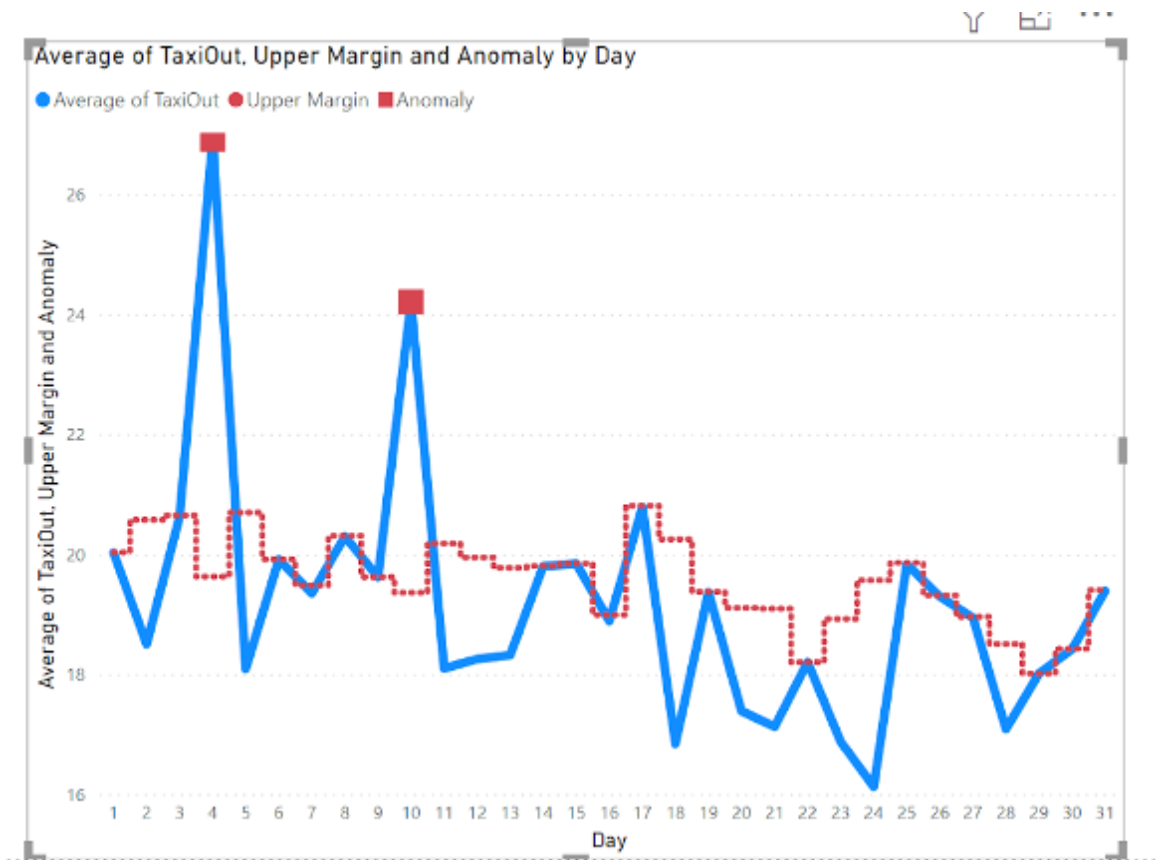
circle to square and increase the marker size to 10 as shown in Figure XX. Set the stroke width to 0.



The image shows the 'Anomaly' settings pane in Power BI. The settings are as follows:

- Anomaly**: Dropdown menu set to 'Anomaly'.
- Stroke width**: Input field set to '0'.
- Join type**: Dropdown menu set to 'Round'.
- Line style**: Dropdown menu set to 'Solid'.
- Show marker**: Toggle switch set to 'On'.
- Marker shape**: Dropdown menu set to a square icon.
- Marker size**: Input field set to '10'.
- Marker color**: Color picker set to red.
- Stepped**: Label at the bottom of the pane.

By now, your line chart should look like in Figure XX and we are almost ready! The only thing missing now is that users can choose the airport they want to examine. We will use the crossfilter feature in Power BI to make this process more convenient for the user.

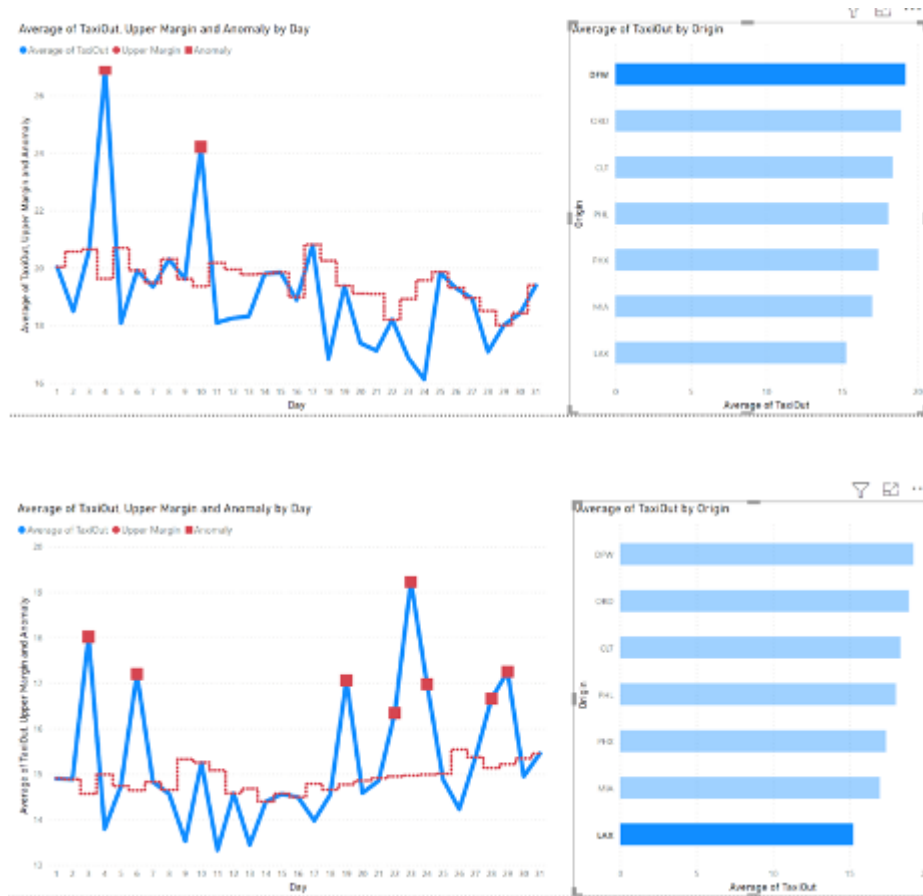


Add a Stacked Bar Chart from the visualizations pane to the report page with a double click. Make sure that your line chart is not selected, otherwise it will replace this visual. The bar chart should automatically fill the empty space right next to the line chart. Add “Origin” to the axis and “TaxiOut” to values. Right click TaxiOut and choose “Average” as the aggregation measure. Under “Format” choose “Y axis” and increase the inner padding to 50%.

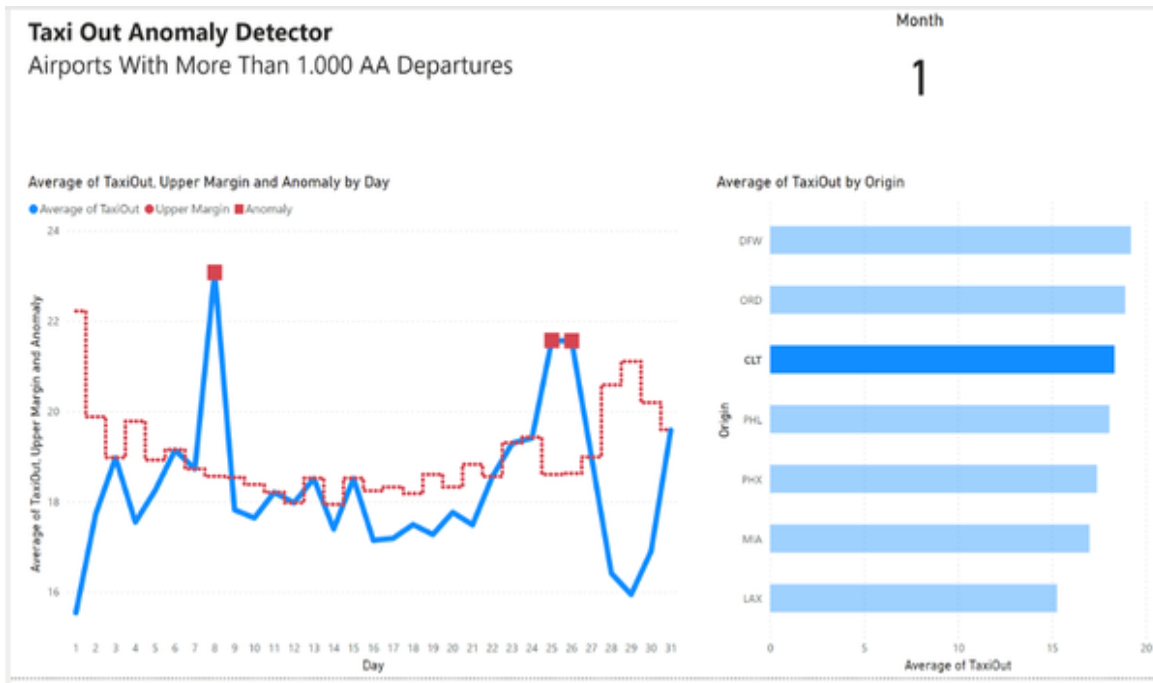
The cross-filter functionality in Power BI will have the effect that once a user selects a bar from the bar chart, the other visuals on the same page will be filtered accordingly. To allow the dynamic filtering for our line chart, select the line chart, open the filter pane and remove the visual filter “Origin is DFW” which we set for the purpose of designing the chart. Leave the page filter untouched.

Now, whenever we select a bar on the right side the left side will show the daily average taxi out time with the anomaly values highlighted as you can

see in Figure XX.



To finish our report off, let's add a headline and the current month to the top of the report by copying the elements from the first report page. Figure XX shows our final dashboard in action. You can find the final state in the accompanied book resource file "Chapter-6-Use-Case-3-AI-Powered.pbi"



Since we now have the attribute `isAnomaly` in our dataset we can easily run further analysis or automated reports for this metric.

To find out more about how the anomaly detection is working in detail, how to run inference results in real time and general best practices around this API, I recommend this [Microsoft resource](#).

With our AI-Powered approach we have helped the operations team to identify anomalies quicker and without adhering to a fixed rule set. Thanks to the AI-prediction, the data label “`isAnomaly`” is now also available in the data model which allows further analysis or automated reporting of this metric, thus supporting the data teams by taking off a lot of manual workload.

Summary

Congratulations! Give yourself a pat on the back because what you just did was nothing less than applying state of the art AI tools to real world data and building actionable BI Dashboards for predictive analytics. We have not only learned how to use historic information in a dataset to predict

future categorical or numeric variables. We have also touched on AI services which give us automated estimations for in this case anomaly detection. It does not matter if you are a Power BI or Tableau user, whether you prefer AWS, Azure or Google Cloud Platform. The fundamental building blocks that you learned and applied in this chapter will help you to get up to speed quickly at any of those platforms. Try it out with your own dataset and get hands-on experience. In the next chapter, we will touch the realms of prescriptive analytics, making intelligent recommendations for decisions on a micro-level.

Chapter 8. AI-Powered Prescriptive Analytics

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 8th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

After we have learned how to leverage AI to analyse data from the past and make predictions about the future, we are now going to recommend actions. In this chapter you will learn how to support data-driven decisions-making by letting an algorithm suggest the best option out of a range of possible actions. Let’s go!

Use Case: Next Best Action Recommendation

For this use case we are building on the suggestions of a predictive model and select the best option for a specific customer.

Problem Statement

We are working on the BI team of a large telco provider. The company sells various products such as monthly or yearly cable and cell phone subscriptions and has millions of active customers each year. The company is facing an increasing problem of churning customers. The data scientists on the team have successfully built a customer churn model which predicts the likelihood of single customers to churn by the end of the current quarter. The churn predictions have been calculated as a churn score where 100 means highest churn probability and 0 less churn probability. While the churn predictor has proven to be quite accurate and useful in identifying those customers which are likely to cancel their subscription with the company, the business still struggles with the right measures to counter churn. The BI team has incorporated a churn prevention dashboard which is shown in Figure 7-1.

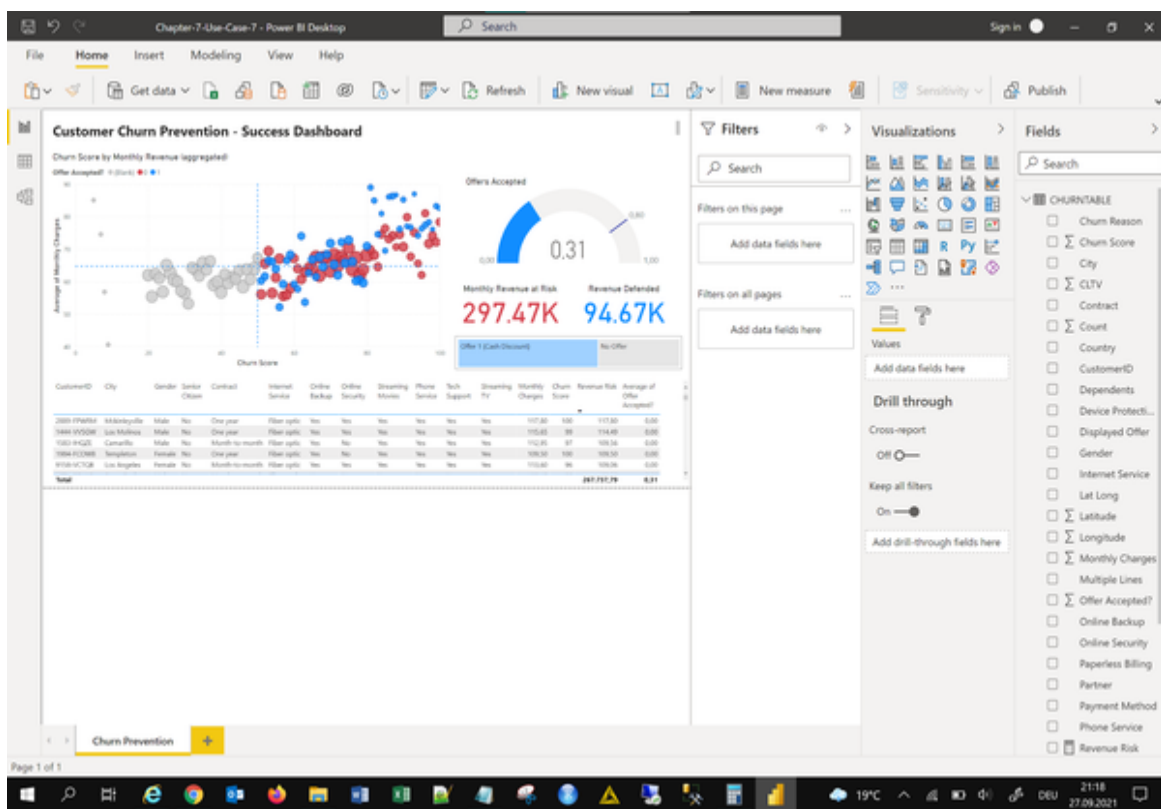


Figure 8-1. Baseline Churn Prevention Dashboard

The dashboard shows a scatterplot on the top left comparing churn score against monthly revenues. In the upper right quadrant are customers who are on an expensive monthly plan and at the same time have a high churn

likelihood, which makes them a particularly relevant target group for churn prevention measures.

Currently, all customers that are labeled as “Churn” get the same retention offer. On average this offer is accepted by 31% of the customers, depicted by the blue dots in the scatter plot. These 31% accepted offers translated to a defended revenue of 94.67 thousand dollars. Sales has indicated that they could provide other offers but were unsure which offer to show to customers. A simple A/B testing has proven to be ineffective due to the large variety of customer segments and different offer types.

The head of the analytics department suggested following a data driven approach to find out which retention offer should be shown to the different customer segments. Our goal is to come up with an approach that at least beats the existing baseline and supports the ability to experiment with new offer types in real time.

Solution Overview

First, we are using the churn prediction results to filter all customers who have a churn likelihood higher than 50%. Next, we will provide them with a customized best offer that reduces the likelihood of quitting the service with the company. In order to find the best offer for customers we will use an approach called reinforcement learning. By experimenting with different offers for different customer groups the reinforcement model will finally figure out which offer to show to which customer group. The model will learn based on a reward system. If a customer accepts the offer, the model will get a reward, if not, the model will be penalized. We will use a Microsoft AI service called Azure Personalizer to run and maintain the model.

In order to get some user interactions, we are also running a small script that mimics offer proposals to users and generates rewards based on the users’ (hidden) preferences. In a real world setting we naturally don’t know these user preferences and have to find them out through experimentation, but for the sake of our simulation, we will keep the ground truth stored in a

plain JSON file. Of course, the model will not have access to this file and needs to find the patterns alone.

The results of our simulation will be logged using a CSV file and stored on an Azure blob storage so we can access them later on through Power BI for further analysis.

Finally, we will use Power BI to monitor our model performance and get an overview which offers are going to be recommended for which user cohorts. With this approach we should be able to see how much better our personalized recommendation is compared to the baseline recommendation

Setting up the AI Service

Navigate to your Azure Dashboard by visiting portal.azure.com. Type “Personalizer” in the search bar type on the top and choose “Personalizers” from the list of services. Hit “Create” which brings you to the “Create Personalizer form as shown in Figure 7-2. Choose the resource group you created in Chapter 6 and give your recommendation service a name such as “offer-engine”. Choose the “Free F0” pricing tier. Hit “Review + Create”. After the final validation passed, confirm with “Create”. The deployment might take some minutes to complete.

Microsoft Azure portal showing the "Create Personalizer" form. The form is titled "Create Personalizer" and has tabs for "Basics", "Virtual network", "Identity", "Tags", and "Review + create". The "Basics" tab is selected.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Azure subscription 1

Resource group * ⓘ ai-powered-bi

[Create new](#)

Instance details

Region * ⓘ East US 2

Name * ⓘ churn-prevention

Pricing tier * ⓘ Free F0 (50K Transactions per month)

[View full pricing details](#)

[Review + create](#) < Previous Next: Virtual network >

Figure 8-2. Create Personalizer Form

Once the deployment of your Personalizer service is finished, navigate to the service by clicking on “Go to resource”. Let’s head over there to see what’s going on. On the service home page you should be greeted with a “Quick start” which looks like Figure 7-3.

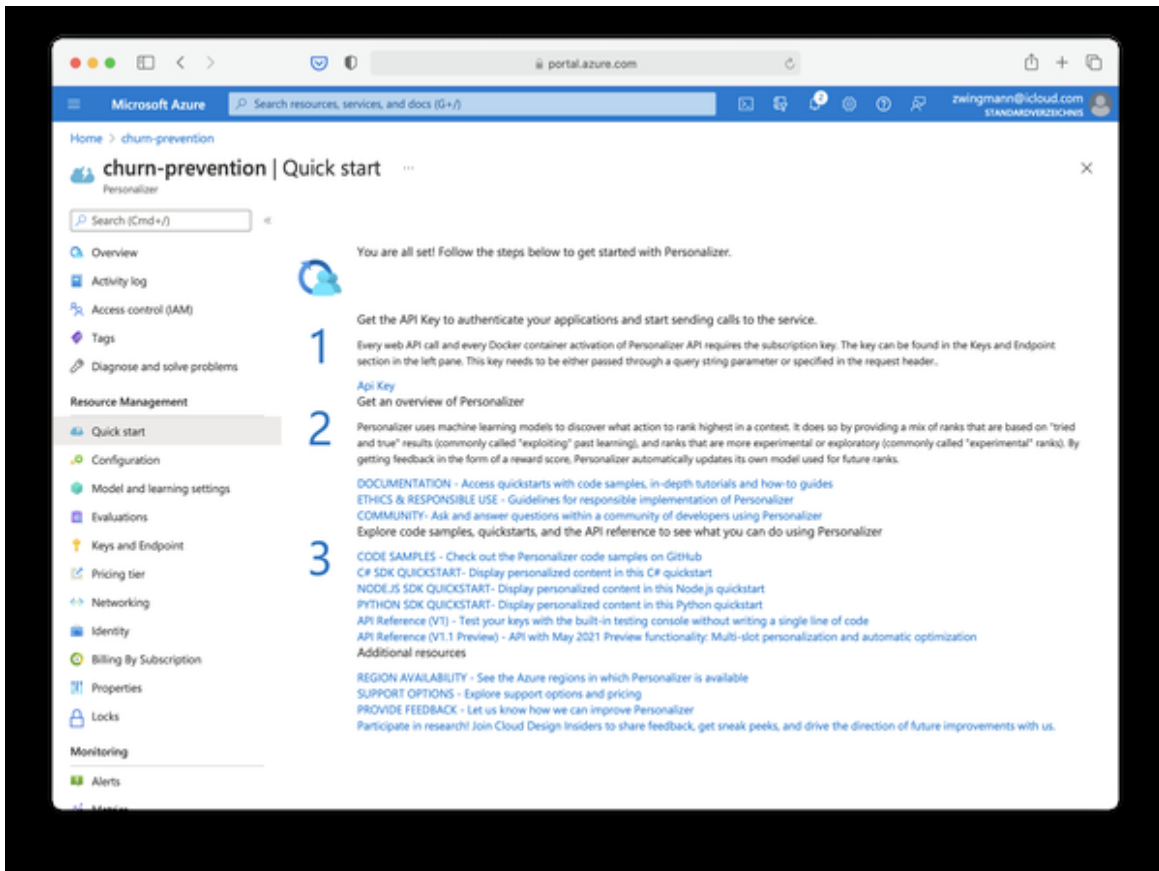


Figure 8-3. Personalizer Service Quickstart

The interface can be a bit overwhelming at first, but we actually don't need to do very much here. First, go to configuration to review your model settings. This is the place where you can tweak the learning behavior of your model. Which time to choose here depends on the data you work with. It is ok normally to start with the standard settings and see how quickly the model picks up any patterns. But for our use case here where we are going to simulate user interactions, we want to keep the wait and model update cycles short. Therefore, set both the "Reward wait time" and the "Model update frequency" to 1 minute as shown in Figure 7-4. Also, we can lower the exploration percentage to 15%, because we know that the user preferences are not changing during our experiments.

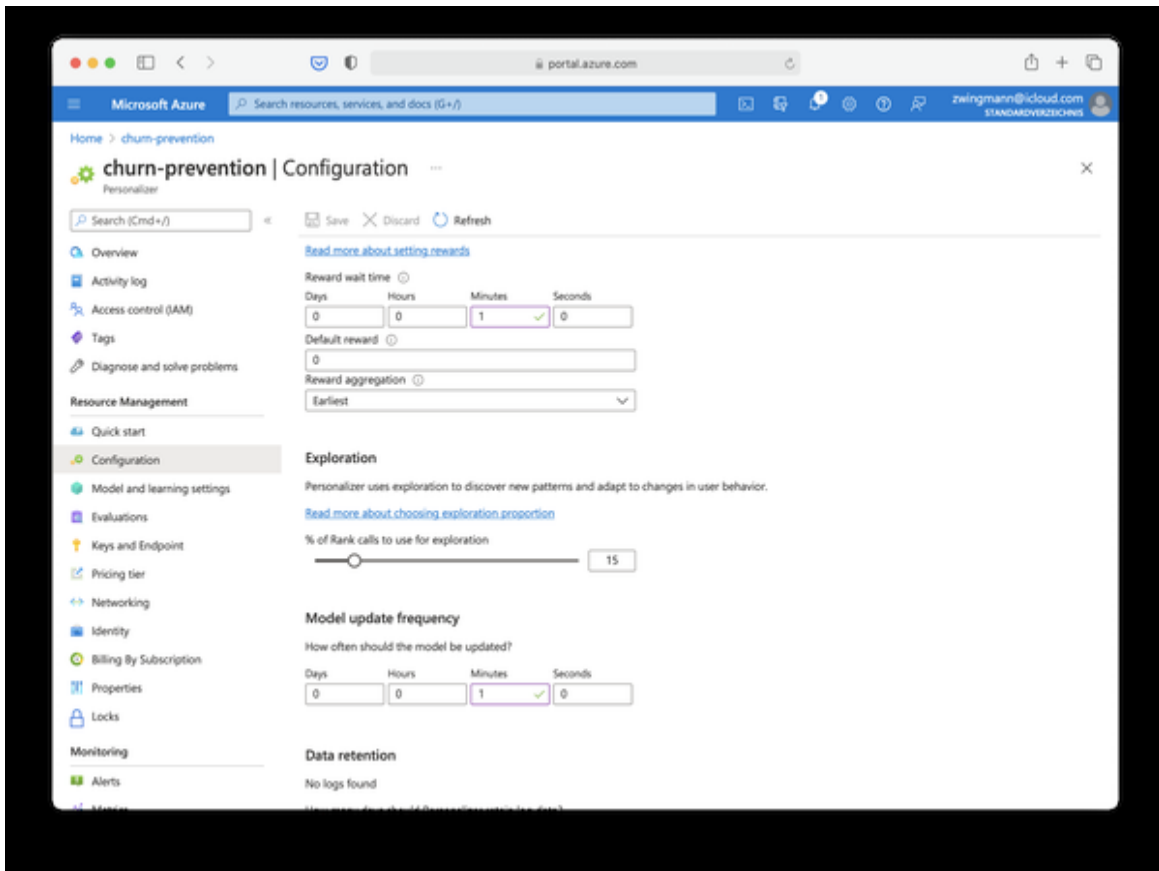


Figure 8-4. Personalizer learning configuration.

Don't forget to save your changes using the small disk icon on the top and verify that the settings have been applied by refreshing the page.

Next, check out the tab learning behavior. This is a very practical feature for real world settings. Here you can adjust if the service should immediately deliver the predicted result or if it should run in a so-called “apprentice mode” which means serving a baseline prediction and collecting data until this baseline can be topped. You can find more information about the learning behavior of the Personalizer service by looking at this [Microsoft support document](#). For our setting, we will leave everything as it is and stay with “Return the best action, learn online”.

To access the model and send requests to it we need both the unique model URL and the corresponding access keys. You can find both by navigating to the “Keys and endpoints” menu in the navigation pane on the left as shown in Figure 7-5.

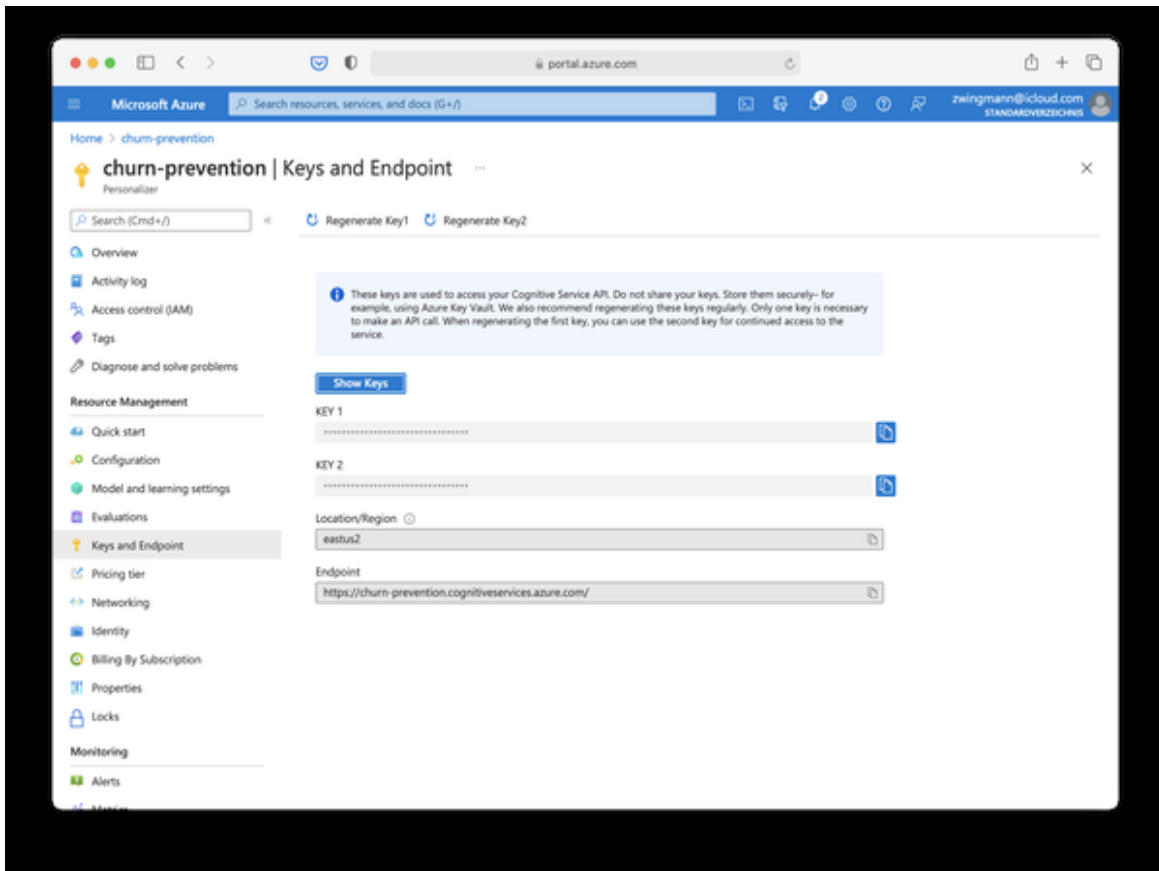


Figure 8-5. Access keys and URL to Personalizer service

This screen is showing you two essential information:

First, it gives you two access keys which you need to authenticate against the API, so you are allowed to make rank and reward requests. And second, you will find the model URL which you will need to access it. Leave this page open for later, because we will need this information when we interact with the model.

Now that we have set up our model and got the credentials to use it, it is a good time to review how the process of learning actually happens and how the model works. This is shown conceptually in Figure 7-6:

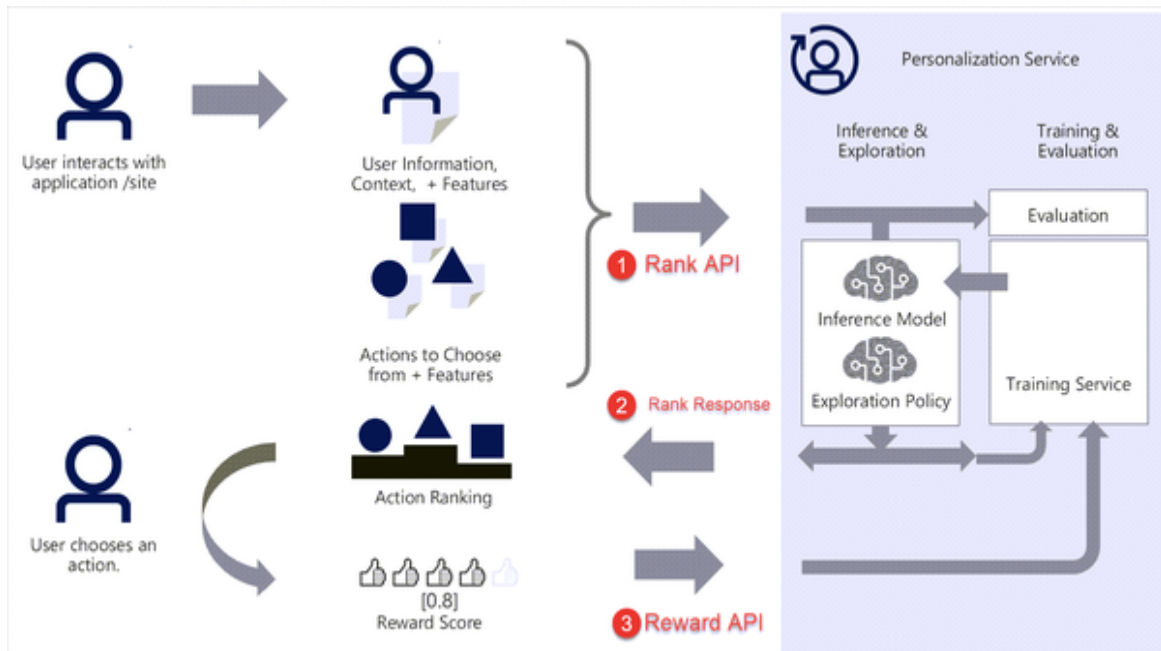


Figure 8-6. Conceptual flow of Personalizer service. (Source)

A typical request to the model incorporates both a rank and a reward call. Both of them are referred to as a learning loop: The rank function suggests something to the user and decides whether it should *exploit*, i.e. show the best action based on past data or *explore* - meaning to select a different action to see if the overall recommendation result can be still improved.

After the service has suggested an action to the user, the model expects a reward. This can be anything from a simple flag such as 0 for “not successful” or 1 for “successful”, in our case if an offer got accepted or not - or it can be a continuous value that can basically represent anything from scroll depth of news articles to profit won from a stock trade – reinforcement learning is a pretty universal concept which can be applied in any scenario where the system follows a defined set of rules.

In the case of the Azure Personalizer service you can see from Figure 1 that the model expects two things to be sent to the rank API: Information about the user (context features) and information about the actions to choose from (1). The model will do its magic and send a rank response (2) suggesting the option which yields the highest chance of a reward (if running in exploit mode). The user will choose an action (or not) and the result will be sent back to the model to the reward API (3). The model will take this feedback

and update itself according to the learning policies that were defined upfront.

Equipped with this new knowledge about Reinforcement Learning, let's move on to get some hands-on experience about how a learning loop looks like in practice and how we can utilize this in our BI.

To simulate user interactions with our offers we need to run some Python or R code outside of Power BI.

Setting up Azure Notebooks

Feel free to skip this section if you already use a program on your computer to run Python or R code (such as PyCharm, VSCode, Jupyter Notebook, RStudio, etc.) and proceed directly to “Simulating user interactions”.

If you don't, then follow along and I'll show you a quick way to execute Python code without having to install any software on your machine.

Navigate to ml.azure.com, choose your preferred workspace and navigate to “Notebooks” as shown in Figure 7-7.

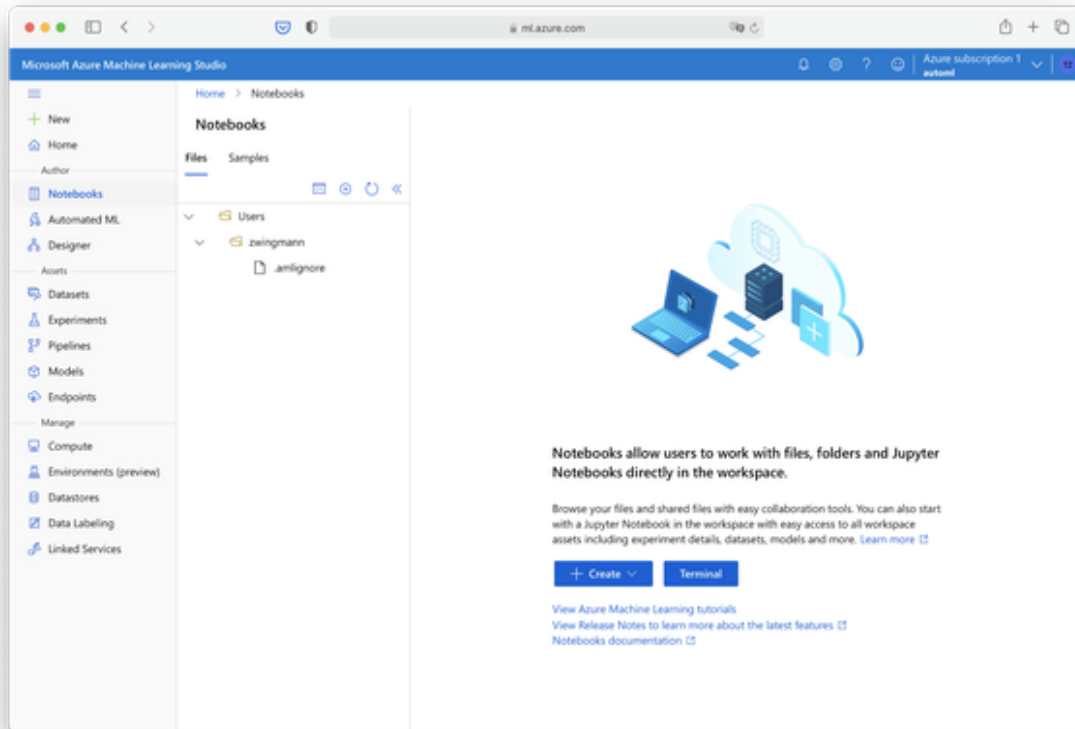


Figure 8-7. Azure Notebook

Choose Create → Create New File as shown in Figure 7-8. Give your notebook name such as “offer-simulation.ipynb” and make sure that the file type is set to “Notebook (*.ipynb)”. Proceed by clicking create.

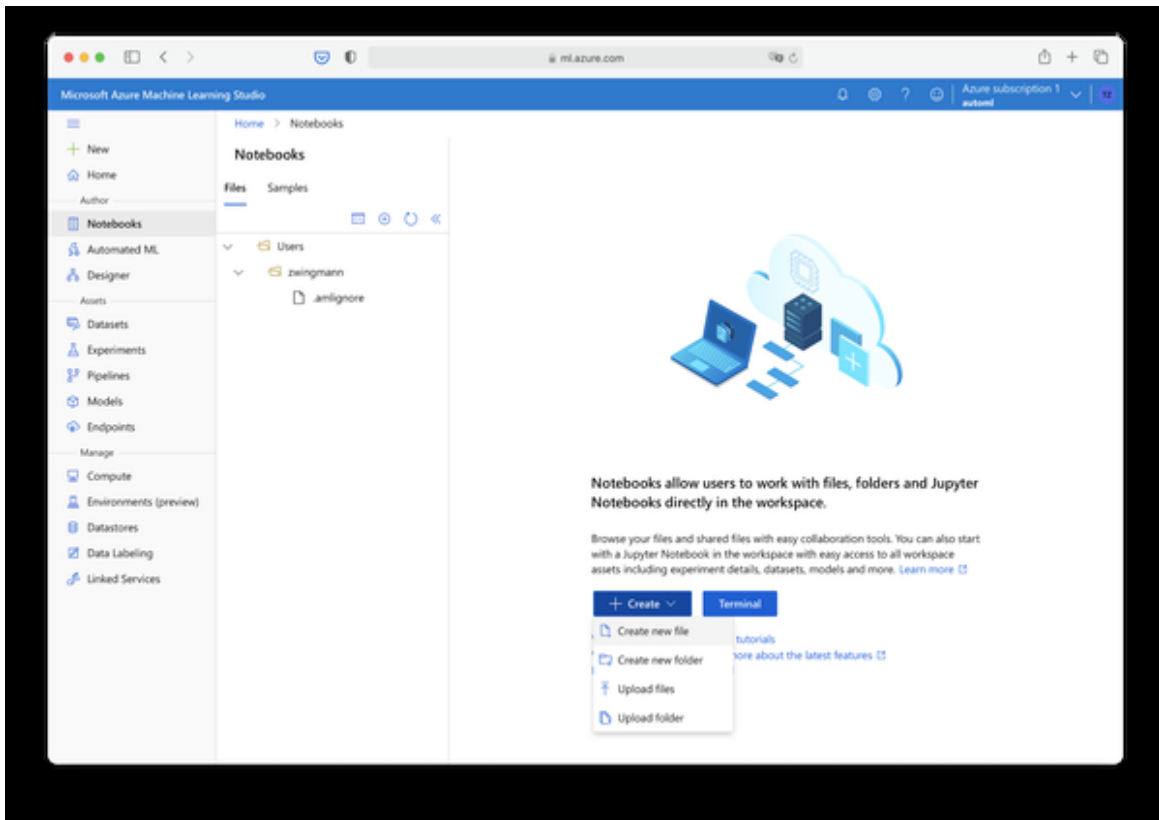


Figure 8-8. Create a new Azure Notebook

You should now find the new notebook in the file tree on the left as shown in Figure 7-9.

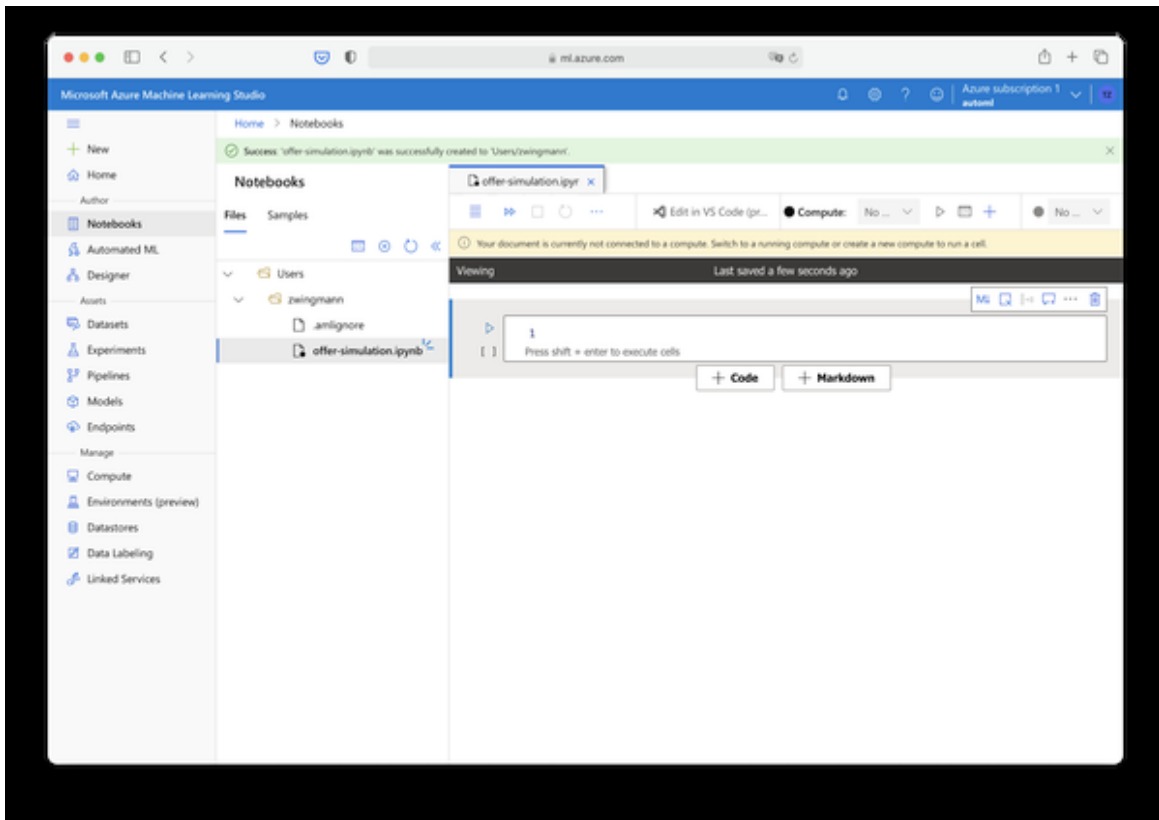


Figure 8-9. Azure Notebook interface

Now that we set up the code editor that lets us interact with a program, we need to provision some compute resources where the code is actually being executed.

Hit the plus icon as shown in Figure 7-9 and choose “Create Compute”. Give your compute resource a name such as “offer-simulation”, make sure the machine type is set to “CPU” and select the cheapest resource available, which should be the machine type “Standard_DS1_v2” as shown in Figure 7-10. You can really choose any machine type here, just make sure the price is cheap because we don’t need performance at this moment.

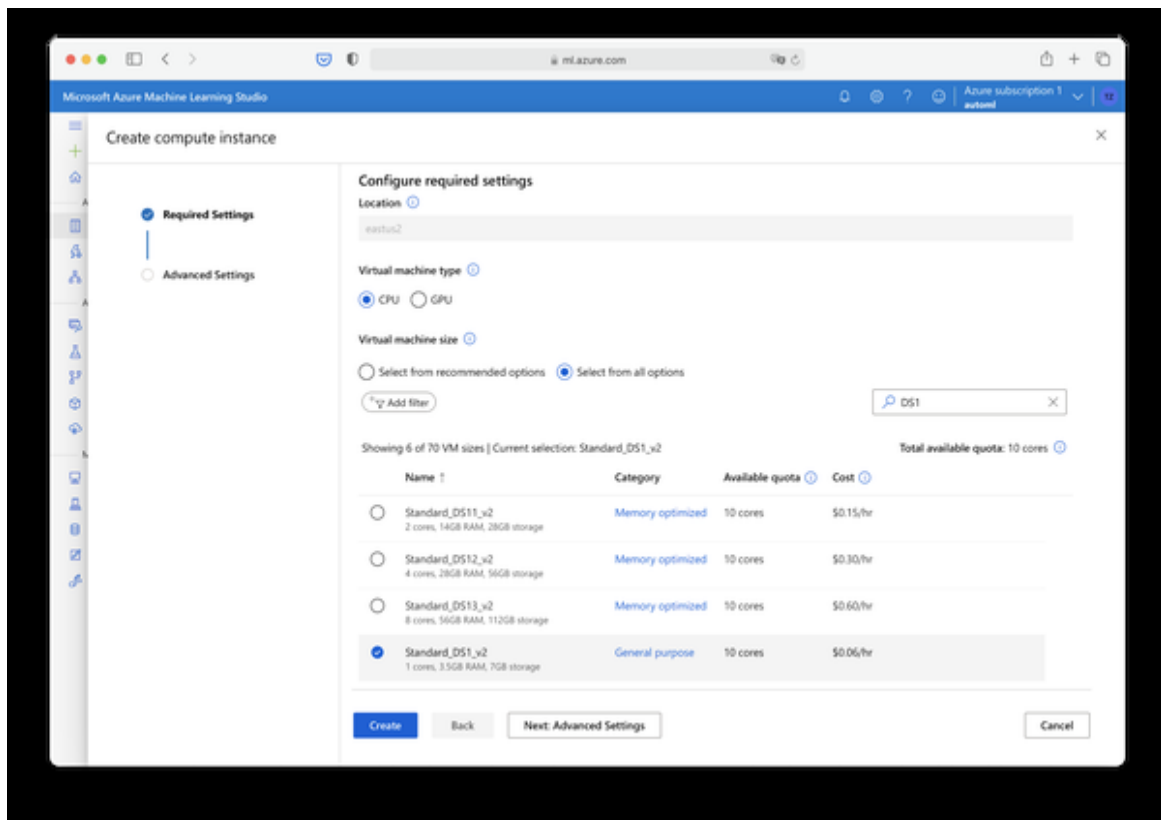


Figure 8-10. Create new compute instance.

Hit “Create” to provision your new compute resource. The creation of your compute resource might take some minutes. Once it is completed you should see a success message and the new resource be shown as in Figure 7-11.

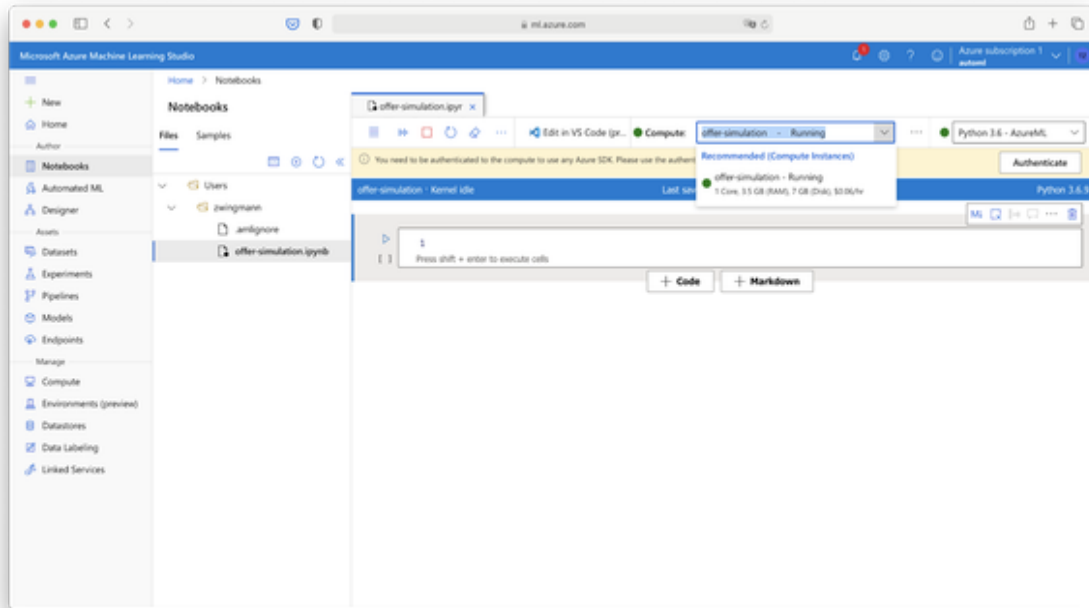


Figure 8-11. Compute instance list

NOTE

Once you create your compute resource and this resource is online you will be charged according to the price mentioned in the compute resources list (e. g. 0.6\$/hour). If you are still using the free Azure credits this will be deducted from them. If not, then your credit card will be charged accordingly. We will only need this compute instance for an hour or so to run the simulation. Once you complete this chapter, make sure to delete your resource. For this go to Manage → Compute and select your compute resource from the list. Choose either “stop” to pause the compute service or “delete” to remove it completely

From the top right you can choose which programming language kernel you would like to use. You can choose either Python 3.6 or R as shown in Figure 7-12 to follow along with the simulation code example in the next step.

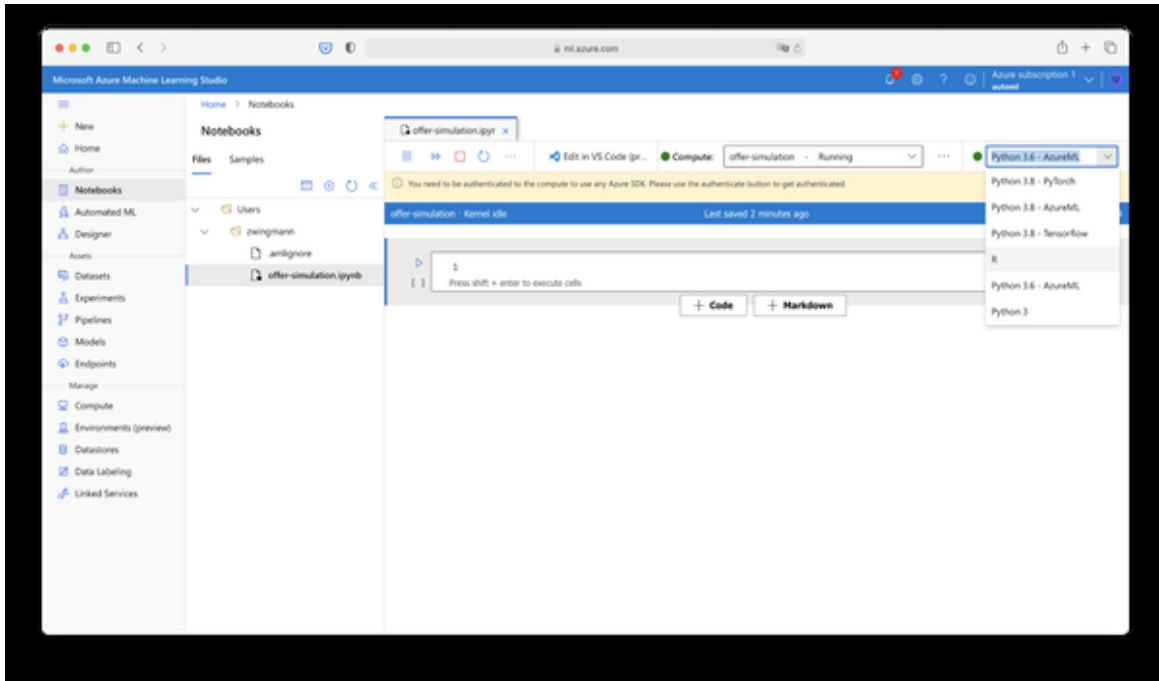


Figure 8-12. Switching between programming languages in Azure Notebooks

Simulating user interactions

Lets finally simulate some user interactions with offers. For this, we will make two assumptions: First, we define a finite set of offer possibilities with a rather simple complexity. And second, we assume that we know which user segment accepts which offer. With this information we are able to let our model start to randomly suggest offers to different customer segments and it should pick up the underlying patterns through exploration. We also keep the complexity of the user segments relatively low because we don't want to exceed our 50k API calls of our free tier. Remember, we need one call each for each rank and reward function and a typical reinforcement model needs a couple of thousands iterations to start learning meaningful patterns. Therefore we'll keep the complexity low. However, we will adhere to one principle: The model does not know which customer segment prefers which offer type and it has to find this out on its own.

Normally a reinforcement learning model needs a couple thousand iterations to learn meaningful patterns. That's why we go for low complexity at the beginning so we can get good learning results with fewer

learning loops (and you stay within the free credits). But don't be limited by the example. You can freely add more context features about customer segments or action features or new offers if you like to shoot for higher goals.

For our simulation we need three components:

1. Information about the available offer types and offer attributes
2. Information about customers (context features) and their true preferences (ground truth) to calculate the reward offline
3. A place where we can store the interaction data so we can analyse them later on through Power BI.

You can find the first two components in the accompanying book repository.

The first component is a file called “offers.json” which lists the available offers and the corresponding features. There are three offers available to prevent churn: Free Month, Free Upgrade and Cash Discount. To be fair, we assume that all of these offers have the same value, so there isn't any offer which is a bigger bargain than the others. It's purely a question of preference.

The second component is a file called “preferences.json”. It includes the information about which customer segment prefers which offer type. You can find the information shown in tabular form in Table XX.

Table 7-1: Rule Table with Customer Preferences

Senior citizen	Contract	Preferred Offer
Yes	Month-to-Month	Cash Discount (Offer 1)
Yes	One Year	Free Month (Offer 2)
Yes	Two Year	Free Month (Offer 2)
No	Month-to-Month	Performance (Offer 3)
No	One Year	Performance (Offer 2)
No	Two Year	Cash Discount (Offer 1)

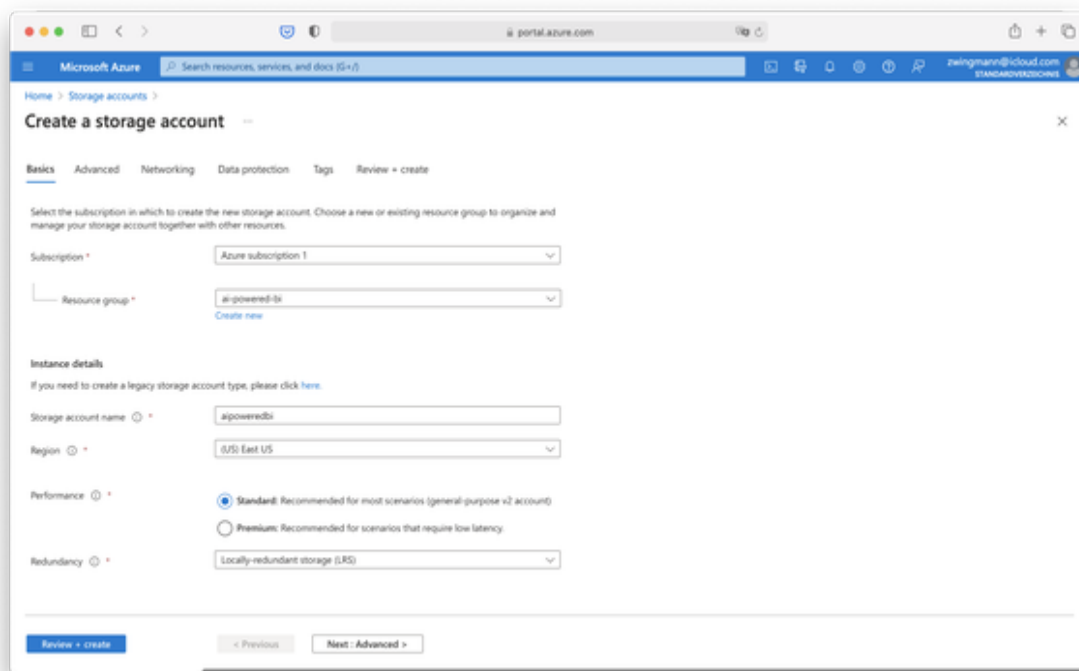
These are the patterns which are unknown to our model and we want it to pick up automatically. As mentioned before, we keep the complexity low by only considering only the variables “Senior citizen” and “Contract”, but of course the Personalizer service is able to handle a lot more features. In fact, the more features we provide about the offer context, the better - given we have enough time / credits to learn.

And finally, we will need a place where we can store the results of our simulation. To keep things simple (and also maintain accessibility via Power BI) we will save the results as a plain CSV file. You can either save the results offline on your computer and load them later on manually into

Power BI or - if you want to have a bit more real-world experience - write the outputs to an Azure blob storage. Trust me, it's not as complicated as it sounds. Follow me for these steps real quick to set up an Azure blob storage:

Setting up Azure Blob Storage

Visit portal.azure.com and search for “storage accounts”. Click “Create” and create a new storage account in a recommended region such as “East US” and give this account a unique name, select standard performance and Locally-redundant storage as shown in Figure 7-13. As this is only test data we don't need to pay more for higher data availability standards. Again, we will only save data here temporarily and the expenses will be more than covered by your free trial budget.



The screenshot shows the 'Create a storage account' wizard in the Microsoft Azure portal. The 'Basics' tab is selected, and the 'Review + create' button is visible at the bottom. The form contains the following fields and values:

- Subscription:** Azure subscription 1
- Resource group:** apowered-bi
- Storage account name:** apoweredbi
- Region:** (US) East US
- Performance:** Standard (Recommended for most scenarios (general-purpose v2 account))
- Redundancy:** Locally-redundant storage (LRS)

Figure 8-13. Create a storage account

Once the deployment is complete click on “Go to resource”. You will see an interface as shown in Figure 7-14 which is very similar to all other Azure resources we have encountered before.

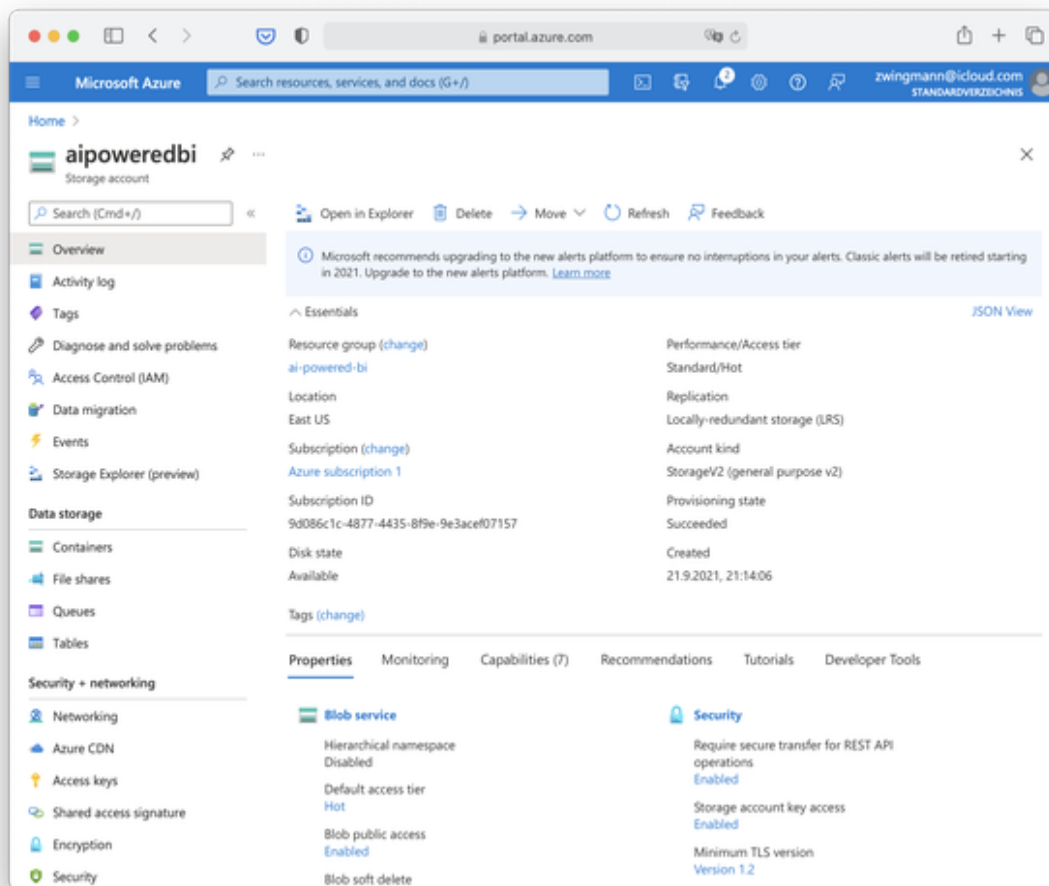


Figure 8-14. Azure storage account interface

Navigate to “Access Keys” as shown in Figure 7-14 and leave this window open as we will need this key in a bit.

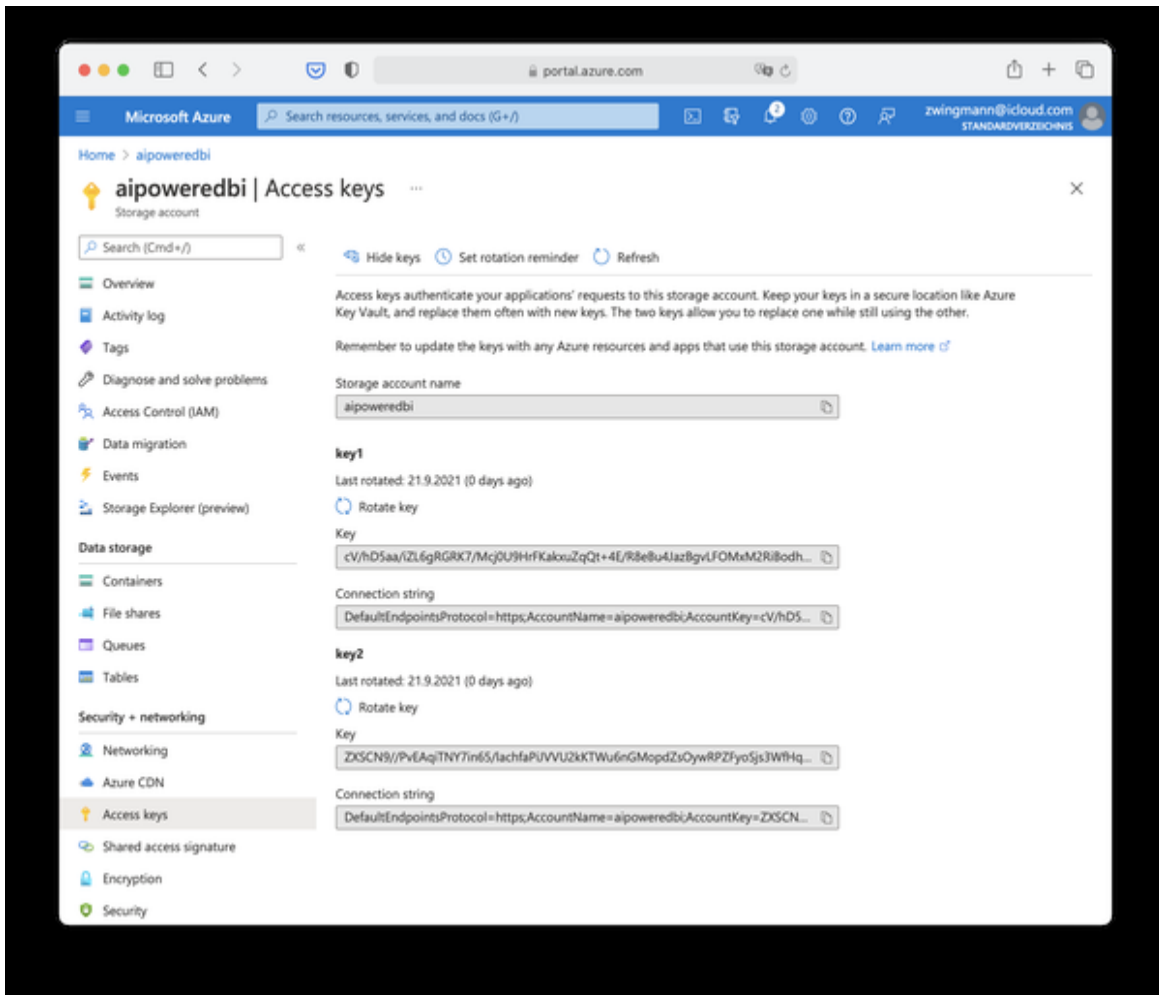


Figure 8-15. Access keys for Azure storage account

Now that we have a storage account and keys to manipulate data here, we have to create a so-called container which is something like a file folder that helps us organize our files.

Navigate to “Container” on the right side of the menu and click “+ Container” on the top. Set the access level to “Container” which will make it easier for us to access the files later through Power BI. Beware, if you deal with sensitive or production data you would of course choose “Private” here to make sure that authorization is needed before the data can be accessed. Voilà, the new container should appear in the list and is now ready to host some files for us.

Let’s leave the container for now and proceed to run the actual code that is doing the user simulation for us.

Running the Simulation with Python or R

Depending on your preferred programming language, continue with the section reviewing either the Python or R code for this task.

Go to the book's repository and find the files “user-simulation.py” and “user-simulation.R”. Open one of these files in your favorite IDE. If you are using Azure Notebooks, open the file in a text editor, select all and copy paste the contents into the notebook which you have created previously. The result should look like in Figure XX. Make sure that the language kernel is set to R or Python accordingly, depending on what you prefer.

However, don't run this code yet!

There are some custom edits you have to make first, but first, let's run briefly through what the code does.

The structure of the code is the same for both the Python and R version of the script.

The code consists of five sections:

The first code section is called “User Inputs”. This is the area where you need to customize some things:

- `OFFERS_FILE_PATH`: The file path to where the “offers.json” file is located.
- `PREFERENCES_FILE_PATH`: The file path to where the “preference.json” file is located.
- `USER_TABLE_FILE_PATH`: The file path to the churn user table
- `PERSONALIZATION_BASE_URL`: The endpoint which is shown in the “Keys and endpoint” in the Azure Portal
- `KEY`: One of the access keys from the same page in the Azure portal (does not matter which of both keys). Make sure you replace this key if you decide to regenerate it.

- `AZURE_CONNECTION_STRING`: The Azure connection string to authorize for the blob storage, will be ignored if empty.
- `AZURE_CONTAINER_NAME`: The name of the Azure blob container, will be ignored if empty.

The second code section is called “Dependencies” and loads all required packages for this script. Verify that you have these packages installed on your machine or otherwise the code execution will fail. Especially the package `azure-storage-blob` is only needed if you plan to upload your results to Azure blob storage.

The third section is called “Functions” and basically contains four main functions:

- `add_event_id`: This function generates a unique ID for each rank call. The ID is used to identify the rank and reward call information. This value could come from a business process such as a web view ID or transaction ID.
- `add_action_features`: This function adds the entire list of offers to the JSON object to send to the Rank request
- `get_reward_from_preferences`: This function is called after the Rank API is called, for each iteration. It compares the user’s preference for an offer, based on their seniority and contract type, with the Personalizer’s suggestion for the user for those filters. If the recommendation is correct, a reward of 1 is returned, otherwise the reward is 0.
- `loop_through_user_table`: This function contains the main work of the script. It will iterate through each line of the provided user table, request a personalized offer from the API, compare to the actual user preferences and calculate a reward score which is then sent back to the Personalizer service. All results will be collected in a list called “results” and this will be returned by the function upon completion.

The fourth section called “Execution” will invoke the functions mentioned above and run the actual workflow.

The fifth section “Export” will persist the results by saving locally to a flat file called “results.csv”. If Azure storage credentials were provided, the results file will also be uploaded to Azure blob storage.

Verify that you have customized all the inputs in the first code section.

Now, run the code!

An iteration over the whole customer table should roughly take 15 minutes.

When the process is completed, you should see the new file “results.csv” being created in your respective working directory. This is basically the documentation of our experiment (and would be the log files of a real-world scenario). If you specified an Azure connection string and a valid container name in the inputs, then this file will automatically be uploaded to your Azure container. If something fails with the upload, make sure that you don’t run the whole script again but only the last part in section 5 which handles the file upload.

Evaluate Model Performance in Azure Portal

To quickly get a glimpse of how the model is working we can check the model performance in Azure Portal.

Navigate to your Personalizer resource and select “Evaluations” from the menu on the left side. Scroll down and you should see the average reward score as shown in Figure 7-15.

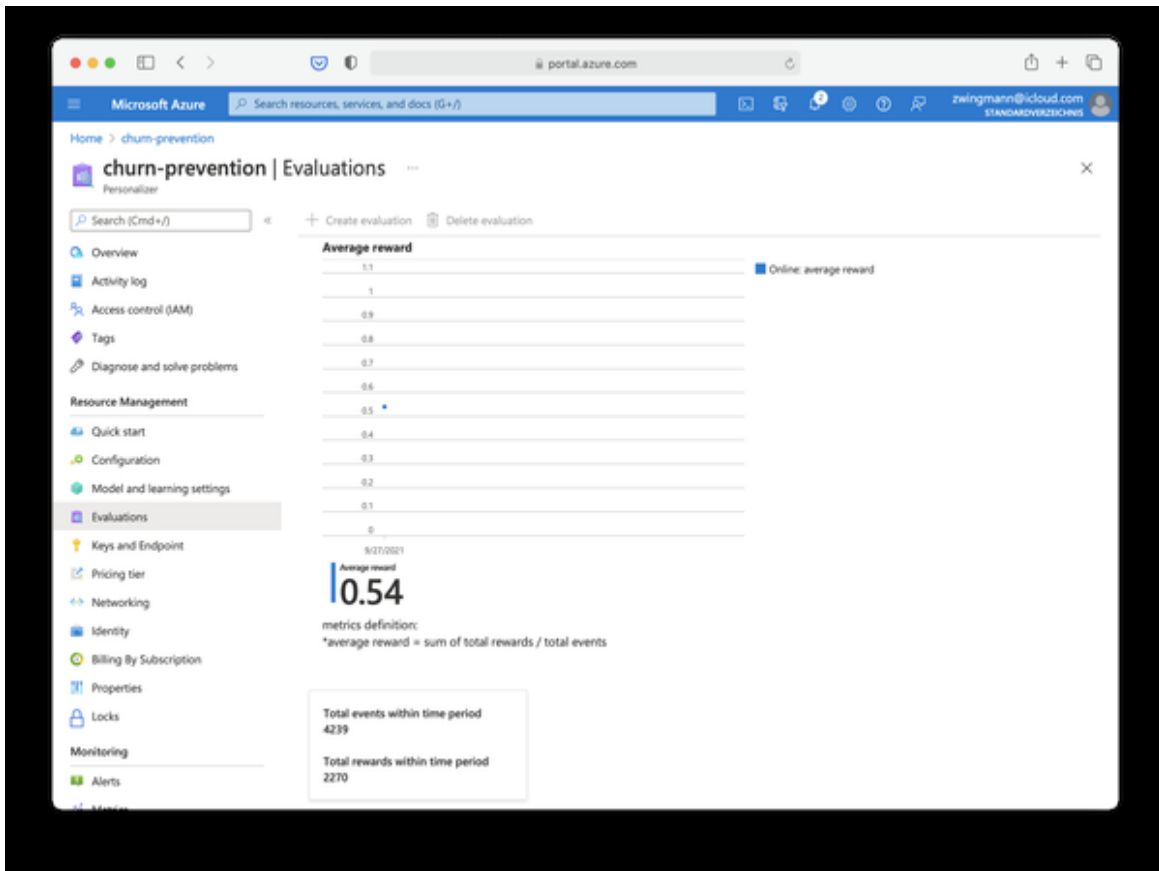


Figure 8-16. Personalizer evaluation

In our case you can interpret this number as a percentage of how often the model chose the correct offer for a customer. By this time the reward score should be still relatively low.

Remember that we started with a naive model making random guesses and only figuring out potential patterns after a couple thousands of experiments. Personalization services typically require tens of thousands of examples to really perform well in real world scenarios. If you run the above simulation code 2-3 times more, you should consequentially see this number increasing. If this is not the case then it's a clear sign that something is going wrong with the learning behavior of the model. You can check these [Microsoft FAQ to troubleshoot errors in model learning](#).

There is one thing, though, we can still do to improve the performance without having to run through the whole dataset again. This feature is called offline evaluation. It is a technique which allows you to use the existing

data and retrain your model with different learning policies to find out which one works best for your data. Let's try this out to find if we can get an even better model than the one we trained so far. From the top menu choose "Offline evaluation" as shown in Figure 7-16.

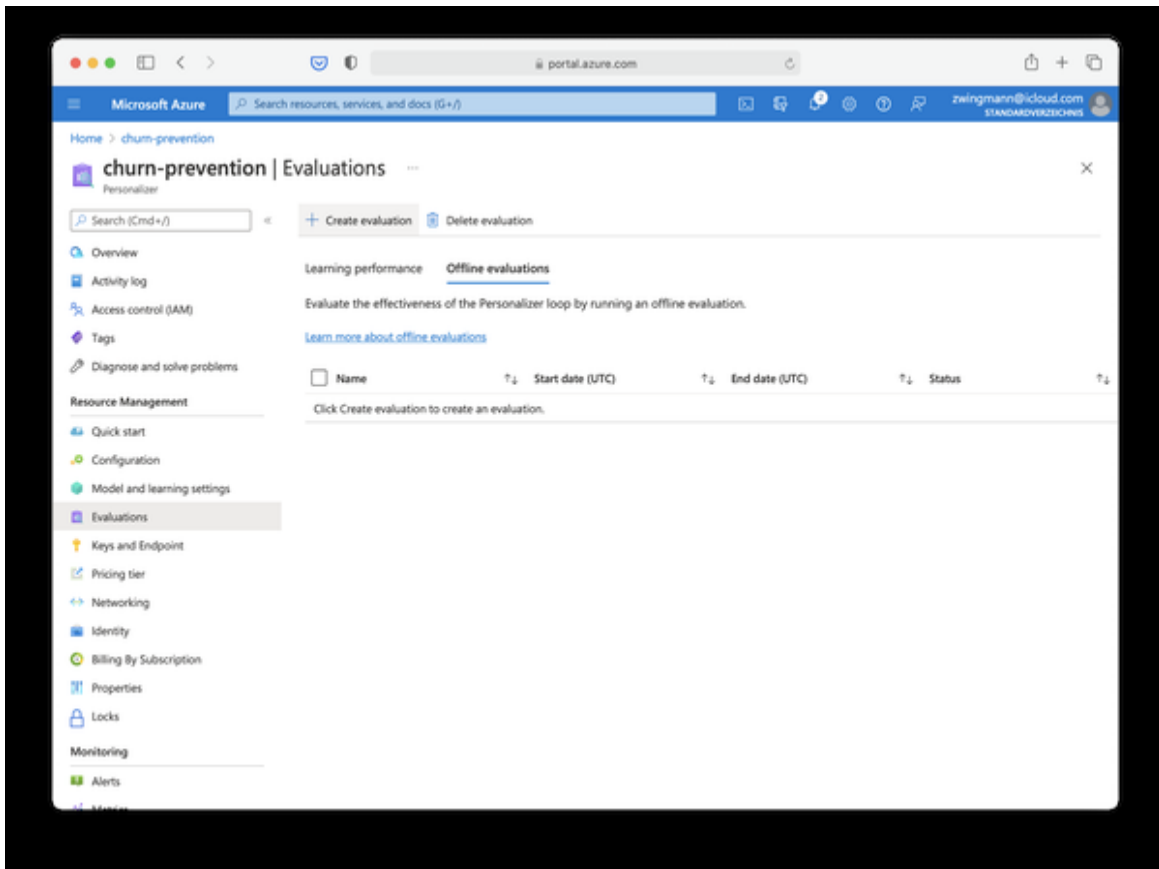


Figure 8-17. Create an offline evaluation

Double check that the date range matches the day where you run the simulation experiment and leave all other settings as they are. Click "OK" to start the offline evaluation as shown in Figure 7-17.

The screenshot shows the 'Create an evaluation' form in the Microsoft Azure portal. The browser address bar shows 'portal.azure.com'. The page title is 'Create an evaluation'. The form includes the following fields and options:

- Name ***: A text input field containing 'simulation-evaluation' with a green checkmark.
- Start date (UTC) ***: A date picker showing '09/28/2021'.
- End date (UTC) ***: A date picker showing '09/28/2021'.
- Additional evaluation options**:
 - Optimization discovery**: A section with the text 'Should Personalizer discover new learning settings that show high effectiveness during the evaluated period?' and two radio buttons, 'Yes' (selected) and 'No'.
 - Compare additional learning settings (optional)**: A section with the text 'Compare the performance of your model with the performance of additional learning settings.' and a button 'Add learning settings'.
 - Learning settings**: A section with a button 'Ok'.

Figure 8-18. Offline evaluation Form

The evaluation process takes a couple of minutes. Once it is completed, select the evaluation from the list to see further details. Click the link “Compare the score of your application with other potential learning settings” to see the evaluation metrics as shown in Figure 7-18.

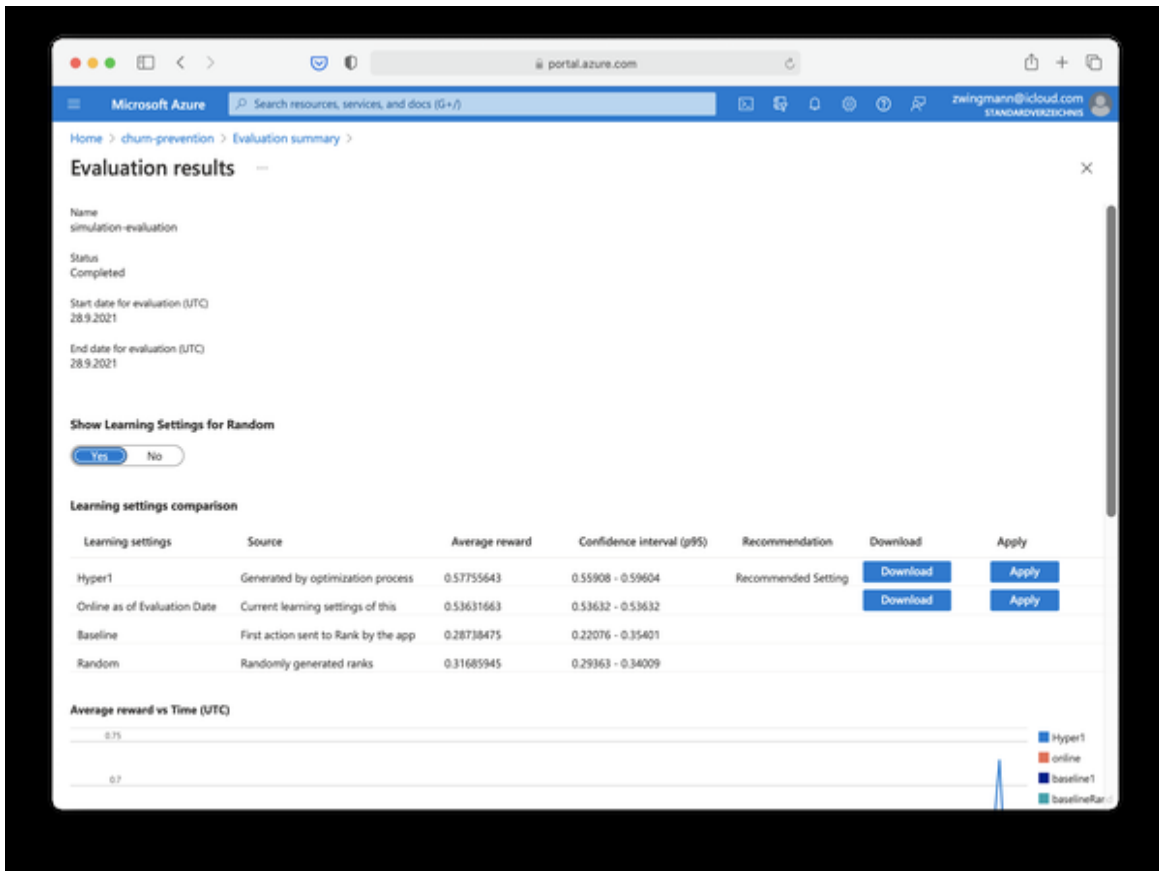


Figure 8-19. Offline evaluation results

As you can see, from the offline evaluation we learned that we can improve our model from 0.5363 average reward (“Online as of Evaluation”) to an 0.5775 average reward if we apply the learning setting “Hyper1”. This is closer to the ideal scenario of 0.8 considering that we leave 20% out for exploration. Choose the new learning settings by clicking the “Apply” button next to it. This will retrain your model and deploy it with the new learning settings.

You could now go back to your code and run the simulation again to see how the model improves. But before we do that we want to incorporate our AI recommendations into our BI dashboard. Let’s explore this in the last step of this use case.

Access Results from Power BI

As you might remember, in our use case scenario we are already provided with a BI dashboard that shows the current customer churn prediction as well as the overall acceptance rate of the standard offer, indicating the success of our customer retention strategy. What we want to do now is to blend this information with insights on how our Personalization service is performing and see if our initiative leads to more customers accepting offers.

To start, open the Power BI workbook “Use-Case-7.pbix” from the book repository and head over to the data model. You should see the existing table structure here for the churn predictions. Let’s add the results from our model by importing the result csv file which we created during the simulation. If you decided earlier to save the csv locally, then click “Get data” → import Text / CSV and find the csv file on your local computer.

If you decide to save the file on the Azure Blob storage, click “Get data”, choose “Azure” and select “Azure blob storage”. Provide the name of your storage account which you provided earlier and in the next step, paste your access key which you got from the Azure Portal. When you confirm, you should see a list with all containers and files on your Blob storage as shown in Figure 7-19.

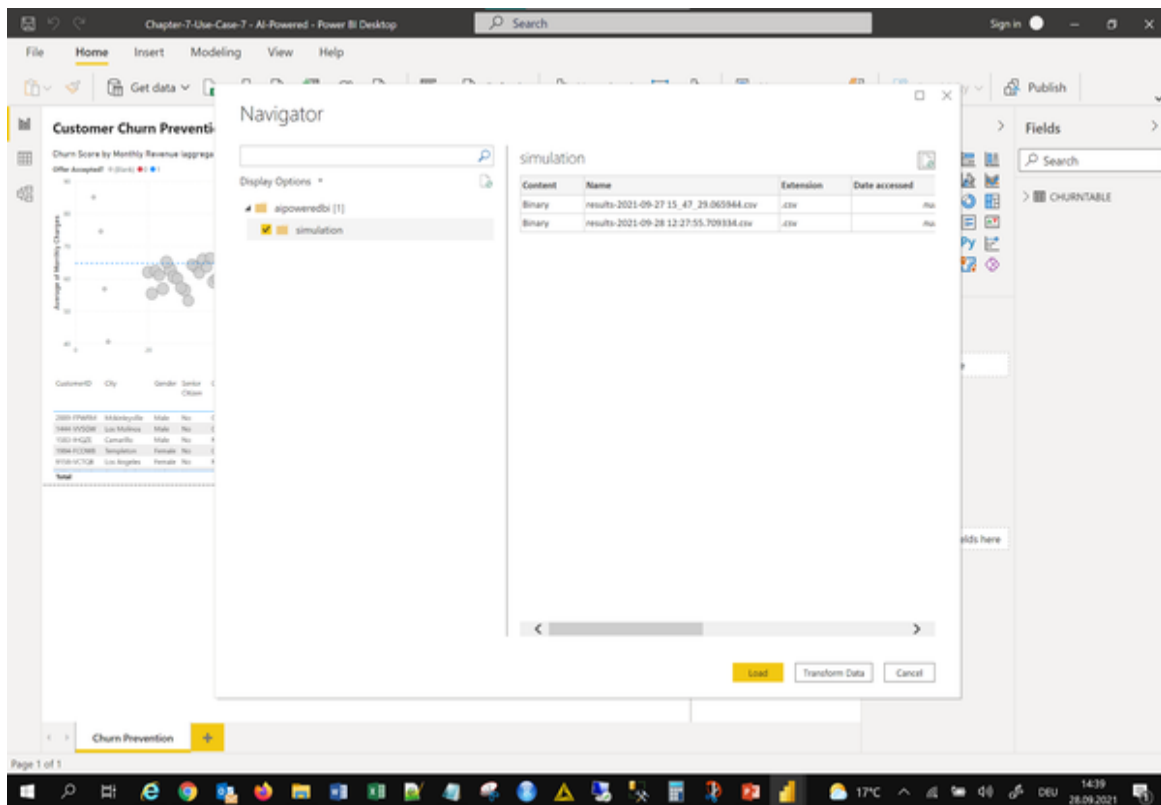


Figure 8-20. Accessing Azure Blob Storage from Power BI

Choose the container with your simulation data. But don't load the files yet! Instead, click the button "Transform data" as shown in Figure 7-19.

This will allow us to use Power Query to extract the actual contents of the CSV file. Otherwise we would only see the CSV metadata such as its filename, creation date, size, etc.

In the Power Query editor, click on the binary data link as shown in Figure 7-20.

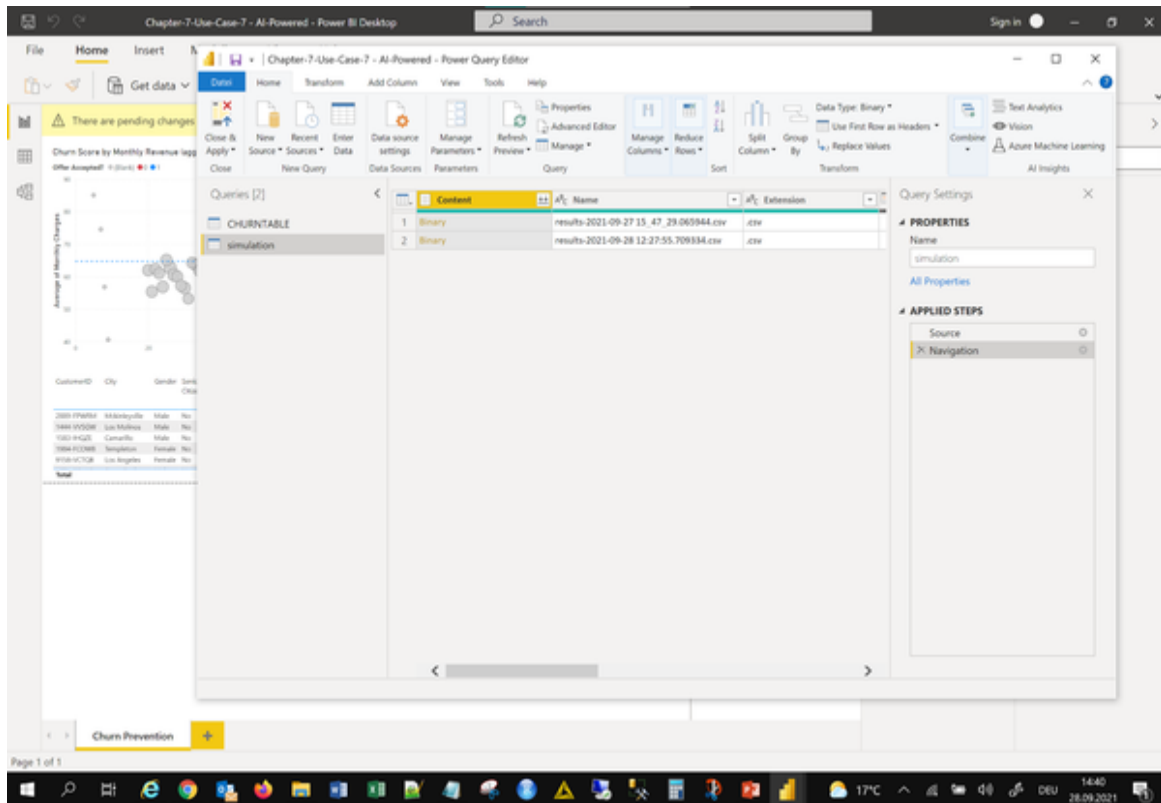


Figure 8-21. Accessing blob files with Power Query Editor

This will bring you to the contents of the CSV file where the conversion from binary to tabular data is shown as a processing step on the right side in the “Applied Steps” pane as seen in Figure 7-21.

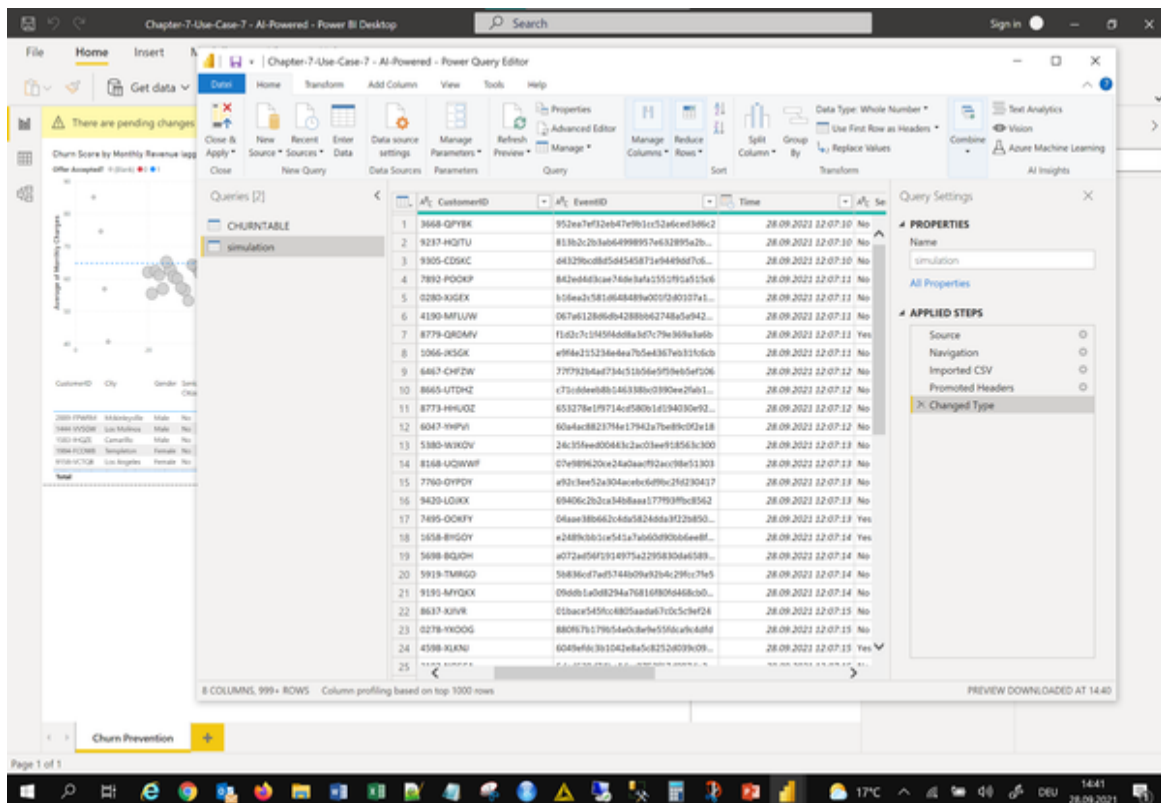


Figure 8-22. Importing table data from blob storage

Confirm the transformation by selecting “Close & Apply” from the top menu. You should see the contents of the new CSV file have been added to your data model and Power BI should have automatically generated the relation between the “Churntable” and the “simulation” data models based the field CustomerID as shown in Figure 7-22.

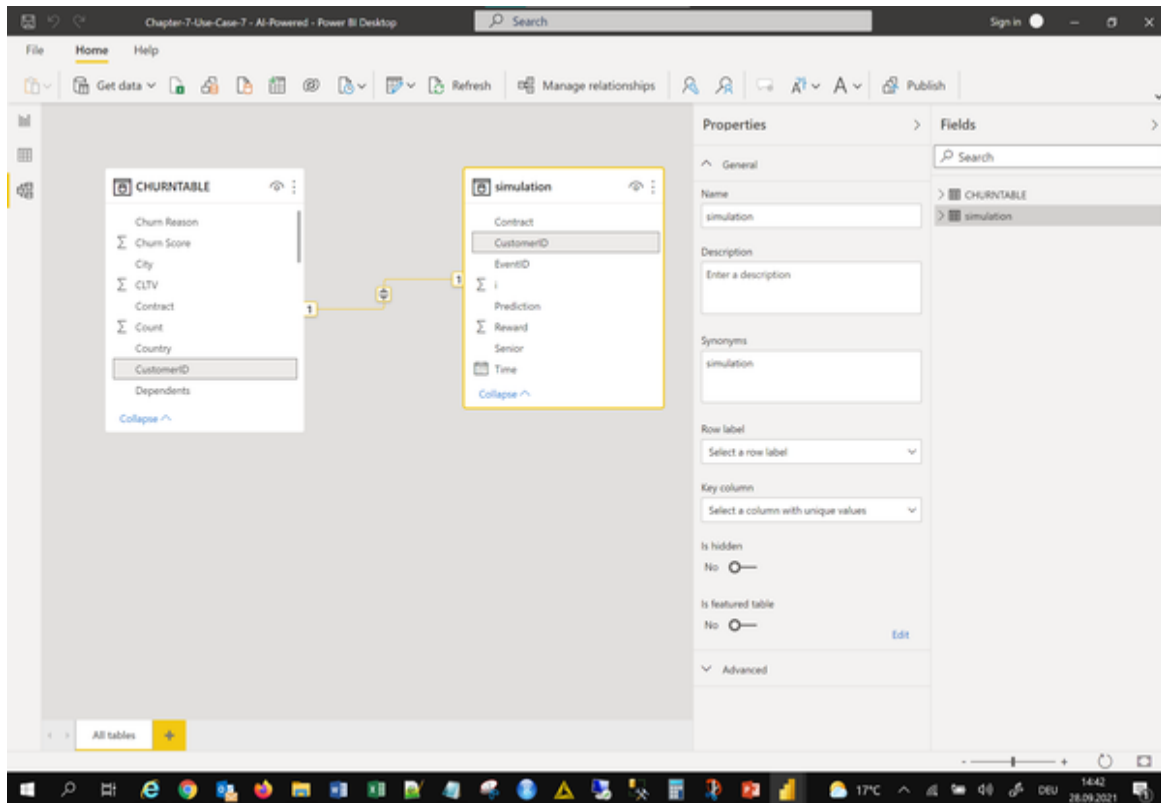


Figure 8-23. Data relation in Power BI.

Head over to the BI report page and let's do some modifications here to include our new AI-based recommendations. Make the following changes to the report:

Update the scatterplot visual :

- Legend field: Replace “Offer accepted?” with “Reward” from the simulation data model
- Change color scheme under formatting and data colors: Set “Blank” to grey, “0” to red and “1” to blue

Update the gauge visual :

- Replace „Average Offer accepted“ with „Reward“ from the Simulation table. Apply the calculation „Average“ to “Reward”.

Update the metric visual for the defended revenue :

First, the gauge has increased from 0.31 to 0.54 which is an increase of 23 percentage points in accuracy compared to our existing baseline. This means that by using the AI-based approach we could convince more customers to accept our retention offers. The defended monthly revenue

consequently increased as well to 154.38K which is a plus of almost 60k dollars.

And second, we can see that the proportions of offers increased. The former most popular Offer 1 (Cash offer) is now only very rarely happening. Most customers accepted Offer 3 - the higher performance

Can we improve these KPIs even more? Remember that we tweaked our model through an offline evaluation previously. But so far, we haven't applied the new model yet.

Go back to the simulation code and re-run it once again without making any changes. After another 15 minutes the code will have gone through the dataset once so the Personalizer service could use the updated learning settings. The results.csv file in your Azure blob storage should have been automatically updated. Therefore, all we need to do in Power BI is just to go to the data model and hit "Refresh data" for the Simulation table as shown in Figure 7-24.

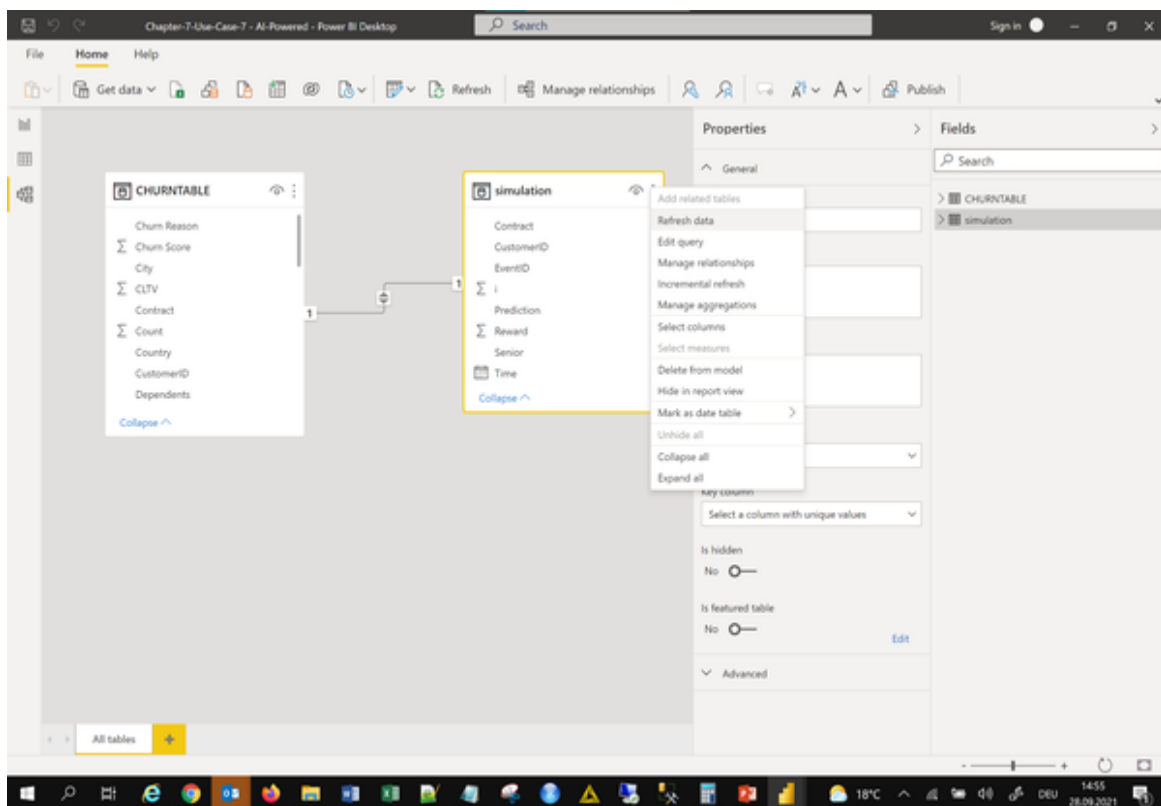


Figure 8-25. Refresh data

Customer Churn Prevention - Success Dashboard

Churn Score by Monthly Revenue (aggregated)

Reward: (Blank) ● ● ● 1

Offers Accepted

0.00 0.59 1.00

Monthly Revenue at Risk: 297.47K

Revenue Defended: 172.06K

Offer 3

CustomerID	City	Gender	Senior Citizen	Contract	Internet Service	Online Backup	Online Security	Streaming Movies	Phone Service	Tech Support	Streaming TV	Monthly Charges	Churn Score	Revenue Risk	Average of Offer Accepted?
2889-FPWRM	McKinleyville	Male	No	One year	Fiber optic	Yes	Yes	Yes	Yes	Yes	Yes	117.80	100	117.80	0.00
1444-VVSGW	Los Molinos	Male	No	One year	Fiber optic	Yes	Yes	Yes	Yes	Yes	Yes	115.65	99	114.49	0.00
1583-3HQZE	Camarillo	Male	No	Month-to-month	Fiber optic	Yes	No	Yes	Yes	Yes	Yes	112.95	97	109.56	0.00
1984-FHQBW	Templeton	Female	No	One year	Fiber optic	Yes	No	Yes	Yes	Yes	Yes	109.50	100	109.50	0.00
9158-VCTQB	Los Angeles	Female	No	Month-to-month	Fiber optic	Yes	Yes	Yes	Yes	Yes	Yes	113.60	96	109.06	0.00
Total														267.737,79	0,31

Churn Prevention

through the dataset once more with the updated model. This time, we improved our recommendations by another 5 percentage points. The accuracy of 59% now. The defended revenue has

For a real-world scenario, imagine that you are not iterating over the same dataset over and over again but over new customer data for each quarter, month or day, depending on your business. This way you can monitor how your recommendation model is performing and you can decide whether it is good enough to be released into production or not. At least the dashboard will give you a solid guideline to assess business value of the AI-backed recommendation service in contrast to your existing baseline.

Cleaning up resources

If you used Azure Notebooks with an Azure compute engine for this chapter, make sure to delete the compute resource after the exercise to prevent it from charging the bill. Also, consider removing the CSV files from the blob storage if you don't need them any more.

Summary

I hope you could see from this use case how we can utilize AI to turn predictions into actions and how to find out which actions fit in a given context. While Azure Personalizer is only one of many services, the ideas and principles behind these reinforcement learning-based recommendation engines are very similar and you should find it easy to pick up other services as well if needed. Also, with Azure Blob storage, you learned a way to monitor model updates in realtime and bring them into your BI without too much friction.

In the next chapter, we will tackle unstructured data and see how we can leverage AI on images, documents and audio files to analyse their contents in our regular BI workflow and blend with existing information from the business.

Chapter 9. Leveraging Unstructured Data with AI

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 9th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

In the previous chapters we used AI quite a lot with structured data, or how most people call it: tables. However, a lot of data in businesses actually is not stored in clean tables, but comes in a plethora of different formats such as pdfs, images, raw text, websites or emails. With AI we can unlock these treasure troves and get insights from data that has hardly been touched before by analysts or data which otherwise needed a lot of manual effort before anyone could get insights from it. In this chapter we are exploring how AI will help us to analyse texts, documents and image files.

Use Case: Getting Insights From Text Data

Not only the internet is full of text. Written language is one of the biggest and most diverse data sources humanity has collected. And businesses are no exception. The biggest creators of data here are people, either within or outside of the organisations. Customers for example - thanks to the internet

- become content producers and share their opinions about products or services across the web and on various channels.

In this use case, we are going to deploy an AI service that is going to help us to make sense of this data. In the concrete problem at hand, we are going to analyse user reviews at scale and communicate key insights through a BI dashboard. Let's go!

Problem Statement

Small rooms, unfriendly staff and horrible breakfast - or not? As the operator of a large hotel, the management is overstrained by the variety of customer feedback. Are there really problems the management needs to address or is just the regular grouse you have to accept as someone running an accommodation business? The head of operations has hired us as an external analyst to find out how customers think about the hotel and how this trend has developed over time. As a data source they provide a sample of text files of customer feedback which they have collected over time through their website and gathered from booking portals. Since the new season is just about to start, they want the results better today than tomorrow, so speed is clearly prioritized over accuracy. The management wants to know if there is something fundamentally going wrong and to have the possibility to dive in deeper if necessary.

Solution Overview

In order to analyse many text files automatically, we will make use of a Natural Language Processing (NLP) AI service. Our goal is to extract information about whether the opinions in these texts are negative or positive, which is also called sentiment analysis. Furthermore, we want to extract keywords so we can relate back to word phrases that carry a positive or negative emotion. The presentation should be in the form of a BI dashboard, in our case Power BI.

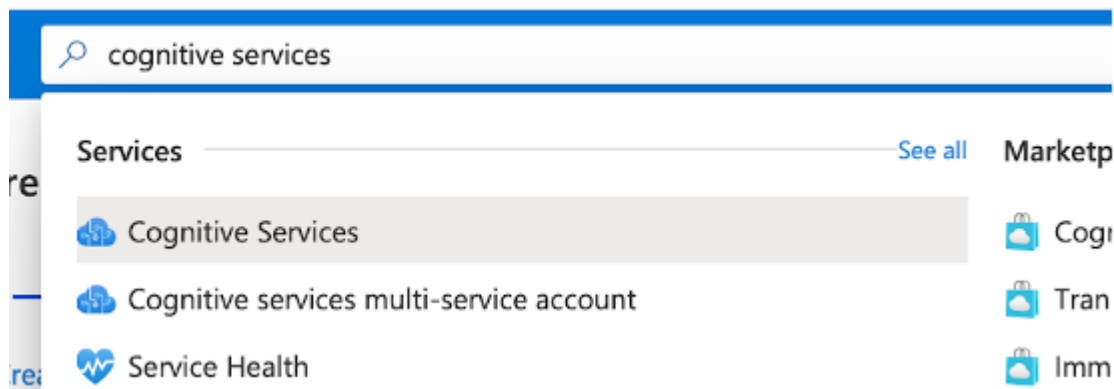
To tackle this task, we are first going to set up an AI service on Azure called Cognitive Services Text Analytics. We will then set up a small data

processing pipeline which takes our raw text files as inputs and creates flat csv files as output that contain all the results needed to conduct our analysis in Power BI (or any other BI tool of your choice).

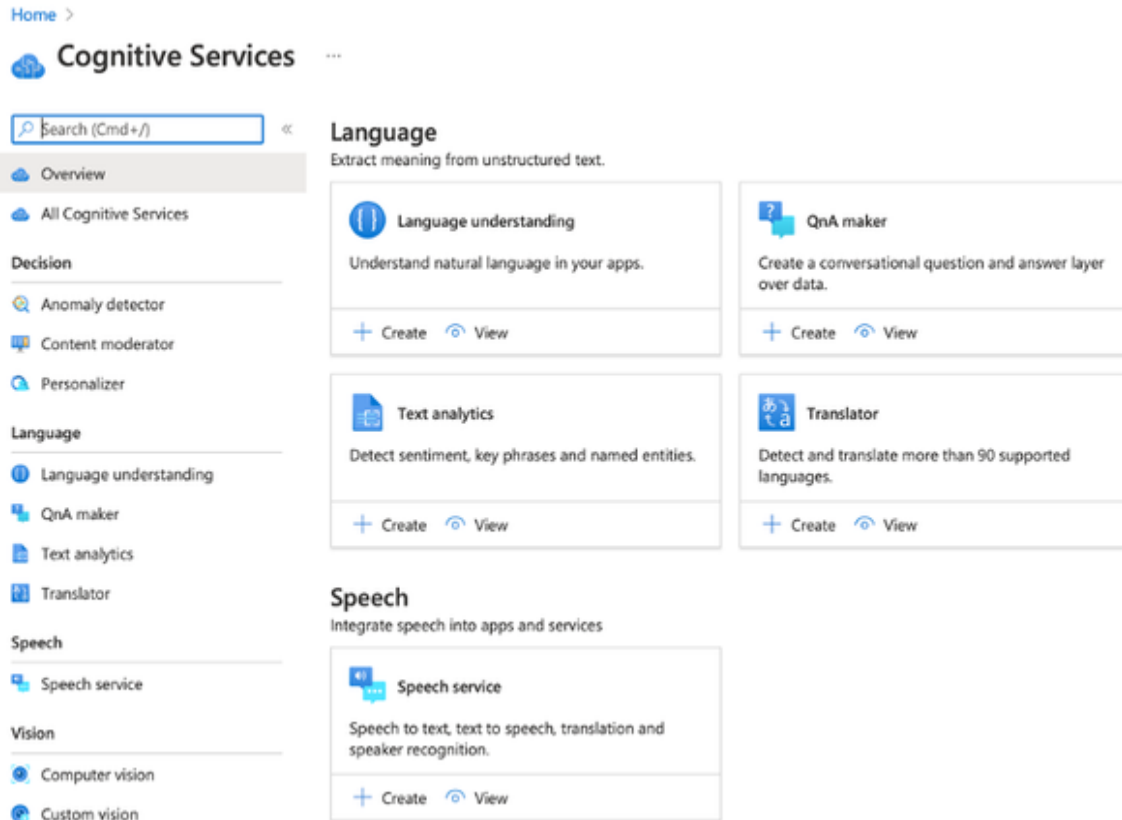
Setting up the AI Service

First of all, we need to activate the Cognitive Services Text Analytics in our Azure subscription. This point is very straightforward and very similar to what we did previously for the other AI services.

To activate Cognitive Services Text Analytics go to your [Azure Portal](#) and search for Cognitive Services in the search bar as shown in Figure XX.



Select Cognitive Service from the suggestions list and head over to the corresponding resource page. At this page you can enable Cognitive Services for all kinds of data and use cases. Scroll down until you see the “Language” section as shown in Figure XX.



From here you can deploy a new Text analytics resource by clicking the “+ Create” button. In the following form, which you should be quite used to by now, select your Azure subscription, the resource group and give your Text analytics service a name. Also, make sure that you choose the Free Tier (F0) which allows 5,000 transactions per 30 days. One transaction is a text record which corresponds to the number of 1,000-character units within a document that is provided as input to a Text Analytics API - that’s more than enough for our use case here. You can see an example of the filled form in Figure XX.

Create ...

Text Analytics

*** Basics** Tags Review + create

Unlock insights from unstructured text using advanced natural language processing. Use sentiment analysis to find out what customers think of your brand. Find topic-relevant phrases using key phrase extraction and identify the language of the text with language detection. Detect and categorize entities in your text with named entity recognition. [Learn more](#) 

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *		Azure subscription 1	▼
Resource group *		ai-powered-bi	▼

[Create new](#)

Instance details

Region *	(US) East US	▼
Name *		ai-powered-bi-text-analytics 
Pricing tier (Learn More) *		Free F0 (5K Transactions per 30 days) ▼

Responsible AI Notice

Microsoft provides technical documentation regarding the appropriate operation applicable to this Cognitive Service that is made available by Microsoft. Customer acknowledges and agrees that they have reviewed this documentation and will use this service in accordance with it.

[Responsible Use of AI documentation for Text Analytics for Health](#)

[Responsible Use of AI documentation for Text Analytics PII](#)

☒ I certify that I have reviewed and acknowledge the terms in the Responsible AI Notice. *

[Review + create](#)

[Next : Tags >](#)

Select “Review + create” and then “Create” after the automatic review process has passed. The deployment will take some minutes, but after that you should see a notification prompting that the new resource is ready. Navigate to the new resource either by clicking on the notification, or - if you left the page - simply by searching for the resource name you provided in the form in the Azure search bar.

In both cases, you should be greeted with the resources’ “Quickstart” guide as shown in Figure XX.

Home > Microsoft.CognitiveServicesTextAnalyticsWithConfigurations > ai-powered-bi-text-analytics

ai-powered-bi-text-analytics | Quick start

Text analytics

Search (Cmd+F)

- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems

Resource Management

- Quick start**
- Keys and Endpoint
- Pricing tier
- Networking
- Identity
- Billing By Subscription
- Properties
- Locks

Monitoring

- Alerts
- Metrics
- Diagnostic settings
- Logs

Automation

Explore the Quickstart guidance to get up and running with Text Analytics.

- 1** Get the API Key to authenticate your applications and start sending calls to the service.
All Text Analytics calls and docker container activations require a key. The key can be found in the Keys and Endpoint section in the left pane. Specify the key either in the request header (Web API), the Text Analytics client (SDK), or through the command-line (Docker container).
[API Key](#)
- 2** Try the service in the API console - requires API Key and selecting your location.
Use the API Console to quickly try the API without writing code. Be sure to select the correct location for this resource. The API key and the location can be found in the Keys and Endpoint section in the left tab.
[API Console \(V3\)](#)
[API Console \(3.1 Preview\)](#)
- 3** Make a web API call - requires API Key and location.
Use the sample code in these quickstarts to begin integrating Text Analytics into your applications to analyze sentiment, extract key phrases and entities, and detect languages. The API key and the location can be found in the Keys and Endpoint section in the left tab.
[C# Quickstart](#)
[C# sample solution \(zip\)](#)
[Java](#)
[Python Quickstart](#)
[NodeJS Quickstart](#)
[What's New](#)

There are many things we can do here, but for now we just want to find out two things:

First, how do we authenticate against the API and second, where do we find out how the API works, i.e. which code we need to write (or better: copy and paste)?

To answer the first part of this question, you can click on the “Keys and Endpoints” menu in the left menu pane. Here you will find two keys just as you did in the Azure Personalize example in the previous chapter. You will only need one of these to call the API. Leave this window open in a new tab since we will soon need these resource keys.

To find out how the API works you can explore the resources in “Step 3” of the quickstart guide. If you click on the “Python Quickstart” you will find a document that explains various scenarios where you could use the Text Analytics API and which code you would use for these. This might seem a lot for now, but don’t get intimidated by it. You won’t need all of this

information. For now, it is enough to acknowledge that the majority of the code we are writing later in the data preparation step can actually be taken from this documentation page and adapted to your own needs.

Looking through the documentation we can actually discover some more useful information regarding our new AI service. Navigate to “Concepts” and “data limits” and you should find a table as shown in Figure XX.

The maximum number of documents you can send in a single request will depend on the API version and feature you're using, which is described in the table below.

Version 3

Version 2

The following limits are for the current v3 API. Exceeding the limits below will generate an HTTP 400 error code.

Feature	Max Documents Per Request
Language Detection	1000
Sentiment Analysis	10
Opinion Mining	10
Key Phrase Extraction	10
Named Entity Recognition	5
Entity Linking	5
Text Analytics for health	10 for the web-based API, 1000 for the container.
Analyze endpoint	25 for all operations.

If we look at this table and locate the row “Sentiment Analysis” we can actually see that the API accepts 10 documents per request. It means that we can send up to 10 text files at once to the API which will greatly speed up our inference process, compared to sending the text files to the API one by one. We will come back to this during the data processing.

Now, everything has been set up for the AI service. Let’s head over to build our mini data pipeline and run some inference on the customer feedback

data.

Setting up the data pipeline

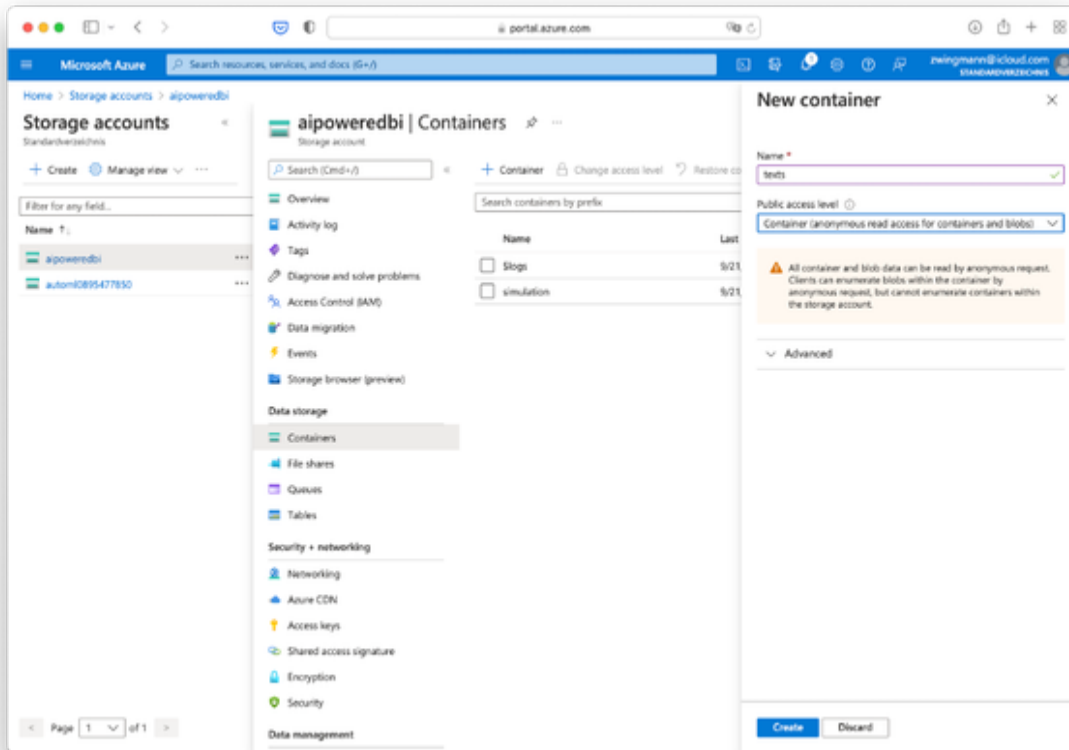
In order to get from raw text files to a beautifully designed BI dashboard we have to solve some intermediate steps. In a productive environment this would be called our ETL or Extract-Transform-Load process. For our prototype we won't go as far as naming this a fully fledged ETL process, but in its essence we are doing just that. Let's call it our mini ETL job.

We will write a short script that will handle the essential parts of an ETL process, that is:

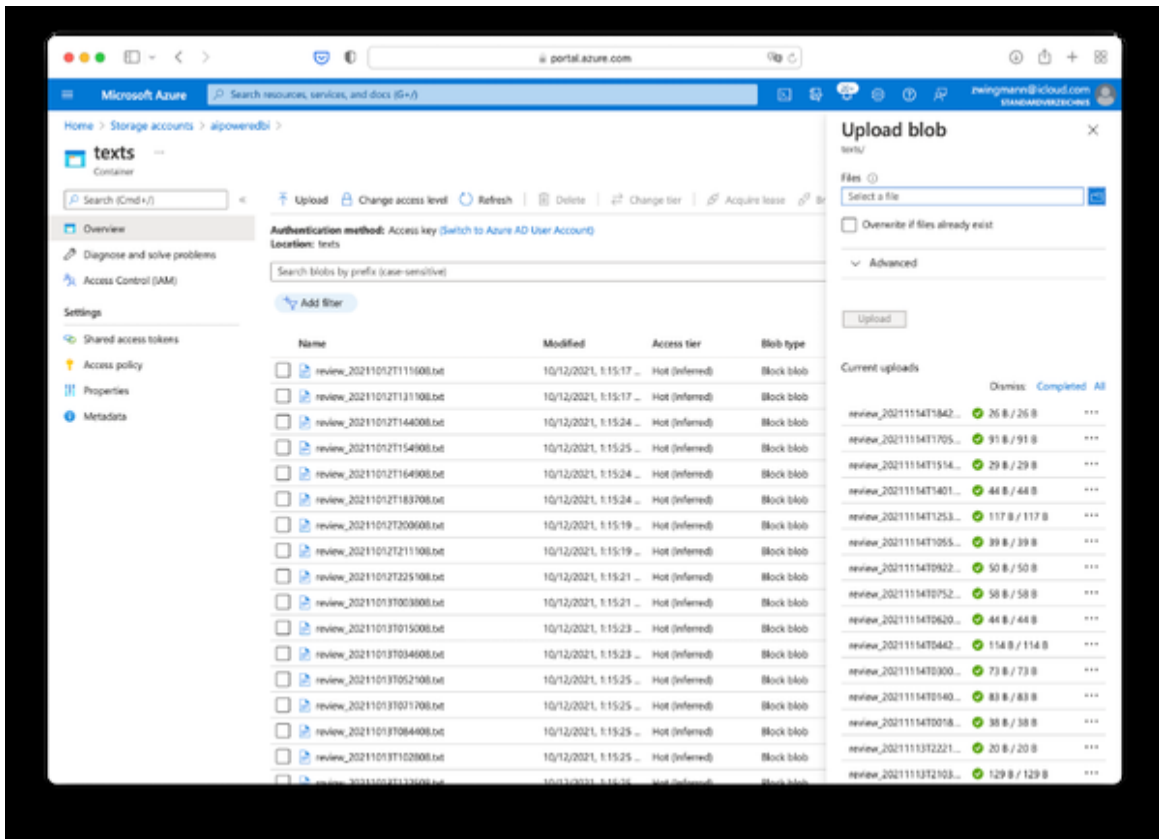
- Step 1: Read plain text files from file.
- Step 2: Send the file contents to the AI service API.
- Step 3: Collect the results, transform and store them in a structured data object.
- Step 4: Export the files as flat tables so they can be easily consumed by our BI software.

For step 1 we need to get the text files and store them at some place where our script can access them. We will use Azure Blob storage for this.

Navigate to the book repository and download the file “reviews.zip”. In your Azure portal, navigate to storage, select the storage account you have set up previously and create two new containers: “texts” and “tables” as shown in Figure XX. The container “texts” will hold the raw input text files and the container “tables” will receive the tabular output of our mini ETL process.



After you created the “texts” container, upload all the individual text files here. Don’t upload the zip file, but its contents. Your container should now look like Figure XX.



Our input data is now ready to be consumed by a script. You can leave the other container “tables” empty for now. We will populate it through our script, at which we will take a look next.

Download the file “ETL_For_Text_Analytics.ipynb” from the book’s repository. This notebook contains all the code you will need to execute the mini ETL process. If you have a local IDE such as Jupyter notebooks installed on your computer, you can go ahead and use this. If not, I suggest using Azure Notebooks as a hosted IDE as it comes as part of your Azure subscription.

I will walk you through the example using Azure Notebooks, but the same steps apply when you use your local IDE instead.

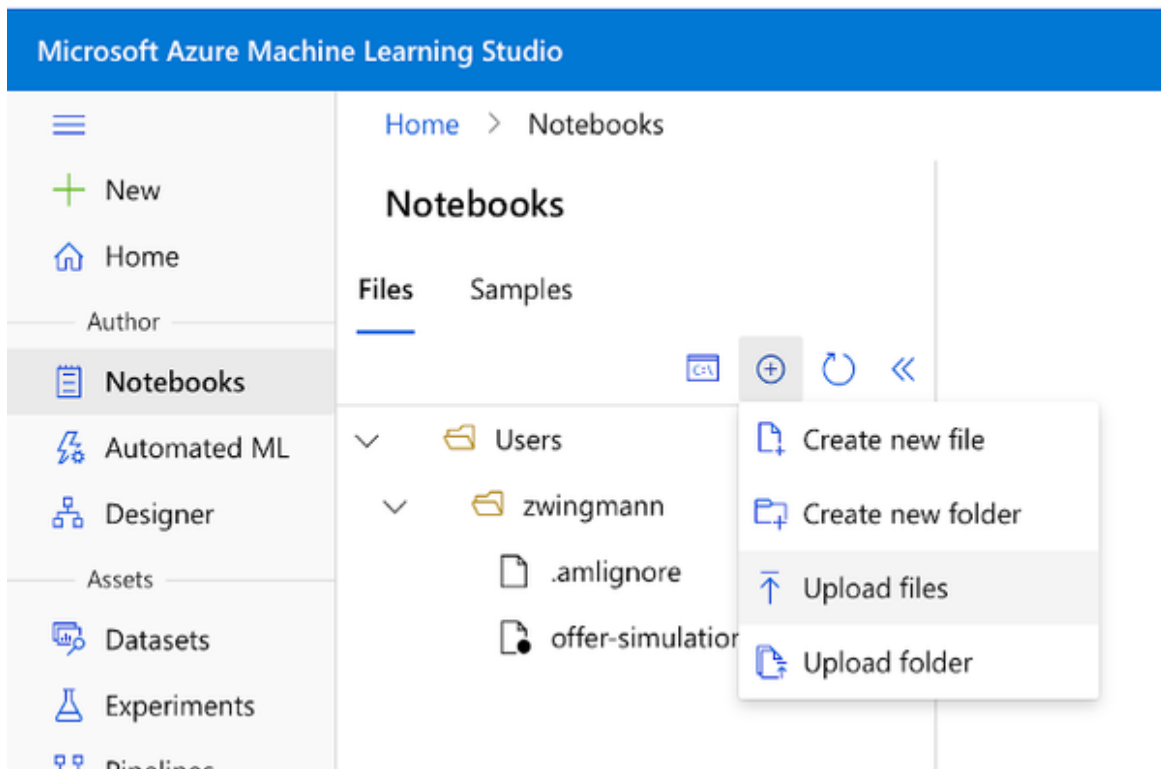
NOTE

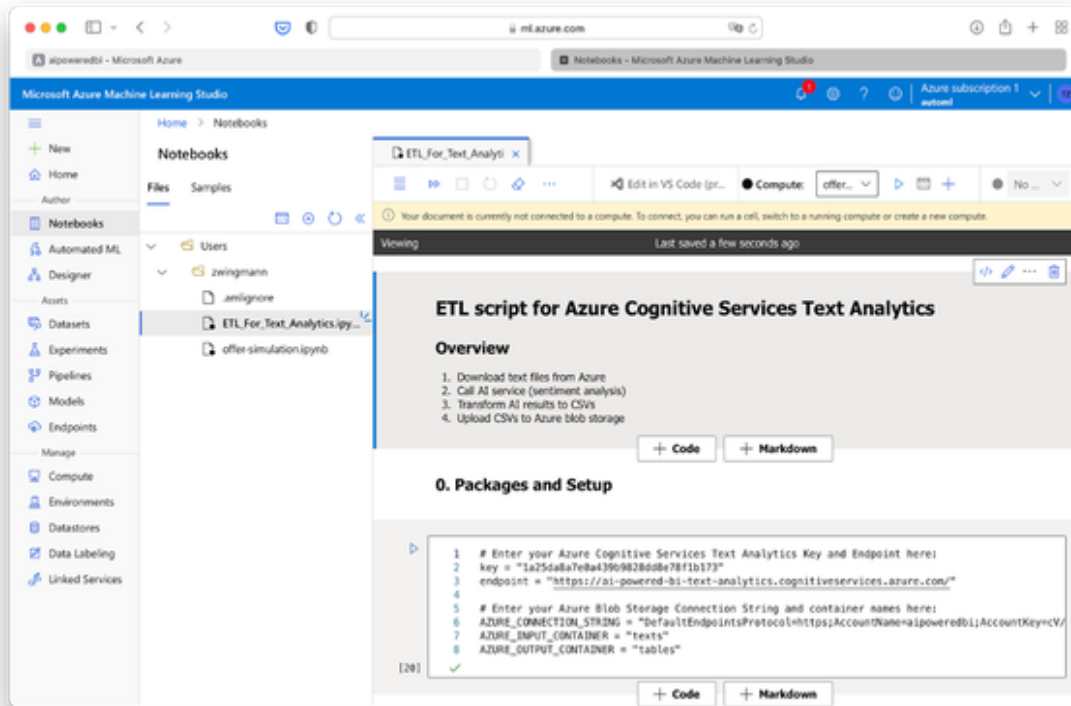
Also provide an R version? If not, this text will apply:

We will make use of some Azure packages needed in this code for Authentication and Blob storage that are currently not supported in R. While it is still possible to execute this workflow in R it would be very heavy. So in this case I will only provide the Python version. All R users, please follow along here.

Let's start by creating a new Azure Notebook:

- Navigate to **Microsoft Azure Machine Learning Studio** and select the workspace you have been using throughout the book.
- Select the “+” Icon and choose “Upload files” as shown in Figure XX.
- Locate the “ETL_For_Text_Analytics.ipynb” on your computer, check the “I trust the contents of this file” and hit “upload”.
- You should see the Notebook as shown in Figure XX.





The notebook will be displayed on the right side. While this file contains quite a lot of code, don't get intimidated by it. In fact, the only thing you need to modify are just the custom parameters for your access credentials and AI service endpoint.

As an intermediate solution between letting you run the whole notebook at once, trusting that it will somehow work out, and discussing the code line by line, I will give you a high-level overview of what this code is doing by explaining the different code sections. I recommend following along and just executing the code cells as we discuss the sections here in the book.

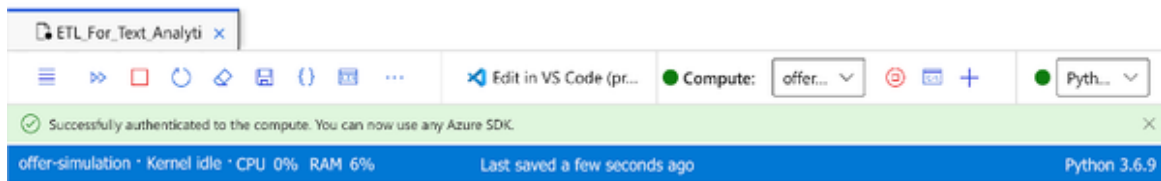
There are four main parts to this code which roughly align to the four steps I mentioned previously during the ETL process:

1. **Download text files from Azure:** This is where we will download the raw text files from Azure to the local machine that is running the notebook.

2. **Call AI service (sentiment analysis):** This is where we send the text files to the AI services and save the results.
3. **Transform AI results to CSVs:** In this section we will transform the AI results from JSON outputs to flat CSV files so it can be consumed easily by our BI.
4. **Upload CSVs to Azure blob storage:** Finally, we will upload the files back to Azure Blob storage that is connected to our BI.

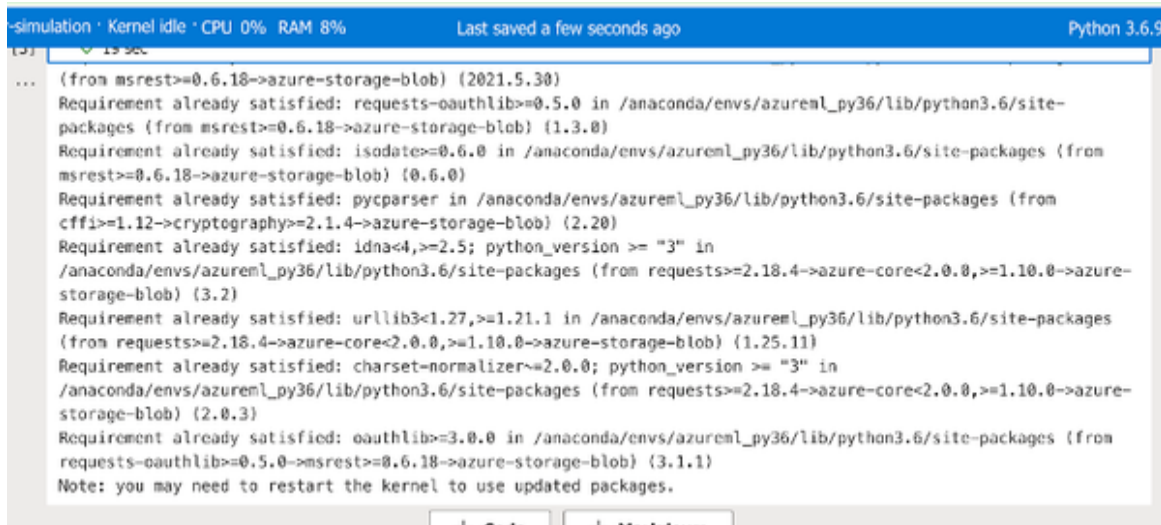
Before we start, make sure that the Notebook is connected to a compute resource that is running. If you have not created a compute resource yet, check back the section “The AI Prototyping Toolkit” in Chapter 3.

If you are asked to authenticate again after the compute resource has started, just press the button that shows up and the authentication should be done automatically for you as shown in Figure XX.



Now, head over to section “0” of the code. This is the place where we will make sure that all packages that we need are installed and loaded and - more importantly - your personal Azure credentials are provided. This is actually the only place in the whole notebook where you need to change something. Replace the dummy strings for the key, endpoint and the Azure Connection String with your custom values from the Azure portal of the Text Analytics Cognitive Service and the Storage account respectively. Once you’ve updated your personal credentials, run the first code cell. If you still feel unsure about how this Notebook works, check back on Chapter 3 “The Prototyping Toolkit”. The output of this code should be relatively simple. All packages should have been installed on the Azure compute resource already and you should simply see the “Requirements already satisfied”

notifications as shown in Figure XX.



```
... (from msrest>=0.6.18->azure-storage-blob) (2021.5.30)
Requirement already satisfied: requests-oauthlib>=0.5.0 in /anaconda/envs/azureml_py36/lib/python3.6/site-
packages (from msrest>=0.6.18->azure-storage-blob) (1.3.0)
Requirement already satisfied: isodate>=0.6.0 in /anaconda/envs/azureml_py36/lib/python3.6/site-packages (from
msrest>=0.6.18->azure-storage-blob) (0.6.0)
Requirement already satisfied: pycparser in /anaconda/envs/azureml_py36/lib/python3.6/site-packages (from
cffi>=1.12->cryptography>=2.1.4->azure-storage-blob) (2.20)
Requirement already satisfied: idna<4,>=2.5; python_version >= "3" in
/anaconda/envs/azureml_py36/lib/python3.6/site-packages (from requests>=2.18.4->azure-core<2.0.0,>=1.10.0->azure-
storage-blob) (3.2)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /anaconda/envs/azureml_py36/lib/python3.6/site-packages
(from requests>=2.18.4->azure-core<2.0.0,>=1.10.0->azure-storage-blob) (1.25.11)
Requirement already satisfied: charset-normalizer<=2.0.0; python_version >= "3" in
/anaconda/envs/azureml_py36/lib/python3.6/site-packages (from requests>=2.18.4->azure-core<2.0.0,>=1.10.0->azure-
storage-blob) (2.0.3)
Requirement already satisfied: oauthlib>=3.0.0 in /anaconda/envs/azureml_py36/lib/python3.6/site-packages (from
requests-oauthlib>=0.5.0->msrest>=0.6.18->azure-storage-blob) (3.1.1)
Note: you may need to restart the kernel to use updated packages.
```

NOTE

While we store our Azure credentials in some plain text here within the code, please be aware that this is generally no good coding practice. We are doing this to keep things simple and also because the notebook is still hosted in a protected Azure environment. If you want to learn best practices around handling authentication mechanisms I do recommend checking out this Microsoft resource: [Azure Key Vault Overview - Azure Key Vault](#)

Let's move on to the first actual part of our mini ETL process, the file download.

This first code cell of step 1 is calling a class object which we defined in the setup part above. It is essentially establishing a connection to your Azure storage account using your connection string and downloading all files from the AZURE_INPUT_CONTAINER which you provided. Run this cell and it should be finished after a minute printing some outputs as shown in Figure XX.

offer-simulation · Kernel idle · CPU 0% RAM 8% Last saved a few seconds ago Python 3.6.9

1. Download text files from Azure

1 azure_blob_downloader = AzureBlobDownloader()
2 azure_blob_downloader.download_all_blobs_in_container()

[6] ✓ 1 min 24 sec

... review_20211104T055808.txt
review_20211104T075408.txt
review_20211104T091808.txt
review_20211104T102408.txt
review_20211104T115808.txt
review_20211104T131508.txt
review_20211104T143108.txt
review_20211104T161108.txt
review_20211104T174308.txt
review_20211104T192408.txt
review_20211104T211208.txt
review_20211104T224408.txt
review_20211105T001108.txt
review_20211105T015508.txt
review_20211105T025908.txt
review_20211105T045008.txt
review_20211105T060008.txt
review_20211105T071808.txt

NOTE

If you wonder why this download takes so long, rest assured that there are methods to speed up the downloading process, for example running multiple downloads at once. In fact, you could also pass over the file URLs of the AI services directly without downloading them at all. For our prototype the slow download method is still OK. But if you move to production, you would consider faster alternatives.

Next, let's move on to section 3 of our code - "Calling the AI service". In this section we will actually send the texts to the AI service for analysis. As mentioned previously, we are sending batches of 10 text files (documents) at once to the API so that we get the results faster. Run the code cell and once it is finished, you should see the first result containing the first 10 results of the sentiment analysis as shown in Figure XX.

```
offer-simulation · Kernel idle · CPU 0% RAM 8% Last saved a few seconds ago Python 3

21 # Call the AI service
22 results = []
23 for document in documents:
24     results.append(sentiment_analysis_with_opinion_mining(document, client))
25
26 # Print the first result
27 results[0]

[8] ✓ 36 sec

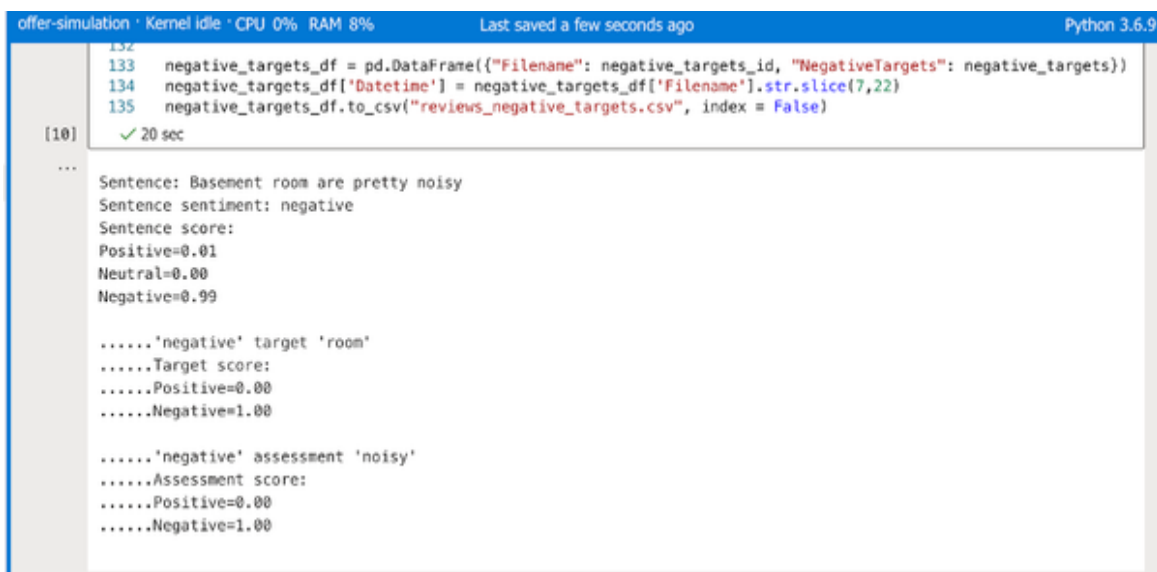
... [AnalyzeSentimentResult(id=review_20211012T111608.txt, sentiment=negative, warnings=[], statistics=None,
confidence_scores=SentimentConfidenceScores(positive=0.42, neutral=0.0, negative=0.58), sentences=
[SentenceSentiment(text=A bit more noise insulation would have been good, sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.42, neutral=0.0, negative=0.58), length=48, offset=0,
mined_opinions=[]), is_error=False),
AnalyzeSentimentResult(id=review_20211012T131108.txt, sentiment=negative, warnings=[], statistics=None,
confidence_scores=SentimentConfidenceScores(positive=0.03, neutral=0.0, negative=0.97), sentences=
[SentenceSentiment(text=A bit noisy at night., sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.01, neutral=0.0, negative=0.99), length=21, offset=0,
mined_opinions=[]), SentenceSentiment(text=The shower did not work very well, sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.05, neutral=0.0, negative=0.95), length=33, offset=22,
mined_opinions=[MinedOpinion(target=TargetSentiment(text=shower, sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.02, neutral=0.0, negative=0.98), length=6, offset=26),
assessments=[AssessmentSentiment(text=work, sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.02, neutral=0.0, negative=0.98), length=4, offset=41,
is_negated=True)]), is_error=False),
AnalyzeSentimentResult(id=review_20211012T144008.txt, sentiment=negative, warnings=[], statistics=None,
confidence_scores=SentimentConfidenceScores(positive=0.01, neutral=0.0, negative=0.99), sentences=
[SentenceSentiment(text=A bit pricey., sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.01, neutral=0.0, negative=0.99), length=13, offset=0,
mined_opinions=[]), SentenceSentiment(text=The corridors to our room on the third floor needed a refresh.,
sentiment=neutral, confidence_scores=SentimentConfidenceScores(positive=0.01, neutral=0.99, negative=0.0),
length=62, offset=14, mined_opinions=[MinedOpinion(target=TargetSentiment(text=corridors, sentiment=negative,
confidence_scores=SentimentConfidenceScores(positive=0.1, neutral=0.0, negative=0.9), length=9, offset=18),
```

As you might notice from this preview output, the structure of the results is very nested. The AI service does not only provide us with the sentiment score for each text, but also with a detailed breakdown for opinions on a word level for each text that was analysed. This will be very useful for the interpretation of the data, but on the downside it creates some hassle for us to untangle the whole object and convert it back to some nice flat tables. This is what section 3 of this code is all about. Let's move on!

Section 3 of the data preparation code is actually the most exhaustive one. While the main AI workload has been done by now, there still needs to be a bit of post processing to extract the sentiment scores for each text and the corresponding opinions discovered in each item.

Now, if you ask yourself: "How on earth should I ever come up with this code?", then I have good news for you: Most of this code was actually borrowed from the AI services' documentation page. Do you remember the quickstart reference back in the Azure console? This is the place where most of this code is coming from. Once you understand the general behavior, it is then pretty straightforward to make just the adjustments you need.

In our case, I made three adjustments: First, I extracted only the high-level sentiment scores per text item and saved them as a flat table for better CSV compatibility. This table, or dataframe, also contains the original filename, the original text, and a datetime column which I extracted from the texts' filenames. This will result in a nice flat CSV output which we can then later display in our BI. And second and thirdly, I created a flat CSV each for all positive and negative terms or opinions found in the text documents, again stored with the reference to the original filename and the extracted datetime column so these terms can be linked back to their original context and also possibly filtered by time. If you feel lucky, go through the code line by line and see what's happening. If not, then that's also fine. We're not wanting to become Software Engineers. In any case, hit the run button for this cell to see the output as shown in Figure XX.



The screenshot shows a Jupyter Notebook interface. At the top, a status bar indicates 'offer-simulation · Kernel idle · CPU 0% RAM 8%' and 'Last saved a few seconds ago' on the left, and 'Python 3.6.9' on the right. The notebook cell contains three lines of Python code:

```
132  
133 negative_targets_df = pd.DataFrame({"Filename": negative_targets_id, "NegativeTargets": negative_targets})  
134 negative_targets_df['Datetime'] = negative_targets_df['Filename'].str.slice(7,22)  
135 negative_targets_df.to_csv("reviews_negative_targets.csv", index = False)
```

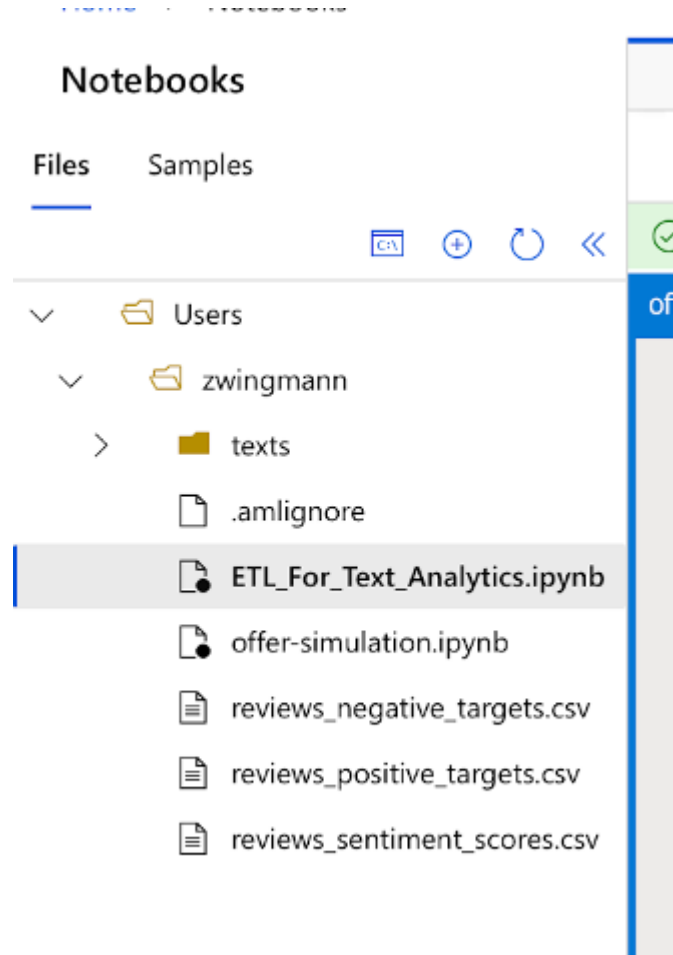
Below the code, the output is displayed. It starts with a prompt '[10]' and a green checkmark indicating successful execution in '20 sec'. The output consists of several lines of text:

```
...  
Sentence: Basement room are pretty noisy  
Sentence sentiment: negative  
Sentence score:  
Positive=0.01  
Neutral=0.00  
Negative=0.99  
  
.....'negative' target 'room'  
.....Target score:  
.....Positive=0.00  
.....Negative=1.00  
  
.....'negative' assessment 'noisy'  
.....Assessment score:  
.....Positive=0.00  
.....Negative=1.00
```

For convenience reasons the output printed the sentiment and opinions for each text document in an easy-to-read format. For example, you can see from Figure XX that the sentence “Basement room are pretty noisy” - despite the grammar mistake - was recognized by the AI as a negative sentiment and the AI service was able to identify the words “room” and “noisy” as the main drivers behind this negative opinion. Isn’t that pretty?

At the end, this code also created the CSV files we wanted. If you take a look at the file explorer on the left and hit refresh, you should see three new

CSV files as shown in Figure XX.



To make these csv files accessible to our BI, let's head over to code section 4 which is all about uploading these files to our Azure Blob storage. Compared to section 3 this will be a breeze.

Execute the last code cell and within a blink of a second you should see the output as shown in Figure XX confirming that the files have been uploaded successfully.

offer-simulation · Kernel idle · CPU 0% · RAM 8% · Last saved a minute ago · Python 3.6.9

4. Upload CSVs to Azure Blob Storage

```
1 blob_service_client = BlobServiceClient.from_connection_string(AZURE_CONNECTION_STRING)
2
3 results_files_list = ["reviews_sentiment_scores.csv", "reviews_positive_targets.csv", "reviews_negative_targets.csv"]
4
5 for results_file_name in results_files_list:
6     # Create a blob client using the local file name as the name for the blob
7     blob_client = blob_service_client.get_blob_client(container=AZURE_OUTPUT_CONTAINER, blob=results_file_name)
8
9     print("\nUploading to Azure Storage as blob:\n\t" + results_file_name)
10
11     # Upload the created file
12     with open(results_file_name, "rb") as data:
13         blob_client.upload_blob(data, overwrite = True)
```

[11] ✓ <1 sec

...

Uploading to Azure Storage as blob:
reviews_sentiment_scores.csv

Uploading to Azure Storage as blob:
reviews_positive_targets.csv

Uploading to Azure Storage as blob:
reviews_negative_targets.csv

+ Code + Markdown

Congratulations! You completed the hardest part and survived the mini ETL challenge. Now, as with every ETL process, this process is reproducible. That means, whenever you update the text files in the Blob input container and the files keep the same structure, you can simply rerun this whole workflow (at once) and the CSV files in the output container will be replaced and overwritten with the new results.

But let's leave the realms of ETL for now and move on to the fun part: Visualizing and synthesizing our results in our BI.

NOTE

Don't forget to stop your compute resource after this exercise!

Synthesizing the results into BI

To display our results we will once more come back to Power BI. Create a new report in Power BI and select "Azure Blob Storage" as the data source. Connect with your storage account and set the check box next to the folder

“Tables”. Click “Transform”. In the Power Query editor you will find a list of all CSV tables found in the folder. We need to go through all three of them one by one. Choose the first one and click on the link “Binary” to see a preview of your data as shown in Figure XX. Double check that the Datetime column has been converted correctly to a datetime format. Hit “Close & Apply” and redo these steps for the other two CSV files.

Filename	Text	Sentiment
review_202110157231308.txt	Breakfast was excellent. Dining good. The staff were polite. Bed comfo...	positive
review_202110197193308.txt	Good bread at breakfast and generally good selection	positive
review_202110307060708.txt	Rates for room were bait. Some staff are helpful and friendly, but othe...	mixed
review_20211017230408.txt	Staffs are nice good location	positive
review_202110157024608.txt	Bed was really comfortable. Location was great	positive
review_202110287200808.txt	Nothing not to like	positive
review_20211047030508.txt	Super helpful staff great location	positive
review_202110277192908.txt	Nice quiet area, excellent location, staff very pleasant, room small but ...	positive
review_202110137102808.txt	Appalling room at basement level with high window. Practically like sl...	negative
review_20211097222708.txt	The staff did their best within the systems established for them by thei...	positive
review_202110207000608.txt	Good location	positive
review_202110287064208.txt	Noise freezing room	negative
review_202110127111608.txt	A bit more noise insulation would have been good	negative
review_20211027051408.txt	So many stairs	neutral
review_20211097021708.txt	The room we stayed the first night was very cold because the wind co...	neutral
review_20211087100808.txt	The room was excellent and our wedding anniversary was acknowledg...	positive
review_202110147022508.txt	Basement room and noisy	negative
review_202110237064008.txt	I got two single beds instead of double. Staff was ok but not that frien...	negative
review_202110257063008.txt	Location perfect, staff extremely friendly and supportive	positive
review_20211137004508.txt	Very helpful staff. Location is perfect. Very comfy beds. Renovated bat...	positive
review_202110277164108.txt	Nice location	positive
review_202110317044808.txt	Room very noisy because we slept under the breakfast room. Houseke...	negative
review_20211097165708.txt	The size of the room is very very small, non electric sockets in the roo...	negative
review_202110147195108.txt	Bed super comfy. Couldn't get up in the morning. Best location for a re...	mixed

Your data model should now list three separate tables. We know, however, that these tables are actually related to each other and we want Power BI to know this as well. Right-click on any table and choose “Edit relationship”. In the editor that pops up, create a relationship between the tables “positive_targets” and sentiment based on the column “Filename”. The cardinality should be many to one as shown in Figure XX. Click OK and redo this setup for the table “negative_targets”.

×

Edit relationship

Select tables and columns that are related.

positive_targets

Filename	PositiveTargets	Datetime
review_20211103T230408.txt	Location	03.11.2021 23:04:08
review_20211015T024608.txt	Location	15.10.2021 02:46:08
review_20211104T030508.txt	Location	04.11.2021 03:05:08

sentiment

Filename	Text	Sentiment	Positive	Neutral	N
review_20211019T193308.txt	Good bread at breakfast and generally good selection	positive	10	0	
review_20211103T230408.txt	Staffs are nice good location	positive	10	0	
review_20211015T024608.txt	Bed was really comfortable. Location was great	positive	10	0	

Cardinality

Cross filter direction

Many to one (*:1)

Single

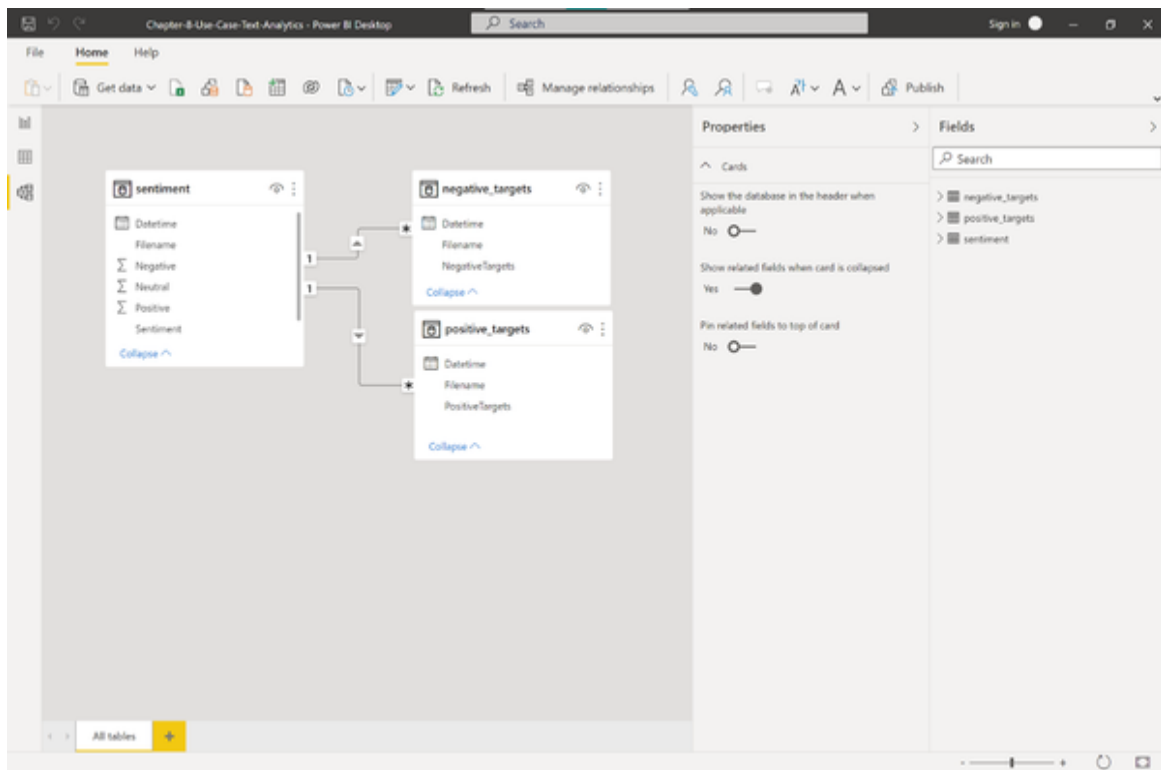
☒ Make this relationship active
 ☐ Apply security filter in both directions

☐ Assume referential integrity

OK

Cancel

Afterwards, the tables in your data model should be connected as shown in Figure XX.

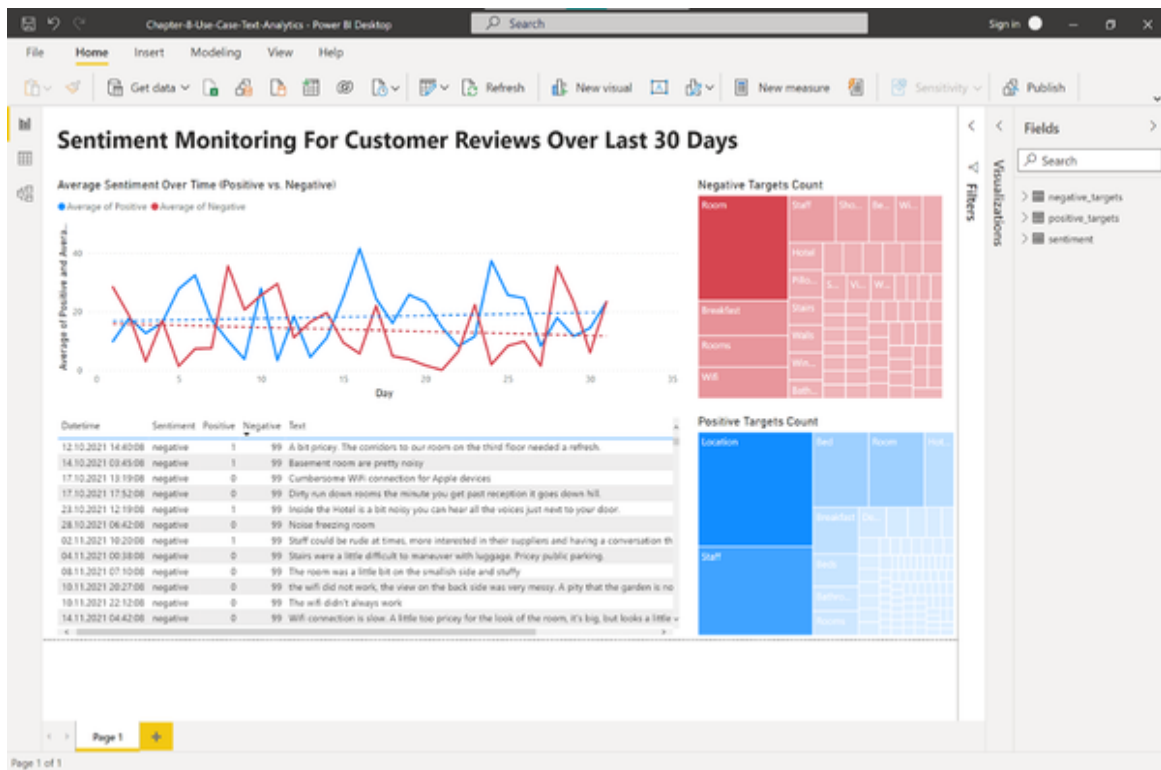


Now that our data model is completed, it's time to move over to handling the actual visuals.

Let's quickly recap the situation: The management wants to know if there is something going on which should be on their radar. To get a high-level overview about the customer reviews we will need four elements:

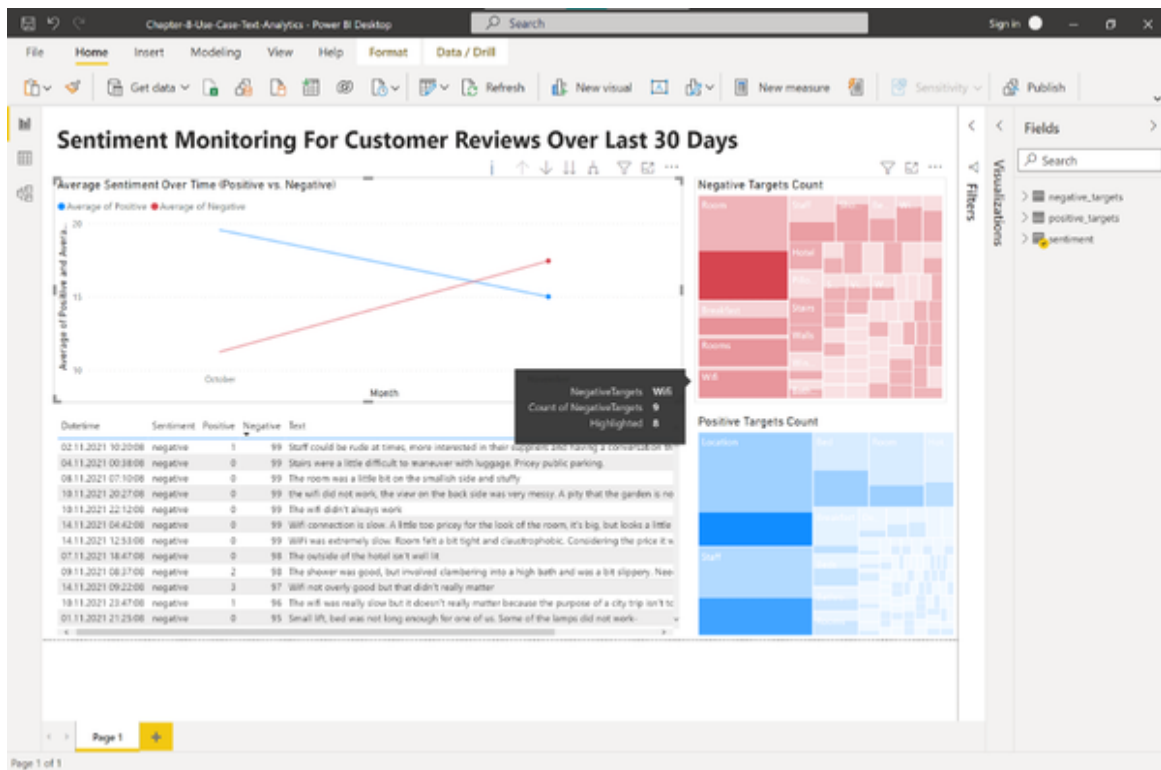
1. The development of customer sentiment over time to check for trends
2. A list of items that customers complain about (negative targets)
3. A list of items that customers like (positive targets)
4. A reference back to the original data so we get more context

There are many different ways to convey this information. I decided to put a report together which consists of a line chart containing the overall trend, two treemaps highlighting the positive and negative targets and a simple table that lists all the customer feedback with plain text. You can see the final result in Figure XX.



Try to recreate this dashboard by yourself in Power BI or come up with your own solution. Or, you can download the complete file “Chapter-8-Use-Case-Text-Analytics.pbix” from the book repository and connect it to your own Azure blob storage.

So what does this dashboard tell us? For once, we can see that both the positive and negative sentiment seems to have a somewhat steady trend with daily ups and downs. If we aggregate the visual to a monthly level, the picture becomes a bit clearer as seen in Figure XX. From a monthly perspective, the negative sentiments seem to have increased a lot. If we explore the reasons for this we can find out that most people actually complain about the rooms, the breakfast and the wifi. While the rooms might be hard to fix in the short term, the breakfast and Wifi provide clear action items that can be addressed quickly by the management.



Also, since we have made the relationship between the different tables, Power BI allows us to filter for certain data points: For example, if we select a date from the line chart, the rest of the dashboard will update accordingly and show only the relevant data that was collected on this day and vice versa. What exactly is it that customers complain about with regards to the room? Just click “Rooms” in the Negative Targets Treemap and see what customers say by exploring the original text at the bottom. This provides a great way to explore the dataset and find out more about the actual customer feedback.

While this dashboard only touches the top of the iceberg of text analytics I hope you have seen how powerful it can be to analyse text data at scale. Thanks to AI services, this can be done with a few lines of code and also integrated in any BI system.

Use Case: Parsing Documents with AI

Have you seen any business that does not use forms? Forms are for businesses what APIs are for programmers. Forms have a great advantage. They typically work very well to exchange information between humans. Given a specific form and some descriptions you should be pretty quick to figure out how to read a form or how to fill it in. The drawback often comes when forms need to be processed by a computer. Oftentimes, forms still live as paper documents or their electronic pendants - PDFs. If you want to process them at scale, you often don't have any other choice than looking through them manually and copy/pasting the data from it. Receipts can also be considered a form - they provide a recurring data structure which is easy to read for humans, but hard to interpret for machines. In this use case we will learn how AI will help us to read documents at scale and extract values from it to display them in a BI system.

Problem Statement

In our next scenario, imagine that we are working for a medium-sized company that provides consulting services. The business is located in Germany where sales representatives travel frequently to potential customers to prepare or close deals. The travels are mostly done via high speed trains that connect the big cities in Germany, called InterCity Express (ICE).

Our travel management department keeps control of the overall expenses. The process for business travels looks as follows: Sales reps can book their train tickets over a self-service portal from the train operator (Deutsche Bahn). These tickets are paid using a company credit card. The travel management team oversees the credit card billing at the end of each month to get an overview of the travel expenses. However, the credit card statements do not provide any further detail except for the amount spent and the date when this happened. To optimize travel expenses and also understand better which business trips are causing most costs, the travel management wants more insights about the trips including a breakdown by popular travel routes (train origin and train destinations).

This information can be found in the booking receipt which the sales representatives get automatically via e-mail after every successful booking. An example of such a receipt is shown in Figure XX.



Online-Ticket

ICE Fahrkarte

Fahrtantritt am 17.01.2020

Flexpreis (Einfache Fahrt)

Klasse: 1

Erw: 1, mit 1 BC50

Hinfahrt: Hamburg+City - Leipzig+City, mit ICE

Über: VIA: (LWL*WBE/UE*SDL)*BSP*(WB/P*KOET)

Storno kostenfrei bis 1 Tag vor 1. Geltungstag

Barcode bitte nicht kassieren!



Zahlungspositionen und Preis

Positionen	Preis	MwSt D: 19%	MwSt (D) 7%
ICE Fahrkarte 1	86,00€		86,00€ 5,63€
Reservierung 1	0,00€		
Summe	86,00€		86,00€ 5,63€

Kreditkartenzahlung

Betrag 86,00€ VU-Nr 4556695619 Transaktions-Nr 685743
Datum 16.01.2020 Gen-Nr N64WYP

Ihre Kreditkarte wurde mit dem oben genannten Betrag belastet. Die Buchung Ihres Online-Tickets erfolgte am 16.01.2020 17:52 Uhr. DB Fernverkehr AG/DB Regio AG, Stephensonstr. 1, 60326 Frankfurt, Steuernummer: 29/001/60002.

Hinfaht:
Gültig ab: 17.01.2020

Zangenabdruck

Herr Jens walter

Auftragsnummer:

6QBUR1

Ihre Reiseverbindung und Reservierung Hinfahrt am 17.01.2020

Halt	Datum	Zeit	Gleis	Produkte	Reservierung
Hamburg Hbf	17.01.	ab 14:35	5	ICE 603	1 Sitzplatz, Wg. 12, Pl. 93, 1 Fenster, Großraum,
Leipzig Hbf	17.01.	an 17:46	11		Nichtraucher, Handy, Res.Nr. 8062 6024 1400 36

Wichtige Nutzungshinweise:

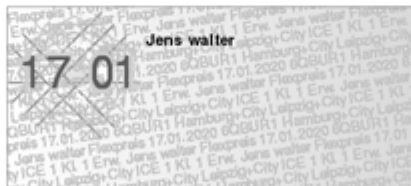
- Ihre Fahrkarte gilt nur zusammen mit einem amtlichen Lichtbildausweis (z.B. Personalausweis) oder Ihrer BahnCard.
- Mit Ihrer Fahrkarte zum Flexpreis können Sie jeden Zug der gewählten Verbindung nutzen: mit einer ICE/EC-Fahrkarte alle IC- und EC-Züge, mit einer ICE-Fahrkarte auch alle anderen Züge.
- Das Online-Ticket gilt nur für den unter "Fahrkarte" angegebenen Reiseabschnitt. Die Übersicht "Ihre Reiseverbindung" enthält gegebenenfalls Reiseinformationen zu Teilstrecken (z.B. Bus oder Straßenbahn), für die eine weitere Fahrkarte erforderlich sein kann.
- Wenn Ihr Ticket den Zusatz "+City" oder "City mobil" zeigt, gilt dieser nur am Tag der Hinfahrt bzw. am Tag der Rückfahrt.
- Es gelten die nationalen und internationalen Beförderungsbedingungen der DB AG. Innerhalb von Verkehrsverbünden und Tarifgemeinschaften gelten deren Bedingungen. Alle Bedingungen finden Sie unter: www.bahn.de/agb und www.diebfahrer.de.

Ihre Reisedaten können sich kurzfristig durch Bauarbeiten oder andere erforderliche Fahrplananpassungen ändern.

Bitte informieren Sie sich kurz vor Ihrer Reise über mögliche Änderungen Ihrer Reisedaten unter www.bahn.de/reiseplan oder mobil über die App DB Navigator. Achten Sie auch auf Informationen und Ansagen im Zug und am Bahnhof. Wir danken Ihnen für Ihre Buchung und wünschen Ihnen eine angenehme Reise!



6QBUR1



Jetzt Online-Ticket in die
App DB Navigator
laden und Echtzeit-Infos
zu Ihrer Reise erhalten!
www.bahn.de/ticket-laden

Seite 1 / 1

In particular, the team is interested in extracting the following information from this form: Booking date, trip origin, trip destination and ticket price. The travel management wants to find out if there is a way to extract these information automatically and ideally report them using the existing BI. They have provided us with a sample of 162 receipts from a single sales representative to work on a first prototype.

Solution Overview

What we are going to do in this scenario is to extract information from a document using a technique called optical character recognition (OCR). Now, OCR isn't new and has been around for a while. However, there are at least two layers in this problem where AI will help us: The first layer is to improve the actual OCR, that is recognizing single text characters and converting them into machine-readable form (plain text string). The second part is actually making sense of the characters, that is to find out which characters belong to a word, a sentence or even recognizing more complex structures like tables.

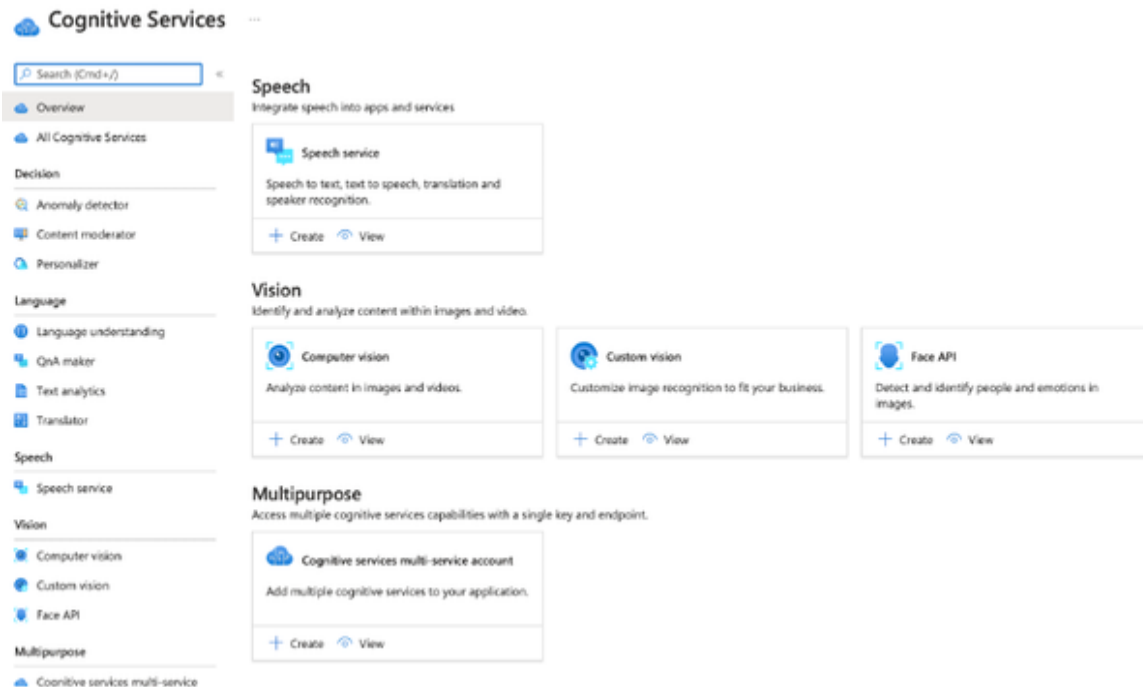
To tackle the problem, we will follow a similar approach as in the use case before:

1. We deploy an out-of-the box AI service on Microsoft Azure, in this case Cognitive Services for Computer Vision
2. We will load the data into a staging area, in this case again using Azure blob storage
3. We will prepare a small ETL script that loads the data from the staging area, applies the AI service and transforms them to a flat CSV.
4. We will upload the CSV file to a location from where it can be easily accessed and visualized with our BI tool.

Let's go!

Setting up the AI service

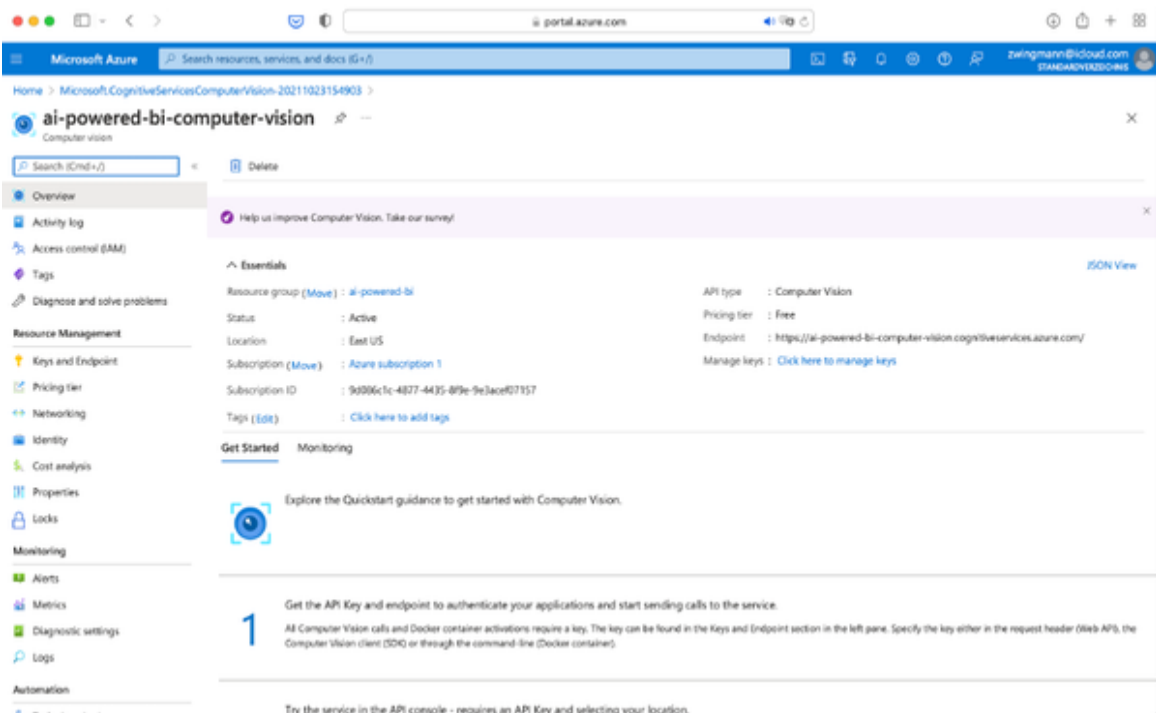
Go to your **Azure portal** and search for “Cognitive Services” in the search bar. Select Cognitive Services from the suggestion list and proceed to the respective resource page. Scroll down to “Vision”. You should see the available services as shown in Figure XX:



Select “+ Create” from the Computer vision box and you should be greeted with the now well-known form to set up a computer vision resource. Again, select a resource group, give your resource a name and select the free F0 pricing tier. Don’t forget to set the checkbox at the bottom that you agree to the “Responsible AI terms” of use.

Take a minute to acknowledge these terms. They essentially mean that you don’t violate personal rights by using the AI. Remember: Every image or document that you send to the API will be processed by services owned and operated by Microsoft. So, especially if you deal with personal data you have to ensure that you have the consent and permission to do this. For our use cases we are dealing with fictional or public domain data so there is no risk involved. However, in a business setting you would think twice before you start sending documents of your customers or employees to a remote AI Service.

Click “Review + create” once you completed the form and “create” once the automatic validation passed. After a few minutes your service should be deployed and you can access it either by clicking the notification link or by searching for your resource name through the search bar in Azure portal. If you open the resource page you should see a screen which looks similar to the one in Figure XX.



The quickstart looks a bit different than the previous AI service but essentially contains the same information: You will find a link to your access keys and some code examples to get started quickly. For now, you can navigate to “Keys and endpoints” where you will find the resource HTTP endpoints and secret access keys. Leave this page open, because we will need these keys in a bit.

The AI setup is completed, let’s move on to load and process the pdf files.

NOTE

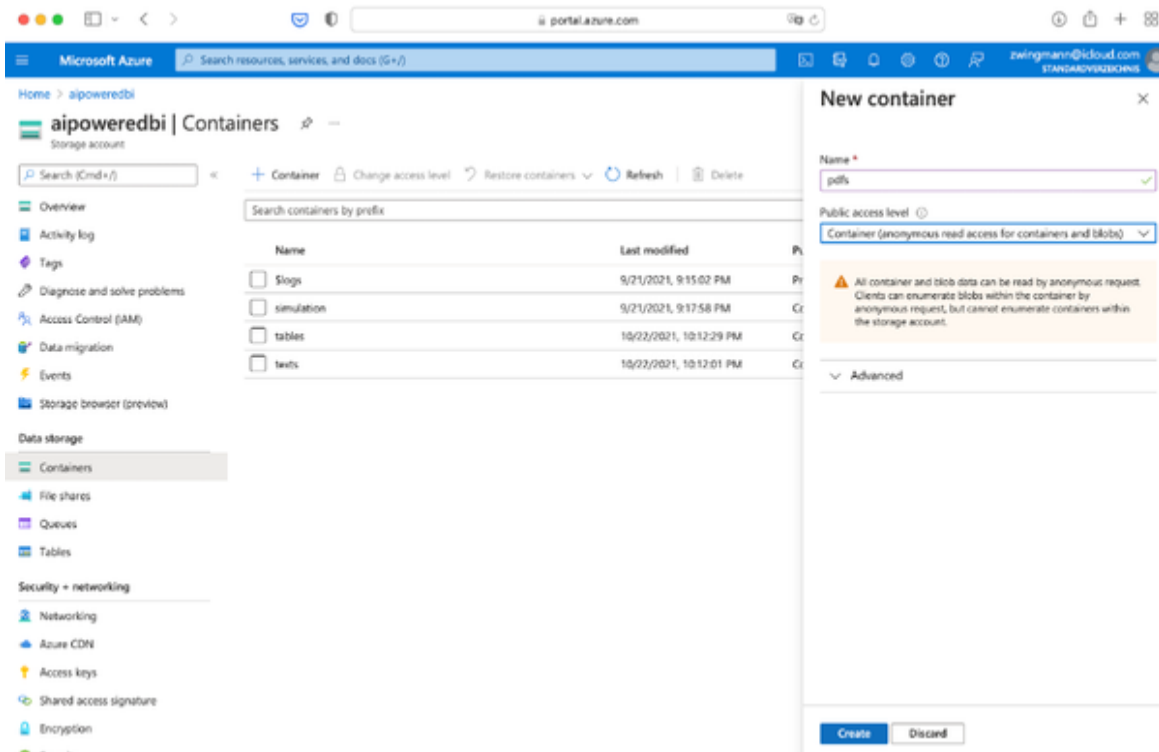
Wait a second, you might say. Why didn't we use the dedicated Azure service "Form recognizer" for this?

Two reasons:

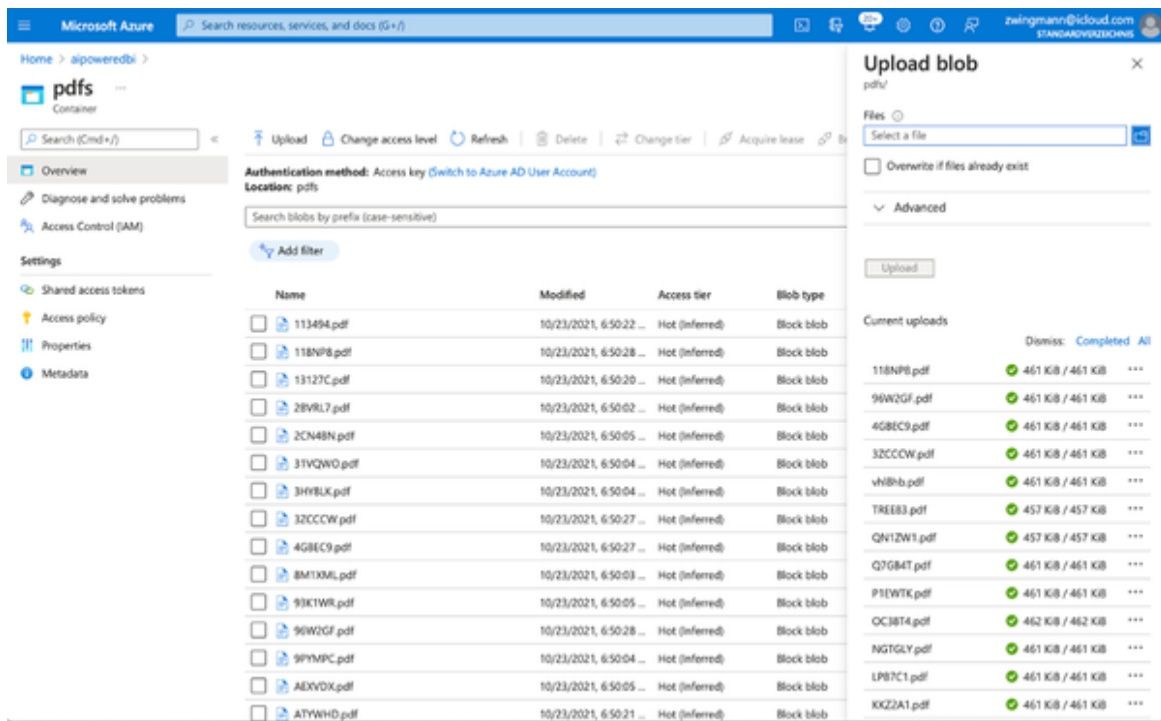
Form Recognizer is a more specialised service that tailors computer vision to form recognition and value extraction. The service is more focused, but also needs a certain degree of custom training and setup before it works, it is no out-of-the box service. On the other hand, the computer vision API is a multi-purpose service that works like plug and play. It is easier to implement, but you will hit limits once the task gets too specialized. It will also be useful for you in other scenarios than form extraction. My recommendation would be to start with the general computer vision first and switch over to more customized / tailored services later, if needed. If you want to learn more about Form Recognizer, you can find more details here: [Form Recognizer - Azure Applied AI Services](#)

Setting up the data pipeline

Let's first upload our PDF files to an Azure Blob storage container from where we can access them easily. Open a new Azure Portal window and navigate to Azure blob storage using the search bar or looking in your "recent items" in your dashboard. Navigate to "Containers" and create a new container called "pdfs" as shown in Figure XX.

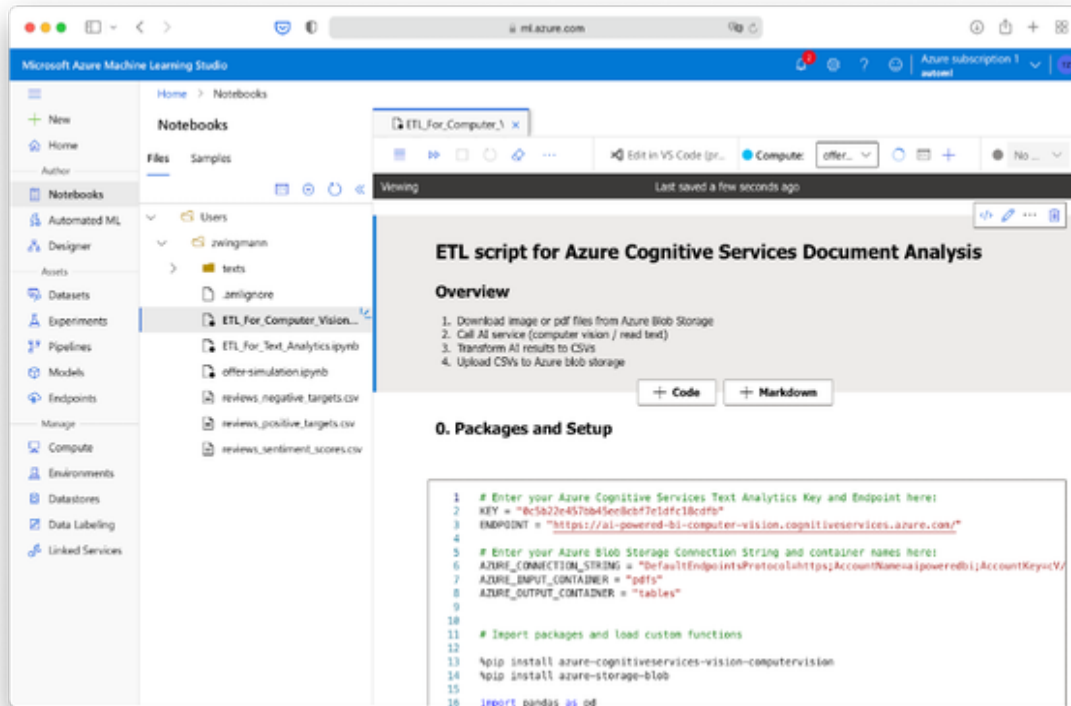


Now, download the file “pdfs.zip” from the book repository and unzip it on your local computer. Download all the contents from your local “pdfs” folder to the container in Azure blob storage. At the end, all pdf files should be stored in the “pdfs” container on Azure blob storage without any subfolder or any zip file as shown in Figure XX.



Now that the files are ready for analysis we can move ahead to the ETL process. As in the previous example I will walk you through the code step by step and we will run the script on Azure Notebooks. Of course you can also use your local IDE if you prefer to.

First things first, download the file “ETL_For_Document_Analysis.ipynb” from the book’s repository to your local computer. Open **Azure Machine Learning Studio**, select your preferred workspace and click “Get started”. Navigate to “Notebooks” and choose “Upload files” as shown in Figure XX. Select the file “ETL_For_Document_Analysis.ipynb” and upload it to Azure Notebooks. After the upload, the notebook shows up on the right side of the screen. Don’t forget to connect your notebook to a compute resource as we did in the previous use case. When everything is ready your screen should look similar to Figure XX.



I will now walk you through the code and explain what is going section by section. Please follow along by executing the code cells in the order as we cover it in the book.

First, the section “0. Packages and Setup” again is the place where you need to input your custom Azure Connection String as well as your custom key and endpoint for your computer vision service. Do you still have the window open? Good, then you can just copy and paste both values here. In the following part, the code ensures that all required packages are imported and we define some custom functions. I will explain these functions in more detail once we are going to use them. Now, run the first code cell.

Section 1 is covering the file download from the Azure blob storage. This one is quite straightforward and identical to what we did in the previous use case. Run this code cell as well. As a result you should see a folder in the file browser called “pdfs”. If not, click the small refresh icon in the file browser.

Section 2 is again a very short one. Here, we are just calling the AI service and fetching the raw results from the analysis. Two important things to note here: First, you might notice the `time.sleep(6)` command in the code. I included this line to make sure we are not exceeding the free tier limit of 20 requests per minute. So why do we wait 6 seconds and not only 3, because $3 \text{ seconds} * 20 \text{ requests} = 60 \text{ seconds}$? This is because in the case of computer vision we are running a so-called “asynchronous operation”. Remember that in the previous example for the sentiment analysis we just sent the text to the API and immediately received the response with the AI results. In the case of computer vision, the operation is more complex and the results can actually take a while (like seconds). Therefore, for each document, we need to make actually two API calls: One POST request to send the document to the AI service. And one GET request to fetch the results. If you look carefully into the function `document_analysis` under section 0, you will notice both API calls. And in fact, we are waiting 2 seconds each time before we make the GET request to increase the chances that the result is there for every call that we make. If you move to a paid plan you can safely delete the `time.sleep(6)` line from the loop in Step 2. But I do recommend to keep a short pause between the first POST and GET request, otherwise you will run into a lot of empty (but still billed) API calls. Considering the limitations of the free plan, the overall analysis for the 162 pdf documents should take around 30 minutes. Run the code cell. And now it's time for a coffee break before we head on for the dirty work!

Calling the AI service so far has been pretty easy. Extracting the information from the response can be messy, though. To understand why, let's take a look at what the result from the AI actually looks like. To do this, we can't actually just print the result object. The result object is a nested structure which we need to unpack. Fortunately, the documentation with Python code examples shows us how to do this:

```
# Inspect the first result
read_result = results[0]
# Print the detected text, line by line
if read_result.status == OperationStatusCodes.succeeded:
    for text_result in read_result.analyze_result.read_results:
```

```

for line in text_result.lines:
    print(line.text)
    print(line.bounding_box)

```

The code above will yield the following output (truncated for brevity):

```

DB
[0.6349, 0.5273, 1.0222, 0.5165, 1.0222, 0.764, 0.6349, 0.764]
Online-Ticket
[3.0799, 0.5784, 4.6416, 0.5784, 4.6416, 0.7642, 3.0799, 0.7642]
ICE Fahrkarte
[0.5206, 1.1069, 1.4109, 1.1069, 1.4109, 1.2117, 0.5206, 1.2117]
...
Betrag
[0.5614, 3.5237, 0.873, 3.5237, 0.873, 3.6278, 0.5614, 3.6278]
92,00€
[1.109, 3.5237, 1.455, 3.5237, 1.455, 3.6197, 1.109, 3.6197]
...
Datum
[0.5621, 3.6571, 0.8733, 3.6571, 0.8733, 3.7384, 0.5621, 3.7384]
02.02.2018
...
Halt
[0.5207, 4.6438, 0.7449, 4.6438, 0.7449, 4.7352, 0.5207, 4.7352]
Datum
[2.4111, 4.6438, 2.7826, 4.6438, 2.7826, 4.7352, 2.4111, 4.7352]
Zeit
[2.9165, 4.6429, 3.1326, 4.6429, 3.1326, 4.7352, 2.9165, 4.7352]
Gleis
[3.4701, 4.6429, 3.7657, 4.6429, 3.7657, 4.7358, 3.4701, 4.7358]
Produkte
[4.104, 4.6438, 4.6302, 4.6438, 4.6302, 4.7352, 4.104, 4.7352]
Reservierung
[4.852, 4.6429, 5.6354, 4.6429, 5.6354, 4.7605, 4.852, 4.7605]
Hamburg Hbf
[0.5214, 4.8136, 1.2506, 4.8136, 1.2506, 4.932, 0.5214, 4.932]
02.02.
[2.4063, 4.8167, 2.7382, 4.8167, 2.7382, 4.9068, 2.4063, 4.9068]
ab 16:36 5
[2.9179, 4.8148, 3.5288, 4.8148, 3.5288, 4.9068, 2.9179, 4.9068]
ICE 1527
[4.1059, 4.8209, 4.6115, 4.8209, 4.6115, 4.9153, 4.1059, 4.9153]
1 Sitzplatz, Wg. 38, Pl. 12, 1 Fenster,
[4.8551, 4.8209, 6.8913, 4.8209, 6.8913, 4.9404, 4.8551, 4.9404]
Leipzig Hbf
[0.5213, 4.9636, 1.1351, 4.9636, 1.1351, 5.082, 0.5213, 5.082]

```



```
...
Seite 1 / 1
[7.1081, 11.2414, 7.5931, 11.2414, 7.5931, 11.3253, 7.1081,
11.3253]
```

To make sense of this, let's compare the output to the original document as shown in Figure XX:



Online-Ticket

ICE Fahrkarte

Fahrtantritt am 02.02.2018

Flexpreis (Einfache Fahrt)

Klasse: 1

Erw: 1, mit 1 BC50

Hinfahrt: Hamburg+City → Leipzig+City, mit ICE

Über: VIA: (LWL*WBE/UE*SDL)*BSP*(WB/P*KOET)

Umtausch/Erstattung kostenlos bis 1 Tag vor Reiseantritt (Hinfahrt).

Zahlungspositionen und Preis

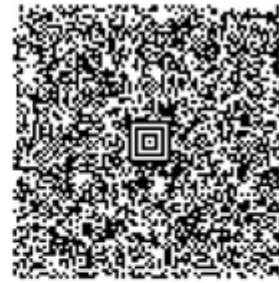
Positionen		Preis	MwSt (D) 19%	MwSt D: 7%
ICE Fahrkarte	1	92,00€	92,00€	14,69€
Reservierung	1	0,00€		
Summe		92,00€	92,00€	14,69€

Kreditkartenzahlung

Betrag	92,00€	VU-Nr	4556895619	Transaktions-Nr	787489
Datum	02.02.2018	Gen-Nr	NBUNEB		

Ihre Kreditkarte wurde mit dem oben genannten Betrag belastet. Die Buchung Ihres Online-Tickets erfolgte am 02.02.2018 13:27 Uhr, DB Fernverkehr AG/DB Regio AG, Stephensonstr. 1, 60326 Frankfurt, Steuernummer: 29/001/60002.

Barcode bitte nicht knicken!



Hinfahrt:
Gültig ab: 02.02.2018

Zangenabdruck

Herr Jens walter

Auftragsnummer:

113494

Ihre Reiseverbindung und Reservierung Hinfahrt am 02.02.2018

Halt	Datum	Zeit	Gleis	Produkte	Reservierung
Hamburg Hbf	02.02.	ab 16:36	5	ICE 1527	1 Sitzplatz, Wg. 38, Pl. 12, 1 Fenster,
Leipzig Hbf	02.02.	an 19:42	11		Panorama / Lounge, Nichtraucher, Ruhebereich, Res.Nr. 8091 3012 3081 12

Wichtige Nutzungshinweise:

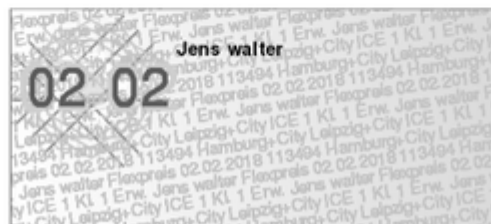
- Ihre Fahrkarte gilt nur zusammen mit einem amtlichen Lichtbildausweis (z.B. Personalausweis) oder Ihrer BahnCard.
- Mit Ihrer Fahrkarte zum Flexpreis können Sie jeden Zug der gewählten Verbindung nutzen: mit einer IC/EC-Fahrkarte alle IC- und EC-Züge, mit einer ICE-Fahrkarte auch alle anderen Züge.
- Das Online-Ticket gilt nur für den unter "Fahrkarte" angegebenen Reiseabschnitt. Die Übersicht "Ihre Reiseverbindung" enthält gegebenenfalls Reiseinformationen zu Teilstrecken (z.B. Bus oder Straßenbahn), für die eine weitere Fahrkarte erforderlich sein kann.
- Wenn Ihr Ticket den Zusatz "+City" oder "City mobil" zeigt, gilt dieser nur am Tag der Hinfahrt bzw. am Tag der Rückfahrt.
- Es gelten die nationalen und internationalen Beförderungsbedingungen der DB AG. Innerhalb von Verkehrsverbünden und Tarifgemeinschaften gelten deren Bedingungen. Alle Bedingungen finden Sie unter: www.bahn.de/agb und www.diebefoerderer.de.

Ihre Reisedaten können sich kurzfristig durch Bauarbeiten oder andere erforderliche Fahrplananpassungen ändern.

Bitte informieren Sie sich kurz vor Ihrer Reise über mögliche Änderungen Ihrer Reisedaten unter www.bahn.de/reiseplan oder mobil über die App DB Navigator. Achten Sie auch auf Informationen und Ansagen im Zug und am Bahnhof. Wir danken Ihnen für Ihre Buchung und wünschen Ihnen eine angenehme Reise!

bahn.bonus
Das Bonusprogramm der Bahn

113494



Jetzt Online-Ticket in die
App DB Navigator
laden und Echtzeit-Infos
zu Ihrer Reise erhalten!
www.bahn.de/ticket-laden

Seite 1 / 1

In both figures I have highlighted the elements of the document that we are interested in (Date, Price, Origin, Destination)

If we compare the AI output with the original document we can see that the AI parsed the document line by line. It starts with “DB” and “Online-Ticket” from the top left and ends with “Seite 1/1” on the bottom right.

Each line of text is an element of the result object and the numbers that follow are the so-called bounding boxes for these lines, indicating where the line was positioned in the document. The six numbers correspond to the x and y coordinates as shown in Figure XX with the coordinate system starting at the top left of the document with coordinates (x = 0, y = 0). Each list of coordinates is mapped as [x1, y1, x2, y2, x3, y3, x4, y4].

In order to extract just the information we want from the document, we need to rely on the assumption that the document was parsed in western reading order (from left to right, top to bottom - this can actually be adjusted for the AI service if needed).



Let's start with the date information by running the next code cell labeled “part 1” of section 3. In this code cell most work is done by the custom function `get_text_by_keyword(result, “Datum”)`. The function which, was defined in section 0, takes a given keyword argument, in this case “Datum” (German for “Date”) and returns the item which follows just after that in the AI response. Since the AI parses the text from left to right the result will be the actual date that we are looking for. We do some post processing to turn it into a more conventional date format and that's it. Extracting a form value which has a distinct form label next to it is pretty straight forward.

Moving on to part 2, extracting the origin and destination information. This one is a bit trickier since the information is not listed on the same line, but on different lines in the document, similar to a table structure. To make it even more complicated this list can be even longer if there are connecting trains in between the origin and final destination. How do we tackle this? We will use a combined approach of a keyword search and a bounding box filter. Let's take a look before we run the cell.

First, we are calling the function

`get_text_between_keywords(result, "Halt", "Wichtige Nutzungshinweise:")` which will give us all text elements between the start and stop markers. In our case that is the first column header of the table ("Halt") and the first text object that follows after the table ("Wichtige Nutzungshinweise"). This function will extract all text elements in the table as shown in Figure XX.

Ihre Reiseverbindung und Reservierung Hinfahrt am 02.02.2018

Halt	Datum	Zeit	Gleis	Produkte	Reservierung
Hamburg Hbf	02.02.	ab 16:36	5	ICE 1527	1 Sitzplatz, Wg. 38, Pl. 12, 1 Fenster,
Leipzig Hbf	02.02.	an 19:42	11		Panorama / Lounge, Nichtraucher, Ruhebereich, Res.Nr. 8091 3012 3081 12

Wichtige Nutzungshinweise:
- Ihre Fahrkarte gilt nur zusammen mit einem amtlichen Lichtbildausweis (z.B. Personalausweis) oder Ihrer BahnCard

However, we don't want all text elements, but only the items of the first column. How do we get this? There are possibly many ways to approach this, but I found it the easiest to just locate this information using the respective bounding boxes. The idea is that, from the text fragment we have just collected, we just extract just the part which is indicated by the dashed line in Figure XX.

Ihre Reiseverbindung und Reservierung Hinfahrt am 02.02.2018

Halt	Datum	Zeit	Gleis	Produkte	Reservierung
Hamburg Hbf	02.02.	ab 16:36	5	ICE 1527	1 Sitzplatz, Wg. 38, Pl. 12, 1 Fenster,
Leipzig Hbf	02.02.	an 19:42	11		Panorama / Lounge, Nichtraucher, Ruhebereich, Res.Nr. 8091 3012 3081 12

Wichtige Nutzungshinweise:

So how do we get the coordinates for this dashed box? The answer is that we just take the lower left corner of the first column header "Halt" and define it to be the upper left corner of our dashed box. To get the width of

the box we take the x-coordinate of the lower left corner of the text “Datum”, because we know this will always be the second column. This will become the x2 value of our dashed box. And finally, to get the height of the dashed box we search for the highest y3 coordinate in our extracted text area. These coordinates are all we need to draw a rectangle. Now that we have defined our “filter” box we can just keep the text findings where the bounding box is inside the boundaries of our filters. This is what the function `get_text_by_position(result, filter_box)` is handling for us. This function will return a clean list with all the train stops. Now, we just need to take the first and the last item of this list and voilà - there are our origin and destination values. With the conceptual understanding of what’s going on here, run the code cell for part 2.

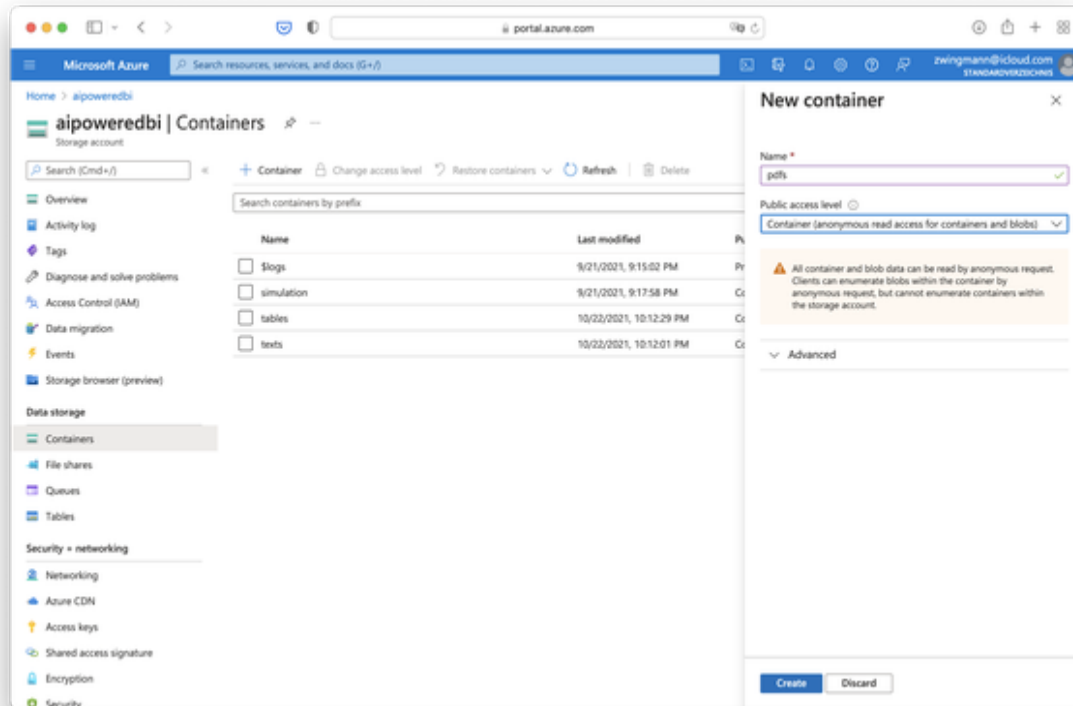
Part 3 is again a bit simpler. In this case we are extracting the price information based on identifying the keyword “Betrag” in the document, very similar to how we extracted the date. The only difference is that this time we are not only extracting the next item after the keyword, but the next two items after the keyword as you can see by the parameter “2” in the function `get_text_by_keyword(result, "Betrag", 2)`. Why is that? The short answer is: It makes our script a bit more robust. We can safely identify the price information based on the “€” sign and we know it should come soon after the label “Betrag”, but it does not necessarily have to be the first item, it could also be the second. Go ahead and run the cell.

By having executed part three there is only one step left for the data transformation step, and that is bringing our results together and writing the CSV table. By now you should have 4 list objects: dates, prices, origins and destinations all having the same length. In this last step we are binding them together to a dataframe and export this as a csv file. Run this last cell and you should see the new file `public-transportation-costs.csv` in the file explorer on the left after a quick refresh. The result table is shown in Figure XX.

	Date	Origin	Destination	Cost	Receipt
0	2017-10-29	München Hbf	Hamburg Hbf	122.00	KKZ2A1.pdf
1	2018-06-08	Leipzig Hbf	München Hbf	101.25	8M1XML.pdf
2	2018-05-21	Leipzig Hbf	Hamburg Hbf	92.00	SYGLKK.pdf
3	2018-07-21	Bonn-Oberkassel Nord	Frankfurt(M) Flughafen Fernbf	56.10	bahn-B3QNOU.pdf
4	2018-04-06	Hamburg Hbf	Leipzig Hbf	89.50	OG4EB6.pdf
...	...	...	...	...	...
157	2020-01-11	Leipzig Hbf	Hamburg Hbf	81.90	bahn-17B6DU.pdf
158	2018-10-02	Konstanz	Bonn Hbf (tief)	109.10	bahn-UKW9RT.pdf
159	2018-11-18	Leipzig Hbf	Hamburg Hbf	89.50	bahn-G4SUFW.pdf
160	2017-10-22	Leipzig Hbf	Hamburg Hbf	87.75	GBEOH3.pdf
161	2017-11-12	Bonn Hbf	Hamburg Hbf	77.75	vhl8hb.pdf

162 rows × 5 columns

Finally, let's upload our result to our container called "tables" on Azure Blob storage. The last code cell in section 4 will handle this for you. Run this cell and give yourself a pat on the back: The biggest part of the work has been completed. We can now move ahead and visualize the results in our BI.



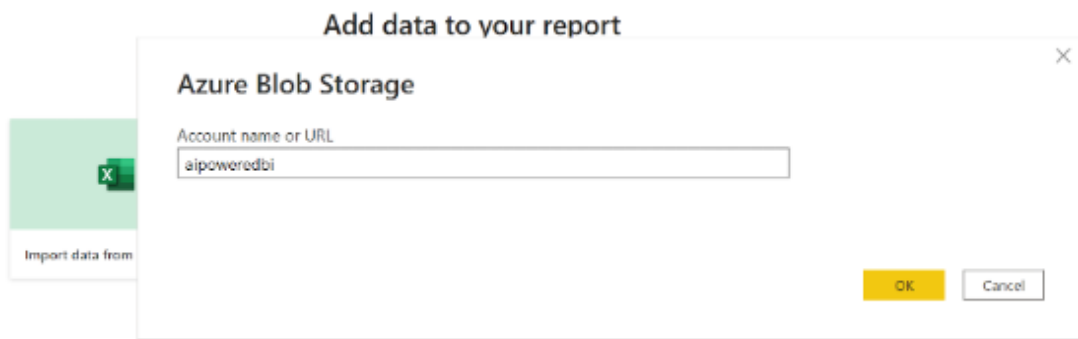
NOTE

Don't forget to pause or delete your compute resource from the notebook!

Visualising the results in BI

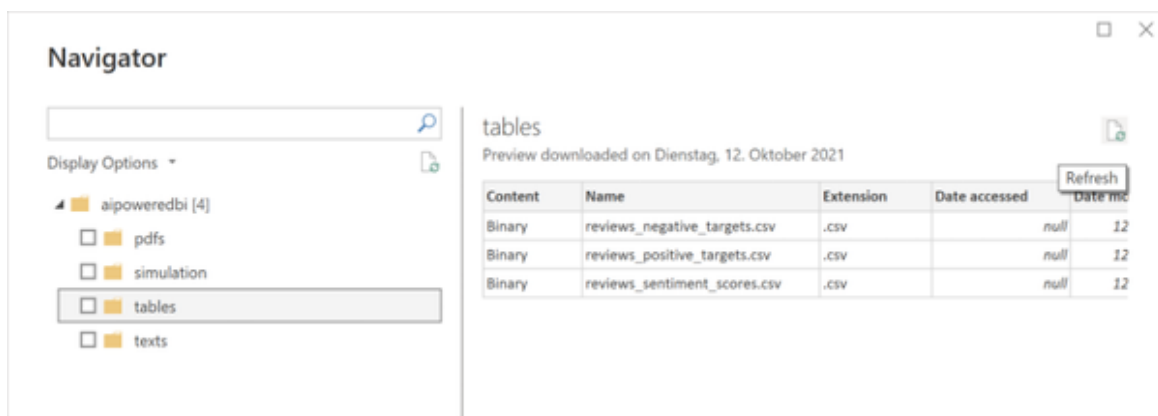
Let's move on to the fun part. Open your favorite BI and load the csv which we just created to get more insights. I will walk you through the process again on the example of Power BI.

In Power BI, create a new report and choose "Azure Blob Storage" from "Get data". Provide the name of your Azure Blob Storage account as shown in Figure XX.

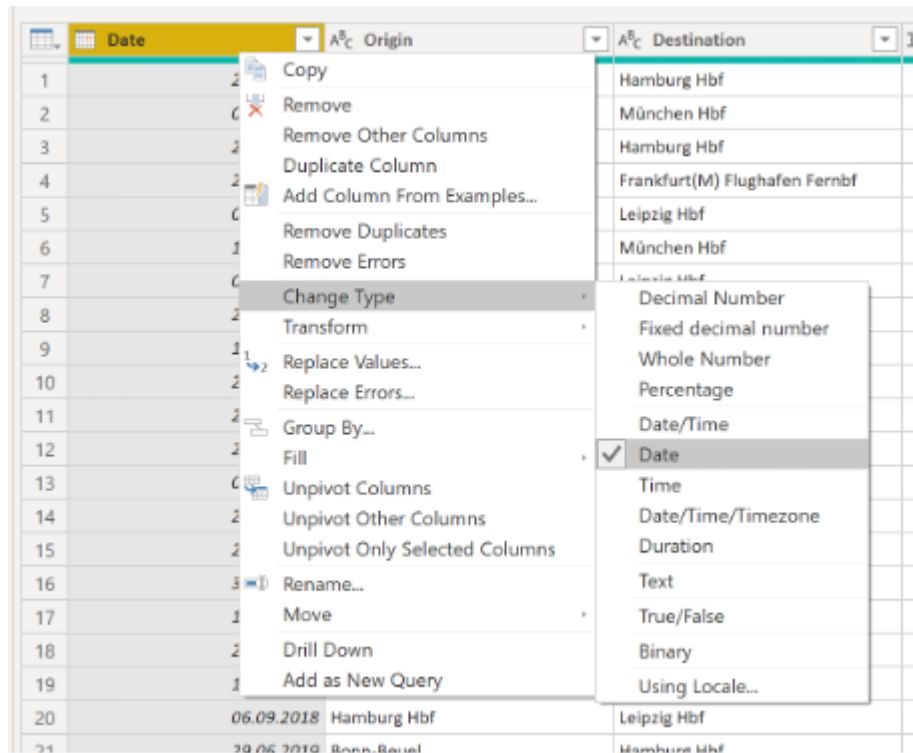


You should see the file list of your blob storage. Navigate to the folder “tables” and hit “Refresh” as seen in Figure XX if you still can’t see the new file `public-transportation-costs.csv`.

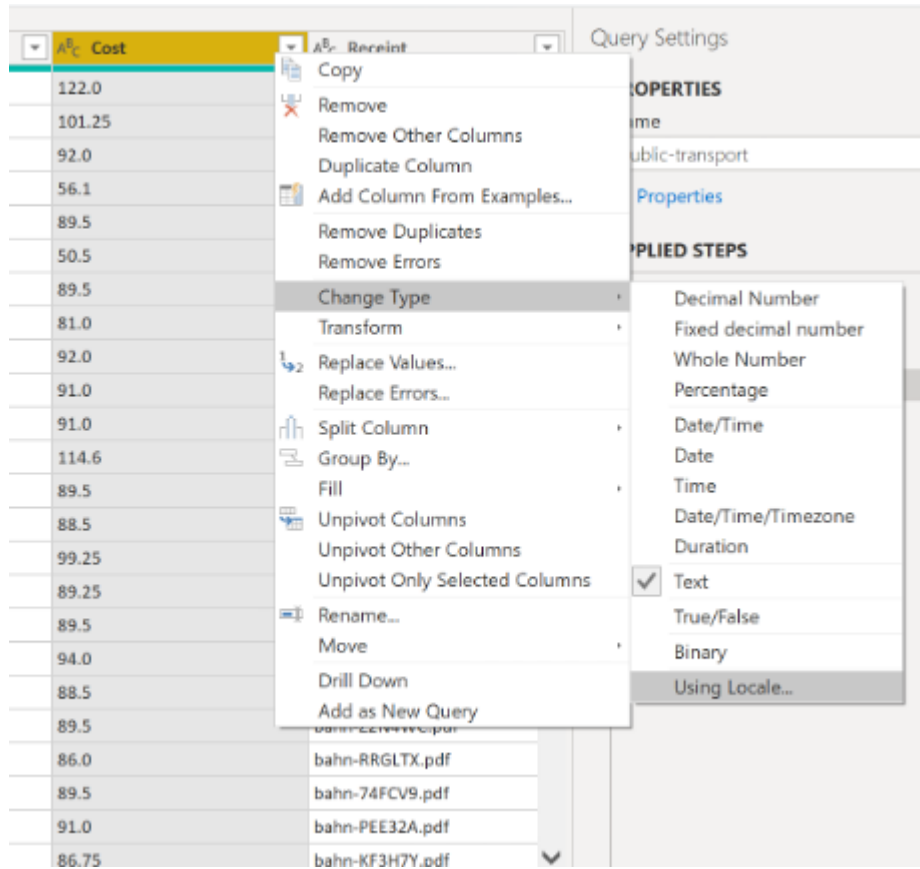
Set the checkbox on the tables folder and click “Transform Data” which will open up Power Query. Click the link “Binary” next to the name of the CSV file “public-transportation-costs.csv”



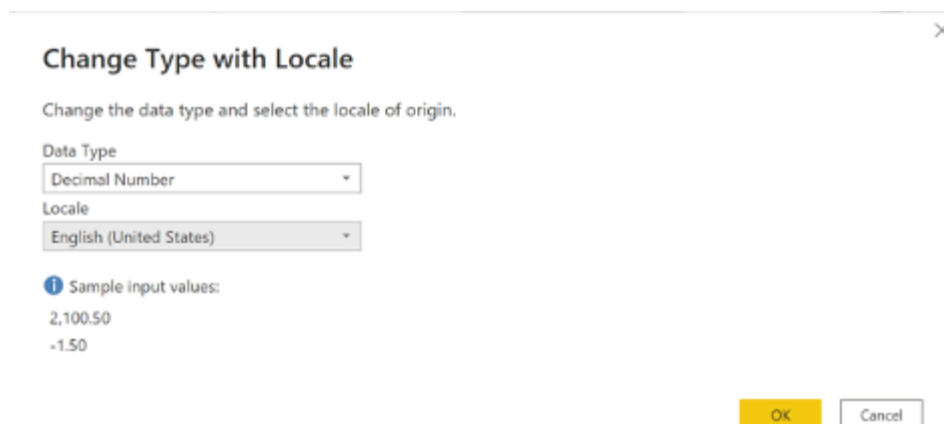
There are two things you want to make sure here: First, that the date column is correctly identified as “Date” data type and second, that the prices are actually converted to a numeric value. If Power BI does not automatically recognize the correct data types, do a right click on the “Date” column and select “Date” as shown in Figure XX.



If the prices are not showing up as numeric value, right click on the column and choose Choose Type → Using Locale from the context menu as shown in Figure XX.



Change the data type to Decimal Number and set the locale to English (United States) as shown in Figure XX.



Finally, your data set should look similar to Figure XX. Close Power Query by selecting “Close & Apply”

Untitled - Power Query Editor

Queries [5]

- Diagnostics [4]
 - public-transport_Changed Ty...
 - public-transport_Changed Ty...
 - public-transport_Changed Ty...
 - public-transport_Changed Ty...
- Other Queries [1]
 - public-transport

	Date	Origin	Destination	Cost
1	29.10.2017	München Hbf	Hamburg Hbf	
2	08.06.2018	Leipzig Hbf	München Hbf	
3	21.05.2018	Leipzig Hbf	Hamburg Hbf	
4	21.07.2018	Bonn-Oberkassel Nord	Frankfurt(M) Flughafen Fernbf	
5	06.04.2018	Hamburg Hbf	Leipzig Hbf	
6	12.11.2019	München Hbf	München Hbf	
7	07.12.2018	Hamburg Hbf	Leipzig Hbf	
8	23.01.2019	Hamburg Hbf	Bonn Hbf	
9	19.01.2018	Hamburg Hbf	Leipzig Hbf	
10	22.06.2019	Leipzig Hbf	Hamburg Hbf	
11	25.01.2018	Hamburg Hbf	Bonn Hbf	
12	21.10.2018	Bonn-Oberkassel Nord	Leipzig Hbf	
13	08.07.2018	Leipzig Hbf	Hamburg Hbf	
14	24.05.2018	Hamburg Hbf	Bonn Hbf	
15	21.11.2019	Hamburg Hbf	Leipzig Hbf	
16	30.01.2020	Hamburg Hbf	Leipzig Hbf	
17	12.08.2018	Leipzig Hbf	Hamburg Hbf	
18	28.03.2018	Hamburg Hbf	Leipzig Hbf	
19	10.04.2018	Hamburg Hbf	Bonn Hbf	
20	06.09.2018	Hamburg Hbf	Leipzig Hbf	
21	29.06.2019	Bonn-Beuel	Hamburg Hbf	
22	10.08.2018	Hamburg Hbf	Leipzig Hbf	
23	12.01.2019	Leipzig Hbf	Hamburg Hbf	
24	10.06.2019	Bonn Hbf	Hamburg Hbf	
25				

5 COLUMNS, 162 ROWS Column profiling based on top 1000 rows

PREVIEW DOWNLOADED AT 23:30

Query Settings

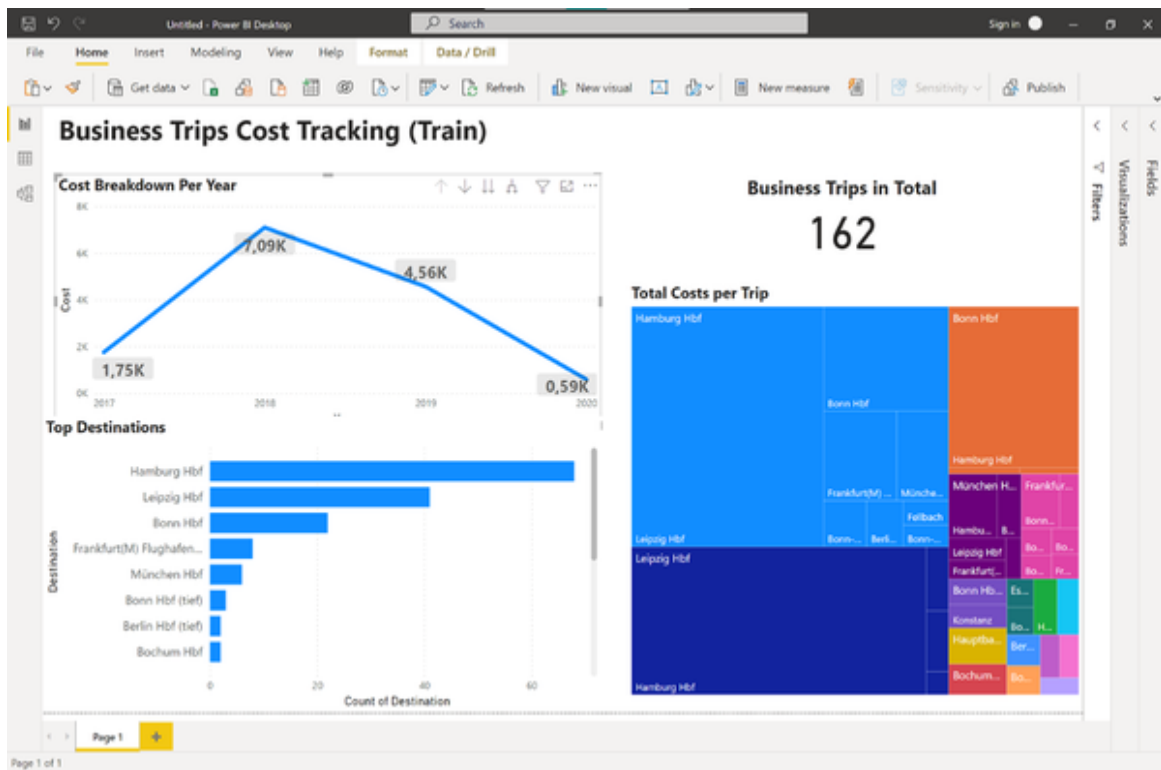
PROPERTIES

Name: public-transport

APPLIED STEPS

- Source
- Navigation
- Imported CSV
- Promoted Headers
- Changed Type with Locale
- Changed Type

Head over to the report and create some new visuals. I decided to show the total travel expenses per year in a line chart, the total number of business trips as a metric, the top destinations as a horizontal bar chart and finally, all routes (combinations of origins and destinations) as a treemap where the size equals the money spent as shown in Figure XX.



With this visual the travel management team can clearly see the overall cost trend and absolute trip numbers. And they can also identify which routes contribute the most to the overall travel expenses. In our case we can see that the connection from Hamburg to Leipzig and back accounted for the largest proportion of travel costs. We can also see that most trips actually started in Hamburg with Bonn and Frankfurt being the second and third most popular destinations from this origin.

Feel free to recreate this dashboard on your own or see if you can find even better ways to display the data.

If you want to see the final dashboard that I did, you can download the file [Chapter-8-Use-Case-Document-Analytics.pbix](#) from the book's repository.

Use Case: Detecting Objects in Images

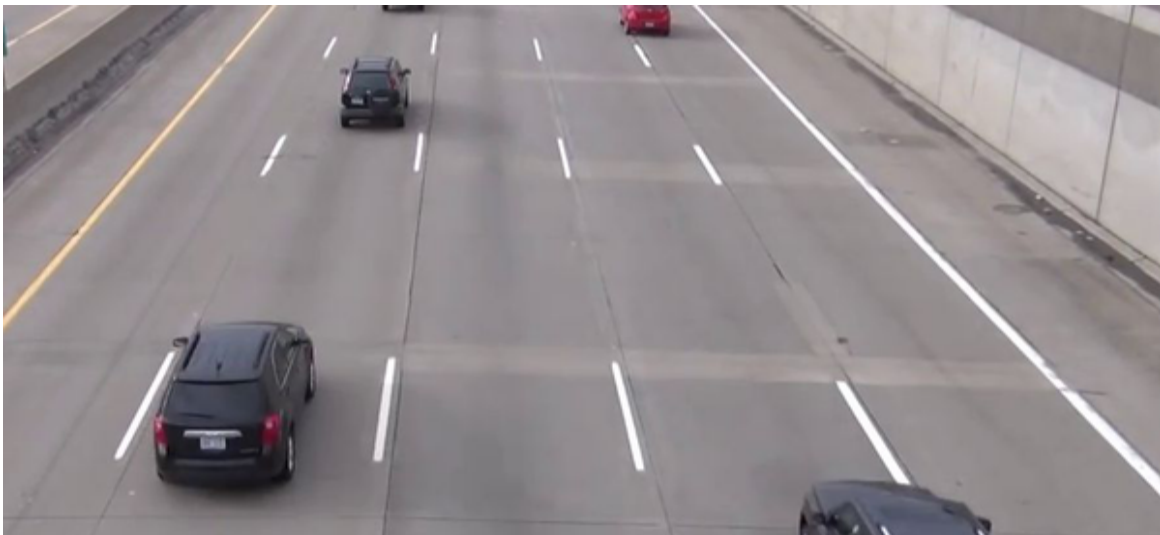
With the previous use case we have only touched the vast capabilities of computer vision. In the following scenario I would like to show you how

you can use the existing infrastructure even further, namely to detect objects in images. For this, let's take a closer look at the problem at hand.

Problem Statement

We are working for a transport and road authority that tries to improve overall traffic management and traffic flow to minimize road congestion. One important tool is to control the speed limits on highways. To set speed limits, the authority needs an ideally constant measurement of the traffic flow on the roads. While new roads are equipped with respective measurement sensors and technology, old roads often have only CCTV cameras installed that were used by traffic managers for manual traffic inspections. The operations team approached us to check if it is possible to count the traffic using the existing camera infrastructure and has provided us with some sample CCTV footage for testing purposes.

Figure XX shows an example of the CCTV footage.



The CCTV images for this use case were provided via a dataset on Kaggle ([Highway CCTV footage images](#)).

Solution Overview

To tackle the problem at hand we are building on the existing computer vision AI service from the previous use case. Instead of a “Read” operation we are now calling a service called “Detect objects”. In this case we want to analyse data on the fly. So instead of loading a dataset to an Azure Blob storage we are consuming the CCTV footage directly from a web URL and will feed this web URL into the AI service. We will still have a small ETL job in place that consumes the http URLs, calls the AI service and writes the output as a flat CSV file to an Azure Blob storage so we can consume it with our BI tool. Let’s start!

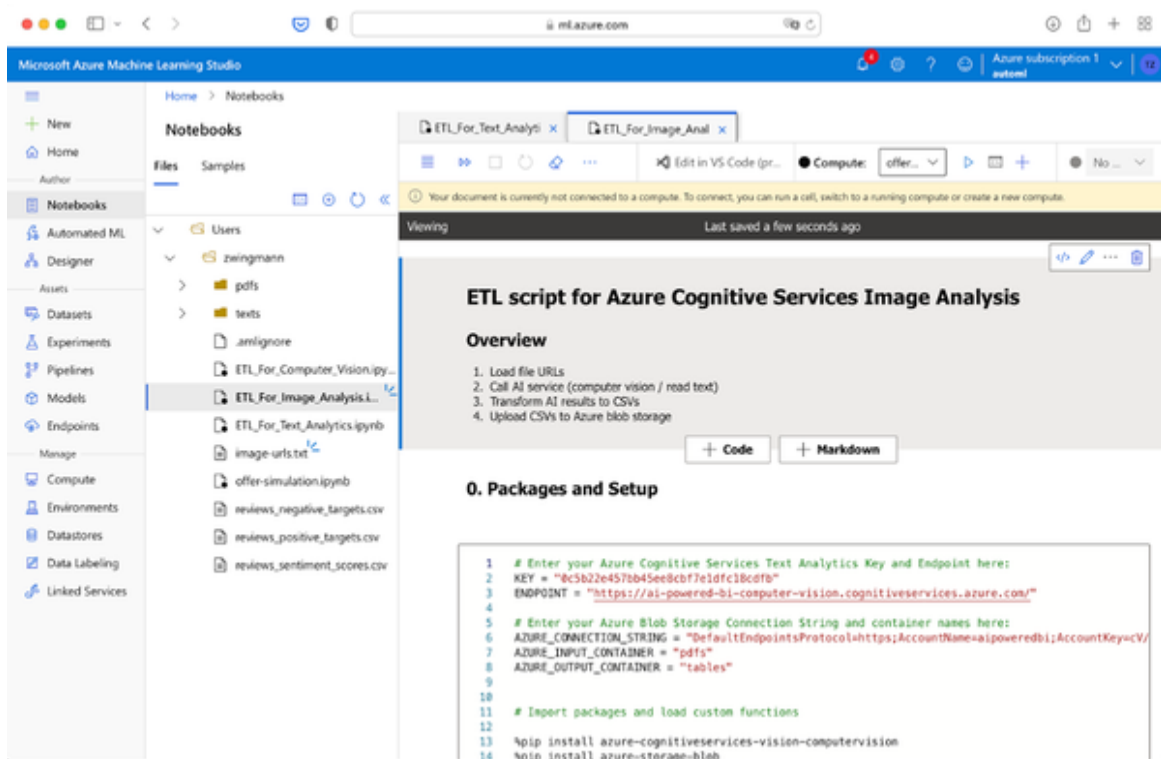
Setting up the AI Service

We are using the same AI service which we deployed previously in the use case for parsing documents. If you haven’t completed it yet, go to the previous section, complete the steps listed under “Setting up the AI service” and come back here. If you did it already, just follow along - there’s nothing more you need to do. As it turns out, computer vision has many different use cases that run under the same service. With the computer vision service that you have just deployed you can not only extract text or detect objects, but also recognize brand names, popular landmarks, tag images and many more. For a complete guide check out the complete [Computer Vision documentation for Azure Cognitive Services](#).

Setting up the data pipeline

Download the file “ETL_For_Image_Analysis.ipynb” from the book’s repo. Just as before, open the file in your local IDE or upload it to Azure Notebooks. In addition to this file you will also need the file “image-urls.txt” which needs to be located in the same directory as the notebook file.

If you open the file ETL_For_Image_Analysis.ipynb, you will see that it looks very similar to the ones which we used previously as you can see in Figure XX. I’ll quickly walk you through the main steps.



In section 0 update your Azure credentials. These are the same as you used in the previous use case. After you enter these, run the first code cell.

In section 1, execute the code cell to import the txt file with the image URLs. Make sure that you downloaded the file from the books' repository and placed them in the same folder as the .ipynb file.

Section 2 is calling the computer vision AI service. In contrast to the use case before, object detection is a synchronous operation, so we actually only need one API call per image and can reduce the wait time to 3 seconds after each image to stay within the 20 images / minute limit of the free tier. If you are on a paid tier, you can safely delete the line `time.sleep(3)`

from the script. Execute the code cell. While you wait for the results, this is a good time to draw your attention to some restrictions or preconditions for the object detection AI service - and these also apply more or less also for other providers, not only for Microsoft Azure. As we can read in the [Azure documentation](#), the following limitations apply:

It's important to note the limitations of object detection so you can avoid or mitigate the effects of false negatives (missed objects) and limited detail.

- Objects are generally not detected if they're small (less than 5% of the image).
- Objects are generally not detected if they're arranged closely together (a stack of plates, for example).
- Objects are not differentiated by brand or product names (different types of sodas on a store shelf, for example). However, you can get brand information from an image by using the Brand detection feature.

Most importantly we have to consider the first point. We basically have to make sure that the objects of interest are large enough compared to the overall picture size. If you pay close attention to the image URLs in the text file, you will note that we are actually not passing the raw footage to the AI service but instead cropping these images on the fly using a content delivery network (CDN). This way we can focus on the parts of the image that actually matter to us and ensure that a potential car object covers more than 5% of the image area. So, if you see poor results with an object detection AI service try cropping the image before or splitting it up into different parts.

By now, the AI service should have done it's job and you can run the following code cell to print out an example result as shown in Figure XX.


```
1 # Print a sample result
2 results[0].as_dict()
```

✓

```
{'metadata': {'format': 'Jpeg', 'height': 400, 'width': 900},
 'model_version': '2021-04-01',
 'objects': [{'confidence': 0.862,
  'object_property': 'car',
  'parent': {'confidence': 0.872,
  'object_property': 'Land vehicle',
  'parent': {'confidence': 0.872, 'object_property': 'Vehicle'}},
  'rectangle': {'h': 89, 'w': 107, 'x': 521, 'y': 185}},
 {'confidence': 0.729,
  'object_property': 'car',
  'parent': {'confidence': 0.744,
  'object_property': 'Land vehicle',
  'parent': {'confidence': 0.744, 'object_property': 'Vehicle'}},
  'rectangle': {'h': 135, 'w': 122, 'x': 336, 'y': 263}}],
 'request_id': 'a0248e40-98a8-4dd0-bd4e-0d86e3f82a1e'}
```

As you can see, the AI service does not only provide us with the names of the detected objects but also with a little more context, such as the position of the object in the image and some additional semantic information.

Let's move on to section 3 - data transformations where we want to parse these results to get the actual count of vehicles in an image.

The approach here is rather simple. We are calling a function called `count_objects(result, "Car", 0.7)` that is filtering the outputs from the result according to two criteria: the object property name and the confidence score of the detection. I choose 0.7 which translates to something like “the AI service is 70% sure that the detected object is a car”. Feel free to experiment with the value and find out where the sweet spot is for your use case. Run this code cell and you will see a list with the counts.

The only task left is to put this into a nice table format which also includes the original image file URL to reference back to the data source and also an index which makes sorting the data easier. Execute the second code cell of section 3 and you should see an output similar to Figure XX.

```

1 # Create the final data frame for CSV export
2 output_df = pd.DataFrame({"Count": counts, "URL": urls})
3 output_df.to_csv("traffic_counts.csv", index = True)
4 output_df

```

	Count	URL
0	2	https://ucarecdn.com/de191530-3136-4b1d-818e-9...
1	2	https://ucarecdn.com/bb436a7e-b384-439b-857e-b...
2	2	https://ucarecdn.com/f3c419a2-5a31-48ef-9945-9...
3	1	https://ucarecdn.com/77deacb2-b5f4-439f-a1fe-b...
4	3	https://ucarecdn.com/cf8e7f6c-c172-4110-8c9f-8...
...	...	...
90	1	https://ucarecdn.com/0828690d-7c57-469b-85aa-9...
91	3	https://ucarecdn.com/5b2b1766-ac27-4081-a7e0-7...
92	2	https://ucarecdn.com/eb7efb73-5748-4061-a089-f...
93	4	https://ucarecdn.com/9af57430-1cba-4b7c-ad37-0...
94	3	https://ucarecdn.com/ccdc77ba-4946-43a6-a298-b...

95 rows × 2 columns

Finally, run the last code cell in the next section 4 to upload the CSV file to our Azure Blob storage.

And that's everything we had to do for our prototype ETL job. Let's move on to visualise the results in BI!

Visualising the results in BI

Since we have stored the AI results in a flat CSV, any BI tool can make use of this data. I will show you again how I have done it with Power BI - and there will be a little surprise!

First, create a new Power BI file and choose Get Data → Azure Blob Storage, provide your storage account name and select the container called "tables". Click "Transform" and click on the "Binary" link in the row where the new table name "traffic_counts.csv" is shown (see Figure XX). If you can't see this file, refresh the preview in Power Query.

	Content	A ^B C Name	A ^B C Extension	Date accessed
1	Binary	public-transportation-costs.csv	.csv	
2	Binary	reviews_negative_targets.csv	.csv	
3	Binary	reviews_positive_targets.csv	.csv	
4	Binary	reviews_sentiment_scores.csv	.csv	
5	Binary	traffic_counts.csv	.csv	

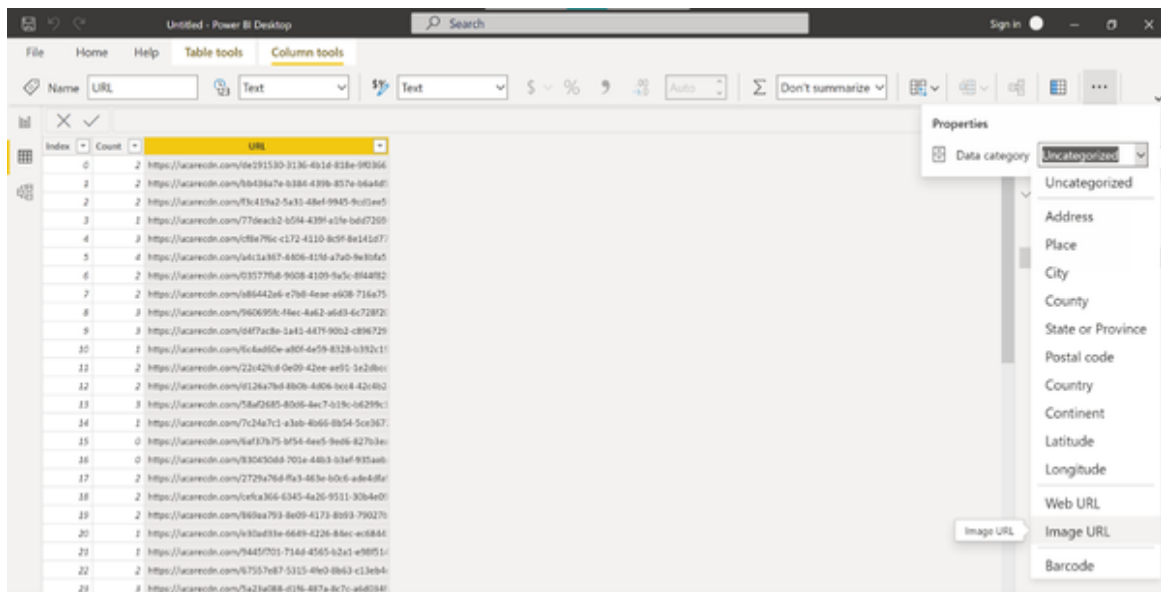
In the Power Query editor there is actually not much for us to do here. Just give the index column a name by right-clicking the column name and choosing “Rename” from the context menu. Rename the column to “Index” as shown in Figure XX.

	1 ² 3	A ^B C
1		2 htt
2		2 htt
3		2 htt
4		1 htt
5		3 htt
6		4 htt
7		2 htt
8		2 htt
9		3 htt
10		3 htt
11	10	1 htt

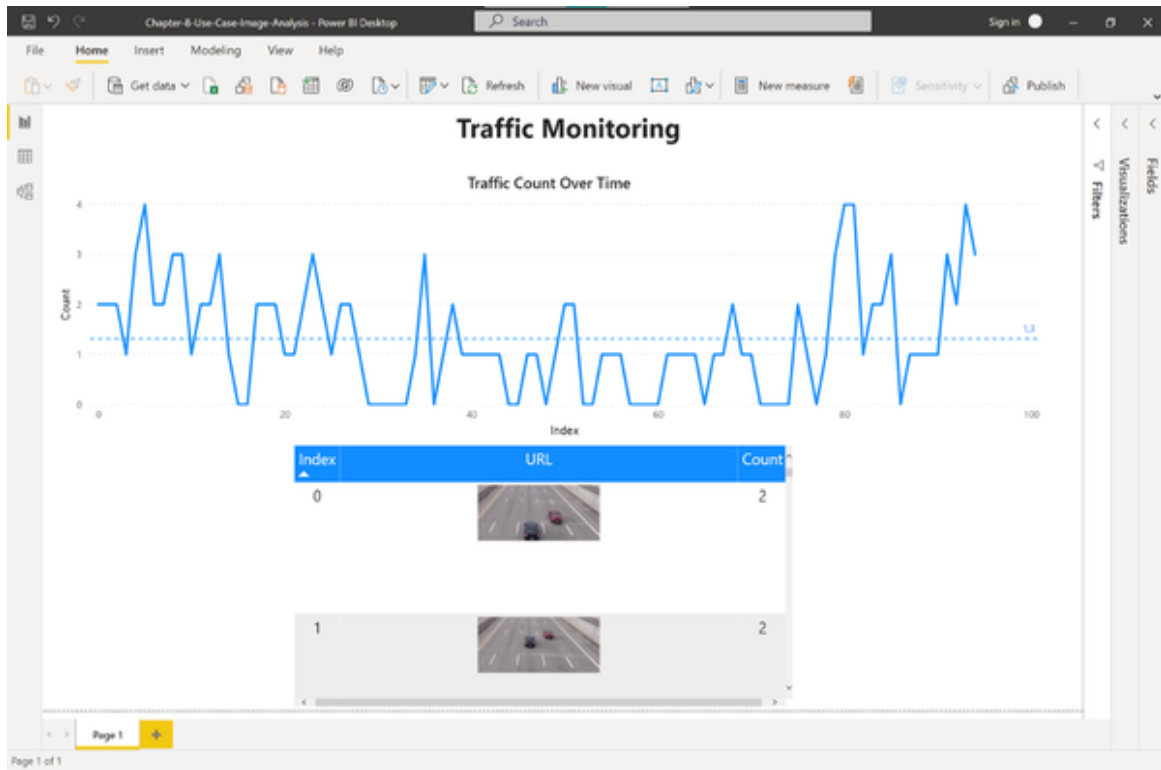
Double-check that all three columns are in the table: the index (numeric), the count (numeric) and the file URL (string). Confirm the query by clicking “Close & Apply”.

Before we head over to build the report, let's include one tiny but powerful surprise that Power BI is offering for us. Click the small table icon that is located between the data model and the report icon on the left side as shown in Figure XX

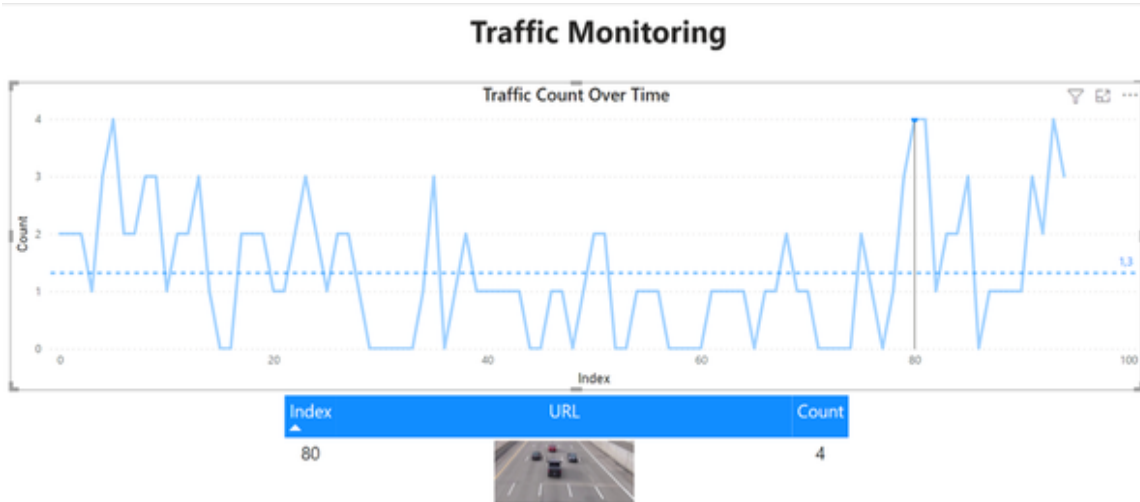
In this window, select the URL column and choose “Column” tools from the menu pane on the top. Find the field “Data category” which is a bit hidden in the top menu and change the datatype from text to “Image URL” as shown in Figure XX



Now, let's move on to the report page. In my case, I chose only two components: A line chart for the count over time (in our case that is the index) and a table showing the count, index and image URL. And because we have formatted the URL field as an “Image URL” datatype, Power BI will actually fetch the image from this URL and show it in our dashboard. Isn't this marvellous? Take a look at Figure XX to see the final dashboard in action.



Of course, this table inherits the full interactivity that you expect from a Power BI visual. So when you select for example one point with the highest traffic count on index 80, the table will actually show the relevant image for this data point as Figure XX shows. This way we can also ensure that the AI detection was actually correct by spotting 4 cars on the image.



Feel free to recreate this dashboard by yourself or modify it as you like.

You can find the final version also in the book repo by downloading the file “Chapter-8-Use-Case-Image-Analysis.ipynb”

Conclusion

This chapter has only touched the surface of what AI services can do with data sources that are not the regular CSV or Excel format. I hope that you had fun trying out these AI services and I wish even more that you got more inspiration on how to use these tools for your own use cases.

This chapter concluded the building blocks of AI services that can empower your BI. In the next chapter we are not going to introduce new tools, but we will take a look at how we can combine what we learned to build a fully-function, integrated AI-powered BI dashboard.

Chapter 10. Bringing It All Together: Building an AI-Powered Customer Analytics Dashboard

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 10th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

You have come a long way in this book and (hopefully!) learned a lot of new things about how AI services can be used at different levels of the analytics stack. In this chapter I want to show you how these different layers of analytics and AI services can be combined together to deliver an improved experience for BI users and leverage the potential of AI and BI by blending the different approaches.

Problem Statement

In this scenario, we are part of the data analytics team of a telecommunications provider. The head of sales and marketing of the consumers’ division has initiated a project that should look further into the

topic of customer churn. As per the businesses' definition, churn happens the moment when a customer cancels their contract, no matter how long the remaining contract duration is (the business is offering monthly and 24-months contracts).

The business is currently facing the following challenges:

- Churn rates are measured, but marketing and sales can't yet make sense of it. The churn metrics seem to go up and down sporadically and they struggle to get any meaningful insights from these metrics.
- As an effective measure to counter churn, the business has identified counter-offers to be a viable strategy. Offers are presented to customers who are quitting their contract as a means to win them back or prevent them from quitting at all. The offers turned out to be effective, but costly. Therefore, the business wants to know which customers are most lucrative to be targeted by such counter-churn offers and ideally like to predict churn before it is happening to ensure a good fit between the offer type and the customer segments.
- The business is running regular surveys among customers which includes a lot of open-text feedback. Looking at some samples of the survey data, the survey team suggests that the text answers could provide important signals for churn modeling.
- The business expects a certain level of interactivity to comb through the data in order to get a feeling for what's happening with regards to customer churn in different customer segments.

We are now part of the project team and are expected to identify viable ways to present the requested information to the business stakeholders - of course as soon as possible and with heavy budget constraints.

Solution Overview

To tackle the problem above, we will use a combination of our existing BI data (which could come from a data warehouse, but in our case is delivered

as CSV files).

The BI data contains both dimensional information about customers such as demographics, services, payment methods, etc. as well as fact tables with information such as customer tenure, monthly charges and contract types. The fact tables we have been provided stretch over the past 5 months and already include a churn flag which has been calculated by our in-house analysts. Besides the churn label, the fact tables also contain a flag whether customers accepted a counter-churn or not in the past.

With regards to the survey data, the team has sent us a CSV dump of the latest survey results from the past months, including the open-end text answers and corresponding customer IDs.

With this data we will try to build a prototype for a BI dashboard that provides the information as requested by sales and marketing. At the heart of the dashboard, we will use a predictive model which calculates the individual churn risk for each customer based on historical data.

To make sense of the open-end text answers we will run a sentiment analysis on the customer feedback and see if these sentiment labels will help us increase the performance of the predictive model. To try this out we will build two models - one which includes the sentiment data and one which doesn't so we can compare the results.

Instead of handling the workflow for this manually, we will use a software called Azure ML Designer which comes as part of our Azure subscription. The ML Designer is a no-code AI platform which allows us to build and deploy managed machine learning workflows in the Azure cloud with as few technical friction as possible.

Finally, we will pull the results together in our BI tool. In our case, that's Power BI again, but we will try to do the heavy lifting outside of Power BI as much as possible to be compatible with other BI platforms as well.

Preparing the Datasets

Download the following files from the book's repository:

- customers_factTable_Jan-May_2022.csv - The file contains information about customer metrics and churn labels for January - May 2022.
- survey_responses_Jan-May_2022.csv - A customer survey across a sample of customers conducted between January and May 2022 containing customer feedback.
- customers_factTable_June_2022.csv - A file containing customer facts for the current month of June 2022
- survey_responses_June_2022.csv - New survey data from June 2022.

Open **Microsoft Azure Machine Learning Studio**, choose your preferred workspace and you should see your dashboard as shown in Figure XX.

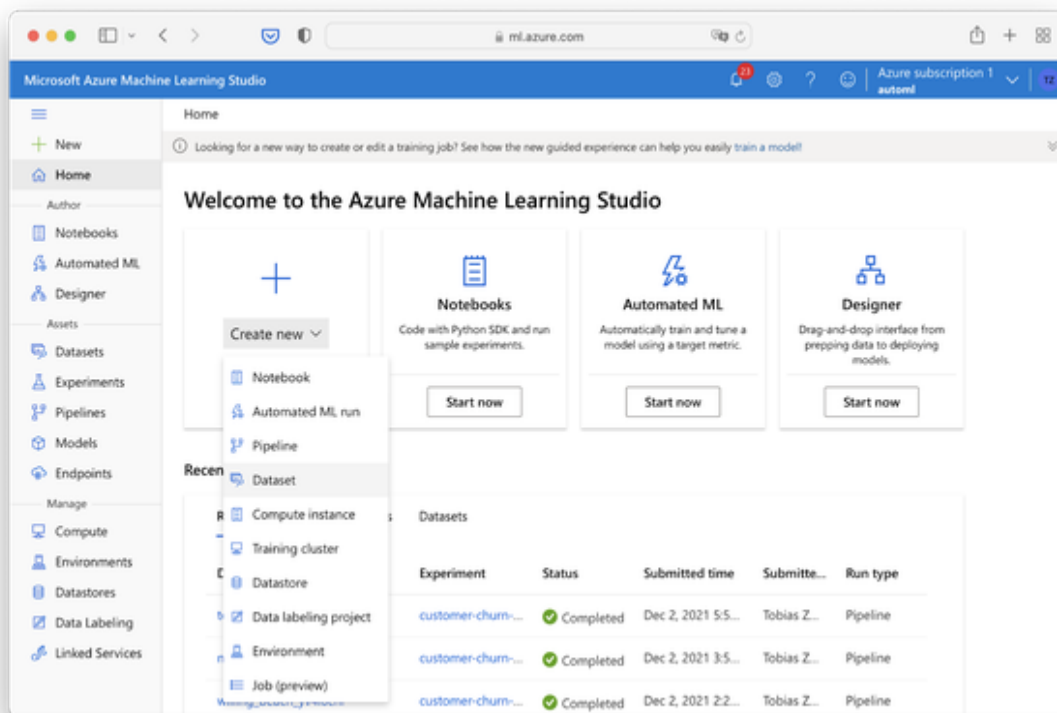


Figure 10-1. Azure Machine Learning Studio Dashboard

First, we will upload all four CSV files as Datasets into Azure ML Studio one by one in order to be able to access them through the ML Designer later.

Click the “+”-Icon and choose Datasets or select Datasets from the menu on the left.

For each of the CSV files, select “Create dataset” → from local files.

Provide the names for the dataset as in the filename, for example

“customers_factTable_Jan-May_2022” for the file

customers_factTable_Jan-May_2022.csv. Follow along through the form, upload the csv files and check the data preview if everything looks fine.

The delimiter in all files should be Comma and the encoding is UTF-8.

When it comes to the schema, you have to make some adjustments.

Please make sure that the schema is as follows for the customer fact tables (Note: The columns Churn and OfferAccepted don’t appear in the file from June as this represents the most current month and the churn info isn’t available yet - that’s what we are going to predict in a bit):

- customerID: String
- tenure: Integer
- Contract: String
- Monthly Charges: Decimal (dot)
- Churn: String
- Don’t include: OfferAccepted
- Don’t include: Month

For both survey files, make sure the columns match the following schema:

- id: String
- text: String

By the end of this, you should see the following four datasets within Machine Learning Studio:

- customers_factTable_Jan-May_2022
- survey_responses_Jan-May_2022
- customers_factTable_June_2022
- survey_responses_June_2022

Allocating a Compute Resource

Now that we have the data in place, we need to make sure that there are some resources available where the actual computations can happen. To add a compute resource, select Compute from the menu on the left of the Azure Machine Learning Studio as shown in Figure XX.

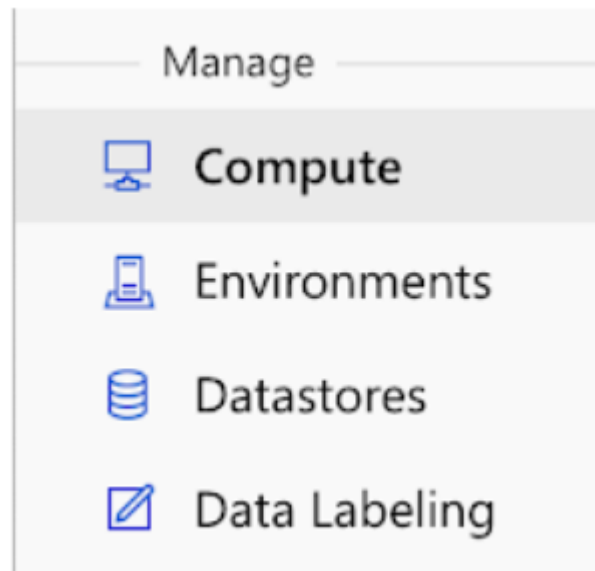


Figure 10-2. Compute Resource in Azure ML Studio

Here, you should still see the compute resource which you used previously throughout the chapters. If you see the resource, but it hasn't been started yet, select the resource and click Start. If you don't have any resource yet or you deleted it previously, click New and create a new compute resource with the default settings and the cheapest machine type

(STANDARD_DS1_V2) which will be sufficient for this case study. Once you have at least one compute instance up and running you can proceed to the ML Designer.

Building the ML Workflow

Go ahead by choosing Designer from the menu on the left as seen in Figure XX. The ML Studio Designer will be our drag-and-drop interface to build so-called machine learning pipelines. We can train our own models, but we can also do some basic data preprocessing and run custom scripts.

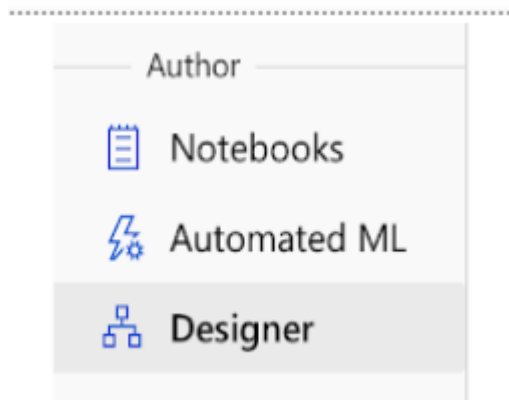


Figure 10-3. ML Designer in Azure ML Studio

Create your first pipeline by clicking “New pipeline”. You should now see the empty canvas for the ML Studio Designer as shown in Figure XX. The ML Studio designer has roughly main components: The asset library on the left which contains pre-built modules, a canvas on the right to build your pipeline and a toolbar on the top to Save, Edit and Publish the entire pipeline.

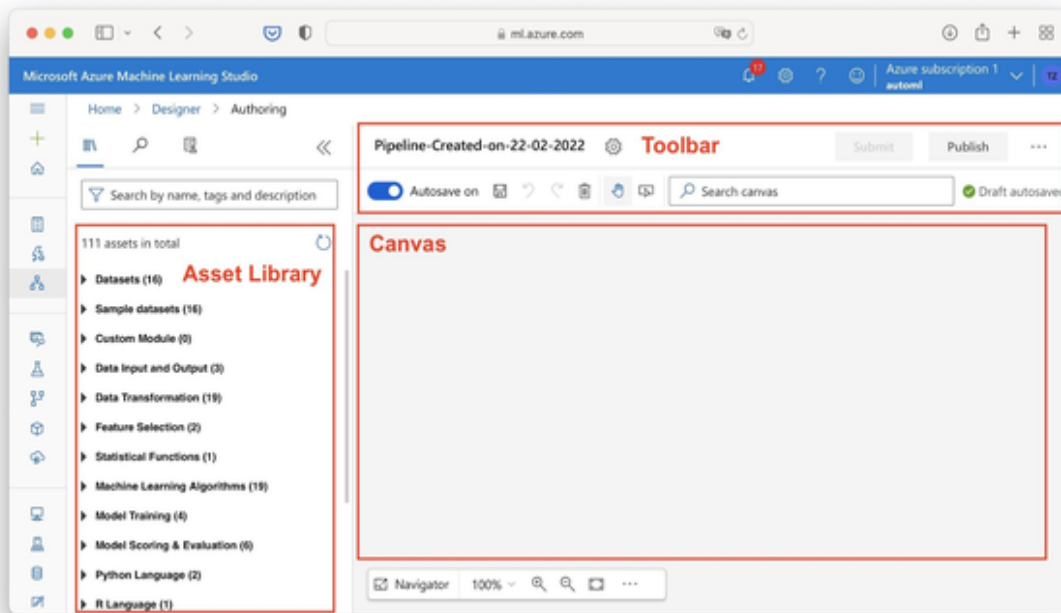


Figure 10-4. Azure ML Designer Interface

Before we can start, assign the running compute instance to our session. If the settings window did not pop up automatically, you can access it by clicking the cog icon next to the pipeline name in the toolbar. Talking about pipeline names, double click the pipeline title and rename it to “Customer-Churn-Predictor”.

We can now start to populate the empty canvas. The idea is that you build a custom workflow using the pre-built modules on the left and combining them on the canvas. Each module can have input and output ports and provides different settings.

Choose Datasets from the toolbar on the left which contains the assets library and select the dataset “customers_factTable_Jan-May_2022”. Drag and drop it onto the canvas as shown in Figure XX.

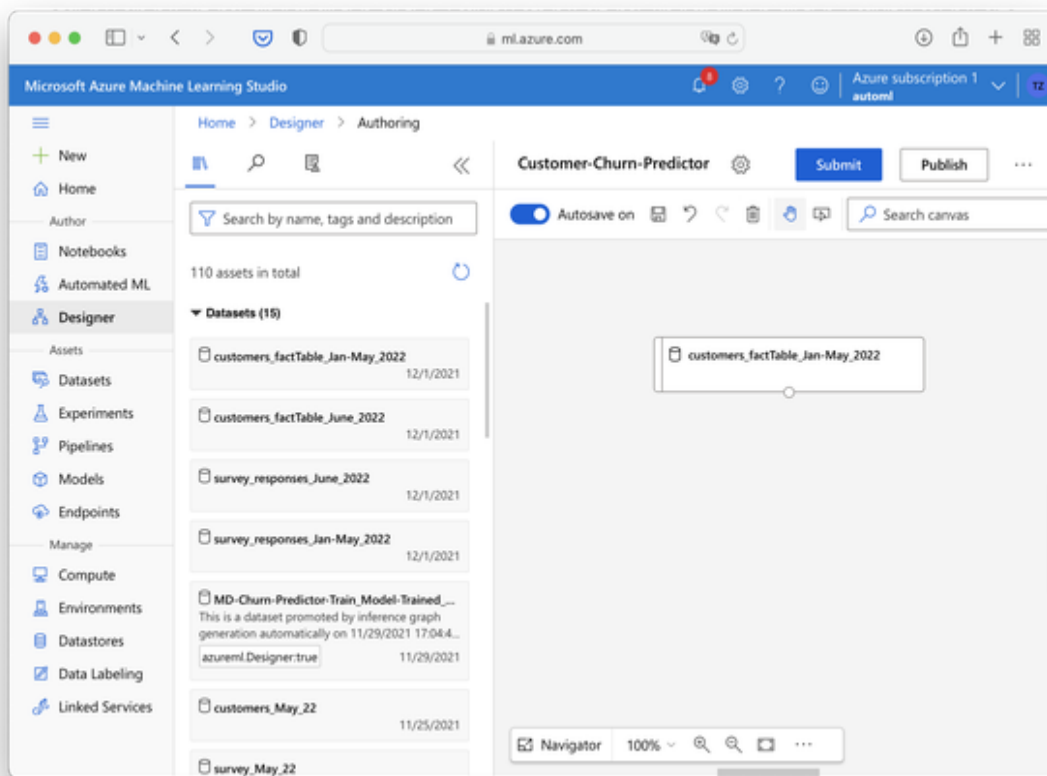


Figure 10-5. Adding a Dataset to the Canvas

The pipelines will be executed from top to bottom. So everything will start by loading this dataset. If you like, you can preview the data by doing a right-click on the module and select “Preview data”. Previewing outputs of individual modules on the canvas is an intuitive way for debugging, especially at the beginning when you are just familiarizing yourself with the interface.

Next, let’s do something with the data. Our goal is to build a very simple machine learning pipeline that will predict the “Churn” column given our input features, similar to the Auto ML use case we have seen in Chapter X.

Using the search bar in the module pane on the left search for the “Select Columns in Dataset”. The purpose of this module is to get rid of the columns we don’t need for the predictions, such as the Customer ID. Drag the module onto the canvas below the Dataset module. Close the settings for now, if they pop up automatically. Connect the output port from the

Dataset module to the Select Columns in Dataset module as seen in Figure XX.

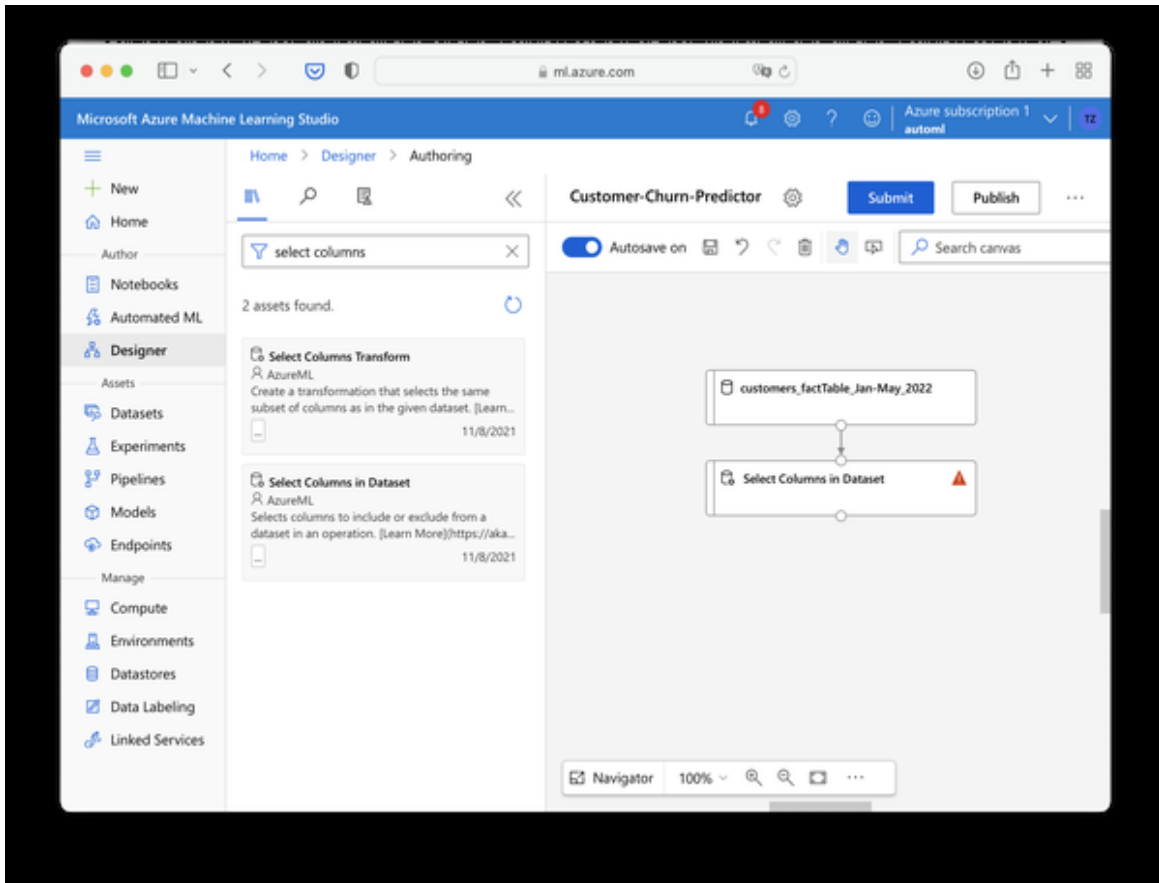


Figure 10-6. Adding module *Select Columns in Dataset*

Drawing a connection between two modules allows for metadata to flow downstream in our pipeline. That means our “Select Columns in Dataset” module will now know the available column names. Click the module on the canvas to open the settings once again. Click “Edit columns” and choose “Select columns by name” as seen in Figure XX.

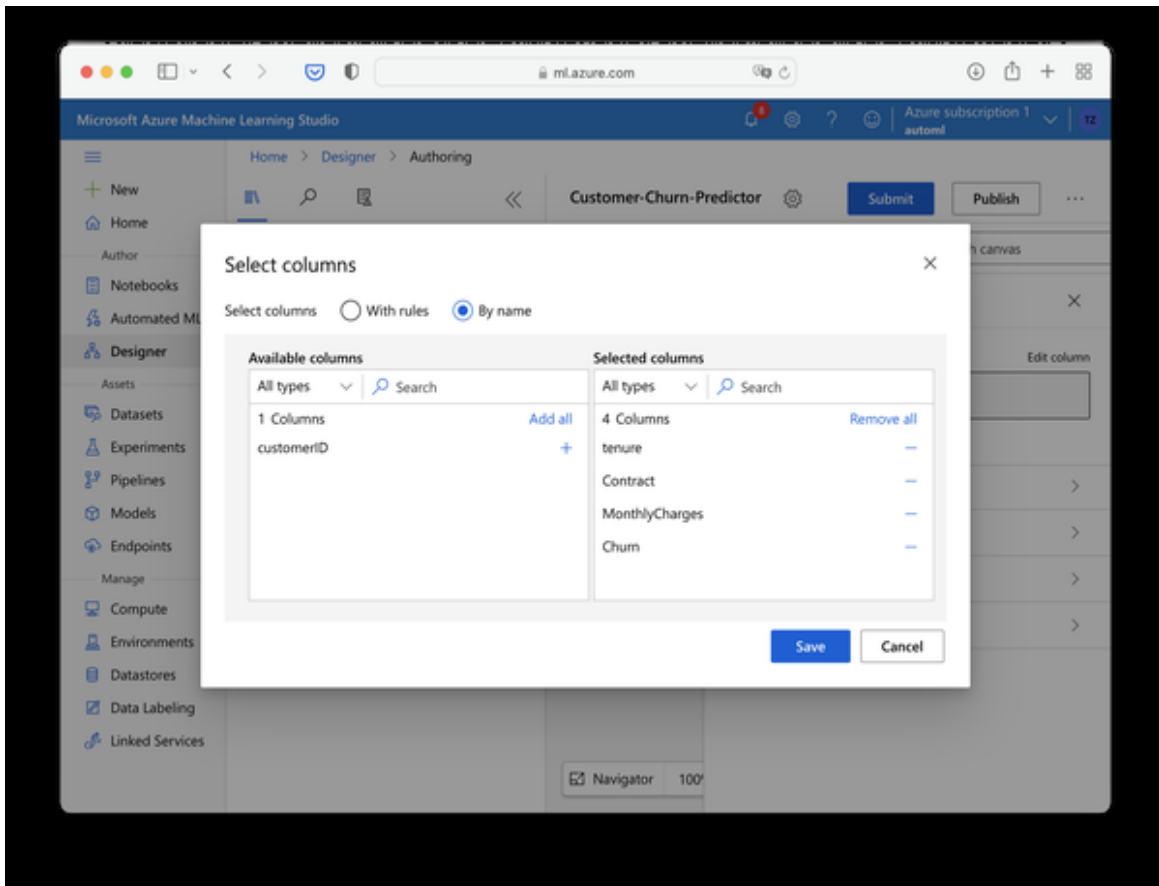


Figure 10-7. Filtering columns

Select all but the CustomerID column. Confirm by clicking “Save”. Close the module’s settings pane to make some more space for the canvas.

Next, we want to split our data into a training and a test set. Search for “Split Data” in the modules search bar and drag the module onto the canvas. In the module settings, define the following split parameters:

- Splitting mode: Split Rows
- Fractions of rows in the first output: 0.75
- Randomized split: True
- Random seed: 1234 (for reproducibility)
- Stratified split: True
- Stratification key columns: Churn

With these settings we are doing a stratified train-test-split with 75% of the data for training and 25% for testing. Stratified means that the class distribution for the Churn label will be the same for both the training and test set.

Close the settings and connect the output port of the “Select Columns in Dataset” module to the input port of the “Split Data” module. As you can see in Figure XX the Split Data module has two outputs now, one for the training (left) and one for the test dataset (right).

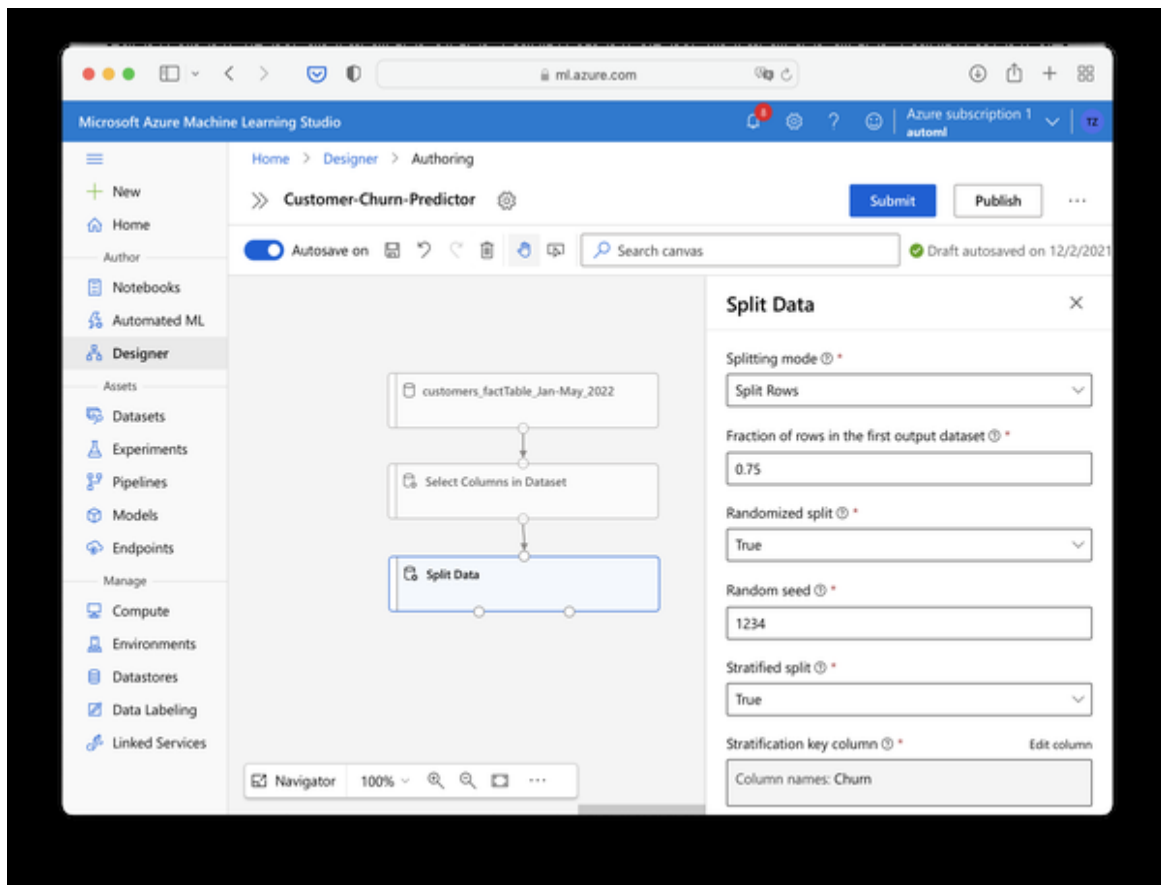


Figure 10-8. Adding a Split Data component

We can now add the Train Model component. Search for it in the module pane and drag it onto the canvas. You will realize that this module has two inputs: The left one expects the training algorithm and the right one the training dataset. Connect the left output node of the Split Data module to the right input port of the Train Model module. We still need to tell the Train Model component which column we actually want to predict. So click

on the component to open the settings and provide “Churn” as the Label column.

Now it’s up to us to provide the actual training algorithm. In contrast to Auto ML we have to decide for the training algorithm ourselves. Now, this isn’t a Machine Learning Fundamentals book, but in most supervised learning scenarios you will achieve pretty good results by choosing an ensemble method such as Decision Forest or Boosted Trees as a first baseline. These models try to combine several weak learners such as decision trees to a strong learning algorithm that tries to minimize the prediction error on your training dataset. These models work for both classification and regression problems, so they are good all-round algorithms, although they are rather complex. The good thing about the ML Designer is that you can easily try out different, even simpler algorithms such as linear or logistic regression, and see how they perform on your dataset.

For our exercise, search for the “Two-Class Boosted Decision Tree” from the module pane and drag it over to the canvas. We can leave all it’s settings to default for now. Connect the Decision Tree module to the Train Model component and your pipeline should look like in Figure XX.

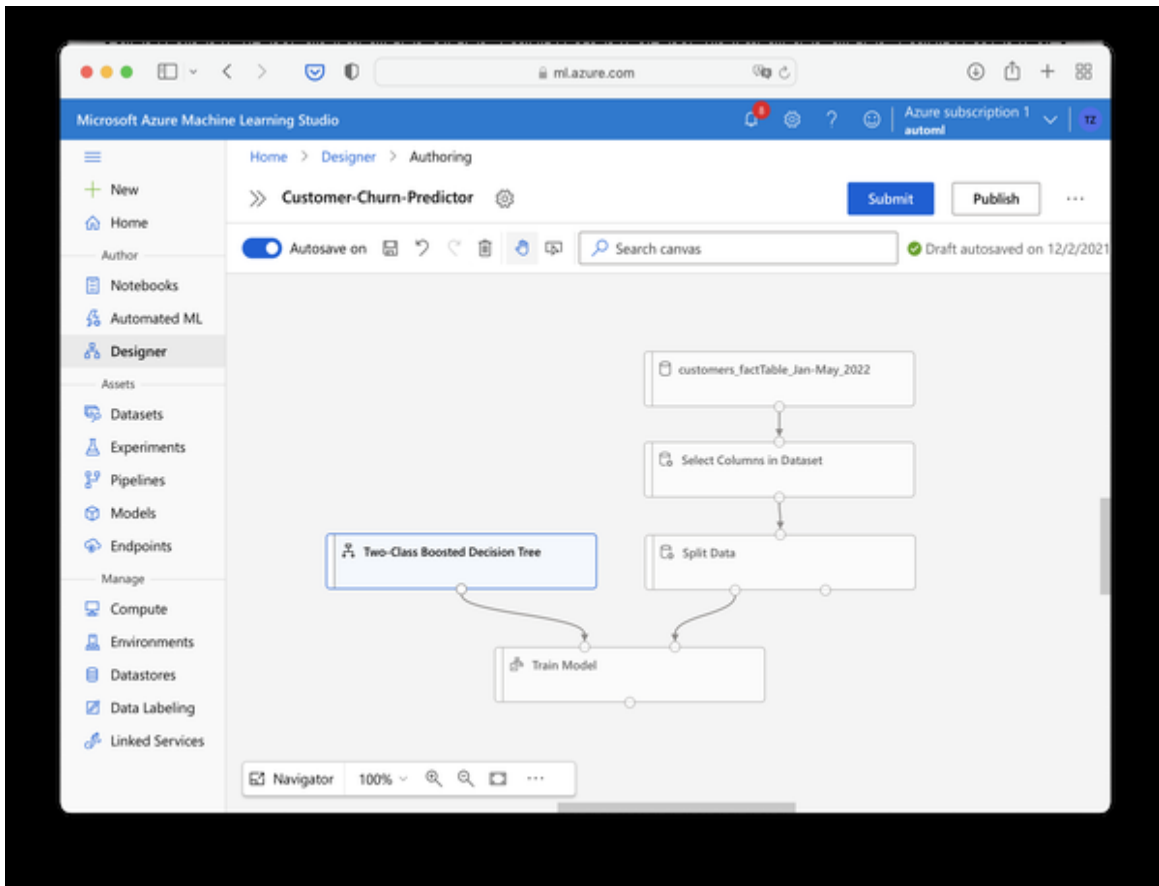


Figure 10-9. Adding the Model and Train Model Component

We're almost there! There's only one thing missing and that is adding a component that scores some actual predictions based on the trained model and calculates some evaluation metrics for it.

Look for the "Score Model" component and drag it over to the canvas. Connect the output port from the Train Model component to the left input port of the Score Model component.

Then connect the right output port from the Split Data component to the right input port of the Score Model component. This will allow us to calculate predictions for the test dataset using our model trained on the training dataset. Finally, find the Evaluate Model module, pull it over to the canvas and connect the Score Model output port to the left input port of the Evaluate Model component. Your final pipeline should now look similar to Figure XX.

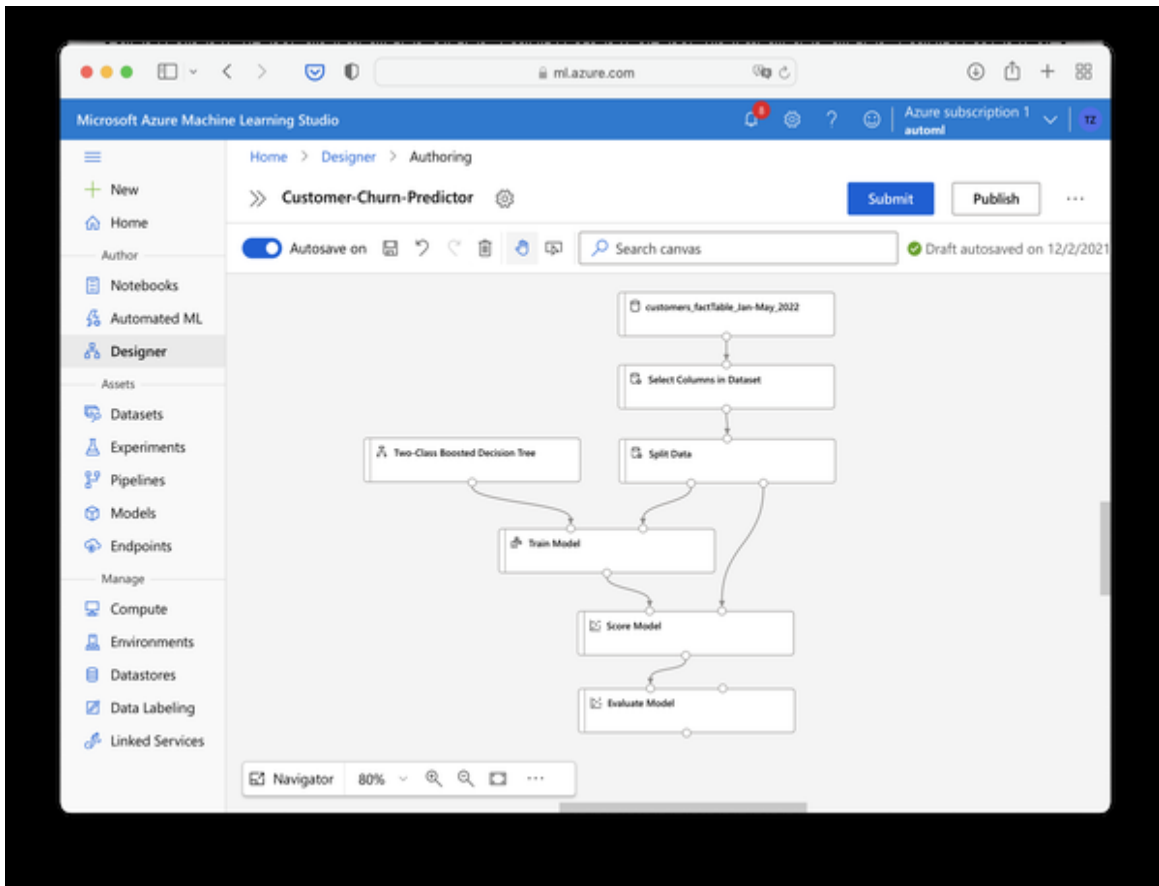


Figure 10-10. First End-to-end Workflow in ML Designer

Congratulations! You have just built your first Machine Learning pipeline from scratch. Press the “Submit” button to run the pipeline. All runs will be gathered in so-called experiments as we are used to from the AutoML scenario. I suggest creating a new experiment for this pipeline so you keep your work organized. Create a new experiment and provide a telling name such as “customer-churn-prediction-training”. Hit Submit and lean back while Azure ML Studio is doing it’s magic behind the scenes.

You can follow along with the training process as the Designer will indicate which step is currently running and which step is completed. Even if the process fails for some reason or gets interrupted, the steps that have been successfully executed up to this point will be stored. As long as you don’t modify any preceding module in the pipeline, the output of the pipelines will be persisted. The whole run should take approximately 10 - 15 mins. The reason why this is taking so long is that Azure is actually spinning up

separate processes for each module which creates some overhead. The advantage is that this interface will also work well for really large datasets because each module runs independently from each other. That is why running the designer for some rather small datasets as in our case will actually take longer compared to running them on your local notebook. But the larger the dataset gets, the more performance advantages you will see.

When the process has finished you should see the “Completed” badge on the last Evaluate Model module. In this case, right-click the module and choose “Preview Data → Evaluation resultss” from the context menu. This should give you the outputs as seen in Figure XX.

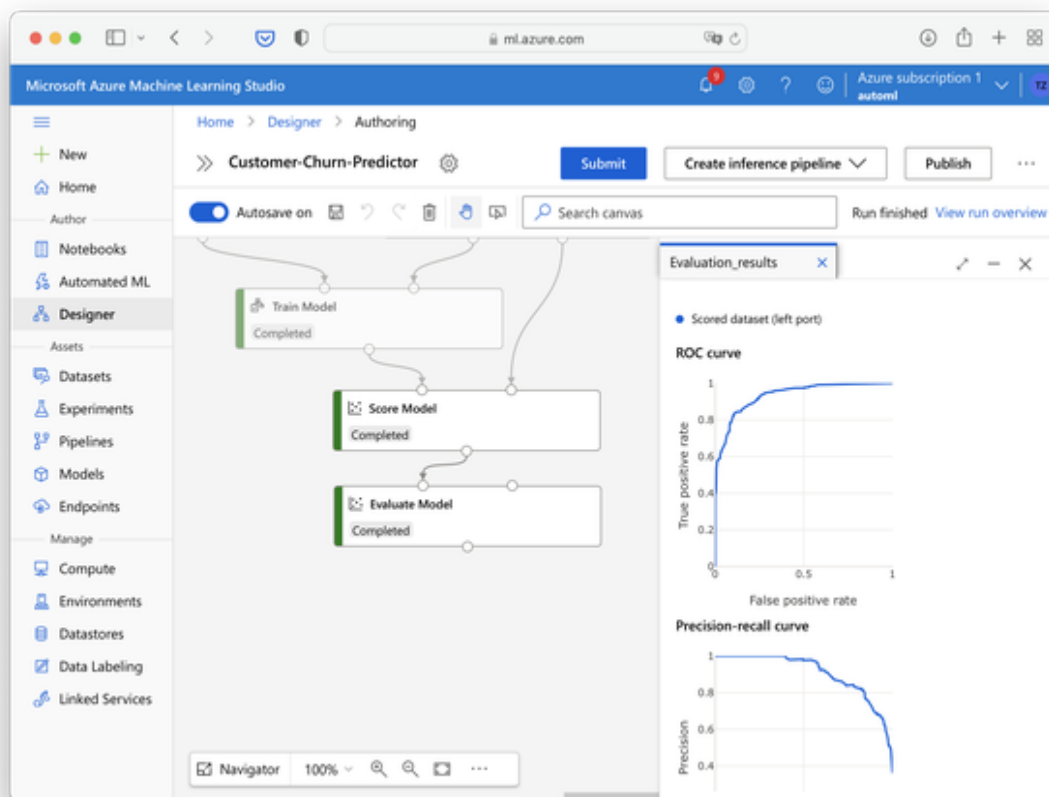


Figure 10-11. Evaluation metrics for first workflow

Click the Enlarge icon to make some more space and the whole interface should look very familiar to what you have seen in the previous chapters. With our initial run we have reached an Accuracy of 86.6% and an F1-

Score of 80.9%. That's not bad for a first try, but let's see if we can improve this metric even more by blending our training dataset with some more data.

Close the evaluation metrics and go back to the canvas. This time we want to add the analysis of the customer feedback and see if that helps to improve our customer churn predictions. The way we approach this is that we build a second pipeline next to the one we already have and feed the results into the right input port of the Evaluate Model module. This will give us a direct comparison of both modeling approaches.

Let's tackle this step by step. First, drag and drop the Dataset module `customers_factTable_Jan-May_2022` from the module pane on the left to the canvas. In addition, locate the Dataset module for `survey_responses_Jan-May_2022` and drag them next to it. Your canvas should now look like in Figure XX.

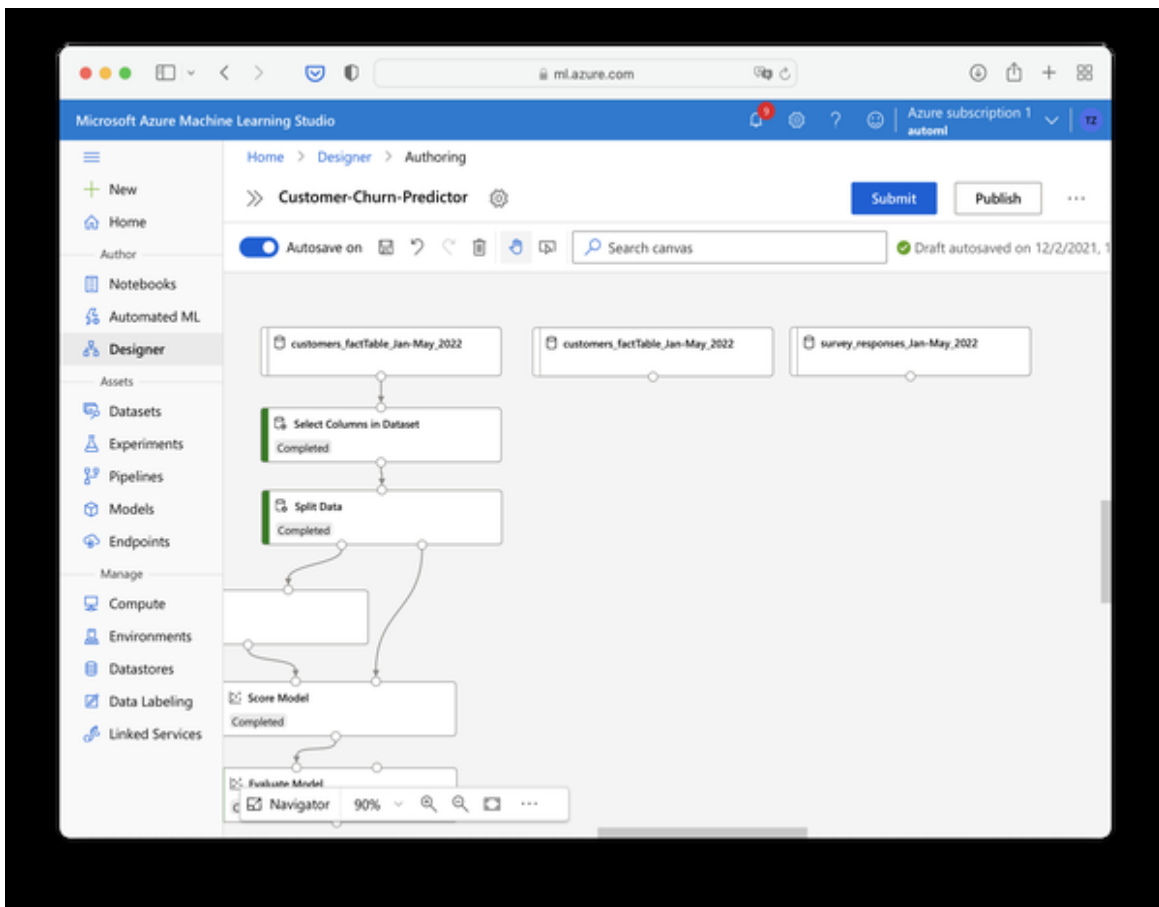


Figure 10-12. Adding more data sources to ML designer

To get more familiar with the survey data, do a right-click on the module and select “Preview data”. The dataset contains only two columns, the customer id and the text response, both values that came from the original customer survey results.

So far, our machine learning model can’t handle the raw text inputs very well as predictive features. To make them more accessible will read the open-end text answers from the survey dataset, send them to Azure Cognitive Service and get the sentiment value for each text. The classification “negative” or “positive” will then become an additional feature for our classification model. For this purpose we will use the Azure Cognitive Services for Sentiment Analysis which we have deployed in Chapter 8.

Let’s bring this AI service component to the canvas! Believe it or not, but (at least at the time of this writing) there is no prebuilt module for Azure Cognitive services which you can just drag onto the canvas in the Azure ML Designer. But what we can do is simply add a module that runs some Python or R code for us. And that is just the approach we are going to take: We will take the responses from the survey, send them to a remote AI service API using Python or R and feed the results back into the ML designer pipeline.

To start, search for the “Execute Python Script” and drag it over to the canvas. Connect the output port of the survey_responses Dataset to the first input port of the Execute Python Script module

The way the script module works is that it expects up to two dataframes (tables) which can be then modified within a function called `azureml_main`. The result will again be returned as one or two dataframes. The underlying Python runtime comes with a limited number of packages. To find more about it, check the resource [Execute Python Script in the designer - Azure Machine Learning](#). All we need to do now is to replace the pre-build demo code with our custom code for the task we want to do.

Open the file `ml-designer.py` from the book’s repository in a text editor. Replace the key and endpoint with your custom parameters as seen in

Chapter 8. Now select all of the script, copy. Replace all of the sample code of the Execute Python Script module in the Azure ML Designer with the contents of your clipboard. The script module should now only contain the code from the ml-designer.py file and nothing else as seen in Figure XX.

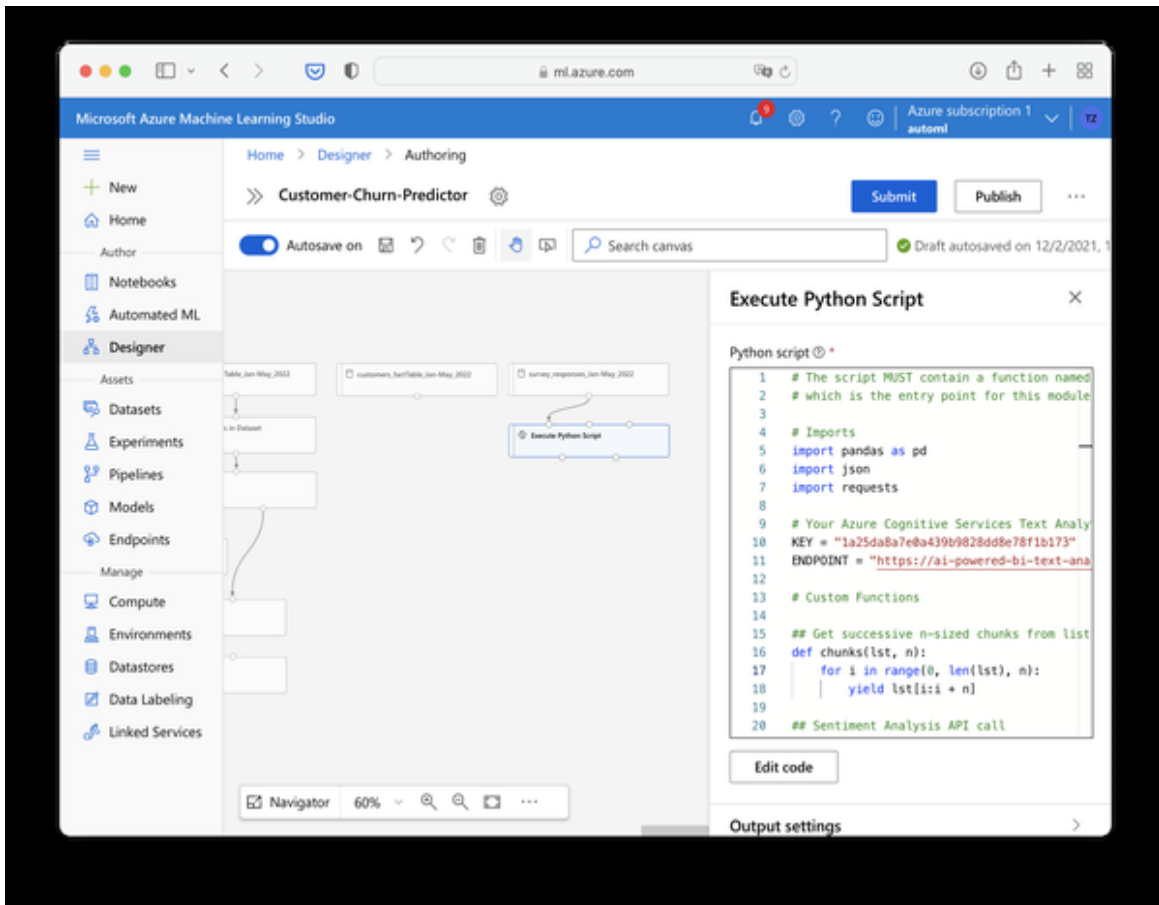


Figure 10-13. Python scripting module in Azure ML designer

The code will basically read the data frame as an input, bring it to a format that the API service can handle. It will return the original dataframe with the sentiment predictions attached.

Now, click “Submit” to run the new part of this pipeline and verify everything is running as expected.

Note that all steps we ran previously on the left side of the graph will not be re-executed since we didn’t change anything here. Instead, this run will only read in the new dataset and feed it into the Script module. After some minutes, you should see that the script was executed successfully. If an

error comes up, click the Script module and take a look at the log files. The error message there will help you to debug, for example if your resource key is not valid or your free quota has been exceeded.

When the run has been completed, right click on the Script module and choose “Preview data → Result dataset”. As shown in Figure XX, you should see that each customer feedback has now gotten a sentiment label.

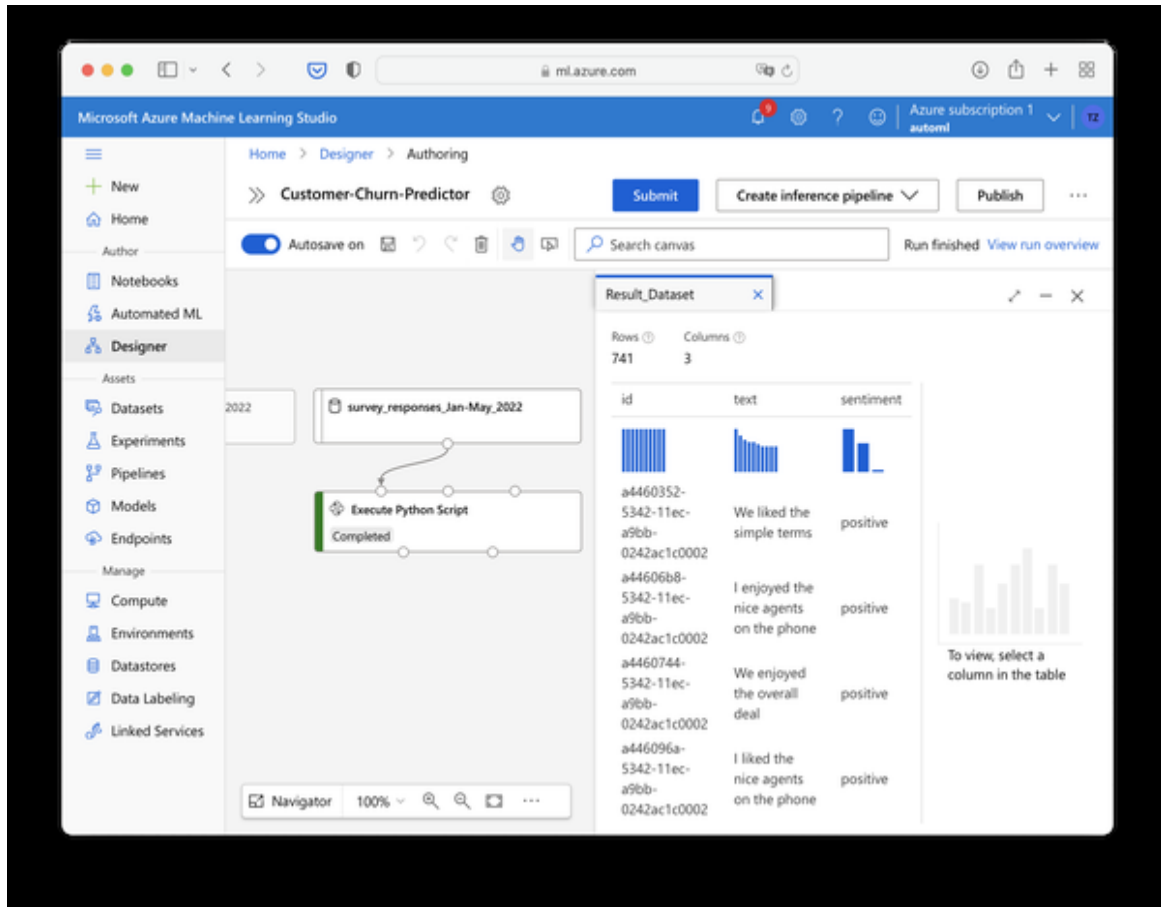


Figure 10-14. Sentiment analysis results in Azure ML Designer

Our goal is now to add this information to our main dataset that is used for training the churn model. What we need to do is to join this table to the main table without losing any records in the main table - remember that this survey wasn't done for all customers but only for a sample. This process is called a Full Outer Left Join where the left table is our main dataset and the right table is the dataset from where we want to join additional information.

Find the corresponding module by searching for “Join Data” in the asset library.

Drag the Join Data module to the canvas. Connect the first output port of the previous scripting module to the right input port of the Join Data module. Then connect the output port of customers_factTable_Jan-May_2022 (the one we didn’t use yet) to the left input port of the Join Data module. Select the Join Data module on the canvas to bring up its settings. We have to define the key columns on which both datasets will be merged. You can edit the key columns by clicking “Edit column” for both the left and the right dataset. For the left dataset, choose the column customerID and for the right dataset the column “id”. Set the join type to Left Outer Join and set “Keep right key columns in joined table” to False. Your settings should ultimately look like as shown in Figure XX.

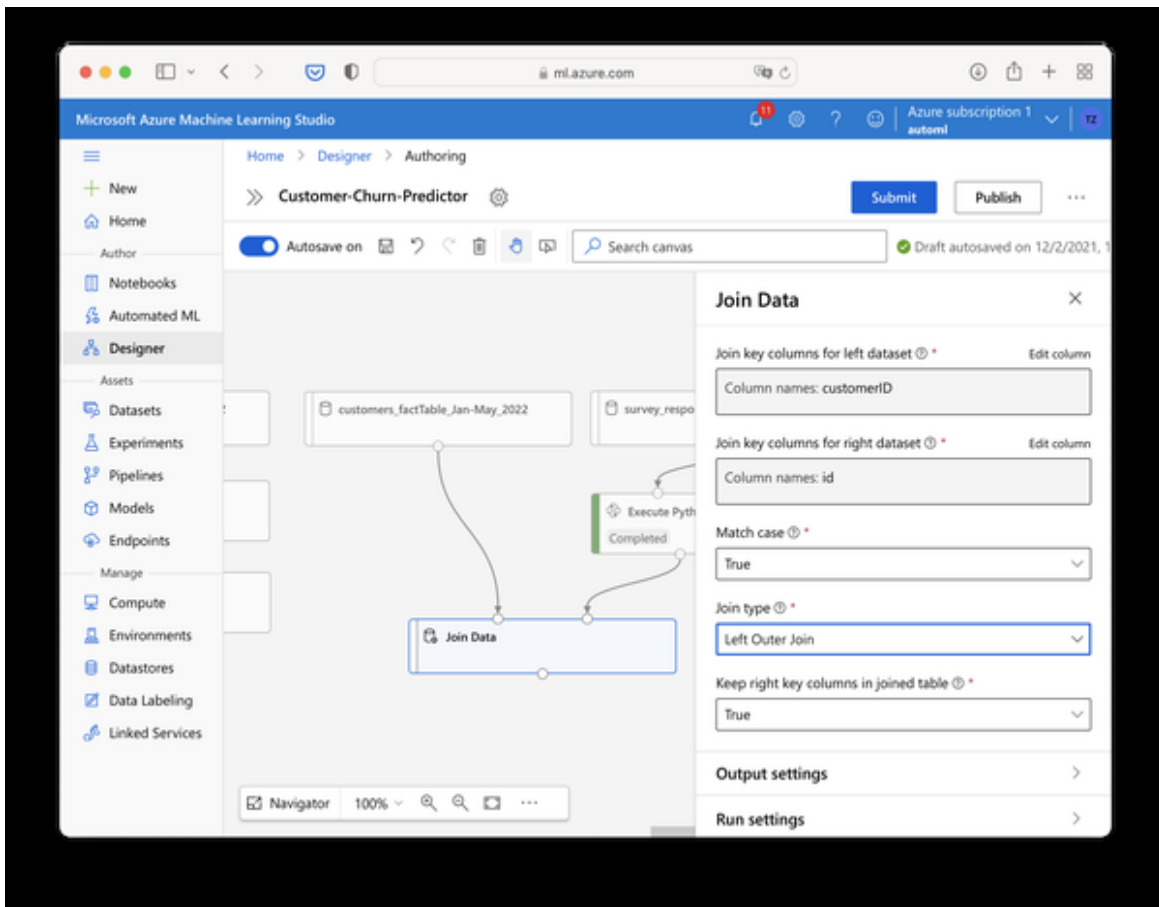


Figure 10-15. Joining data from different sources

Run the pipeline by hitting submit to verify that the join works as expected. Your merged dataset should have 7 columns and 3,814 rows which you can verify with a right-click on the Join Data module → Preview data → results dataset after the run has been completed.

We now pretty much have to recreate the rest of the pipeline as we did previously. I won't go through these steps in detail as these are repetitive. In brief, add the following modules and apply these settings:

- Select Columns in Dataset
 - Remove the column “text”, keep the new column “sentiment”
- Split Data:
 - Same as before, 0.75 stratified sample on the column “Churn”
- Train Model
 - Label column is “Churn”
- Two-Class Boosted Decision Tree
 - Leave default settings
- Score Model

Connect the last output port from the Score Model component to the right input port of the Evaluate Model component we created before. The final pipeline is shown in Figure XX.

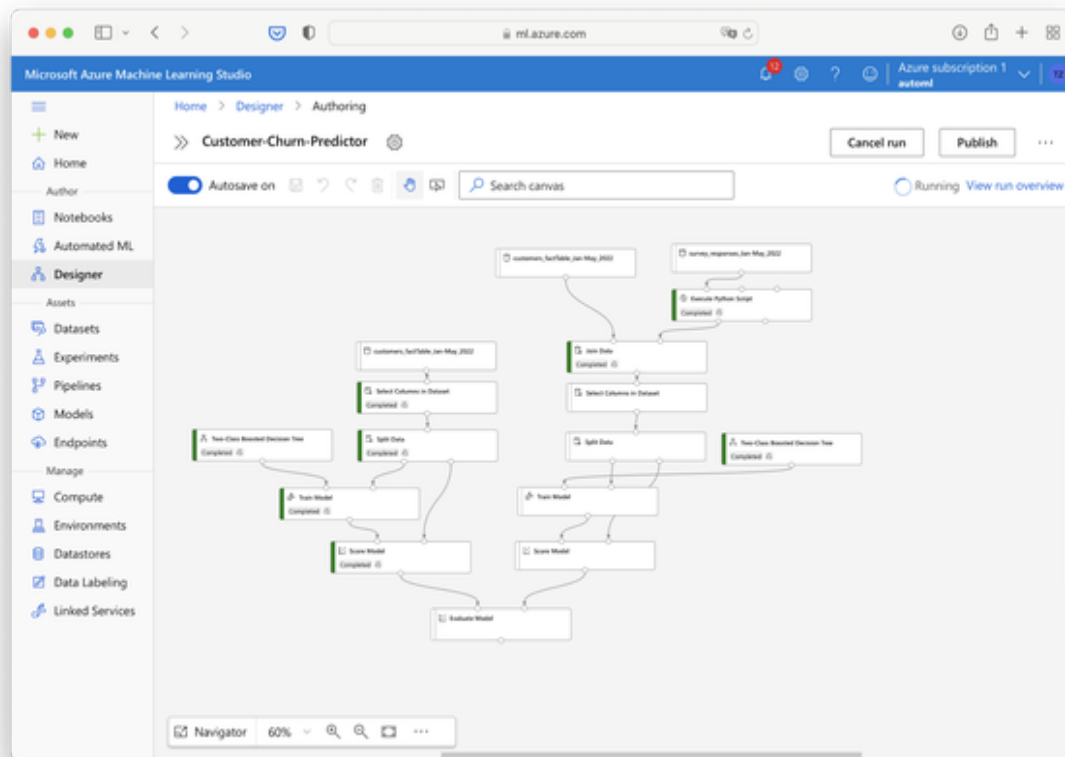


Figure 10-16. Second end-to-end workflow

Hit Submit to run the whole graph. If you think that it's time for another coffee then you're just right! The whole process might take a couple of minutes. Check back when the process has been completed.

When the run has been completed, let's check our evaluation module and see if the model performance could be improved. Right click the Evaluate Model module and choose Preview Data → Evaluation Results. The output is shown in Figure XX.

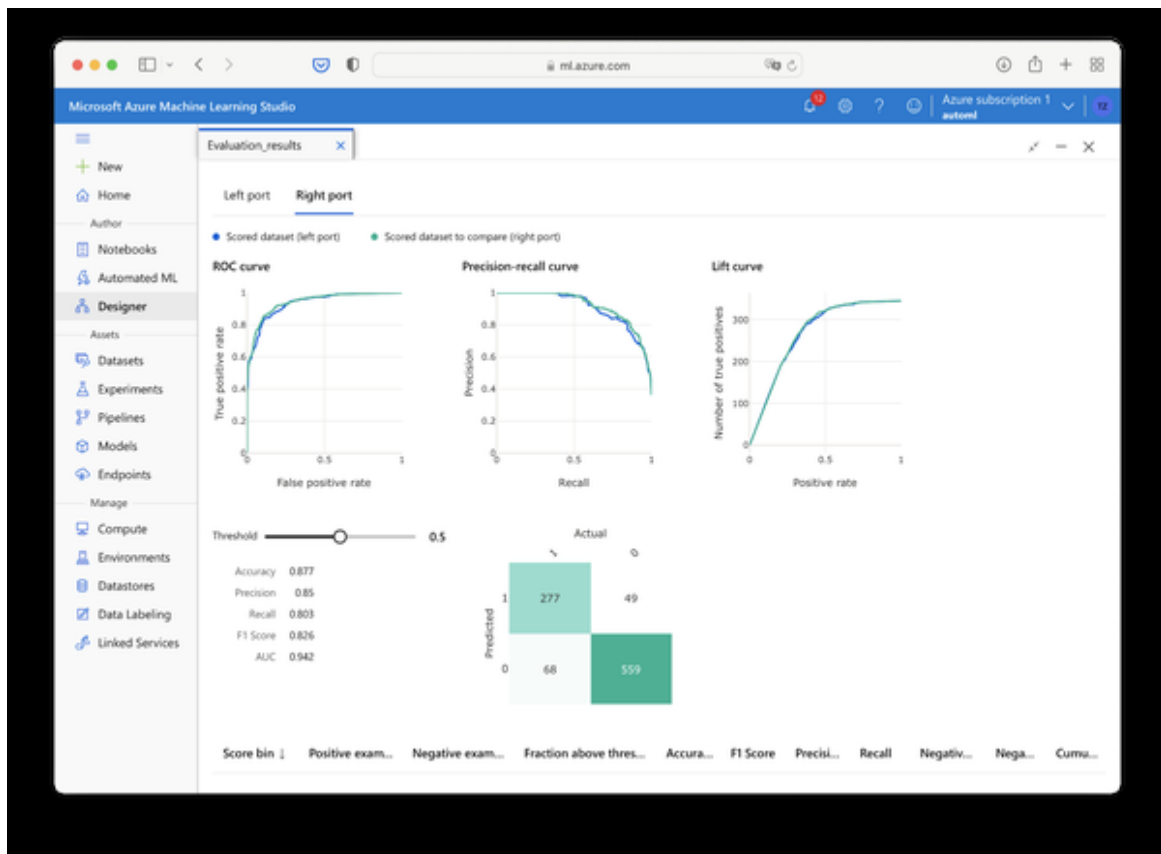


Figure 10-17. Evaluation metrics of second workflow

Looking at this we can see that the Accuracy could be improved by another percentage point to 87.7% and the F1-Score has increased to 82.6%. That's not a groundbreaking but still solid performance increase, considering that the sentiment label was only provided for a fraction of the data. This means that the sentiment information itself has very high predictive power for our use case. So it's good, keeping this information in our dataset and using it whenever possible.

By now, our model training is complete. Congratulations! You have just combined an out-of-the-box AI service with an advanced machine learning algorithm to craft a powerful custom AI model yourself! It's now time to put the model at work and make it ready for inference.

Before we proceed, let's tidy up our graph by selecting all parts on the left and deleting them. Since our previous runs have all been logged within our experiment, we can still access the results of this part of the model training later if we need them.

To select multiple modules, choose the selection tool from the toolbar and select all graph modules on the left side of the graph as shown in Figure XX. Click the delete icon to delete the modules.

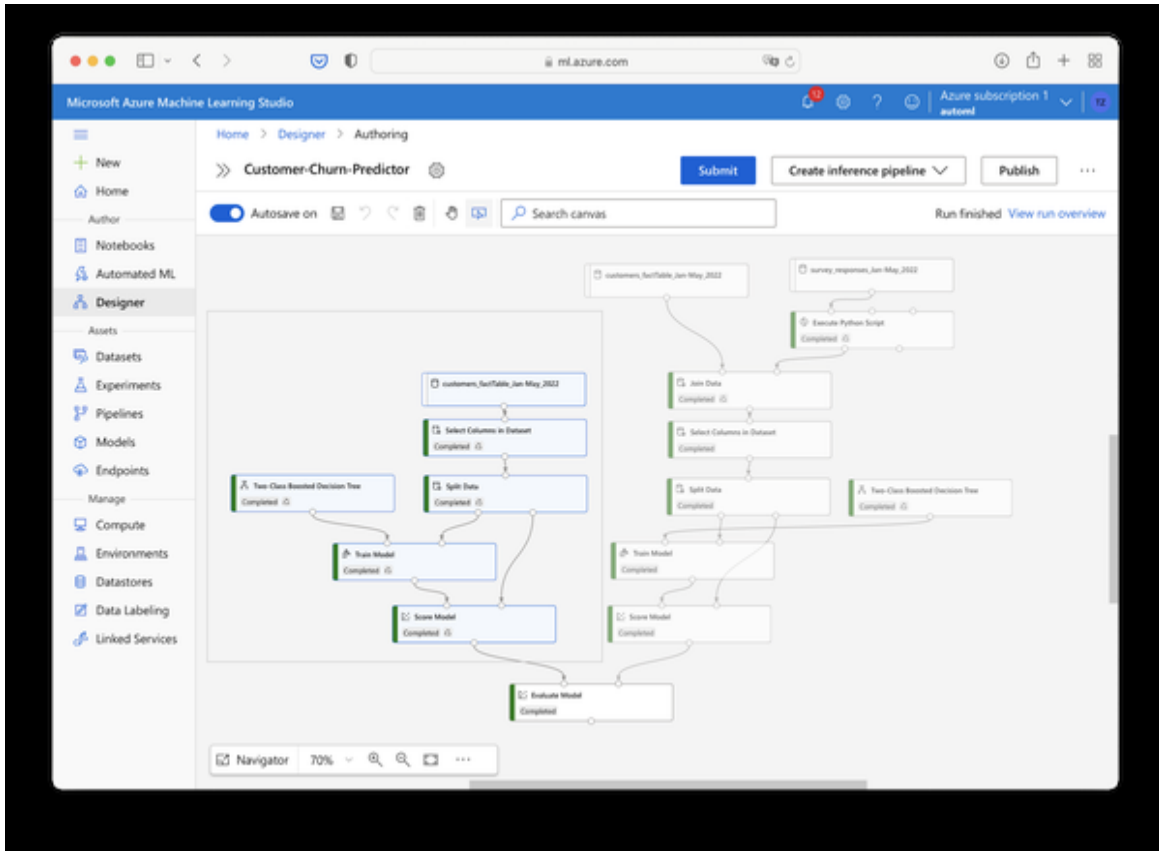


Figure 10-18. Select multiple components on canvas

Finally, re-connect the Score Model module to the first input port of the Evaluate Model module by selecting the existing connection between both, deleting it and redrawing to the first input port. Submit the training pipeline for a last time - this will be quick since it will only recalculate the results for the last module.

We can deploy this graph now as an inference pipeline. All we need to do for this is select Create Inference pipeline from the top menu as seen in figure XX and then choose Batch inference pipeline.

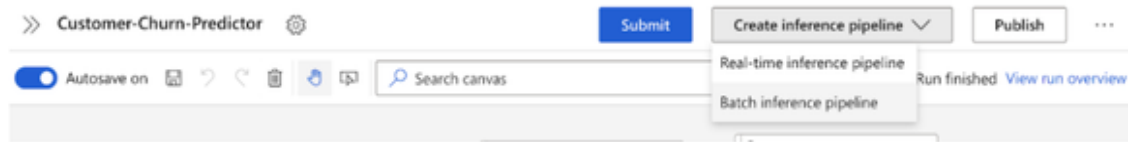


Figure 10-19. Create batch inference pipeline

The main difference between online and batch inference pipelines in this case is that the batch pipeline takes a blob dataset as an input whereas the online pipeline takes data submitted through an online API.

In any case, the training pipeline will be converted to an inference pipeline. If you look closely at the header of the ML Designer you will see a new tab called “Batch inference pipeline” as shown in Figure XX.

The main difference between the training and the inference pipeline is that the inference pipeline does not contain any training components. Note how the Boosted Decision Tree model and the Train model modules have been replaced by a new module called ML-Customer-Churn-Predictor-Train_Model_Trained_model as seen in Figure XX. This module will automatically reference our trained model so we can use it for inference. The rest of the data preparation pipeline, including our Python script will remain the same.

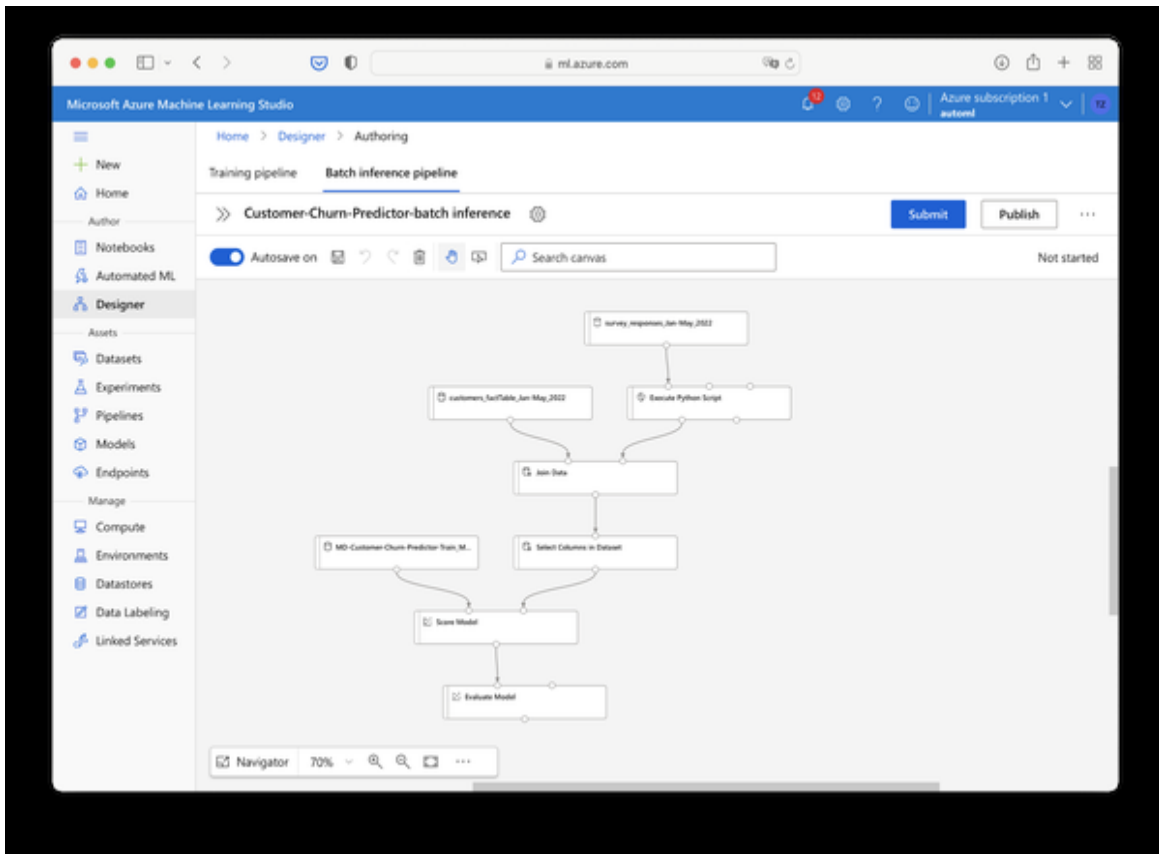


Figure 10-20. Batch inference pipeline (raw)

To run inference for new data replace the input datasets with `customers_factTable_June_2022` and `survey_responses_June_2022` respectively. These two datasets represent new data from customers where we don't know yet if they are going to churn or not. To replace the datasets, simply delete the existing datasets from the canvas and pull the new datasets over from the asset library. Your canvas should look like Figure XX.

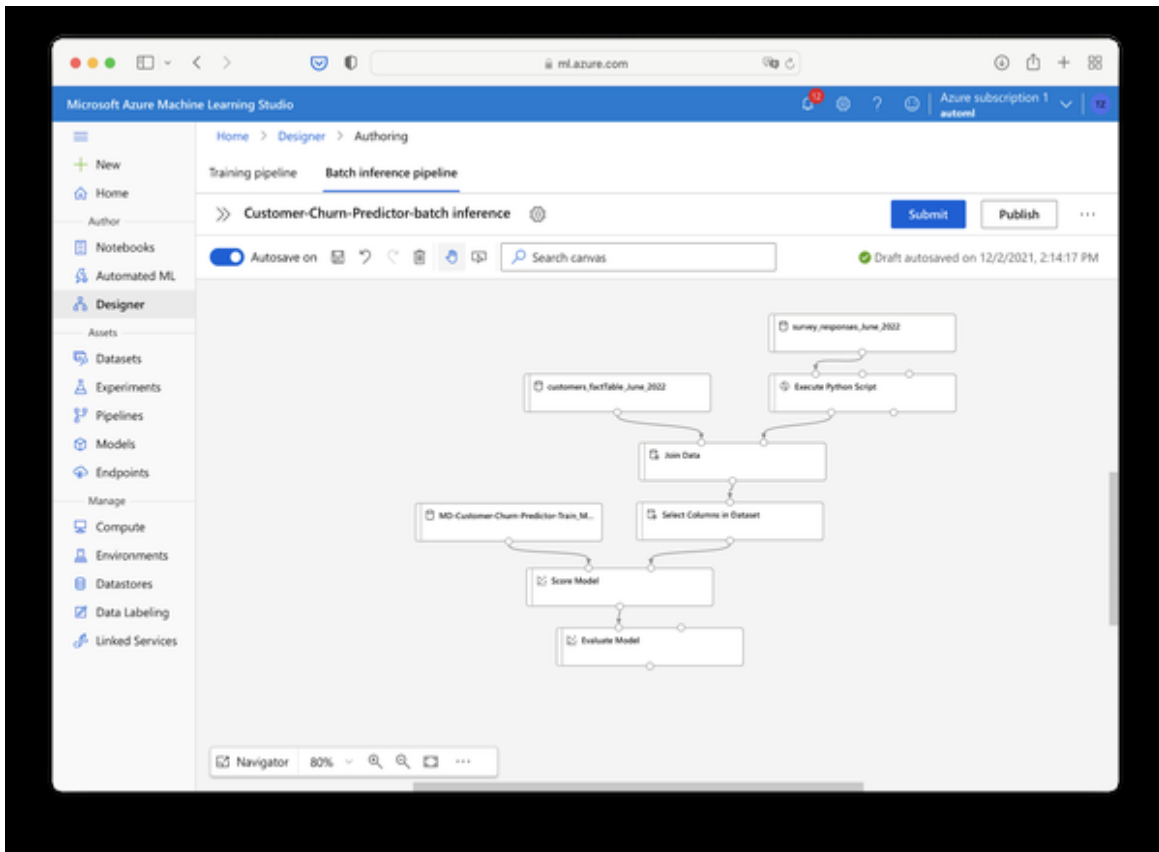


Figure 10-21. Batch inference pipeline (trimmed)

Before we can run the inference, we still need to make some small adjustments: Locate the Module Select Columns in Dataset on the canvas. It still contains the column Churn we used for training. We don't have this column during inference (that's the one we want to predict!). So edit the settings and delete the Churn column from the module. Also, keep the column customerID, because we will need this later to match the churn predictions to the customer records.

Also, if you jump to the end of our pipeline you will still see the Evaluate Model module as the last component here. However, we don't need this component at this stage, because we don't have any ground truth to compare to. Instead, we want to export the scored data to some location. To do this, delete the Evaluate Model module from the canvas and drag the "Export Data" module from the asset library (Data Input and Output) to the canvas.

In the settings, select Azure blob storage as data type. For the Datastore, click New Datastore and give it a name such as churn_model_scoring. For the storage account, choose the storage account that you used previously in the Chapters and select the “tables” under “Blob container”. Your settings should look similar to Figure XX.

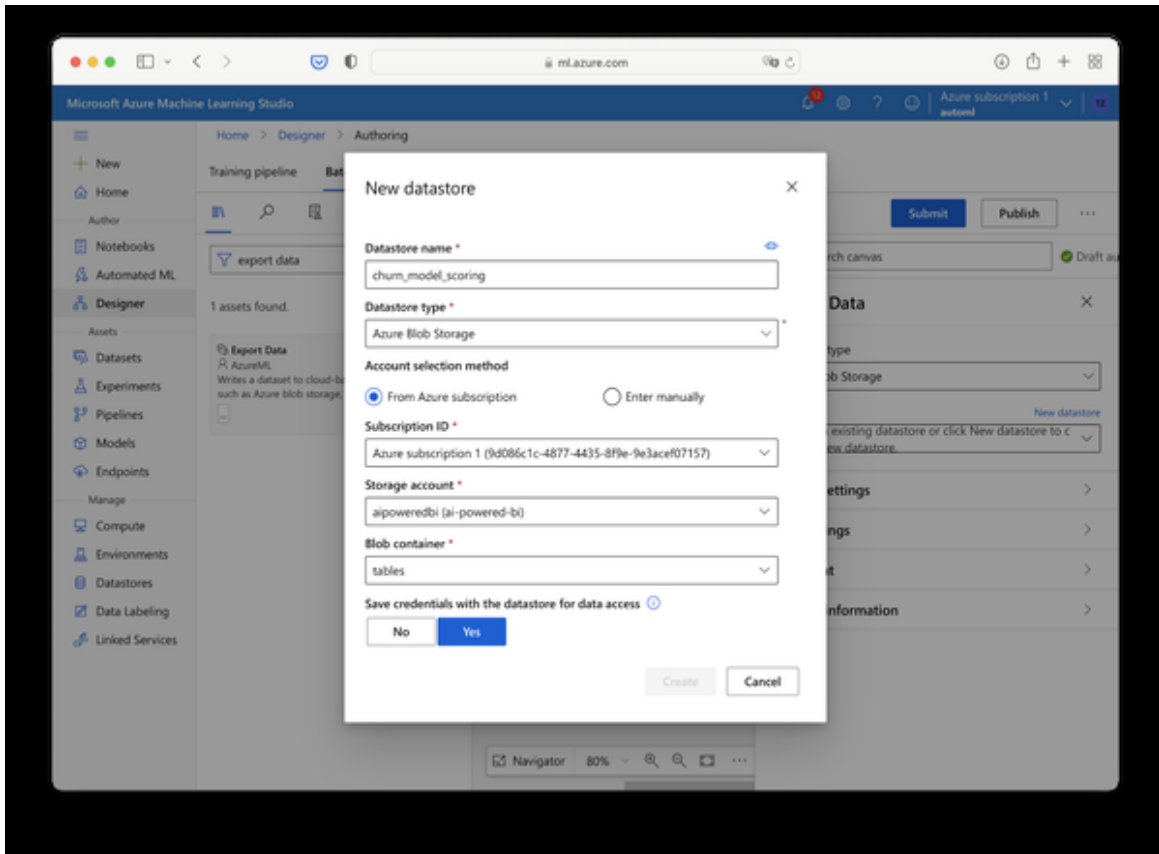


Figure 10-22. Create new datastore

Scroll down and paste your Account Key from the Azure Storage Account resource to the respective field in this settings window. Click Create and you should find yourself back in the Export Data settings. Double check that your newly created datastore has been selected and give your output file a name in the Path field such as churn-scoring.csv. Finally, set the file format to CSV. Figure XX shows an example of the export settings.

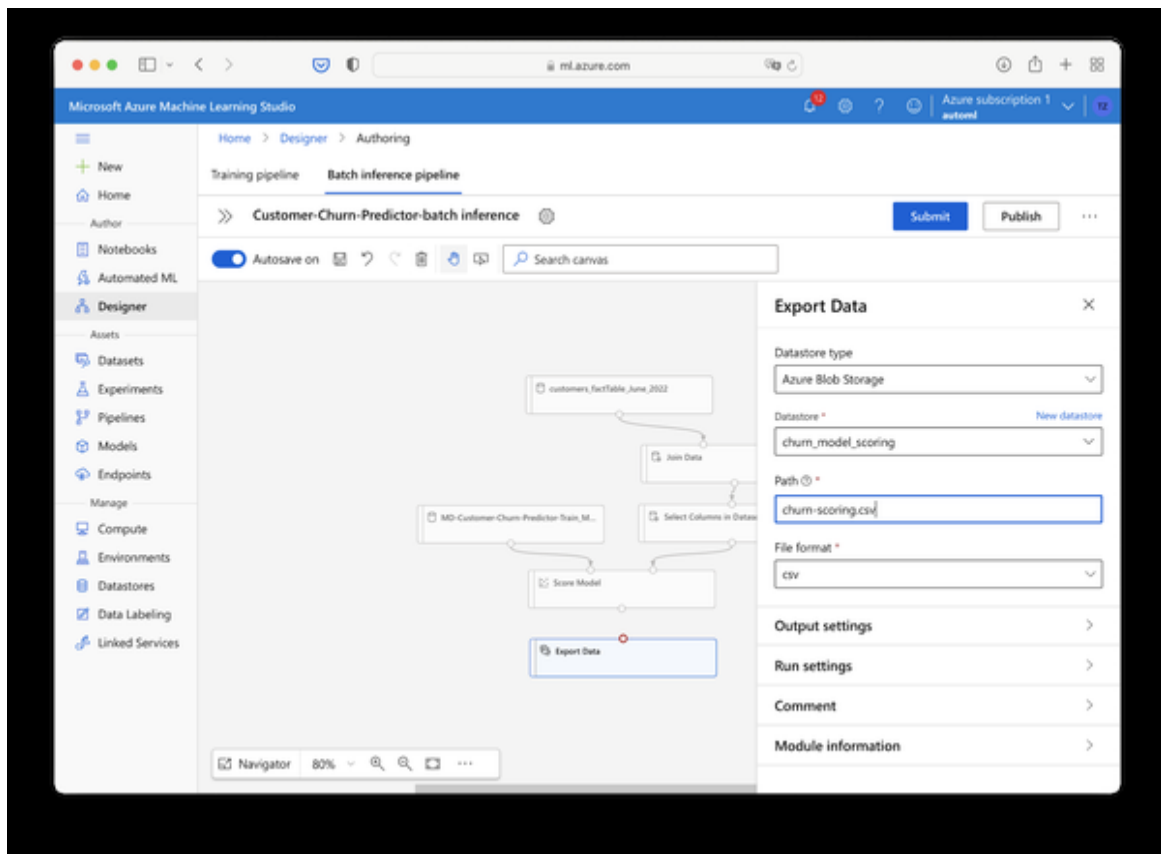


Figure 10-23. Add Export Data module and settings

Lastly, connect the Score Model module to the Export Data module. The final pipeline is shown in Figure XX.

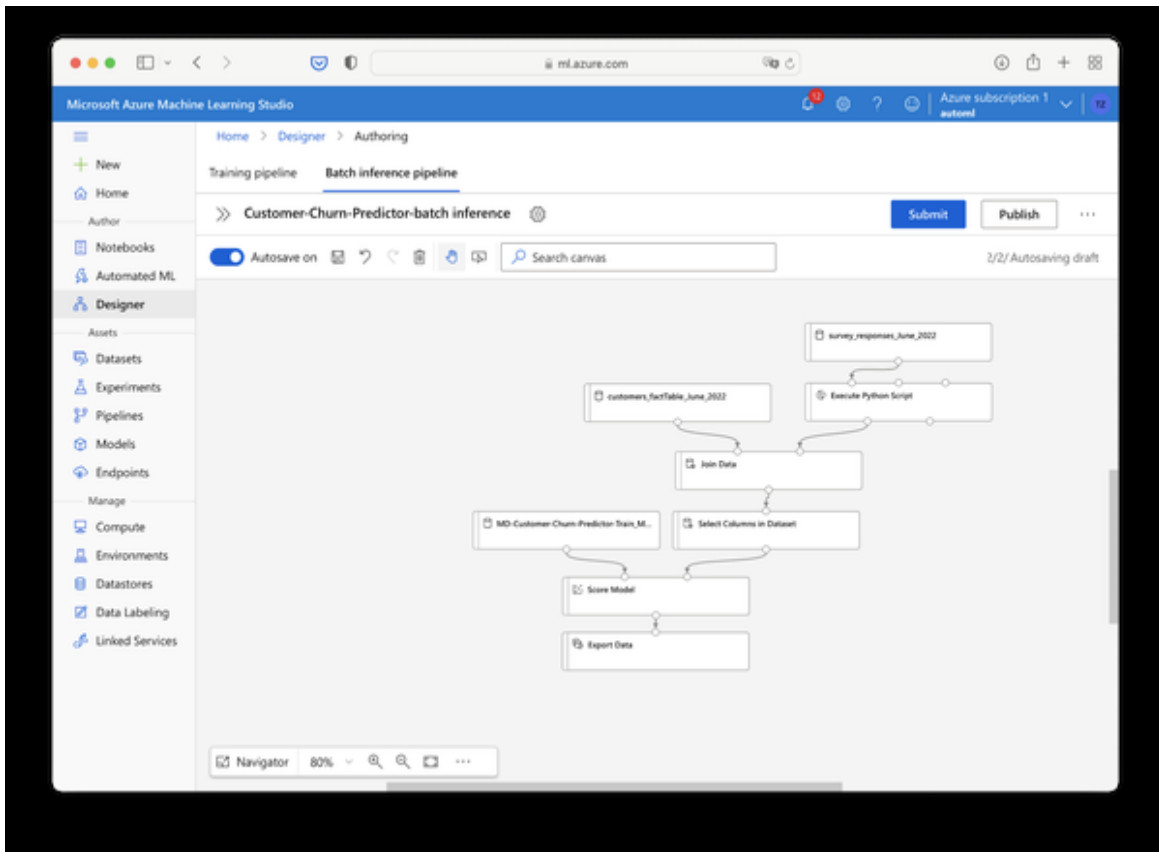


Figure 10-24. Final inference pipeline

Now, click “Submit” to run your inference pipeline. To keep things organised, create a new experiment such as “customer-churn-inference” that separates your inference runs from your training runs. The inference run will take a couple of minutes. Again, we’re doing this on relatively small data samples here but the process itself does scale pretty well also to very large datasets without scaling proportionally in time.

Once the inference run has been completed, you should see the new file churn-scoring.csv in your tables folder in your storage container as seen in Figure XX.

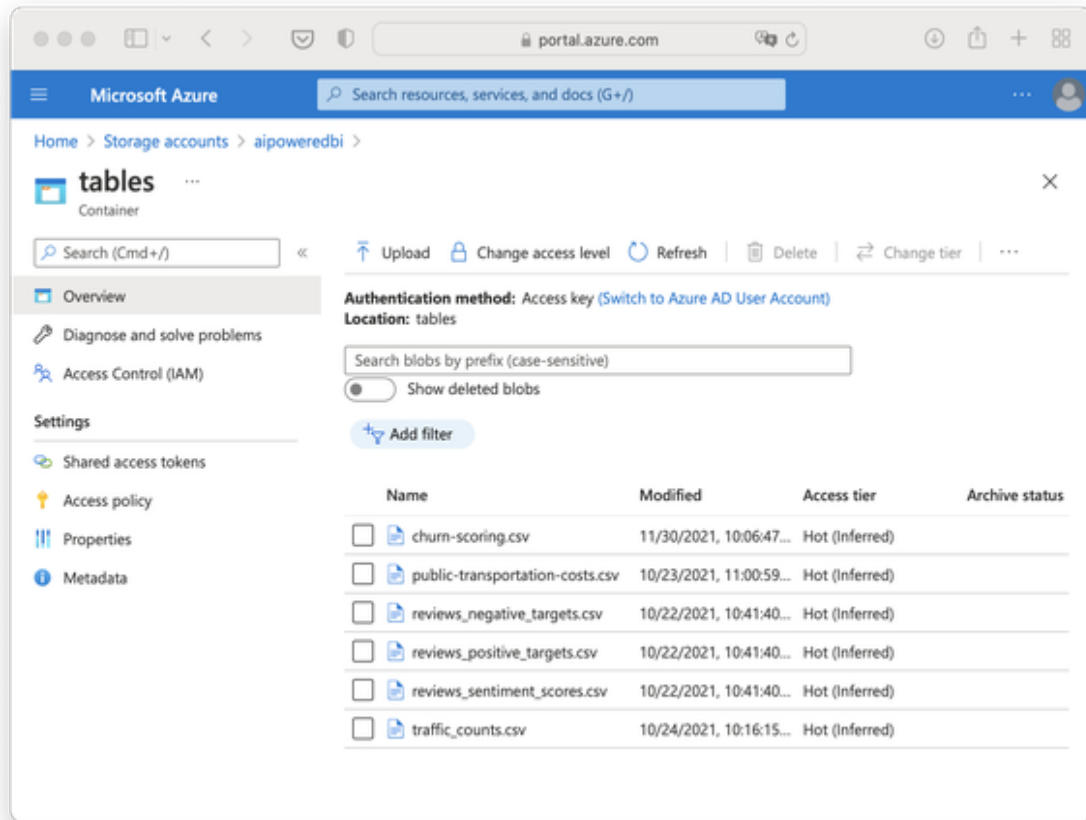


Figure 10-25. Table output in Azure blob storage

Time to head over to our BI!

Combining Different AI Services in BI

Now that we have built our predictive model and also turned unstructured text into easy-to-analyze categorical data, let's bring it all together by building a AI-Powered Customer Dashboard.

Remember, our goal was to build a dashboard for marketing and sales which provides a high-level overview how customer churn is developing, if things are 'still normal' and how effective counter-actions were taken. The dashboard should also provide some further functionality to dive deeper for custom insights.

In contrast to the previous chapters, I won't give you detailed step-by-step instructions on how the dashboard was built, but instead focus on the main

concepts and the different components at work here. You can download the PBIX file Chapter-9-customer-dashboard.pbix or recreate it yourself. I was using the different steps that were outlined previously.

NOTE

For a step-by-step video tutorial on how to build the dashboard, visit www.aipoweredbi.com/customer-dashboard for a free video tutorial

To build our customer dashboard, we will need the data model as seen in Figure X which is based on the following four tables:

- **churn_predictions_June_2022:** This table includes the scored churn dataset which we got from the Azure ML Designer
- **customer_dimensions_June_2022:** This table contains further BI data for customers such as tenure, services, demographic information, etc. It has a 1-to-1 relation to the table churn_predictions_June_2022 based on the field customer ID.
- **Churn_Metrics_2021:** A table which includes aggregated churn rates and offers acceptance rates per month from previous months. This data could come from a SQL data warehouse, in our case it's provided as a CSV file from the book repo.
- **Aggregated_metrics_2022:** This table is generated by Power BI by taking the dataset churn_metrics_2021 and adding the predicted churn rate for June 2022. It will take this time series and send it to Azure Anomaly detection to infer the expected lower and upper bounds for the metrics and set a flag for anomalies found.

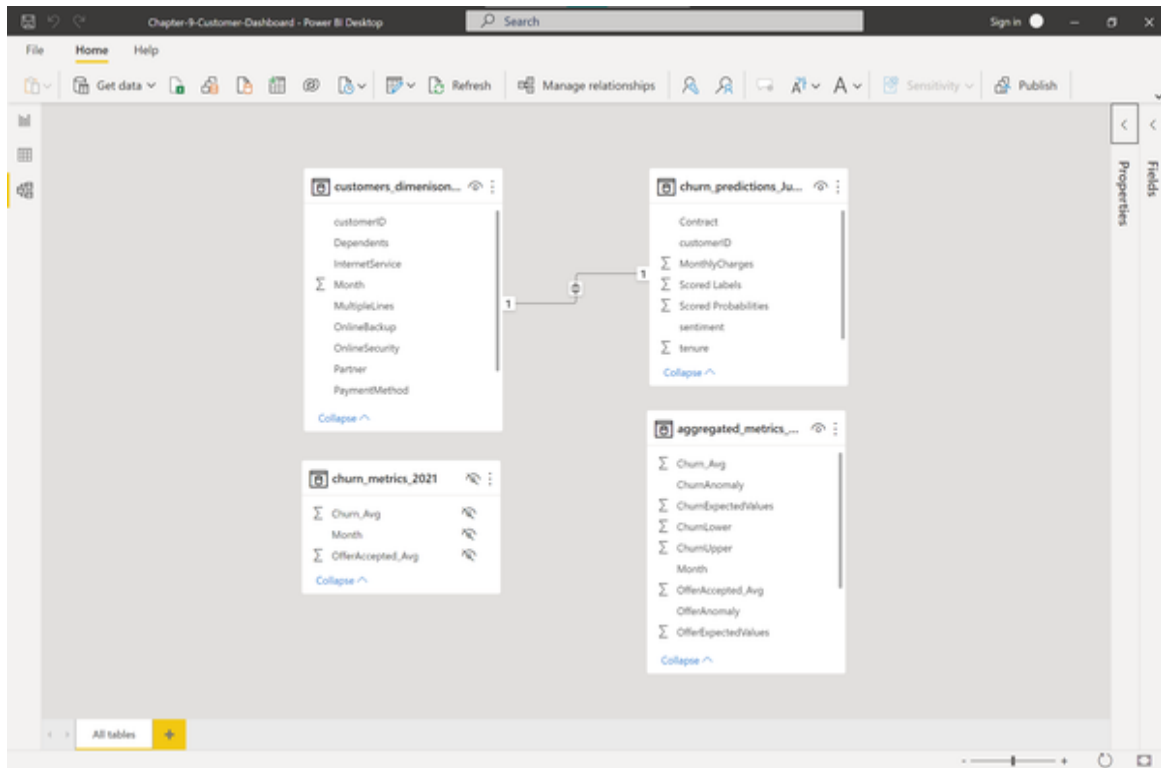


Figure 10-26. Data model in Power BI

With this data, we can generate a dashboard, that looks like shown in Figure XX:

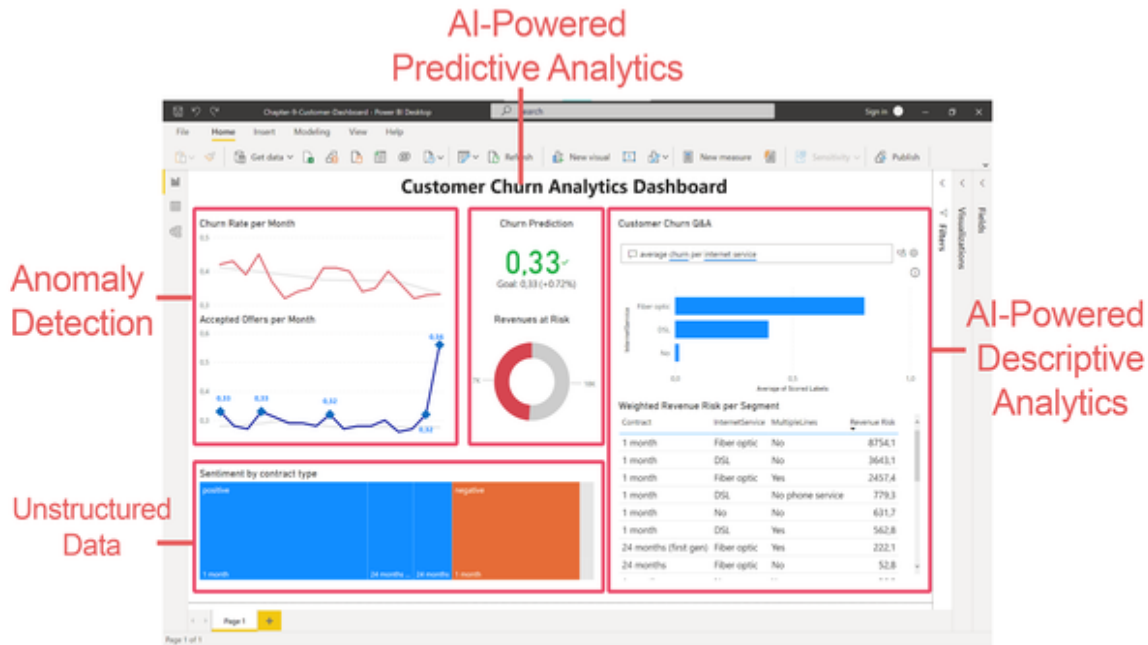


Figure 10-27. Different components of the final dashboard

Let's unpack the different components.

Anomaly Detection

On the top left you will see two line charts that show the churn rate and offer acceptance rate by month. You can see this part of the dashboard in Figure XX. While the line charts themselves 'just' represent basic descriptive analytics ("What happened?"), we are enriching the chart by blending two more pieces of information here. The first one is that we are showing a grey line which represents the expected value of the respective metrics based on the data we have seen so far. The second is that we highlight data points which show an abnormal positive spike (either 'too' high churn rate or 'too' high offer acceptance rate. These anomalies are depicted by square data markers. With this additional information we can see that the overall churn trend seems to be negative (which is good!) and at the same time we did not really observe any clear outlier although the line seems to be rather wriggly. For the offers, we can clearly observe that the high spike in the last month was detected as an anomaly, but so were some

data points before. The overall trend therefore seems to be rather steady, except for some outliers. All these information came from the anomaly detection AI service which we were calling with an R or Python script directly from Power BI.

→ Assess whether changes in Churn rate and Offer Acceptance are normal or abnormal (anomaly)

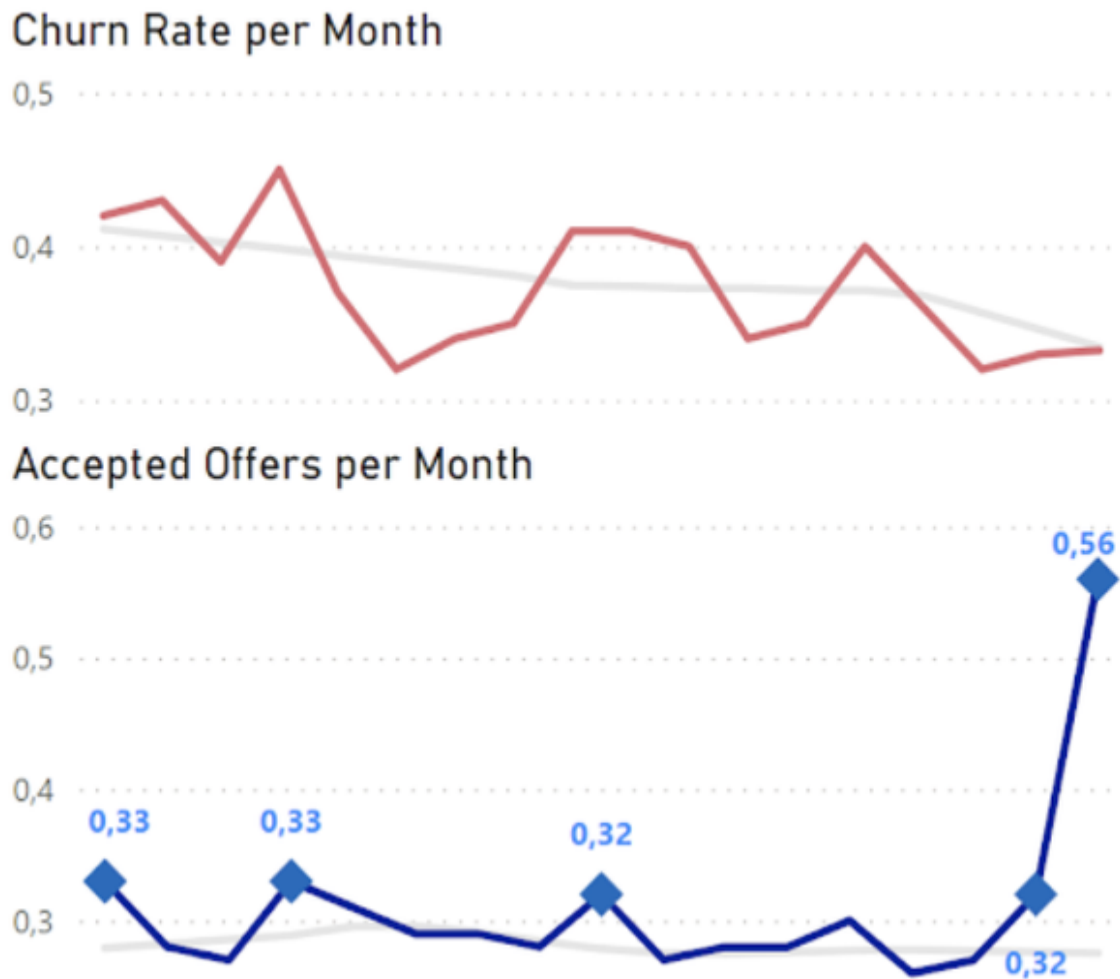


Figure 10-28. Churn and offer acceptance metrics per month

Predictive Analytics

This part will combine historic information with the AI predictions from above. The visual can be seen in Figure Figure XX. This includes the predicted customer churn rate for the current month, as well as a translation

to what this means to revenues at risk. The predicted customer churn rate for the current month is a KPI which we are calculating as a summary metric based on the individual AI predictions we have seen from the model we used in the ML Designer. We can compare this metric to a goal which in this case is defined as a churn rate of 0,33 - a criteria that is probably met in the current month. To make this information more accessible for business users, we are blending this metric with a donut chart called “Revenue at risk” which sums up all the revenues of the current month from customers that were marked with a churn flag = 1. Note that this metric is rather simplistic as it does not account for customer lifetime values. We could add this information if we wanted, but for the sake of this demo, I decided to stick to the basics. The core idea here is that we can blend AI predictions with historical or transactional BI data to make the predictions more relevant for the business. We will see another, more detailed example of this in the next component.

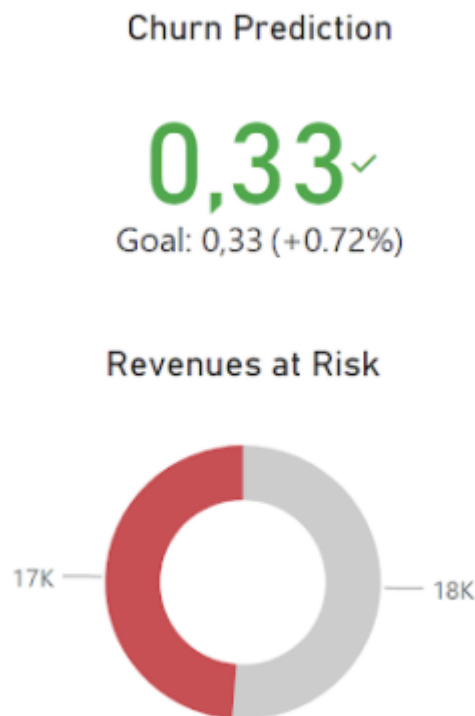


Figure 10-29. Churn prediction KPI metric

AI-Powered Descriptive Analytics

This part of the dashboard is the one where the AI at work is the most seamless, but yet possibly the most intuitive one for users. The AI-powered descriptive part of this analysis consists of two charts as shown in Figure XX. The upper part is a Q&A tool and the lower part is a table visual.

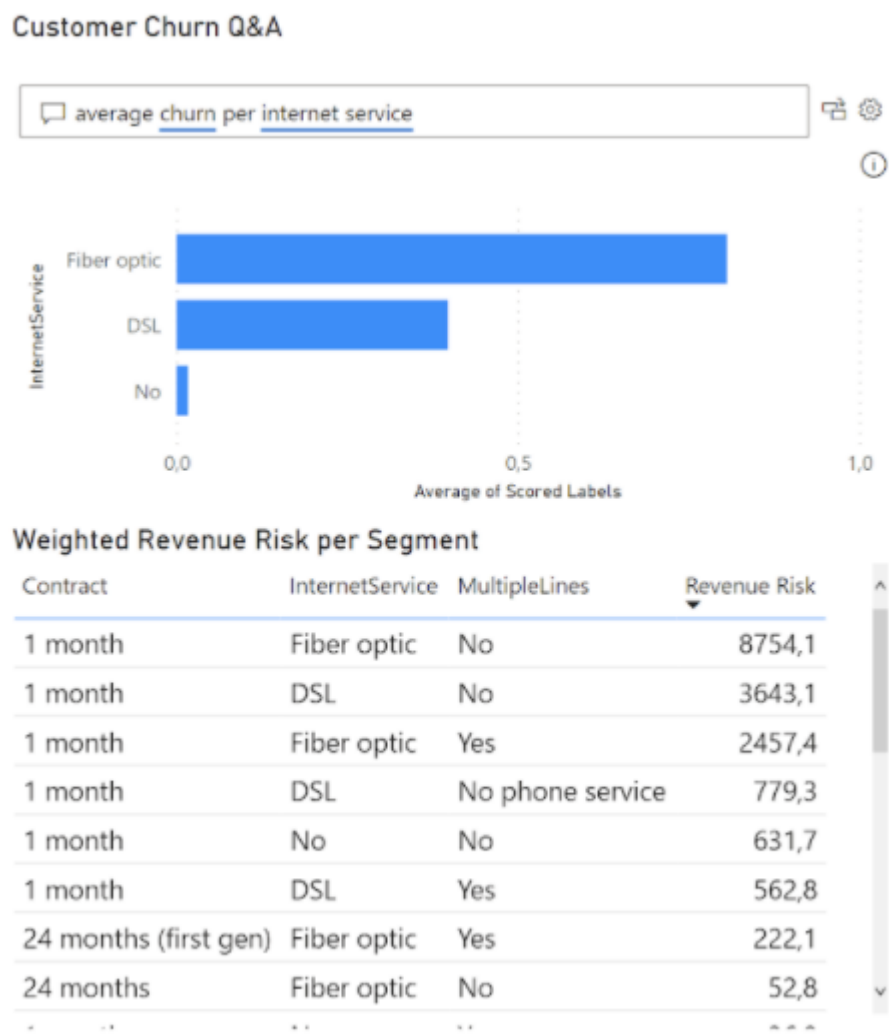


Figure 10-30. Descriptive analytics component

Let’s explore the Q&A tool first. The Q & A Tool combines churn scores with BI data to allow queries such as: What’s the average predicted churn rate in segment xyz? This visual can be used to run queries against our data models using natural language. Thanks to the relationship between the predicted churn outcomes and the dimensional BI data from the warehouse, the user can seamlessly blend AI-powered predictions with additional

properties about the customers. As shown in the example above, the user can query the “Average churn per internet service” and see that the churn rate for Fiber optics is much higher than in the other internet service categories. The only settings we needed to make for this is to define synonyms in the Q&A tool so that “average churn” can be matched to the data field “scored labels” as it is called originally in the source data table.

The second part in the form of the table below is another example of how users can leverage this combination between historic BI data and AI predictions. The user can calculate revenue at risk by combining churn prediction with current revenues. To do this, the table uses a new field called “Revenue Risk” which was generated in Power BI as a “quick measure” that multiplies a customer’s monthly revenue with their individual churn probabilities. If we now sum that up over different customer categories we can easily spot the customer segments where most money is going to be lost, making the aggregate churn number even more actionable for sales and marketing. If we look at the chart above, we can clearly see that the segment of customers who are on a monthly contract with Fiber optics internet service and without multiple lines provide the biggest leverage to defend revenue with a revenue risk of \$8.754 per month. If marketing was to think about personalized counter-churn offer incentives, this should be an interesting category to look further into.

Unstructured Data

And lastly, we provide more insights on the sentiment scores in combination with dimensional data. The sentiment analysis of the unstructured data eventually became part of our churn prediction model, but it can also stand on its own to provide further insights. The tree map visual as shown in Figure XX contrasts the sentiment labels to any customer dimension we want - in this case the contract type. From this visual we can see that while most customers are actually happy with regards to the sentiment of their survey comments, the majority of all customers who provided negative feedback were on a monthly plan. Considering the high churn likelihood of customers in this category, it should be the first priority to look into this category about what’s fundamentally going wrong here.



Figure 10-31. Sentiment analysis

If we pull everything together and look at the dashboard itself as seen in Figure 9-32, we can provide Marketing and Sales with a customer centric dashboard that uses AI under the hood not to show off, but with one goal in mind: To make data more actionable and more meaningful to business stakeholders.

Try it out by yourself, opening the accompanying Power BI file on your computer and playing around with the different components therein. Which information would you add, change or delete altogether?

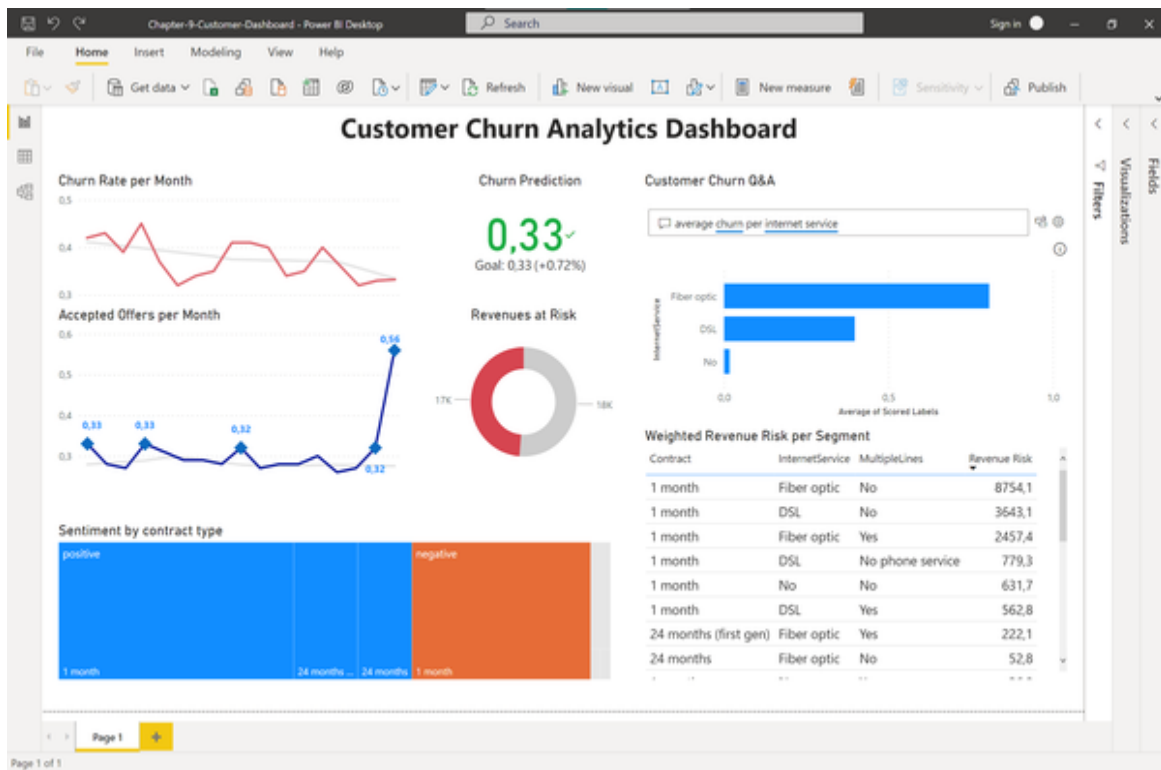


Figure 10-32. Final dashboard

Conclusion

I hope that this last chapter has shown you that using AI services in your BI isn't a "either" - "or" decision. In fact, the biggest benefits come from combining multiple AI services. However, don't let the technology pull you down into a rabbit hole where the BI functionality is overloaded with AI components at work. Remember that each AI service which you add, also adds another layer of technical debt - someone has to eventually take care of the models, the Q&A tools, the data integration and so on. Pulling different AI services together, however, is a great way of validating if the business benefits from further insights or carving out what it is exactly that the business is looking for. With your flexible analytics stack ranging all the way from accessing unstructured data, descriptive analytics up to prescriptive analytics from within the same platform will put you in a great position to find the sweet spots between adding technical complexity and delivering actual business value.

Congratulations, you've successfully built your first prototype which contains not less than 4 different AI services! But what's next? As mentioned, it is a prototype and it is still a good way to go before we could ship this dashboard to a productive environment. In the next chapter we will learn more about what it takes to move from prototype to production and how to tackle the complexities that come with it.

Chapter 11. Taking the Next Steps: From Prototype to Production

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 11th chapter of the final book.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the author at *tobias@rapyd.ai*.

Congratulations!

If you have made it this far and followed the different examples and case studies in this book, you should have acquired a solid knowledge of the main AI/ML techniques and their application in the context of various BI scenarios.

This is truly a great achievement and puts you in a great position to successfully launch your next AI use case.

Discovery vs. Delivery

Keep in mind, however, that what did so far has been “just” prototyping. You did what product managers call a “discovery” process. Discovery is

about validating value, usability, feasibility or viability of a product, or a use case.

It's worth noting that most prototypes are unlikely to survive the discovery phase. That's the nature of the game: you want to learn fast and fail fast. That's fine. By now, you should not have too many resources invested, and ideally you have other ideas in your backlog to pursue.

But what happens if your prototype actually survives the validation phase? What happens if it takes off?

Let's say your early BI test users love the new feature or the model predictions. Everybody gets excited about your new solution and some people may even spread the word about it in the company.

How do you scale your prototype for production?

The simple answer is: you don't scale it at all.

In our prototyping example, we did everything ourselves: we created the virtual machine for computation jobs, we managed the access key for storage, and we even set the security policy for our containers.

You may have guessed it: this is not what you should do in a production scenario. The Azure cloud platform (and all other platforms) may look super intuitive and easy to use after some training. But in reality they are super complex landscapes that require expert knowledge to manage and control securely and effectively. Otherwise, you risk a data breach or a skyrocketing resource bill, or both.

If a prototype turns out to be successful and passes the tests for impact and feasibility, it's time to let go of it and move on to a different stage: Delivery.

In enterprises, this is about much more than just creating a "clean" version of your code and dashboard.

When you proceed to Delivery you should prioritize the following things:

- Scalability
- Reliability

- Performance
- Maintainability

In addition, prototypes usually have multiple security vulnerabilities because they are not properly tested at this stage of development. Therefore, you should not immediately move a prototype into production, but rethink across these dimensions. Let us unpack these concepts briefly:

Scalability refers to both the technical and non-technical challenges of making your solution available to more users. Technically, it's about the upper bounds of how many users can access the solution simultaneously, which are limited by your infrastructure. In non-technical terms, you want to figure out how processes can hinder the deployment of an application. For example, if you'd to manually set up the dashboard for each user, the maximum size of your solution would be limited by the people you can assign to it. If you deploy the dashboards automatically or use a dedicated BI server, you can run your solution at a much larger scale.

Reliability simply means that your solution does what it is expected to do. When a failure occurs, can it be fixed easily and with little or no downtime? Sooner or later, you will need to respond to user inquiries and often provide online help. How quickly can these problems be fixed?

Performance refers to the speed at which your application responds to user requests. As the number of users increases, this becomes an important factor. First impressions are crucial, and slow-loading applications or sluggish interactions can hinder growth. After all, you want users to be able to complete their tasks as quickly as possible without experiencing delays.

Maintainability is about how easy it is to make changes once the solution is up and running. Can features be added or changed without requiring a programmer? Is your code properly documented so that no unexpected errors occur when a new user works on it?

Your production application should have scalability, reliability, performance and maintainability baked in from the start. The actual priorities depend on the needs of your business. If you want to provide your solution to many

users, scalability is very important. On the other hand, if an enterprise application is designed for only a certain number of users, it's ok to deprioritize the scalability goal and document this constraint in your delivery project.

Either way: In most cases, your prototyping stack doesn't meet these delivery principles because you've sacrificed them for speed and developer convenience.

That's why you shouldn't try to scale a prototype, but rebuild it properly step by step.

Success Criteria For AI Product Delivery

So how do you achieve a successful transition from the Discovery to the Delivery phase?

To develop a successful enterprise AI product in the context of BI, you need to consider 5 dimensions: People, Processes, Data, Technology, and MLOps, as shown in Figure 11.

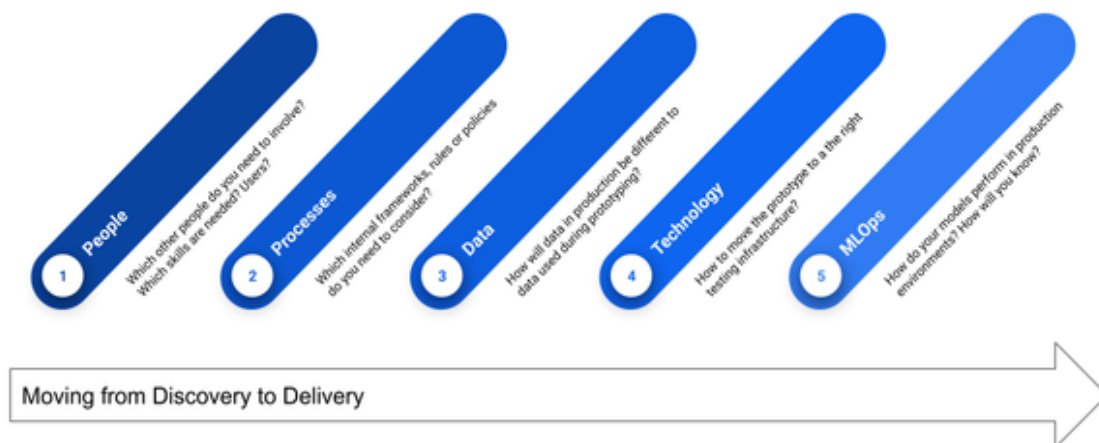


Figure 11-1. Success Criteria For AI Product Delivery In the Context Of BI

Let's go through these in more detail:

People

People determine the success or failure of your AI product.

During the discovery phase, your team probably did not exceed the 2-pizza rule. That is: all people in the project team were satiated by two large pizzas. That's usually 3-5 people.

Once you aim for delivery in enterprise scenarios, this will change. Enterprise AI projects are essentially enterprise software projects. Instead of two pizzas, consider ordering a big buffet, because there are all sorts of people who might be involved in a new software development project. These may include, to name only a few:

- IT security
- IT application management
- IT infrastructure management
- Data governance office
- Business leaders of various other departments
- Lawyers
- Works council
- ...

The various functions you need to include depend heavily on how your company is organized and how large it is. These people often go hand-in-hand with the process frameworks you need to follow (see next point).

While I can't give you a comprehensive list of the people you should talk to, I have generally found it useful to organize your different stakeholders in a stakeholder matrix that maps people along two broad dimensions:

Power over the project and their interest in the work.

The goal is to identify stakeholders in each category to determine the right strategies for communicating with them. There are five categories: Manage closely, Keep satisfied, Keep informed and Monitor, which we will go through briefly.

Manage closely: This category includes people with a lot of power and high interest in your project. These are your most important stakeholders. You need to talk to them a lot, identify concerns early, and engage to keep them on the same page and manage their expectations.

Examples: Sponsors, top management, directors, PMs.

Keep satisfied: These are people with a lot of power but little interest in your project. They are often under the radar, but very important because they have the power to decide whether the project should continue or not, depending on current outcomes or progress.

Examples: Program management, financial planning

Keep informed: People with a lot of interest but little power in your project need to be kept informed on a regular basis. These are usually people who will ultimately use what is being built. Even if their formal power is small, you should communicate with them frequently. They are so involved that they are likely to influence other (more powerful) stakeholders.

Examples: End users, clients

Monitor: People with little power and interest in your project. These people may not be a priority for you, and you do not want to bother them too much. However, they still need to be monitored and informed sporadically, even if they are unlikely to cause major problems.

Examples: Employees from other departments

Once you start mapping different people along these categories, you should get a feeling for the complexity of your project and you can adjust this matrix as your project progresses.

Processes

Since AI projects are essentially software projects you need to watch out for frameworks and processes in your company that govern the development process of software artifacts. Depending on your organization, there may be different rules in place and it's beyond the scope of this book to list all of them. However, if you're generally not familiar with how software projects are handled, I'll give you some key terms to look out for to see if they might apply to your organization:

ISO/IEC: The ISO /IEC 25000 series of standards is also known as SQuaRE (System and Software Quality Requirements and Evaluation). These standards aim to provide a framework for evaluating the quality of software products. If your organization uses ISO -certified standards, your project might fall under these rules and needs to be set up accordingly.

Scrum: Scrum is a software development framework used by companies for agile software development. Although the goal is to enable a lightweight, iterative development process, it comes with a relatively strict overhead and fixed roles, for example a Product Owner and a Scrum Master. Be sure if something like Scrum exists in your organization as this will define the project structure.

BPM: The concept of “Business Process Management” (BPM) is commonly used in large organizations to manage key business processes and their respective owners. BPM is also used in software development and you may need to document your project accordingly.

PMO: A Project Management Office (PMO) is an internal department where company-specific project standards are established and maintained. Depending on the size of your project and the way your company works, you may at least have to adhere to their rules and sometimes even report regularly.

Once your prototype has a chance to move to the delivery phase, I strongly recommend that you contact your IT / DevOps department (if you have not already done so during prototyping) to ensure that your project setup meets all formal requirements necessary for software development projects.

Data

The data shouldn't surprise you anymore by now. During prototyping, you should have been able to identify the biggest data traps, and if you made it to delivery, you somehow found a way to overcome them. The only question that remains is: Will the approaches you have taken work in production? Are they safe and reliable? Does it scale? Is there anything else you need to consider?

Coming from BI, you should have a solid knowledge of what levers you can pull in your organization to move data around and what the requirements or constraints are there.

Here are some areas you should pay particular attention to:

Data pipelines: Remember how we used to manually upload CSV files to Azure ML Studio and store the results in Azure Blob Storage? Most likely, this approach would not work in production scenarios because there are too many things that can break. Ideally, you pull your data from a data warehouse and store the exports according to a consistent schema in a place where your production application has direct access to it - and that can even be Azure Blob Storage.

Data cleansing and management: Once you have automated the data pipelines and created the actual ETL scripts, you need to make sure they are both maintainable and can be automated (you won't be able to do manual data cleansing). Depending on the size of the project, this data cleansing part alone can be a major hurdle for the entire initiative.

Data access: I hope the use cases have shown you the importance of being able to easily access and share data throughout the various stages of the development process. If you had all the data stored in a centralized cloud data warehouse, accessing it would be "just" a matter of access rights, but not a technical hurdle. I know that many companies are still struggling to break down their data silos and improve sharing across different business units. Ideally, your prototype use case has provided a clear example of the benefits of making data easily accessible and connecting your applications to those data sources with a few simple scripts. Hosting your data

warehouse in the cloud can have its drawbacks, but in terms of fast and flexible development, it's definitely a big advantage.

Data protection: As you move into production and serve more users, consider whether your data is subject to other constraints compared to the data you used in prototyping. For example, in prototyping, you may have used customer data from the U.S. and worked with it on Microsoft Azure. In production, your users might also be located in the EU and you would need special permission to process that data if it contains personal information. Storing data outside the EU can also quickly become problematic for this user group. If you have a data governance framework in place, get your data governance team involved early enough, ideally at the prototyping stage, to identify major obstacles early on.

Technology

In 99% of cases, the technology stack you use for prototyping will not be the technology stack you use in production. And if it is - congratulations - you have figured out what digital transformation is all about. It's far from common to use the same infrastructure for both rapid prototyping and testing new features, as well as running production workloads at scale.

The reason is that systems for production are optimized to be scalable, reliable, maintainable, performant and secure. As we have learned, we generally sacrifice these goals for faster and more flexible development when prototyping.

If you want to deliver a new software application or even a single feature within an existing (BI) system, you should be guided by the requirements of the testing environment. The testing stage will usually resemble your production environment and is your first milestone to deliver something that can be used for testing.

The most important question you need to ask yourself at this point is:

Which parts of my prototype do I need to recreate to deliver it to the testing environment and which can I reuse?

The answer to this question can lie between the two extremes: A) You do not need to rebuild anything because the prototyping stack is the same as the testing stack. In this case, you just need to focus on actually improving things and making sure that what you built meets the production environment's criteria for scalability, reliability, maintainability, performance, and security. The other extreme would be option B, where you basically have to rebuild everything.

In most cases, you will find yourself somewhere between these two poles and you will need to identify the components that you can reuse. Anything left over will need to be rebuilt and will determine the size of your development team and budget. The more complex your product becomes, the higher the cost of development and maintenance will be.

Based on the use cases we have explored in this book, here are some components we have used in prototyping that you may be able to reuse even in production:

Model Endpoint APIs: The models we have implemented using Azure ML Studio generally meet all the requirements of production workloads. Be sure to check out the next section, “MLOps,” for the criteria for machine learning models in production. But the infrastructure itself will be compatible.

User Interface / Reports: If you are using Power BI in your existing reporting landscape, you can probably reuse the report layouts and dashboards we created in the exercises. The way you deliver them to users might be different (e.g., using Power BI Server), but the look and feel would remain the same, so you would not need to recreate this again.

Data pipelines: We used Datasets in Azure ML Studio to train our models. As long as you keep the schema, you should be able to incorporate new data fairly quickly, either by importing data from a file or from other resources in Azure. As long as you manage to get your data into the Datasets using the same schema, the pipeline for data preparation within Azure ML Studio remains the same.

Documentation: Ideally, you have used the prototyping phase to make some notes about your overall development process. This can be non-technical documentation such as “What problem did you solve?”, “What approaches did you take?”, “What people did you involve?” as well as more technical documentation (data schemas, API documentation, Power BI data models). It is very likely that you can reuse these components, or at least use them as a starting point for moving the prototype project into the delivery phase.

Try to reuse as much as possible and rebuild everything else properly.

MLOps

The process of managing and operating machine learning models in production systems is an area commonly referred to as MLOps.

At a minimum, this requires adequate API documentation and staff to take care of the model once it is deployed.

In the best case, MLOps spans the entire lifecycle of a machine learning solution: from data acquisition through version control, experimentation, performance monitoring, endpoint management to model sunset.

Machine learning models can therefore be considered an end-to-end software project in themselves and should be managed accordingly. That’s why it shouldn’t surprise you that you will find some of the same concepts within MLOps that were also applied to the overall solution in which the machine learning model is used.

The following areas are among most important success factors for MLOps:

People

Truth be told: If you don’t have someone to at least monitor your model from time to time and take care of its maintenance, you better not implement any machine learning model at all. There are many things that can happen after implementation. The most common phenomenon is a concept called “data drift.” This means that the data your model sees in production becomes more and more different from the data it

saw during training and makes the predictions less accurate and less reliable over time. Although there are tools to help automate this process, it is ultimately a human's job to decide whether a new model needs to be trained again and what data to use for that. Apart from model performance, debugging is also a very important component. If the model produces errors or other problems occur, who should be responsible to fix these issues?

Automation

One of the main goals of MLOps is to automate the process as much as possible. If you can't easily reproduce an analysis or a model training, it's unlikely that your team will be able to iterate on new models quickly enough to deliver value. For this reason, automated pipelines are considered one of the most important aspects in MLOps, especially if you run more than one or two models in production.

Reliability and Reusability

If your organization is expanding the use of ML models, it's important that you establish a standard process for creating them that's reliable and whose correct operation can be easily verified each time. If you fail to do this, the cost of creating new models or making subsequent changes to existing models can exceed the value of a machine learning model. You also need to ensure that when there's a change within the team, there's a smooth transition and new colleagues can take over quickly.

Security

It's important that your MLOps team understands what data they're working with and what governance rules apply. In some cases, this data includes sensitive user information that may not leave certain geographic areas, may not be accessible to all individuals, or may have its use restricted by privacy laws. MLOps also ensures that all data and

infrastructure resources are managed through appropriate identity and access management (IAM) rules and technical arrangements.

Scalability

Keeping up with an ever-growing ML resource footprint is another major concern of MLOps. One of the best ways to avoid this is by being able to scale up your infrastructure as needed, both during training and during inference. On the one hand you want to avoid a situation where you are either limiting your machine learning training progress by not providing enough or not the right infrastructure. On the other one you don't want to reserve a large computer cluster for an ML job that is only running once every couple of months.

Dedicated machine learning platforms like Azure ML studio, will help you address some of the issues that MLOps deals with. But ultimately, your organization needs a process and people framework to manage these aspects. Be sure to think about them before moving your own custom ML services into production. Some of these areas also apply to AI services, although in this case you usually have a vendor who will either take care of most things or support you accordingly.

Start by delivering increments

Yes, that's a lot. That's why you shouldn't embark on this journey alone. Consult your IT department or outside experts as needed if you want to take this seriously and expand your AI efforts. You'll need a lot of support, especially in the beginning, to avoid serious mistakes.

At the same time, don't be too intimidated by the process. By now, you should have the confidence that the step from discovery to delivery is actually worth all the effort.

When you start, do not aim for a big bang, but try to deliver incrementally. That way, you will stay flexible for changes and you have opportunities to celebrate the small wins along the way.

Conclusion

You did it!

I hope this book has given you some insight into the world of AI and how it can be used in BI. There are a number of use cases we have explored here to give you a taste of the many possibilities you can explore with machine learning applications.

With the help of this book, you should now be well on your way to developing your own AI-powered Business Intelligence applications.

So, what have you learned? Here are some concepts we covered:

- How to find use cases with business impact
- How to assess the feasibility of AI projects
- Basic things to know about machine learning
- How to use prototyping for product discovery and what tools to use in the context of AI prototyping
- Practical use cases of how AI can be used to improve BI results.
- How to move to the next phase to take your project from prototype to production

I hope the use cases in this book have piqued your interest in how artificial intelligence can improve business outcomes at BI in the areas of data collection, descriptive, diagnostic, predictive, or prescriptive analytics.

If so, do not stop here. The next step is to build out your own Business Intelligence application using artificial intelligence techniques.

The world is changing at a rapid pace, but with the information in this book, you will be well equipped to navigate this field on your own.

I wish you all the best in your future AI journey and hope that you will launch many successful projects.

You can always contact me if you have feedback, questions, or ideas for prototypes around AI-powered business intelligence and want to succeed in any of these areas.

About the Author

Tobias Zwingmann is an experienced Senior Data Scientist and currently Cofounder at RPYD.AI, a Germany-based AI startup focused on prototyping AI solutions. His mission is to help companies adopt machine learning and AI solutions faster while creating meaningful business impact and reducing the risk of project failure. Before founding RPYD.AI, he has worked for more than 15 years in a corporate setting where he has been responsible for creating a company-wide data strategy and building out data science use cases.